HANDOUTS | MATERIAL
HIGHLIGHTED AND PROVIDED BY :

VU TALK

CLICK TO JOIN OUR WHATSAPP GROUP :

VU TALK WHATSAPP GROUP

VU TALK YouTube CHANNEL : click me

For Any Help Contact Me On Whatsapp : click me

Follow VU TALK on Whatsapp Channel : click me

CS511-Web Engineering

Handouts Organized by Scammer404

This course has been divided in two parts. First part deals with basic topics and second part deals with advance topics.

Outline of this course is as follows:

Week – 1

Lesson 1 : Introduction to Web Engineering

Topics (Basics):

1. What is web and how does it work?
2. Structuring web contents using HTML

   .. Initial learning of HTML will be included.
3. Setting visual layout using CSS

   .. CSS deals with coloring and font styling of web page.
4. Making web interactive using JavaScript

   ..Java script also being used for full scale development these days.
5. Web backend programming using PHP and databases

   ..PHP is used for dynamic web applications

Topics (Advanced)

6. Engineering web application

   .. study of different architectures and pattern
7. Making websites secure

   .. to avoid the hacking attacks and other vulnerabilities
8. Introduction to web services

   .. to perform some functions and task performed on remote server
9. Front-end and back-end frameworks

   .. nodejs, react, angular, view

10. Hosting websites
11. Search engine optimization (SEO)

    .. how to make your website on top position of search engine results.

Lesson 2 : Ecosystem of Web

A Complicated Ecosystem

As visualized in following figure , web development can also be understood as an ecosystem, one that builds on existing technologies (URL, DNS, and Internet), and contributes new protocols and standards (HTTP, HTML, and JavaScript) that facilitate client-server interactions. As this ecosystem matures, new client and server technologies, frameworks, and platforms continue to be developed in support of the web (PHP, jQuery, Bootstrap, etc.).



The model shows web-development as a three-storey building. Each floor depicts several aspects of web development with an illustration. Ground floor shows Servers, Configuration, Networks, and Protocols which are the basic building blocks. Middle floor shows different languages which are a part of web development, like CSS, HTML, PHP, Javascript, databases, APIs, and Tools. Top level shows advanced concepts like Design, Search, Integration, Frameworks, and Security.

A Short History of the Internet

- Telephone Network

  ..Telephone networks provides the basic infrastructure.

- Packet Networks
- ARPANET (1969)

..This early ARPANET network was funded and controlled by the United States government, and was used exclusively for academic and scientific purposes. The early network started small, with just a handful of connected university campuses and research institutions and companies in 1969.

- **X.25 (1974)**

..At the same time, alternative networks were created like X.25 in 1974, which allowed (and encouraged) business use.

- **USENET (1979)**

..USENET, built in 1979, had fewer restrictions still, and as a result grew quickly to 550 connected machines by 1981. It focuses on unrestricted access.

- **TCP/IP (1983) ⇓ INTERNET**

..To promote the growth and unification of the disparate networks, a suite of protocols was invented to unify the networks. A protocol is the name given to a formal set of publicly available rules that manage data exchange between two points. Communications protocols allow any two computers to talk to one another, so long as they implement the protocol. By 1981, protocols for the Internet were published and ready for use. New networks built in the United States began to adopt the TCP/IP (Transmission Control Protocol/Internet Protocol) communication model.

Internet vs. (World Wide) Web



"Internet is the hardware part - it is a collection of computer networks connected through either copper wires, fiber-optic cables or wireless connections whereas, the World Wide Web can be termed as the software part – it is a collection of web pages connected through hyperlinks and URLs. In short, the World Wide Web is one of the services provided by the Internet. Other services over the Internet include e-mail, chat and file transfer services. All of these services can

be provided to consumers for use by businesses or government or by individuals creating their own networks or platforms."

…[https://www.diffen.com/difference/Internet_vs_World_Wide_Web]

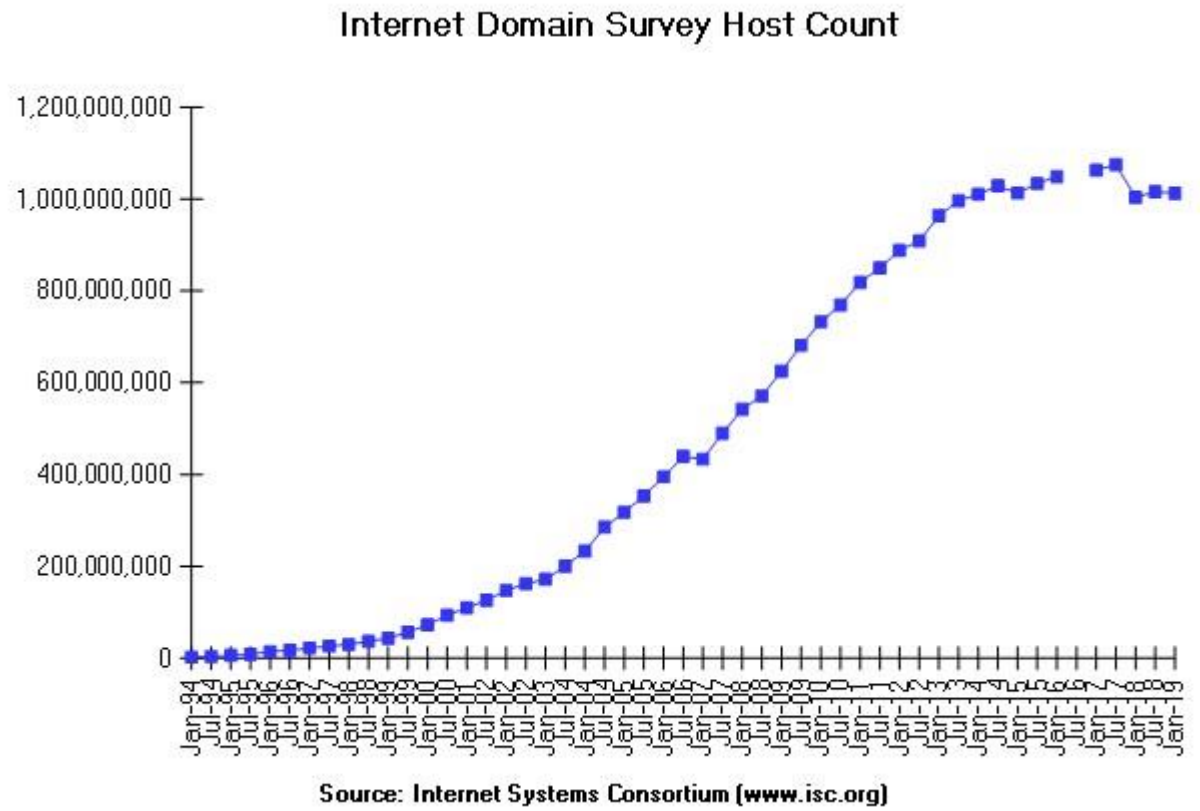
The Birth of the Web (1990)

The British Tim Berners-Lee (now Sir Tim Berners-Lee), who, along with the Belgian Robert Cailliau, published a proposal in 1990 for a hypertext system while both were working at CERN (European Organization for Nuclear Research) in Switzerland. Shortly thereafter Berners-Lee developed the main features of the web. This early web incorporated the following essential elements that are still the core features of the web today:

A Uniform Resource Locator (URL) to uniquely identify a resource on the WWW. The Hypertext Transfer Protocol (HTTP) to describe how requests and responses operate. A software program (later called web server software) that can respond to HTTP requests. Hypertext Markup Language (HTML) to publish documents. A program (later called a browser) that can make HTTP requests to URLs and that can display the HTML it receives.

Growth of Internet

Number of Internet Hosts increases tremendously between 2000 to 2015 for a figure of 1 Billion hosts.



Web Apps vs. Desktop Apps

Advantages

- Accessible from any Internet-enabled computer
- Cross platform and browser compatibility
- Easier to roll out program updates
- Fewer security concerns about local storage

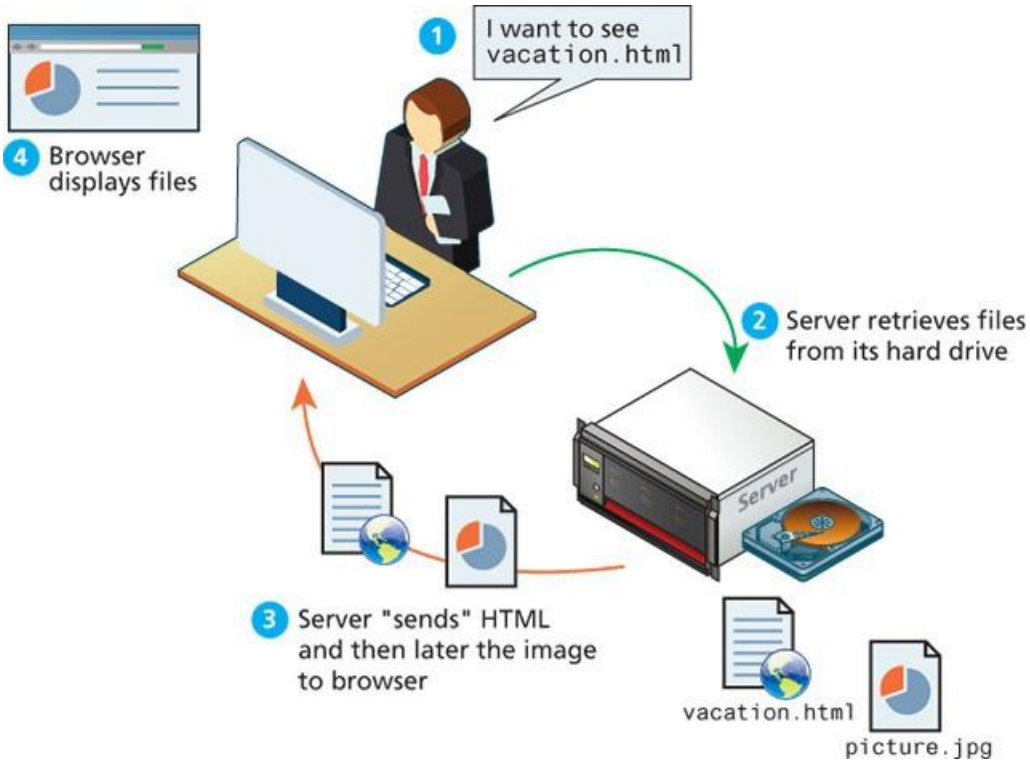
Disadvantages

- Requires internet
- Security concerns about sensitive private data being transmitted over the Internet
- Concerns over the storage, licensing, and use of uploaded data
- Problems with certain websites not having an identical appearance across all browsers
- Restrictions on access to operating system resources
- May need plugins for functioning

Lesson 3 : Static vs Dynamic Websites

Static website

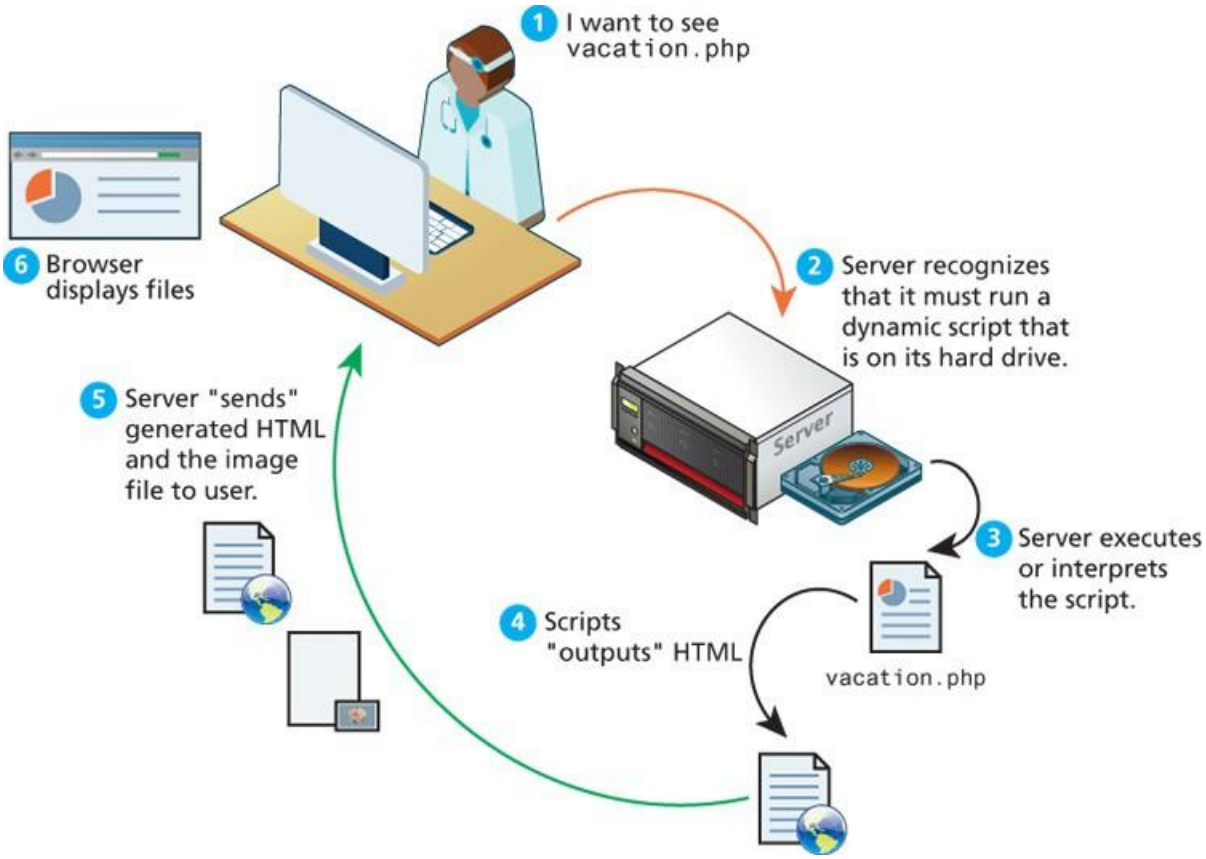
Static website consists only of HTML pages that look identical for all users at all times. A publisher would publish web pages and periodically update them. Users could read the pages but could not provide feedback. Following figure illustrates a simplified representation of the interaction between a user and a static website.



The illustration shows user at a desktop monitor saying, “I want to see vacation.html”. An arrow points from the user to a server machine which has its hard drive open. Server has second step defined as, “Server retrieves files from its hard drive”. Two files labeled “vacation.html” and “picture.jpg” are shown next to the hard drive. This leads to third step, depicting two files being transferred from server to the user's machine. This step is labeled as, “Server 'sends' HTML and then later the image to browser”. This leads to final and fourth step, which shows a webpage consisting a picture and text. Picture and text depicted on webpage is same as depicted on “vacation.html”, and “picture.jpg” files. This step is labeled as “Browser displays files”.

(Traditional or server-side) Dynamic Websites

Within a few years of the invention of the web, sites began to get more complicated as more and more sites began to use programs running on web servers to generate content dynamically. These server-based programs would read content from databases, interface with existing enterprise computer systems, communicate with financial institutions, and then output HTML that would be sent back to the users' browsers. This type of website is called a dynamic server-side website because the page content is being created at run time by a program created by a programmer; this page content can vary from user to user. Following figure illustrates a very simplified representation of the interaction between a user and a dynamic website.



The illustration shows six steps of the interaction as follows:

1. User at a terminal says, “I want to see vacation.php"”

2. Server recognizes that it must run a dynamic script that is on its hard drive (an arrow points from the user terminal to the Server, with an open hard drive).

3. Server executes or interprets the script. (an arrow points from the server to a page titled “vacation.php”

4. Scripts “outputs” HTML (an arrow points from vacation.php to a html script.

5. Server “sends” generated HTML and the image file to user. (an arrow points from html file to user terminal).

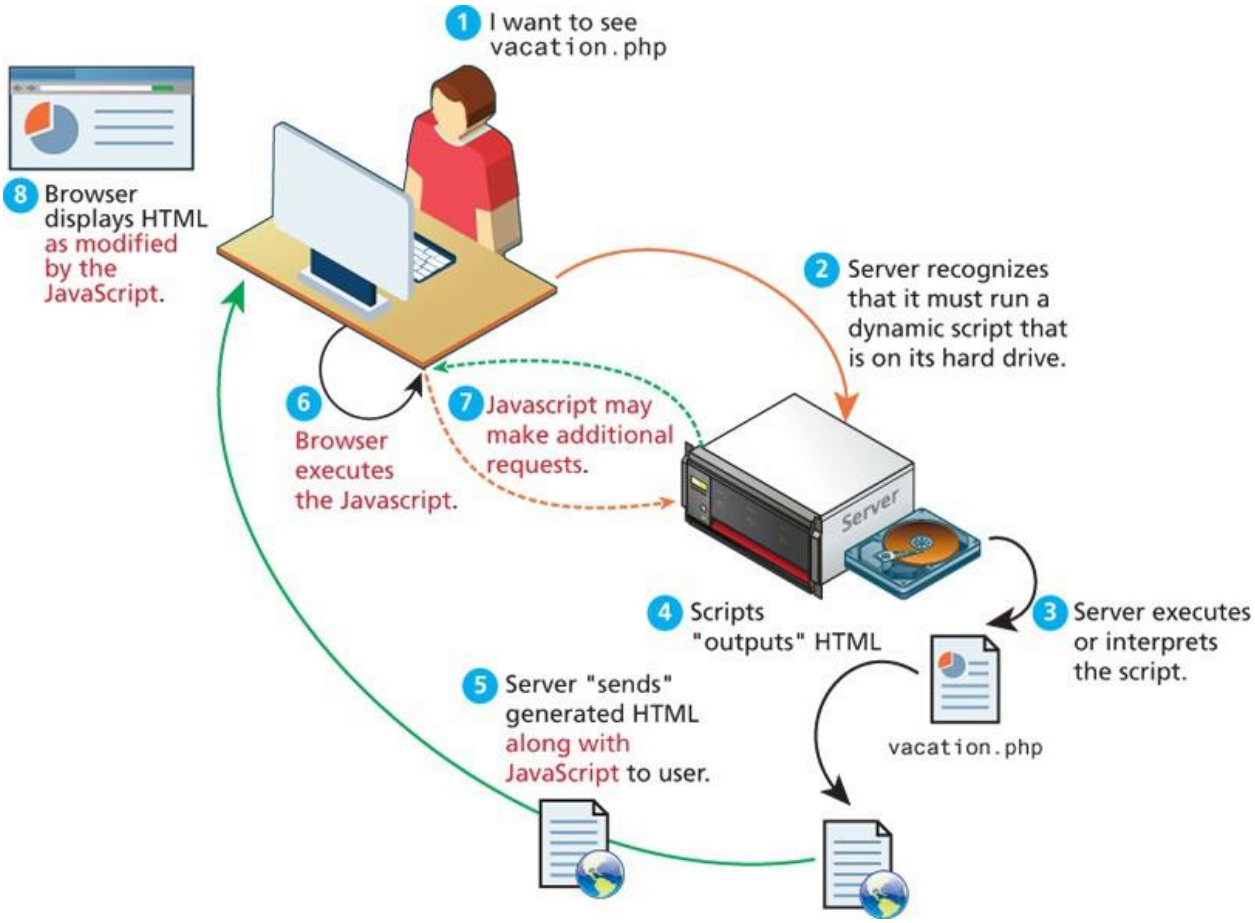
6. Browser displays files (An assembled webpage is displayed on the user's terminal).

(Modern) Dynamic Websites

Dynamic Websites of modern era (Web 2.0) refer to an interactive experience where users could contribute and consume web content, thus creating a more user-driven web experience.

For software developers, (Web 2.0) require more programming logic, which previously existed only on the server, began to migrate to the browser (see following figure).

This required learning JavaScript, a rather tricky programming language that runs in the browser, as well as mastering the rather difficult programming techniques involved in asynchronous communication.



The illustration shows six steps of interaction between an user and a dynamic website depicted as:

1. User at a terminal says, “I want to see vacation.php”

2. Server recognizes that it must run a dynamic script that is on its hard drive (an arrow points from the user terminal to the Server, with an open hard drive).

3. Server executes or interprets the script. (an arrow points from the server to a page titled “vacation.php”).

4. Scripts “outputs” HTML (an arrow points from vacation.php to an html script).

5. Server “sends” generated HTML and the image file to user. (an arrow points from html file to user terminal).

6. Browser executes the Javascript. (An arrow points back to the user's terminal).

7. Javascript may make additional requests (back and forth arrows between the user terminal and the server).

8. Browser displays HTML as modified by the Javascript (An assembled webpage is displayed on the user's terminal).

Lesson 4 : Brief History of Web

Web 1.0

In the earliest days of the web, a webmaster (the term popular in the 1990s for the person who was responsible for creating and supporting a website) would publish web pages and periodically update them.

Users could read the pages but could not provide feedback.

- Contents generated by publishers
- Company generated companies

Web 1.0 include both type:

- Static websites
- Dynamic websites

Web 2.0

Web 2.0 is mainly characterized by:

- User Generated Contents (UGC)
- Dynamic websites

In the mid-2000s, a new buzzword entered the computer lexicon: Web 2.0.

This term had two meanings, one for users and one for developers. For the users, Web 2.0 referred to an interactive experience where users could contribute and consume web content, thus creating a more user-driven web experience. Some of the most popular websites today fall into this category: Facebook, YouTube, and Wikipedia. This shift to allow feedback from the user, such as comments on a story, threads in a message board, or a profile on a social networking site has revolutionized what it means to use a web application.

For software developers, Web 2.0 also referred to a change in the paradigm of how dynamic websites are created. Programming logic, which previously existed only on the server, began to

migrate to the browser. This required learning JavaScript, a rather tricky programming language that runs in the browser, as well as mastering the rather difficult programming techniques involved in asynchronous communication.

Web 3.0

There is no standard consensus over Web 3.0 but some people have, however, argued that Web 3.0 will be something called the semantic web. It is mainly characterized by:

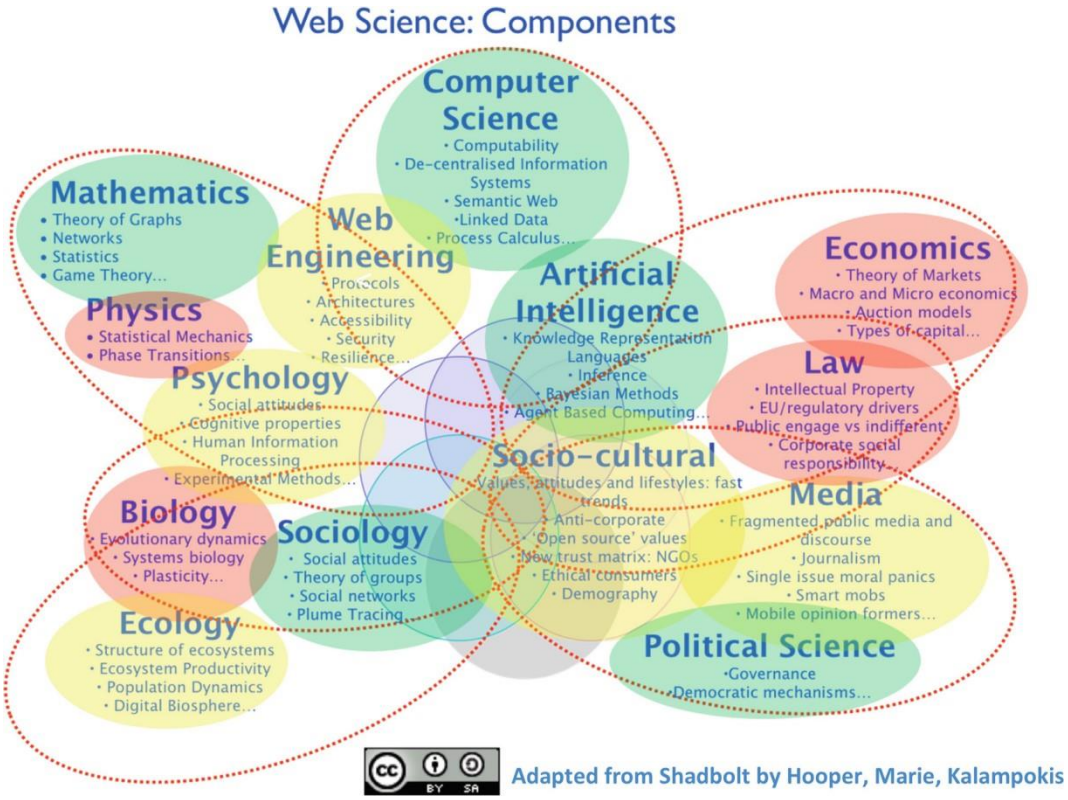
- Machine understandable contents
- Semantic web

Semantic is a word from linguistics that means, quite literally, "meaning." The semantic web thus adds context and meaning to web pages in the form of special markup. These semantic elements would allow search engines and other data-mining agents to make sense of the content. The goal of the semantic web is to make it easier to figure out those meanings, thereby dramatically improving the nature of search on the web.

Web Science

- Interdisciplinary study of sociotechnical integration of the world wide web.

Web Science, as it is known, studies the sociotechnical systems that apply the web in areas as diverse as finance, politics, activism, romance, and hate speech. If you look at each interaction on the web as more than just a technical exchange using protocols and file transmission, you can see there is often an underlying social need motivating each exchange. The technical system facilitates a social interaction and social interactions span nearly the entire human experience, so there is now an entire area of study looking at the web as a sociotechnical system. This is just another example of how the web can facilitate entirely new areas of study.

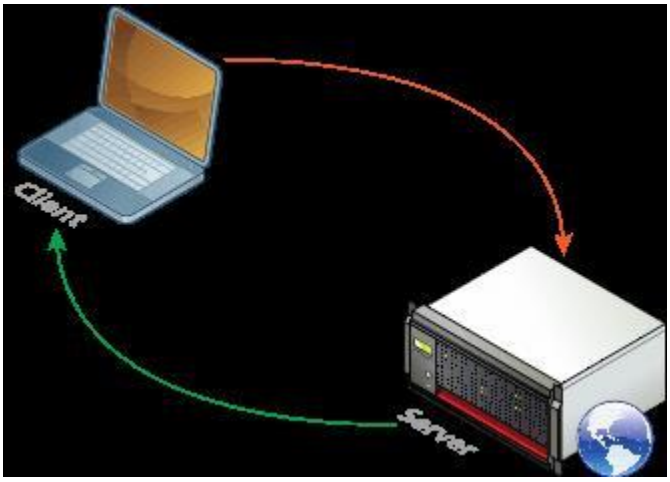


Lesson 5 : Client Server Model

Client server model

The web is sometimes referred to as a client server model of communications. In the client-server model, there are two types of actors: clients and servers. The server is a computer agent that is normally active 24/7, listening for requests from clients. A client is a computer agent that makes requests and receives responses from the server.

The request-response Loop



The diagram shows a laptop labelled as “client”. An arrow, labelled as “Request”, is pointing from the laptop to a server machine, depicted with a blue globe next to it. This server machine is labelled as “server”. Another arrow from the server machine to the laptop is labelled as “Response”.

Server Types

Most real-world websites are typically not served from a single server machine, but by many server machines. It is common to split the functionality of a website between several different types of server, as shown in following figure. These include the following:

Web servers

A web server is a computer servicing HTTP requests. This typically refers to a computer running web server software, such as Apache or Microsoft IIS (Internet Information Services).

Application servers

An application server is a computer that hosts and executes web applications, which may be created in PHP, ASP.NET, Ruby on Rails, or some other web development technology.

Database servers

A database server is a computer that is devoted to running a Database Management System (DBMS), such as MySQL, Oracle, or MongoDB, that is being used by web applications.

Mail servers

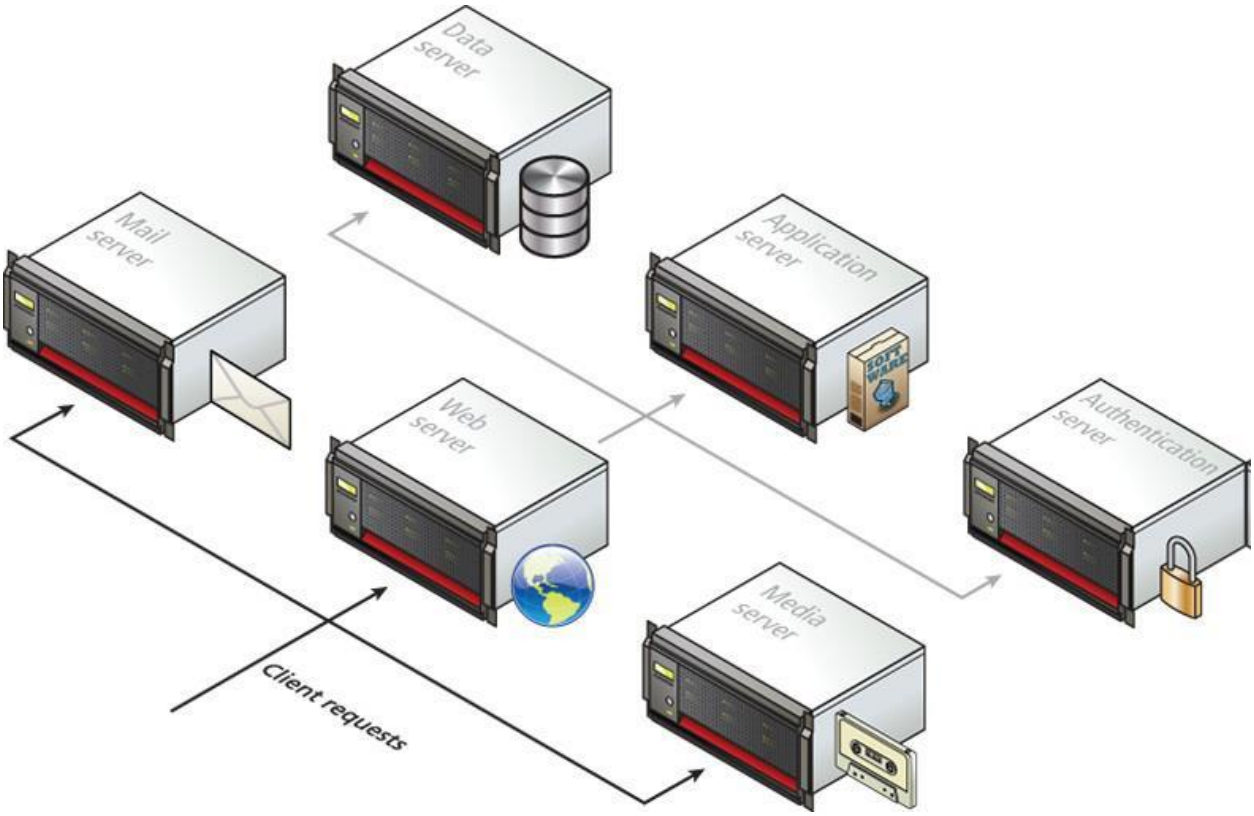
A mail server is a computer creating and satisfying mail requests, typically using the Simple Mail Transfer Protocol (SMTP).

Media servers

A media server (also called a streaming server) is a special type of server dedicated to servicing requests for images and videos. It may run special software that allows video content to be streamed to clients.

Authentication servers

An authentication server handles the most common security needs of web applications. This may involve interacting with local networking resources, such as LDAP (Lightweight Directory Access Protocol) or Active Directory.

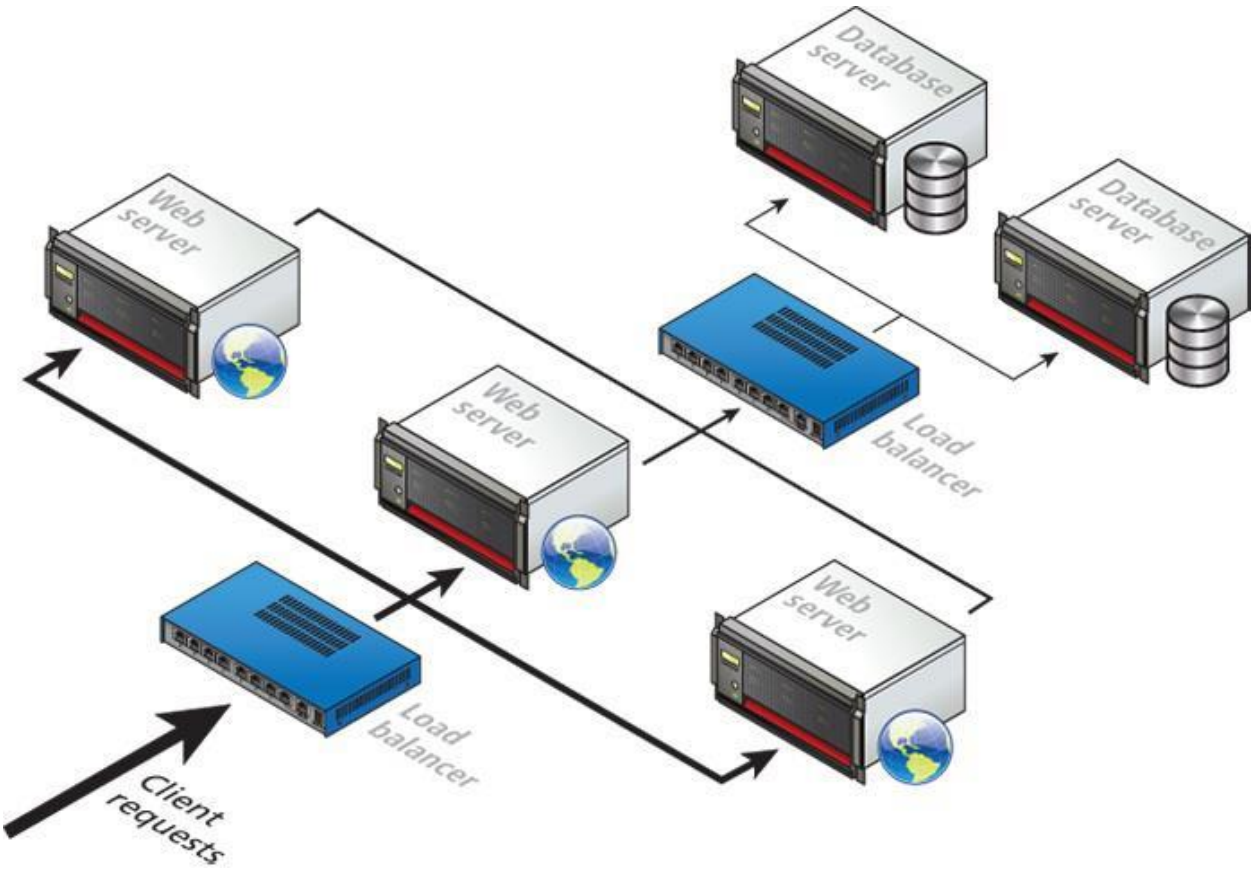


In smaller sites, these specialty servers are often the same machine as the web server.

Server Farms

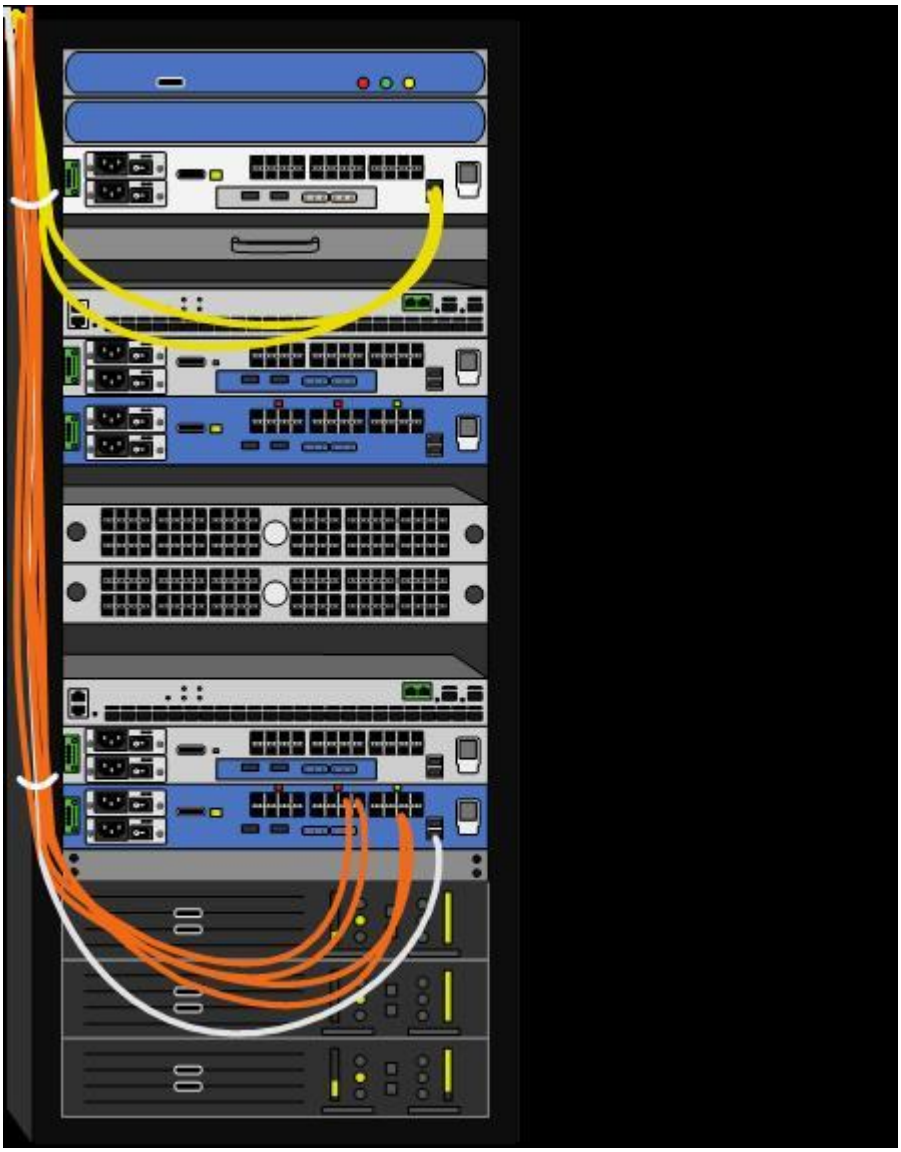
A single web server that is also acting as an application or database server will be hard-pressed to handle more than a few hundred requests a second. A busy site can receive thousands or even tens of thousands of requests a second; globally popular sites such as Facebook receive millions of requests a second.

The usual strategy for busier sites is to use a server farm. The goal behind server farms is to distribute incoming requests between clusters of machines so that any given web or data server is not excessively overloaded, as shown in following figure. Special devices called load balancers distribute incoming requests to available machines.



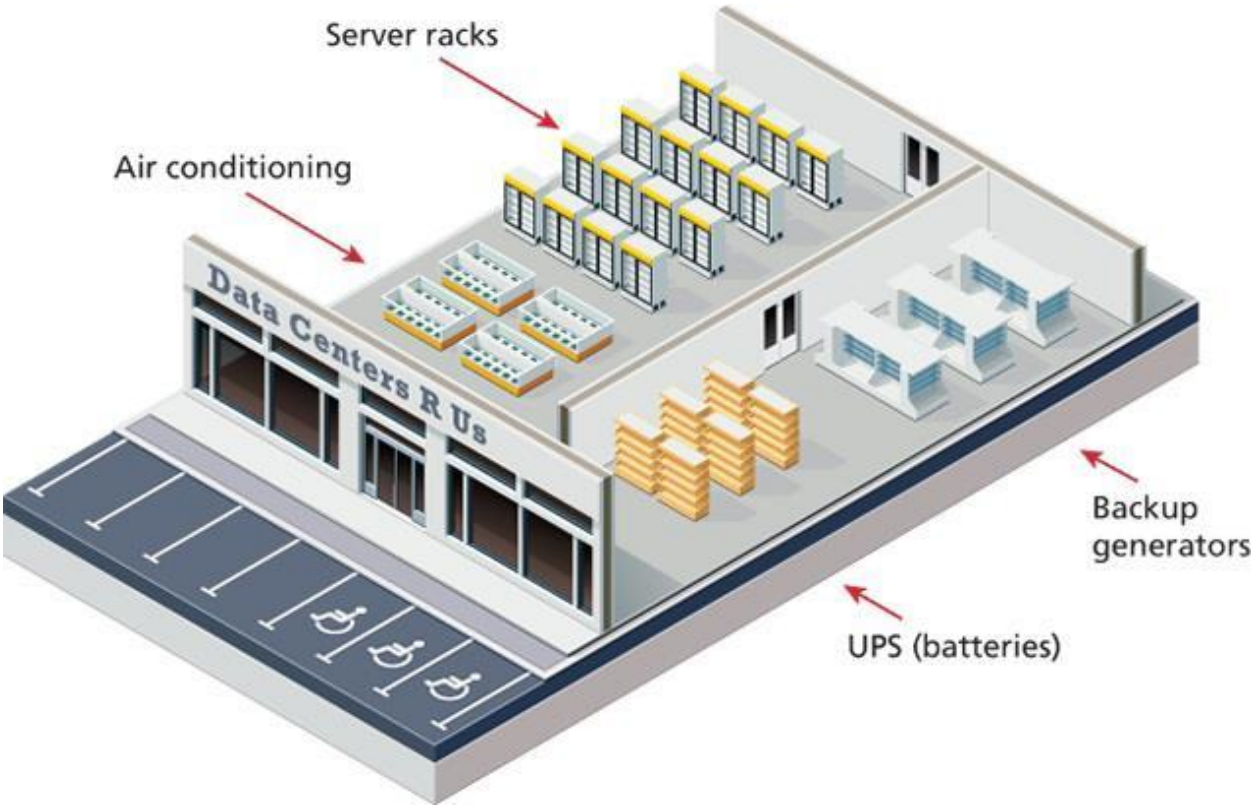
Server Racks

In a server farm, the computers do not look like the ones in your house. Instead, these computers are more like the plates stacked in your kitchen cabinets. That is, a farm will have its servers and hard drives stacked on top of each other in server racks. A typical server farm will consist of many server racks, each containing many servers, as shown in following figure.



Data Centers

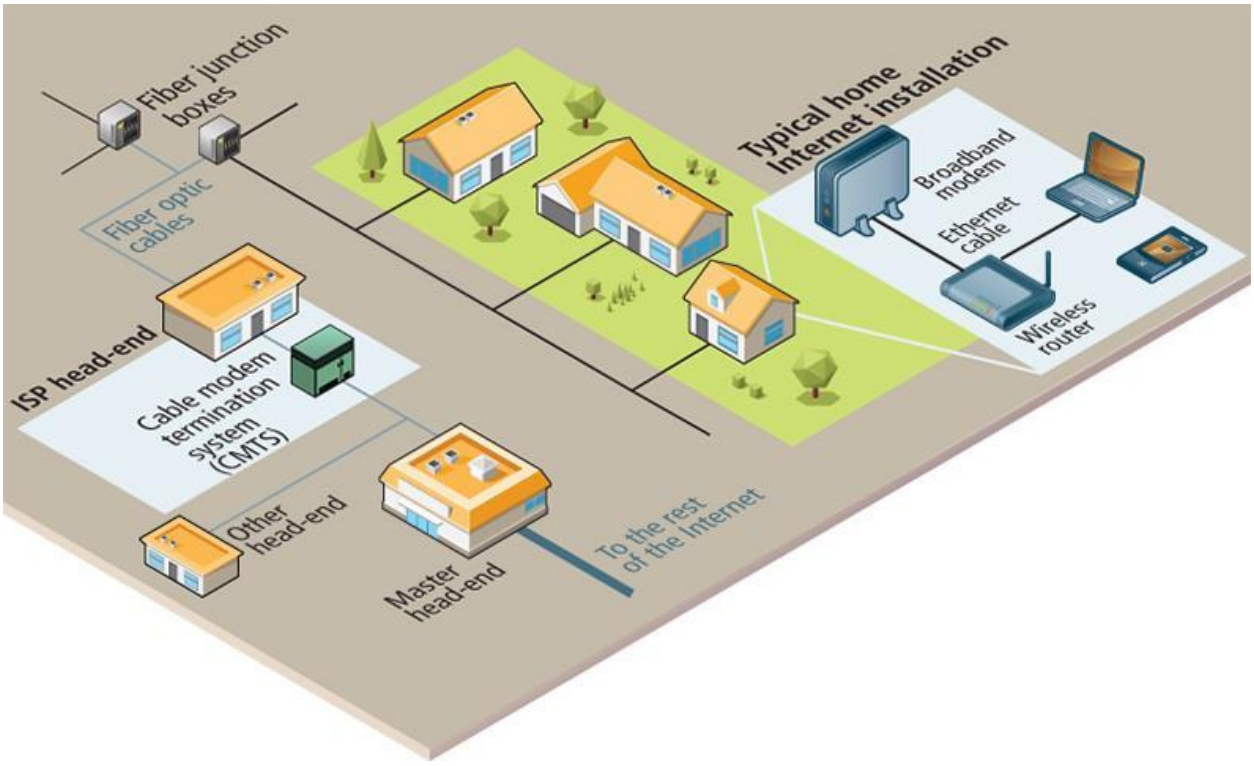
Server farms are typically housed in special facilities called data centers. A data center will contain more than just computers and hard drives; sophisticated air conditioning systems, redundancy power systems using batteries and generators, specialized fire suppression systems, and security personnel are all part of a typical data center, as shown in following figure.



Lesson 6 : How Internet Works

Internet from Home to ISP

While there are many configuration possibilities, following figure does provide an approximate simplification of a typical home to local Internet Server Provider (or ISP) setup.



The broadband modem, also called a cable modem or DSL (digital subscriber line) modem, is a bridge between the network hardware outside the house (typically controlled by a phone or cable company) and the network hardware inside the house. The wireless router is perhaps the most visible manifestation of the Internet in one's home.

The various neighbourhood broadband cables are aggregated and connected to fiber optic cable via fiber connection boxes.

These fiber optic cables eventually make their way to an ISP's head-end, which is a facility that may contain a cable modem termination system (CMTS) or a digital subscriber line access multiplexer (DSLAM) in a DSL based system. This is a special type of very large router that connects and aggregates subscriber connections to the larger Internet. These different headends may connect directly to the wider Internet, or instead be connected to a master head-end, which provides the connection to the rest of the Internet.

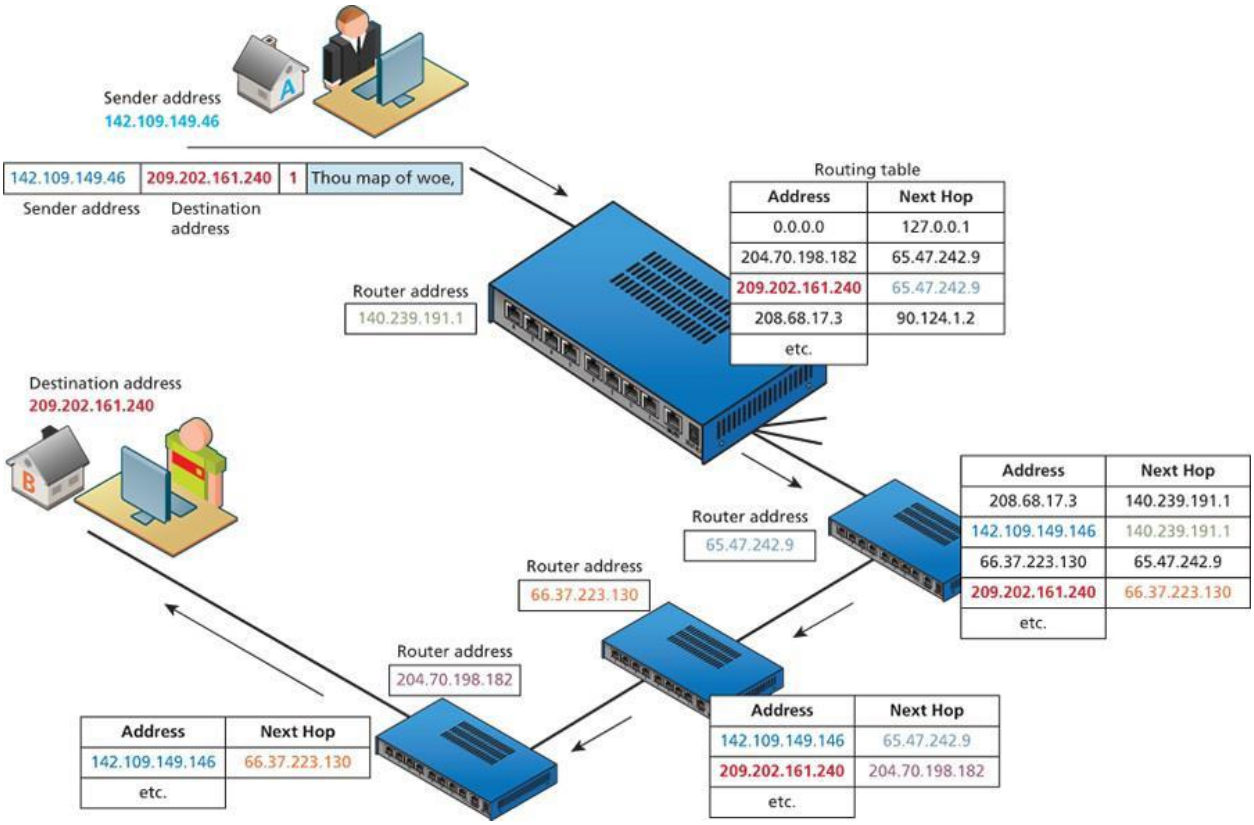
Routing Tables

The flow of internet traffic or information is carried out mainly by routers. A router is a hardware device that forwards data packets from one network to another network. When the router receives a data packet, it examines the packet's destination address and then forwards it to another destination. A router uses a routing table to help determine where a packet should be sent. It is a table of connections between target addresses and the destination (typically another router) to which the router can deliver the packet. In following figure, the different routing tables use next-hop routing, in which the router only knows the address of the next step of the

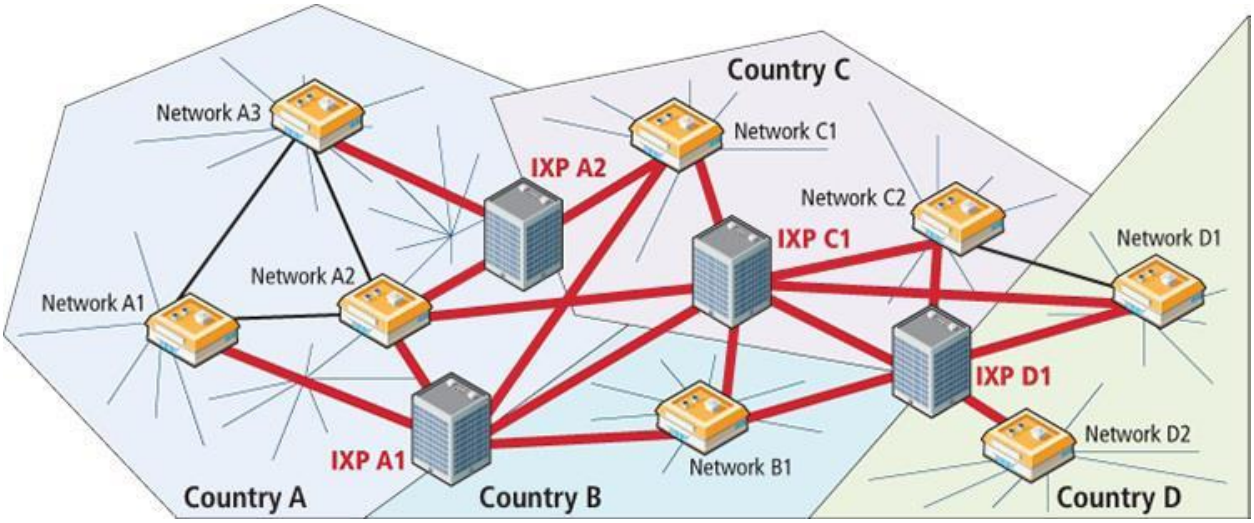
path to the destination; it leaves it to that next step to continue the routing process. The packet thus makes a variety of successive hops until it reaches its destination.

From ISP to the World

Internet exchange point (IX or IXP): Dedicated infrastructure to make the internet more connected. Internet exchange point (IX or IXP) provides a way for networks

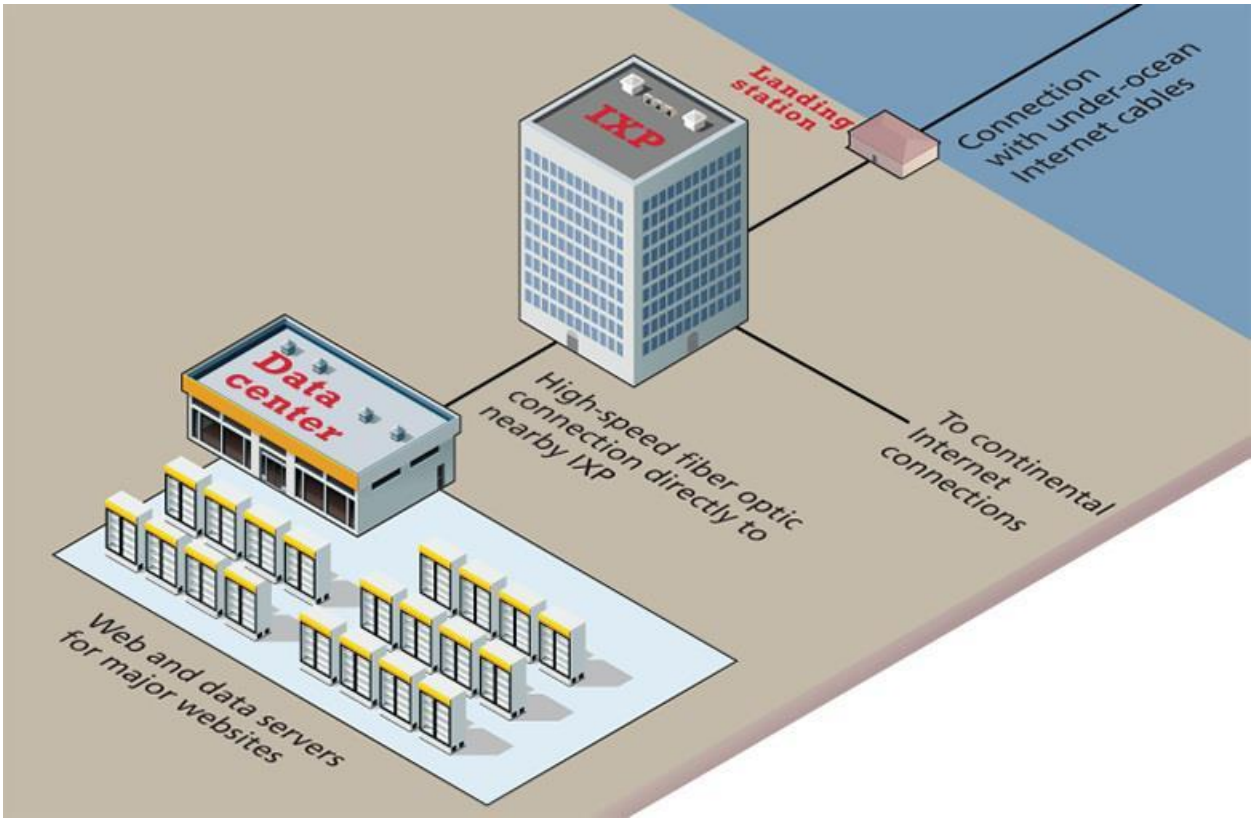


within a country to interconnect and between different countries. More and more networks began to interconnect with each other using an Internet exchange point (IX or IXP). Multiple paths between IXPs provide a powerful way to handle outages and keep packets flowing. Another key strength of IXPs is that they provide an easy way for networks to connect to many other networks at a single location.



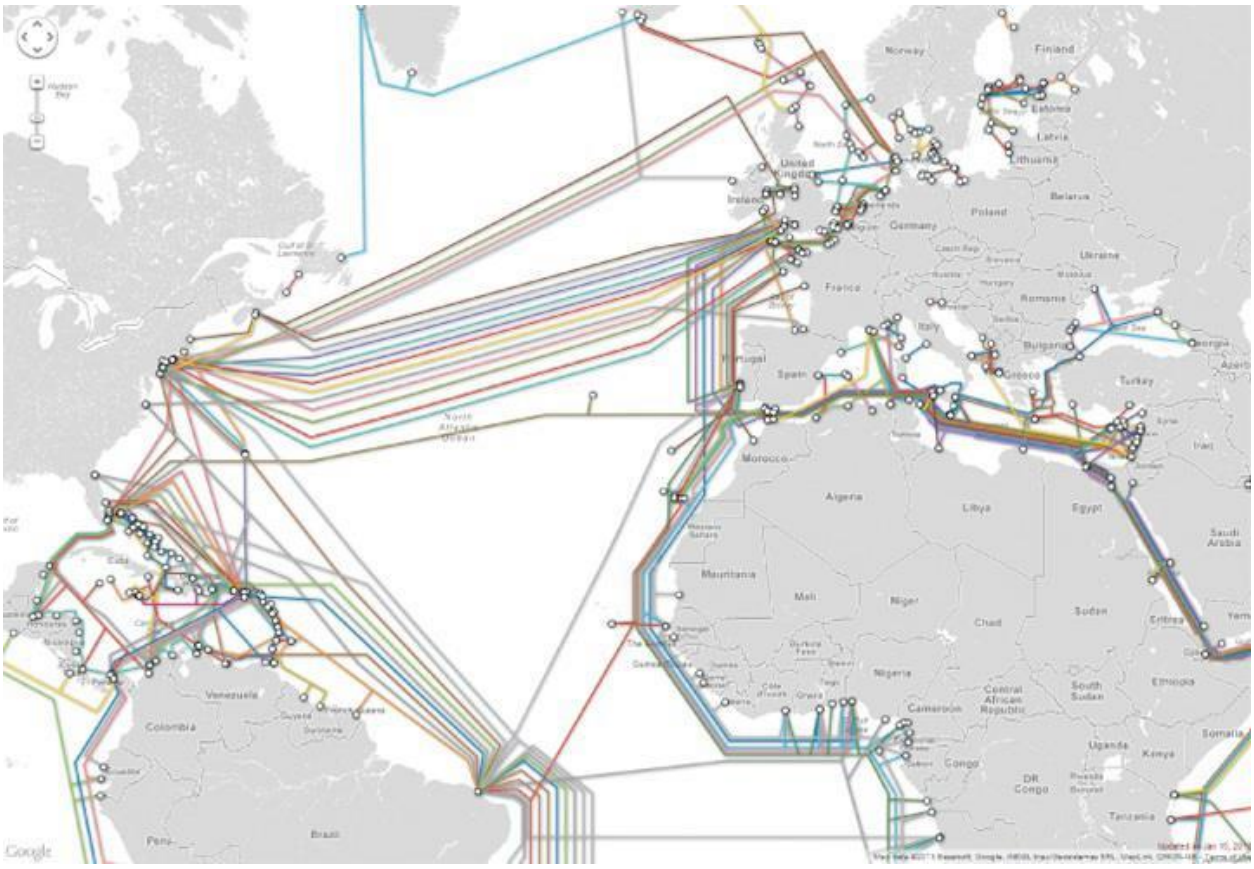
IXPs and data centers

In recent years, major web companies have joined the network companies in making use of IXPs. As shown in following figure, this sometimes involves mirroring (duplicating) a site's infrastructure (i.e., web and data servers) in a data center located near the IXP. The website will have incremental speed enhancements (by reducing the travel distance for these sites) across all the networks it is peered with at the IXP, while the network will have improved performance for its customers when they visit the most popular websites.



Under-ocean connectivity

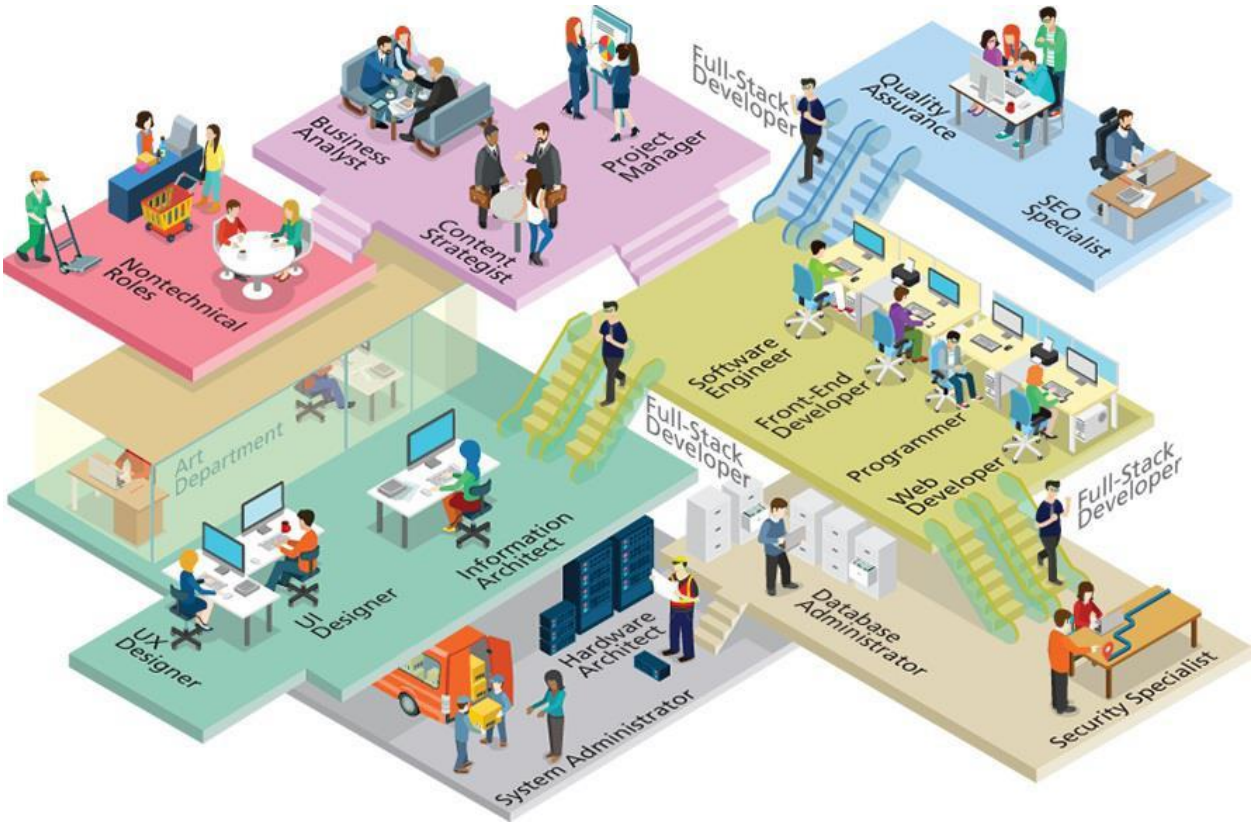
Eventually, international Internet communication will need to travel underwater. The amount of undersea fiber optic cable is quite staggering and is growing yearly. As can be seen in following figure, over 250 undersea fiber optic cable systems operated by a variety of different companies span the globe.



Lesson 7 : Web Development Jobs

Web Engineering Jobs

Following figure illustrates the complex ecosystem that is contemporary web development. Seeing that diagram, you should learn that there are many different jobs that one can do within the web development world.



Roles and Skills

Following list of job titles provides an overview of the roles typically available in a web development company as part of a team.

Hardware Architect/Network Architect/Systems Engineer

The people who design the specifications for the servers in a data center, and design and manage the layout of the physical and logical network are essential somewhere along the way, whether at your company or your host's. Typically, these roles require networking and operating systems knowledge.

System Administrator

Once the system is built and wired to the network, system administrators are the next people required to get things up and running. Often they choose and install the network operating system, then manage the shared operating system environments for other users. This position is often combined with the hardware architect in smaller firms.

Database Administrator/Data Architect

The database administrator (sometimes abbreviated as DBA) is a role found in larger companies. In these companies, there are many databases, often from many divisions, all of which need to be managed, secured, and backed up. Database administrators will perform maintenance on the

databases as well as manage access for user and software accounts. They sometimes write triggers and advanced queries for users upon request as well as manage database indexes.

A data architect has some overlap with database administrator, but the role is more focused on the design and integration of data. In recent years, managing and making use of large sets of often unrelated data has become increasingly important for web companies. In smaller companies, these different data roles are often combined with the system administrator and/or developer ones.

Security Specialist/Consultant/Expert

A good system administrator and network architect will certainly have insights into security as they perform their duties. However, because security is so vital to web development in general, and because the knowledge necessary to do security work is complicated and ever changing, it is not uncommon for companies to outsource their security needs to security specialists. These specialists will test for vulnerabilities, implement security best practices, and make updates and changes to programming code or hardware infrastructure to protect a site against well-known or newly emerging (called zero-hour) threats.

Developer/Programmer

Programmers can be assigned a wide range of tasks aside from simple coding. Writing good documentation, using version control software, engaging in code reviews, running test cases, and more might be typical tasks, depending on company practices. Programmer positions often begin at the entry level, with higher-level design decisions left to software engineers and senior developers. In terms of the web development world, the terms programmers or developer are quite broad; typically, however, this term is used to indicate a job focused more on server-side development using languages like PHP.

Front-End Developer/UX Developer

Increasingly complex front-end development requires software developers with an aptitude for graphical user interface design (nowadays more typically referred to as user-experience or UX design) and an understanding of human–computer interaction (HCI) principals. This typically requires in-depth JavaScript expertise along with good CSS skills. Another increasingly commonly used synonym for front-end developer is UX developer. The main difference between a UX developer and a UX/UI designer (described below), is that the UX developer is involved mainly in the implementation of the user experience and less in the actual design of it.

Software Engineer

A software engineer is a programmer who is adept at the language of analysis and design, and uses established best practices in the development of software. Sometimes the role of a programmer and software engineer are used interchangeably, but a software engineer has more knowledge of the software development life cycle and can effectively gather requirements and speak with clients about technical and business matters.

UX Designer/UI Designer/Information Architect

These are names used somewhat interchangeably for jobs that focus on the structure, design and usability of a website. Once referred to as the user interface, the term UX has become the preferred term because improving how a website is used is just as important (or even more important) nowadays as improving how a website appears. While coding skills can be helpful, this type of work more often involves the development of prototypes, making mockup designs, and analyzing user experience data. In larger web development firms, this type of work also commonly involves working in conjunction with creatives in the art department.

Tester/Quality Assurance

Testers are the people who try to identify flaws in software before it gets released. This type of work is often called quality assurance (QA). Although some test roles are for nonexperts, many testers know how to program and might write automated tests as well as develop testing plans from requirements. Although these duties are often integrated with developers, they can form a job all their own.

SEO Specialist

Search engine optimization (SEO) refers to the process of improving the discoverability of web content by search engines. Chapter 23 covers both the above board (as well as the under-handed) techniques used to improve SEO results. An SEO specialist needs to be familiar with these techniques as well as analytics, testing approaches, social networking APIs, and even content creation strategies.

Content Strategists/Marketing Technologist

Regardless of technological features, websites ultimately succeed due to the quality of their content. A content strategist (sometimes also called a marketing technologist) is someone who uses his or her experience with existing and emerging web technologies in conjunction with knowledge about the audience to craft engaging web content. This type of work might also be done by an SEO specialist or an information architect. Writing and marketing skills as well as knowledge of content management systems, email services, and social networking interfaces are important for this job.

Project Manager/Product Manager

Websites are complicated projects often involving the work of many different people with different skill sets and personalities. Getting all these people to work together in a timely and effective manner typically requires the committed effort and knowledge of project managers (also called product managers). Knowledge of planning and estimation methodologies is helpful, as are more general people management skills.

Business Analyst

Although a software engineer in an analysis role might speak to clients and get requirements, that role is often given a different name and assigned to someone with especially good

communication skills. A business analyst is the interface between the various divisions of the company and the website (and IT in general). These people can easily speak to the HR, marketing, and legal divisions, and then translate those requirements into tasks that software engineers can take on.

Nontechnical Roles

Aside from all the technical roles above, there are additional important roles that require expertise outside of technology. These roles include traditional ones found in almost every company: accountants, writers, designers, editors, lawyers, salespeople, and managers. There are also a wide variety of new roles that are unique to the web space,10 such as analytics manager, motion designer, social media analyst, cloud architect, and the intriguingly named growth hacker. Getting people from different backgrounds with different expertise to work together is how companies balance the business, technology, and art of website development.

Full-stack developer

Rather than specializing in server-side development, or client-side user experience construction, or database administration, a full-stack developer ideally has competency and experience in all of these domains. Indeed, many companies even expect full-stack developers to be knowledgeable about various system administration tasks, such as setting up a web server and handling security issues.

Web Development Companies

A major factor to consider when thinking about a career in web development is what kind of company you want to work for. Sure, everyone needs a website, but there are multiple kinds of companies that work together to make that a possibility (illustrated in following figure).



Hosting Companies

There are companies that will manage servers on your behalf. These hosting companies or data centers offer many employment opportunities, especially related to hardware, networking, and system administration roles.

Design Companies

Design companies are at the opposite end of the spectrum, with few technical positions available. These firms will provide professional artistic and design services that might go beyond the web and include logos and branding in general. Some companies produce mockups in Photoshop, for example, which a web developer (at another company) can then turn into a website.

Website Solution Companies

Website solution companies focus on the programming and deployment of websites for their clients. There are technical positions to help manage the existing sites (working in conjunction with hosting companies) as well as development jobs to build the latest custom site.

Vertically Integrated Companies

Vertically integrated companies are increasingly becoming the one-stop shop for web development. They are called vertically integrated because these companies combine hosting, design, and application solutions into one company. This allows these companies to achieve economies of scale and appeal to nontechnical clients who can go there for all their web-related needs, large or small.

Start-Up Companies

Start-up ventures in web development have been some of the biggest success stories in the business world. Start-ups are often attractive places for new graduates to work, with less competition from experienced candidates and potentially lots of jobs available from developers to designers and system administrators. The smaller start-ups companies often require full-stack developers, who can take on any role from system administrator through to lead developer.

Internal Web Development

Although many companies outsource their web presence, others assign the work to an internal division, normally under the umbrella of IT or marketing. Although many of these roles are simple caretaker positions, others can be quite engaging, requiring real programming expertise. Many companies have lots of internal data they would not share with outsiders and thus prefer inhouse expertise for the development of web interfaces and systems to manage and display that confidential data. Often these websites exist only with an organization's Intranet rather than as public websites on the Internet.

Lesson 8 : Internet Protocols

Internet Protocols

A protocol is a set of rules that partners use when they communicate.

TCP/IP

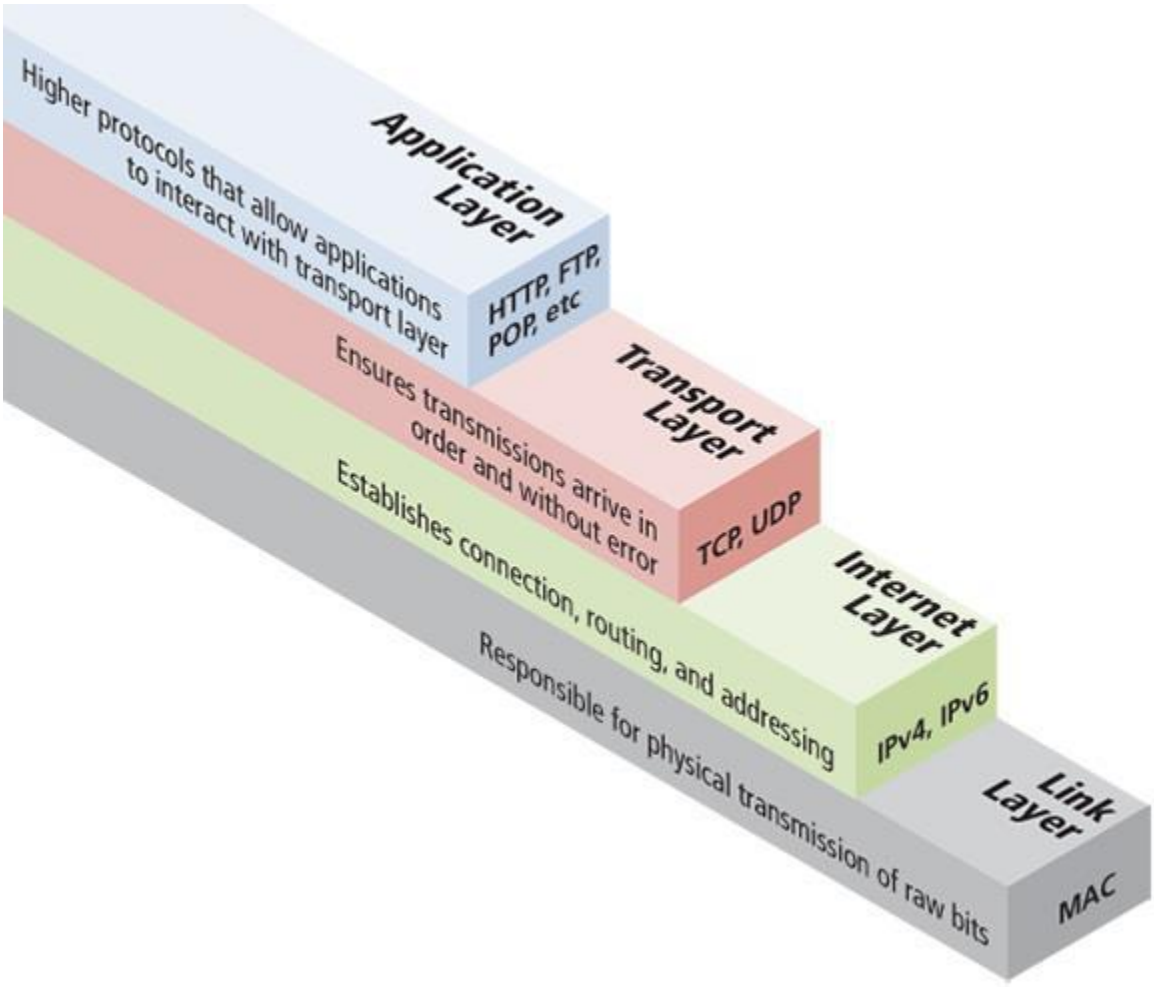
These protocols have been implemented in every operating system, and make fast web development possible.

Networking is its own entire discipline.

Web developer needs general awareness of what the suite of Internet protocols does.

Layered Architecture

The TCP/IP Internet protocols were originally abstracted as a four-layer stack.



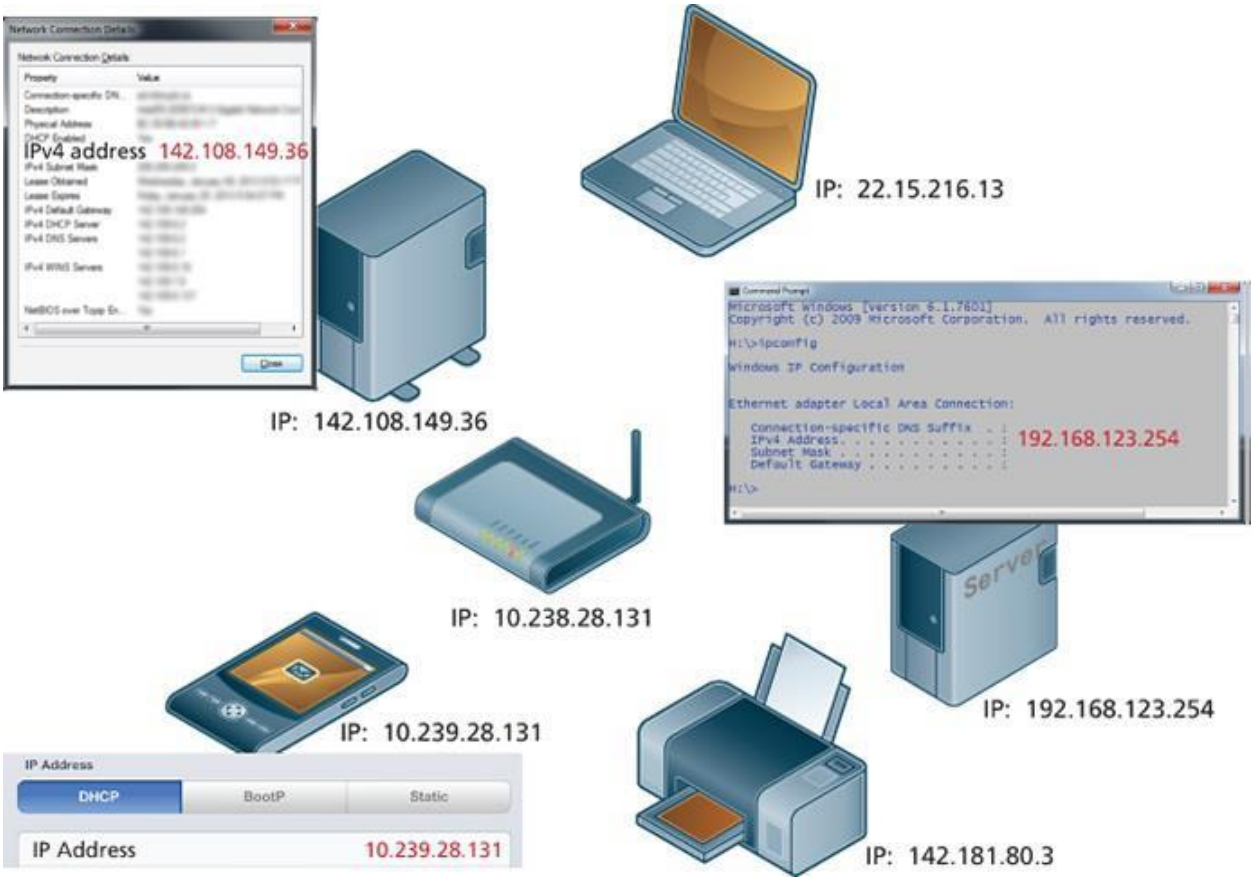
Link Layer

The link layer is the lowest layer, responsible for both the physical transmission of data across media (both wired and wireless in the form of raw bits, light waves etc) and establishing logical links. It handles issues like packet creation, transmission, reception, error detection, collisions, line sharing, and more. MAC (media access control) addresses are unique 48- or 64-bit identifiers assigned to network hardware and which are used at the link layer.

Internet Layer

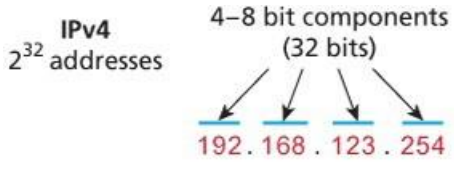
The Internet layer (sometimes also called the IP Layer) routes packets between communication partners across networks. The Internet layer provides “best effort” communication. It sends out a message to its destination, but expects no reply, and provides no guarantee the message will arrive intact, or at all.

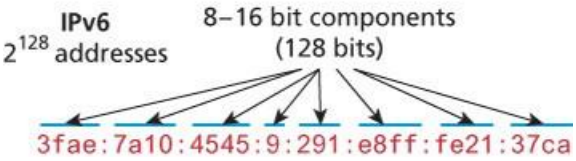
The Internet uses the Internet Protocol (IP) addresses, which are numeric codes that uniquely identify destinations on the Internet. As can be seen in following figure, every device connected to the Internet has such an IP address.



IP-Address

There are two types of IP addresses: IPv4 and IPv6. IPv4 addresses are the IP addresses from the original TCP/IP protocol. In IPv4, 12 numbers are used (implemented as four 8-bit integers), written with a dot between each integer (see following figure). Since an unsigned 8-bit integer's maximum value is 255, four integers together can encode approximately 4.2 billion unique IP addresses.



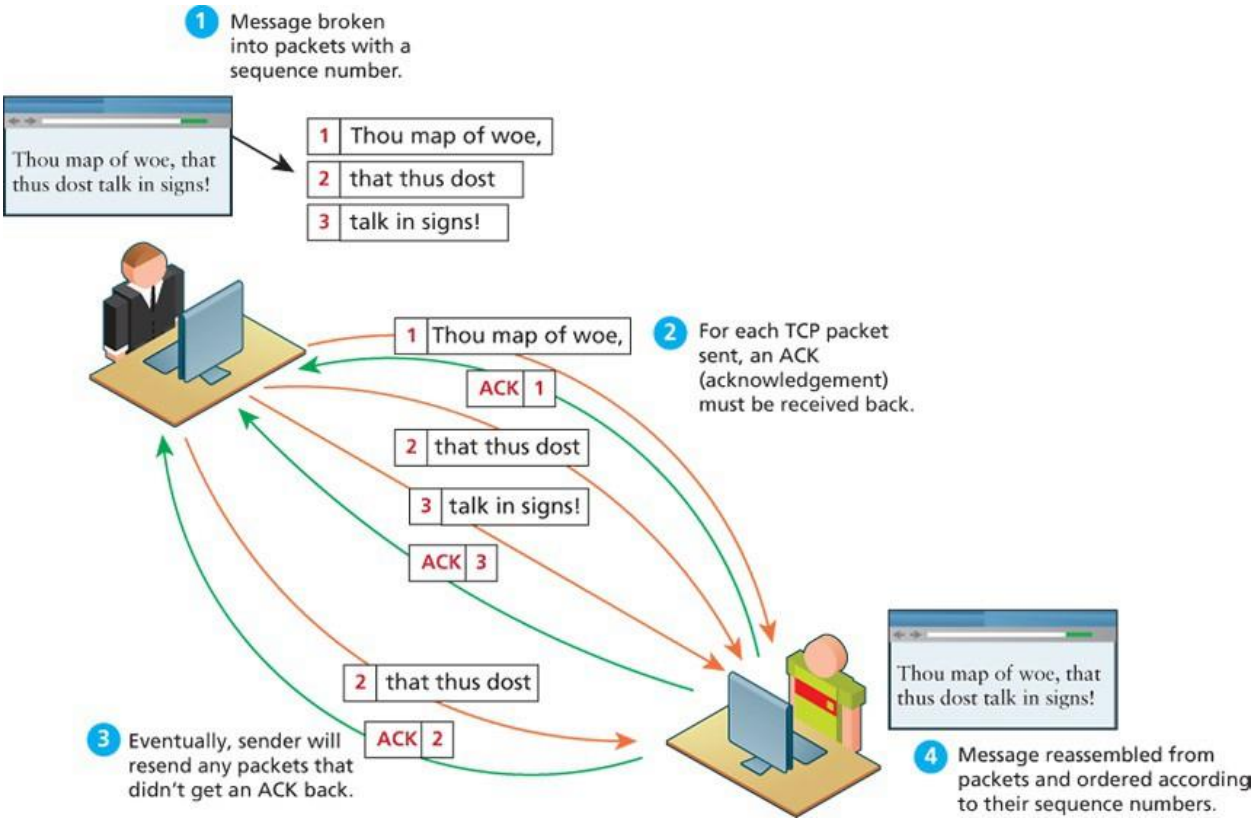


The decision to make IP addresses 32 bits limited the number of hosts to 4.2 billion. As more and more devices connected to the Internet the supply of addresses dwindled, especially in some local areas that had already distributed their allotment.

To future-proof the Internet against the 4.2 billion limit, a new version of the IP protocol was created, IPv6. This newer version uses eight 16-bit integers for 2128 unique addresses, over a billion billion times the number in IPv4. These 16-bit integers are normally written in hexadecimal, due to their longer length.

Transport Layer

The transport layer ensures transmissions arrive in order and without error. This is accomplished through a few mechanisms. First, the data is broken into packets formatted according to the Transmission Control Protocol (TCP). Each data packet has a header that includes a sequence number, so the receiver can put the original message back in order, no matter when they arrive. Secondly, each packet acknowledges its successful arrival back to the sender so in the event of a lost packet, the transmitter will realize a packet has been lost since no ACK arrived for that packet. That packet is retransmitted, and although out of order, is reordered at the destination, as shown in following figure. This means you have a guarantee that messages sent will arrive and will be in order.



Sometimes we do not want guaranteed transmission of packets. Consider a live multicast of a soccer game, for example. Millions of subscribers may be streaming the game, and the broadcaster can't afford to track and retransmit every lost packet. A small loss of data in the feed is acceptable, and the customers will still see the game. An Internet protocol called User Datagram Protocol (UDP) is used in these scenarios in lieu of TCP. Other examples of UDP services include Voice Over IP, many online games, and Domain Name System (DNS).

Application Layer

Application layer protocols implement process-to-process communication and are at a higher level of abstraction in comparison to the low-level packet and IP address protocols in the layers below it. There are many application layer protocols. A few that are useful to web developers include the following:

HTTP. The Hypertext Transfer Protocol is used for web communication.

SSH. The Secure Shell Protocol allows remote command-line connections to servers.

FTP. The File Transfer Protocol is used for transferring files between computers.

POP/IMAP/SMTP. Email-related protocols for transferring and storing email.

DNS. The Domain Name System protocol used for resolving domain names to IP addresses.

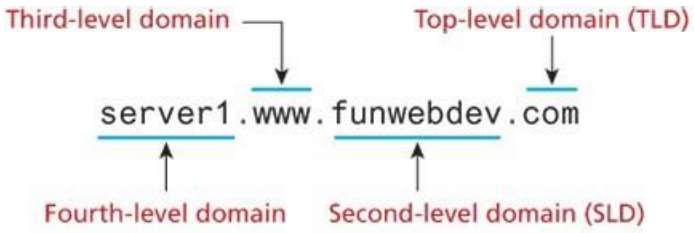
Lesson 9 : Domain Name System

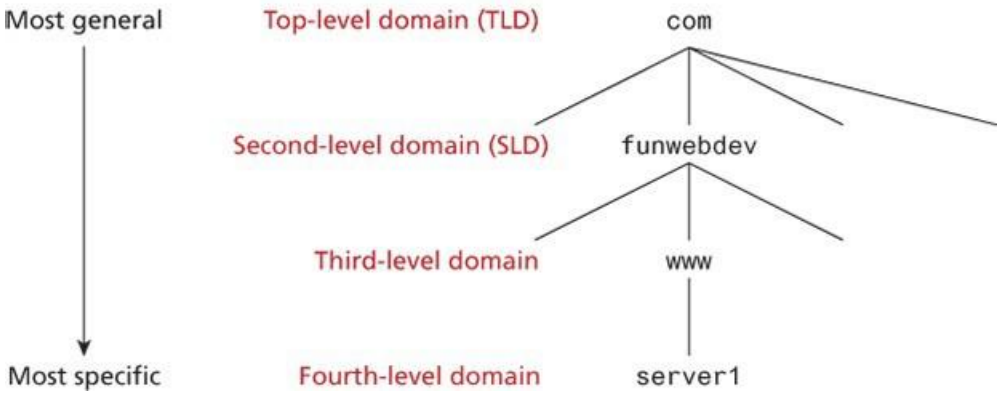
Domain Name System

DNS is one of the core systems that make an easy-to-use Internet possible (DNS is used for email as well).One can imagine how unpleasant the Internet would be if you had to remember IP addresses instead of names. Rather than google.com, you'd have to type 216.58.216.78.

Name levels in a domain

A domain name can be broken down into several parts. They represent a hierarchy, with the rightmost parts being closest to the root at the “top” of the Internet naming hierarchy. All domain names have at least a top-level domain (TLD) name and a second-level domain (SLD) name. Most websites also maintain a third-level WWW subdomain and perhaps others. Following figure illustrates a domain with four levels.





Types of top level domains

The rightmost portion of the domain name (to the right of the rightmost period) is called the top-level domain. For the top level of a domain, we are limited to two broad categories, plus a third reserved for other use. They are:

Generic top-level domain (gTLD)

Unrestricted. TLDs include .com, .net, .org, and .info.

Sponsored. TLDs including .gov, .mil, .edu, and others. These domains can have requirements for ownership and thus new second-level domains must have permission from the sponsor before acquiring a new address.

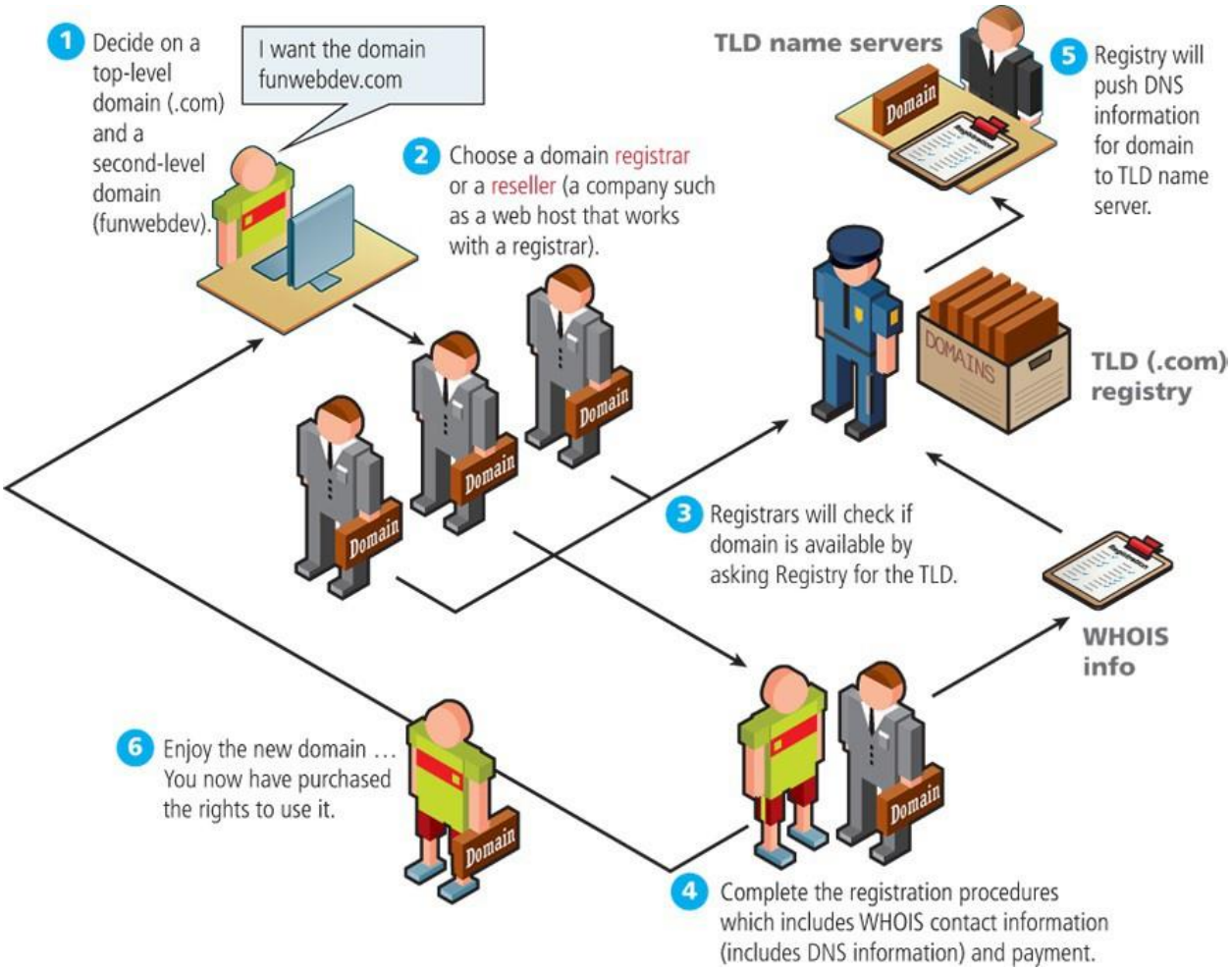
New. From January to May of 2012, companies and individuals could submit applications for new TLDs. TLD application results were announced in June 2012, and include a wide range of both contested and single applicant domains. These include corporate ones like .apple, .google, and .macdonalds, and contested ones like .buy, .news, and .music.5

Country code top-level domain (ccTLD)

- TLDs include .us, .ca, .uk, and .au. At the time of writing, there were 252 codes registered.6 These codes are under the control of the countries which they represent, which is why each is administered differently. In the United Kingdom, for example, commercial entities and businesses must register subdomains to co.uk rather than second-level domains directly. In Canada, .ca domains can be obtained by any person, company, or organization living or doing business in Canada. Other countries have peculiar extensions with commercial viability (such as .tv for Tuvalu) and have begun allowing unrestricted use to generate revenue.
- Since some nations use nonwestern characters in their native languages, the concept of the internationalized top-level domain name (IDN) has also been tested with great success in recent years. Some IDNs include Greek, Japanese, and Arabic domains (among others) which have test domains at http://παράδειγμα.δοιμή, http:// 例え.テスト, and http:// إختبار.مثال , respectively.

Domain registration

Special organizations or companies called domain name registrars manage the registration of domain names. These domain name registrars are given permission to do so by the appropriate generic top-level domain (gTLD) registry and/or a country code top-level domain (ccTLD) registry. In the 1990s, a single company (Network Solutions Inc.) handled the com, net, and org registries. By 1999, the name registration system changed to a market system in which multiple companies could compete in the domain name registration business. A single organization—the nonprofit Internet Corporation for Assigned Names and Numbers (ICANN)—still oversees the management of top-level domains, accredits registrars, and coordinates other aspects of DNS. At the time of writing this chapter, there were almost 1000 different ICANN-accredited registrars worldwide. Following figure illustrates the process involved in registering a domain name.



The process involves six steps shown as follows:

1. "Decide on a top-level domain (.com) and a second level domain (funwebdev)." The illustration shows a user at a server who says, "I want the domain funwebdev.com"

2. "Choose a domain registrar or a reseller (a company such as a web host that works with a registrar." The illustration shows three formally dressed figures carrying suitcases labeled as "Domain."

3. "Registrars will check if domain is available by asking Registry for TLD." The illustration points to a security guard standing next to a box full of domain slabs, labeled "TLD (.com) registry."

4. "Complete the registration procedures which includes WHOIS contact information (includes DNS information) and payment." The illustration shows the user talking to a domain registrar. An arrow points to a paper labeled "WHOIS info" which points to the security guard standing next to "TLD (.com) registry."

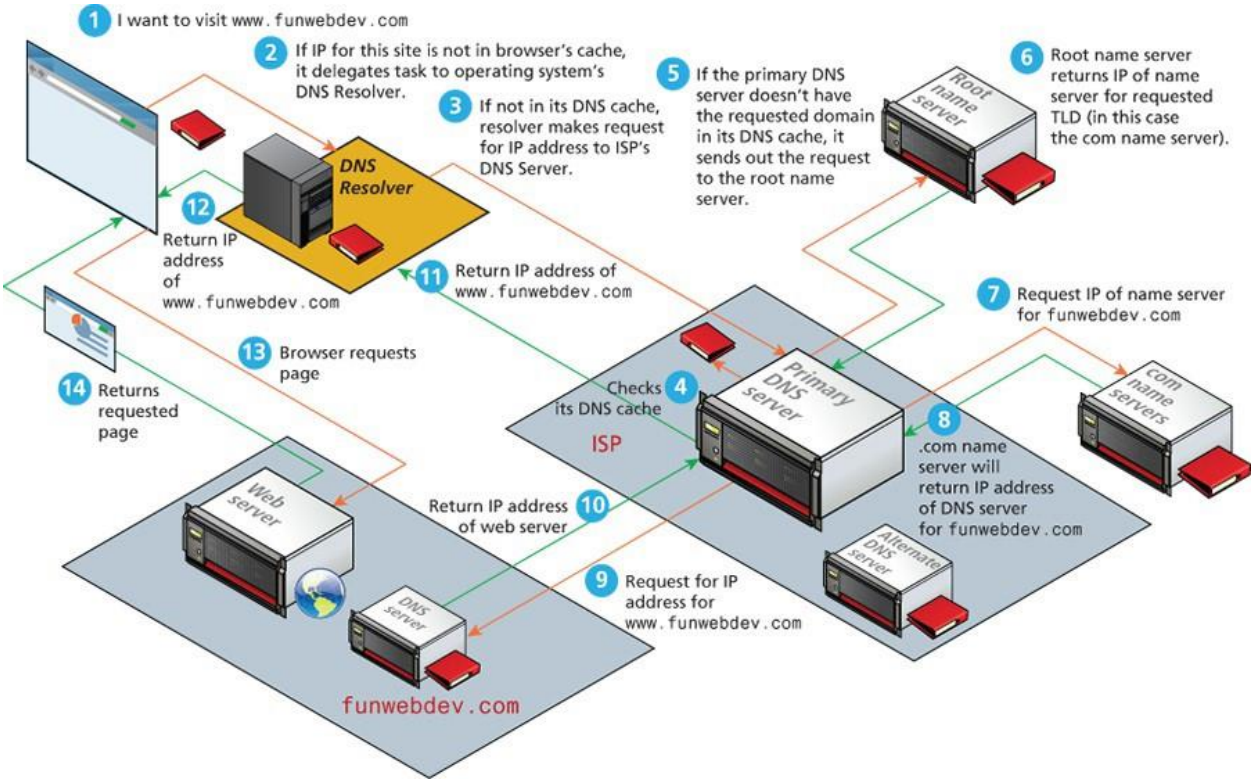
5. “Registry will push DNS information for domain to TLD name server.” The illustration shows a formally dressed figure at a desk labeled “TLD name servers”, with a domain and registration form.

6. “Enjoy the new domain...You now have purchased the rights to use it.”

The illustration shows the user holding a suitcase labeled as “Domain”.

Address Resolution

While domain names are certainly an easier way for users to reference a website, eventually your browser needs to know the IP address of the website in order to request any resources from it. DNS provides a mechanism for software to discover this numeric IP address. This process is referred to as address resolution. When you request a domain name, a computer called a domain name server will return the IP address for that domain. With that IP address, the browser can then make a request for a resource from the web server for that domain. What actually happens during address resolution is more complicated, as can be seen in following figure.



1. The resolution process starts at the user's computer. When the URL www.funwebdev.com is requested (perhaps by clicking a link or typing it in), the browser will begin by seeing if it already has the IP address for the domain in its cache. If it does, it can jump to step in the diagram.

2. If the browser doesn't know the IP address for the requested site, it will delegate the task to the DNS resolver, a software agent that is part of the operating system. The DNS resolver also keeps

a cache of frequently requested domains; if the requested domain is in its cache, then the process jumps to step .

3. Otherwise, it must ask for outside help, which in this case is a nearby DNS server, a special server that processes DNS requests. This might be a computer at your Internet service provider (ISP) or at your university or corporate IT department. The address of this local DNS server is usually stored in the network settings of your computer's operating system. This server keeps a more substantial cache of domain name/IP address pairs. If the requested domain is in its cache, then the process jumps to step .

4. If the local DNS server doesn't have the IP address for the domain in its cache, then it must ask other DNS servers for the answer. Thankfully, the domain system has a great deal of redundancy built into it. This means that in general there are many servers that have the answers for any given DNS request. This redundancy exists not only at the local level but at the global level as well.

5. If the local DNS server cannot find the answer to the request from an alternate DNS server, then it must get it from the appropriate top-level domain (TLD) name server. For funwebdev.com this is .com. Our local DNS server might already have a list of the addresses of the appropriate TLD name servers in its cache. In such a case, the process can jump to step .

6. If the local DNS server does not already know the address of the requested TLD server (for instance, when the local DNS server is first starting up it won't have this information), then it must ask a root name server for that information. The DNS root name servers store the addresses of TLD name servers. IANA (Internet Assigned Numbers Authority) authorizes 13 root servers, so all root requests will go to one

of these 13 roots. In practice, these 13 machines are mirrored and distributed around the world (see http://www.root-servers.org/ for an interactive illustration of the current root servers); at the time of writing, there are over 500 root server machines. With the creation of new commercial top-level domains in 2012, approximately 2000 or so new TLDs has come online, creating a heavier load on these root name servers.

7. After receiving the address of the TLD name server for the requested

domain, the local DNS server can now ask the TLD name server for the

address of the requested domain. As part of the domain registration

process, the address of the domain's DNS servers are sent to the TLD name servers, so this is the information that is returned to the local DNS server in step .

8. The user's local DNS server can now ask the DNS server (also called a second-level name server) for the requested domain (www.funwebdev.com); it should receive the correct IP address of the web server for that domain. This address will be stored in its own cache so that future requests for this domain will be speedier. That IP address can finally be returned to the DNS resolver in the requesting computer,

as shown in step .

9. The browser will eventually receive the correct IP address for the requested domain, as shown in step . Note: If the local DNS server were unable to find the IP address, it would return a failed response, which in turn would cause the browser to display an error message.

10. Now that it knows the desired IP address, the browser can finally send out the request to the web server, which should result in the web server responding with the requested resource.

Lesson 10 : Uniform Resource Locators

Uniform Resource Locators

Uniform Resource Locators (URL) play an important role in World Wide Web. URL has different components. They components are protocol, domain, path, query string and fragment. The functions of these parts are explained below.

10.1 Protocol

This part describes the protocol through which a web-page is accessed. When we open a website, we need to specify the protocol first. Most of the unsecured websites are accessed through HTTP protocol while in the case of secured websites, HTTPS protocol is used. Web browsers also support other protocols. For example, FTP protocol is used for accessing files. In figure 1, the protocol used is HTTP.

10.2 Domain

The address of a website is called domain. Domain consists of top-level domain, country code top-level domain, second level domain, third level domain etc. Domain name Server finds IP address against this name and this website can be accessed using that IP address. In figure 1, www.funwebdev.com is domain.

10.3 Port

It is an optional part of a URL. Usually, it is not visible. Most of the times, HTTP servers listen at port 80. A colon is added after domain name and then integer port number is specified.

10.4 Path

After domain, the path, of the file we want to access, is specified. In figure 1, index.php is path of a file being accessed. When path is not mentioned, default files like index.html or index.php are returned.

10.5 Query String

It is an information passed to the file. Query strings are mostly used in dynamic websites. Information in query string is sent to server in the form of key value pairs. It starts from delimiter

‘?’, and if there are more than one keys in a query string, they are separated using ‘&’. In figure 1, query string is page=17.

10.6 Fragment

Fragment is used to request a specific portion of a web-page. It is represented using ‘#’symbol. In figure 1, article is a fragment, and browser automatically scrolls the website to it.



Figure-1: Uniform Resource Locators (taken from Fundamentals of Web Development second edition 2017)

Lesson 11 : Hypertext Transfer Protocol

Hypertext Transfer Protocol

Hypertext Transfer Protocol (HTTP) is the backbone of World Wide Web (WWW) which is used to access and visit websites. Headers are used in HTTP. A sample illustration of HTTP is given in figure 2. When a client requests to access a web-page, a piece of information is sent to server called request header. It has different components like methods (GET, POST), the path a client wants to visit, and the file a client wants to access on a server. These headers are automatically generated by a browser, and sent to server side. It can also contain some additional information e.g. version of protocol, host, and IP address. If multiple websites are hosted on a single web server, there IP address will remain the same. Another important part of HTTP is user agent which contains information about client’s operating system and browser. A server can generate dynamic contents on the basis of this information.

When a web server receives a request, it sends response header and content against it. Response header can contain response code, date & time, information about server, and content length etc.

When a client fills a form, its data is sent to server suing POST method. The same can be done by using GET method, but it uses query string to send information to server. The comparison of POST and GET is shown in figure 3.

Different response codes are used to determine how a server handled a client’s request. 200 codes are for successful responses, 300 are for redirection-related responses, 400 codes are client errors and 500 codes are server errors. For example, some response codes and their purposes are given below.

200: OK

301: Moved Permanently

400: Bad Request

404: Not found

500: Internal server error

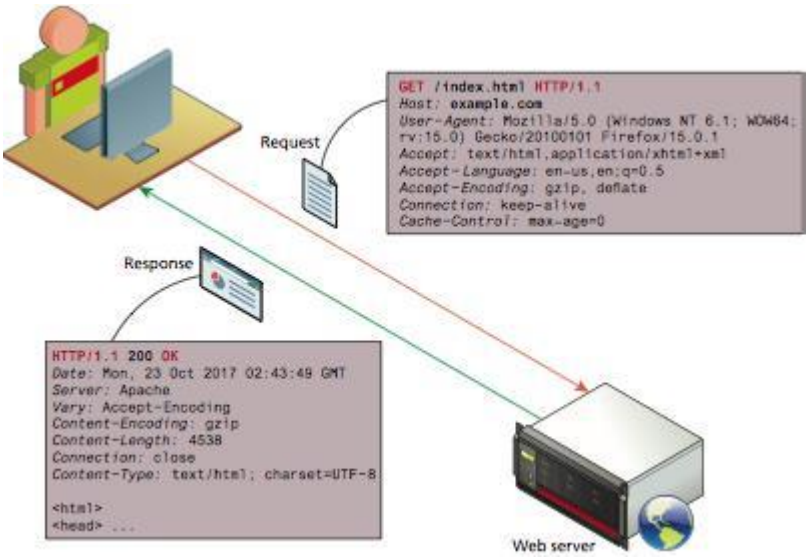


Figure-2: HTTP (taken from Fundamentals of Web Development second edition 2017)

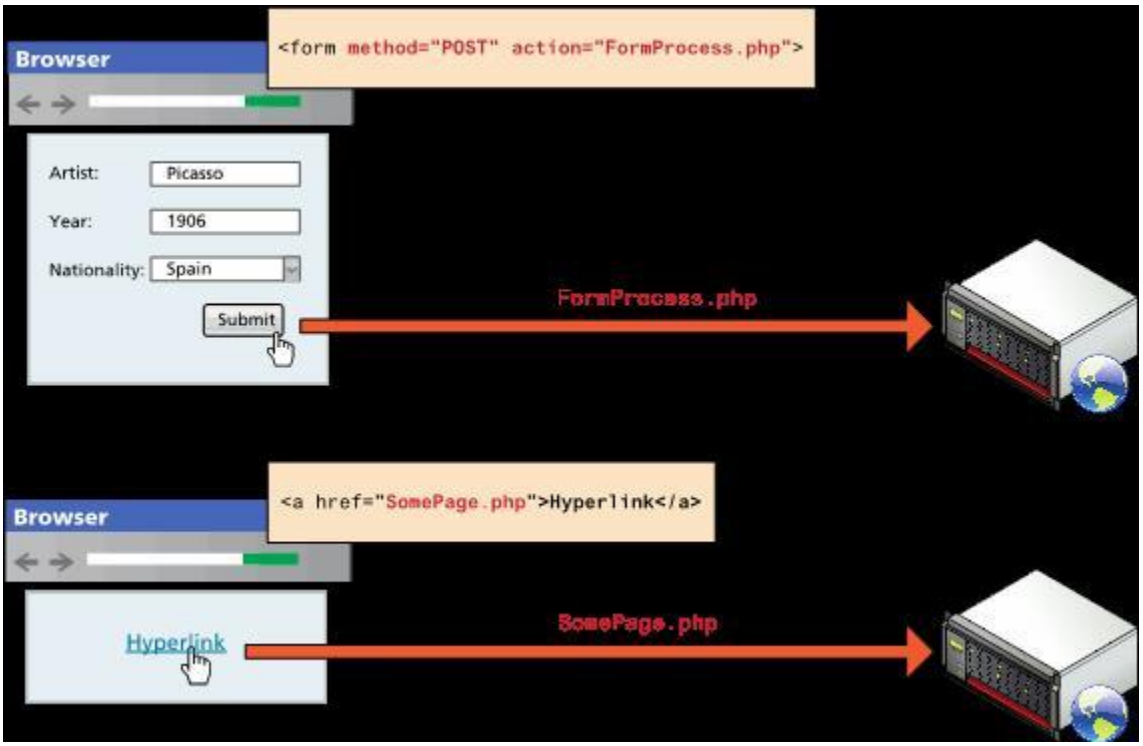


Figure-3: POST Vs GET (taken from Fundamentals of Web Development second edition 2017)

Week – 2

How a web browser works?

When we open a website, many processes run at backend. When we request for a webpage, we get its HTML contents first, which are in textual form. If that webpage contains any images, videos, styles sheets or scripts, all of these external files are fetched one by one. Browser requests and receives these files from server. One request may contain further multiple requests to get webpage content. When browser gets these contents, it renders and displays our page by using them.

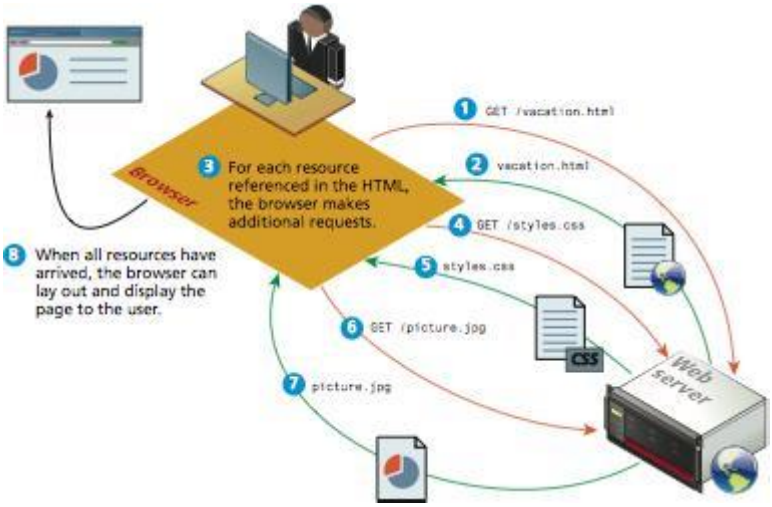


Figure-4: Web browser working (taken from Fundamentals of Web Development second edition 2017)

12.1 Load times and cascades

We can check the requests and responses against them, and time taken for responses, when we open a website. It can be done using developer tools in Chrome/Firefox. This tool is particularly intended for developers and thus called a developer tool. For example, in Chrome, we can open developer tools using browser menu (Ctrl+Shift+I shortcut can also be used). These developer tools contain different components for different purposes. For example, the component 'Network' contains information related to network.

12.2 Browser Rendering

When a browser receives all the contents of a webpage, it renders and then displays it. Different browsers, for this purpose, use different procedures according to their standards which have been devised by World Wide Web Consortium (W3C). Therefore, a web developer should test a website using different browsers.

12.3 Browser Caching

When we visit a website and fetch some images and files from it, they are stored in browser cache. Next time, when we need that file from the same server, the file will be retrieved from

browser cache instead of server, resulting in fast browsing. Caching can also be disabled to avoid caching, like done in banking websites.

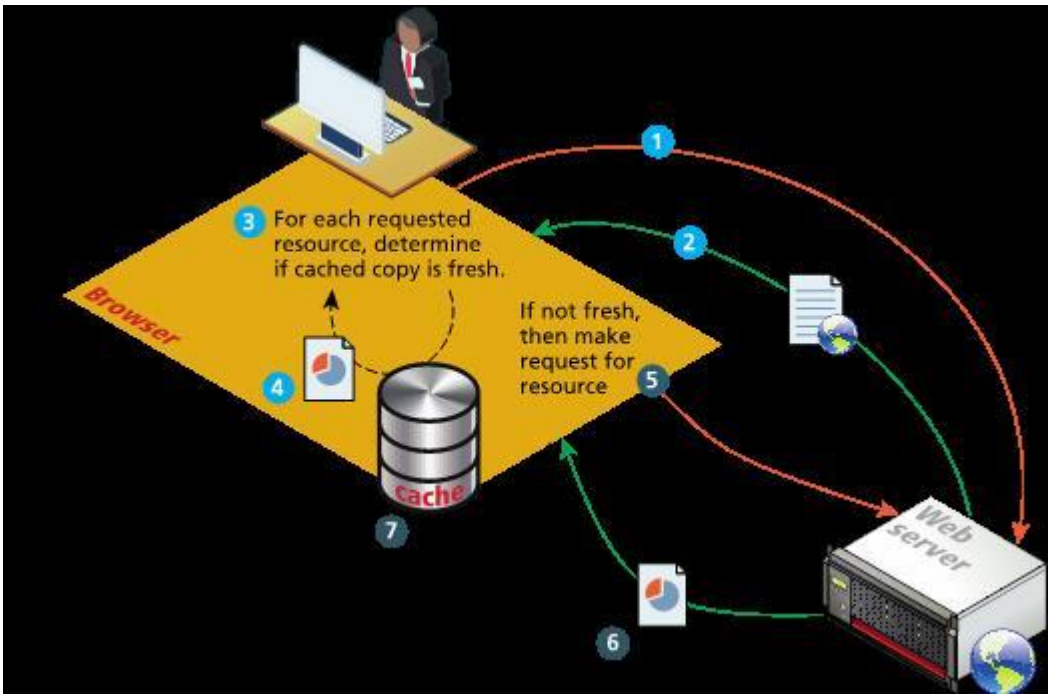


Figure-5: Browser Caching (taken from Fundamentals of Web Development second edition 2017)

12.4 Browser Features

Browsers contain certain features to facilitate users. Some features and their purposes are given below.

Search Engine Integration: Suggests search query in search bar.

URL Auto-completion: Gives suggestions on the basis of address bar input.

Form Auto-completion: Browsers saves some values and fills them automatically in upcoming forms.

Cloud sync of user data: Search data becomes independent of device and can be available at any device against specific login credentials.

Phishing website detection: Some browsers also identify websites which can hack user's data.

Secure connection visualization: A lock symbol is shown to represent secure connection.

12.5 Browser Extensions

Extensions can modify the page being rendered. They are designed for specific purposes. For example, extensions like ‘Lighthouse’ and ‘React developer tools’ are very useful for developers while extensions like ‘Adblock’ and ‘Third Party Plugins’ are used by general public.

Lesson 13 : Introduction to web servers

Introduction to web servers

It is also a computer but more powerful than the computers used at homes. Web servers follow a certain application stack. This application stack consists of operating system, web server software, a database, and scripting language for dynamic requests.

13.1 Application Stacks

The most commonly used application stack is LAMP which contains Linux operating system, Apache web server, MySQL database, and PHP scripting language. It is traditionally used for dynamic websites. There are other stacks as well. For example, WAMP, WISA, and MEAN. Therefore, a full-stack developer is supposed to have expertise of working on operating system, web server, frontend development, and backend development.

13.2 Operating Systems

Mostly used operating systems are Linux, Mac, and Windows.

13.3 Web server software
Some popular web server software are Apache, Nginx, and IIS.

13.4 Database software
Relational databases like MySQL, PostgreSQL, and Sqlite are mostly used. However, document based databases are also being used these days. One important example of document based databases is MongoDB.

13.5 Scripting languages
The content of a dynamic website can be generated in almost every scripting language. However, PHP is world-wide used for this purpose. These days, ASP.NET, Python, and Node.js are also being used as scripting languages.

Lesson 14 : Architecture of web-based system

Architecture of web-based system

14.1 Multi-tier architecture
Web development can be realized at multiple tiers.

2-tier architecture

When we access a website, at least 2-tiers are created i.e. Client-side tier and Server-side tier. However, in the perspective of web development, it becomes easy if we divide it into more tiers. It also eases the maintenance of a website.

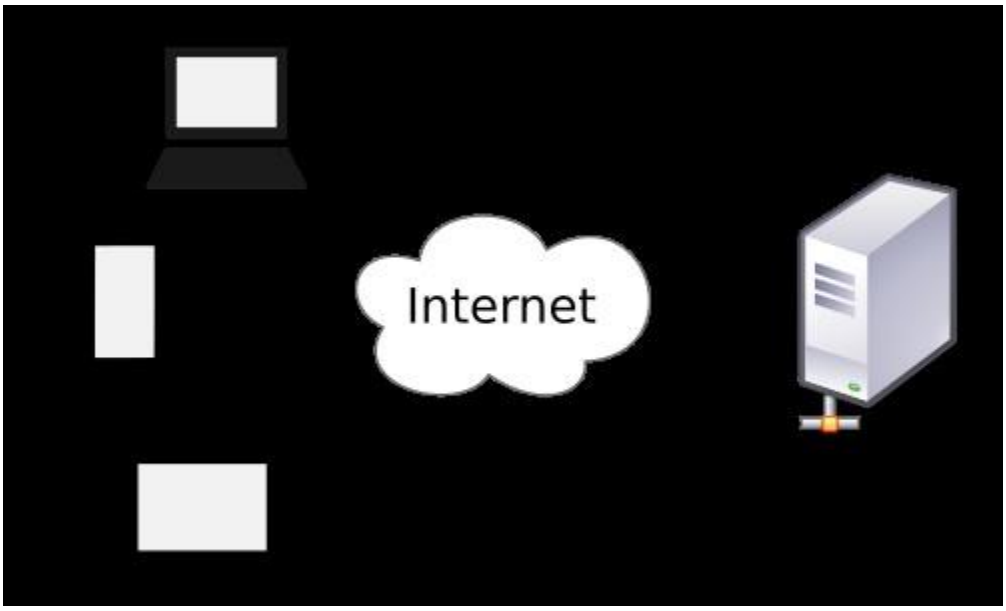


Figure-6: 2-tier architecture (taken from https://en.wikipedia.org/wiki/File:Client-server-model.svg)

3-tier architecture

In software engineering perspective, these tiers can be broken down to further tiers. In 3-tier architecture, first tier is responsible for content display (Presentation Layer), second tier is business logic for generating dynamic content, third tier is database layer which stores data and manages database. The advantage of separating presentation and business logic layers is that when we change business logic for some users, we do not need to change presentation layer necessarily.

Lesson 15 : Introduction to HTML

Introduction to HTML

HTML stands for hypertext markup language. Markup is a way to indicate information about the content that is distinct from the content. There has been different version of HTML, the latest is HTML5.

To understand markup, think of an assignment you submitted to your teacher. Teacher checks it and marks it for what can be improved or which mistakes student made which can be corrected. The feedback of teacher can be thought as markup. To make distinction between assignment and markup, teacher may use different color pen.

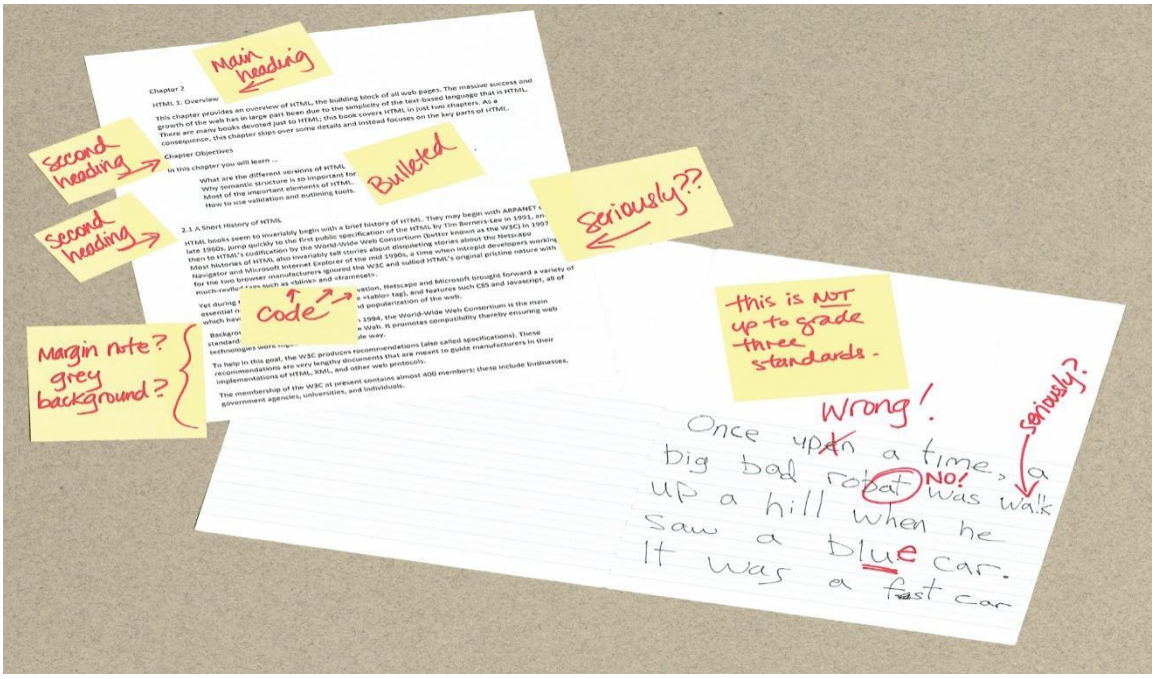


Figure-7: Markup (taken from Fundamentals of Web Development second edition 2017)

HTML validator, a tool of W3C, is used to identify errors in an HTML page and give recommendations in the form of warnings.

15.1 HTML5

It is the latest version of HTML supported by all of the modern browsers.

15.2 Elements and attributes

An HTML document contains HTML elements, also called HTML tags. Every HTML element has a unique name which can be written after the angle bracket (<). HTML elements can also contain attributes and the content within the tag. The figure given below shows the syntax of anchor tag, an attribute of link, and content. Most of tags come in pair, means they have closing tags as well.



Figure-8: Anchor Tag(taken from Fundamentals of Web Development second edition 2017)

15.3 An empty element

An empty element does not contain any text content; instead, it is an instruction to the browser to do something.

In XHTML, empty elements had to be terminated by a trailing slash. However, the trailing slash in empty elements is optional in HTML5.



Figure-9: Empty Element(taken from Fundamentals of Web Development second edition 2017)

15.4 Nesting element

Developers must be careful while using nesting elements. While closing, the nested element must be closed first. The example of correct and incorrect nesting is shown below in figure 10. In figure 11, the relationship of nested tag is shown. Nested elements can also be shown in the form of tree containing roots and children (figure 12).



Figure-10: Correct and incorrect Nesting (taken from Fundamentals of Web Development second edition 2017)

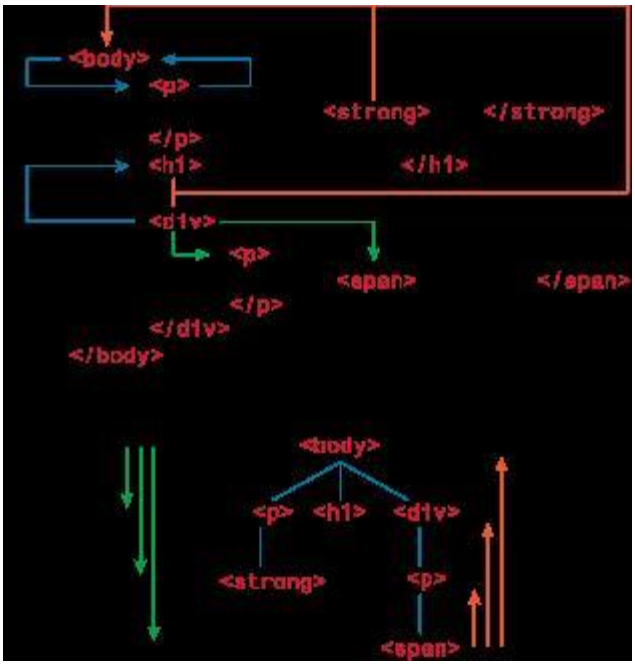


Figure-11: Relationship between tags (taken from Fundamentals of Web Development second edition 2017)

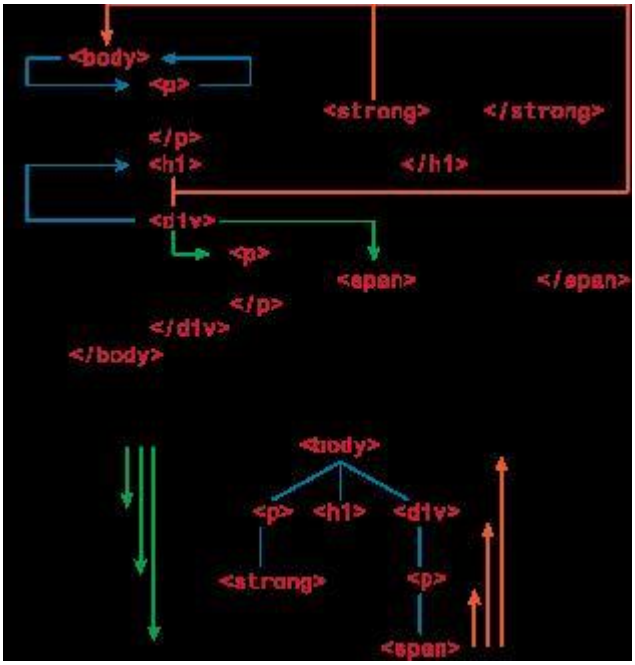


Figure-12: Tree structure of relationships (taken from Fundamentals of Web Development second edition 2017)

Lesson 16 : Common HTML Tags

Common HTML tags; Heading, Paragraphs, Divs; Inline Text Elements; Images; HTML Character Entities

HTML is only used for structuring and storing content. It is not used for styling but style sheets can be used for this purpose. Common HTML tags are explained below.

16.1 DOCTYPE

This tag is used to inform web server and browser that the document is an HTML document. It is written in the start of document. Its syntax is <!DOCTYPE html>.

16.2 Head and Body tags

External files, document title, and meta data about document are mentioned in the 'head' tag. The contents of document are mentioned in the 'body' tag. Earlier, parent 'html' tag was necessary but this compulsion has been removed in HTML5. However, most developers still use this tag.

16.3 Meta Tag

This tag contains information about coding details of HTML contents.

16.4 Link Tag

This tag is used to link the external styles sheets to an HTML document.

16.5 Script Tag

To specify a certain script from an external file, script tag is used.



Figure-13: Head, body, meta, link and script tags (taken from Fundamentals of Web Development second edition 2017)

Heading Tag

HTML provides six levels of headings (h1 - h6). Headings are also used by the browser to create a document outline for the page. It is recommended not to use different headings for setting font size.



Figure-14: Heading tags (taken from Fundamentals of Web Development second edition 2017)

Paragraphs and Divisions

Paragraph is a container for text and other HTML elements. <p> tag is used of paragraph.

Divisions are used to create a logical grouping of content. It is also a container element, and tag is used for division. Following figure demonstrates a paragraph tag.

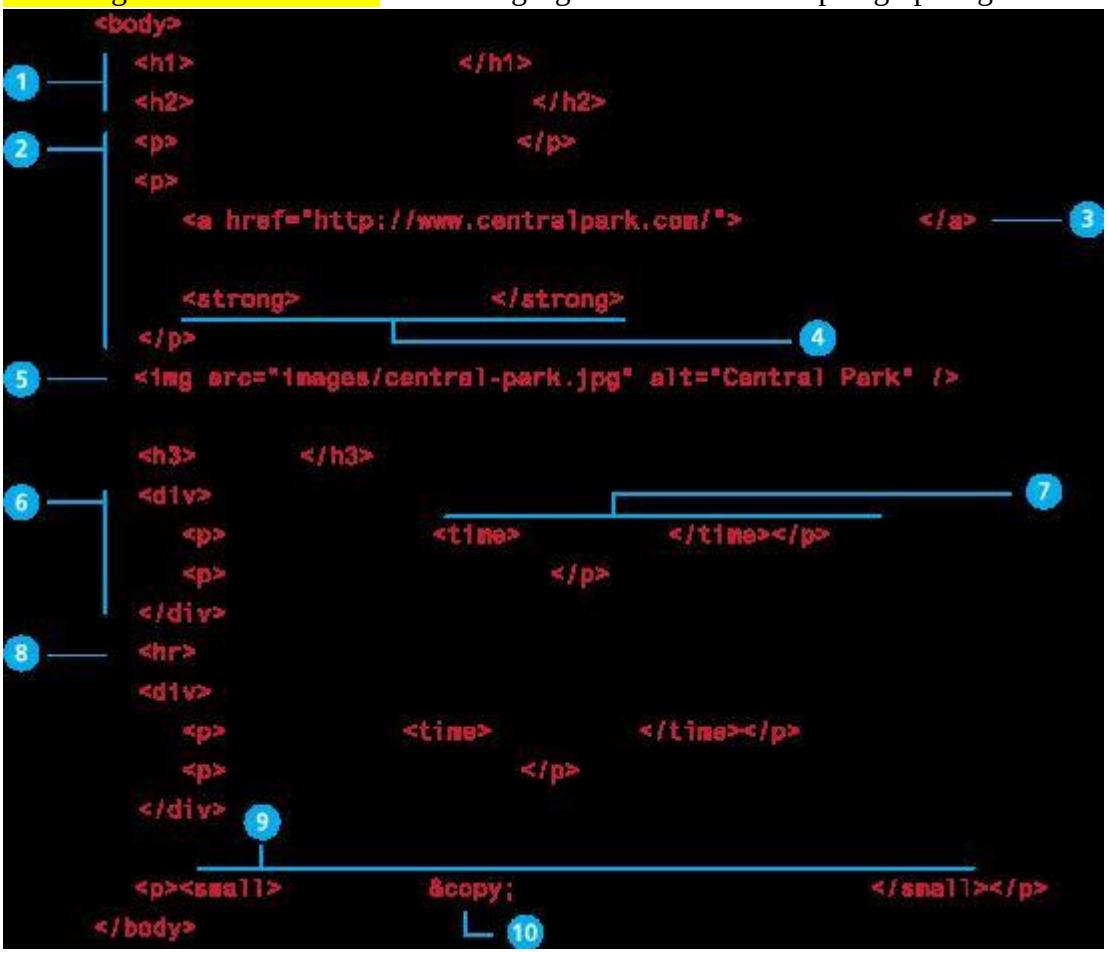


Figure-15: Paragraph tag (taken from Fundamentals of Web Development second edition 2017)

These tags do not create their space separately when they are used in other tags. In other words, these tags do not disrupt the flow. Its examples are <a>,
, and <site> etc.

16.9 Block elements

These elements disrupt the existing flow and take their own space. Its examples are , <hr>, and <li> etc.

16.10 Images

<img> element is used to display image. It is mandatory to use 'alt' attribute in image tag with the purpose to display text against an image which also helps visually impaired. The figure, given below, explains image tag properly.



Figure-16: Image tag (taken from Fundamentals of Web Development second edition 2017)

16.11 Character entities

These are special characters for symbols for which there is either no easy way to type them via a keyboard or which have a reserved meaning in HTML e.g. '<', which means that '<' cannot be displayed by just typing '<' because it is a keyword. Some entities and their description is given below in the table.

Entity	Description
	Nonbreakable space
<	<
>	>
©	©
™	™

Lesson 17 : Links

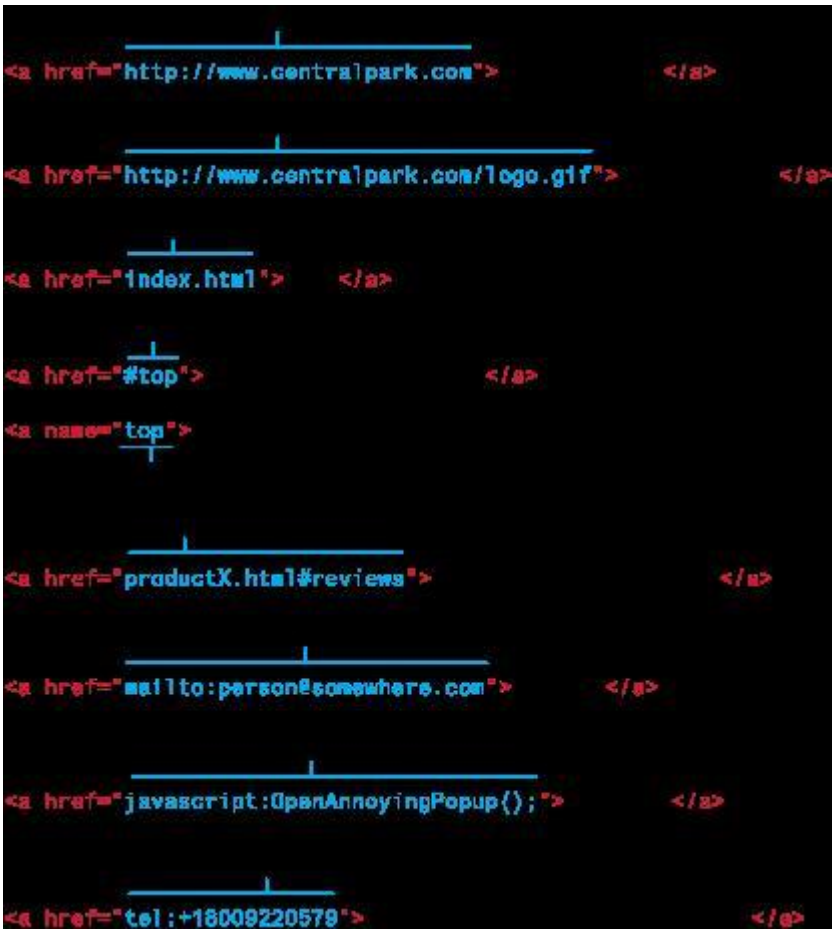
Links

Links are an essential feature of all web pages. Links are created using the element (the "a" stands for anchor). It has two main parts: the destination and the label.

As it can be seen in Figure 17, the label of a link can be text or another HTML element such as an image.



Figure-17: Parts of Link (taken from Fundamentals of Web Development second edition 2017)



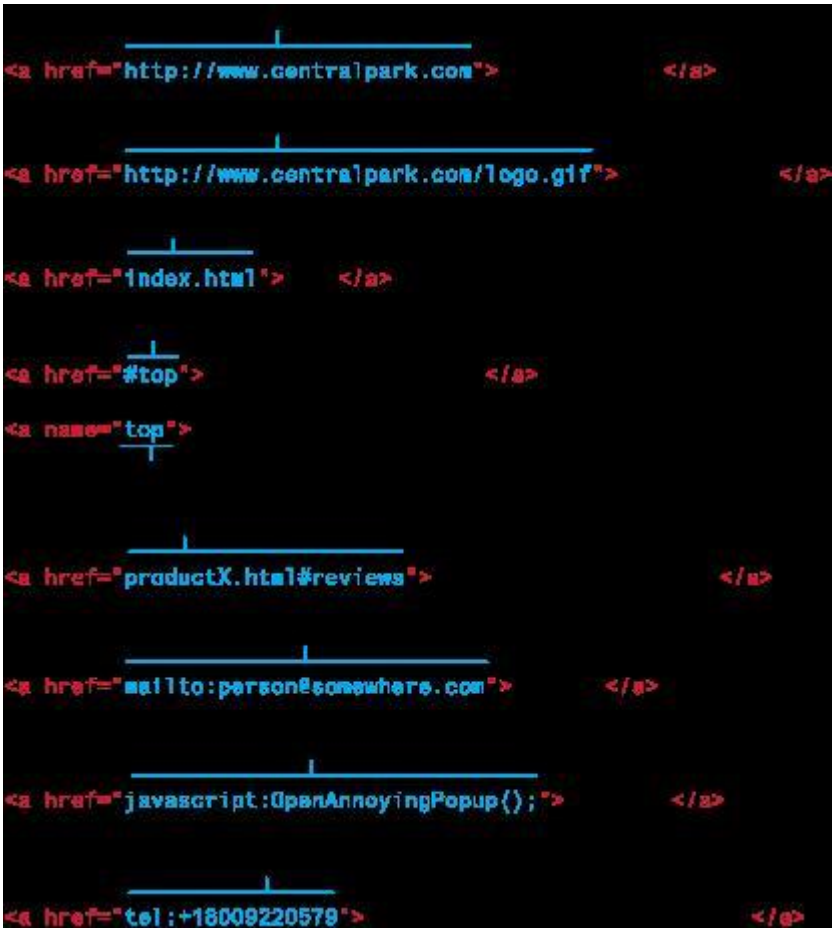


Figure-18: Examples of links (taken from Fundamentals of Web Development second edition 2017)

17.1 Relative Referencing

Whether we are constructing links with the element, referencing images with the <img> element, or including external JavaScript or CSS files, we need to be able to successfully reference files within our site. This requires learning the syntax for so-called relative referencing. Table 17.1 provides examples of the different types of URL referencing.

Relative Link Type	Example
Same directory	<a href="example.html">
Child Directory	<a href="images/logo.gif">
Grandchild/Descendant Directory	<a href="css/images/background.gif">

Parent/Ancestor Directory	<a href="../about.html"> <a href="../../about.html”>
Sibling Directory	<a href="../images/about.html">
Root Reference	<a href="/images/background.gif">

Table 17.1 Relative Referencing

Lesson 18 : Lists

Topic 18: Lists

HTML provides three types of lists which are explained below.

18.1 Unordered lists

Collections of items in no particular order is called unordered list. These are by default rendered by the browser as a bulleted list. However, it is common in CSS to style unordered lists without the bullets. Unordered lists have become the conventional way to markup navigational menus.

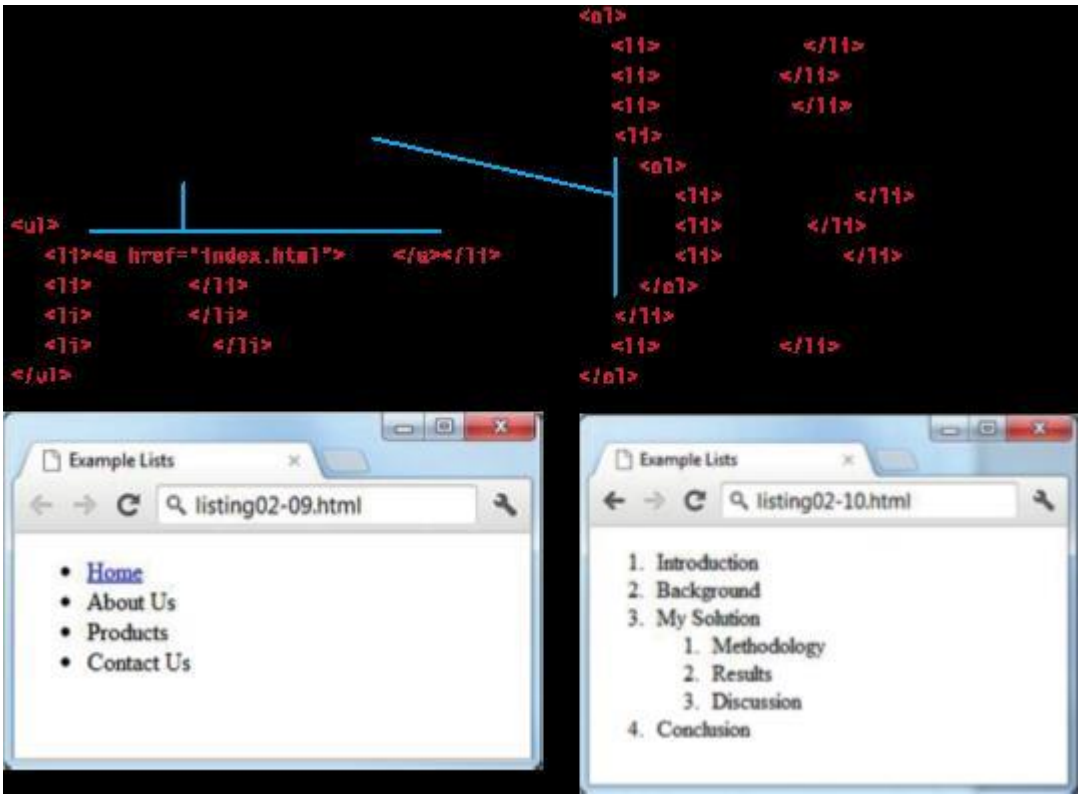


Figure: Unordered lists (taken from Fundamentals of Web Development second edition 2017)

18.2 Ordered lists

Collections of items that have a set order is called ordered list. These are by default rendered by the browser as a numbered list.

```
<!DOCTYPE html>
<ol>
    <li>Introduction</li>
    <li>Background</li>
    <li>My Solution</li>
    <li>Conclusion</li>
</ol>
```

1. Introduction
2. Background
3. My Solution
4. Conclusion

Figure: Ordered lists (taken from Fundamentals of Web Development second edition 2017)

18.3 Nested lists

A list within a list is called nested list, for example:

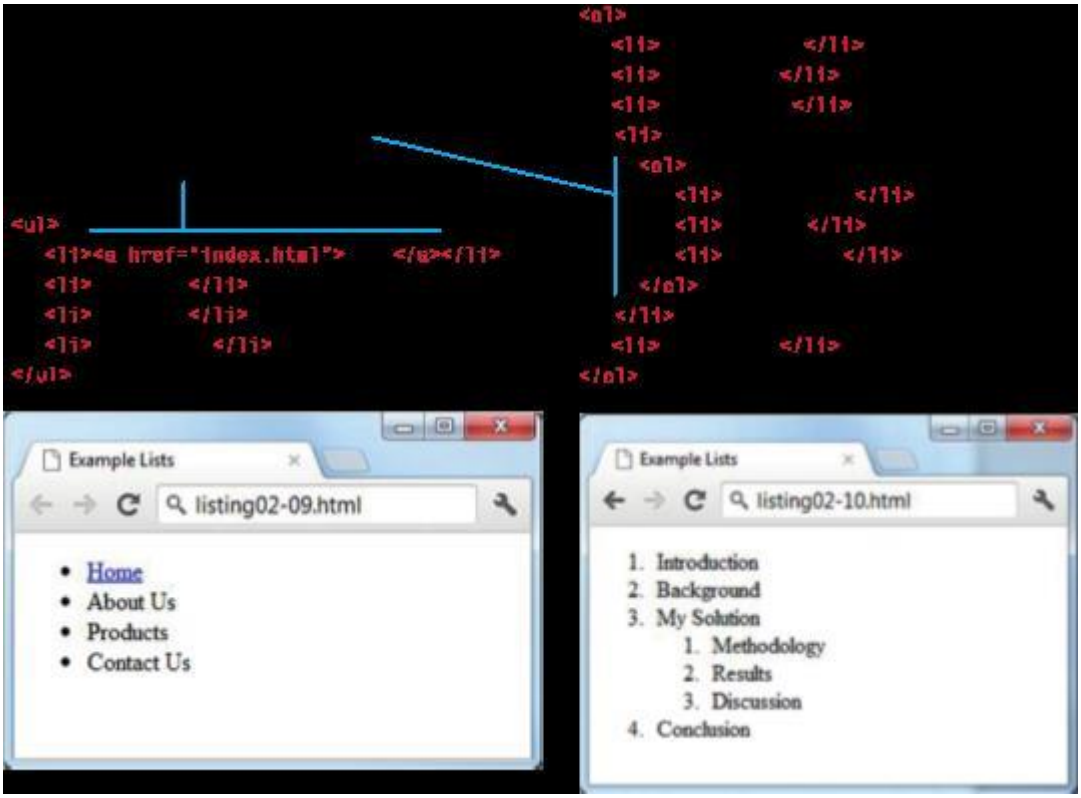


Figure: Nested lists (taken from Fundamentals of Web Development second edition 2017)

18.4 Description lists

Collection of name and description/definition pairs is called a description list. These lists tend to be used infrequently. Perhaps, the most common example would be an FAQ list. Unlike the other two lists (which contain <li> items within either a <ul> or <ol> parent container), the container for a description list is the <dl> element. It contains <dt> (term or name to be described) and <dd> (describes each term) pairs for each item in the list.

```
<dl>
    <dt>HTML</dt>
    <dd>Hyper Text Markup Language</dd>
    <dt>CSS</dt>
    <dd>Cascading Style Sheet</dd>
</dl>
```

HTML

Hyper Text Markup Language

CSS

Cascading Style Sheet

Figure: Description lists (taken from Fundamentals of Web Development second edition 2017)

Lesson 19 : Semantics in HTML 5

1. **HTML Semantic Structuring**

HTML semantic structuring adds meaning to a web page rather than merely making it look pretty. A <p> tag, for example, denotes that the contained text is a paragraph. Because people understand what paragraphs are and how to display them, this is both semantic and presentational. Tags like <b> and I on the other hand, do not have any semantic value. They merely specify how the text should be formatted (bold or italic) and add no further meaning to the markup.

Examples of semantic HTML tags include:

Header tags <h1> through <h6>

<blockquote>

<code>

There are many more semantic HTML tags to use as you build a standards-compliant website.

1.

    1. **Advantages Semantic Structuring**

Eliminating presentation-oriented markup and writing semantic HTML markup has a variety of important advantages:

1. ***Maintainability.***

Web pages with a lot of presentation markup are more difficult to edit and change than those with semantic markup. Our students are frequently startled to hear that maintaining and altering old code takes more time than writing new code. This is especially true for web initiatives. Web projects, in our experience, incur more "requirements drift" because of end user and customer input than traditional software development projects.

2. ***Performance.***

Semantic web pages are typically quicker to author and faster to download.

3. ***Accessibility.***

The content on web pages is not accessible to all web users. Voice reading software is used by users with vision problems to use the internet. If a website does not employ semantic markup, visiting it with speech reading software might be an extremely irritating experience. Many governments also require that websites for organizations that receive federal support follow particular accessibility criteria. The US government, for example, maintains its own set of Section 508 Accessibility Guidelines (http://www.section508.gov).

4. ***Search Engine Optimization***

The numerous search engine crawlers are, for many website owners, the most significant users of a website. Crawlers are automated programmes that explore the internet, looking for content to utilise in user searches. Semantic markup gives these crawlers better instructions: it tells them what material on the site is most important.

2. **Header Tag**

The header of a page or section is defined with the <header> tag. It usually includes a logo, a search engine, and navigational connections, among other things. This tag does not create a new outline section. The heading (an <h1>–<h6> element) of a surrounding section is typically contained in a <header> element. This, however, is not essential. One of the HTML5 elements is the header tag. Several <header> tags can be used in an HTML document, and they can be positioned in any area of it. It is forbidden to use the <header> tag inside the <footer> and <address> components, as well as inside another <header> tag.

The tag <header> is used in pairs. Between the opening (<header>) and closing (</header>) tags, the content is written.

3. **Footer Tag**

A web page's or section's footer is defined by the <footer> tag. It usually includes copyright information, contact information, navigation links, and so on. On a web page, there might be many <footer> tags. You can, for example, include a footer within the <article> element to keep information about the content (links, footnotes, etc.). Except for the <footer> and <header> tags, the tag can include other HTML components. One of the HTML5 components is the footer. If the footer has contact information, provide it in the <address> tag.

The <footer> element can contain the following:

- copyright, authorship and contact information
- related documents
- sitemap
- back to top links

The <footer> tag comes in pairs. The content is written between the opening (<footer>) and closing (</footer>) tags.

The <footer> tag supports the Global Attributes and the Event Attributes.

4. **Navigation Tag**

One of the HTML5 elements is the <nav> tag. It specifies a set of navigation links, either within the current content or to other papers. Tables of contents, menus, and indexes are examples of navigation blocks.

A single HTML document can include multiple <nav> tags, such as one for site navigation and another for intra-page navigation.

It's worth noting that the <nav> element doesn't contain all of the links in the HTML content; it only contains the primary navigation blocks. The nav> element can be used to define links in the website's footer, however the <footer> tag is more commonly used in such circumstances.

The <nav> tag is not allowed to be nested within the address> element.

The <nav> tag is a pair of tags. The material is written in the middle of the page.

5. **Main Tag**

In the HTML5 specification, the <main> tag is a new block-level element. The dominating content of the document's <body> is specified by this tag. The <main> tag's content must be unique and not duplicate blocks of the same type that appear in other pages, such as a site's header, header, footer, menu, search, copyright information, and so on. The <article>, <aside>,

<footer>, <header>, or <nav> tags cannot include the main> element. There can't be more than one <main> tag in a document that doesn't have a hidden attribute.

Unlike elements like <body>, <h2>, and others, this element has no effect on the DOM's understanding of the page's structure. Landmarks can be used by assistive technology to easily identify and navigate to the document's big portions. It is preferable to utilise the <main> element instead of declaring role="main."

The "skipnav" technique allows users of assistive technology to skip huge chunks of repetitive material. This allows the user to quickly reach the primary material of a website. If you give the <main> element an id property, it will become the target of a skip navigation link.

The tag main> is used in pairs. Between the opening (<main>) and closing (</main>) tags, the text is written. The Global Attributes and Event Attributes are supported by the <main>tag.

6. **Article Tag**

One of the HTML5 elements is the <article> tag. It's used to describe stuff that's self-contained and self-contained. An article should have a distinct significance and stand out from the rest of the web page's material.

A blog entry, forum post, news item, or user remark can all be included in the <article> element.

The internal element represents an article that is associated to the external element when the <article> element is nested. The address> element can be used to provide author information for the <article> element. It will not, however, apply to nested <article> components. In a single HTML document, many article> tags can be used. The <h2> to <h6> tags can be nested within the <article> tag. If the <h1> tag has never been used previously, it can be used now. A <time> element with a datetime attribute can be used to express the <article> tag's publishing date and time.

The tag article> is used in pairs. Between the opening (article>) and closing (/article>) tags, the text is written.

7. **Section Tag**

One of the HTML5 components is the <section>. It's utilised to make separate sections of a website with logically connected content (news block, contact information, etc.). When designing a landing page, the <section> tag is frequently used to break the page into logical blocks.

A navigation menu, for example, must be wrapped in a <nav> tag, while a map display and a list of search results do not need to be wrapped in a <nav> tag and can be placed inside a <section>.

The nested <section> tag can be used to divide information into groups within the <article> tag. As a result, using <h1> -<h6> heads within the <article> and <section> tags is essential. The usage of a <h2> title in each section defined by the <section> tag is permitted. Instead of using

the <section> element as a universal container for generating page structure, use the tag instead.

The tag <section> is used in pairs. Between the opening (<section>) and ending (</section>) tags, the text is written.

8. **Figure Tag**

The <figure> tag is used to indicate content that is self-contained. Diagrams, photos, illustrations, code samples, and so forth can all be included in the tag. One of the HTML5 elements is the figure tag. The <figure> tag's content is tied to the main flow's content. It is, nonetheless, regarded as a self-contained unit. The <figcaption> element is used to add a caption or explanation to the content of the figure> tag. The <figcaption> tag is a child element of the figure> tag, and it is either the first or last child element. The content will be shown at the top of the image if <figcaption> is the first nested element.

The tag <figure> is used in pairs. Between the beginning (<figure>) and closing (</figure>) tags, the content is written.

1.

9. **Caption Tag**

The table's header is defined using the <caption> tag. The tag must be the first child of its parent <table> element and must be inside the <table> element immediately after the opening (<table>) tag. There is only one caption per table that can be defined. You must use the <figcaption> element instead of the <caption> element when the table> element containing the caption> is the only descendent of the <figure> element. A table caption is center-aligned above a table by default. However, the caption can be aligned and placed using the text-align and caption-side attributes.

The tag <caption> is used in pairs. Between the opening (<caption>) and closing (</caption>) tags, the text is written.

Lesson 20 : Tools Insight

1. **Tools for web development**

HTML pages can be created in a variety of ways. An HTML editor can be used with any program that can edit and save text files. However, the use of the right technology can make

writing web content much easier. You may have selected an HTML editor based on lab availability, familiarity, or some other factor.

While we will not be recommending specific tools for creating web content, we believe it is critical to understand the many types of web development tools and their relative benefits and drawbacks. Web development tools are divided into five categories:

- WYSIWYG editors,
- Code editors,
- Full IDEs,
- Cloud-based environments,
- Code playgrounds.

1. **WYSIWYG editors.**

Web tools that provide a user experience similar to that of a word processor are known as What-You-See-Is-What-You-Get. The benefit of such tools is that you don't require much (if any) HTML knowledge. However, there is a significant downside to using such technologies. These tools are never fully WYSIWYG, and they frequently have trouble previewing increasingly complex CSS. Indeed, in order to remedy such issues, these technologies nearly always have to give users with a typical HTML view. While we would never recommend relying only on such a tool, inexperienced end users may find them useful. In this genre, Adobe Dreamweaver and Adobe Muse are two prominent editors. WYSIWYG editors are also used in web-based publishing programs like blogs and content management systems.

1.

2. **Code editors.**

Since web developers typically need knowledge of HTML, CSS, JavaScript, and more, many web developers prefer to use tools that allow them to focus on viewing and editing these text files. Nonetheless, it is helpful to use a tool that "understands" HTML, CSS, and so on. Such a tool might provide color coding, intelligent hints, tag completion, and so on. There are a wide range of choices in this genre, many of them open source. Some of the options include Atom, BlueFish, Brackets, Notepad++, Sublime Text, and Visual Studio Code.

1.

3. **Full IDEs.**

Integrated Development Environments provide a more full featured programming experience. They not only provide most of the same functionality as the previously mentioned code editors, but also typically provide extra capabilities, such as comprehensive help files, build tools, multiple-language support, and integration with other enterprise tools, such as databases. Some of the options in this genre include Eclipse, NetBeans, and Visual Studio. This extra power does

come at a price, both figuratively and literally. The figurative costs is these complicated IDEs typically have a more substantial learning curve and can often have steep hardware requirements.

1.

4. **Cloud-based environments.**

One of the fastest growing approaches to developing web applications is to do one's development, testing, and hosting all within an online environment. The key advantage of such an approach is that you don't have to worry about installing, supporting, and synchronizing different web development tools, since it is all done for you by the online environment. As well, using such online environments means that you don't really care what device you have; if you have an Internet connection, you can do your coding. Of course, that's also the key disadvantage. Since you need an Internet connection, you can't code while on the plane or in a forest (though these environments sometimes provide a mechanism for offline usage). At the time of writing, CodeAnywhere and Cloud9 are two popular sites providing a complete IDE for web development.

1.

5. **Code playgrounds.**

Our final approach to web development tools also makes use of online environments. Code playgrounds are not about constructing complete sites. Instead, they provide a way to experiment, demonstrate, and share smaller snippets of code. Some of the most popular include CodePen , JSFiddle, and CSS Deck. These environments are especially valuable for students as a way to construct online portfolios and to show off their skills to prospective clients and employers.

Lesson 21 : Introduction to CSS

Introduction to CSS

In current web development best practices HTML should not describe the formatting or presentation of documents. Instead that presentation task is best performed using Cascading Style Sheets (CSS). CSS is a W3C standard for describing the appearance of HTML elements.

Another common way to describe CSS's function is to say that CSS is used to define the presentation of HTML documents. With CSS, we can assign font properties, colors, sizes, borders, background images, and even position elements on the page. CSS can be added directly to any HTML element (via the style attribute), within the <head> element, or, most commonly, in a separate text file that contains only CSS.

Benefits of CSS

Before digging into the syntax of CSS, we should say a few words about why using CSS is a better way of describing appearances than HTML alone. The benefits of CSS include the following:

Improved control over formatting.

The degree of formatting control in CSS is significantly better than that provided in HTML. CSS gives web authors fine-grained control over the appearance of their web content.

Improved site maintainability.

Websites become significantly more maintainable because all formatting can be centralized into one CSS file, or a small handful of them. This allows you to make site-wide visual modifications by changing a single file.

Improved accessibility.

CSS-driven sites are more accessible. By keeping presentation out of the HTML, screen readers, and other accessibility tools work better, thereby providing a significantly enriched experience for those reliant on accessibility tools.

Improved page-download speed.

A site built using a centralized set of CSS files for all presentation will also be quicker to download because each individual HTML file will contain less style information and markup, and thus be smaller.

Improved output flexibility.

CSS can be used to adopt a page for different output media. This approach to CSS page design is often referred to as responsive design.

Lesson 22 : CSS Syntax (Properties)

CSS Syntax

A CSS document consists of one or more style rules. A rule consists of a selector that identifies the HTML element or elements that will be affected, followed by a series of property:value pairs (each pair is also called a declaration).

Each individual CSS declaration must contain a property. These property names are predefined by the CSS standard. The CSS2.1 recommendation defines over a hundred different property names, so some type of reference guide, whether in a book, online, or within your web development software, can be helpful.5 This section will only be able to cover most of the common CSS properties. The most used CSS properties are given below. Properties marked with an asterisk contain multiple subproperties not listed here (e.g., border-top, border-top color, border-top-width, etc).

Fonts

font

font-family

font-size

font-style

font-weight

@font-face

Text

letter-spacing

line-height

text-align

text-decoration*

text-indent

Color and background

background

background-color

background-image

background-position

background-repeat

box-shadow

color

opacity

Borders

border*

border-color

border-width

border-style

border-top, border-left, …*

border-image*

border-radius

Spacing

padding

padding-bottom, padding-left, …

margin

margin-bottom, margin-left, …

Sizing

height

max-height

max-width

min-height

min-width

width

Layout

bottom, left, right, top

clear

display

float

overflow

position

visibility

z-index

Lists

list-style*

list-style-image

list-style-type

Effects

animation*

filter

perspective

transform*

transition*

Lesson 23 : CSS Syntax (Values)

CSS Syntax (Values)

Each CSS declaration also contains a value for a property. The unit of any given value is dependent upon the property. Some property values are from a predefined list of keywords. Others are values such as length measurements, percentages, numbers without units, color values, and URLs.

Color Values

Name

Use one of 17 standard color names. CSS3 has 140 standard names.

color: red; color: hotpink; /* CSS3 only */

RGB

Uses three different numbers between 0 and 255 to describe the red, green, and blue values of the color.

color: rgb(255,0,0); color: rgb(255,105,180);

Hexadecimal

Uses a six-digit hexadecimal number to describe the red, green, and blue value of the color; each of the three RGB values is between 0 and FF (which is 255 in decimal). Notice that the hexadecimal number is preceded by a hash or pound symbol (#).

color: #FF0000; color: #FF69B4;

RGBa

This defines a partially transparent background color. The “a” stands for “alpha,” which is a term used to identify a transparency that is a value between 0.0 (fully transparent) and 1.0 (fully opaque).

color: rgba(255,0,0,0.5);

HSL

Allows you to specify a color using Hue Saturation and Light values. This is available only in CSS3. HSLA is also available as well.

color: hsl(0,100%,100%); color: hsla(330,59%,100%,0.5);

Just as there are multiple ways of specifying color in CSS, so too there are multiple ways of specifying a unit of measurement. these units can sometimes be complicated to work with. When working with print design, we generally make use of straightforward absolute units such as inches or centimeters and picas or points. However, because different devices have differing physical sizes as well as different pixel resolutions and because the user is able to change the browser size or its zoom mode, these absolute units don't always make sense with web element

measures.

Below is a list of the different units of measure in CSS. Some of these are relative units, in that they are based on the value of something else, such as the size of a parent element. Others are absolute units; in that they have a real-world size. Unless you are defining a style sheet for printing, it is recommended you avoid using absolute units. Pixels are perhaps the one popular exception (though, as we shall see later, there are also good reasons for avoiding the pixel unit). In general, most of the CSS that you will see uses either px, em, or % as a measure unit.

Units of Measure Values

px

Pixel. In CSS2 this is a relative measure, while in CSS3 it is absolute (1/96 of an inch).

em

Equal to the computed value of the font-size property of the element on which it is used. When used for font sizes, the em unit is in relation to the font size of the parent.

%

A measure that is always relative to another value. The precise meaning of % varies depending upon the property in which it is being used.

ex

A rarely used relative measure that expresses size in relation to the x-height of an element's font. Relative

ch

Another rarely used relative measure; this one express size in relation to the width of the zero (“0”) character of an element's font.

rem

Stands for root em, which is the font size of the root element. Unlike em, which may be different for each element, the rem is constant throughout the document.

vw, vh

Stands for viewport width and viewport height. Both are percentage values (between 0 and 100) of the viewport (browser window). This allows an item to change size when the viewport is resized.

in

Inches Absolute

cm

Centimeters Absolute

mm

Millimeters Absolute

pt

Points (equal to 1/72 of an inch)

Pc

Pica (equal to 1/6 of an inch)

Lesson 24 : Location of styles

Location of styles

As mentioned earlier, CSS style rules can be in three different locations. These three are not mutually exclusive, in that you could place your style rules in all three. In practice, however, web authors tend to place all their style definitions in one (or more) external style sheet files.

Inline Styles

Inline styles are style rules placed within an HTML element via the style attribute. An inline style only affects the element it is defined within and overrides any other style definitions for properties used in the inline style. Notice that a selector is not necessary with inline styles and that semicolons are only required for separating multiple rules. Using inline styles is generally discouraged since they increase bandwidth and decrease maintainability (because presentation and content are intermixed and because it can be difficult to make consistent inline style changes across multiple files). Inline styles can, however, be handy for quickly testing out a style change.

Embedded Style Sheet

Embedded style sheets (also called internal styles) are style rules placed within the <style> element (inside the <head> element of an HTML document). While better than inline styles, using embedded styles is also by and large discouraged. Since each HTML document has its own <style> element, it is more difficult to consistently style multiple documents when using embedded styles. Just as with inline styles, embedded styles can, however, be helpful when quickly testing out a style that is used in multiple places within a single HTML document. We sometimes use embedded styles in the book or in lab materials for that

reason.

External Style Sheet

External style sheets are style rules placed within a external text file with the .css extension. This is by far the most common place to locate style rules because it provides the best maintainability. When you make a change to an external style sheet, all HTML documents that reference that style sheet will automatically use the updated version. The browser can cache the external style sheet, which can improve the performance of the site as well.

Lesson 25 : CSS selectors (Element, Class, ID)

Selectors

As teachers, we often need to be able to relay a message or instruction to either individual students or groups of students in our classrooms. In spoken language, we have a variety of different approaches we can use. We can identify those students by saying things like: “all of you talking in the last row,” or “all of you sitting in an aisle seat,” or “all of you whose name begins with ‘C’,” or “all first-year students,” or “John Smith.” Each of these statements identifies or selects a different (but possibly overlapping) set of students. Once we have used our student selector, we can then provide some type of message or instruction, such as “talk more quietly,” “hand in your exams,” or “stop texting while I am speaking.” when defining CSS rules, you will need to first use a selector to tell the browser which elements will be affected by the property values. CSS selectors allow you to select individual or multiple HTML elements. The topic of selectors has become more complicated than it was when we started teaching CSS in the late 1990s. There are now a variety of new selectors that are supported by most modern browsers. Before we get to those, let us look at the three basic selector types that have been around since the earliest CSS2 specification.

Element Selectors

Element selectors select all instances of a given HTML element. You can select all elements by using the universal element selector, which is the * (asterisk) character. You can select a group of elements by separating the different element names with commas. This is a sensible way to reduce the size and complexity of your CSS files, by combining multiple identical rules into a single rule.

Class Selectors

A class selector allows you to simultaneously target different HTML elements regardless of their position in the document tree. If a series of HTML elements have been labeled with the same class attribute value, then you can target them for styling by using a class selector, which takes the form: period (.) followed by the class name.

Id Selectors

An id selector allows you to target a specific element by its id attribute regardless of its type or position. If an HTML element has been labeled with an id attribute, then you can target it for styling by using an id selector, which takes the form: pound/hash (#) followed by the id name.

Lesson 26 : CSS Selectors (Attribute)

Attribute Selectors

An attribute selector provides a way to select HTML elements either by the presence of an element attribute or by the value of an attribute. This can be a very powerful technique, but because of uneven support by some of the browsers in the past, not all web authors have used them. Followings are the most common ways one can construct attribute selectors in CSS3.

[]

A specific attribute.

[title]

Matches any element with a title attribute

[=]

A specific attribute with a specific value. a[title=“posts from this country”] Matches any <a> element whose title attribute is exactly “posts from this

country“

[~=]

A specific attribute whose value matches at least one of the words in a space delimited list of words. [title~=“Countries”] Matches any title attribute that contains the word “Countries“

[^=]

A specific attribute whose value begins with a specified value. a[href^=“mailto”] Matches any <a> element whose href attribute begins with “mailto“

[*=]

A specific attribute whose value contains a substring. img[src*=“flag”] Matches any <img> element whose src attribute contains somewhere within it the text “flag“

[$=]

A specific attribute whose value ends with a specified value. a[href$=“.pdf”] Matches any <a> element whose href attribute ends with the text “.pdf“

Lesson 27 : CSS Selectors (Pseudo)

Pseudo-Element and Pseudo-Class Selectors

A pseudo-element selector is a way to select something that does not exist explicitly as an element in the HTML document tree but which is still a recognizable selectable object. For instance, you can select the first line or first letter of any HTML element using a pseudo-element selector. A pseudoclass selector does apply to an HTML element, but targets either a particular state or, in CSS3, a variety of family relationships. Below is a list of some of the

more common pseudo-class and pseudo-element selectors.

a:link pseudoclass

Selects links that have not been visited.

a:visited pseudoclass

Selects links that have been visited.

:focus pseudoclass

Selects elements (such as text boxes or list boxes) that have the input focus.

:hover pseudoclass

Selects elements that the mouse pointer is currently above.

:active pseudoclass

Selects an element that is being activated by the user. A typical example is a link that is being clicked.

:checked pseudoclass

Selects a form element that is currently checked. A typical example might be a radio button or a check box.

:firstchild pseudoclass

Selects an element that is the first child of its parent. A common use is to provide different styling to the first element in a list.

:firstletter pseudoelement

Selects the first letter of an element. Useful for adding drop-caps to a paragraph.

:firstline pseudoelement

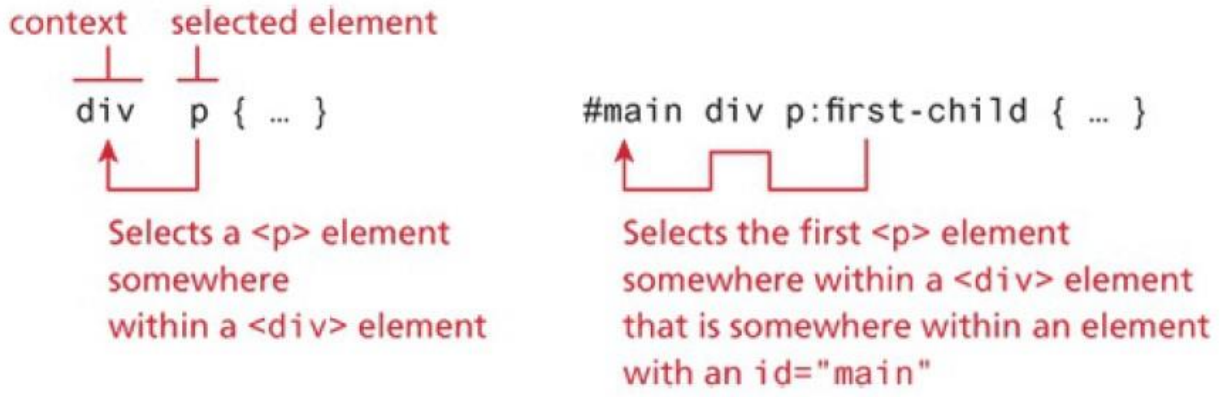
Selects the first line of an element.

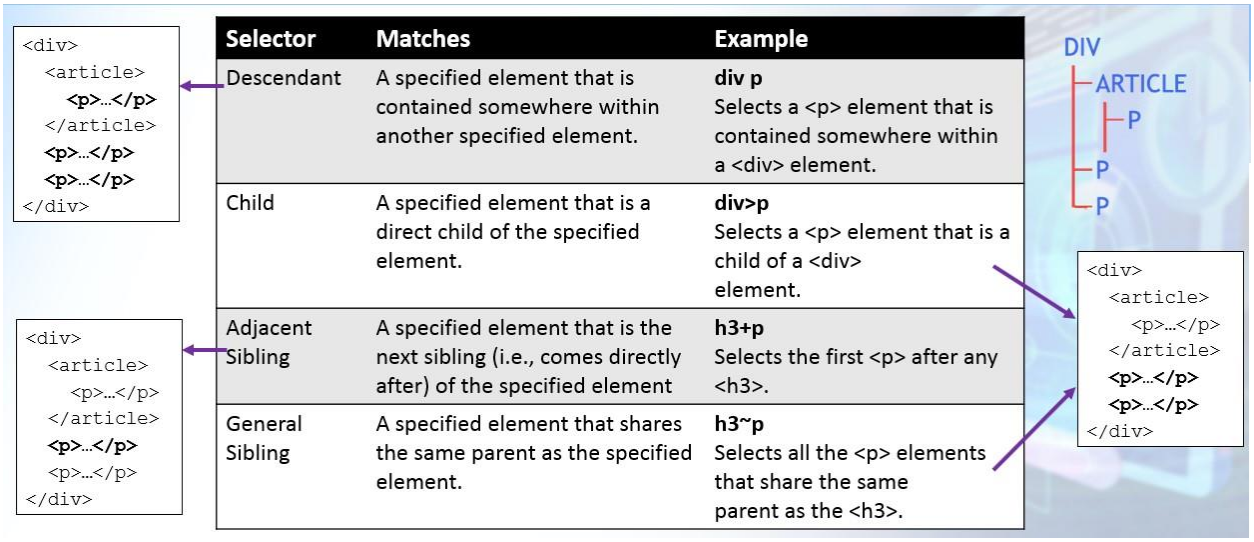
Lesson 28 : CSS selectors (Contextual)

CSS Selectors (Contextual):

A contextual selector (in CSS3 also called combinators) allows you to select elements based on their *ancestors*, *descendants*, or *siblings*.

A descendant selector matches all elements that are contained within another element.





Selector	Matches	Example
Descendant	A specified element that is contained somewhere within another specified element.	**div p** Selects a <p> element that is contained somewhere within a element.
Child	A specified element that is a direct child of the specified element.	**div>p** Selects a <p> element that is a child of a element.
Adjacent Sibling	A specified element that is the next sibling (i.e., comes directly after) of the specified element	**h3+p** Selects the first <p> after any <h3>.
General Sibling	A specified element that shares the same parent as the specified element.	**h3~p** Selects all the <p> elements that share the same parent as the <h3>.

Lesson 29 : CSS cascading

CSS Cascading:

- The “Cascade" in CSS refers to how conflicting rules are handled.

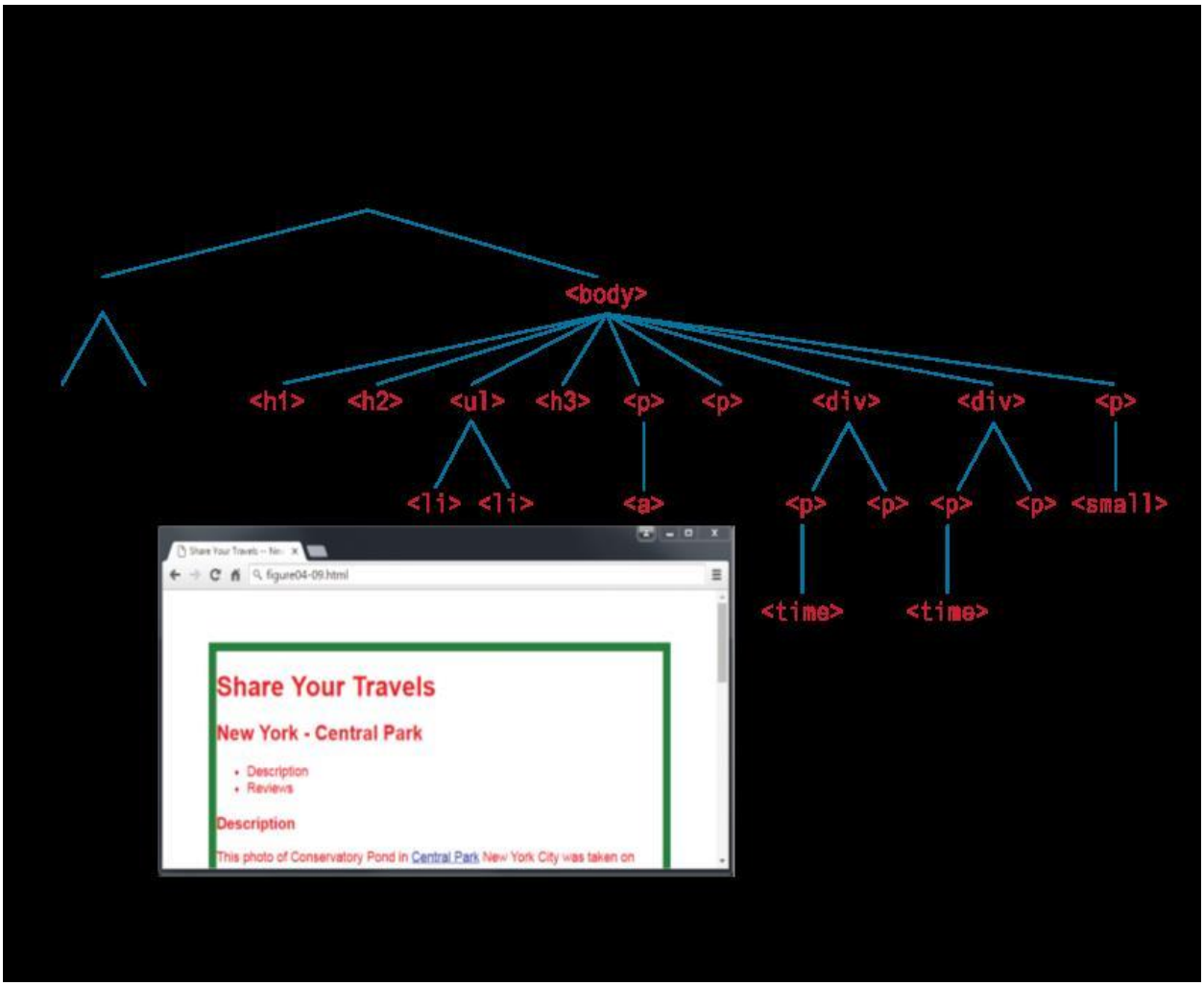
- The downward movement of water down a cascade is meant to be analogous to how a given style rule will continue to take precedence with child elements

- CSS uses the following cascade principles to help it deal with conflicts:
    - inheritance
    - specificity
    - location

Inheritance:

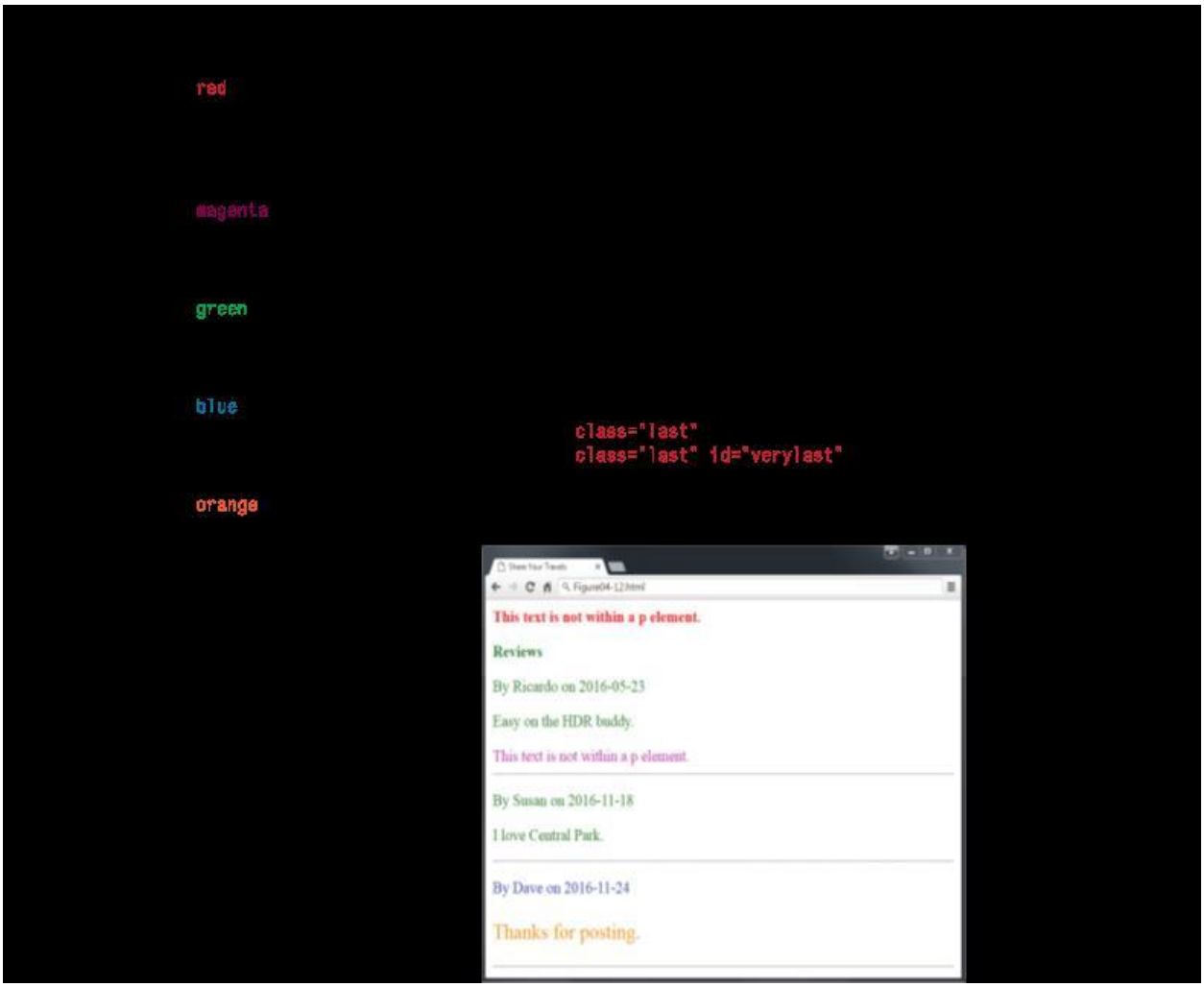
- Many (but not all) CSS properties affect not only themselves but their descendants as well
- Inheritable:
    - font, color, list, and text properties
- Not inheritable:
    - layout, sizing, border, background, and spacing properties

In the first example, only some of the property rules are inherited from the <body> element. That is, only the body element (thankfully!) will have a thick green border and the 100-px margin; however, all the text in the other elements in the document will be in the Arial font and colored red.



Specificity:

Specificity is how the browser determines which style rule takes precedence when more than one style rule could be applied to the same element. In CSS, the more specific the selector, the more it takes precedence (i.e., overrides the previous definition).



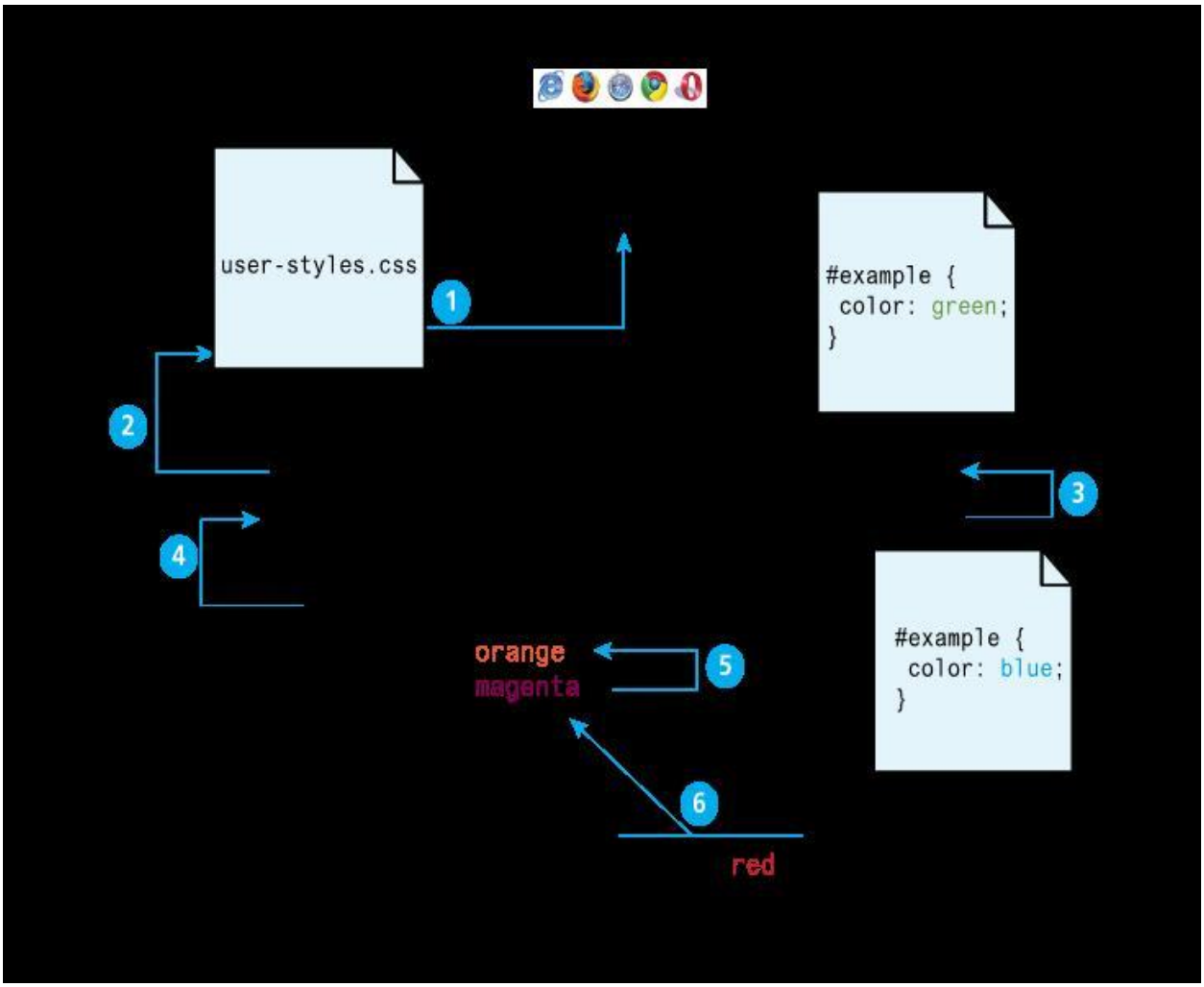
In the example shown in image, the color and font-weight properties defined in the <body> element are inheritable and thus potentially applicable to all the child elements contained within it. However, because the and <p> elements also have the same properties set, they *override* the value defined for the <body> element because their selectors (and <p>) are more specific. As a consequence, their font-weight is normal and their text is

colored either green or magenta.

Location:

When inheritance and specificity cannot determine style precedence, the principle of location will be used. The principle of location is that when rules have the same specificity, then the latest are given more weight. For instance, an inline style will override one defined in an external author style sheet or an embedded style sheet. Similarly, an embedded style will override

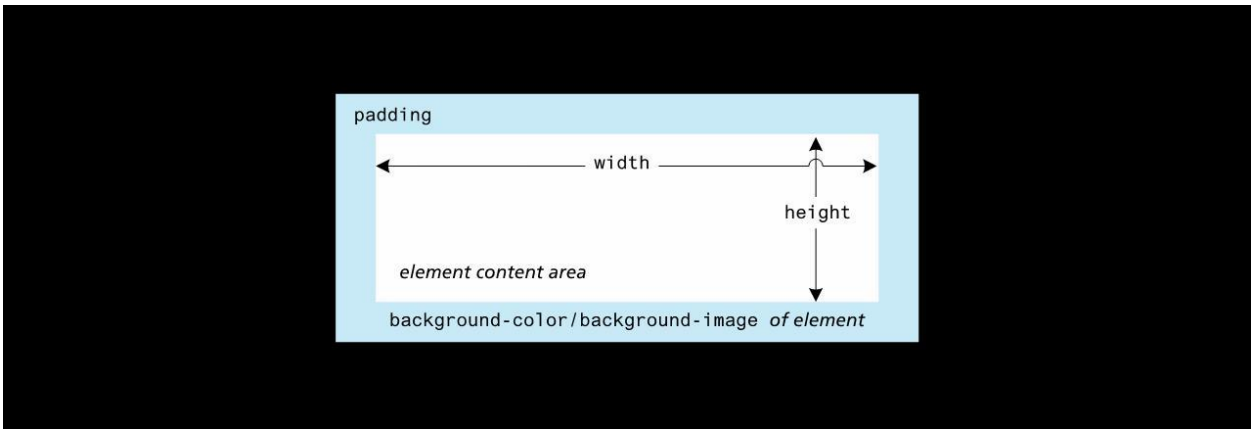
an equally specific rule defined in an external author style sheet if it appears after the external sheet's <link> element.

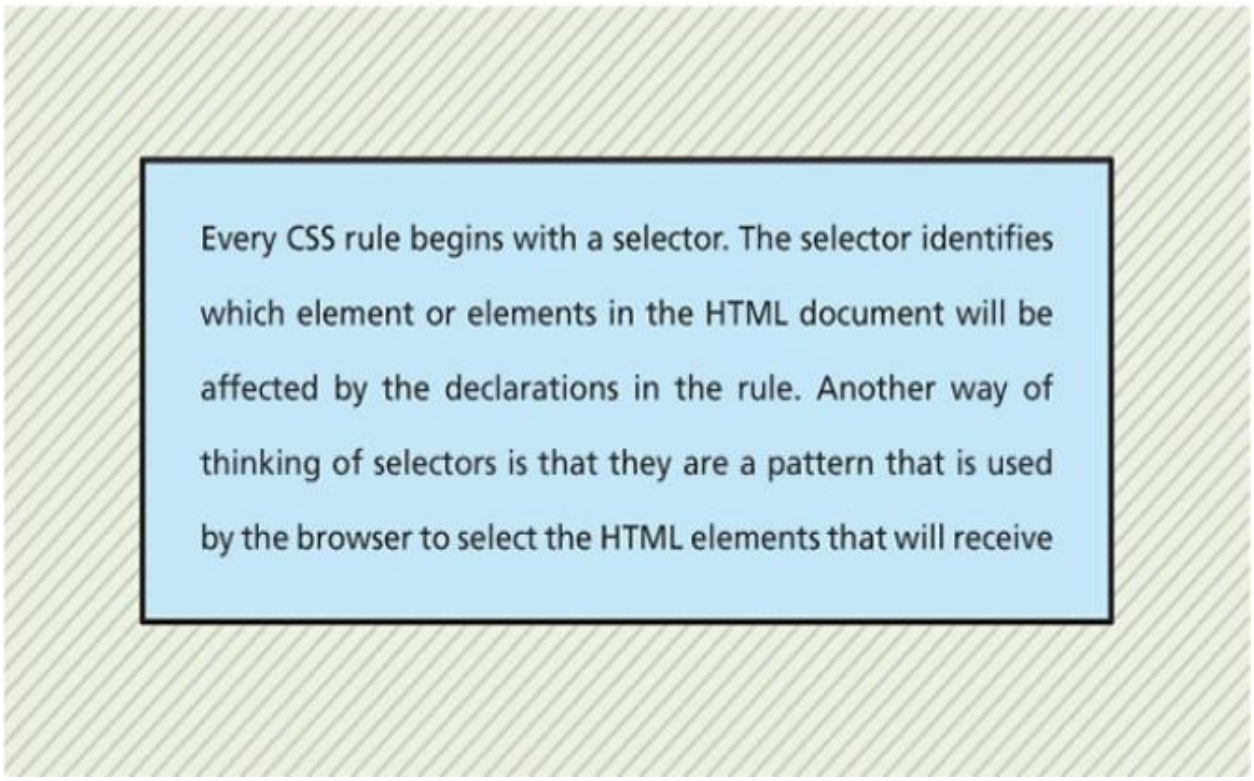


Lesson 30 : The Box Model (1)

The Box Model

In CSS, all HTML elements exist within an element box shown in Figure below. In order to become proficient with CSS, you must become familiar with the element box.





Background:

the background color or image of an element fills an element out to its border (if it has one, that is).

Common Background Properties

Reference: taken from https://www.w3schools.com/

Value	Description

background-color	Specifies the background color to be used
background-image	Specifies ONE or MORE background images to be used
background-position	Specifies the position of the background images
background-size	Specifies the size of the background images
background-repeat	Specifies how to repeat the background images
background-origin	Specifies the positioning area of the background images
background-clip	Specifies the painting area of the background images
background-attachment	Specifies whether the background images are fixed or scrolls with the rest of the page

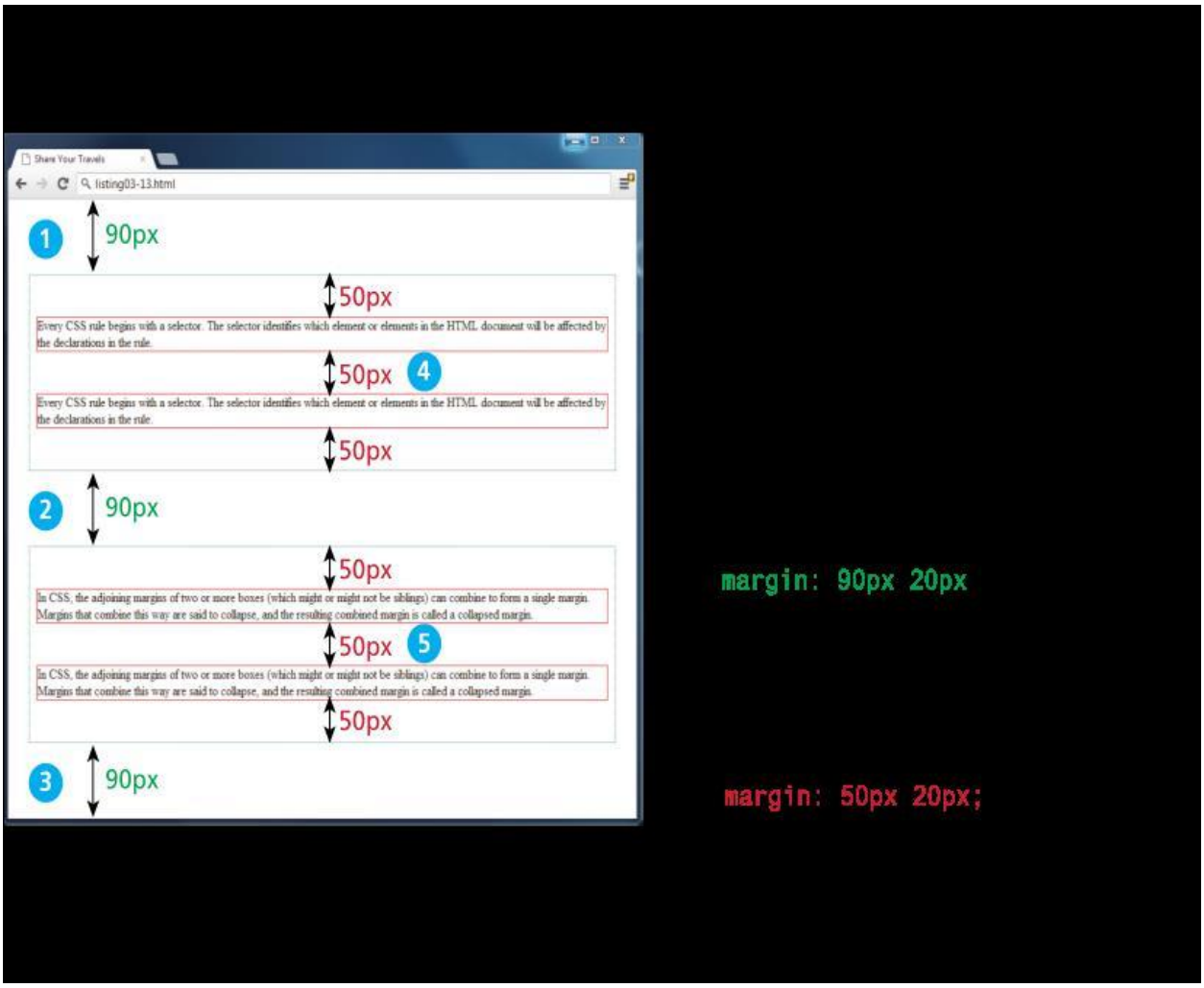
CSS TRBL Shortcut

- **border-top-color: red;** /* sets just the top side */
- **border-right-color: green;** /* sets just the right side */
- **border-bottom-color: yellow;** /* sets just the bottom side */
- **border-left-color: blue;** /* sets just the left side */
- Alternately, we can set all four sides at once:
- **border-color: red;** /* sets all four sides to red */

- **border-color: red green orange blue;** /* sets 4 colors ***/**



Collapsing Margins



Lesson 31 : The Box Model (2)

Box Dimensions

Content-box

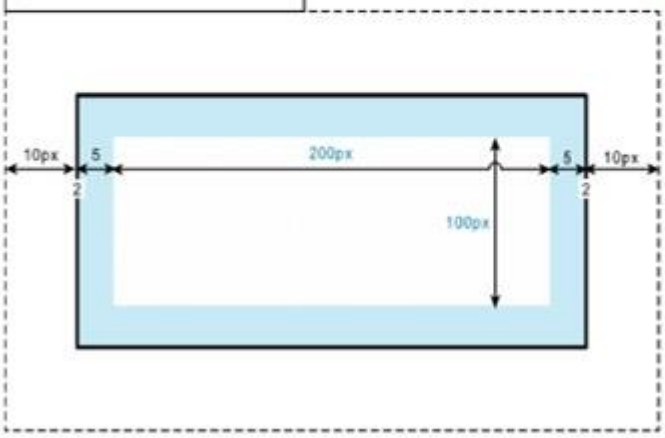
- width/height applied to contents only
- borders and padding counted separately
- margins counted separately

Border-box

- width/height include border and padding
- margins counted separately

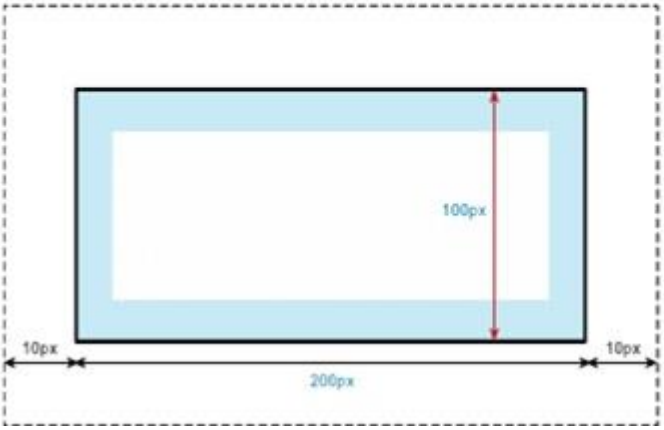
```
div {
  box-sizing: content-box;
  width: 200px;
  height: 100px;
  padding: 5px;
  margin: 10px;
  border: solid 2pt black;
}
```

True element width = 10 + 2 + 5 + 200 + 5 + 2 + 10 = 234 px
True element height = 10 + 2 + 5 + 100 + 5 + 2 + 10 = 134 px




```
div {
  ...
  box-sizing: border-box;
}
```

True element width = 10 + 200 + 10 = 220 px
True element height = 10 + 100 + 10 = 120 px



Limitations of Height Property

For block-level elements such as <p> and elements, there are limits to what the width and height properties can actually do. You can shrink the width, but the content still needs to be displayed, so the browser may very well ignore the height that you set.

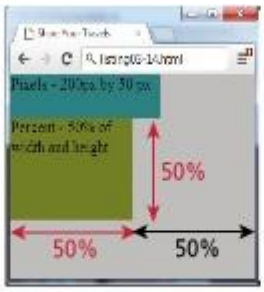


Overflow Property

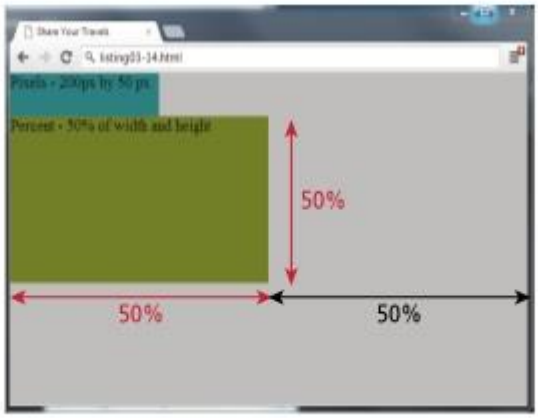
It is possible to control what happens with the content if the box's width and height are not large enough to display the content using the overflow property.

Box Sizing Using Percentages

```
<style>
  html,body {
     margin:0;
     width:100%;
     height:100%;
     background: silver;
  }
  .pixels {
     width:200px;
     height:50px;
     background: teal;
  }
  .percent {
     width:50%;
     height:50%;
     background: olive;
  }
```




```
<body>
   <div class="pixels">
     Pixels - 200px by 50 px
   </div>
   <div class="percent">
     Percent - 50% of width and height
   </div>
</body>
```



Box Sizing Using Pixels

Lesson 32 : CSS Text Styling

CSS Text Styling

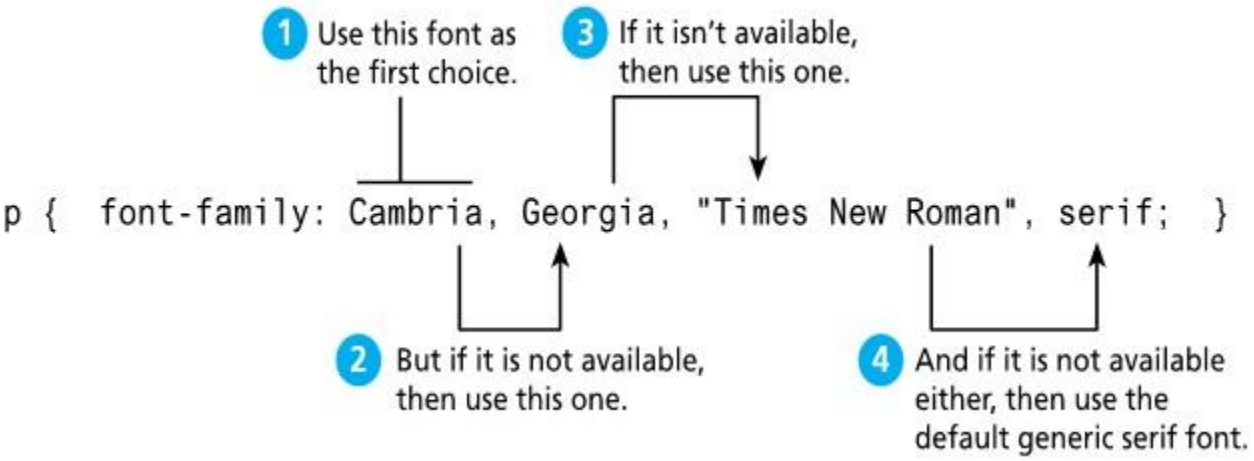
CSS provides two types of properties that affect text. The first we call font properties because they affect the font and its appearance. The second type of CSS text properties are referred to here as paragraph properties since they affect the text in a similar way no matter which font is being used.

Font Properties

Property	Description
font	A combined shorthand property that allows you to set the family, style, size, variant, and weight in one property. **style weight variant size font-family**
font-family	Specifies the typeface/font to use. More than one can be specified.

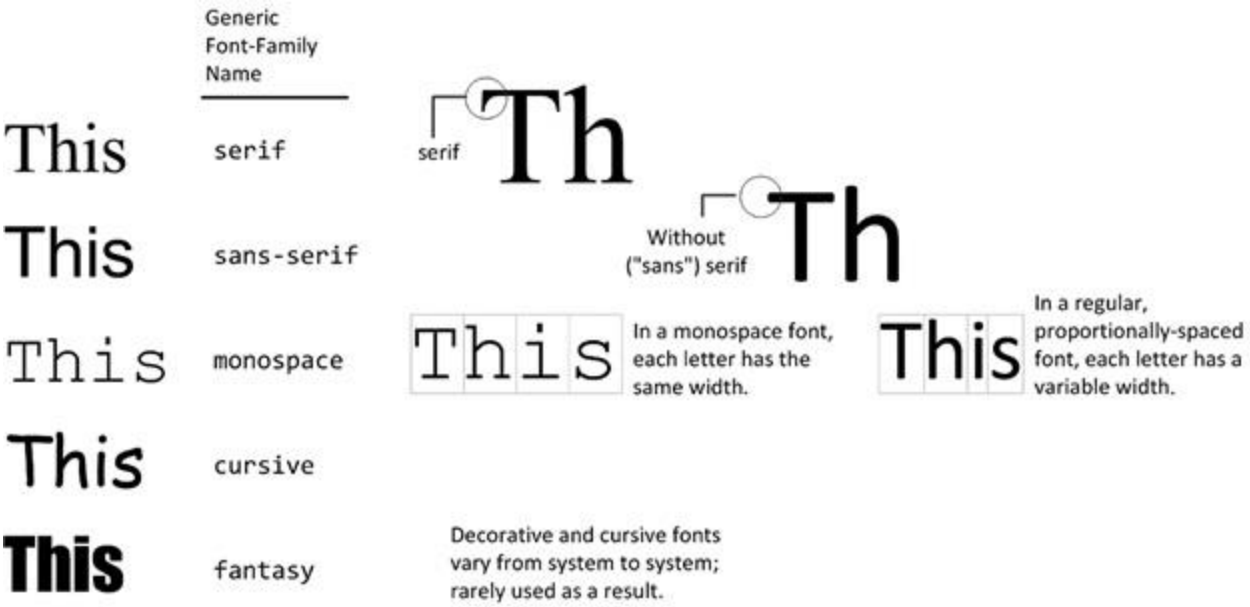
font-size	The size of the font in one of the measurement units
font-style	Specifies whether italic, oblique, or normal
font-variant	Specifies either small-caps text or none
font-weight	Specifies either normal, bold, bolder, lighter, or a value between 100 and 900 in multiples of 100, where larger number represents weightier (i.e., bolder) text.

Specifying the Font Family



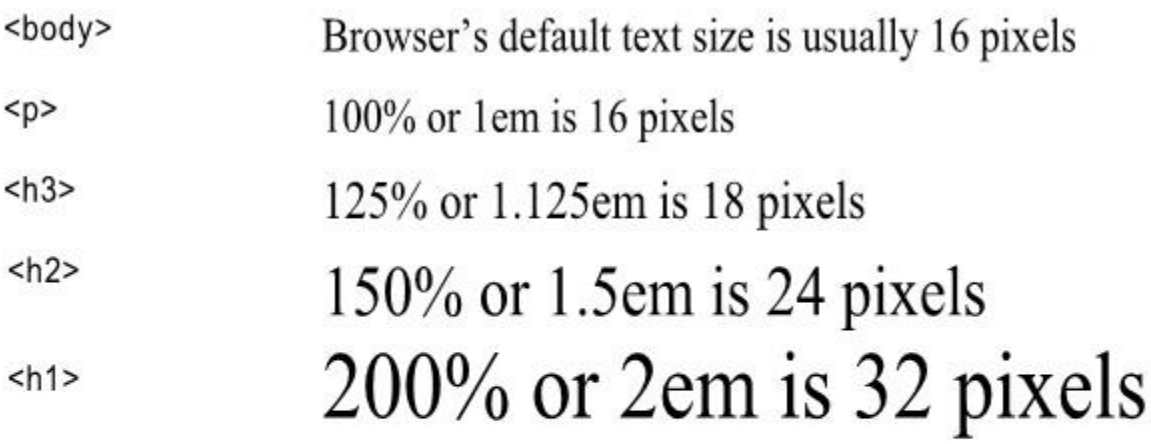
Different Font Families

The font-family property supports five different generic families; the browser supports a typeface from each family.



Font Sizes

Another potential problem with web fonts is font sizes. In a print-based program such as a word processor, specifying a font size is unproblematic. Making some text 12 pt will mean that the font's bounding box (which in turn is roughly the size of its characters) will be 1/6 of an inch tall when printed, while making it 72 pt will make it roughly one inch tall when printed. However, as we saw absolute units such as points and inches do not translate very well to pixel-based devices. Newer mobile devices in recent years have been increasing pixel densities so that a given CSS pixel does not correlate to a single device pixel.




```
/* using 16px scale */

body { font-size: 100%; }
p { font-size: 1em; }       /* 1.0  x 16 = 16 */
h3 { font-size: 1.125em; }  /* 1.25 x 16 = 18 */
h2 { font-size: 1.5em; }    /* 1.5 x 16  = 24 */
h1 { font-size: 2em; }      /* 2 x 16 = 32 */
```

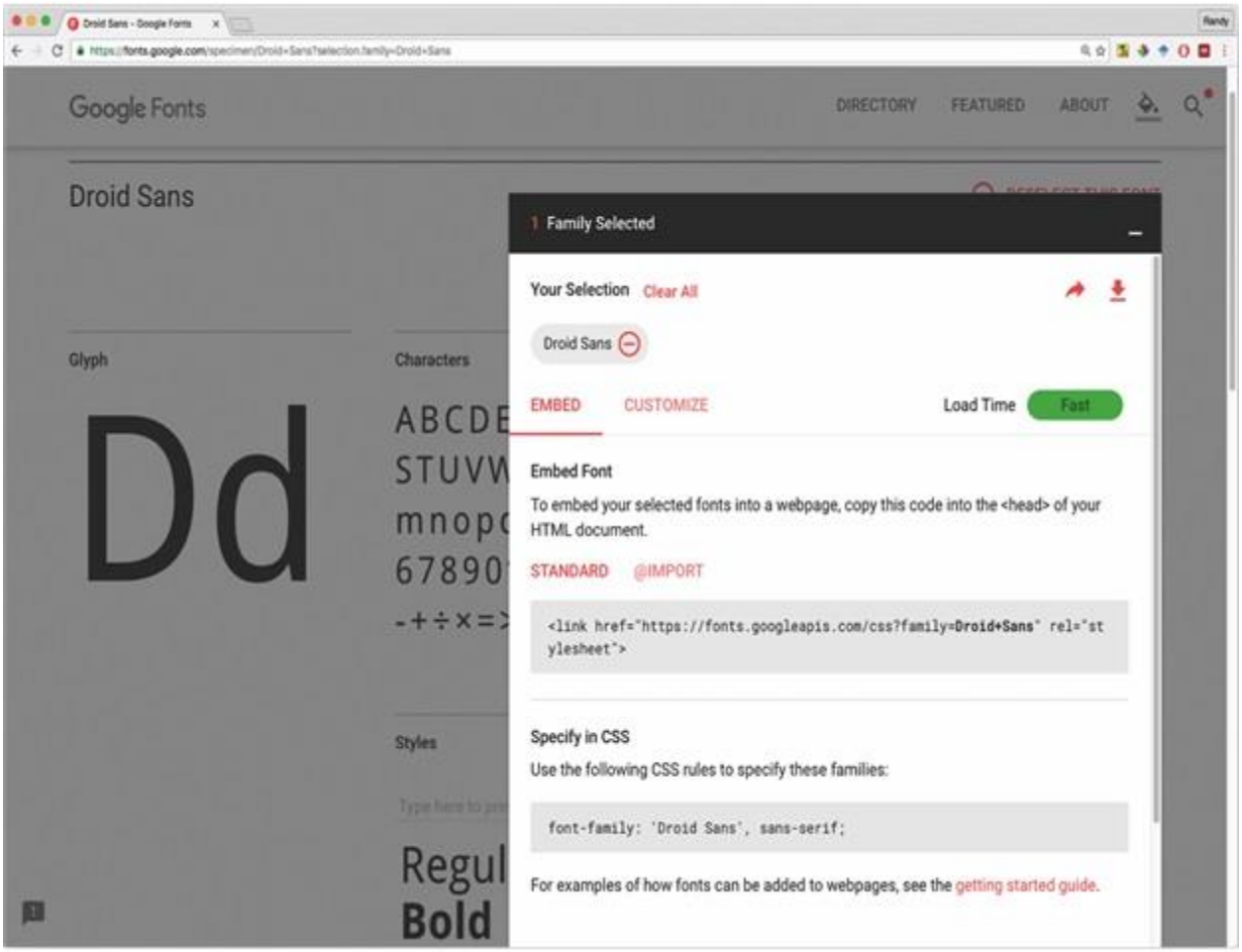
```
<body>
  Browser's default text size is usually 16 pixels
  <p>100% or 1em is 16 pixels</p>
  <h3>125% or 1.125em is 18 pixels</h3>
  <h2>150% or 1.5em is 24 pixels</h2>
  <h1>200% or 2em is 32 pixels</h1>
</body>
```

CSS Text Styling

Embedding Fonts

Due to the ongoing popularity of open source font sites such as Google Web Fonts (https://fonts.google.com) and Font Squirrel (http://www.fontsquirrel.com/), @font-face seems to have gained a critical mass of widespread usage.

The following example illustrates how to use Droid Sans (a system font also used by Android devices) from Google Web Fonts using @font-face.



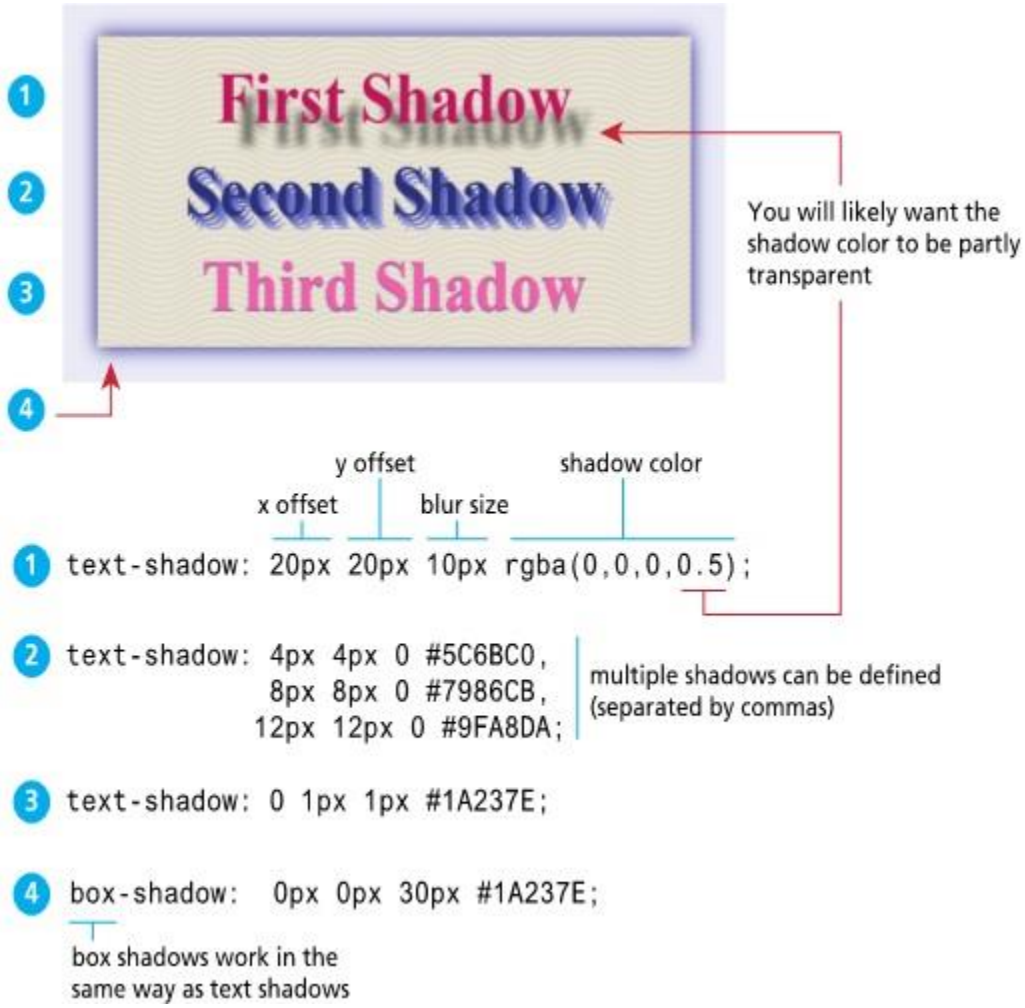
Paragraph Properties

Reference: taken from https://www.w3schools.com/

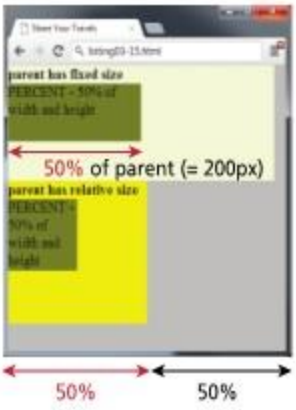
Property	Description
letter-spacing	Specifies the space between characters in a text

line-height	Specifies the line height
text-indent	Specifies the indentation of the first line in a text-block
white-space	Specifies how to handle white-space inside an element
word-spacing	Specifies the space between words in a text
text-shadow	Specifies the shadow effect added to text

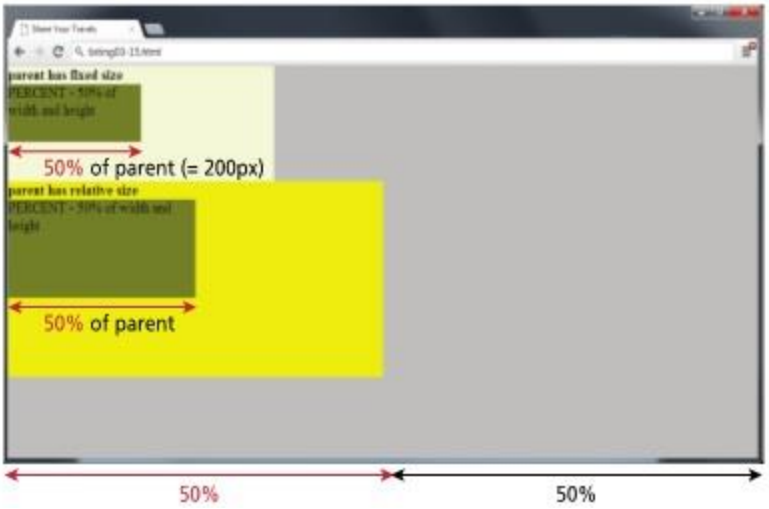
Shadow Example:



```
  .parentFixed {
     width:400px;
     height:150px;
     background: beige;
  }
  .parentRelative {
     width:50%;
     height:50%;
     background: yellow;
  }
</style>
```



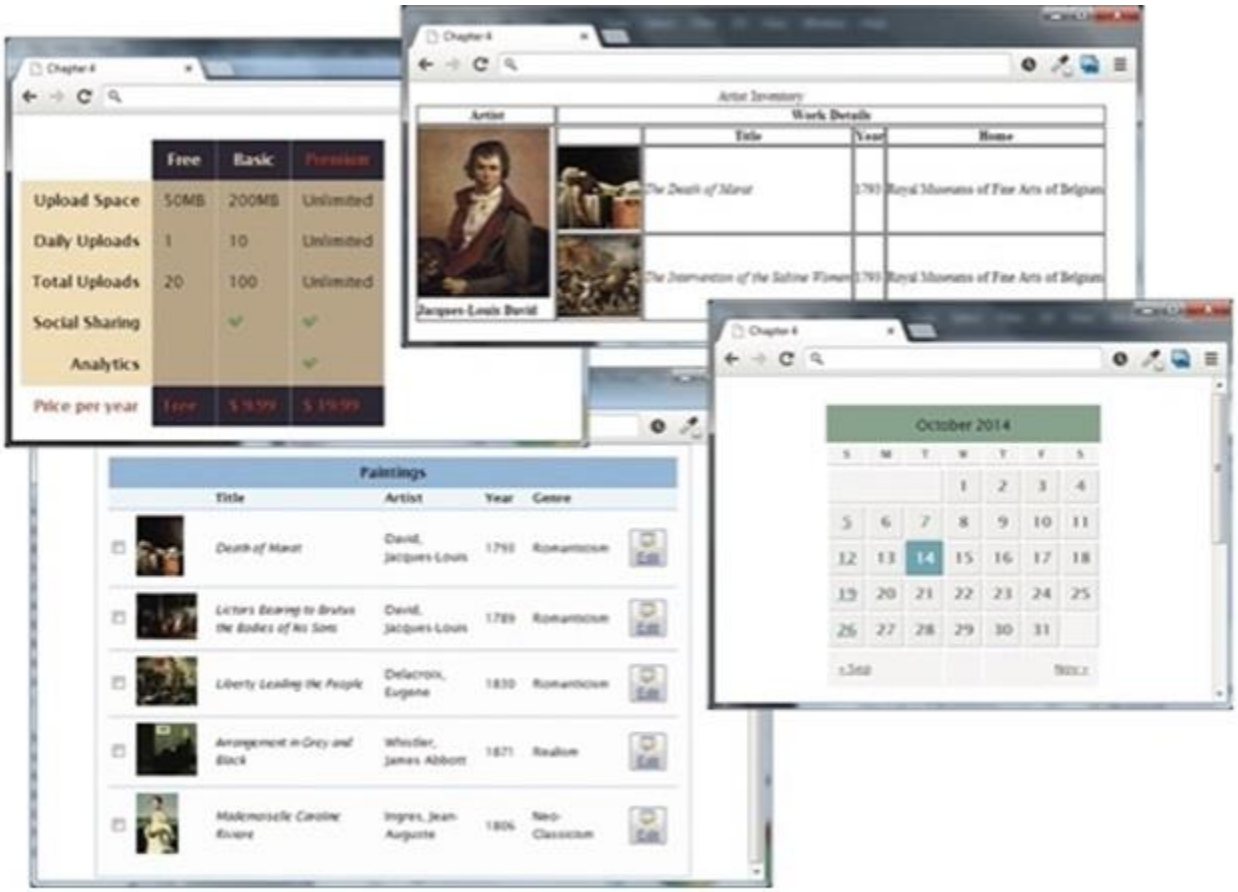

```
<body>
<div class="parentFixed">
   <strong>parent has fixed size</strong>
   <div class="percent">
      PERCENT - 50% of width and height
   </div>
</div>
<div class="parentRelative">
   <strong>parent has relative size</strong>
   <div class="percent">
      PERCENT - 50% of width and height
   </div>
</div>
</body>
```



Lesson 33 : Introducing Tables

Introducing Tables

A table in HTML is created using the <table> element and can be used to represent information that exists in a two-dimensional grid. Tables can be used to display calendars, financial data, pricing tables, and many other types of data. Just like a real-world table, an HTML table can contain any type of data: not just numbers, but text, images, forms, even other tables, as shown in the Figure below.



Basic Table Structure

Create a Basic Table Complex Content in Tables As can be seen in Figure, an HTML <table> contains any number of rows (<tr>); each row contains any number of table data cells (<td>). The indenting shown in Figure is purely a convention to make the markup more readable by humans.

<table>
<tr>
<td>The Death of Marat</td>
<td>Jacques-Louis David</td>
<td>1793</td>
<td>162cm</td>
<td>128cm</td>
</tr>
<tr>
<td>Burial at Ornans</td>
<td>Gustave Courbet</td>
<td>1849</td>
<td>314cm</td>
<td>663cm</td>
</tr>
</table>

```
<table>
   <tr>
      <td>The Death of Marat</td>
      <td>Jacques-Louis David</td>
      <td>1793</td>
      <td>162cm</td>
      <td>128cm</td>
   </tr>
   <tr>
      <td>Burial at Ornans</td>
      <td>Gustave Courbet</td>
      <td>1849</td>
      <td>314cm</td>
      <td>663cm</td>
   </tr>
</table>
```



Adding Headings

Many tables will contain some type of headings in the first row. In HTML, you indicate header data by using the <th> instead of the <td> element.

Spanning Rows and Columns

If you want a given cell to cover several columns or rows, then you can do so by using the colspan or rowspan attributes.

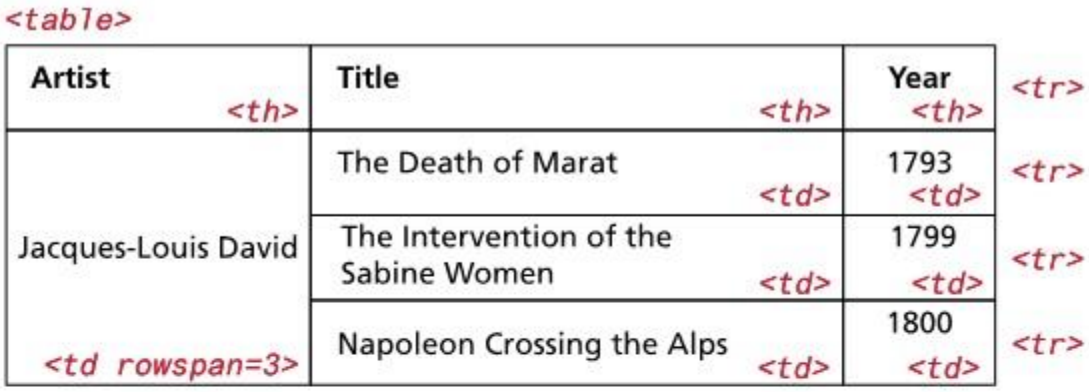
Spanning columns



Notice that this row now only has four cell elements.

```
<table>
  <tr>
    <th>Title</th>
    <th>Artist</th>
    <th>Year</th>
    <th colspan="2">Size (width x height)</th>
  </tr>
  <tr>
    <td>The Death of Marat</td>
    <td>Jacques-Louis David</td>
    <td>1793</td>
    <td>162cm</td>
    <td>128cm</td>
  </tr>
  ...
</table>
```

Spanning Rows




```
<table>
   <tr>
      <th>Title</th>
      <th>Artist</th>
      <th>Year</th>
   </tr>
   <tr>
      <td rowspan="3">Jacques-Louis David</td>
      <td>The Death of Marat</td>
      <td>1793</td>
   </tr>
   <tr>
      <td>The Intervention of the Sabine Women</td>
      <td>1799</td>
   </tr>
   <tr>
      <td>Napoleon Crossing the Alps</td>
      <td>1800</td>
   </tr>
</table>
```

Notice that these two rows now only have two cell elements.

Lesson 34 : Introducing Tables (2)

Introducing Tables

Additional Table Elements

While the previous sections cover the basic elements and attributes for most simple tables, there are some additional table elements that can add additional meaning and accessibility to one's tables.

A title for the table is good for accessibility.

These describe our columns, and can be used to aid in styling.

Table header could potentially also include other <tr> elements.

Yes, the table footer comes *before* the body.

Potentially, with styling the browser can scroll this information, while keeping the header and footer fixed in place.

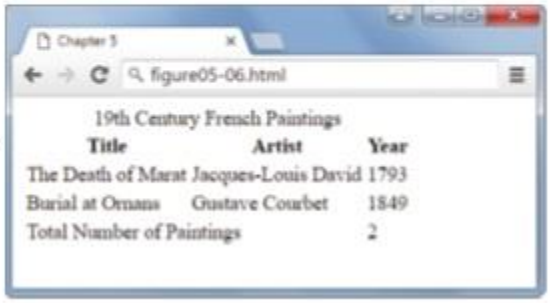
```
<table>
    <caption>19th Century French Paintings</caption>
    <col class="artistName" />
    <colgroup id="paintingColumns">
        <col />
        <col />
    </colgroup>

    <thead>
        <tr>
            <th>Title</th>
            <th>Artist</th>
            <th>Year</th>
        </tr>
    </thead>

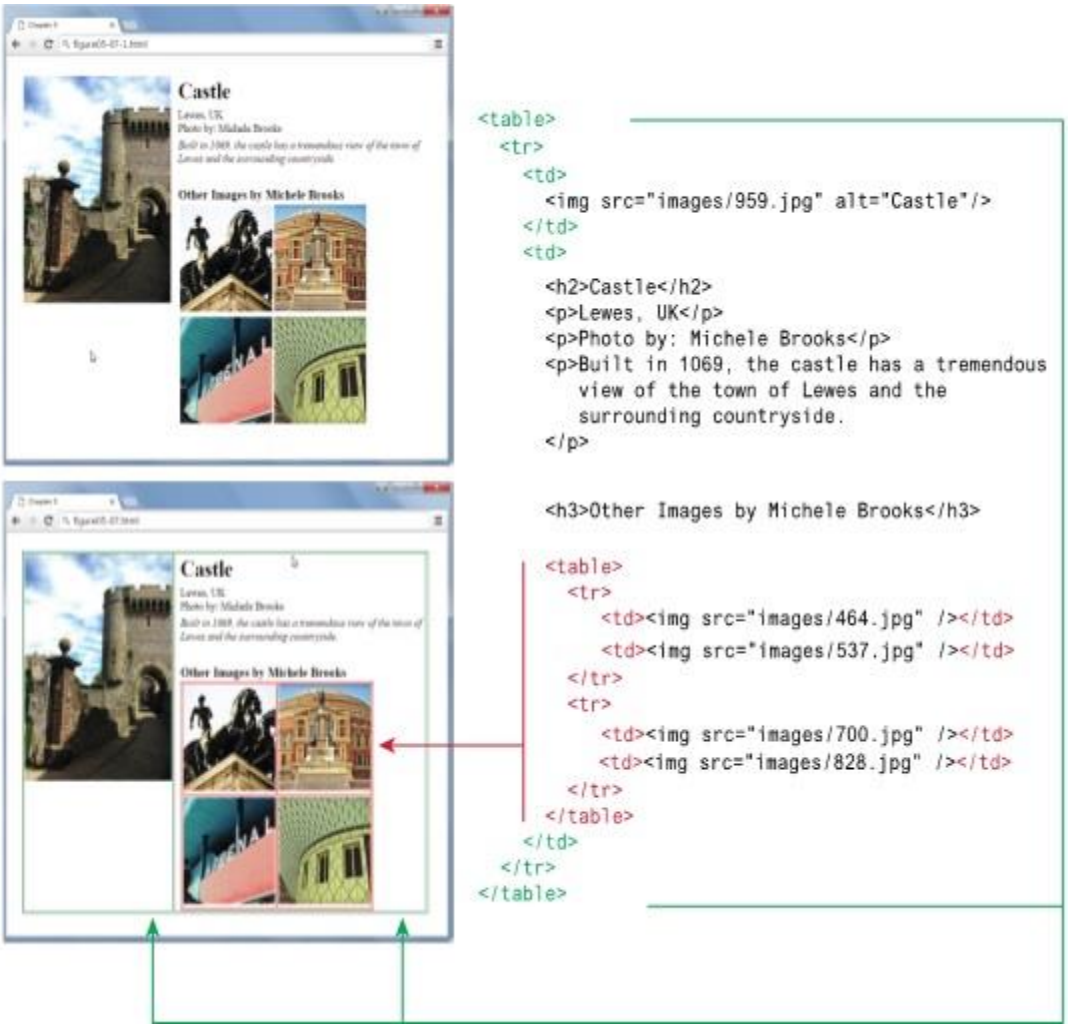
    <tfoot>
        <tr>
            <td colspan="2">Total Number of Paintings</td>
            <td>2</td>
        </tr>
    </tfoot>

    <tbody>
        <tr>
            <td>The Death of Marat</td>
            <td>Jacques-Louis David</td>
            <td>1793</td>
        </tr>
        <tr>
            <td>Burial at Ornans</td>
            <td>Gustave Courbet</td>
            <td>1849</td>
        </tr>
    </tbody>

</table>
```



Using Tables for Layout



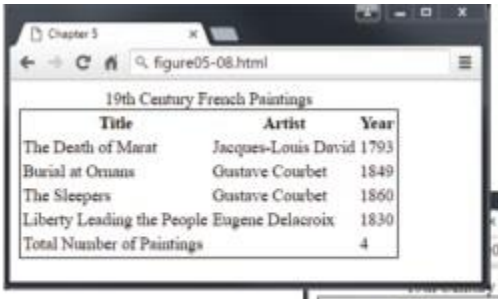
Lesson 35 : Styling table using CSS; Making tables accessible using CSS

Styling Tables

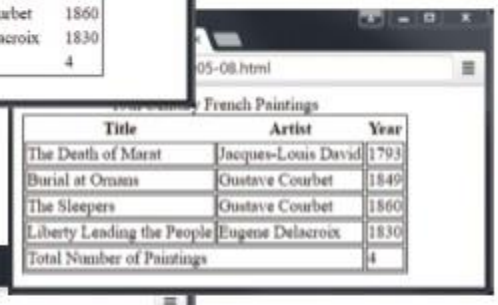
You can style the tables in different way outs.

Table Borders

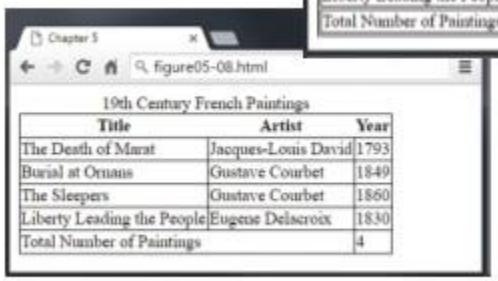
Following are some examples of the table with different borders:



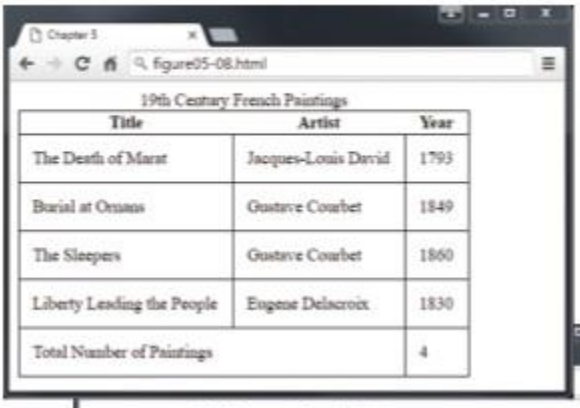

```
table {
   border: solid 1pt black;
}
```



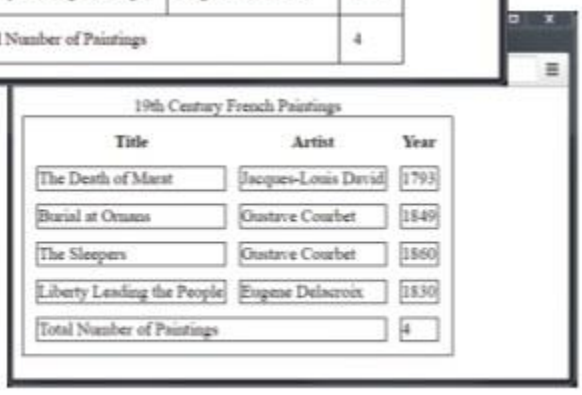

```
table {
   border: solid 1pt black;
}
td {
   border: solid 1pt black;
}
```



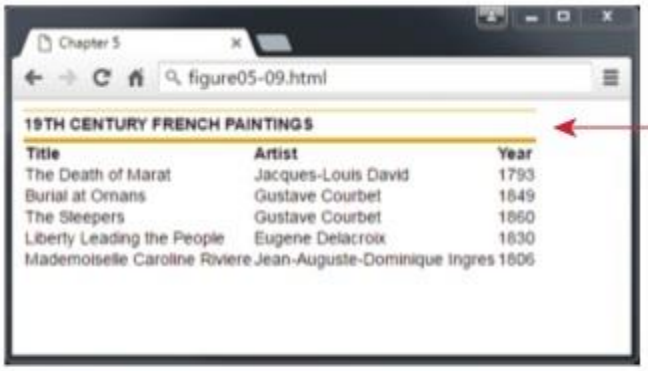

```
table {
   border: solid 1pt black;
   border-collapse: collapse;
}
td {
   border: solid 1pt black;
}
```



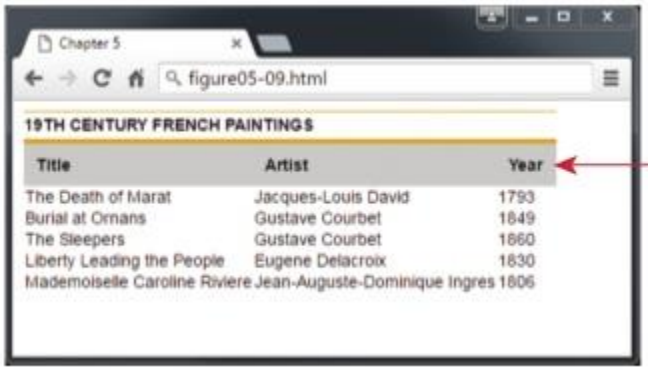

```
table {
   border: solid 1pt black;
   border-collapse: collapse;
}
td {
   border: solid 1pt black;
   padding: 10pt;
}
```



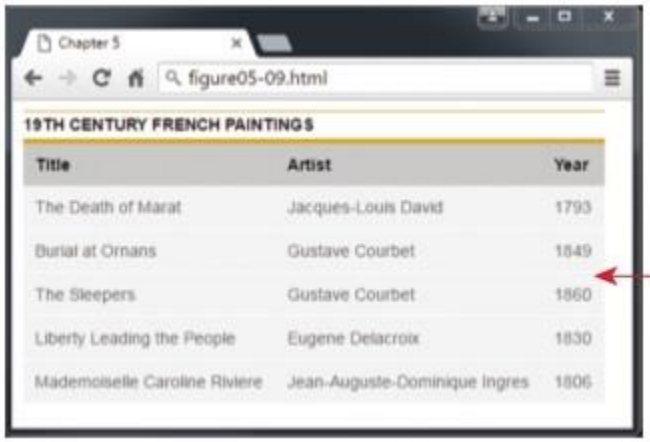

```
table {
   border: solid 1pt black;
   border-spacing: 10pt;
}
td {
   border: solid 1pt black;
}
```




```
caption {
   font-weight: bold;
   padding: 0.25em 0 0.25em 0;
   text-align: left;
   text-transform: uppercase;
   border-top: 1px solid #DCA806;
}
table {
   font-size: 0.8em;
   font-family: Arial, sans-serif;
   border-collapse: collapse;
   border-top: 4px solid #DCA806;
   border-bottom: 1px solid white;
   text-align: left;
}
```

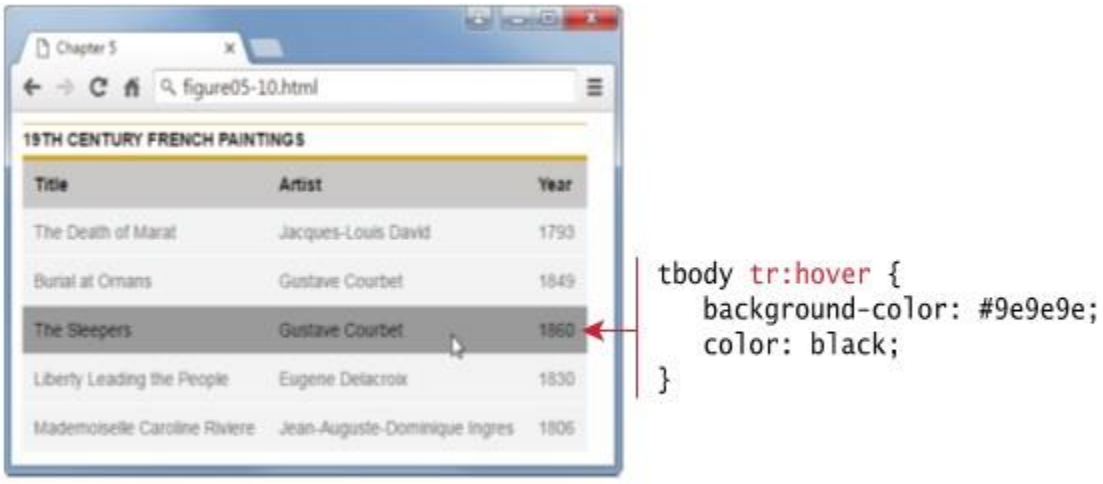


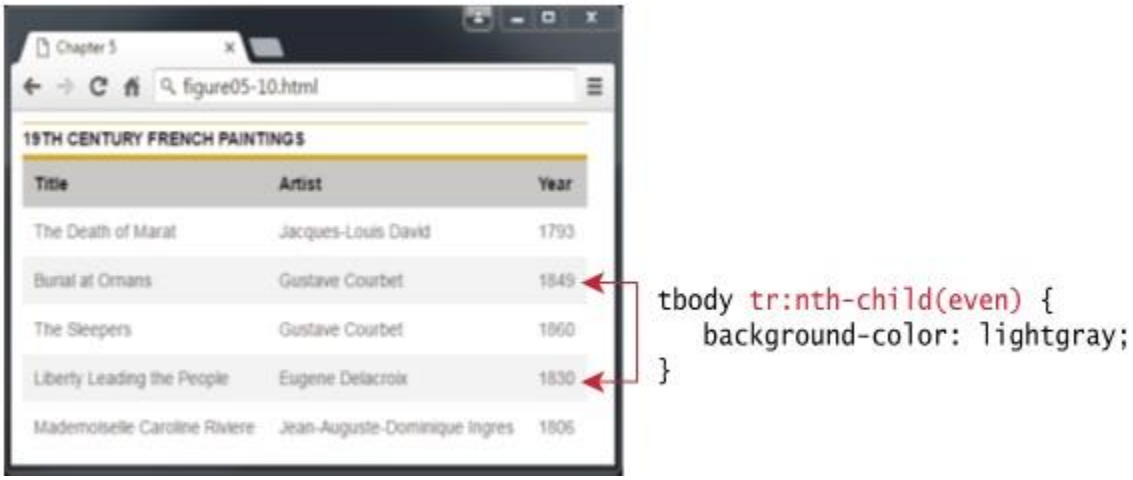

```
thead tr {
   background-color: #CACACA;
}
th {
   padding: 0.75em;
}
```




```
tbody tr {
   background-color: #F1F1F1;
   border-bottom: 1px solid white;
   color: #6E6E6E;
}
tbody td {
   padding: 0.75em;
}
```

.





Accessible Tables

- Use tables for data, not layout
- Use the <caption> element
- Connect cells with a textual description in the header

Code:

<caption>Famous Paintings</caption>

<tr>

<th scope="col">Title</th>

<th scope="col">Artist</th>

<th scope="col">Year</th>

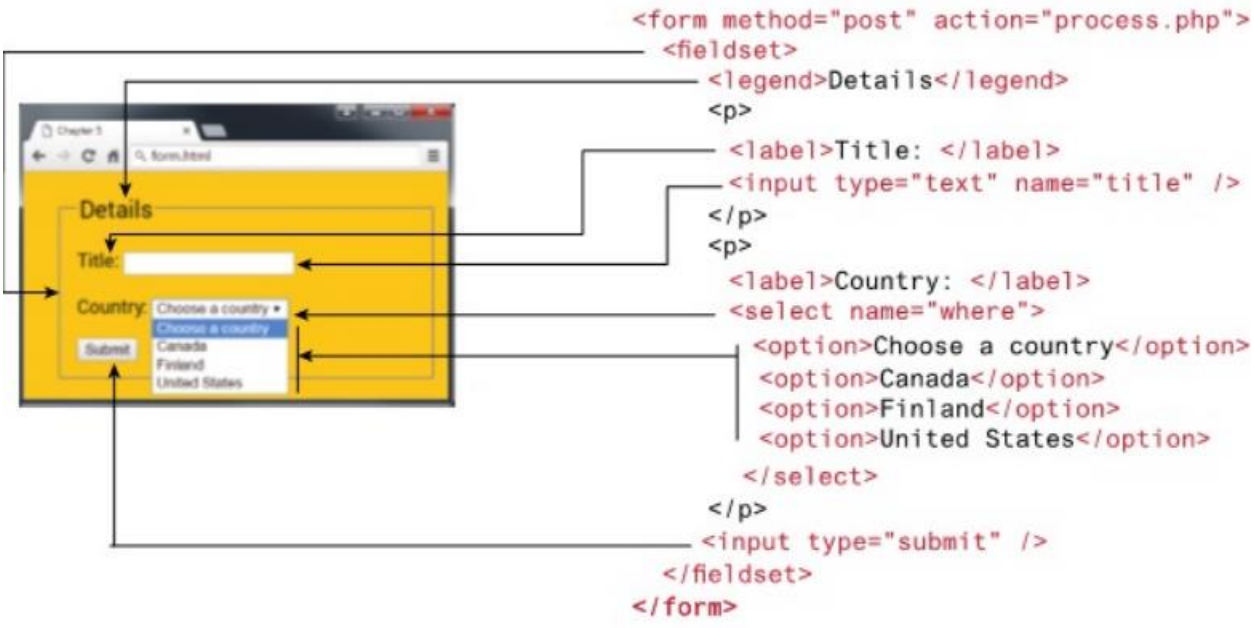
```
    <th scope="col">Width</th>
    <th scope="col">Height</th>
</tr>
```

Week – 4

Lesson 36 : Introducing Forms

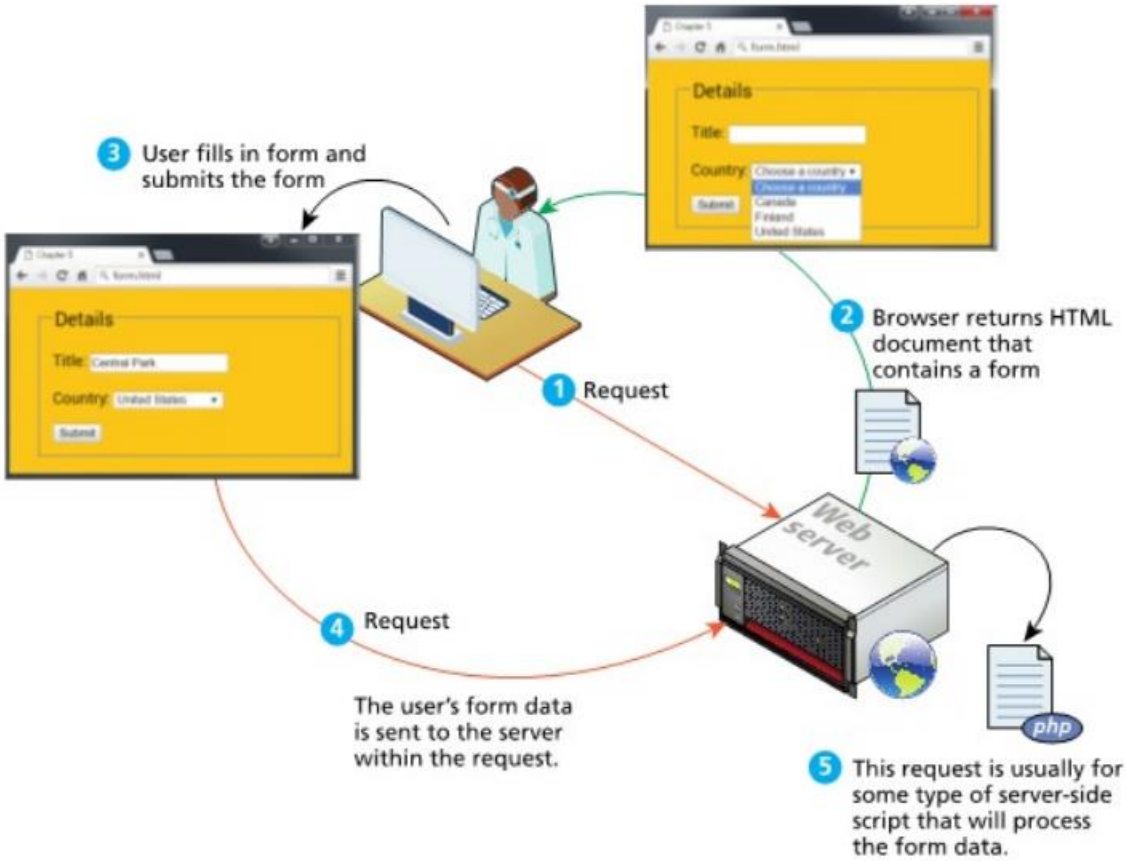
Introducing Forms

Forms provide the user with an alternative way to interact with a web server. Up to now, clicking hyperlinks was the only mechanism available to the user for communicating with the server.



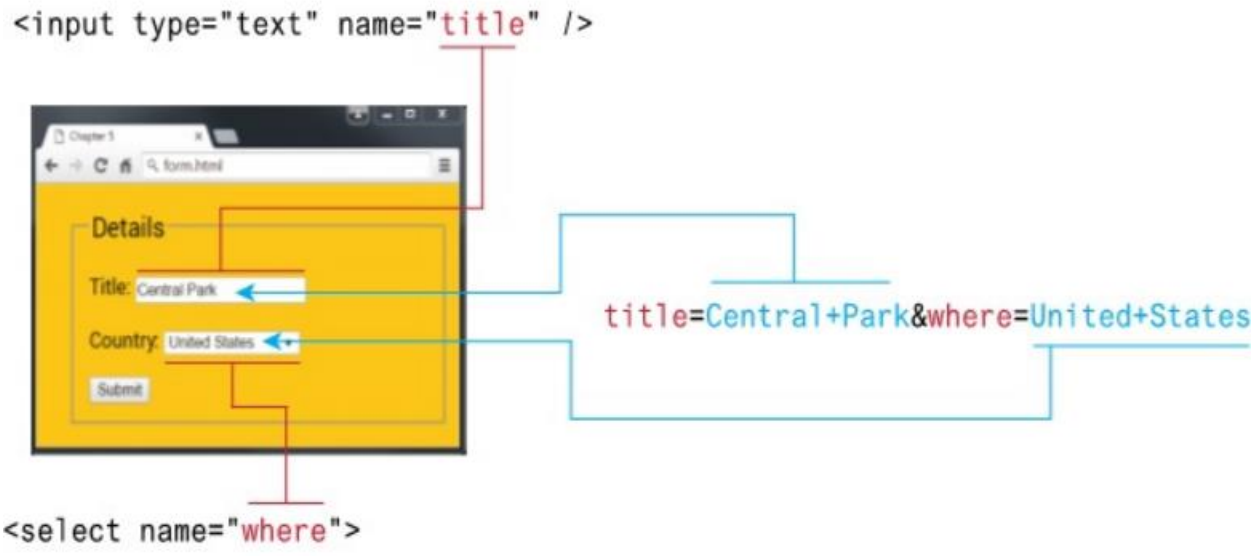
How Forms Work

The process begins with a request for an HTML page that contains some type of form on it. This could be something as complex as a user registration form or as simple as a search box. After the user fills out the form, there needs to be some mechanism for submitting the form data back to the server. This is typically achieved via a submit button, but through JavaScript, it is possible to submit form data using some other type of mechanism.



Query Strings

The browser packages the user's data input into something called a query string. A query string is a series of name=value pairs separated by ampersands (the & character).



GET and POST

The method attribute specifies how the query string data will be transmitted from the browser to the server. There are two possibilities: GET and POST.

What is the difference between GET and POST?

The difference resides in where the browser locates the user's form input in the subsequent HTTP request. With GET, the browser locates the data in the URL of the request; with POST, the form data is located in the HTTP header after the HTTP variables. Following Figure illustrates how the two methods differ.



GET versus POST

Type	Advantages and Disadvantages
GET	• Data can be clearly seen in the address bar. This may be an advantage during development but a disadvantage in production. • Data remains in browser history and cache. Again this may be beneficial to some users, but a security risk on public computers. • Data can be bookmarked (also an advantage and a disadvantage). • Limit on the number of characters in the form data returned.
POST	• Data can contain binary data. • Data is hidden from user. • Submitted data is not stored in cache, history, or Bookmarks.

Lesson 37 : Form control elements 1

Form Control Elements:

Most forms need to gather text information from the user. Whether it is a search box, or a login form, or a user registration form, some type of text input is usually necessary. Different text input controls are mentioned below:

Type	**Description**
Text	Creates a single-line text entry box. <input type="text" name="title" />
Textarea	Creates a multiline text entry box. <textarea rows="3" ... />
password	Creates a single-line text entry box for a <input type="password" ... />
search	Creates a single-line text entry box suitable for a search string. <input type="search" ... />
email	Creates a single-line text entry box suitable for entering an email address. <input type="email" ... />
tel	Creates a single-line text entry box suitable for entering a telephone. <input type="tel" ... />
url	Creates a single-line text entry box suitable for entering a URL. <input type="url" ... />

Datalist:

<datalist> element is new addition to HTML5. This element allows to define a list of elements that can appear in a drop-down autocomplete style list for a text element.

Choice Controls:

Forms often need the user to select an option from a group of choices. HTML provides several ways to do this.

1. **Select Lists:**

The <select> element is used to create a multiline box for selecting one or more items. The options can be hidden in a dropdown list or multiple rows of the list can be visible. Option items can be grouped together via the <optgroup> element.

2. **Radio Buttons:**

Radio buttons are useful when you want the user to select a single item from a small list of choices and you want all the choices to be visible.

3. **Check Boxes:**

Checkboxes are used for getting yes/no or on/off responses from the user.

Button Controls:

HTML defines several different types of buttons which are mentioned below:

Type	Description
Submit	Creates a button that submits the form data to the server.
Reset	Creates a button that clears any of the user's already entered form data.
button	Creates a custom button. This button may require JavaScript for it to actually perform any action.
image	Creates a custom submit button that uses an image for its display.

Lesson 38 : Form control elements 2

Form Control Elements:

Date & Time Controls:

Types	Description
Date	Creates a general date input control. The format for the date is "yyyy-mm-dd."
Time	Creates a time input control. The format for the time is "HH:MM:SS," for hours:minutes:seconds.

month	Creates a control in which the user can enter a month in a year. The format is “yyyy-mm.”
week	Creates a control in which the user can specify a week in a year. The format is “yyyy-W##.”

Lesson 39 : Microformats

Microformats:

The web has millions of pages in it. Yet, despite the incredible variety, there is a surprising amount of similar information from site to site. Most sites have some type of Contact Us page, in which addresses and other information are displayed; similarly, many sites contain a calendar of upcoming events or information about products or news. The idea behind microformats is that if this type of common information were tagged in similar ways, then automated tools would be able to gather and transform it. A microformat is a small pattern of HTML markup and attributes to represent common blocks of information such as people, events, and news stories so that the information in them can be extracted and indexed by software agents.

One of the most common microformats is hCard, which is used to semantically markup contact information for a person. Google Map search results now make use of the hCard microformat so that if you used the appropriate browser extension, you could save the information to your computer or phone’s contact list.

Lesson 40 : Image formats for the web

Image Formats for the Web:

When you see text and images on your desktop monitor or your mobile screen, you are seeing many small squares of colored light called pixels that are arranged in a two-dimensional grid. There are two basic categories of digital representations for images: raster and vector. In a **raster image** (also called a bitmap image) the smaller components are pixels. That is, the image is broken down into a two-dimensional grid of colored squares. Each colored square uses a number that represents its color value. A raster image has a set number of pixels, dramatically increasing or decreasing its size can dramatically affect its quality. A **vector image** is not composed of pixels but instead is composed of objects such as lines, circles, Bezier curves, and polygons.

File Formats:

1. **JPEG:**

JPEG (Joint Photographic Experts Group) or JPG is a 24-bit, true-color file format that is ideal for photographic images. It uses a sophisticated compression scheme that can dramatically reduce the file

size of the image. JPEG is the ideal file format for photographs and other continuous-tone images such as paintings and grayscale images.

2. **GIF:**

The GIF (Graphic Interchange Format) file was the first image format supported by the earliest web browsers. Unlike the 24-bit JPEG format, GIF is an 8-bit or less format, meaning that it can contain no more than 256 colors. GIF files use a much simpler compression system that is lossless, which means that no pixel information is lost.

3. **PNG:**

The PNG (Portable Network Graphics) format is a more recent format. Its main features are:

v. Lossless compression.

v. 8-bit (or 1-bit, 2-bit, and 4-bit) indexed color as well as full 24-bit true color

v. From 1 to 8 bits of transparency.

For normal photographs, JPEG is generally still a better choice because the file size will be smaller than using PNG.

4. **SVG:**

The SVG (Scalable Vector Graphics) file format is a vector format. SVG graphics do not lose quality when enlarged or reduced.

Other Formats:

There are many other file formats for graphical information. The TIF (Tagged Image File) format is a cross-platform lossless image format that supports multiple color depths, 8-bit transparency, layers and color channels, the CMYK and RGB color space, and other features especially useful to print professionals.

Lesson 41 : Understanding normal document flow

Normal Flow:

To understand CSS positioning and layout, it is essential to first understand the idea of normal flow, and how the browser will normally display block-level elements and inline elements from left to right and from top to bottom.

Block-level elements: such as <p>, , <h2>, <ul>, and <table> are each contained on their own line. Because block-level elements begin with a line break, without styling, two block-level elements can't exist on the same line. Block-level elements use the normal CSS box model, in that they have margins, paddings, background colors, and borders.

Inline elements: do not form their own blocks but instead are displayed within lines. Normal text in an HTML document is inline, as are elements such as , <a>, <img>, and .

Lesson 42 : Positioning elements (Relative, absolute, fixed)

Positioning Elements:

It is possible to move an item from its regular position in the normal flow. The position property is used to specify the type of positioning, and the possible values. The left, right, top, and bottom properties are used to indicate the distance the element will move; the effect of these properties varies depending upon the position property.

Relative positioning:

In relative positioning an element is displaced out of its normal flow position and moved relative to where it would have been placed.

Static:

The element is positioned according to the normal flow. This is the default.

Fixed:

The element is fixed in a specific position in the window even when the document is scrolled.

Absolute:

The element is removed from normal flow and positioned in relation to its nearest positioned ancestor.

Lesson 43 : Absolute positioning

Absolute Positioning:

When an element is positioned absolutely, it is removed completely from normal flow. Thus, unlike with relative positioning, space is not left for the moved element, as it is no longer in the normal flow. Its position is moved in relation to its container block.

Lesson 44 : z-index

Z- Index:

Each positioned element has a stacking order defined by the z-index property (named for the z-axis). Items closest to the viewer (and thus on the top) have a larger z-index value, which can be seen in the first. Unfortunately, working with z-index can be tricky and seemingly counterintuitive.

Lesson 45 : Fixed positioning

Fixed Position:

The fixed position value is used relatively infrequently. It is a type of absolute positioning, except that the positioning values are in relation to the viewport (i.e., to the browser window). Elements with fixed positioning do not move when the user scrolls up or down the page. The fixed position is most commonly used to ensure that navigation elements or advertisements are always visible.

Lesson 46 : Floating elements (1)

Floating Elements:

It is possible to displace an element out of its position in the normal flow via the CSS float property. An element can be floated to the left or floated to the right. When an item is floated, it is moved all the way to the far left or far right of its containing block and the rest of the content is “re-flowed” around the floated element.

Floating within a Container:

Floated item moves to the left or right of its container (also called its containing block).

Floating Multiple Items:

One of the more common usages of floats is to place multiple items side by side on the same line. When you float multiple items that are in proximity, each floated item in the container will be nestled up beside the previously floated item. All other content in the containing block (including other floated elements) will flow around all the floated elements.

Clear Property:

You can stop elements from flowing around a floated element by using the clear property. Values for clear property are shown below:

Values	Description
Left	The left-hand edge of the element cannot be adjacent to another element.
Right	The right-hand edge of the element cannot be adjacent to another element.
Both	Both the left-hand and right-hand edges of the element cannot be adjacent to another element.
None	The element can be adjacent to other elements.

Containing Floats:

Another problem that can occur with floats is when an element is floated within a containing block that contains only floated content. In such a case, the containing block essentially disappears.

Lesson 47 : Floating elements (2)

Overlaying and Hiding Elements:

One of the more common design tasks with CSS is to place two elements on top of each other, or to selectively hide and display elements. Positioning is important to both of these tasks. Positioning is often used for smaller design changes, such as moving items relative to other elements within a container. In such a case, relative positioning is used to create the positioning context for a subsequent absolute positioning move.

There are in fact two different ways to hide elements in CSS: using the display property and using the visibility property. The **display property** takes an item out of the flow: it is as if the element no longer exists. The **visibility property** just hides the element, but the space for that element remains.

Lesson 48 : Multi-column layouts using floats

Constructing Multicolumn Layout (using Float):

A typical layout may very well use both positioning and floats. There is unfortunately no simple and easy way to create robust multicolumn page layouts.

Using Floats to Create Columns:

Using floats is perhaps the most common way to create columns of content. The first step is to float the content container that will be on the left-hand side. Remember that the floated container needs to have

a width specified. The background of the floated element stops when its content ends. If we wanted the background color to descend down to the footer, then it is difficult (but not impossible) to achieve this visual effect with floats.

Three- Column Example:

Another approach for creating a three-column layout is to float elements within a container element. This approach is actually a little less brittle because the floated elements within a container are independent of elements outside the container. The floated content must appear in the source before the nonfloated content. This is the main problem with the floated approach: that we can't necessarily put the source in an SEO-optimized order.

Using Positioning to Create Columns:

Positioning can also be used to create a multicolumn layout. This approach typically uses some type of container that establishes the positioning context. With positioning it is easier to construct our source document with content in a more SEO-friendly manner; in this case, the main can be placed first.

Problem with Absolute Positioning:

When an item is positioned, it is removed entirely from normal flow, so subsequent items will have no "knowledge" of the positioned item. One solution to this type of problem is to place the footer within the main container.

Lesson 49 : Multi-column layouts using flexbox

Constructing Multicolumn Layout using flexbox:

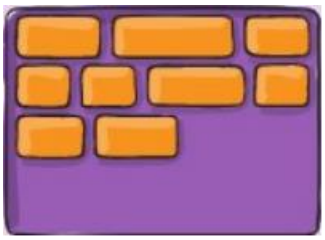
Flex layout is to give the container the ability to alter its items' width/height to best fill the available space.

Flexbox parent properties:

1. **flex-start**:

Items are packed toward the start of the flex-direction.

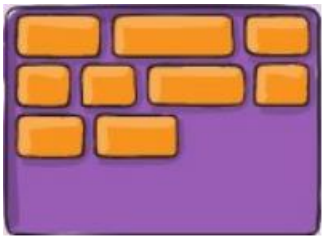
Example:



2. flex-end:

Items are packed toward the end of the flex-direction.

Example:



3. Center:

Items are centered along the line.

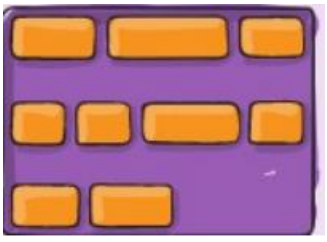
Example:



4. Space-between:

Items are evenly distributed in the line; first item is on the start line, last item on the end line.

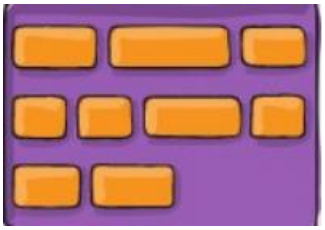
Example:



5. space-around:

Items are evenly distributed in the line with equal space around them.

Example:



Flexbox Child properties:

1. **Flex-grow:**

It tell us what amount of the available space inside the flex container the item should take up. By default we have equal space for flex grow and it set to 1, this means container will be distributed equally to all children.

Example:



Flexgrow set to be 1

If one of the children has a value of 2, that child would take up twice as much of the space either one of the others.

Example:



Flexgrow set to be 2

2. Flex-basis:

This defines how much space an element is taking initially.

3. Flex-shrink

This defines the ability for a flex item to shrink if necessary.

Lesson 50 : Approaches to CSS layout (Fixed and liquid)

Approaches to CSS Layout:

One of the main problems faced by web designers is that the size of the screen used to view the page can vary quite a bit. Users with the large monitor might expect a site to take advantage of the extra size; users with the small monitor will expect the site to scale to the smaller size and still be usable. Satisfying

both users can be difficult; the approach to take for one type of site content might not work as well with another site with different content.

Fixed Layout:

The first approach is to use a fixed layout. In a fixed layout, the basic width of the design is set by the designer, typically corresponding to an “ideal” width based on a “typical” monitor resolution. Fixed layouts are created using pixel units, typically with the entire content within a container (often named "container", "main", or "wrapper") whose width property has been set to some width. The advantage of a fixed layout is that it is easier to produce and generally has a predictable visual result.

Fixed layouts have other drawbacks. For larger screens, there may be an excessive amount of blank space to the left and/or right of the content. Much worse is when the browser window shrinks below the fixed width; the user will have to horizontally scroll to see all the content.

Liquid Layout:

The second approach to dealing with the problem of multiple screen sizes is to use a liquid layout (also called a fluid layout). In this approach, widths are not specified using pixels, but percentage values. The obvious advantage of a liquid layout is that it adapts to different browser sizes, so there is neither wasted white space nor any need for horizontal scrolling.

Week – 5

Lesson 51 : Introduction to responsive design

Responsive Design:

In a responsive design, the page “responds” to changes in the browser size that go beyond the width scaling of a liquid layout. One of the problems of a liquid layout is that images and horizontal navigation elements tend to take up a fixed size, and when the browser window shrinks to the size of a mobile browser, liquid layouts can become unusable. In a responsive layout, images will be scaled down and navigation elements will be replaced as the browser shrinks. There are four key components that make responsive design work.

1. Liquid layouts.

2. Scaling images to the viewport size.

3. Setting viewports via the <meta> tag.

4. Customizing the CSS for different viewports using media queries.

Lesson 52 : Viewports

Setting Viewports:

A key technique in creating responsive layouts makes use of the ability of current mobile browsers to shrink or grow the web page to fit the width of the screen. The way this works is the mobile browser renders the page on a canvas called the viewport. On iPhones, for instance, the viewport width is 980 px, and then that viewport is scaled to fit the current width of the device.

Lesson 53 : Media queries

Media Queries:

The other key component of responsive designs is CSS media queries. A media query is a way to apply style rules based on the medium that is displaying the file. You can use these queries to look at the capabilities of the device, and then define CSS rules to target that device. Unfortunately, media queries are not supported by Internet Explorer 8 and earlier.

Syntax of Media Queries:

1. **Width:** Width of the viewport.
2. **Height:** Height of the viewport.
3. **Device-width:** Width of the device.
4. **Device-height:** Height of the device.
5. **Orientation:** Whether the device is portrait or landscape.
6. **Color:** The number of bits per color.

Lesson 54 : Scaling images for liquid layouts

Scaling Images

Responsive designs begin with a liquid layout, that is, one in which most elements have their widths specified as percentages. We simply need to specify the following rule to make image scale in size:

```
Image {
```

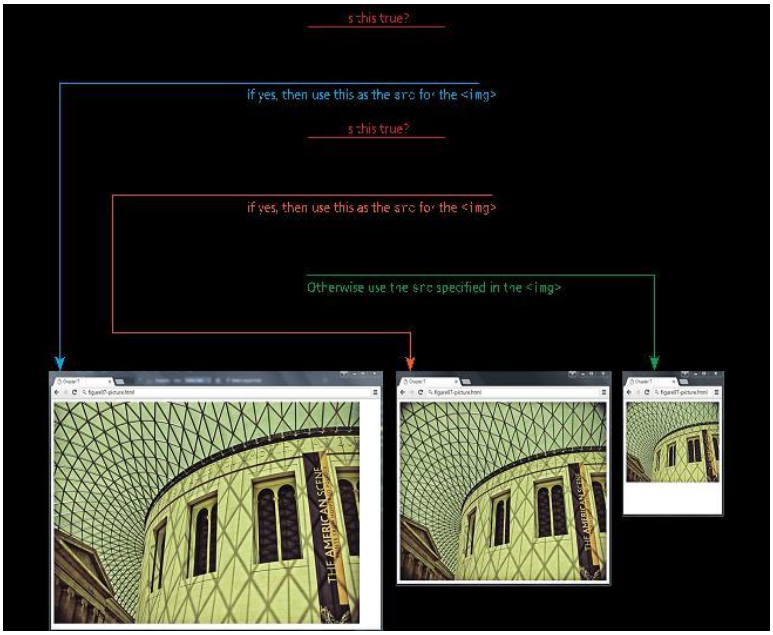
width: 100%;

max-width: 1024px;

}

<picture>

Example:



Lesson 55 : CSS Frameworks and Preprocessors

CSS Frameworks:

A CSS framework is a pre created set of CSS classes or other software tools that make it easier to use and work with CSS. They are two main types of CSS framework: grid systems and CSS preprocessors.

CSS Preprocessors:

CSS preprocessors are tools that allow the developer to write CSS that takes advantage of programming ideas such as variables, inheritance, calculations, and functions. A CSS preprocessor is a tool that takes code written in some type of preprocessed language and then converts that code into normal CSS. The advantage of a CSS preprocessor is that it can provide additional functionalities that are not available in CSS.

In a programming language, a developer can use variables, nesting, functions, or inheritance to handle duplication and avoid copy-and-pasting and search-and-replacing. CSS preprocessors such as LESS, SASS, and Stylus provide this type of functionality.

Lesson 56 : Installing Bootstrap

Installing Bootstrap

Bootstrap has two parts:

1. CSS
2. JavaScript

Ways to get Bootstrap:

There are three main ways to get the bootstrap:

1. Content Delivery Network
2. Web server/local Machines
3. Package Managers

Bootstrap Requirements:

Require certain tags on the web page

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
  </head>
</html>
```

Lesson 57 : Grid layout of bootstrap

Grid Layout:

Grid systems make it easier to create multicolumn layouts. There are many CSS grid Systems. Some of the most popular are Bootstrap, Blueprint, and 960 (960.gs). The most important of these capabilities is a grid system. Print designers typically use grids as a way to achieve visual uniformity in a design. CSS frameworks provide similar grid features. The 960 framework uses either a 12- or 16-column grid. Bootstrap uses a 12-column grid. Blueprint uses a 24-column grid. The grid is constructed using

elements with classes defined by the framework. In Bootstrap, content must be placed within the row container.

Lesson 58 : Typography in bootstrap

Typography in Bootstrap:

Typography is a feature of Bootstrap for styling and formatting the text content. It is used to create customized headings, inline subheadings, lists, paragraphs, aligning, adding more design-oriented font styles and much more. By default, Bootstrap will style the HTML headings (<h1> to <h6>).

Text Utilities:

1. **Text alignment**

Easily realign text to components with text alignment classes.

2. **Text wrapping**

Prevent text from wrapping with a .text-nowrap class.

3. **Text transform**

Transform text in components with text capitalization classes.

a. lowercased text.

b. UPPERCASED TEXT.

c. CapiTaliZed Text.

Lesson 59 : Colors in bootstrap

Colors in Bootstrap:

Bootstrap provides us the color utility due to this we can assign colors to text and the background. Bootstrap supports many theme-wise colors. Colors can be used:

v. Text

Example: .text-light

v. Background

Example: .bg-primary

v. Link

Example: Danger link

v. Alerts

Colors for themes can be changes using Saas CSS pre-processor.

Theme- based colors:

8 theme-based colors are mentioned below:

Text colors	Background colors
Primary	.bg-primary
Secondary	.bg-secondary
Success	.bg-success
Info	.bg-info
Warning	.bg-warning
Danger	.bg-danger
Light	.bg-light
dark	.bg-dark

Lesson 60 : Navigation in bootstrap

Navigations in Bootstrap

Nav is one of the components available in bootstrap and it's used for creating responsive navigation bars. Navigation is implemented using:

Unordered list:

Example: < ul class = “nav”>

List items:

Example: <li class = “nav-item”>

Links:

Example: < a class =”nav-link”….>

Dropdown using nested unordered list:

Parent li:

Example: <li class = “nav-item dropdown”>

Parent link:

Example: < a class = “nav-link dropdown-toggle” data-bs-toggle = “dropdown” href = “#”> Dropdown

Nested ul:

Example: <ul class = “dropdown-menu”>

List item:

Example: <a class = dropdown-item”

Lesson 61 : Introduction to Javascript

Introduction to JavaScript: JavaScript's History:

JavaScript was introduced by Netscape in their Navigator browser back in 1996. JavaScript that is supported by your browser contains language features:

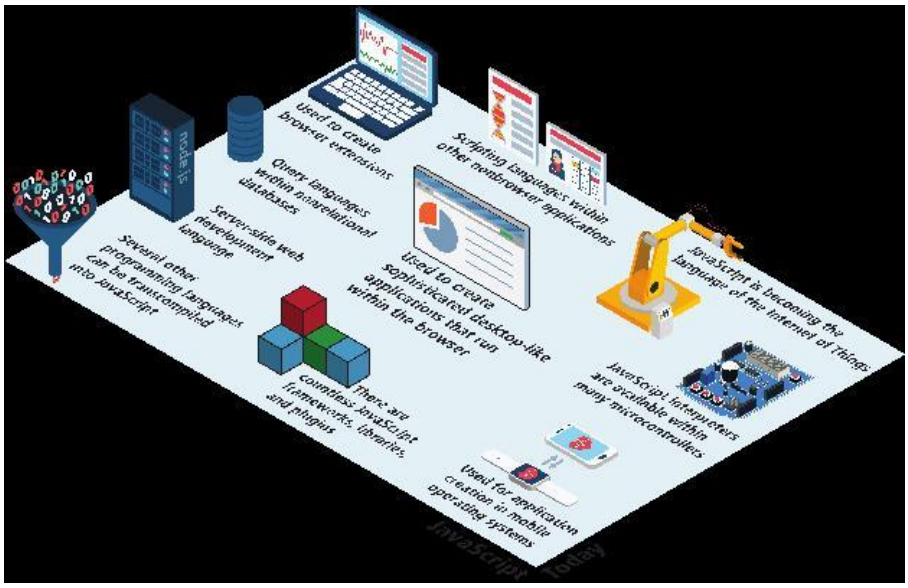
1. not included in the current ECMAScript specification and
2. missing certain language features from that specification

Commonly used version of ECMAScript is the Sixth Edition (ES6). Whereas, 12th edition (ECMAScript 2021) released in 2021.

JavaScript and Web 2.0:

Early JavaScript had only a few common uses. 2000s onward saw more sophisticated uses for JavaScript. **AJAX** as both an acronym and a general term

JavaScript in Contemporary Software Development:



Lesson 62 : Writing and storing the script

Inline JavaScript:

Inline JavaScript refers to the practice of including JavaScript code directly within certain HTML attributes.

```
<a href="JavaScript:OpenWindow();"more info</a>
```

```
<input type="button” onclick="alert('Are you sure?');" />
```

Embedded JavaScript:

Embedded JavaScript refers to the practice of placing JavaScript code within a <script> element.

```
<script type="text/javascript">
/* A JavaScript Comment */
alert ("Hello World!");
</script>
```

External JavaScript:

External files typically contain function definitions, data definitions, and other blocks of JavaScript code. JavaScript external files have the extension .js.

```
<head>
<script type="text/JavaScript" src="greeting.js">
</script>
</head>
```

Lesson 63 : Variables and data types in Javascript

Variables in JavaScript:

There are two types of variables in programming language i-e dynamic and static. In c, c++ and java variables are statically typed. Some programming languages decided the type of variable on runtime and java script is one of them, in python also variables are dynamically typed. This simplifies variable declarations, so that we do not require the familiar type fields like int, char, and String. Instead, to declare a variable x, we use the var keyword, the name, and a semicolon.

Data Types in JavaScript:

There are two types of data types in java script i-e reference data type and primitive data type.
In **reference data types** we have objects and the address of objects will be stored in the variable.
Whereas, in **primitive data types** exact value of variable will be stored e.g Boolean, number variable and string e.t.c. So in primitive data types values will be stored and in reference address will be stored.

Lesson 64 : Javascript output

JavaScript Output:

It's very important in every programming language to view the output. In JavaScript we have three mechanisms to view the output:

1. Alert
2. Console. Log()
3. Document. Write ().

i. Alert:

The alert() function makes the browser show a pop-up to the user, with whatever is passed being the message displayed. The following JavaScript code displays a simple hello world message in a pop-up:

alert ("Good Morning");

The pop-up may appear different to each user depending on their browser configuration. During debugging and testing we may use alert function.

ii. Document. Write:

This method writes a string of text to a document stream opened by document. By using the document. Write whatever we will write it will be the part of the HTML page.

iii. Console log:

It is very good way to debug your application. We can open a console in web browser by using: Ctrl + Shift + I for windows.

Lesson 65 : Conditions in Javascript

Conditionals:

In java script conditions are identical like c++. There are three types of conditional statements:

1. If else if
2. Switch
3. Conditional assignment.

If else if:

Conditional statements are simple in which we have to write logical expressions. If logical expression will be true then one block of expression will be executed if false then it will not be executed.

Example:

```
var hourOfDay; // var to hold hour of day

var greeting; // var to hold the greeting message.

if (hourOfDay > 4 && hourOfDay < 12){

// if statement with condition

greeting = "Good Morning";

}

else if (hourOfDay >= 12 && hourOfDay < 20){

// optional else if

greeting = "Good Afternoon";

}

else{ // optional else branch

greeting = "Good Evening";

}
```

Switch Statements:

The switch statement evaluates an expression, matching the expression's value to a case clause, and executes statements associated with that case, as well as statements in cases that follow the matching case.

Example:

```
const expr = 'Papayas';

switch (expr) {

  case 'Oranges':
```

```
    console.log('Oranges are $0.59 a pound.');
    break;
  case 'Mangoes':
  case 'Papayas':
    console.log('Mangoes and papayas are $2.79 a pound.');
    // expected output: "Mangoes and papayas are $2.79 a pound."
    break;
  default:
    console.log(`Sorry, we are out of ${expr}.`);
}
```

Conditional assignment:

The **conditional (ternary) operator** is the only JavaScript operator that takes three operands: a condition followed by a question mark (?), then an expression to execute if the condition is truthy followed by a colon (:), and finally the expression to execute if the condition is falsy. This operator is frequently used as an alternative to an if...else statement.

Example:

```
/* x conditional assignment */
x = (y==4)                ? "y is 4"                : "y is not 4";
Condition                 Value if true             Value if false
```

Lesson 66 : Arrays in Javascript

Arrays:

Arrays are one of the most used data structures, and they have been included in JavaScript as well. There are two main ways to define an array in java script.

1. Object literal notation
2. Array constructor

Object Literal Notation:

It's a simple form to creating arrays. The values we want to store in arrays we will write in square bracket and separate them with the help of commas.

Example:

```
var greetings = ["Good Morning", "Good Afternoon"];
```

Array Constructor:

Array constructor is the second way for the creation of array. In array constructor “new” keyword has been used and in parenthesis we will write commas separated values.

Example:

```
var greetings = new Array("Good Morning", "Good Afternoon");
```

Common Features of Arrays in Java Script:

1. Index will be start from ZERO.
2. [] is used to access particular element.
3. . length has been used to give the length of the array.
4. .push(): add a new element/item to an array.
5. .pop() : remove the last element from the array.
6. Concat: method is used to concatenate two or more arrays.

Lesson 67 : Loops in Javascript

Loops:

Loops are used for repetitive execution of the statements. Following types of loops are mentioned below:

1. While loop.
2. Do while loop
3. For loop
4. For each

While Loop:

The most basic loop is the while loop, which loops until the condition is not met.

Example:

```
var i=0;
while(i < 10){
//do something with i
i++;
}
```

Do While Loop:

The do...while **statement** creates a loop that executes a specified statement until the test condition evaluates to false. The condition is evaluated after executing the statement, resulting in the specified statement executing at least once.

Example:

```
count=0;

do

{

//do something

//

Count++;

}

While (count<10);
```

For Loop:

A for loop combines the common components of a loop: initialization, condition, and post-loop operation into one statement. This statement begins with the for keyword and has the components placed between ( ) brackets, semicolon (;) separated.

Example:

```
for (var i = 0; i < 10; i++){

//do something with i

}
```

For each Loop:

This loop is used with an array in java script.

Example:

```
const fruits = ["apple", "orange", "cherry"];
fruits.forEach(myFunction);
```

Lesson 68 : Objects in Javascript

Objects in JavaScript:

To create an objects in JavaScript we have multiple ways. One way is to implement by using object literal notation. Object literal notation got very popular and now a days used in many web development. Now a days it's become very popular file format specifically to transfer the data from one place to another.

Example:

```
Var objName= {
Name1: value 1,
Name 2: value 2,
//…
nameN: valueN
};
```

Lesson 69 : Functions in Javascript

Functions:

Functions in JavaScript are one of the most fundamental building blocks. Wherever we want to define a function we will use the keyword of the function followed by the name of the function.

Example:

```
function square(number) {
  return number * number;
}
```

Anonymous Functions:

JavaScript allows/support the anonymous functions. Anonymous function must be assigned to a variable.

Example:

```
var showMessage = function (){
   alert("Hello World!");
};
showMessage();
var sayHello = function (firstName) {
   alert("Hello " + firstName);
};
showMessage();
sayHello("Bill");
```

Arrow Functions:

An **arrow function expression** is a compact alternative to a traditional function expression, arrow functions have a shorter syntax. We can also convert normal functions to arrow functions. Arrow functions are more useful for anomalous and call functions.

Example:

```
/ Traditional Anonymous Function
function (a){
  return a + 100;
}
// Arrow Function Break Down
// 1. Remove the word "function" and place arrow between the argument and opening body bracket
(a) => {
  return a + 100;
}
```

Lesson 70 : Functions – Nested

Nested Functions:

In JavaScript, we can define functions inside functions and call it nested functions. Nested functions are in scope of outer function. The inner function can access variables and parameters of outer function, but out function cannot access variables defined inside the inner functions.

Example:

```
function ShowMessage(firstName)
{
    function SayHello() {
        alert("Hello " + firstName);
    }
    return SayHello();
}
ShowMessage("Steve");
```

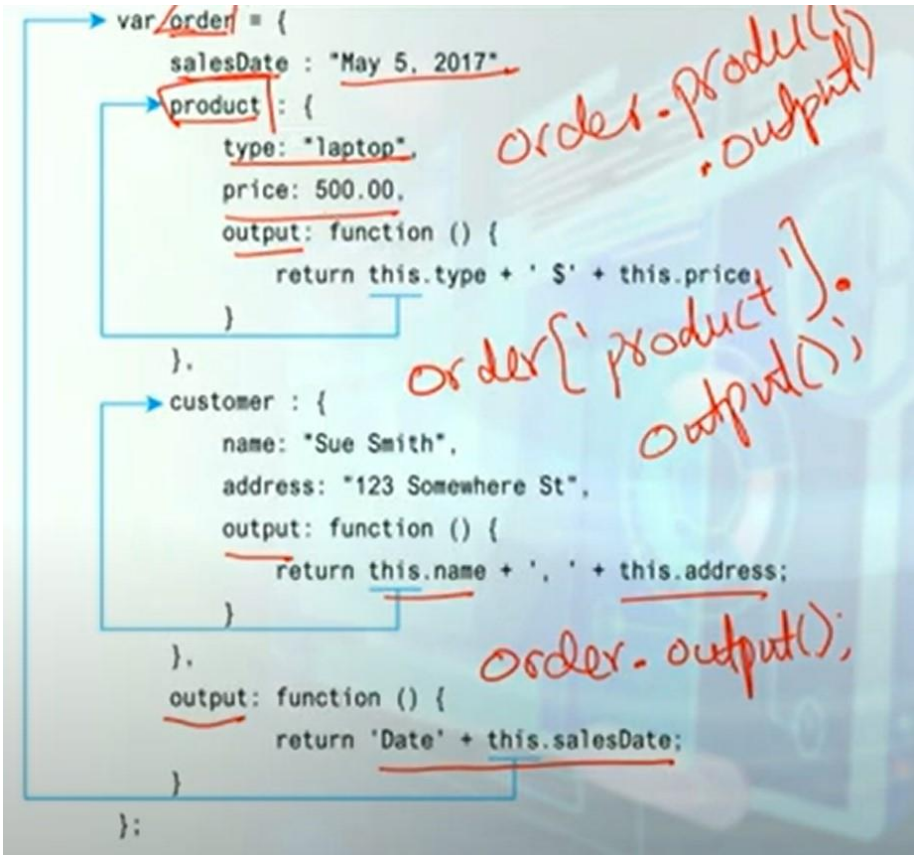
Lesson 71 : Functions – Callback

Callback Functions:

Java script supports the call back functions. In JavaScript, you can pass a function as an argument to a function the function that is passed as an argument inside of another function is called callback function.

Lesson 72 : Functions – Objects

Functions-Object:



Lesson 73 : Functions – Scope

<u>Function scope:</u>

Let us understand how the scope works in JavaScript particularly in the context of function. In JavaScript the scope can be considered as a box. The biggest scope would be called as global cope, implying that anything put inside this area (Global Scope) can be accessible from any program's location. Anything declared/put outside of the function body is recognized as global.

Function can be considered as a directed window, if a function has a nested function of functions then the parent/main function can't access the variables inside the nested function whereas the nested function can access the variables of parent/main function, this property is known as a one-way window.. As shown in below figure no.73.1.

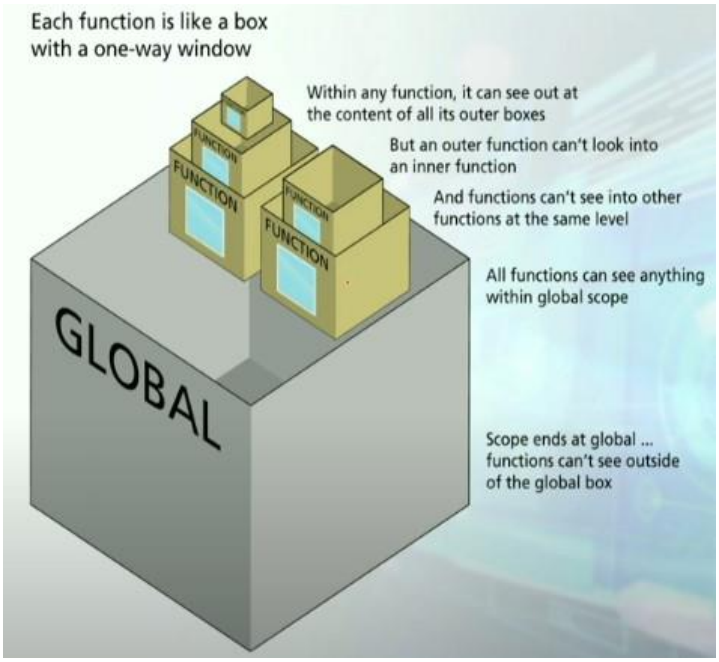


Figure 73.1.

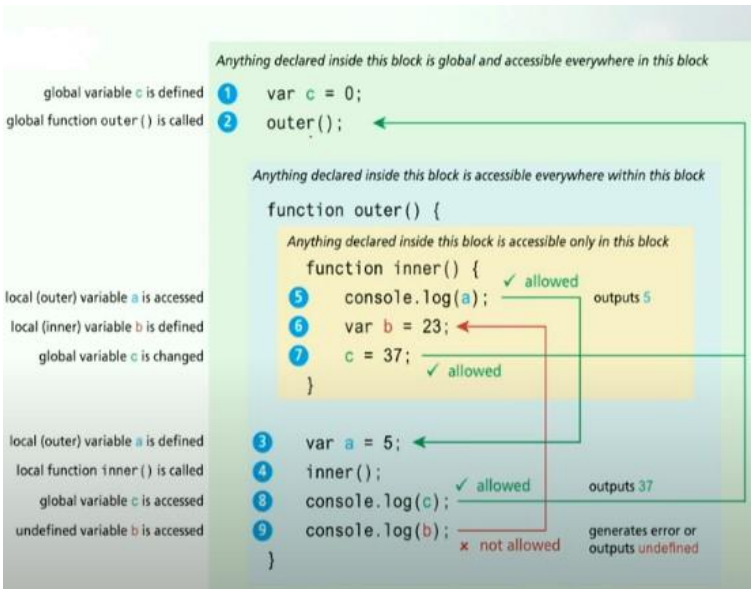
Figure no. 73.1. Of Fundamentals of Web Development-Pearson Education (2017)

The above figure no.1 has been explained in below examples in detail.

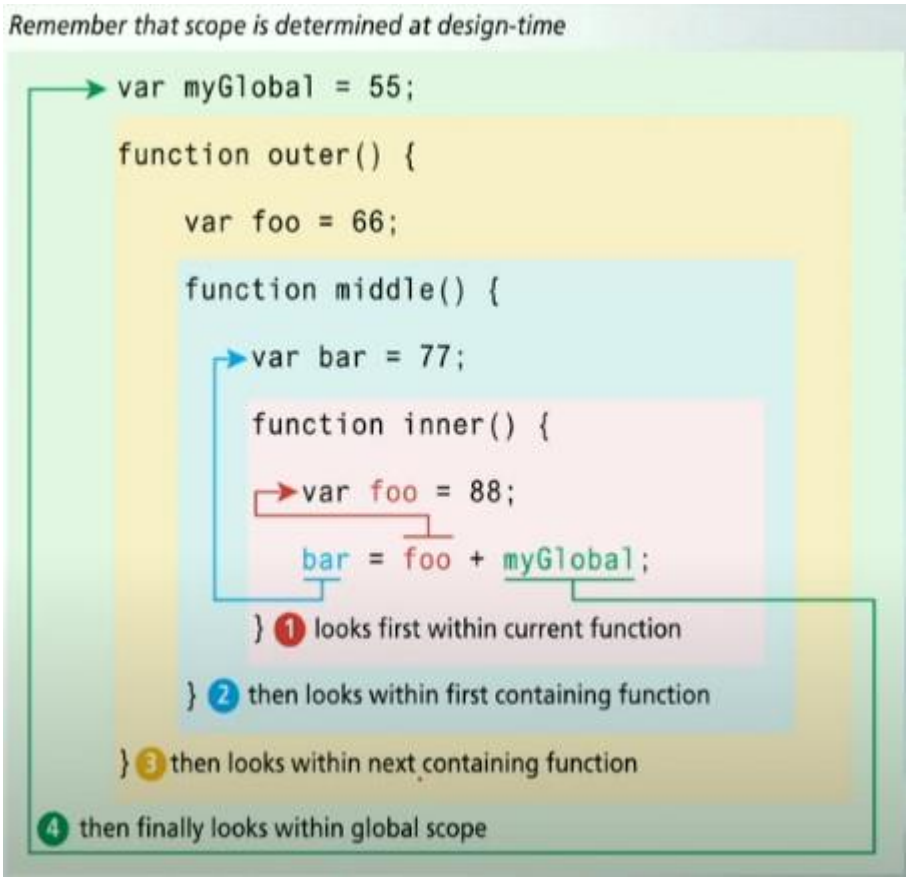
In the given example, the variable c is defined in the global scope and it is not part of a function.

Two functions namely outer () and inner () have been defined with its own variables. **a** is the member variable of outer () function whereas **b** is the member variable of inner () function as shown in below figure. The inner () function can access variable **a** like console.log (a) "it is allowed "as it is defined in outer function whereas the outer () function can't access variable **b** like consol.log (b) "it is not allowed" as it has defined in inner () function.

The same concept has been explained in example no.2.



Example no. 1 of function scope from Fundamentals of Web Development-Pearson Education (2017)



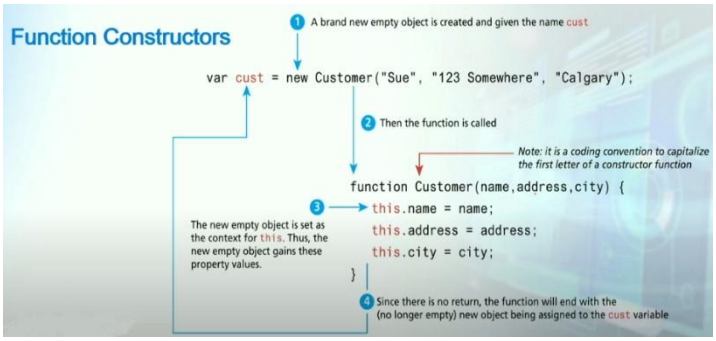
Example no. 2 of function scope from Fundamentals of Web Development-Pearson Education (2017).

Furthermore function can be used as a constructor for defining objects. Object oriented programming was used in the early versions of JavaScript before ES5 and ES6 via functions.

The figure given below shows example of how function is used as a constructor for defining objects.

Let's have a look at one example.

A function with the name customer has been defined in below example on the basis of this function objects can be created. The variables defined in function might be taken as parameter inside the function, or besides this we can also take additional parameters in the function as well. In the given example the customer function has three variables, **name, address and city**, these variables are taken as parameter and it is used as objects. The variables inside the function which defines its (function) properties, let's think of it as an object, so the key word "**this**" will be used before the variable of the function like. "**This.name**" as shown in the example below. Now "**this.name**" is totally different form only the **name** variable that is received as parameter and so on. And it is compulsory to use the key word **this** to differentiate it from the only name variable.



Example no.1 of function constructor. From Fundamentals of Web Development-Pearson Education (2017)

Lesson 74 : Exception handling using Try and Catch

Exception handling using Try and Catch.

When we write program, we occasionally encounter unexpected behavior, we called it exception. All programing languages provide ways to deal with exceptions via exception handling.

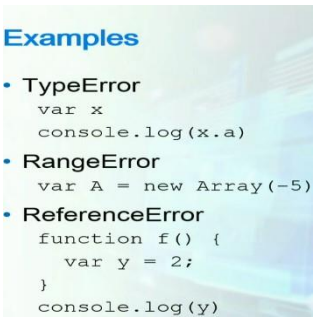
Let's have a look that how java script supports exception handling.

- Errors occur at run time are called exceptions.
- Might result an unexpected behavior if not handled.
- Handling exceptions avoid unexpected termination.

Some errors are syntax errors which we can easily identify while we are coding, but few errors may execute at run time, implying that it is not mandatory that it will be always executed at run time. May be 99% the logic is fine and no error is executed at all but it might happen that just 1% or even 0.1% of the time an unexpected scenario occurs which results an error.

If the exceptions are not properly handled the logic can terminate abruptly. To avoid this problem, the exception or exceptions that may arise will be handled properly.

Few examples of exceptions are given below.



The above exceptions are built-in exception and are predefined in JavaScript.

1. **TypeError:**

There can be multiple scenarios which might generate **TypeError** exception.

Have a look at the example

var x // neither property assigned to **x** nor it is an object. But **x** is a simple variable.

console.log (x.a) // **a** is not the property of x. Here in this line of code it is attempted to access an undefined property of **x**, namely **a**.

As a result, an exception will be generated and thrown, which will be visible/highlighted in red on the console.

2. **RangeError:**

var A = new Array (-5)

When an array is declared, it must be initialized with a positive integer, which is referred to as the array's size. When the size parameter is a –ve integer, the result is an unexpected value/range, resulting in a range exception.

3. **RefrenceError:**

Let's have an example of the RefrenceError

function f () {

var y =2; // local variable inside the function.

}

console.log(y) // access the local variable of function from outside.

This line of code will cause a reference exception and throw it. The local variable is accessed from outside the function in this line of code.

Exception handling:

How exceptions are handled?

- Exceptions are handled using **try/catch**.

The kind of remedy of an exception is defined here.

The statements expected to generate an exception or exceptions are put in try block.

- Statements expected to result in an exception are put in try block.
- Exception handling is done in catch block.

Syntax of try block

try { // Use the key word **try** and define it's block using curly **"{}"** braces.

var A = new Array (-5); // Here it will throw an exception of range error. How to handle it?

} catch (e) { // Here a catch block is defined with curly **"{}"**braces which takes a parameter namely **e,** where **e is** an exception

console.log (e); // this line of code which is written in the catch block will throw the **c**orresponding exception/error onto console.

}

It can also be used in many other practical and professional environments as well, like connection doesn't establish, try to read/write but can't read/write a file located on server and so on.

Throwing exceptions:

One can also throw exception. Like,

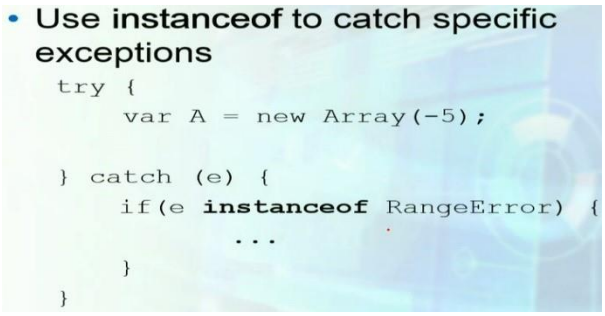
- Exception can be generated (thrown) by the programmer using throw.

If we have a scenario in which no built-in exception exists in JavaScript, We'll generate an exception for ourselves.

```
try {
    …
    if (age < 0 || age > 150) {
      throw("InvalidAge: Valid age
is between 0 and 150");
    }
} catch (e) {
    console.log(e);
}
```

Consider the above example: we take the user's age using a form, but the value must be between 0 and 150; if the user inputs a value greater than 150 or less than 0, an exception of "InvalidAge" will be thrown.

Catching Specific Exception:

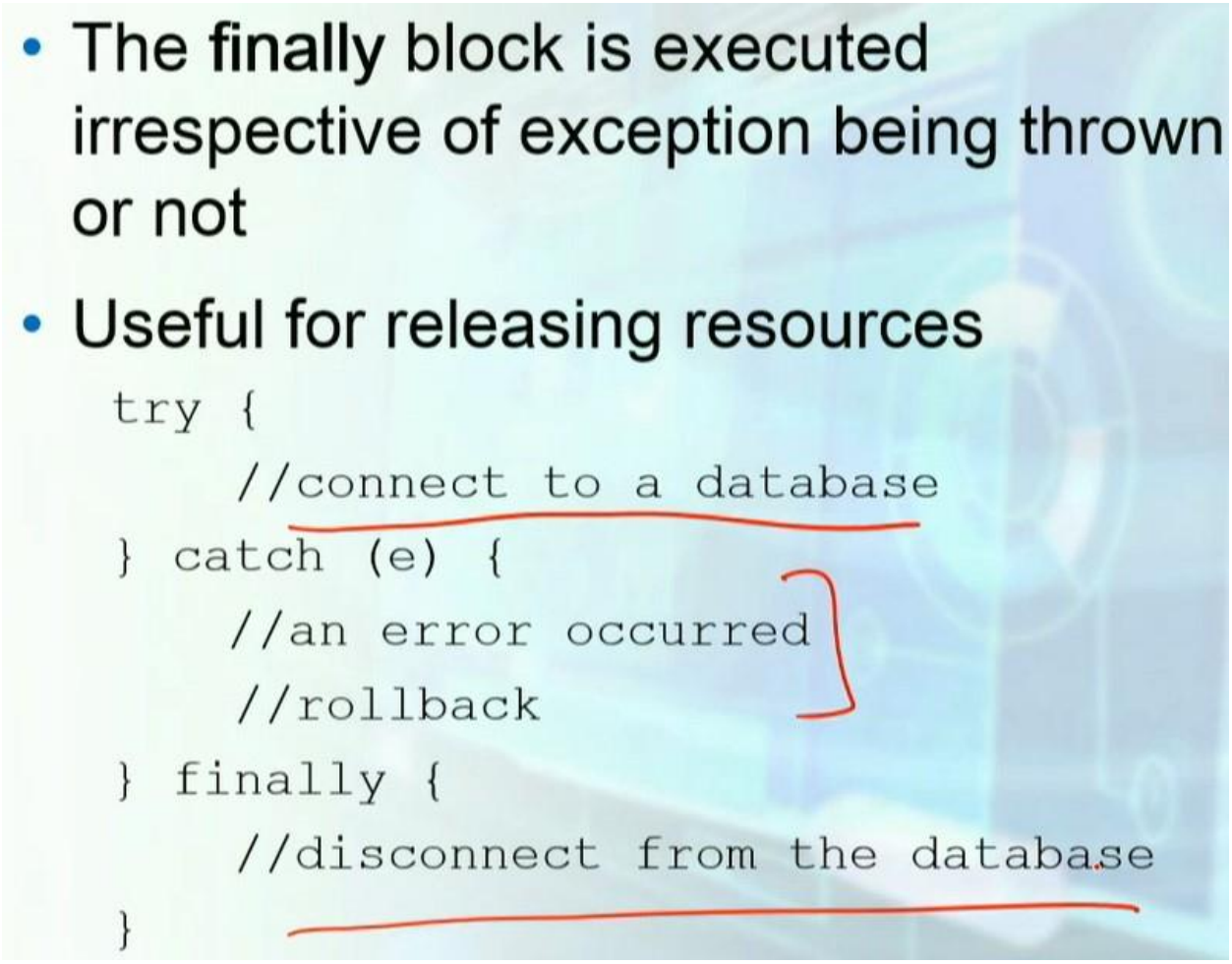


Let suppose we have a kind of code which might throw multiple types of exceptions/errors, and you want to handle them specifically based on the type of the exception.

In the above example if we do expect that in **try** block multiple types of exception are expected.

Then in the catch block the key word instanceof is used before the type of error as used in the given example, and through this one specifically know what type of an error has generated, and likewise multiple if/switch statements in the catch block can be used to cover all the possible exceptions.

The finally block:



The finally block demonstrates that regardless of whether a try block is executed or not, the finally block must be executed.

Lesson 75 : Javascript built-in Objects (Date, Math, Navigator, String, and etc.)

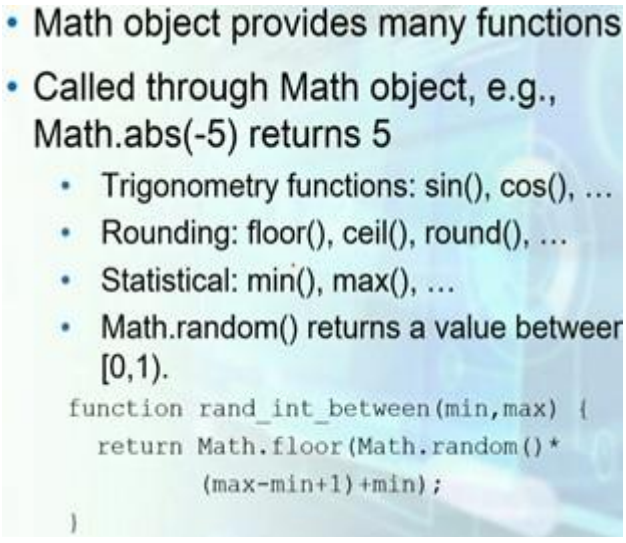
Introduction.

JavaScript has some different built-in objects by which different activities can be work out. The primary goal of JavaScript's built-in objects is to make work easier. Let's have a look at few built-in JavaScript objects.

- Built-in objects facilitate in programing.
- JavaScript provides many built-in –in objects.
    - Math, Number, BigInt, Date // BigInt is used for large number.
    - String, RegExp // Regular expression used for pattern matching.
    - Array
    - JSON // Used for working in JSON format.
    - Intl

There are many more built-in objects as well. Let see a few examples that how these built-in objects can be sued.

Working with math:



Math.abs // abs stands for absolute. // takes an integer as an input and return a +ve number if for example the number is –ve.

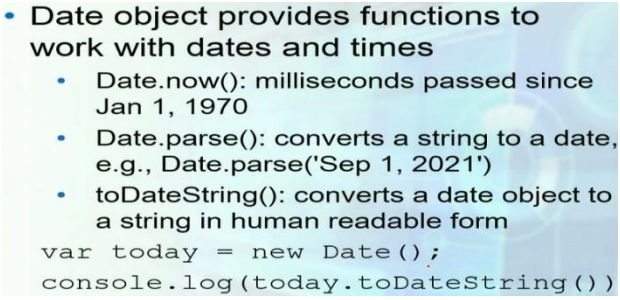
Rounding: This is used for rounding the number for example 2.3. The ceil () would 3, the floor () and round () would be equal to 2.

Statistical: min (), max (). Provides an array to the functions, to return the minimum or maximum value of an array.

Math.random // generate random number from 0 to 9. And the probability to generate a number would be same.

Let suppose if someone wants to use **math.random** to define a function to return a value/integer in some specific range, for example 5 to 20. For achieving this goal we will make use of this (**math.random**) function to define our own function, as given above.

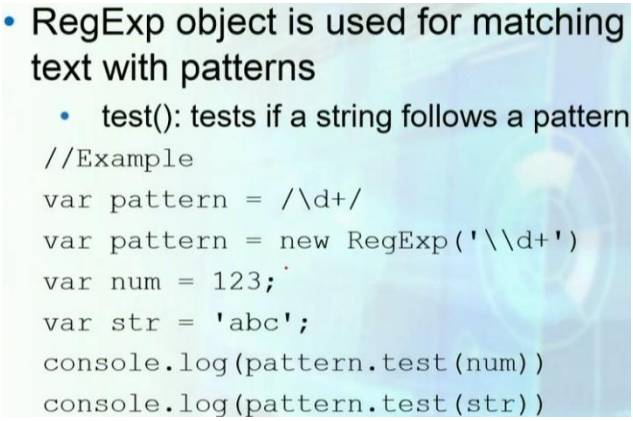
Working with Dates:



Date. Now () // Returns system time in milliseconds which is from 1970 till date and it would be a kind of number, which is not in human readable form. This can be converted to human readable form by calling **toDateString ().**

Date.parse () // if the date is in string form like (Sep 1, 2021), then Date.parse () function is used to convert it to date. However, keep in mind that the implementation of the Date.parse () function varies by browser, so keep that in mind.

Working with Pattern:



/\ **d+/** // slash \d represents digits which can be from 0 to 9 and plus (+) represents one or more reference. So what kind of number can be matched using this **"/\d+/**" pattern>? For example if we have 1, 23, 45, 343… so all these include 1 or more than one digits therefore this (**"/\d+/**) pattern can be used for matching such numbers.

New RegExp ('\\d+') // this is another way of defining pattern and the above given pattern is simply pass on to it. But you must be careful with this ('\\d+') particular format.

If you are using forward slashes "**/\ d+/**" here then whatever pattern is written inside it would be treated as regular expression.

var num = '123';

var = 'abc';

console.log (pattern. test (num)) // when **num** is pass on it returns **true** "1"as it matches the pattern.

console.log (pattern. test (str)) // when **str** is pass on it returns false "0" as it has no digit but only characters. So it would be a false expression.

Lesson 76 : Object prototypes; Prototypes for extending objects

Object prototypes; Prototypes for extending objects

We learned a better way to create many instances of objects with the same properties and methods in the previous section. While the constructor function is simple to use, it can be an inefficient approach for objects that contain methods. Take, for example, the function constructor in Listing 8.18. It allows you to make a single dice object.

Sample inefficient function constructor and some instances

```
function Die(col) {

this.color=col;

this.faces=[1,2,3,4,5,6];

this.randomRoll = function() {

var randNum = Math.floor((Math.random() * this.faces.length) + 1);

return faces[randNum-1];

 };

}

// now create a whole bunch of Die objects and start rolling 'em!

var x1 = new Die("red");

alert(x1.randomRoll());

var x2 = new Die("green");

alert(x2.randomRoll());  // …

var x100 = new Die("blue");
```

Although the function constructor used above works, it is not a memory-efficient approach. Why? Because a new randomRoll() function (object) is created for each new Die object instance. Figure 76.1. illustrates how multiple instances of the Die object contain multiple (identical) definitions of the

randomRoll() method (recall that a function expression is an object whose content is the definition of the function).

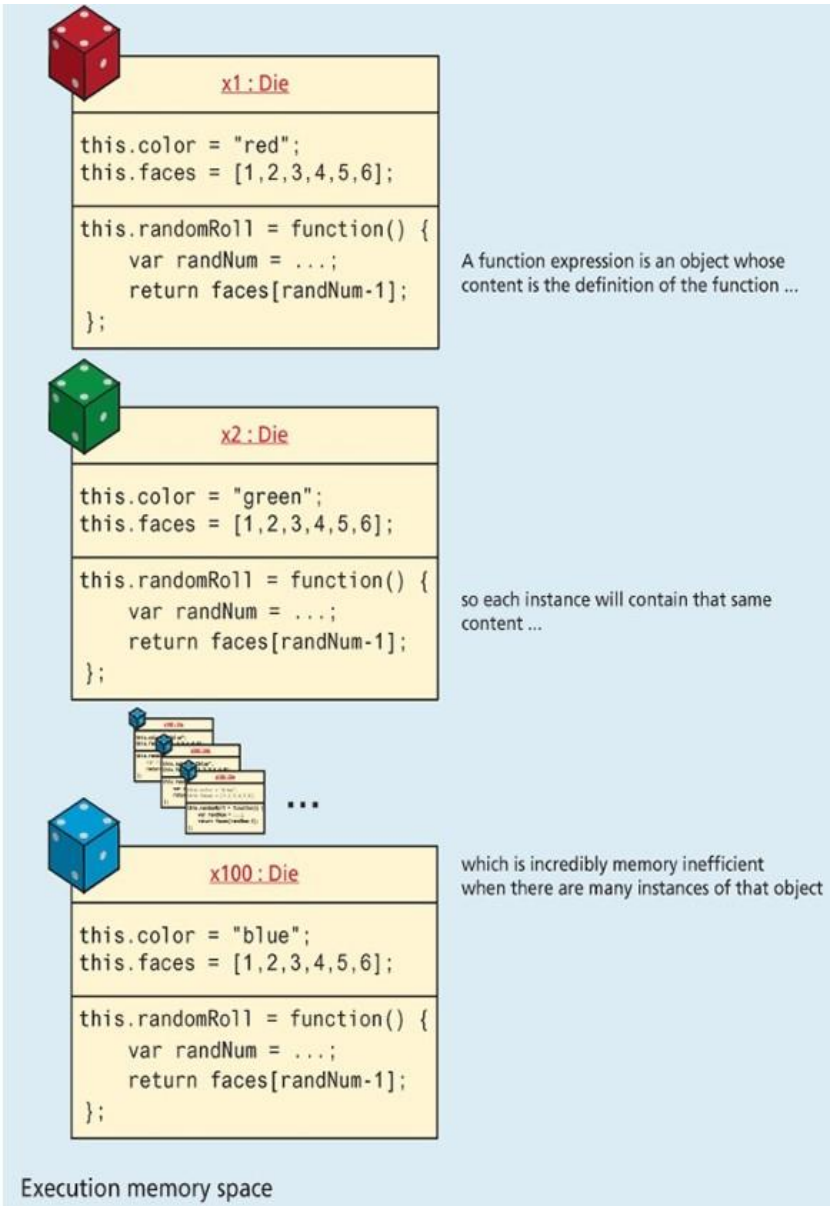


Figure 76.1.

Consider the challenge of creating 1000 or 100,000 Die things. You'd be redefining each method 1000 times or more, which could have an impact on client execution times and browser performance. A preferable technique is to define the method only once, using a prototype of the function, to avoid this unnecessary waste of memory.

Using Prototypes

Prototypes are an important grammar feature of JavaScript that allows it to behave more like an object-oriented language. A prototype attribute is given (inherited) by every function object, which is initially an empty object.

What makes the prototype property powerful is the prototype properties and methods are defined once for all instances of an object created with the new keyword from a constructor function.

So now in our example, we can move the definition of the randomRoll() method out of the constructor function and into the prototype, as shown in

Using Prototypes

```
function Die(col) {

this.color=col;

this.faces=[1,2,3,4,5,6];

}

Die.prototype.randomRoll = function() {

    var randNum = Math.floor((Math.random() * this.faces.length) + 1);

return faces[randNum-1];

};

// now create a whole bunch of Die objects

var x1 = new Die("red");

alert(x1.randomRoll());

var x2 = new Die("green");

alert(x2.randomRoll());

…
```

This way is superior since it just defines the function once, regardless of how many Die instances are created. In contrast to the duplicated code in figure 76.1. Figure 76.2. illustrates how using a prototype boosts productivity.

Figure 76.2. shows how the prototype property is updated to contain the method so that subsequent instantiations reference that one method definition. Since all instances of a Die share the same prototype object, the function declaration only happens one time and is shared with all Die instances.

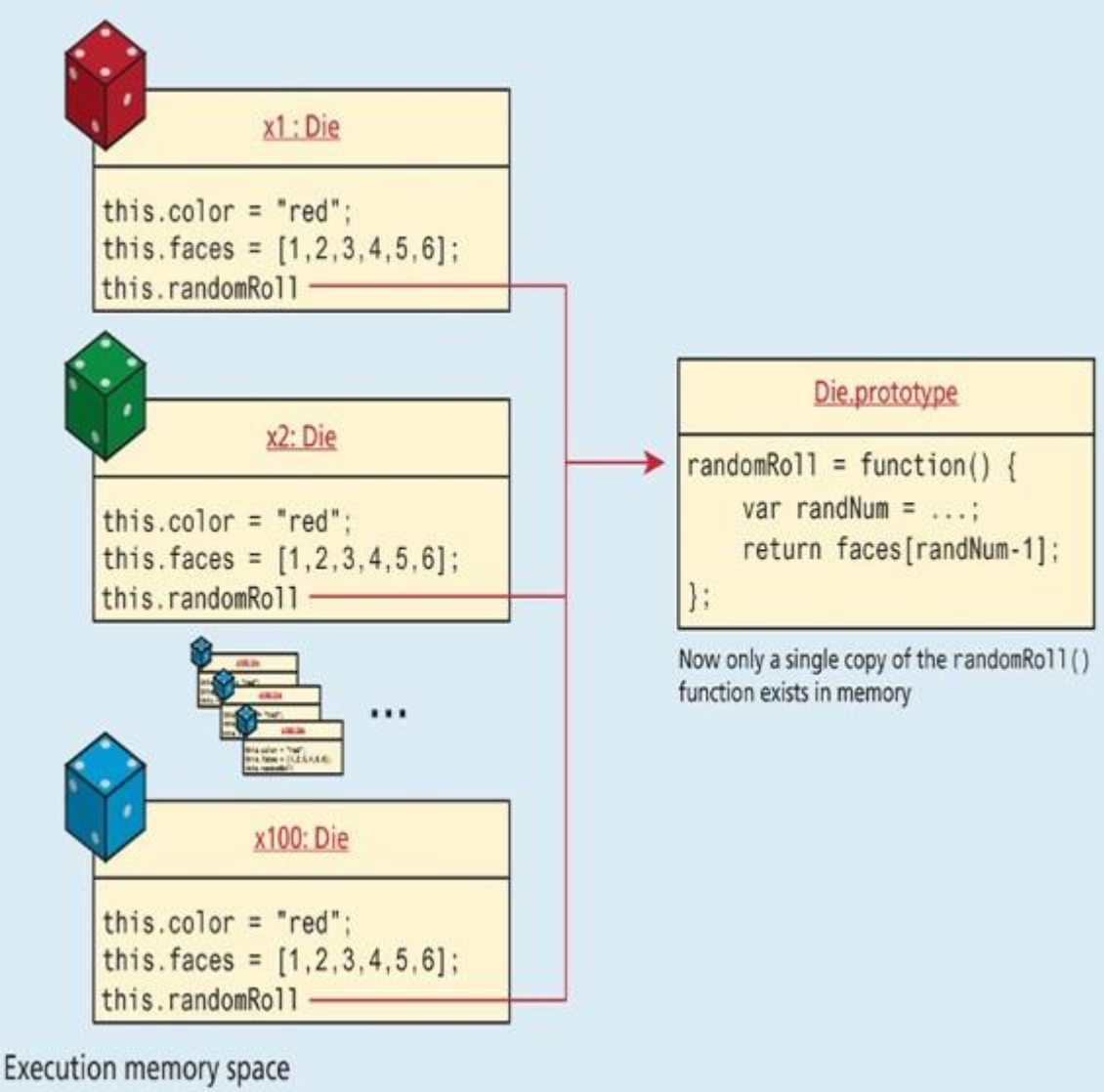


Figure 76.2.

Using Prototypes to Extend Other Objects

In addition to the obvious application of prototypes to our own constructor functions, Prototypes allow you to extend existing objects (including built-in objects) by adding to their prototypes. Imagine a method added to the String object which allows you to count instances of a character. Below example defines just such a method, named countChars() that takes a character as a parameter.

```
String.prototype.countChars = function (c) {
 var count=0;
for (var i=0;i<this.length;i++) {
      if (this.charAt(i) == c)
           count++;
```

}

return count;

}

This method will now be available to any new String objects. The new method could be used on any strings created after the prototype definition was added. For instance the following example will output Hello World has 3 letter l's.

var msg = “Hello World”;

console.log(msg + “has” + msg.countChars(“l”) + “ letter l's”);

Lesson 77 : Introduction to the Document Object Model (DOM)

Introduction to the Document Object Model (DOM):

Let's take a look at the Document Object Model (DOM) and how JavaScript can access it.

Introduction:

- Any HTML document can be represented in form of tree called Document Object Model.
- Available as data structure in JavaScript. When an HTML document is represented as a Tree, the Tree is now a data structure that JavaScript can access. And once it's accessible in JavaScript, it'll be feasible to manipulate it.
- Helps in accessing and manipulating HTML elements through JavaScript.

Overview:

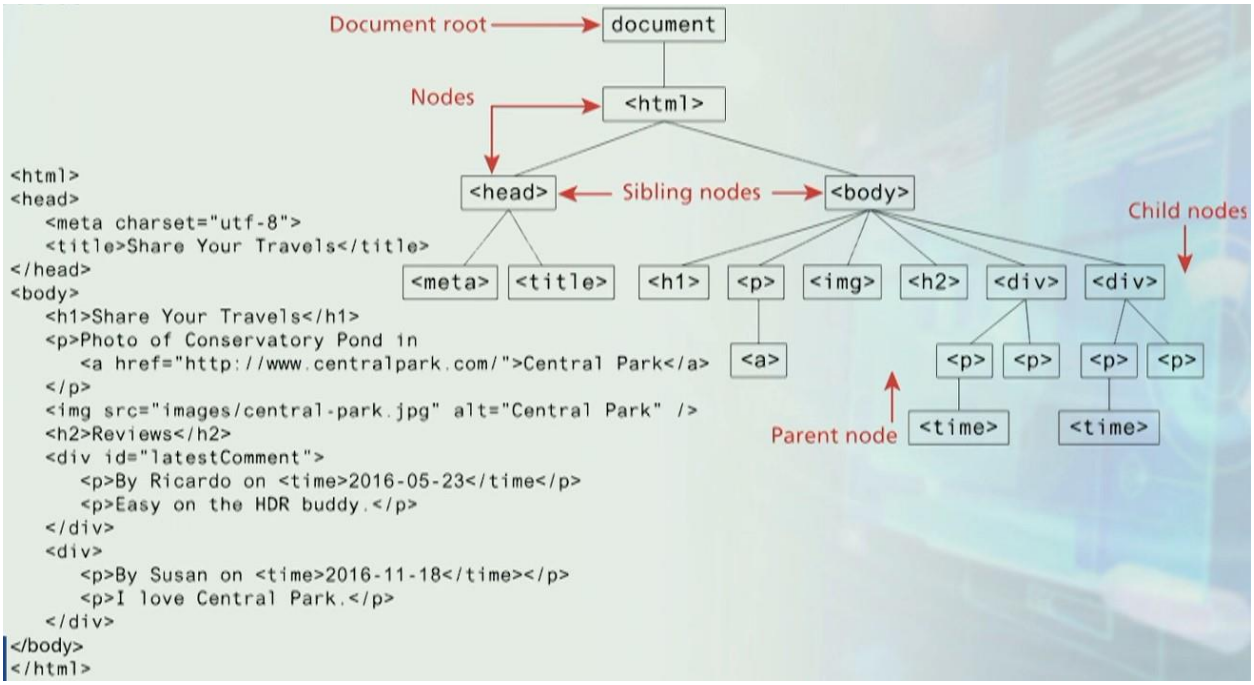


Figure 1, from Fundamentals of Web Development-Pearson Education (2017).

This is an **HTML** document, and **Document** is the primary variable used to access a **DOM** tree via **JavaScript**. **Document** represents the, document object Model (**DOM**) object for the HTML page on which we are working. Besides the "**Document**" in **DOM** tree each element will become node. For example.

<html>, <head>, <body>, <meta>, <title>, <h1>….etc.

Sibling nodes: Sibling nodes are nodes that exist at the same hierarchical level.

Childe and parent node: A child node is any subnode of a parent node, and the parent node is the child's parent.

Note: In the above figure 1 the tree represents the HTML code and vice versa.

Nodes:

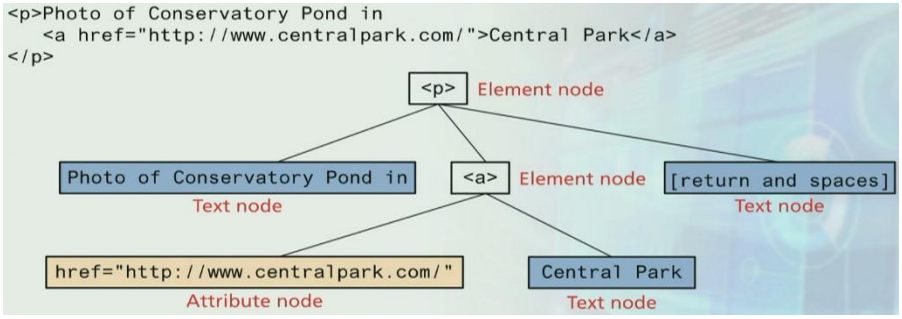


Figure 2, from Fundamentals of Web Development-Pearson Education (2017).

When a tree is built in memory, everything in it becomes a node, weather it is an element, text, spaces or new line character, as seen in figure 2. Consider the paragraph tag<p>, which consists of an anchor tag,

content, and a new line, followed by a closed <p>tag. How will it be transformed into a DOM tree? Figure 2 depicts this quite clearly.

DOM Related Data Types:

- **Document**
    - The root document// represents the whole document.
- **Node**
- Every object in the HTML document is a node (elements, attributes, text, etc).
- **NodeList //** collection of nodes, including all its child node(s).
- Array of elements
- **Element**
- Refers to an HTML element or a node
- **Attribute**
- Refers to an attribute// this would be an element attributes if any.

Lesson 78 : Accessing DOM Elements

Accessing DOM Elements:

Let's see how we can access DOM elements. The DOM form is used to access the web page in JavaScript. We are actually viewing the web page when we access the document object.

Document Object:

The DOM document object is the root JavaScript object representing the entire HTML document.

//retrieve the URL of the current page

var a= **document**.URL;

it's a global property which is not part of the DOM tree.

//retrieve the page encoding, for example ISO-8859-1 or utf-8

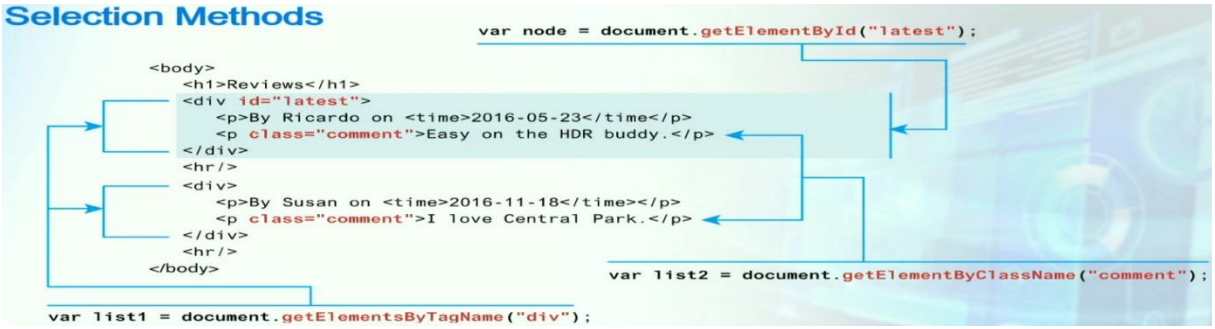
var b= **document**.inputEncoding;

Selection Methods: The document object is used to access the DOM tree, and there are several ways to do so. Let's have a look at main ways.

- **Classic:** in this method either pass on the dement ID/tag name or class name, and we get some node over node.
- getElementById() // each element has a unique ID it will return a single node object/element.

- getElementByTagName() these two functions will return node list.
- getElementByClassName()
- Newer
- querySelector() and //The first match will be returned by this function. And it will return a node.
- querySelectorAII() //this function is used to access multiple elements. And it will return node list

Here are few examples



Example no. 1

This is an HTML document which consists of <body> <h1> it has two paragraphs and a class has been used over there. A dive has an ID. Now we want to access these elements.

var node = document.getElementById ("latest");

The ID is passed as parameter and this function will return a node, and this node will represent the 1st particular dive element under h1 heading.

If we want to access all the div element then we will use the function.

var list = document.getElementsByTagName ("div"); // This will retrieve all of a document's div elements. If there are two div as given in the above example then list will have two div.

At index [0] object will represent the first dive and

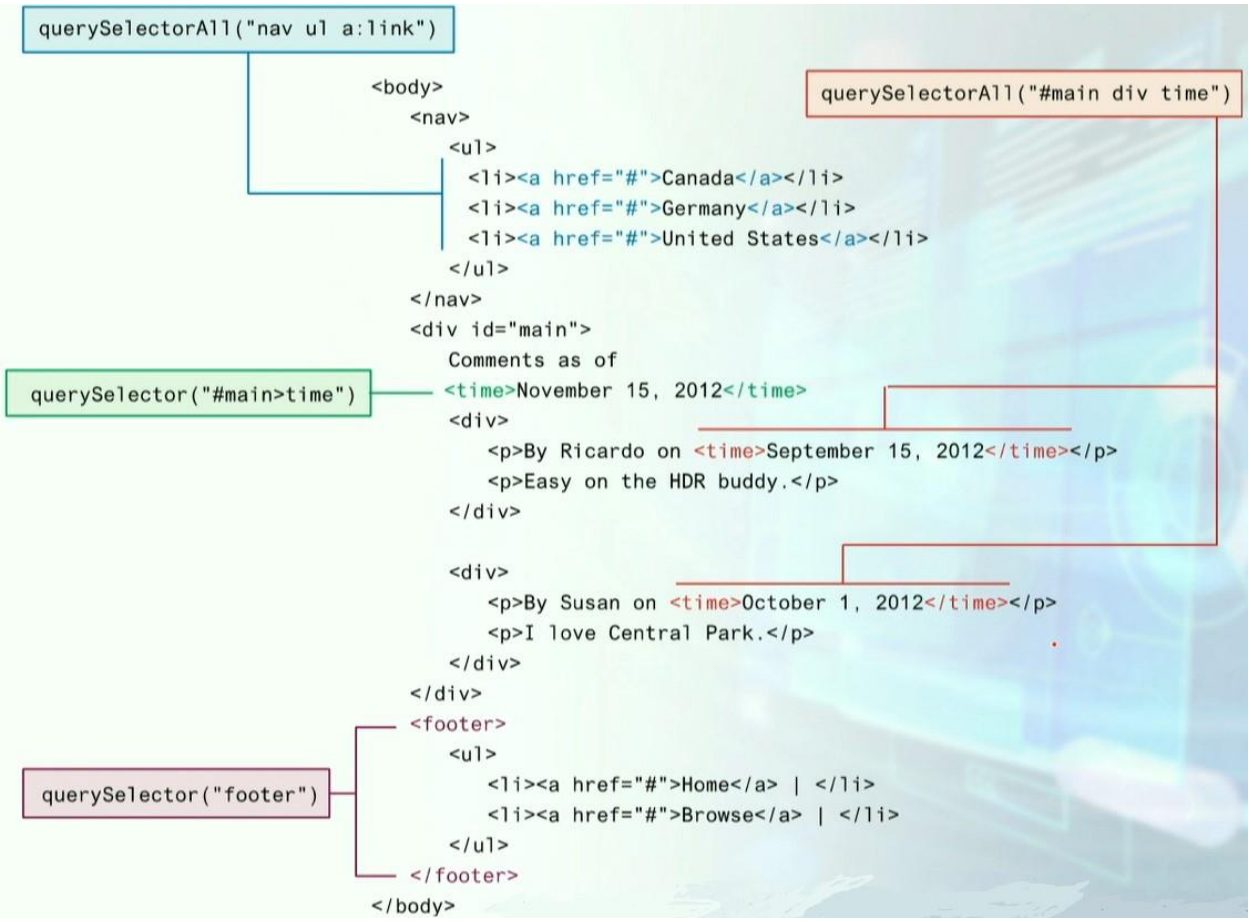
[1] object will represent the 2nd div.

Similarly if we want to access and retrieve all those element whose class name is "comment" such as then the following statement will be used.

var list2 = document.getElementsByClassName ("comment");

We'll get a node list of the elements that use this class.

Query Selector: it is the 2nd way to select element. For selection purposes, the CSS selector is always passed on to the query selector.



Example no. 2

querySelectoAll ("nav ul a: link")// the query selector receives the nav parameter, which includes an unsorted list with a link. All the elements matching with this query will be returned as a result.

Note: In example no.2, the anchor tags have been used in two separate unsorted list (ul), one at the top and the other at the bottom. As a result, this query will return the unsorted list in the **nav**, while ignoring the other unsorted list in the **footer**, which is not in the scope of the **nav**.

Element Node Object:

Element Node object represents an HTML element in the hierarchy, contained between the opening<> and closing</> tags for this element.

Every node has. // these are the properties of a returned node.

- classList // which CSS classes have been used on this element.
- className // what is the class name of a particular element
- id // what is an element ID, if the class for an element has not been defined, we will get an empty string/array.

- innerHTML // everything that exixts between the opening and closing <><</> paragraph(p) tag will be get in **innerHTML form**
- style // associated with each node
- tagName // associated with each node

There are several properties that aren't required to be present in every node.

Element Node object (2)

Some properties are not common in all the elements.

- href // exits in **a** "anchor tag" element and not in **p** element.
- Name // may not be available for each element.
- src // may not be available for each element.
- Value // may not be available for each element.

Note: you can check whether or not a particular property exist in a node. Let supposed to check

'href' in node, if it exist/defined/available in a node, it will return true and vice versa.

Lesson 79 : Modifying DOM elements

Modifying DOM elements.

Let's see how one can modify the DOM elements/elements exists on HTML page via JavaScript. Prior to making any changes, the element must first be accessed. For example, in the example no.1 we have a **div** defined inside <**main**> foremost the div will be selected and there are multiple ways to perform this. But here in this example jut the **queryselector** has been used and not the **querySelectorAll,** So this statement will return the fisrt **div** define in main. the statement is as under.

var node = document.querySelector("main div");

Now once we received it (**div**) in the form of a node, we can change this element/ div node. For example, look at the statement below.

Node.className = "yellowish"; // this yellowish has already been defined in <style> tag

When this code is written in JavaScript the background color of **div** will be changed to yellow. (**Note: box** has already been applied as shown in the example no. 1 and yellowish has been applied using JavaScript) Moreover, you can also remove and re add this color as shown in the example no.1.

Modifying the styles associated with a specific element programmatically is a typical DOM task.

This can be accomplished by altering the style property of that element. For instance, to change an element's background color and add a three-pixel border, we could use the following code:

var node = document.getElementById("someId");

node.style.backgroundColor = "#FFFF00";

node.style.borderWidth = "3px";

Armed with knowledge of CSS attributes you can easily change any style attribute. Note that the style property is itself an object, specifically a CSSStyleDeclaration type, which includes all the CSS attributes as properties and computes the current style from inline, external, and embedded styles. While you can directly change CSS style elements via this style property, it is generally preferable to change the appearance of an element instead using the className or classList properties because it allows the styles to be created outside the code, and thus be more accessible to designers. Using this practice we would change the background color by having two styles defined, and changing them with JavaScript code.



Example no. 1

Example no.1 Manipulating the CSS classes of an element:

Toggling the use of a class is a common use of the classList property. For instance, we might want an element to not be visible until some user action triggers its visibility, which can be done simply using the toggle () function.// toggle is a on/off switch.

// assume that a CSS class called hide has been defined

// if hide is set, remove it; otherwise add it to the element

node.classList.toggle("hide");

DOM family: Some functions are available in DOM which can be used for accessing element(s). For example, if you are at a node you can access it's surrounding or relative node(s) as shown in the figure below. For example, in figure you have a paragraph which is a node. Then what can be the parent node of .

**Node. parentNode=? // it will return the parentNode of **

As a result, it demonstrates that while you are at any node, you may access its relative nodes, such as **parentNode, previousSibling, nextSibling, fistChild, lastChild and so on.**

For navigating between elements and adding or removing elements from the document hierarchy, each node in the DOM includes a number of "family relations" properties and methods. These characteristics are depicted in below figure.

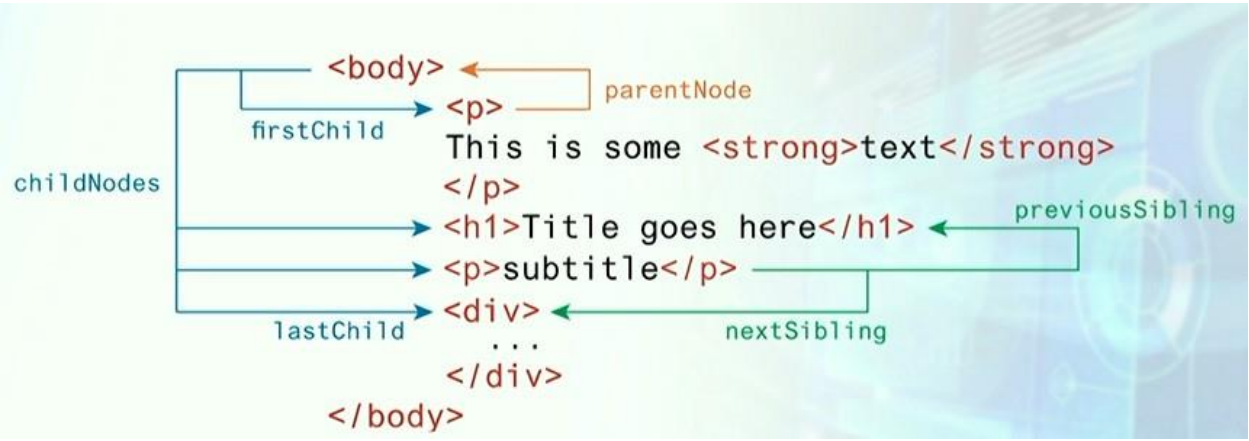


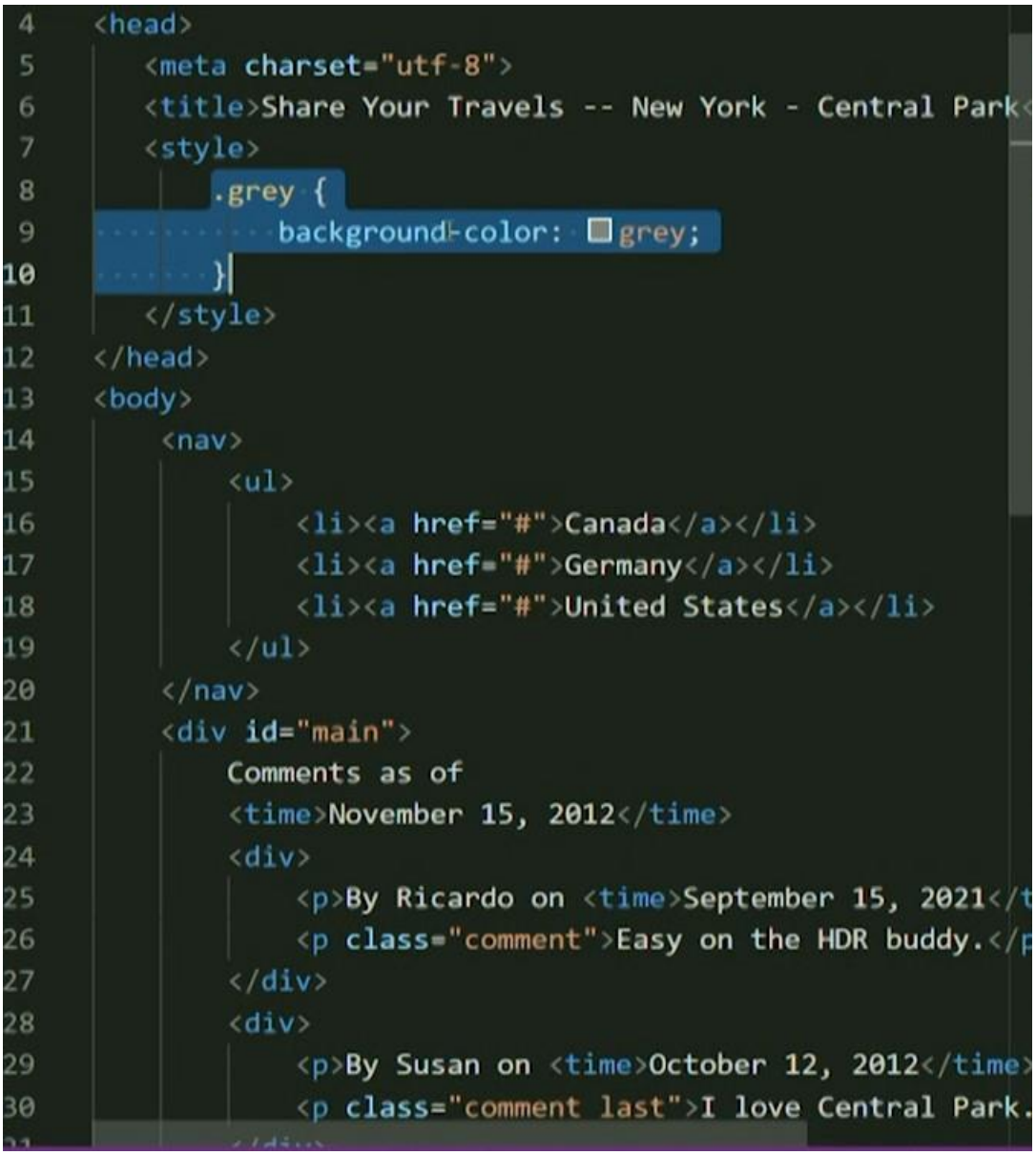
Figure no. 2

Changing an element's content. Let suppose we have a paragraph nod   which is empty, we want to write **foo bar** in this paragraph and bar must be in italic form. So, for doing so, Access the element **via querySelector(); or getElementByTagName(); and then write the statement given below.**

node.innerHTML = "foo bar";

After running this statement , it ("foo bar";)will be come the part of paragraph at run time like this. "foo bar"; .

And it will be displayed in screen like foo*bar*.



```
<head>
    <meta charset="utf-8">
    <title>Share Your Travels -- New York - Central Park
    <style>
        .grey {
            background-color: grey;
        }
    </style>
</head>
<body>
    <nav>
        <ul>
            <li><a href="#">Canada</a></li>
            <li><a href="#">Germany</a></li>
            <li><a href="#">United States</a></li>
        </ul>
    </nav>
    <div id="main">
        Comments as of
        <time>November 15, 2012</time>
        <div>
            <p>By Ricardo on <time>September 15, 2021</t
            <p class="comment">Easy on the HDR buddy.</p
        </div>
        <div>
            <p>By Susan on <time>October 12, 2012</time>
            <p class="comment last">I love Central Park.
```

In this video lecture the following procedure has been used to change the class color to gray

To access **<main>** and to change its's class to gray the following procedure is used.

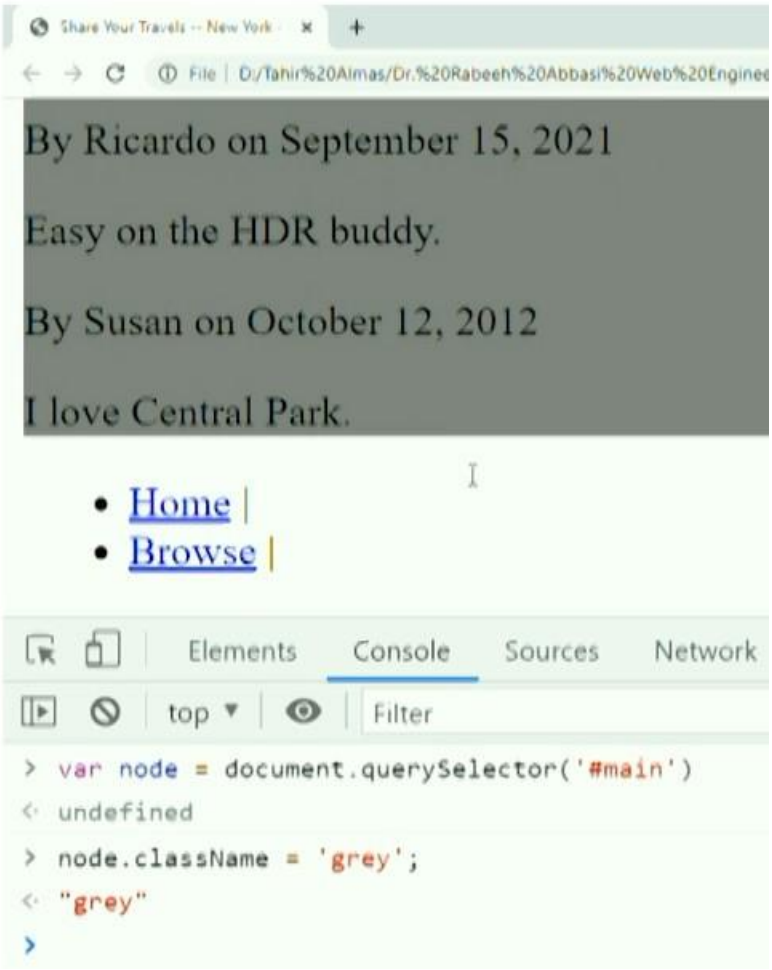
- Goto HTML page
- Goto inspect and then
- Click on console
- Access div// there are two ways to access it, **querySelector ();** the other is **getElementById ();** and write the following code.

var node=document. queryselector('#main') // instead of **node** you can use **name** or simply **div** as well.

No using this we will access its class name as given in below code.

node. className= ‘grey’; // now when you press enter the class will be applied and the background color of **div** will change to grey.

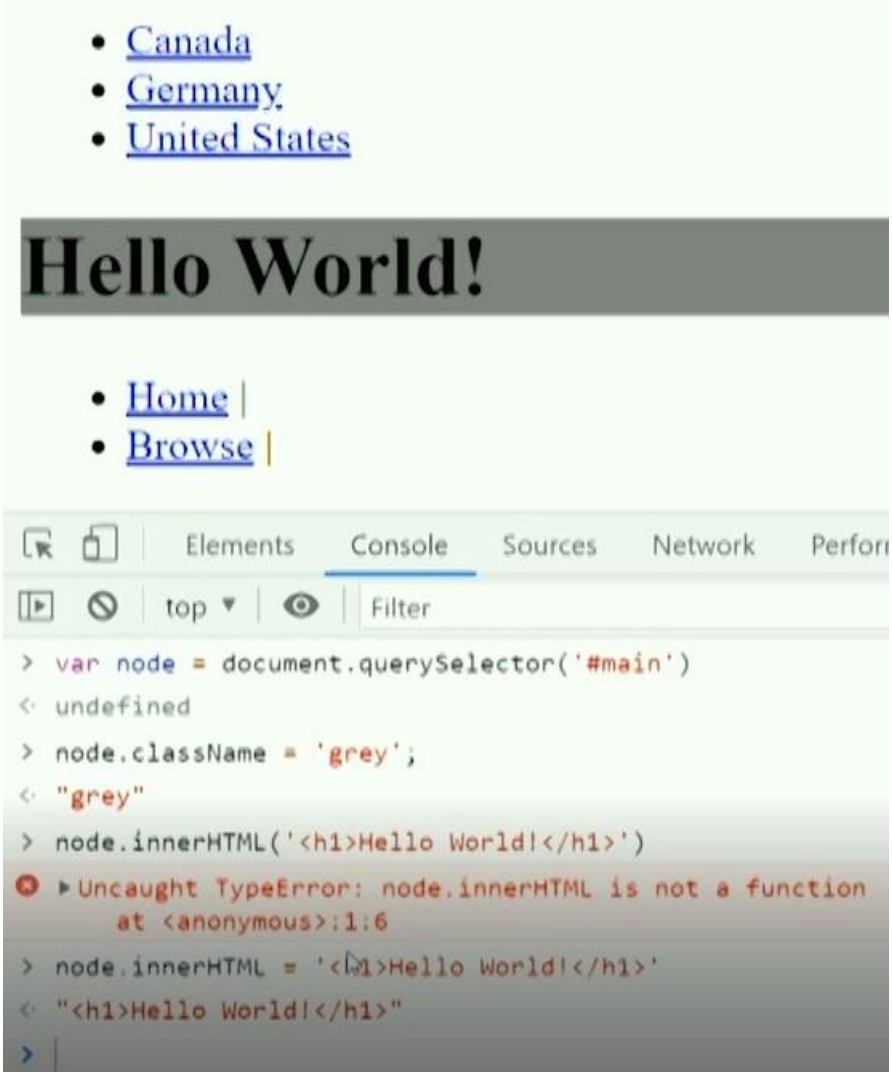
By this way we can access and modify the elements as well.



Now let suppose we want the whole **div** to become empty and to write **Hello word** instead. The following code will be used

node. innerHTML(‘<h1>Hello world! </h1>’) // but if the innerHTML is not a function, it means it is a property and property can’t be called we will just assign it a value. As given below.

Node.innerHTML= **‘<h1>Hello world! </h1>’//** by applying this the previous contents will be replaced by Hello word.



Lesson 80 : Creating DOM elements

creating DOM elements

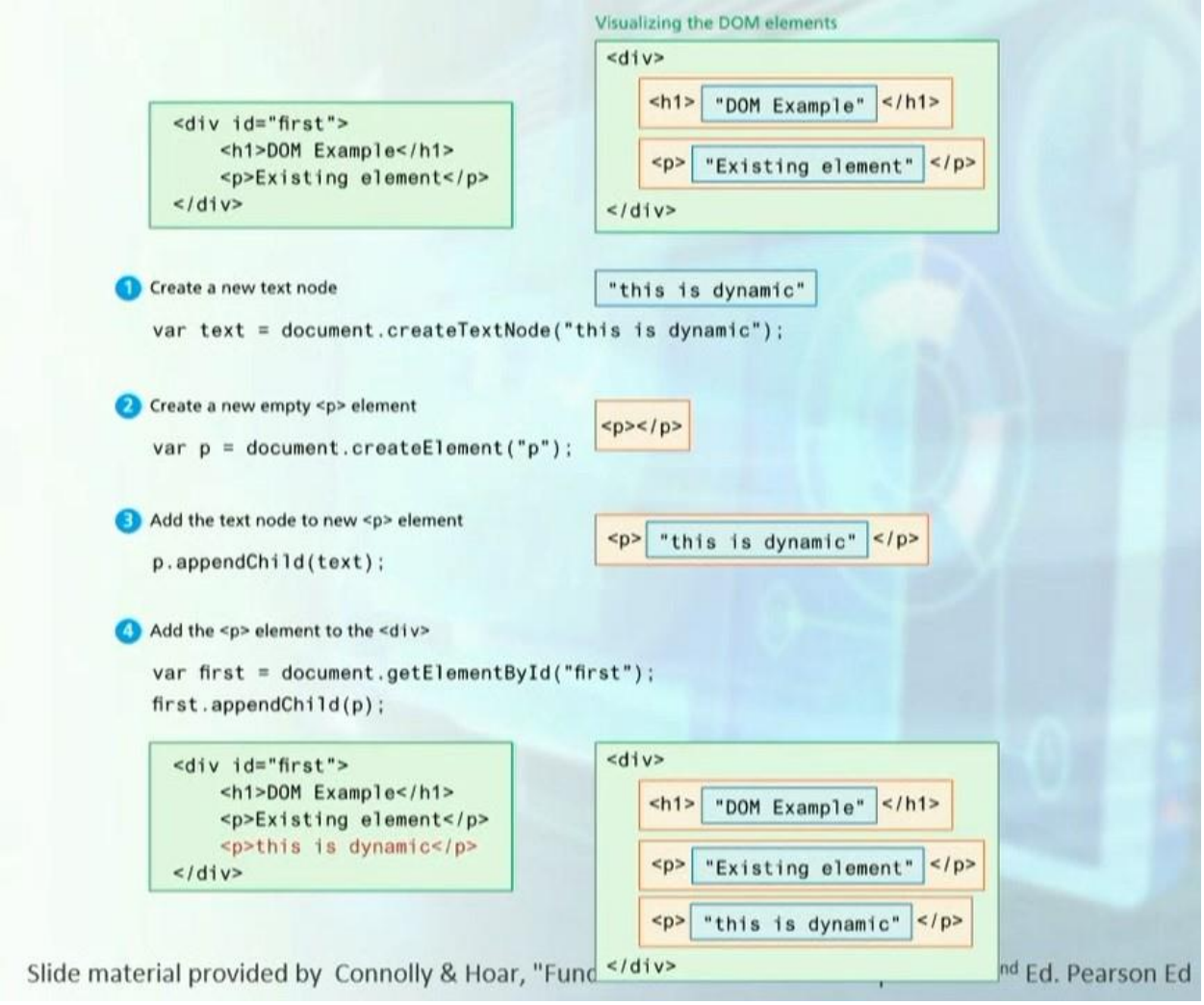


Figure 1

JavaScript and DOM can both be used to add elements to a web page. The element is constructed first in the DOM, and then a text node is created if text is to be added. Using appendChild (); or any other similar accessible functions, whatever nodes are formed will be linked or connected together.

Let's have a look at one example given in figure 1:

<h1> DOM Example</h>

 Existing element

This piece of code contains the following components.

- **div** with id equal's to **first**

- **div** has to children <h1> and

Now we want to add a paragraph having some text in the above piece of. The following statements will be used.

var text = document.createTextNode("this is dynamic");

A text node has been formed that is unconnected to anything else.

Because we want to insert text, the next step is to create a paragraph node. The paragraph is an HTML element so create an element function will be used.

var p=document.createElement ("p"); // it will create an element of paragraph type, and will be stored in p.

In order to make the text as part of the paragraph the following statement will be used.

p.appendChild (text); //A paragraph has been created including some text.

This paragraph will now be included in the HTML document. But how it will be achieved let's have a look at the following statement.

var first = document.getElementById("first");

first. appendChild (p); // the created paragraph node will become the part of DOM tree and will be dispayed on the screen. As shown below and also visible in figure no.1

<h1> DOM Example</h>

 Existing element

 this is dynamic

Week – 7

Lesson 81 : Event handling

Event handling

In every language of computer science event has its own significance, but in JavaScript, which is utilized in web development, it is very significant. An event is an action that occurs as a result of the user or another source, such as a mouse click. Let's see how it is handled in JavaScript.

Introduction:

- JavaScript event is an action that can be detected by JavaScript.
- Many of them are initiated by user actions, some are generated by the browser itself.
- We say that an event is triggered and then is handled by JavaScript functions.

There are multiple ways to handle an event. Let's have a look at them.

The given figure no.80.1 depicts it in detail.

Event Handling Approaches-Inline Hook

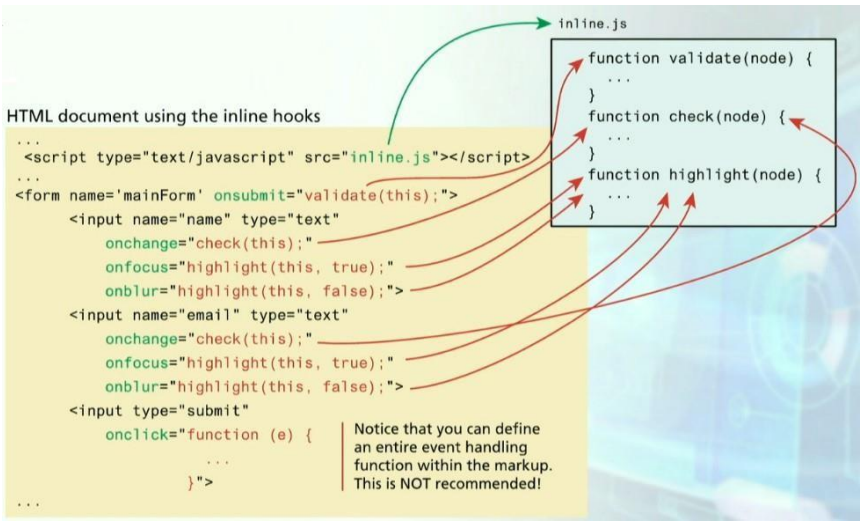


Figure no.80.1

The very first line portrays that a JavaScript code has been stored in an external file which includes the some functions such as

function validated (node){….

} // validate the node

function check (node){….

} // check whether the node is or not an appropriate one.

function highlight (node){….

} // highlight text

The figure 80.1 clearly depicts the events on which these functions are invoked. Another

way of handling event is.

Event –Handling Approaches-Event property Approach:

- **var mybutton=document.getElementById (‘example’**);//In JavaScript, get an element from the DOM tree (Any approach, such as query Selector or get element ById, can be used). After that we write function name on its appropriate event. As shown below.
- **myButton.onclick=alert (‘some message’);**
- **myButon.addEventListener (‘click’, alert (‘some message’));**// it take two parameter
- **myButton.addEventListner (‘mouseout’, funcName);**

Similarly, we may define several events on a single object using this method.

The key benefit of this method is that JavaScript code is segregated from HTML. The script/code may be in an external file or an embedded JavaScript. The code of HTML will be very clean and clear, while making changes in HTML you need not worry about the functionality and vice versa.

Another way to define event.

Event-Handling Approach-Event Listener Approach (anon function)// anonymous function is pass on.

```
myButton.addEventListener (‘click’, function () {
        var d = new Date ();
      alert (“You clicked this on” +d.toString ());
      });
```

// in this piece of code the whole function is pass as a parameter to AddEventListner when a click event is generated.

The last method to handle an event is,

Event Object

When an event is triggered, the browser will construct an event object that contains information about the event.

div.addEventListener (‘click’, function(e)

{

//findout where the user clicked

var x=e. clientX;

and via **e. Target** can be used to manipulate elements of **div.**

Lesson 82 : Capturing and Bubbling

Capturing and Bubbling.

Let's see how JavaScript handle events.

Event Handling:

- Events are handled in three phases.
- Capturing

  Event propagation from parent towards the most nested child.

- Target

  Identifying the target.

- Bubbling

  Elements registered for the events are triggered from the most nested child towards the outermost.

In JavaScript, events progress from the immediate object that caused the event to the document root. This is referred to as event propagation or event bubbling.

While this may appear to be a difficult concept, the underlying notion is extremely straightforward. When you click on an element, you're also clicking on all the ascendants in the DOM tree, as seen in Figure 82.1

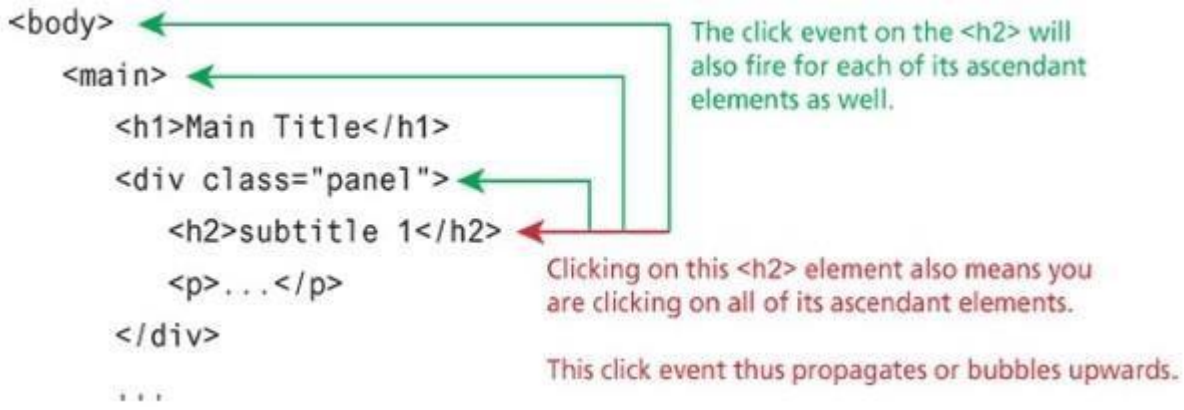


Figure 82.1

Lesson 83 : Event Types

Event Types – mouse, keyboard and touch.

Figure taken from: Fundamentals of Web Development (Second Edition, Pearson), Authors: Randy Connolly, Ricardo Hoar

The click event is perhaps the most visible, but JavaScript and the DOM support a variety of others. In reality, the W3C specifies various classes of event, each having multiple types of events inside each class. Mouse events, keyboard events, touch events, form events, and frame events are some of the most popular event types.

Let's have a look at few types of events which are supported by JavaScript.

Mouse Events: Event related to mouse is called mouse events.

Mouse events are used to describe a variety of mouse-driven interactions. Mouse click and mouse move events are two types of mouse events. Many mouse events can be sent at once, which is unusual. The user may move the mouse from one div to another at the same time, activating the mouseon and mouseout events as well as the mousemove event. To deal with these difficulties, use the Cancelable and Bubbles properties. Following is the list of mouse events.

- **click** The mouse was clicked on an element.
- **dblclick** The mouse was double clicked on an element. // double click
- **mousedown** The mouse was pressed down over an element.// pressed button but did not release yet.
- **mouseup** The mouse was moved (not clicked) over an element.// when the button is released.
- **mouseover** The mouse was moved (not clicked) over an element.

- **mouseout** The mouse was moved off of an element.
- **mousemove** The mouse was moved while over an element.

These are different types of events which can be used in JavaScript.

Note: click event // in this both up and down will be invoked.

And **mouseup** is used only for up and likewise **mousedown** is used only for down.

Keyboard Events: the events related to keyboard is called keyboard events.

Within input fields, these events are most useful. With each key hit, we could, for example, validate an email address or submit an asynchronous request for a dropdown list of suggestions with each key press.
<input type="text" id="key">

For this input box, we could listen for key press events and replay each pushed key back to the user as shown below.

document.getElementById("key").addEventListener("keydown", function (e) { var

keyPressed=e.keyCode;

// get the raw key code

var character=String.fromCharCode(keyPressed);

// convert to string

alert ("Key " + character + " was pressed");

});

Same Above Program By Scammer404 :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <h2>Type in the box below</h2>
    <input type="text" id ="key" placeholder = "Type something..."/>
    <p>Last key pressed: <span id = "display"></span> </p>

<script>

document.getElementById("key").addEventListener("keydown", function(e){
    let character = e.key;
    document.getElementById("display").textContent = character;
});
```

```
</script>

</body>
</html>
```

Note: Regrettably, different browsers handle keyboard properties differently. For example, the keyCode property is not available for the keypress event in FireFox. If we used the same callback method for the keypress event in code above, we'd have to update the code to retrieve the key press as follows:

// use either the which or the keyCode property var

keyPressed = e.which || e.keyCode;

Instead of clogging our code with these kind of browser testing conditional statements, we can use something like the jQuery framework to handle these quirks.

Types of keyboard events as shown below.

- **Keydown** The user is pressing a key (this happens first)
- **Keypress** The user presses a key (this happens after keydown)
- **Keyup** The user releases a key that was down (this happens last)

Touch events: these are relatively new events which are especially used in mobile devices.

- Touch events are a relatively new category of events that can be triggered by devices with touch screens.
- The different events (e.g., touchstart, touchmove, and touchend) are analogous to some of the mouse events (mousedown, mousemove, and mouseup).
- Touch events are only available by default in Chrome and iOS Safari at the time of writing.
- In Edge and FireFox, the user must allow touch events. Pointer events, a new standard specification that combines mouse, touch screen, and pen input into a single event type, are supported by Microsoft Edge. However, pointer events are only supported in Edge at the time of writing.

A form has been created in HTML page, to test a keyboard event.

<form id = “info” method= “post” action= “#”>

<label> Name</label>

<input type = “text” id = “name”>

<label> Email</label>

<input type = “text” id = “ email”>

<input type = “submit”> </form>

Whole Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Form Keyboard Event</title>
</head>
<body>

  <h2>Form Example with Keyboard Event</h2>

  <form id="info" method="post" action="#">
    <div>
      <label>Name:</label>
      <input type="text" id="name">
    </div>
```

```
    <br>
    <div>
      <label>Email:</label>
      <input type="text" id="email">
    </div>
    <br>
    <input type="submit" value="Submit">
  </form>

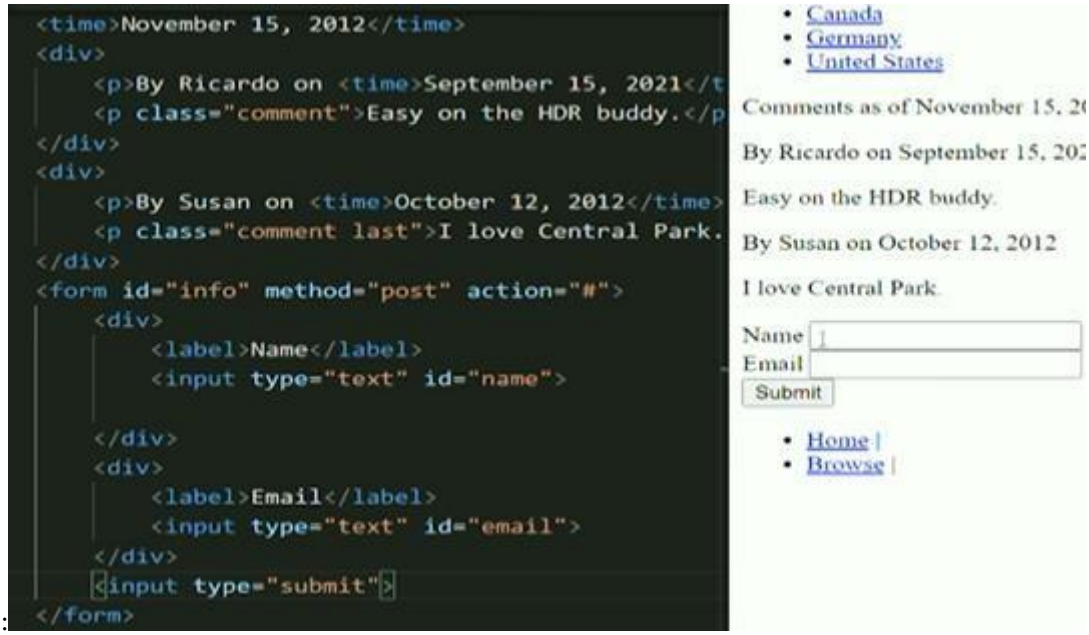
  <p id="output"></p>

  <script>
    const uname = document.getElementById('name');

    uname.addEventListener('keypress', function (e) {
      document.getElementById("output").textContent = "You pressed: " + e.key;

      if (e.key === "_") {
        e.preventDefault();
        alert("Underscore (_) is not allowed!");
      }
    });
  </script>

</body>
</html>
```



:

Figure 83.1

//comment out the div that was built with the event listener as shown in figure 83.2

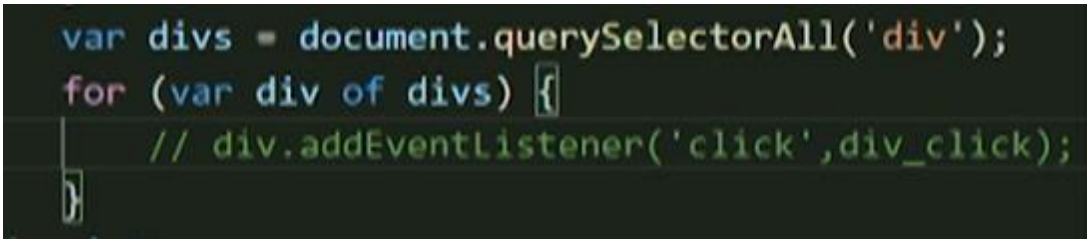


Figure 83.2

<script>

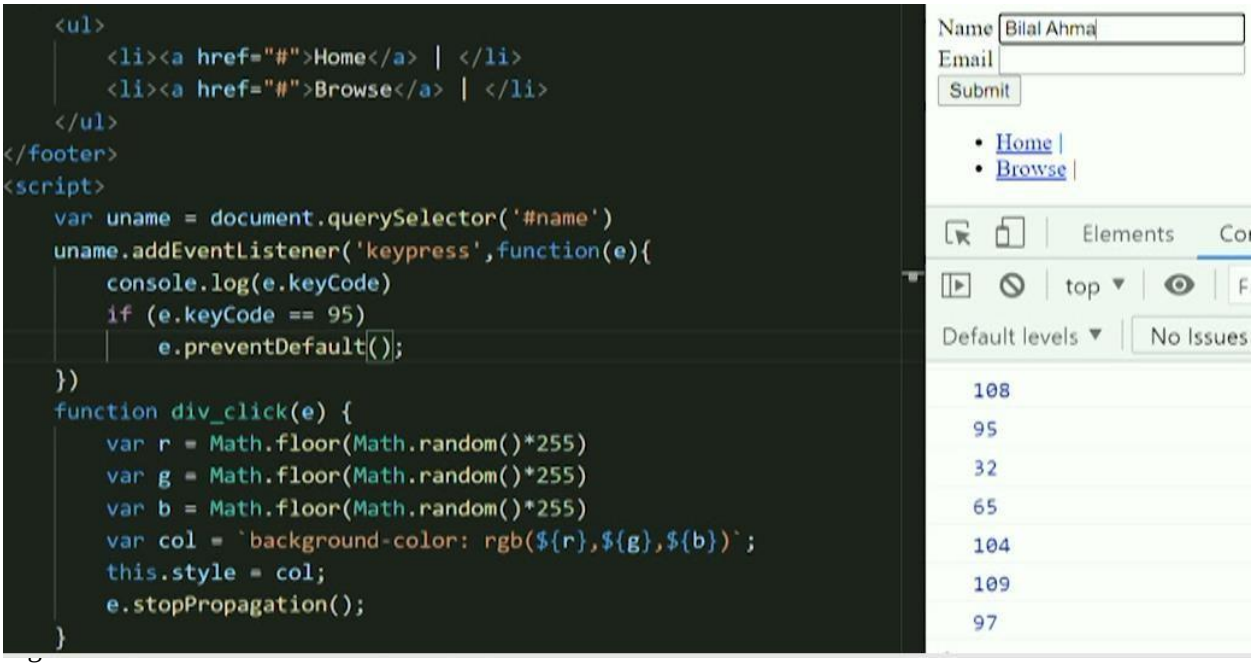
var uname =document. querySelector('name')// reserved word can't be used as variable.

uname.addEventListener('keypress', function(e){console.log(e.keycode)

if (e.keycode==95)

e.preventdefault();})

The event listener is working fine, because we get the ASCII code for the keys which are being pressed, and when the key code is equal to 95 then prevent the default behavior, it implies that the user is not allowed to type underscore in the input box as shown in figure 83.3,



Lesson 84 : Form Validation; Client side input validation

Form Validation; Client side input validation

Form validation is the process of determining whether or not the user has filled out the form correctly. This activity can be done at various stages. E.g

- **HTML Level**

// using built-in HTML element we can do validation e.g type of an email address must be email likewise specify the range for a specific number.

- **JavaScript Level**

// this is also called client side validation.

Note: the user may disable JavaScript. And submit the form. Because similar situations may arise, it is vital to validate the form at the server level as well to prevent erroneous data from being stored in the database. So it good to have validation at all the stages.

- **Webserver Level**

Form Events:

Events related to form are called form events. Forms are the primary means of collecting and transmitting user input to the server. Form events allow us to perform some real-time processing in response to user input. The submit event is the most popular JavaScript listener for forms. We

use preventDefault() to prevent submitting to the server and inform the user if the password field (with id pw) is blank. Otherwise, nothing is done, allowing the default event to occur (submitting the form).

Catching the submit event and validating a password to not be blank

```
document.getElementById("loginForm").addEventListener('submit', function(e)
{  var pass = document.getElementById("pw").value;
if (pass=="")  { alert ("enter a password");
e.preventDefault ();    } });
```

Form events

- submit// When the form is submitted this event is triggered. We can do some prevalidation of the form in JavaScript before sending the data on to the server.
- reset // HTML forms have the ability to be reset. This event is triggered when that happens.
- formdata

Textfield events

- blur // Triggered when a form element has lost focus (that is, control has moved to a different element), perhaps due to a click or Tab key press.
- change// some <input>, <textarea> or <select> field had their value change. This could mean the user typed something, or selected a new choice.
- focus // Complementing the blur event, this is triggered when an element gets focus (the user clicks in the field or tabs to it).
- select // When the users selects some text. This is often used to try and prevent copy/paste.
- ….

Let's look at an example where these events will be used to validate a form. uname.addEventListener (‘change’, check_name);

```
email.addEventListener (‘change’, check_email);
form.addEventListener (‘submit’, validate); function
check_name () {
if (this.value.length<5)
{
    this.style= ‘background-color: pink’;
} else {
    this.style= ‘background-color: white’;
}
}
function check_email () {
        var re= / [^@]+@[^\.]+\.[^\.]+/
         if (! re.test (this.value)){
             This.style= ‘background-color: pink’;
        } else {
this.style= ‘background-color: white;
}
}
function validate (e) {
       var re=/ [^@]+@[^\.]+\.[^\.]+/
       if (uname.value.length<5||re.test (email.value) ==false {
       e.preventDefault ();
       }
}
```

Lesson 85 : Introduction to jQuery

Introduction to jQuery

JQuery is one of the JavaScript libraries that are used to assist us with web development.

jQuery is a popular framework that gives developers access to cross-browser animation tools, user interface elements, and DOM manipulation functions, among other things. Learning about frameworks, especially jQuery, can help you improve your development abilities by allowing you to use ever-evolving frameworks that provide new and increasingly expected functionality.

The Most popular Framework

- Makes working with JavaScript and DOM easy// JQuery also makes it easy to select an element and to perform different operation on it
- Cross browser compatibility // it makes sure that the code written is

executable on all the different browsers.

- Most popular JavaScript library (as of 2021). According to w3techs.com
    - Used by 78% of all website
    - 96% market share

How to include JQuery in our code, there are multiple ways to include it.

Including JQuey

- Use a Content Delivery Network

  (CDN)
- <script src =

  https://code.jquery.com/jquey-

  3.6.0.min.js> </script>

//can include directly from CDN, min.js is best as it has minimum size.

- Can also be Download and used on your web server
- Installed via package manager(npm, yarn, etc)

Note: JQuery can be included in the head, or before body in HTML file. Let's

have a look at an example that how it can be included in HTML file.

Figure 85.1 shows that how it can be included when we have an internet connection.



Figure no 85.1

But if we have no internet connection then download and save it in folder where you have HTML file.

And then include it using absolute path as shown in figure 85.2



Figure no. 85.2

Lesson 86 : jQuery selectors

jQuery selectors

The programmatic selection of DOM elements is one of the most common JavaScript jobs. The element selection method in jQuery is both strong and easy. JavaScript has a variety of functions for selecting an element, such as getElementByID() and querySelector().Although the querySelector() and querySelectorAll() capabilities now let you pick DOM elements using CSS selectors, developers have only been able to assume that these functions are available in most users' browsers since around 2014 (IE 8 and earlier didn't fully support these techniques). One of

the reasons for jQuery's early popularity was that it gave programmers with a straightforward cross-browser way to programmatically select an element using regular CSS selectors.

- The power of jQuery resides in the function named jQuery()
- There's also an alias for this function named $()
- You can combine CSS selectors with the $() notation to select DOM objects that match CSS attributes

JavaScript versus jQuery selection

```
<p id="here">hello <span>there</span></p>
<ul>
 <li>France</li>
<li>Spain</li>
<li>Thailand</li>
</ul>
<script>
/* selecting using regular JavaScript */
var node = document.getElementById("here"); var
link = document.querySelectorAll("ul li");
/* equivalent selection using jQuery */
var node = $("#here");
var link = $("ul li");
</script>
```

So, what is the result of the $() function? The getElementByID() function, as you may recall, returns an Element object, whereas querySelectorAll() returns a NodeList object. The jQuery set object, which is an array-like structure that holds a set of DOM elements that match the selector, is returned instead by the $() function.

Instead of returning a single object, the $() function always returns a group of results. This is simple to overlook and might cause unanticipated frustration. For example, the following code can be used to obtain something that is singular (such as the body> element):

```
$('body') = temp
```

Due to the fact that the $() function always produces a set of results, you must refer to this element as the 0th array element, as illustrated below:

var b = temperature[0];

Nonetheless, the fact that jQuery always produces an array is one of its strengths: regardless of whether a selector returns a single value or numerous values, all of its methods work the same.

Basic Selectors

The four basic selectors were defined back in Chapter 4, and include the universal selector, class selectors, id selectors, and elements selectors. Selectors in jQuery are designed to closely resemble the CSS specification, which is especially useful given that CSS is something you've learnt and used throughout this book:

- $(“*”)—Universal selector matches all elements (and is slow).
- $(“tag”)—Element selector matches all elements with the given element name.
- $(“.class”)—Class selector matches all elements with the given CSS class.
- $(“#id”)—Id selector matches all elements with a given HTML id attribute.

For example to pick the single div> element with id="grab," write:

var singleElement = $(“#grab”);

To get a set of all the <a> elements the selector would be:

var allAs = $(“a”);

Other CSS selectors mentioned in Chapter 4: attribute selectors, pseudo-element selectors, and contextual selectors can be used in addition to these basic selectors, as shown in Figure 86.1. The rest of this section goes over a few of these selections and how they work with jQuery.

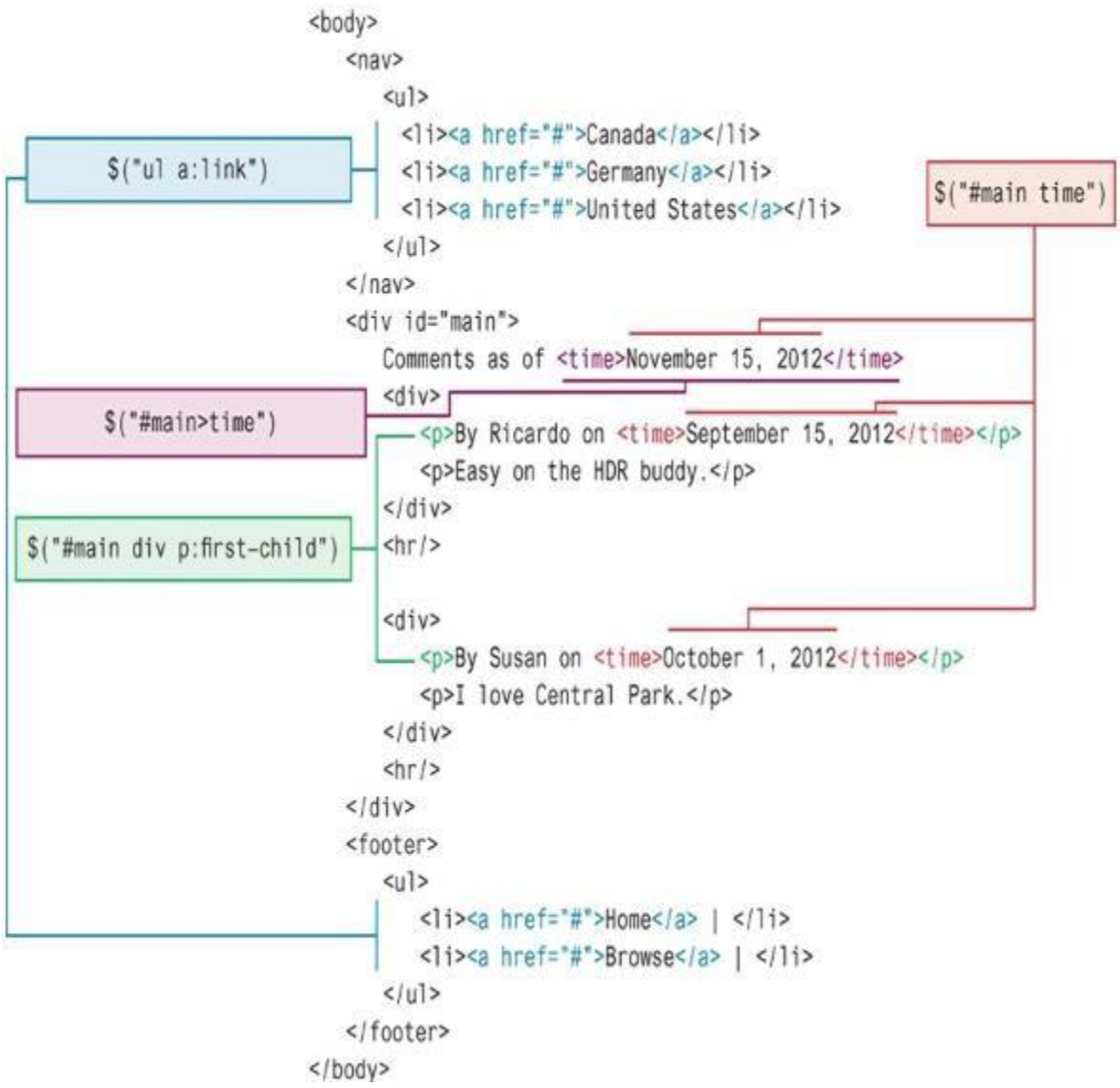


Figure 86.1: figure taken from: Fundamentals of Web Development (Second Edition, Pearson), Authors: Randy Connolly, Ricardo Hoar

jQuery's Content Filter Selection

The jQuery content filters provide you more selection options. You can use:has() to choose elements that have a specific child,:empty to select elements that have no children, and:contains to match a specific piece of text (). Consider the following illustration:

var allWarningText = $("body *:contains ('warning')");

It will provide a list of all DOM elements that include the term warning. You can see how we would want to draw attention to those DOM elements by colouring the backdrop red, as seen in Figure 86.2.

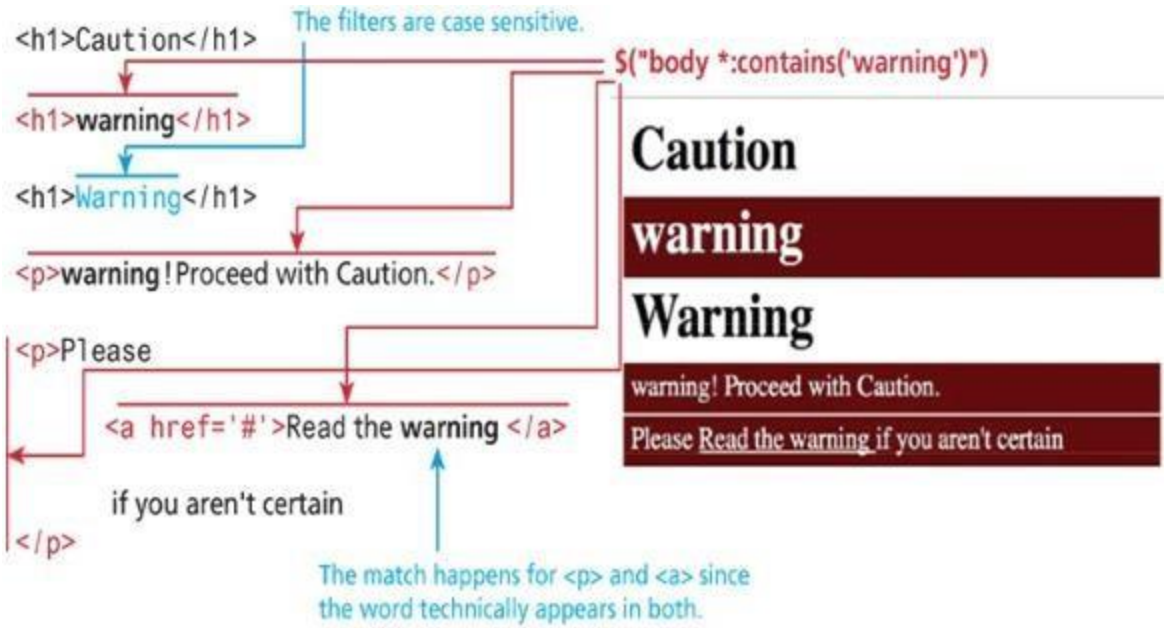


Figure 86.2: figure taken from: Fundamentals of Web Development (Second Edition, Pearson), Authors: Randy Connolly, Ricardo Hoar

Lesson 87 : jQuery Manipulators

jQuery Manipulators

//manipulator can be used for both read and write.

Common Element Manipulations in jQuery

You can choose any set of components from a web page using all of the selectors mentioned in this chapter. You can then alter them in a variety of ways once they've been picked.

- We can both set and get innerHTML through the html() method

//link is assigned the html code contained by the first p var

html = $(“p”).html ();

// change the innerHTML of an element

$(“p#message”).html (“ Invalid Email”);

// change innerHTML of all matching elements

$(p.warning”) .html (“No warning”);

.text () can be used in similar manner for innerText

Common Element Manipulations--------HTML Attributes

The attributes defined in the HTML tags detailed in Chapter 3 are the fundamental collection of attributes associated to DOM elements. You've probably used key attributes like an <a> tag's href attribute, an <img> src attribute, and most elements' class attribute before. Using the attr () method on any element returned by a selector in jQuery, we can both set and obtain an attribute value. The attribute name is specified by the first parameter, and the optional second parameter allows you to edit the attribute by specifying a value. If no second parameter is provided, the procedure returns the attribute's current value. The following are some examples of possible applications:

We can both set and get an attribute value by using the attr () method

// link is assigned the href attribute of the first <a> tag

var link = $ (“a”). attr (“href”);

// change all links in the page to http://funwebdev.com

$(“a”).attr (“href”, http://funwebdev.com”);

//change the class for all images on the page to fancy

$(“img”).attr (“class”, “fancy”);

Common Element Manipulations--------HTML Properties

The checked property of a radio button or checkbox is the most common example of an HTML tag that includes both properties and attributes. The attr () method could be used to set HTML properties in earlier versions of jQuery. However, because properties are not actually characteristics, strange behavior ensued. Although attr () may return some (less useful) values, the prop () method is now preferred for retrieving and setting the value of a property.

To illustrate this subtle difference, consider a DOM element defined by

<input class=“meh” type=“checkbox” checked=“checked”>

The value of the attr () and prop () functions on that element differ as shown below. var

theBox = $(“.meh”);

theBox.prop (“checked”); // evaluates to TRUE theBox.attr

(“checked”); // evaluates to “checked”

val () used for getting/setting values of elements having the value property (e.g., <input type= “text”…>

Common Element Manipulations--------Changing CSS

Modifying a CSS style is fairly similar to changing attributes in terms of syntax. The css() technique in jQuery is incredibly user-friendly. This method has two versions (with two different method signatures), one for getting the value and the other for setting it. The first version takes a single parameter holding the value you want for the CSS attribute and returns it.

var color = $("#element").css ("background-color"); // get the color

To set a CSS attribute, you use the second version of css (), which takes two parameters: the first being the CSS attribute, and the second the value.

// set color to red

$("#element").css ("background-color", "red");

Lesson 88 : jQuery Manipulators – Example

jQuery Manipulators – Example

The background color of name field has been changed using jQuery, as shown in below figure 88.1.

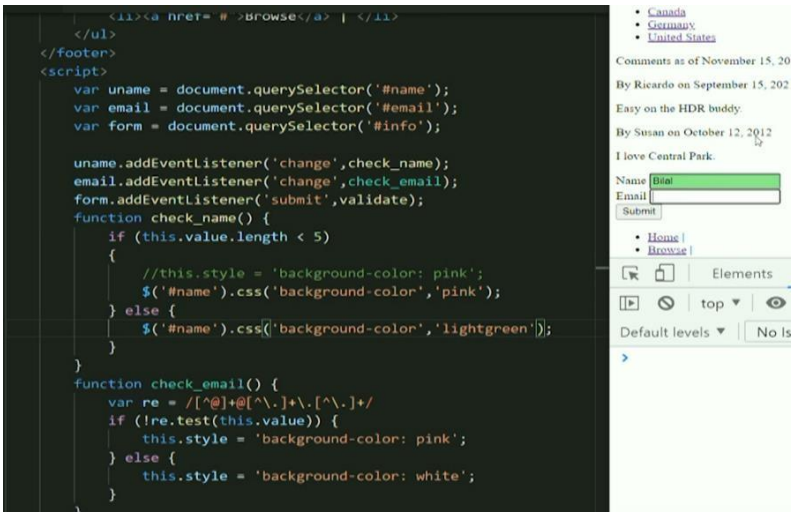


Figure 88.1

You can check out the length as well as the value typed in the name filed as illustrated in figure 88.2.

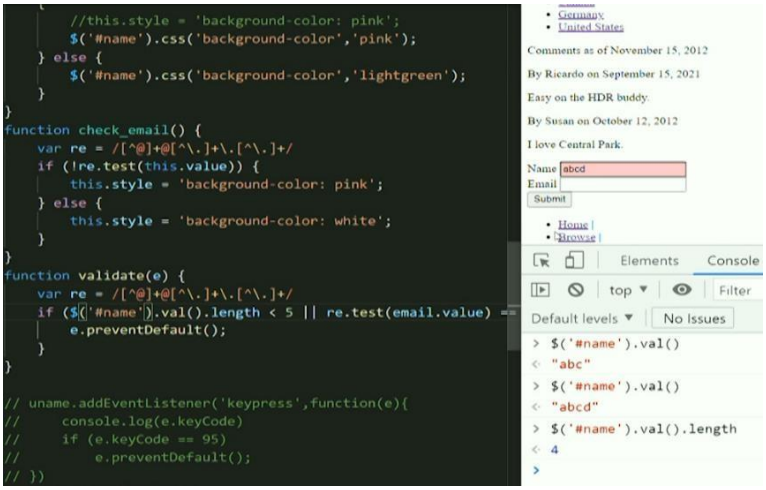


Figure 88.2

Note: You can also do it in the console as demonstrated below.

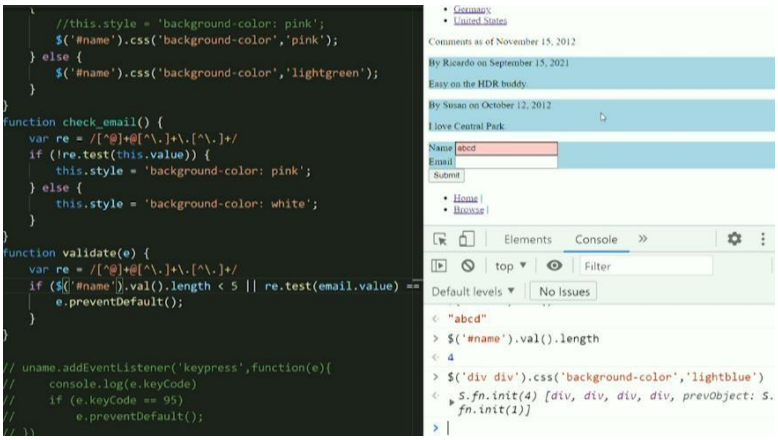


Figure 88.3

Using the code shown in the console, you can easily alter the background color of div and inner dive.

Lesson 89 : Events in jQuery

Events in jQuery

Introduction:

jQuery, like JavaScript, allows you to create and manage JavaScript event listeners/handlers. With a few minor syntax modifications, these events are used in the same way as JavaScript.

Listeners

For jQuery Setting up listeners for specific events is similar to how JavaScript works. While addEventListener () is used in pure JavaScript, jQuery includes on () and off () methods as well as shortcut methods for attaching events.

Looking at figure 89.1. we can see that jQuery is substantially less verbose than JavaScript once again. Is that, however, all it has to offer? When working with events, jQuery also makes many typical chores easier. Figure shows a simple example with three different mouse events. The click event handler, in particular, makes advantage of the jQuery off () function to stop listening to the mouse move event. This is far more straightforward than the JavaScript-only solution.

Binding and unbinding events

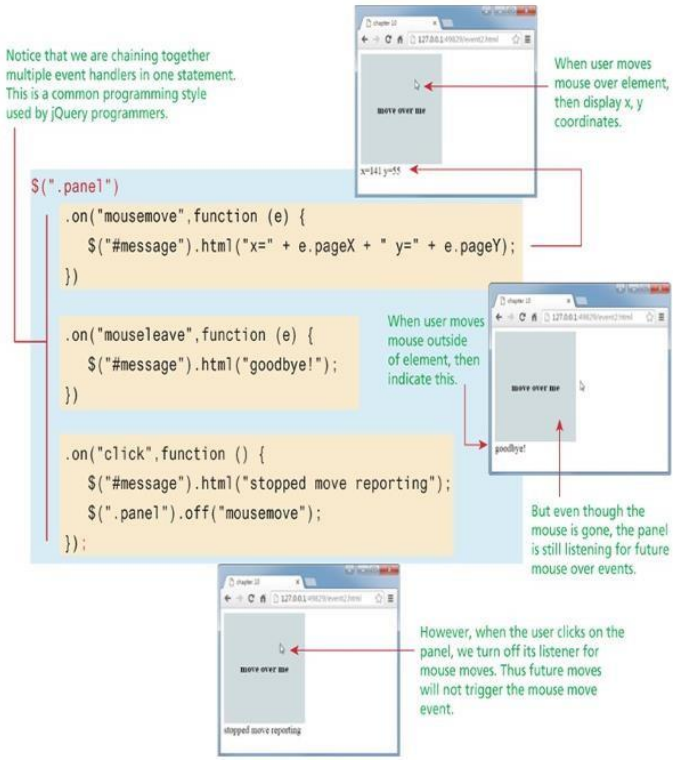


Figure 89.1.

Page loading

You learnt why having your listeners inside the window is important in JavaScript. It was a fantastic idea to use the .addEventListener ("load",...) event.

It did this by ensuring that the complete page and all DOM elements were loaded before attempting to connect listeners to them. With jQuery we do the same thing but use the
$(document).ready () event

$(document).ready (function () {

// set up listeners knowing page loads before this runs

```
$(“#example”).click (function () {

$(“#message”).html (“you clicked”);

}); });

// or the even simpler

$(function () {

$(“#example”).click (function () {

$(“#message”). html (“you clicked”);

}); });
```

Example:

```
//var divs =document.querySelectorAll(‘div’);

//for (var div of divs) {

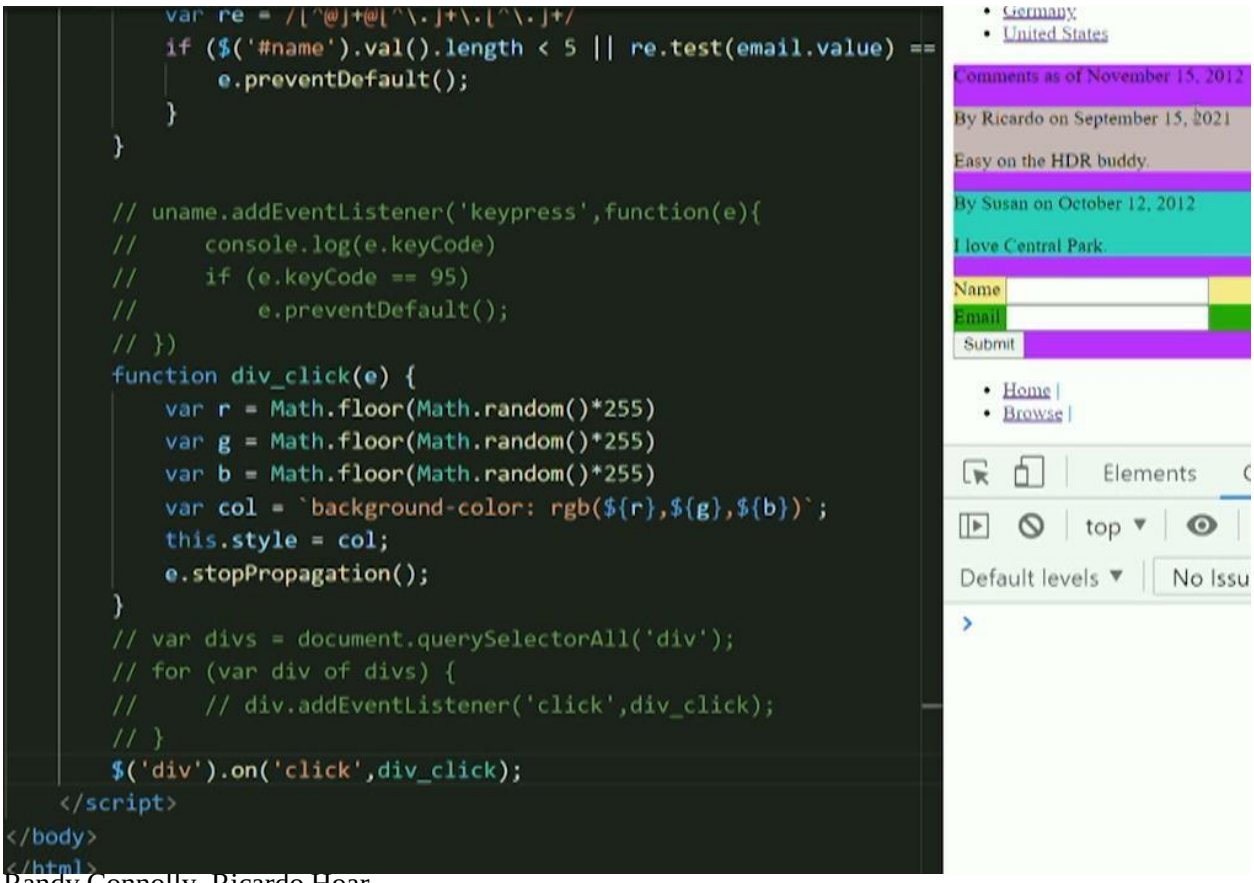
//          .. div.addEventListener (‘click’, div_click);

//}
```

The code above was created in java scripts, and all of the divs were accessed using java script, which was then iterated using for loop, and listeners were added to each individual listener.

Note: the above bunch of code were replaced by a single line of code in jQuery.

```
$(‘div’).on (‘click’, div_click); // click event has been registered.
```



Randy Connolly, Ricardo Hoar

Lesson 90 : DOM manipulation using jQuery

DOM manipulation using jQuery

jQuery has a number of helpful methods for manipulating DOM elements. We've already seen how to edit the inside contents of a DOM element with the html () function, as well as how to modify the internal attributes and styles of an existing DOM element using the attr () and css () methods.

You may recall learning several JavaScript DOM modification methods like appendChild (), createElement (), and createTextNode () in Chapter 9. A similar set of DOM manipulation techniques is offered by jQuery, which extends the capabilities of these native JavaScript DOM functions.

Creating Nodes

If you choose to think of your page as a DOM object, you'll want to work with the tree structure rather than just the strings. Fortunately, jQuery can automatically translate strings containing correct DOM syntax into DOM objects.

Remember that the createElement () method is used to create a DOM node in JavaScript: var

element = document.createElement('div'); //create a new DOM n

However, because the jQuery DOM manipulation functions accept an HTML string, jQuery objects, or DOM objects as parameters, you could opt to describe your element as follows:

var element = $(" "); // create new DOM node

Instead of relying on pure JavaScript, which has fewer shortcuts, you may apply all of the jQuery functions to the object this way. If we consider creation of a simple <a> element with multiple attributes, you can see the comparison of the JavaScript and jQuery techniques in be

Comparison of node creation in JS and jQuery

Example no.90.1.

// pure JavaScript way

```
var jsLink = document.createElement("a");
jsLink.href = "http://www.funwebdev.com";
jsLink.innerHTML = "Visit Us";
jsLink.title = "JS";
// jQuery version 1
var link1 = $('<a href="http://funwebdev.com" title="jQuery">Visit Us</a>');
// jQuery version 2
var link2 = $("<a></a>");
link2.attr("href","http://funwebdev.com");
link2.attr("title","jQuery verbose");
link2.html("Visit Us");
```

// version 3 … also not creating a temporary variable which // will be more typical once we start chaining methods (see next // section)

```
$('<a>', {
        href: 'http://funwebdev.com',
        title: 'jQuery',
```

```
        text: 'Visit Us'
    }
);
```

As you can see, jQuery offers a variety of ways to create an element. It is important to note that running the code in above example no.90.1. has no observable effect on the browser window.
The code has only done one thing: it has created DOM nodes. You must also add the elements to the DOM tree, which is discussed next, in order for them to be visible.

Adding DOM Elements

When an element is defined in any of the ways described in example no.90.1. , it must be inserted into the existing DOM tree. You can also insert the element into several places at once if you desire, since selectors return a set of DOM elements.

Inserting DOM Elements

The append () method accepts as a parameter an HTML string, a DOM object, or a jQuery object as an argument. The item is subsequently added to the element(s) being selected as the last child.

In Figure 90.1.1, we can see the effect of an append () method call. As can be seen in the figure, jQuery has a variety of additional methods for adding content to the DOM tree.

```
<div class="dest">
existing content
</div>
```

```
var link = $('<a href="http://funwebdev.com">Fun</a>');
```

```
$(".dest").append(link);
```

```
<div class="dest">
existing content
<a href="http://funwebdev.com">Fun</a>
</div>
```

```
$(".dest").prepend(link);
```

```
<div class="dest">
<a href="http://funwebdev.com">Fun</a>
existing content
</div>
```

```
link.appendTo($(".dest"));
```

```
<div class="dest">
existing content
<a href="http://funwebdev.com">Fun</a>
</div>
```

```
link.prependTo($(".dest"));
```

```
<div class="dest">
<a href="http://funwebdev.com">Fun</a>
existing content
</div>
```

```
$(".dest").before(link);
```

```
<a href="http://funwebdev.com">Fun</a>
<div class="dest">
existing content
</div>
```

```
$(".dest").after(link);
```

```
<div class="dest">
existing content
</div>
<a href="http://funwebdev.com">Fun</a>
```

```
link.insertBefore($(".dest"));
```

```
<a href="http://funwebdev.com">Fun</a>
<div class="dest">
existing content
</div>
```

```
link.insertAfter($(".dest"));
```

```
<div class="dest">
existing content
</div>
<a href="http://funwebdev.com">Fun</a>
```

Figure 90.1.1

The appendTo () method is similar to append (), except it is used in the opposite syntactic order.

If we were to use appendTo (), we would have to switch the object making the call and the parameter to have the same effect as append ().

The variable link is used in the example in the picture, but a more frequent technique that is functionally comparable is as follows:

$('Fun').appendTo($('.dest'));

The prepend() and prependTo() methods operate in a similar manner except that they add the new element as the first child rather than the last. The diagram also illustrates the before() and after() methods for adding content before or after a specified element.

Wrapping Existing DOM in New Tags

Adding new HTML elements as needed to support some jQuery methods is one of the most common ways to improve a website that supports JavaScript. Imagine for illustration purposes our art galleries being listed alongside some external links as described by the HTML in example 90.2. below.

Uffuzi Museum

National Gallery

funwebdev.com

If we wanted to wrap all the gallery items in the whole page inside, another

 (perhaps because we wish to programmatically manipulate these items later) with class galleryLink we could write:

$(".gallery").wrap(' ');

This modifies the HTML to that shown in example 90.3 below Note how each and every link is wrapped correctly in a that uses the galleryLink class.

Uffuzi Museum

National Gallery

funwebdev.com

In a related demonstration of how succinctly jQuery can manipulate HTML, consider the situation where you wanted to add a title element to each element that reflected the unique contents inside. To achieve this more sophisticated manipulation, you must pass a function as a parameter rather than a tag to the wrap() method, and that function will return a dynamically created element as shown in example 90.3. below.

Using wrap() with callback to create a unique div for each element

```
$(“.gallery”).wrap (function () {

return “<div class='galleryLink' title='Visit ” +   $(this).html() + “></div>”;

 });
```

You might have noticed the use of $(this), and wondered what $(this) meant in example 90.3. above

Among jQuery developers, this is a common idiom. It's used in functions to provide a jQuery wrapper for the this keyword. You may remember from Chapter 8, that this is often a point of confusion for JavaScript developers since its meaning is completely dependent upon the context in which it is used. In example 90.3. above, the $(this) refers to whatever $(“.gallery”) object is passed to the wrap () function. The wrap () method is a callback function, which is called for each element in a set (often an array). Each element then becomes this for the duration of one of the wrap() function's executions, allowing the unique title attributes as shown in example 90.4. below.

```
<div class=“external-links”>

<div class=“galleryLink” title=“Visit Uffuzi Museum”>

<div class=“gallery”>Uffuzi Museum</div>

</div>

<div class=“galleryLink” title=“Visit National Gallery”>

<div class=“gallery”>National Gallery</div>

</div>

<div class=“link-out”>funwebdev.com</div>

</div>
```

There is an inverse method to perform the opposite objective, as there is with practically anything in jQuery.

In this case, unwrap () is a method that does not take any parameters and whereas wrap() added a parent to the selected element(s), unwrap() removes the selected item's parent. WrapAll () and wrapInner (), for example, provide you more control over wrapping DOM elements. The online jQuery documentation has more information on certain methods.

Lesson 91 : Animations using jQuery

Animations using jQuery

A very powerful feature of jQuery is with the reference of Animation. Different elements could be added through animation in html document.

Animation Effects:

Some of the generic animation methods are as show and fade In methods.

Show() and fadeIn():

.show(duration)

This function will display the image gradually over given duration in milli seconds. For

example;

If image was hidden then **.show(1000)** function will gradually show the image in 1000 milli seconds (1 second).

You can also pass string parameters like slow or fast to show function.

.fadeIn(duration)

This function gradually fade the element by decreasing its opacity to given duration.

slide()

This function will move the element from one position to another. For

example;

A menu item will slide down from a button when mouse hover or enter on button element and when mouse leave the button ,sliding menu disappers.

```
<button id="menuBtn">Menu</button>
<ul id="menu">
    <li><a href="#">Menu item 1</a></li>
    <li><a href="#">Menu item 2</a></li>
    <li><a href="#">Menu item 3</a></li>
    <li><a href="#">Menu item 4</a></li>
</ul>
```

```
$(function () {
    $("#menu").hide();      When page loads, hide the list.

    $("#menuBtn").on("mouseenter", function () {
        $("#menu").slideDown(500);
    });                 Slide list down in 0.5 sec when mouse
                        hovers over it.

    $("#menuBtn").on("mouseleave", function () {
        $("#menu").slideUp(300);
    });                 Slide list up faster when mouse is no
                        longer hovering over it.

});
```

Example-1: slide() function (taken from Fundamentals of Web Development second edition 2017)



Figure-1: slide() function (taken from Fundamentals of Web Development second edition 2017)

Use animation to a limited extent, because animations take time and if users want to do work fast, animations will annoy users because of their durations.

Raw Animations in jQuery:

Along with generic animation methods of jQuery, there are some raw animations which could be used for custom type requirements. You can animate any numeric CSS property.

animate()

$("#box").**animate**({left:'495px');

Function animate will move the box about 495 pixels on left side. Box animation will be visible from one position to other. The position of object must be relative not statis.

For example:

Following code example shows a box enter on the page from left side, then move upwards, and by clicking on the button in that dialogue box, the dialogue box get fade.

```
$(function() {
    $('#notifyBtn').on("click", function () {
        $('.notification')
            .animate({right:'0px', opacity: "1"},500)
            .animate({top: "0"});
        window.setTimeout(function() {
            dismissNotification();
        }, 4000);
    });

    $('#dismissBtn').on("click", function () {
        dismissNotification();
    });
});
```

3 Over 0.5 sec, first animate these two properties

These are the **after** values

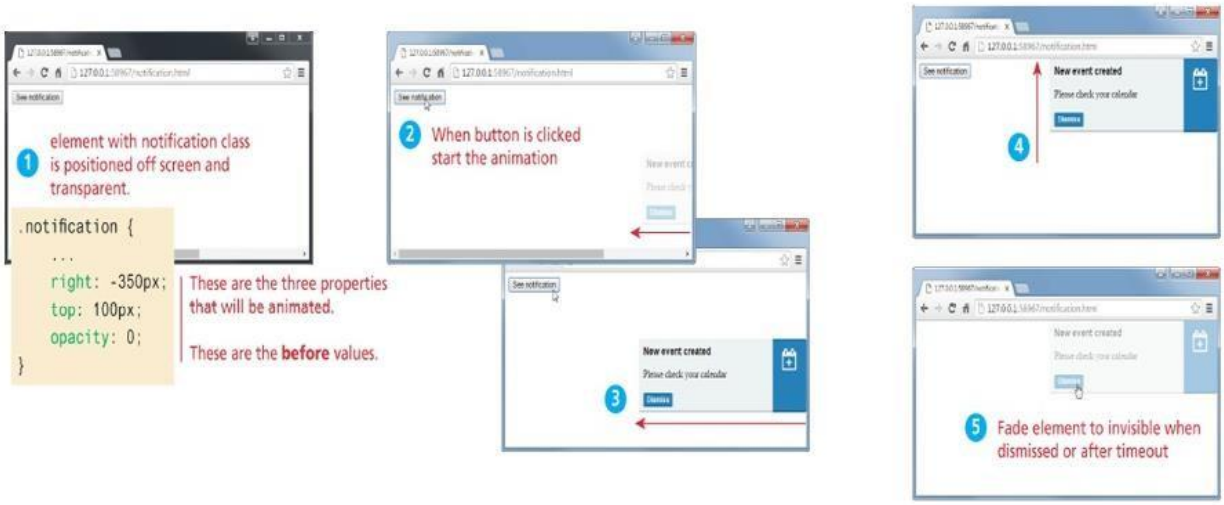
4 and then animate this property

After 4 seconds, call the dismiss notification function

```
function dismissNotification() {
    $('.notification').fadeOut(500);
    $('#notifyBtn').fadeOut(500);
}
```

5

Example-1: animate function code (taken from Fundamentals of Web Development second edition 2017)



Example-1: animate function output (taken from Fundamentals of Web Development second edition 2017)

Week – 8

Lesson 92 : Introduction to Ajax

Introduction to Ajax

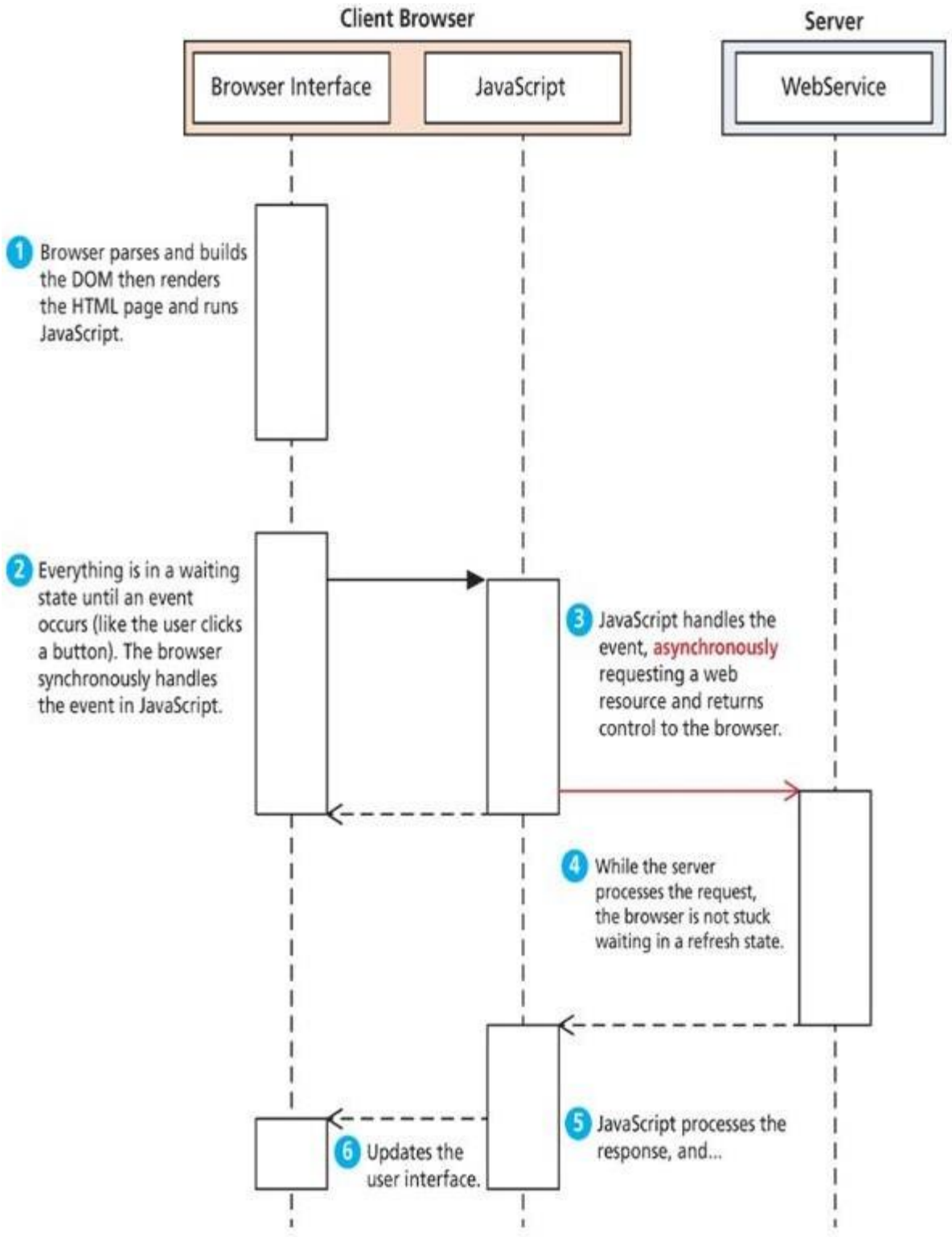
Ajax is a very important feature of JavaScript through which we fetch data from server while staying on client side.

AJAX is known as Asynchronous JavaScript and XML. Earlier it was originally using XML, but now it is using JSON format.

General Sequence of Events:

As shown in Figure 1;

1. In browser Dom will be created.
2. HTML document will be shown and html page will be listening for events(scroll events, click events, etc).



Example-1: UML sequence diagram of an AJAX request (taken from Fundamentals of Web Development second edition 2017)

Asynchronous Request:

When event occur and there is the need to fetch some data from server, then we sends asynchronous request to server and server send back the request with data. We will place that data in appropriate place and then html page layout will be updated.

Synchronous Request:

The page loaded is again sent back to server with query string or other way to get information. Server creates a new web page and send it back to us. For example; we want to change the time on a web page and send a request for it to server, server change the time along with recreate all the content of web page.

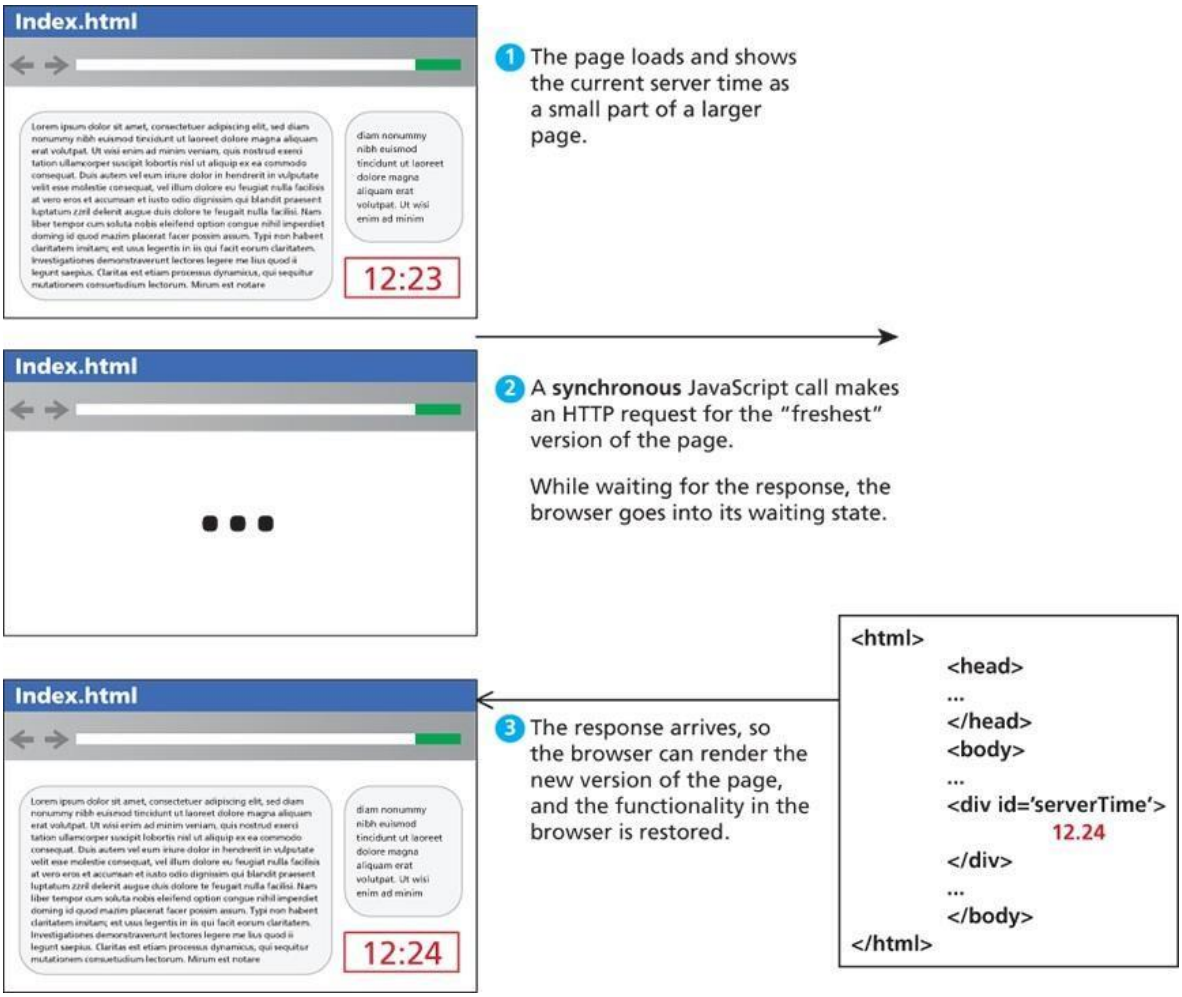


Figure 10.13 Illustration of a synchronous implementation of the server time web page (taken from Fundamentals of Web Development second edition 2017)

Ajax Asynchronous Example(What's Changed?):

In ajax we send request to change time on web page to server and it only sends us the updated time. We update that time on our web page and remaining content of web page are not changed. Ajax is very powerful in web pages because it is not updating whole page to change one element.

The mail pages, social media pages etc. are using ajax in their web pages. Just required data is fetched and updated on the browser page in ajax.



Figure 10.14 Illustration of an AJAX implementation of the server time web page (taken from Fundamentals of Web Development second edition 2017)

Making Asunchrounous Requests-load():

How we can we send asynchronous request through jQuery ?

load():

load() function is used to update content that we get from server.

$("#timeDiv").load("currentTime.php");

Here in JQuery load function will get the updated time from server-side script named currentTime.php in a single string and load that value asynchronously into the <div

id="timeDiv"> element.

Making Asynchronous Requests-GET:

$.get()

$.get() function is used to send request to get data from server. There is a dollar sign and dot before get, that will hint as it is not an element rather it is a jQuery Ajax function.

For example;

$.get("serviceTravelCountries.php?name=Italy");

Here a url and country name is giving as parameter in query string.

The Post Method:

$.post()

This method is used to send data on server and data is sent in parameter.

The POST Method-Form Serialization:

To send data from form to server using Ajax, we use serialization. In given code example, the serialize() method converts(encoded) the data of form into query string that would be stored in postData variable. And that variable is passed as parameter to post method to send data on server.

var postData = $("#someForm").serialize();

$.post("formHandler.php",postData);

Complete Control over Ajax:

Many things can be customized using Ajax. For example, which URL you want to access, what data is to send, whether it is asynchronous or synchronous type of request. In synchronous request you have to wait for the response of server then next statement could be executed. In asynchronous request, next statements could be run without waiting server response. In given code HTTP headers could be set in Ajax query. For example headers and referrer etc properties could be set.

$.ajax({ url: "vote.php",

data: $("#voteForm").serialize(), async:

true,

```
    type: post,
    headers: {“User-Agent” : “Homebrew Vote Engine”, “Referer”:
        “http://funwebdev.com”
        }
});
```

Code 1-Listing10.16 Adding headers to an AJAX post in jQuery

Cross-Origin Resource Sharing:

The Hackers sometimes use cross origin resource sharing for hacking. By default requests related to cross origin resource sharing are disabled and server does not entertain those requests. To deal with these requests servers require allowing Access-Control-Allow-Origin requests. Through this we can fetch data from different websites using Ajax. I on server side this request is not enabled then on client side error will be shown to not have permission to access data. Such type of error could be shown when you try to fetch data from different domains or even on same domain with different urls.

Lesson 93 : Asynchronous file transmission

Asynchronous file transmission

File transmission

The default way a file gets uploaded to the server is through the synchronous mode. In this mode, the user browses and select the file and upload it by pressing the **submit** button

But in Ajax, a file can be uploaded to a server in asynchronous mode as shown below

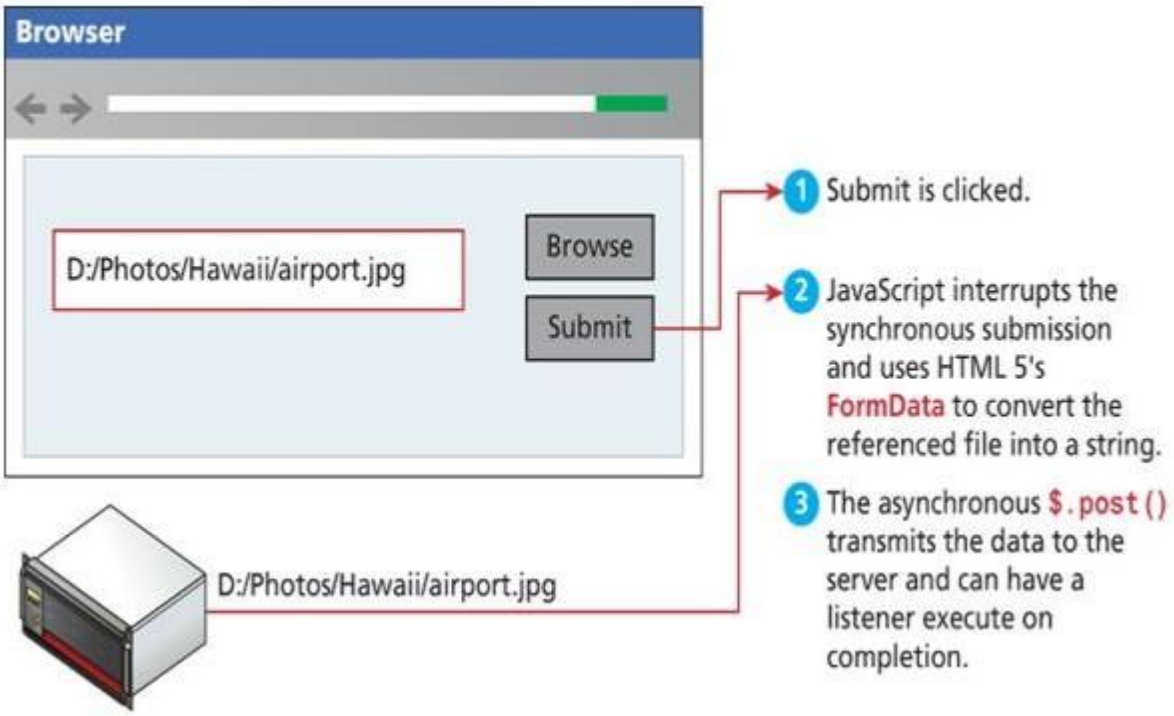


Figure: Posting a file using Form data

(Taken from fundamentals of Web Development Second Edition)

The following code uses the form data interface to post files asynchronously.

```
function uploadFile () {
  // get the file as a string
  var formData = new FormData($("#fileUpload")[0]);
  var xhr = new XMLHttpRequest();
  xhr.addEventListener("load", transferComplete, false);
  xhr.addEventListener("error", transferFailed, false);
  xhr.addEventListener("abort", transferCanceled, false);
  xhr.open('POST', 'upload.php', true);
  xhr.send(formData);                    // actually send the form da
  function transferComplete(evt) {     // stylized upload complet
     $("#progress").css("width","100%");
     $("#progress").html("100%");
  }
  function transferFailed(evt) {
     alert("An error occurred while transferring the file.");
  }
  function transferCanceled(evt) {
     alert("The transfer has been canceled by the user.");
  }
}
```

Example: Using the form data interface to post files asynchronously

(Taken from fundamentals of Web Development Second Edition)

In the upload File() function, a built-in JavaScript form object Form Data () is called which receives the name of the file to be uploaded.

Lesson 94 : Introduction to Server-Side Development; Comparison of Server-side technologies

Introduction:

Up till now we have covered only the client-side server technologies like HTML, CSS, JavaScript, etc. whose primary focus was on creating static web pages. Now in the server-side technology like PHP, ASP.NET, Python, JSP, etc. focuses shift on creating dynamic web pages and web sites from which the user can interact.

Client Site Scripts:

Following figure shows the execution of client site scripts

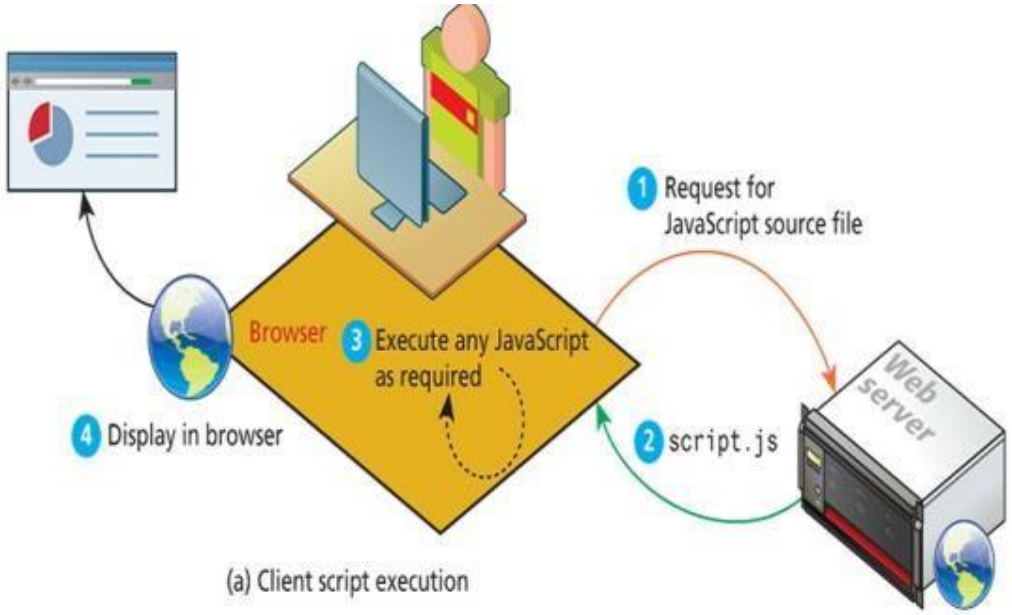


Figure: Execution of client site scripts

(Taken from fundamentals of Web Development Second Edition)

Server Site Scripts:

Similarly, the execution of the server site scripts has been illustrated in the following figure.

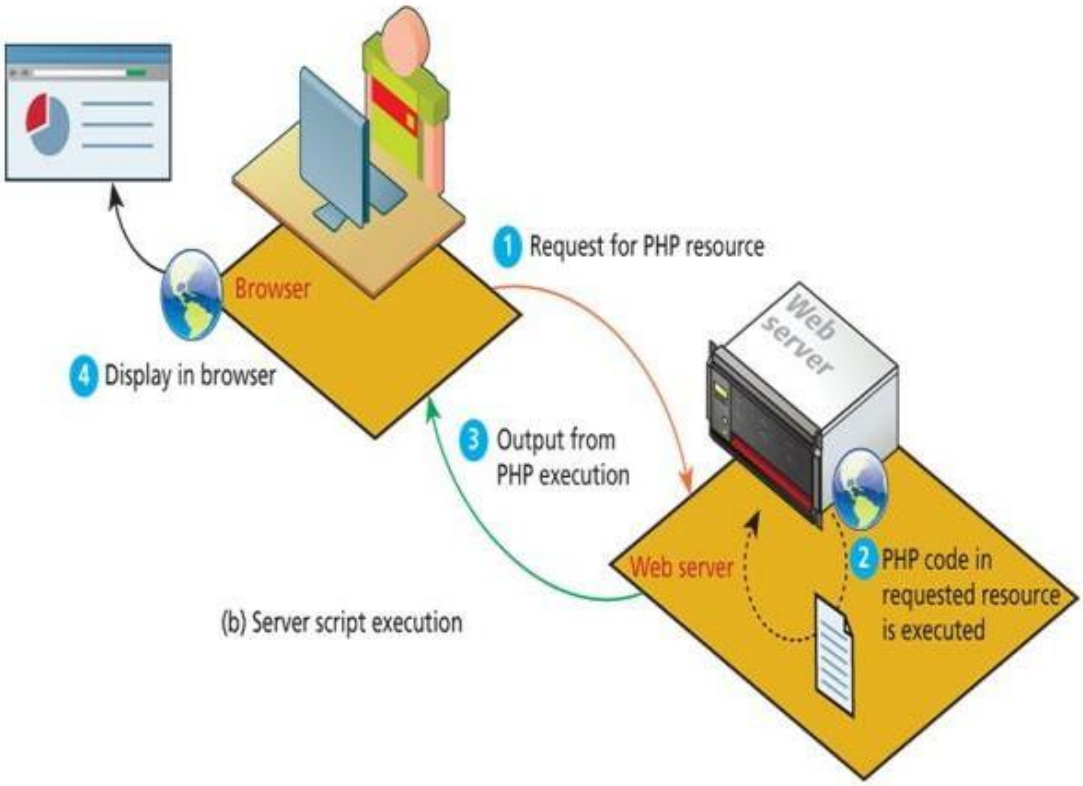


Figure: Execution of server site scripts

(Taken from fundamentals of Web Development Second Edition)

Server -Side Script Resources:

A server-side script running on a webserver can use or access multiple resources made available to it to complete its process. There resources may include a database, local files, webservices, mail server, or a different software, etc.

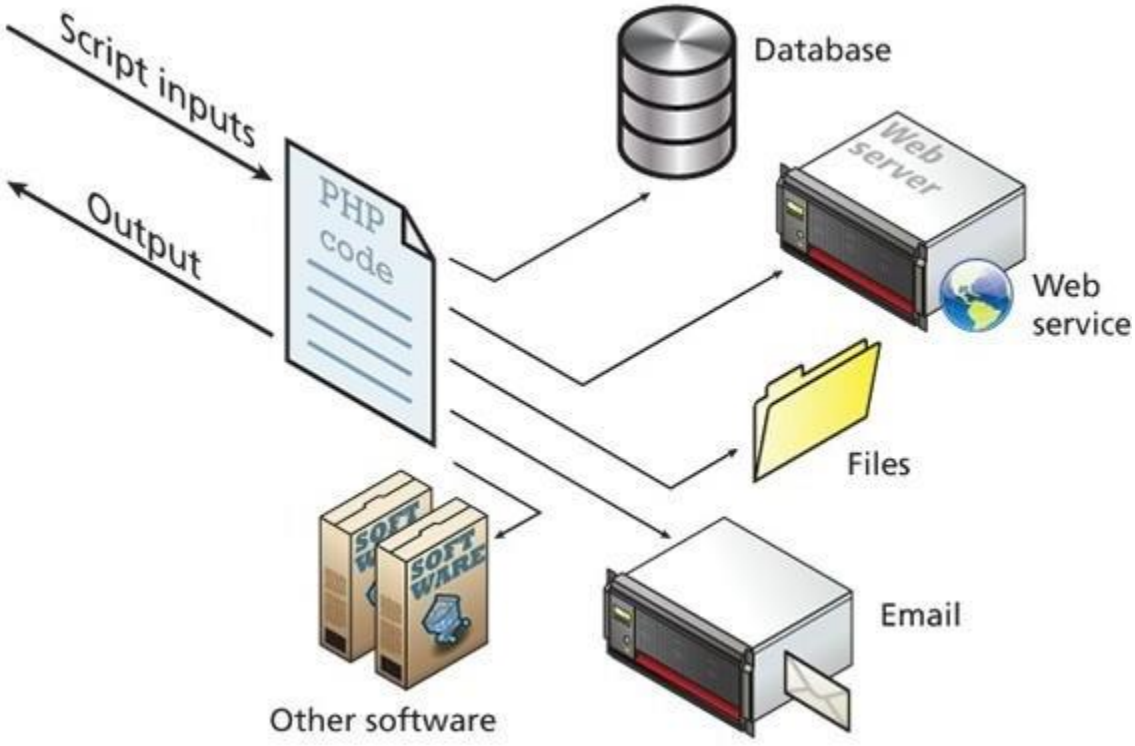


Figure: Server scripts may have access to many resources

(Taken from fundamentals of Web Development Second Edition)

Comparing Server-Side Technologies:

ASP.NET

It is a framework in which C# language is used to create web sites.

JSP

Java Sever Pages uses java language as its core to create dynamic contents.

Perl

Perl is historically one of the earliest languages to create dynamic web contents.

Python

Python is an object-oriented programming language and uses framework like Django, flask to create dynamic web page contents

Ruby on Rails

Ruby is a programming language which uses Ruby on Rails a web development framework for developing websites.

Node.js

Node.js is an open-source platform that uses java script language to develop dynamic contents.

PHP

PHP (Hyper Text Preprocessor) is one of the most popular scripting languages that used to generate dynamic web contents. In this course we will be using the PHP language to create dynamic webpages.

Popularity of PHP:

In terms of popularity, the most popular programming language in creating dynamic websites is PHP. In fact, if we looked into the world wide about 79 percent of the websites uses PHP language.

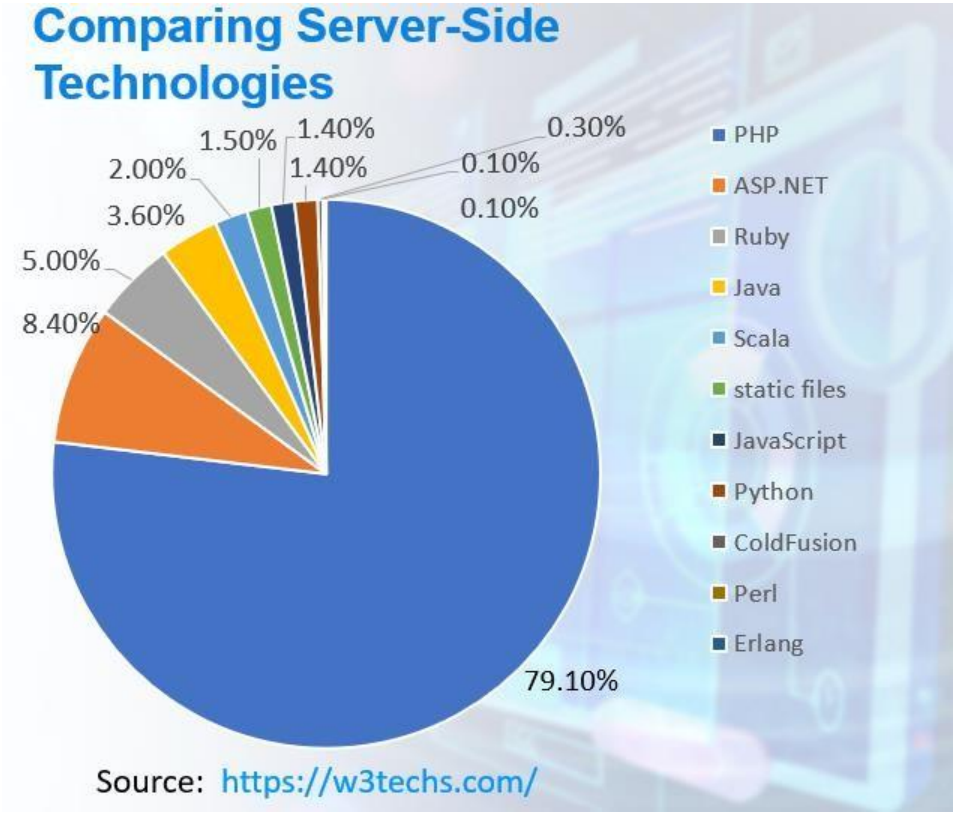


Figure: Popularity of PHP language
Taken from .ppt slides

Comparison of two Server-side scripting languages:

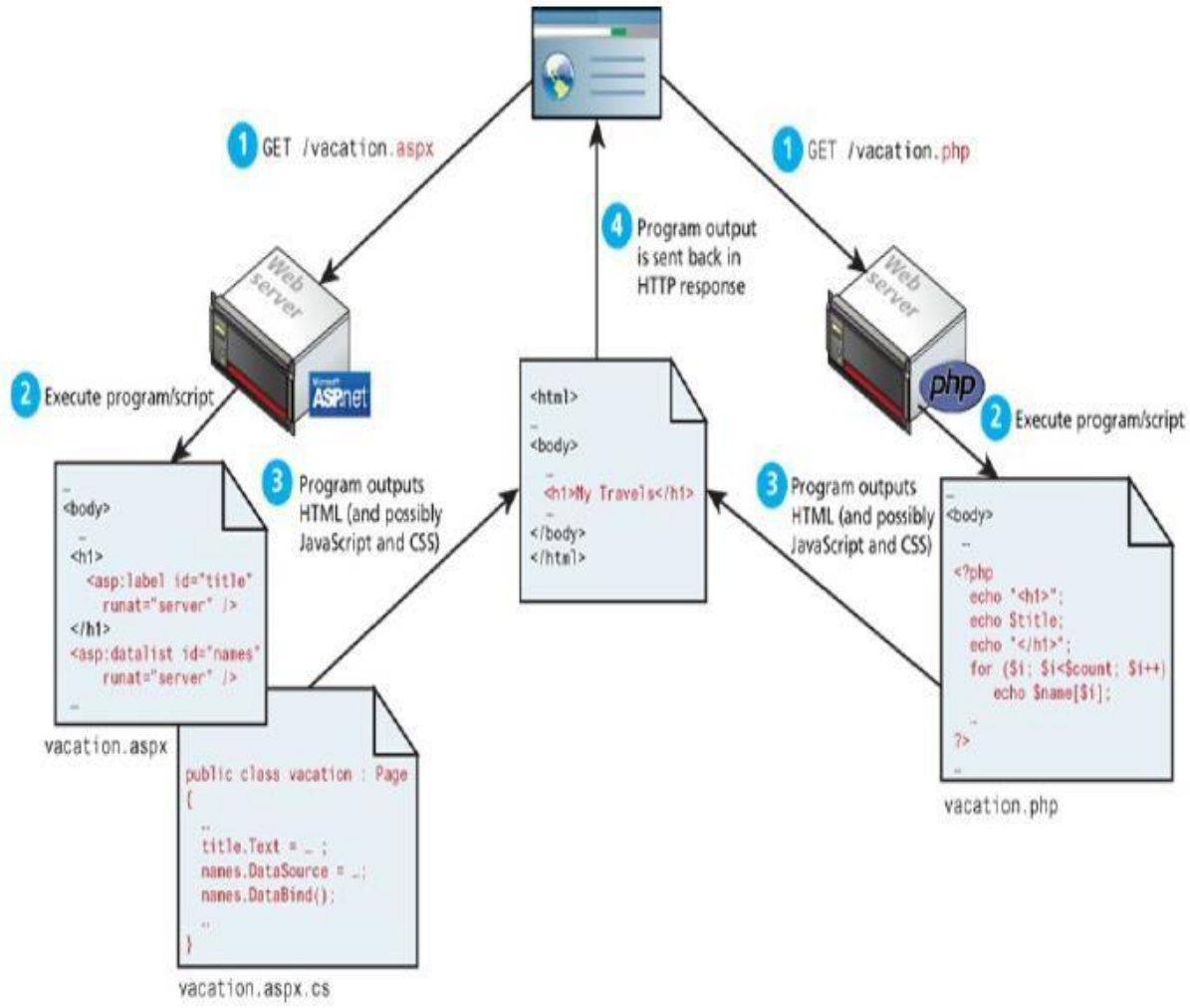


Figure: Comparison of two server-side scripting language
(Taken from Websites fundamentals of Web Development Second Edition)

Lesson 95 : Installation of Apache webserver, PHP and MySQL for local development

Installation of Apache webserver, PHP and MySQL for local development

When we want to start server-side development, a set of software's are required which is also called as software stack. Minimum software's included in the Software stack are Operating System, Webserver, Programming language, Database. etc.

In this course we will be using the XAMMP server which includes the minimum number of software's to develop websites.

The XAMMP server can be downloaded from the following link:

https://www.apachefriends.org/download.html

Lesson 96 : Introduction to PHP

Introduction to PHP

Introduction:

PHP is a very simple programming language and like JavaScript it is a dynamically typed language. Dynamically typed means that values of the variables in the programming language are checked at runtime. PHP also supports the object-oriented language features like C++, C#, Java etc. It provides object-oriented features like classes which is also the back bone of object- oriented programming.

PHP Tags:

PHP language provides a facility that its code can be embedded directly with in an HTML file. However, instead of having an **.html** extension, a PHP file will usually have the extension **.php** HTML, CSS, JavaScript code can also be inserted into the PHP file. PHP programming code must be contained within an opening <?php tag and a matching closing?>tag in order to differentiate it from the HTML. The programming code within the <?php and the ?> tags is interpreted and executed, while any code outside the tags is echoed directly out to the client.

Variables:

PHP Variables are dynamically typed, which means that the programmer does not have to declare the data type of variable and it is determined at run time. PHP variables are also loosely type and can be assigned different data types over time.

To declare a variable in PHP a dollar sign ($) must be placed before the name of the variable indicating to the PHP engine to handle it as a variable. Variables in PHP are assigned value from right to left i.e.

```
<?php

$var = 10;

?>
```

It should also be noted that name of the PHP variables is case sensitive i.e., variables $var and $Var are treated as two different variables in PHP. In PHP, variable names can also contain the underscore character, which is useful for readability reasons.

Constant:

In PHP, a constant is somewhat similar to a variable, except a constant's value never changes, in other words it says constant. A constant in PHP can be defined using the **define()** function and it should be declared at the top of the PHP file. The define() function normally takes two parameters; the name of the constant and second its value. The standard programming convention for naming constant should be in uppercase. It should also be noted that constants in PHP should be used without quotes or ($) sign.

<?php

define ("DATABASE_LOCAL", "local host"); echo

DATABASE_LOCAL;

?>

Data types:

- **Boolean** A logical true or false value
- **Integer** Whole Numbers
- **Float** Decimal Number
- **String** Letters
- **Array** A collection of data of any type
- **Object** Instances of classes

Escaping Characters:

String literals in PHP can be defined using either the single quote or the double quote character. Single quotes define everything exactly as is, and no escape sequences are expanded. If you use double quotes, then you can specify escape sequences using the backslash. For instance, the string "Good\n Morning" contains a newline character between the two words since it uses double quotes, but would actually output the slash n were it enclosed in single quotes.

The following table shows the String Escape Sequences

Sequence	Description
\n	Line Feed
\t	Horizontal tab
\\	Back Slash
\$	Dollar Sign
\"	Double quote

\’	Single quote

Table: String Escape Sequence

(Taken from Websites fundamentals of Web Development Second Edition)

Output using echo and print:

In PHP, pages are programs that output HTML. To output something that will be seen by the browser, you can use the echo () function.

```
<?php
    echo (“hello”);
?>
```

There is also an equivalent shortcut version that does not require the parentheses.

```
<?php
    echo “hello”;
?>
```

Similarly, output can also be generated through the use of print statement.

```
<?php
    print(“Hello”);
?>
```

Output using printf:

In PHP, output can also be displayed using printf() function. The printf() function is derived from the same-named function in the C programming language. The function takes at least one parameter, which is a string and that string optionally take references parameters, which are then integrated into the first string by placeholder substitution.

The printf() function also allows a developer to apply special formatting, for instance, specific date/time formats or number of decimal places.

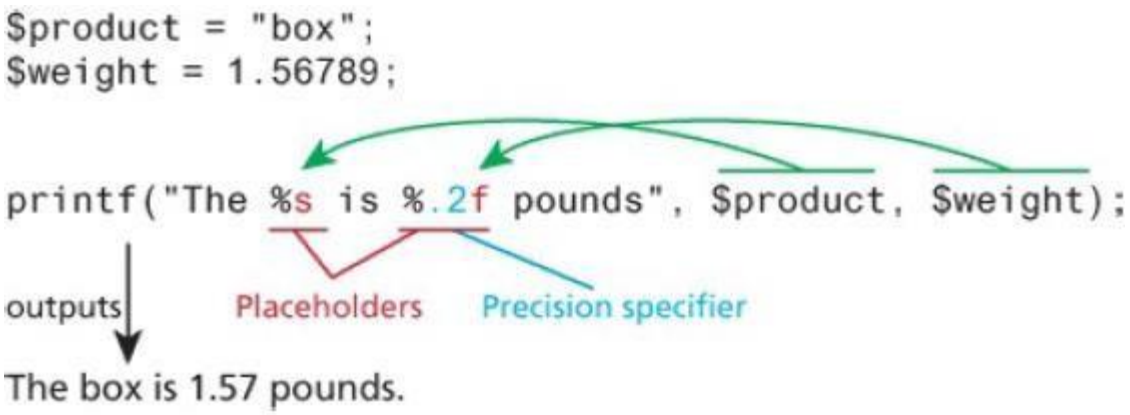


Figure: Illustration of components in a printf statement and output

(Taken from Websites fundamentals of Web Development Second Edition)

Lesson 97 : Introduction to PHP (String Concatenation)

Introduction to PHP – String Concatenation

In PHP, strings can easily be appended together using the concatenation operator, which is the period (.) symbol. For example in the following code

```
<?php

$username = “Ahmed”; echo
“Hello” . $username;
// outputs Hello Ahmed

?>
```

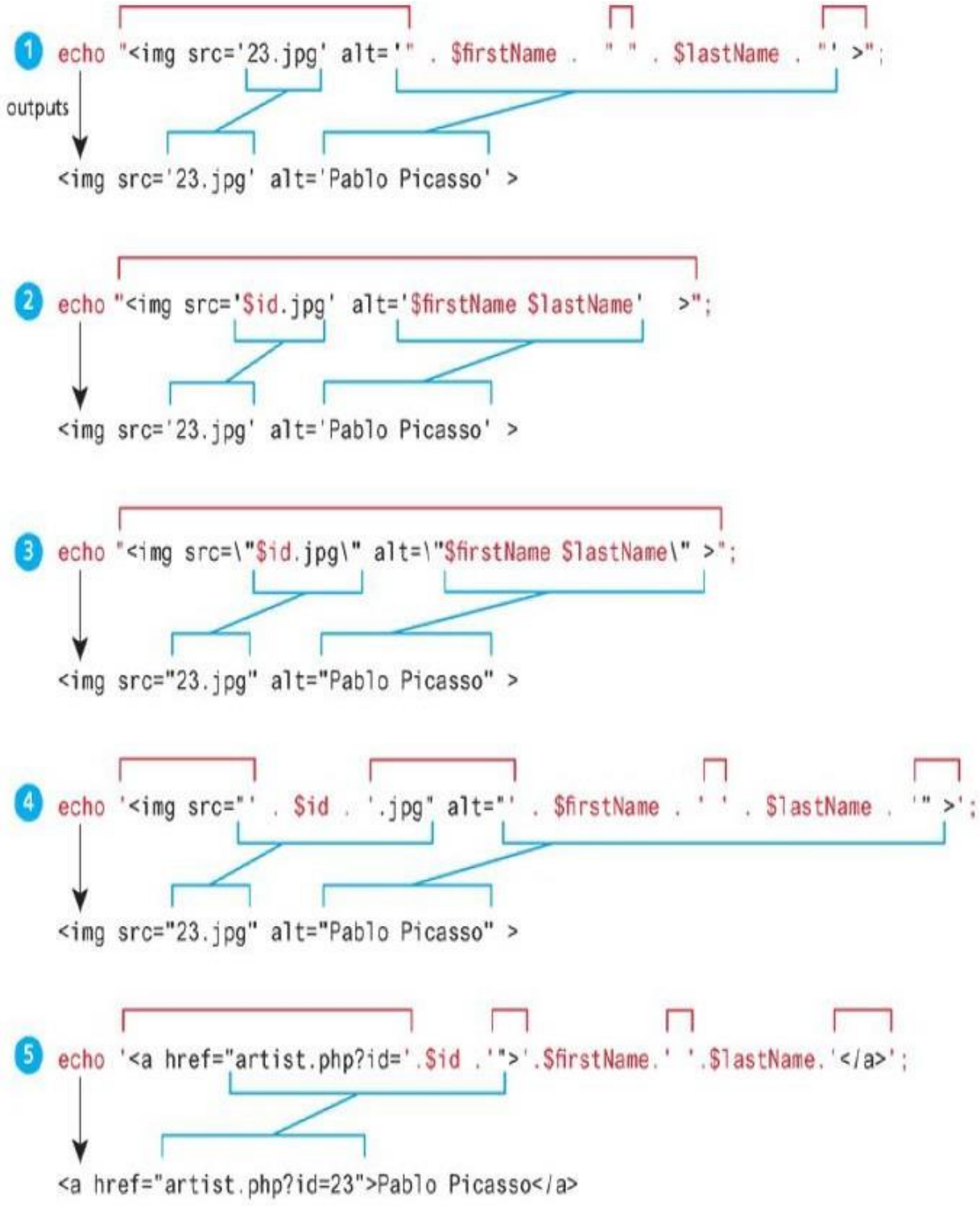


Figure: More Complicated Concatenation Examples explain
(Taken from Websites fundamentals of Web Development Second Edition)

Lesson 98 : Program Control - Conditional Program Statements

Program Control – Conditionals

Just like in any other programming language PHP also uses the conditional statements in it except for few differences.

Comparison Operators:

In PHP, when doing comparison, either you can use the Equal to operator "==" or the Identical operator "===".

- In the Equal to operator "==", two equal operators are used i.e., 5 == '5'; // true

  In PHP, the equal to operator always check the equality of two operands. So, in the above example, PHP will internally treat number 5 without quotes as integer and number 5 with quotes as string and its comparison will be resulted as true which might lead to some logical error.

- In the Identical operator "===", three equal operators are used i.e. 5 === '5' // false

In PHP, the Identical operator check the equality of both the operands and their data types So, In the above example, the comparison will return as false.

if ..else:

The syntax for conditionals statements in PHP is identical to that of JavaScript. In this syntax the condition to test is contained within () brackets with the body contained in {} blocks. Optional else if statements can follow, with an optional else ending the branch.

```
// if statement
if ( $hourOfDay > 6 && $hourOfDay < 12) {
  $greeting = "Good Morning";
}
else if ($hourOfDay == 12) {  // optional else if
  $greeting = "Good Noon Time";
}
else {                        // optional else branch
  $greeting = "Good Afternoon or Evening";
}
```

Example: Conditional snippet of code using if ..else

(Taken from Websites fundamentals of Web Development Second Edition)

Alternative Syntax for Control Structure:

It is also possible to place the body of an if or an else outside of PHP. For instance, in the below figure, an alternate form of an if … else is illustrated (along with its equivalent PHP-only form).

```
<?php  if ($userStatus == "loggedin") {  ?>
  <a href="account.php">Account</a>
  <a href="logout.php">Logout</a>
<?php  } else {  ?>
  <a href="login.php">Login</a>
  <a href="register.php">Register</a>
<?php  }  ?>
```

Example: Combining PHP and HTML in the same script
(Taken from Websites fundamentals of Web Development Second Edition)

switch … case:

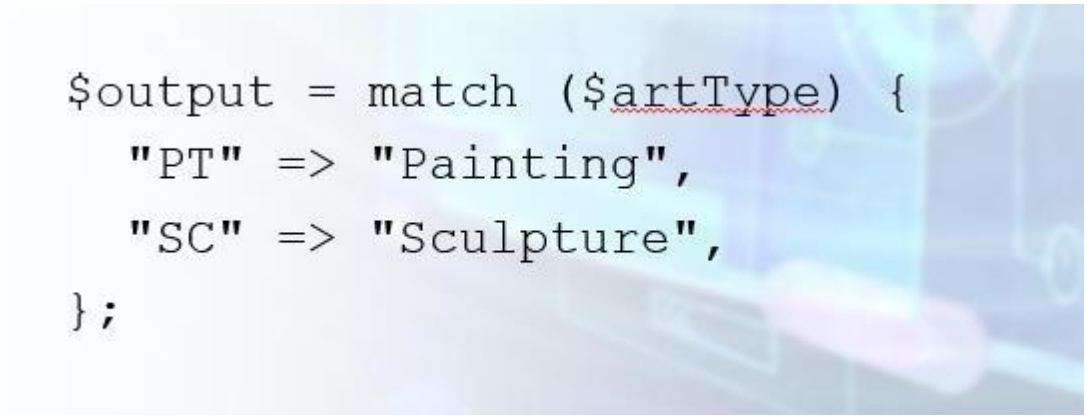
The switch statement is similar to a series of if ... else statements. An example using switch statement is given below:

```
switch ($artType) {
   case "PT":
      $output = "Painting";
      break;
   case "SC":
      $output = "Sculpture";
      break;
   default:
      $output = "Other";
}
```

Example: Conditional statement using switch statement
(Taken from Websites fundamentals of Web Development Second Edition)

match:

PHP 8 introduces a new match expression statement in which multiple conditionals are evaluated and returns a value. Match expression statement is somewhat similar to switch statement in PHP, except for few differences. In match statement, break statement is not required, it uses identical operator also no default case is required.



```
$output = match ($artType) {
    "PT" => "Painting",
    "SC" => "Sculpture",
};
```

Figure: match statement
(Taken from .ppt slides)

Lesson 99 : Program Control – Loops

Program Control – Loops

PHP also supports the same type of loops in any other programming languages. i.e., while loop, for loop and do while loop.

While and do while loop:

The while loop and the do … while loop are quite similar. Both will execute nested statements repeatedly as long as the while expression evaluates to true. In the while loop, the condition is tested at the beginning of the loop; in the do … while loop the condition is tested at the end of each iteration of the loop.

```
$count = 0;
while  ($count < 10)
{
   echo $count;
   $count++;
}

$count = 0;
do
{
   echo $count;
   // this one increments the count by 2 each time
   $count = $count + 2;
} while ($count < 10);
```

Example: while and do while loop

(Taken from Websites fundamentals of Web Development Second Edition)

for loop:

The for loop in PHP has the same syntax as the for loop in JavaScript. The for loop contains the same loop initialization, condition, and post loop operations as in JavaScript.

```
// this one increments the count by 1 each time
for ($count=0; $count < 10; $count++)
{
   echo $count;
}
// this one increments the value by 5 each time
```

Example: the for loops

(Taken from Websites fundamentals of Web Development Second Edition)

Include Files:

PHP does have one important facility that is unlike most other non-web programming languages, namely, the ability to include or insert content from one file into another Almost every PHP page beyond simple practice exercises makes use of this include facility. Include files provide a mechanism for reusing both markup and PHP code, as shown in the following figure.

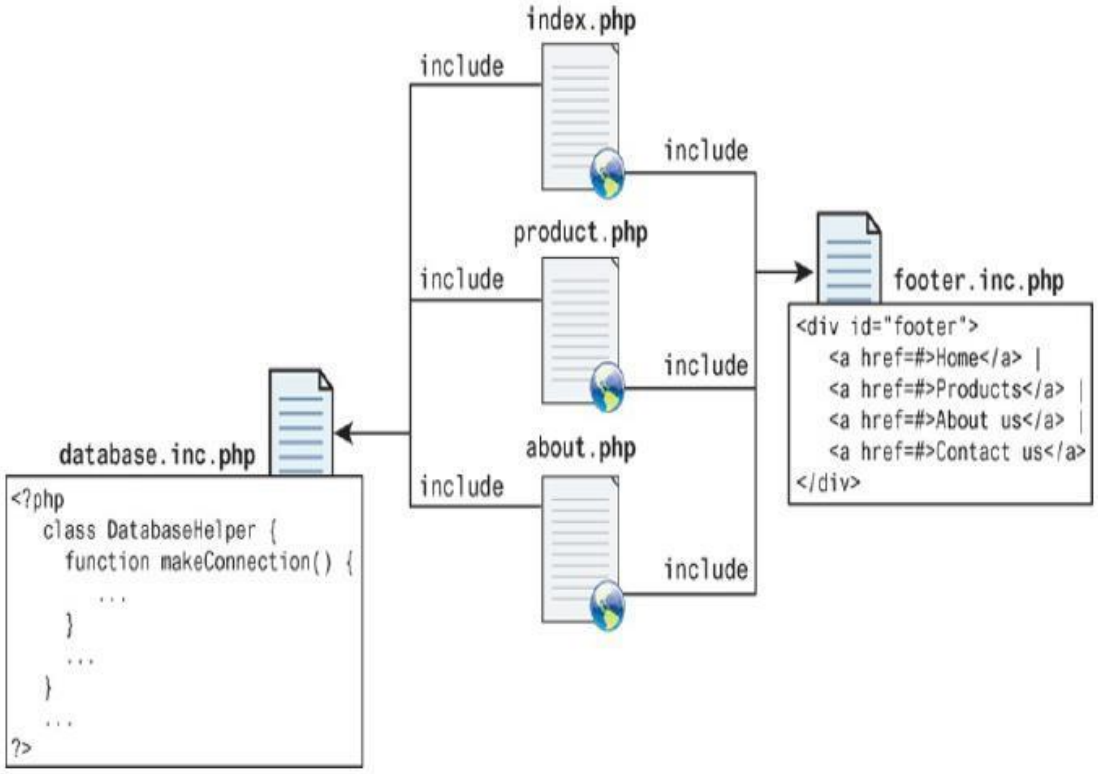


Figure: The include files
(Taken from Websites fundamentals of Web Development Second Edition)

Examples of Include files

- **Include** "somefile.php"
- **Include_once** "somefile.php"

Lesson 100 : Functions in PHP (1)

Topic: 100

Functions:

- Function starts with the keyword **function**
- Can take zero or more parameters
- May or may not return a value

```
function square($num) {
  return $num*$num;
}
```

$sq = square(5);

Function Syntax – Return Type Declaration

- A Return Type Declaration explicitly defines a function's return type by adding a colon and the return type after the parameter list when defining a function

function mustReturnString() **: string** { return

"hello";

}

Parameters – Passing Parameters by Reference

- By default, arguments passed to functions are **passed by value** in PHP
- PHP also allows arguments to functions to be passed by reference, which will allow a function to change the contents of a passed variable
- The mechanism in PHP to specify that a parameter is passed by reference is to add an ampersand (&) symbol next to the parameter name in the function declaration

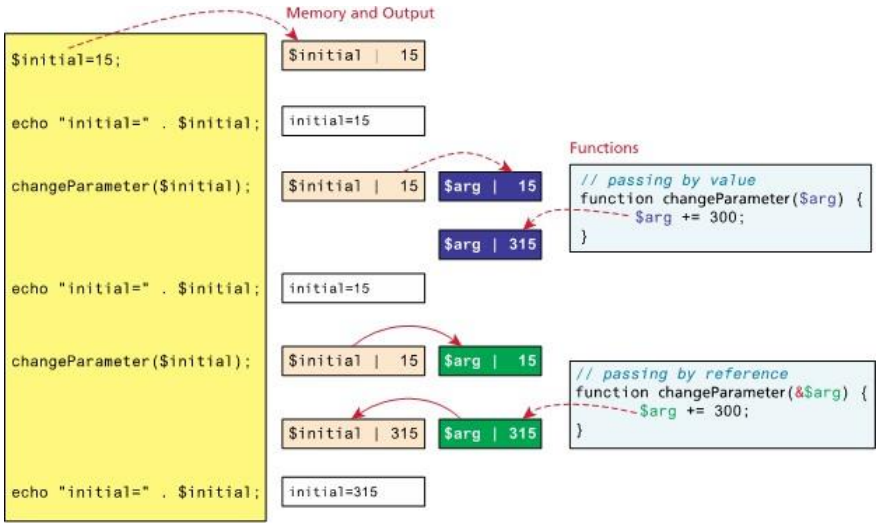


Figure taken from: Fundamentals of Web Development (Second Edition, Pearson), Authors: Randy Connolly, Ricardo Hoar

Lesson 101 : Functions in PHP (2)

Functions:

Parameters – Default Values

Parameters can have default values for example consider the following function. Here default values of variables a, b, and c are 1, 2 and 3. So if the function is called without any parameter, these default values will be used.

```
function func($a=1, $b=2, $c=3) {
  echo $a." ".$b." ".$c;
}
func(5,10,15);       //5 10 15
func(5,10);           //5 10 3
func();                 //1 2 3
```

Variable Scope within Functions

Variables defined within a function have function scope. While variables defined in the main script are said to have global scope, by default, not available within functions. PHP does allow variables with global scope to be accessed within a function using the **global** keyword.

```
$x=10;
function func() { global
  $x;
  echo $x;
}
```

Lesson 102 : Arrays in PHP

Arrays:

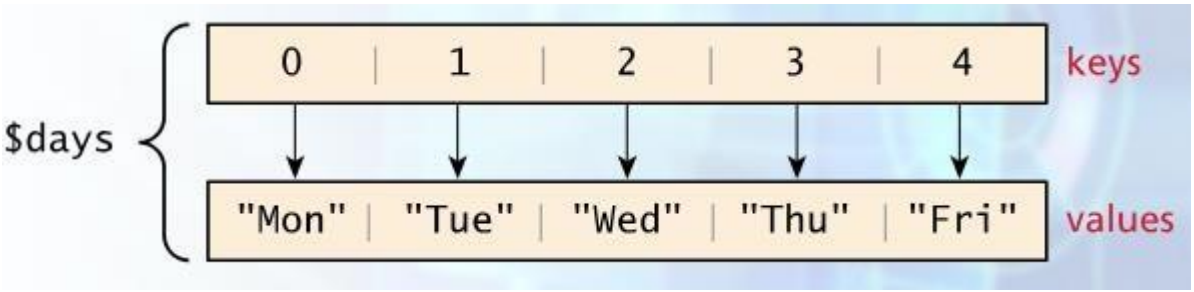
Defining and Accessing an Array

Using array function:

```
$days = array("Mon","Tue","Wed","Thu","Fri");
```

Using array notation:

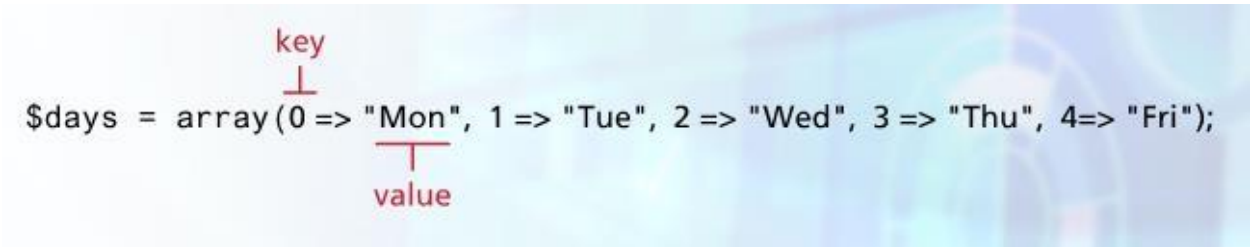
```
$days = ["Mon","Tue","Wed","Thu","Fri"];
```



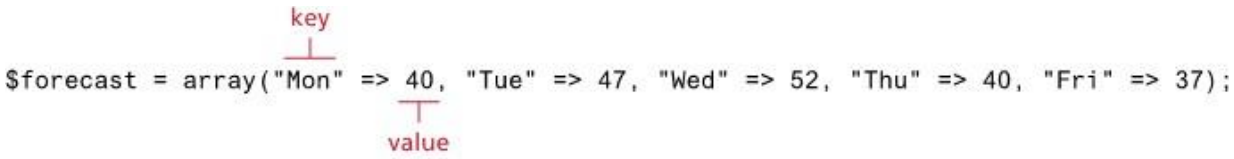
- For debugging, use *var_dump($days)* or *print_r($days)* to see contents of the array (or any variable). Place output in <pre></pre>

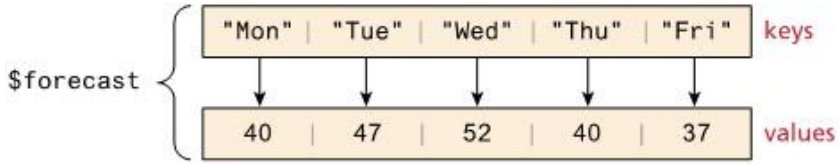
<pre><?php var_dump($days); ?></pre>

All arrays in PHP are generally referred to as **associative arrays**



You can use integer and string keys, not necessarily in order






```
echo $forecast["Tue"];  // outputs 47
echo $forecast["Thu"];  // outputs 40
```

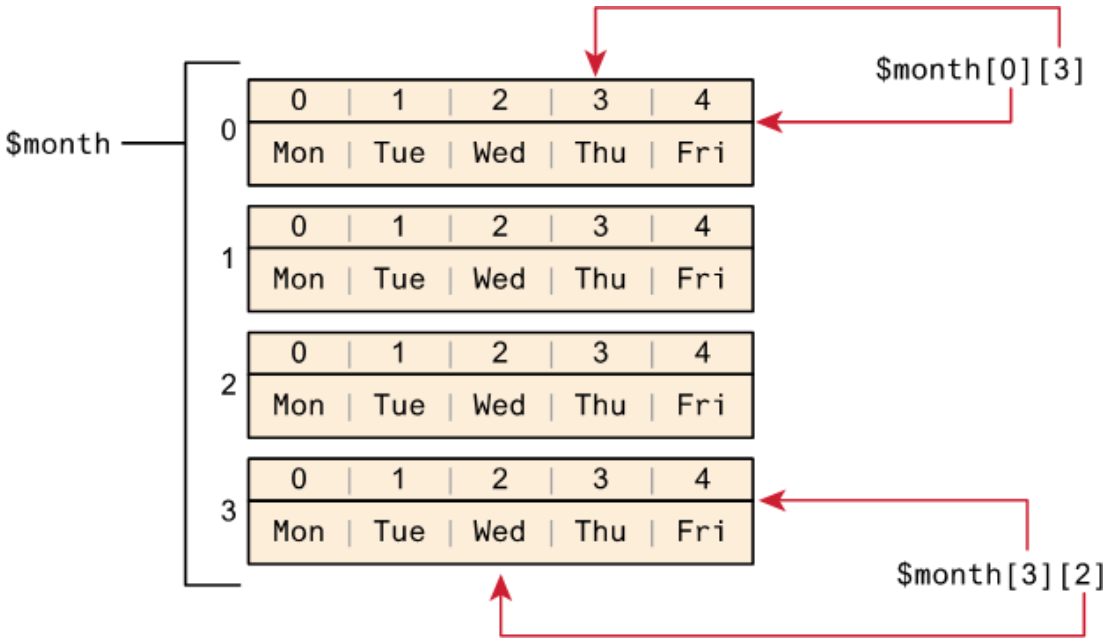
Multidimensional Arrays

A multidimensional array is an array of arrays as follows:

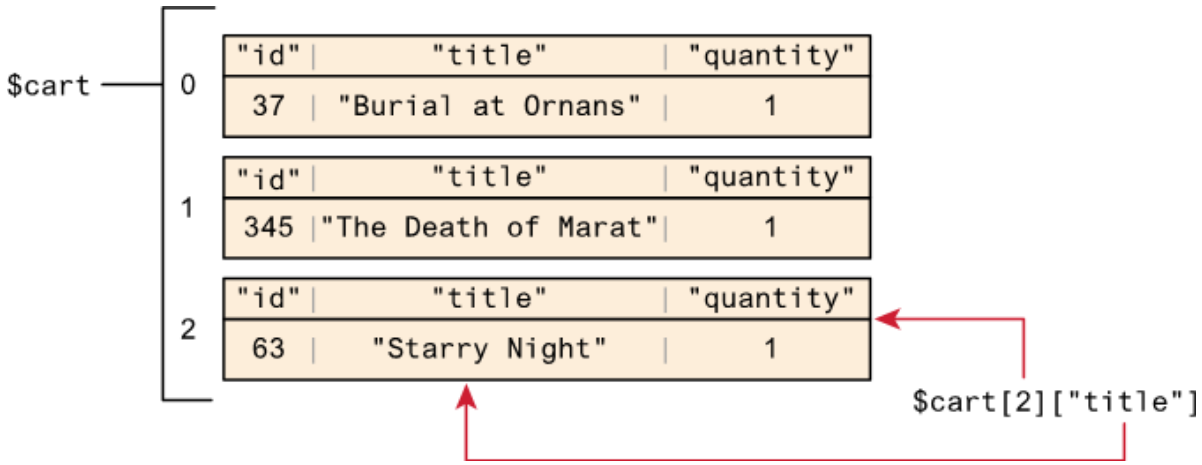
$month = array (

  array("Mon","Tue","Wed","Thu","Fri"),

```
  array("Mon","Tue","Wed","Thu","Fri"),
  array("Mon","Tue","Wed","Thu","Fri"),
  array("Mon","Tue","Wed","Thu","Fri")
);
```




```
$cart = array();
$cart[] = array("id" => 37, "title" => "Burial at Ornans", quantity" => 1);
$cart[] = array("id" => 345, "title" => "The Death of Marat", "quantity" => 1);
$cart[] = array("id" => 63, "title" => "Starry Night", "quantity" => 1);
```



Iterating through an Array - for

for loop can be used to iterate through an array as follows:

```
for ($i=0; $i<count($days); $i++) {
  echo $days[$i] . "<br>";
}
```

Iterating through an Array - foreach

Similarly can also be used to iterate through an array as follows:

```
//iterating through the values
foreach ($forecast as $value) {
echo $value . "<br>";
}
// iterating through the values AND the keys
foreach ($forecast as $key => $value) { echo
"day[" . $key . "]=" . $value;
}
```

Adding and Deleting Elements

An element can be added to an array simply by using a key/index that hasn't been used, as shown below:

$days[5]= "Sat";

As an alternative to specifying the index, a new element can be added to the end of any array using empty square brackets after the array name, as follows:

$days[]= "Sun";

Delete with unset()

Lesson 103 : Sorting arrays

Arrays Sorting

Arrays can be sorted using “sort” and “assort” as follows:

sort($days);

As the values are all strings, the resulting array would be:

Array ([0] => Fri [1] => Mon [2] => Sat [3] => Sun [4] => Thu [5] => Tue [6] => Wed)

asort($days);

The resulting array in this case keeps associations so is:

Array ([4] => Fri [0] => Mon [5] => Sat [6] => Sun [3] => Thu [1] => Tue [2] => Wed)

Here are some more arrays related functions:

array_keys($someArray)

array_values($someArray)

array_rand($someArray, $num=1)

shuffle($someArray) array_reverse($someArray)

array_walk($someArray, $callback, $optionalParam)

in_array($needle, $haystack, $optionalStrict)

Lesson 104 : Arrays - Super Globals

Arrays – Super Globals

PHP uses special predefined associative arrays called **superglobal variables** that allow the programmer to easily access HTTP headers, query string parameters, and other commonly needed information.

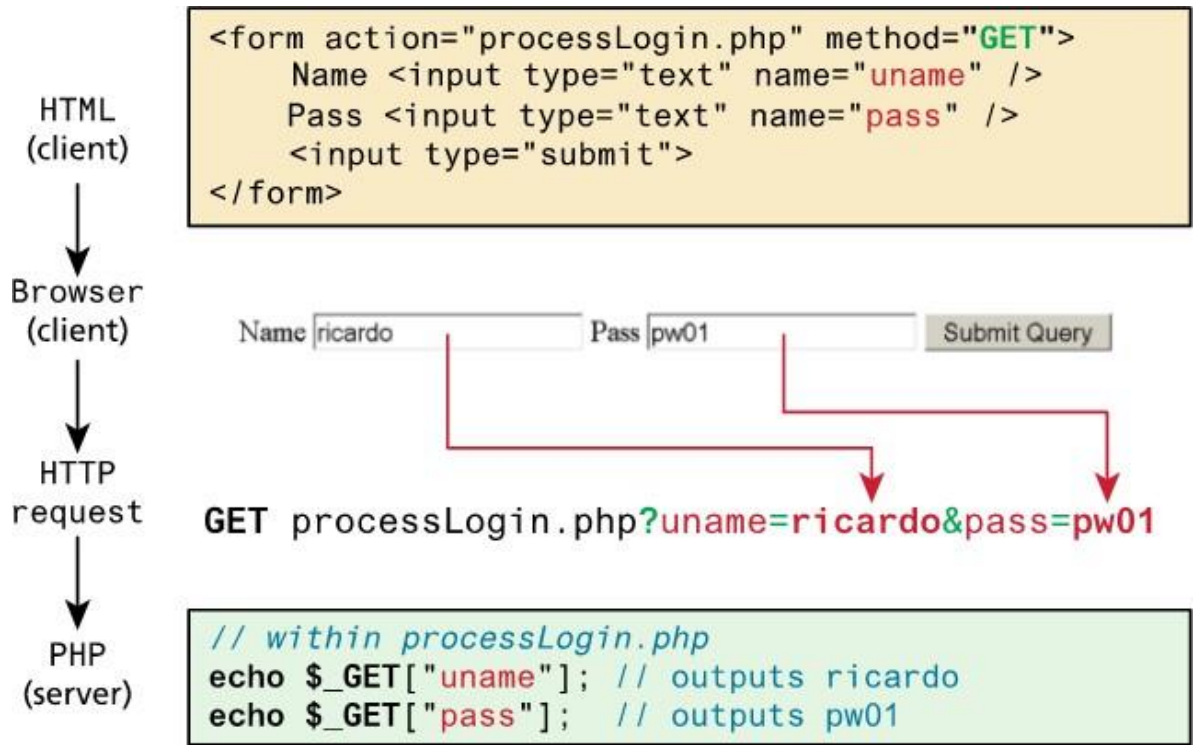
Here are some super global arrays along with the task it is used for:

- $GLOBALS
    - Array for reading and storing data that needs superglobal scope
- $_COOKIES
    - Array of cookie data passed to page via HTTP request
- $_SESSION
    - Array that contains session data
- $_ENV
    - Array of server environment data (mostly OS related and paths)
- $_SERVER
    - Array containing information about the request and the server
- $_ENV
    - Array of server environment data (mostly OS related and paths)
- $_SERVER
    - Array containing information about the request and the server

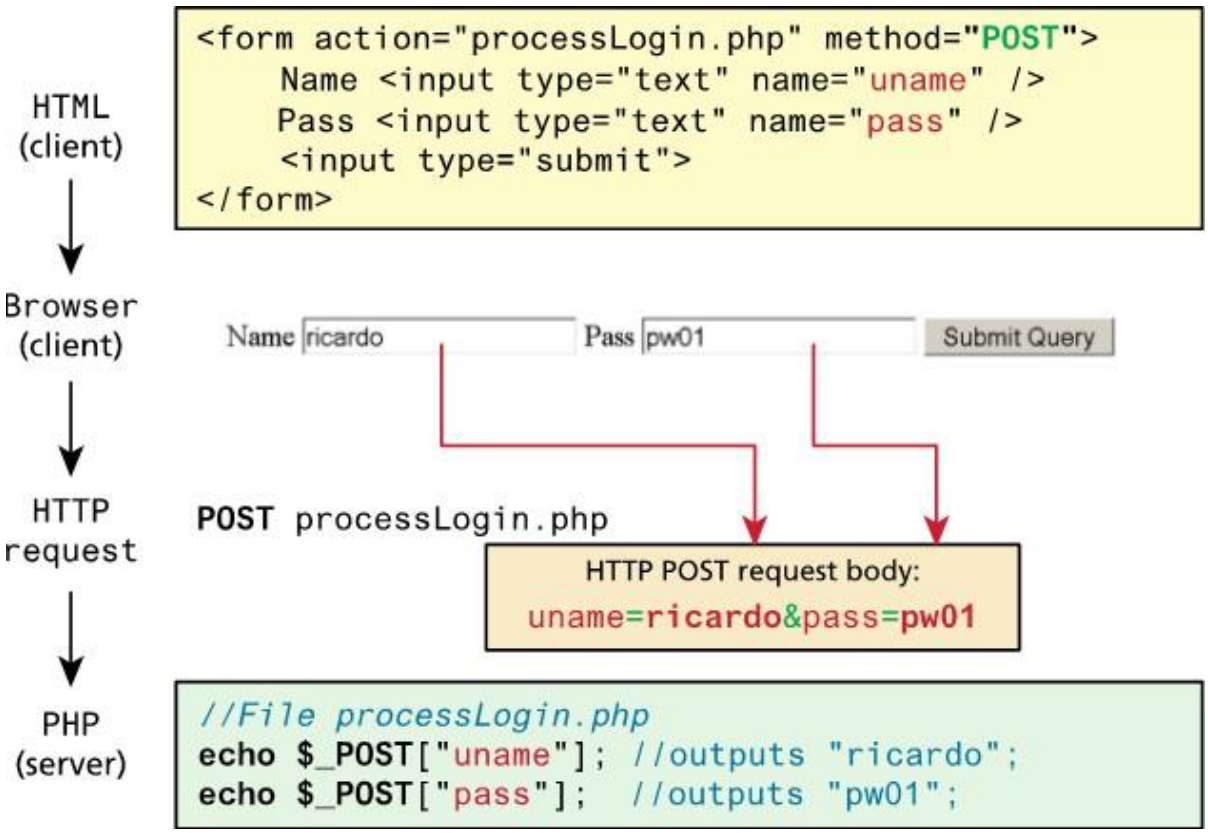
Lesson 105 : GETandGETand_POST arrays

$_GET and $_POST superglobal arrays

Both $_GET and $_POST superglobal arrays can be used to get the data (submitted through HTML form) for further processing as shown in figure below:



Above figure shows about how we can extract data using $_GET array. Whereas the figure below shows the use of $_POST for getting the data.



Sanitizing Query Strings

Note that just because you are expecting a proper query string, it doesn't mean that you are going to get one. Your program must be able to handle:

- If query string parameter doesn't exist
- If query string parameter doesn't contain a value
- If query string parameter value isn't the correct type or is out of acceptable range
- If value is required for a database lookup, but provided value doesn't exist in the database table

Lesson 106 : GETandGETand_POST arrays – Example

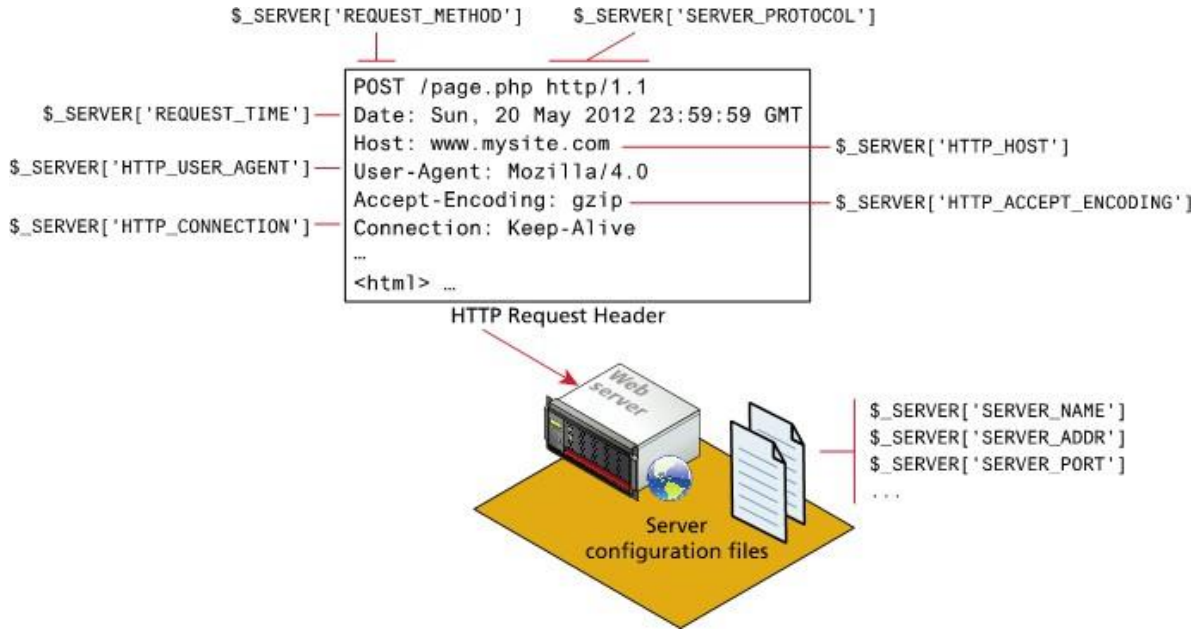
$_GET and $_POST hands on practice:

Please refer to video tutorial of topic 106.

Lesson 107 : $_SERVER array

$_SERVER Array

Contains all the necessary information related to the current server. Lets look at the figure below



Reference: figure taken from: Fundamentals of Web Development (Second Edition, Pearson), Authors: Randy Connolly, Ricardo Hoar

Request Header Information Keys:

Note that for advanced browser detection we need to get browscap.ini and configure php.ini [browscap] to use it.

```
<?php

echo $_SERVER['HTTP_USER_AGENT'];

$browser = get_browser($_SERVER['HTTP_USER_AGENT'], true); print_r($browser);

?>

$previousPage = $_SERVER['HTTP_REFERER'];

// Check to see if referer was our search page if

(strpos($previousPage,"search.php") != 0) {

  echo "<a href='search.php'>Back to search</a>";

}
```

// Rest of HTML output

Lesson 108 : $_FILES array

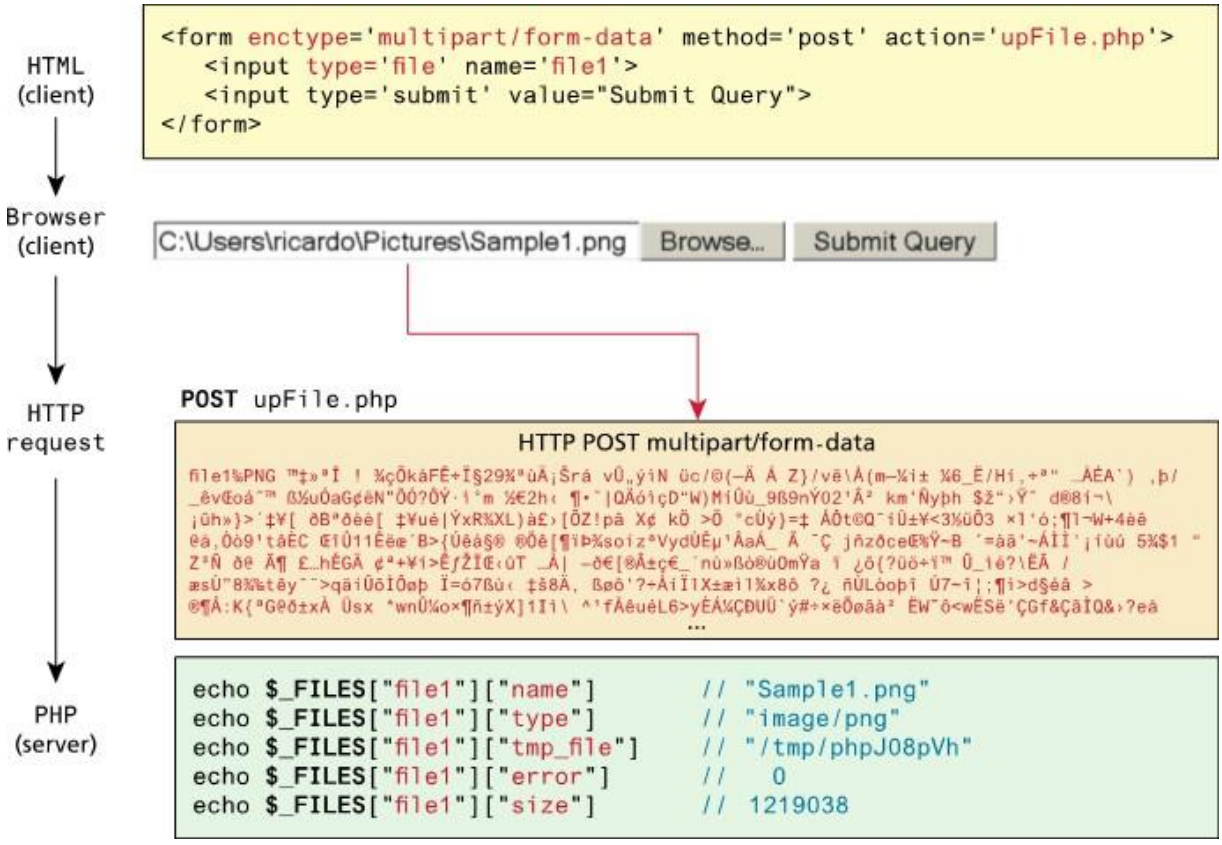
$_Files array

Sometimes you require uploading files through web pages. It can be done as follows:

First, you must ensure that the HTML form uses the HTTP POST method. Second, you must add the enctype="multipart/form-data" attribute to the HTML form that is performing the upload. Finally you must include an input type of file in your form.

```
<form enctype='multipart/form-data' method='post'>
  <input type='file' name='file1' id='file1'>
  <input type='submit'>
</form>
```

Figure below shows the flow of how the files are uploaded.



Reference: figure taken from: Fundamentals of Web Development (Second Edition, Pearson), Authors: Randy Connolly, Ricardo Hoar

The code below shows about how we can check the error in file handling: foreach

```
($_FILES as $fileKey => $fileArray) {
  if ($fileArray["error"] != UPLOAD_ERR_OK) { // error
    echo "Error: " . $fileKey . " has error" .          $fileArray["error"] . "<br>";
  }
  else { // no error
    echo $fileKey . "Uploaded successfully ";
  }
}
```

We can also restrict the file size that should be uploaded. It can be limited in multiple ways e.g. by using HTML form attributes in inputs (browser) or JavaScript (browser) or PHP validation (server). The following code is an example of limiting the file type.

```
$validExt = array("jpg", "png");
$validMime = array("image/jpeg","image/png");
foreach($_FILES as $fileKey => $fileArray ){
  $extension = end(explode(".", $fileArray["name"])); if
  (in_array($fileArray["type"],$validMime)
   && in_array($extension, $validExt)) {
    echo "All is well. Extension and mime types valid";
  }
  else {
    echo $fileKey." has an invalid mime type or extension";
  }
}
```

Moving the file:

```
$fileToMove = $_FILES['file1']['tmp_name'];
$destination = "./upload/" . $_FILES["file1"]["name"];
```

```
if (move_uploaded_file($fileToMove,$destination)) { echo
  "The file was uploaded and moved successfully!";
}
else {
  echo "There was a problem moving the file.";
}
```

Lesson 109 : Reading and writing files

Reading and writing files:

Mostly, when we have to save the user's data, or date we are getting from forms, we save it in the database. Because, in the database we are saving data in extracted form and it becomes easy for us to revertive the data efficiently and easily. However, there are certain situations in which you have to create the files on the server side by writing and reading the data in such files. For this, we have different functions in PHP.

Before getting into these functions, let's first know about the ways of reading files. Mainly, there are two basic techniques for reading/writing files in PHP.

First is "Streaming" in which we open the file and read the data line by line, after reaching at the end of the file after reading close the file. This method is used when we have large files and loading them in memory at once is not possible.

Another method is "All -In- Memory access" in which we read the complete file in the memory at once and write the data residing in memory at once in the file. It does make the processing of files extremely easy.

How Streaming works?

Stream access works in the same way as 'fopen(), fgets() and fclose() works in the C/C++ programming language. First, we open the file using the fopen() function and then by
thorough loop read the data from the file line by line using the fgets() function, till we reach the end of the file. After reading all data from the file, close the file by fclose() function.

How In-Memory File Access works?

If you have a file of size which can be fit into memory, then we have such functions to read/write the complete file at once. The function "file()" reads the entire file and returns the file content as an array, with each array element corresponding to one line in the file, such as index [0] of the file having the first line, index [1] have the second line and so on.

Other functions are "file_get_contents()" and "file_put_contents()" . file_get_fucntion reads the file and stores its all content into a string variable. If we have data in string form and we want to

save it into the file then we will use the function file_put_content () function and pass that string to the function to write all the data in the file.

Week – 9

Lesson 110 : Classes and Objects in PHP; Object properties

Object oriented programing facilitate us to think in terms of object and program coding. So, in program we keep the object in mind and then implement the properties and methods related to object. By this way, our code becomes maintainable, making logic and program/code becomes easy as well.

To define a class, we use the keyword “class” then write the class name, after that list down the methods and properties. For example, we have a class named “Artist” and have some related information, we will write this as follows;

class Artist { public

$firstName; public

$lastName; public

$birthDate; public

$birthCity; public

$deathDate;

}

Instantiating Objects:

Use the new keyword:

After making class, we create the instances of the class called object. Creating object is simple, like we use the “new” keyword with class name. By this an object is created in memory, which we save to a variable. Example is given as follows:

$picasso = new Artist ();

$dali = new Artist ();

Here, two objects are created of class "Artist". In OO programming we can use as many objects as required.

Properties:

Once you have instances of an object, you can access and modify the properties using the object's variable name and an arrow (->)

$picasso = new Artist();

$dali = new Artist();

$picasso->firstName = "Pablo";

$picasso->lastName = "Picasso";

$picasso->birthCity = "Malaga";

$picasso->birthDate = "October 25 1881";

$picasso->deathDate = "April 8 1973";

After creating object, its properties are accesses through "->" sign. If the property is public, we can directly store value in it, like "Pablo". Similarly, to retrieve the value we write the object name following -> sign and then property name like $picasso->firstName.

Lesson 111 : Constructor; Methods

Constructor:

Before method calling, we have to initialized the object, and in OO programming, to initialize the value constructor is used. In PHP, defining of constructor is bit different as compared to C++ or Java. Here, to define the constructor of any class, we will define as functions and its name will always be **" construct"**. By this, function will be called automatically while creating object. Constructor can be parameterized; its parameters can be taken from user or some values can be default.

class Artist {

function__construct ($firstName, $lastName, $city, $birth, $death=null) {

$this->firstName = $firstName;

$this->lastName = $lastName;

$this->birthCity = $city;

```
$this->birthDate = $birth;

$this->deathDate = $death;

  }

}
```

In the above example, $death default value is null, while all other values are taken as parameter and saving them in properties of class' objects. Here you can see that parameter of function is $firstName and same is the property of the class, it doesn't make much difference and doesn't have to be same always.

In PHP, $this is used for object property which represent the current object, then -> sign is used, after that property name of object is used.

```
$picasso = new Artist("Pablo","Picasso","Malaga","Oct 25,1881", "Apr

8,1973");

$dali = new Artist("Salvador","Dali","Figures","May 11 1904",

"Jan 23 1989");
```

For example, $this. firstName = $firstName; here any name passed through constructor will becomes property of this object. Parameter name can be different from property name, the difference between them is that the local parameter can be accessed as it is while object property is accessed through "$this". An object is created through "new" keyword, then write class name, after that pass the parameters which the constructor is taking.

Methods:

Methods define the tasks each instance of a class can perform and are useful since they associate behavior with objects. For example, if we have

information about artist and want to display it in HTML table format then we will write its method as follows:

```
class Artist {

  // . . .

  public function outputAsTable() {

 // . . .

  }

}
```

To call the above method:

$a-> outputAsTable ();

Here $a is the object of the class Artist.

Lesson 112 : Visibility

In PHP we define the visibility of properties and methods of class. Visibility means that some properties of objects are accessible and some should be visible only to function. Similarly, some functions or methods should be accessible through object and some should not be accessible.

To define all this, we use the visibility keywords which are public and private. When we set a variable, property or method as public, we can access it through object. While if we set a variable, property or method as private, we can't access it through object. However, we can access variable or method inside the class but cannot access directly through object.

For example, we have class named "Painting", having public property "$title" and private property "$profit". Similarly, we have a public function "doThis()" and private function "doSecretThat()". In the class code we can access all these public and private methods and properties, like we are accessing public property "title" and private property "profit" in the following way:

$a = $this->profit;

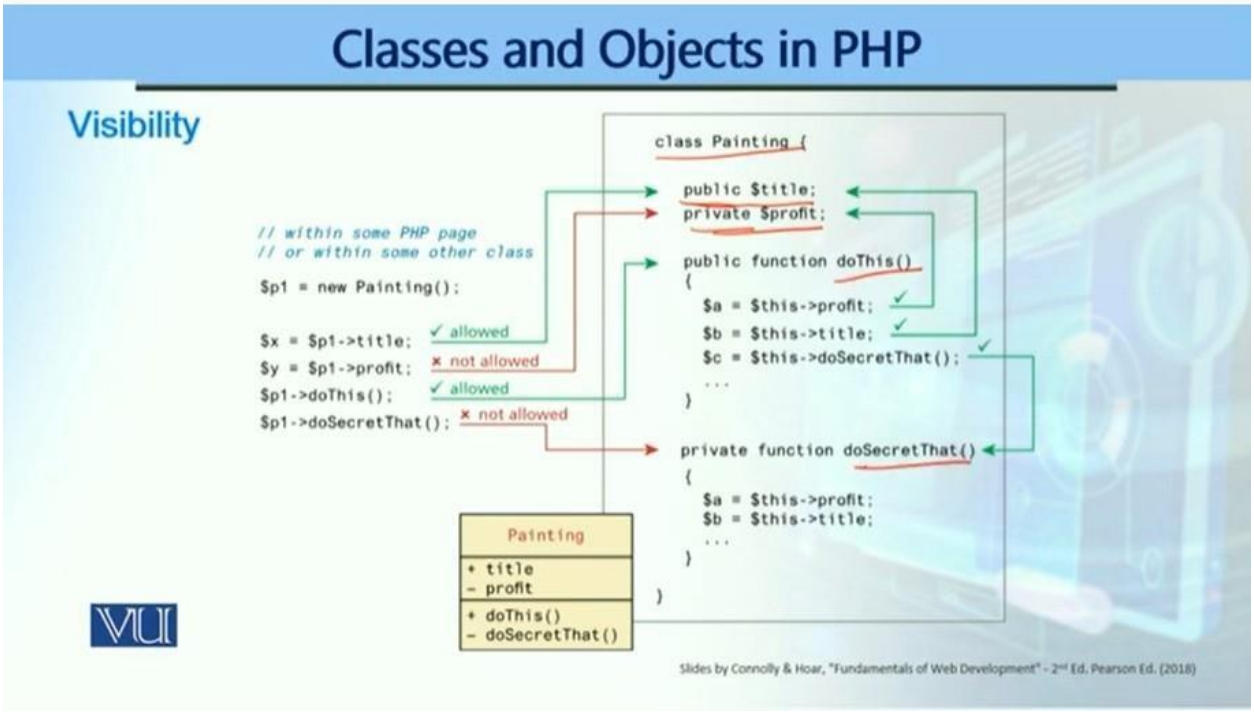
$b = $this->title;

Similarly, method can be accessed. However, when we look towards object side that which thing we can access or not. Let say, we have created an object of paining as "$p1 = new Paining ();" by this we can access public properties/methods like "title" and "dothis()" in following way:

$x = $p1->title;

$p1->doThis();

But we can not access private properties or methods, such as if we try to access "profit" which is a private property, we will get an exception. Similarly, private method like "doSecretThat()" is not accessible.



will reduce the chance of accidental bugs. Such as if you set the most of the properties as private, there is no need to access them through object, it can be accessed through functions where required. The benefit of accessing properties through function is that you will get the exact logic about the use of property in class. If you set each property as public and you make changes in the code related to object, then there is a chance that logical bugs arise accidently. So, make sure to set the visibility of properties and methods appropriately.

Lesson 113 : Static members; Constants

In OO programming, there is a concept of static variable, any property or method which is set as static is shared among all objects. In fact, we can set the variable or methods as static without creating object. In PHP, if we have to set a property as static, the syntax will be as follows:

public static variable name = initializing value;

For example, in class “Artist” we will write the static property like this: public

static $artistCount = 0;

By this we can access this static property without creating an object. Similarly, the mechanism to define static function will be as follows:

public static method name() {

Statement;

}

For class “Artist” we will write the static function like this:

public static function addArtist() {

self::$artistCount++;

}

If we have to access the above static method, we can access this without creating any object, by using class name directly like this:

Artist :: addArtist();

And to access static variable we will write it as follows:

Print(Artist::$artistCount();

We can change its value directly, if required.

Class Constants:

Constants is another similar concept like static variable in class, the only difference it that you can change the value of static variable while constant values can’t be changed. Constants can also be accessed directly through class name like static variables. To define the constant in the class body, syntax will be as follows:

const constant name = ‘constant value’; For

example,

Const EARLIEST_DATE = ‘January 1, 1200’;

By convention write the constant name in capital letters. These can be accessed both inside and outside the class, to access in the body of the class we will write it as follows:

self::EARLIEST_DATE

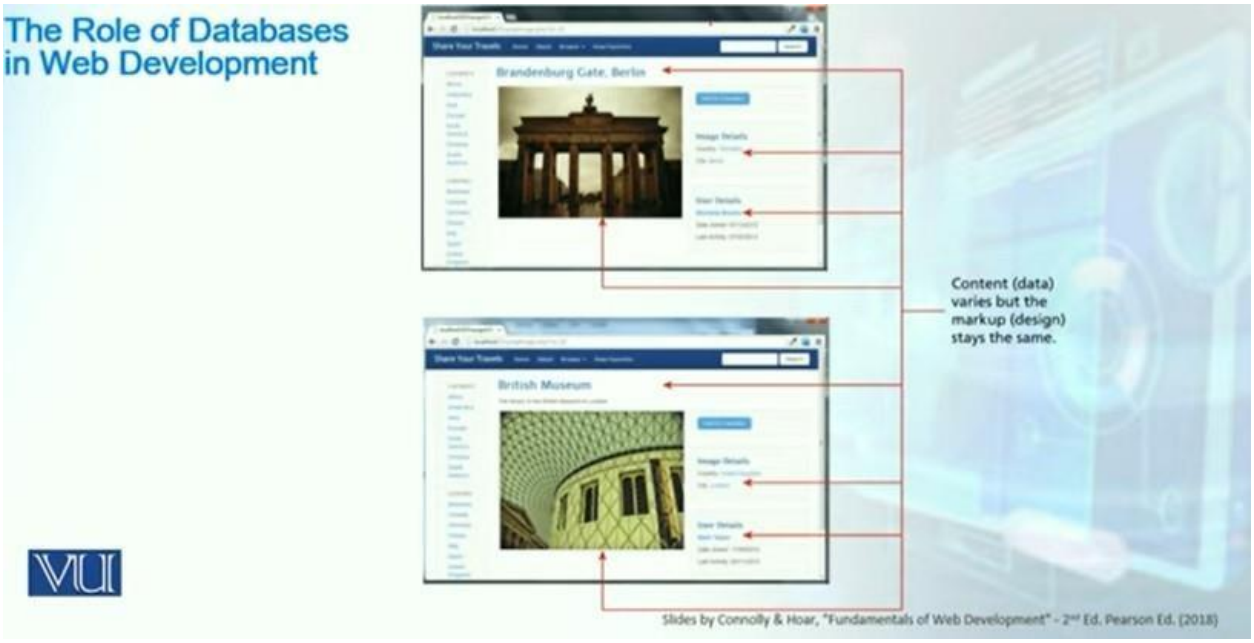
while to access it outside the class write as follows: classRefrence::EARLIEST_DATE

Lesson 114 : Databases and Web Development

The Role of Database in Web Development:

Database plays very important role in web development; all dynamic website is associated with database in some way. If we talk about only HMTL or CSS pages they are static i.e., their content remains same. If we want to make our HTML pages dynamic, we can do this by server side by

using any language like PHP. PHP will fetch the data from database and fit it into HTML blocks which we want to make dynamic.

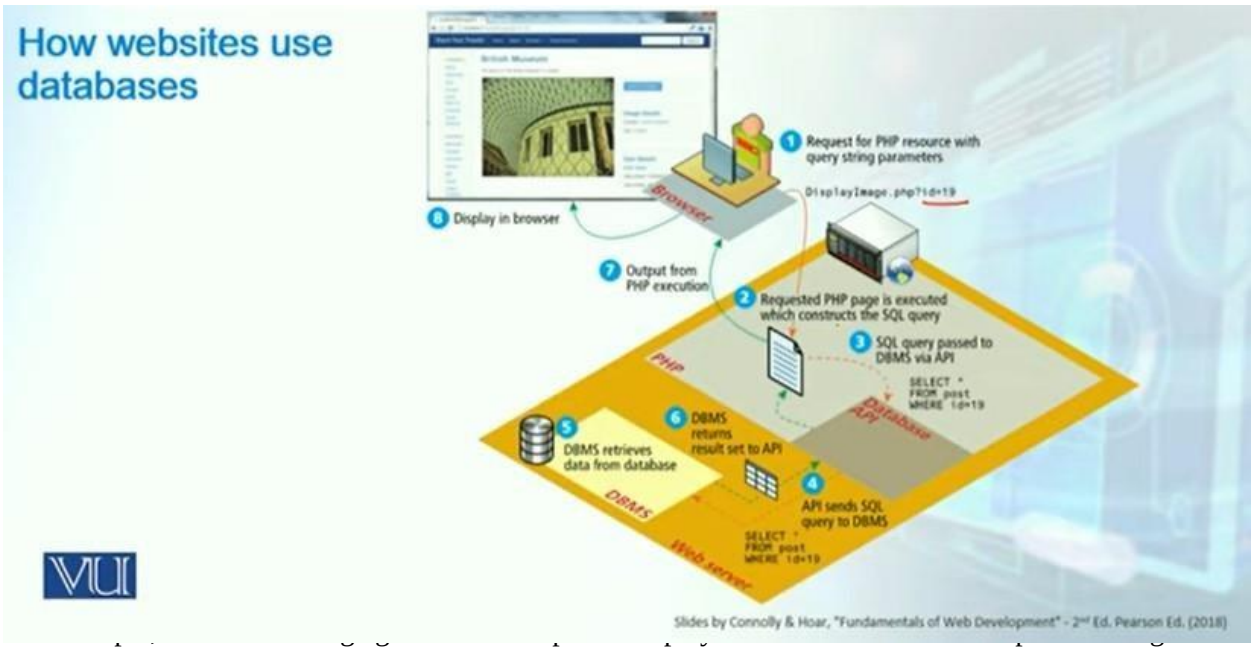


HMTL, CSS and JAVA scripts would be the same for all the palaces. The static or constant part would be covered in HMTL part, while dynamic contents will be fetched from database and embedded in HMTL part. Here in the example website some parts are constant like header, nav bar, and layout, but there are some differences as well. The above both pages are generated by the same script.

The main idea behind is that we make a HTML template and then save the information about different places into database. After that we will fetch the information of a particular place and plugin it into the HTML template. By this, the user will see the information of different places in the same way which is generated by same script.

How Websites Use Database?

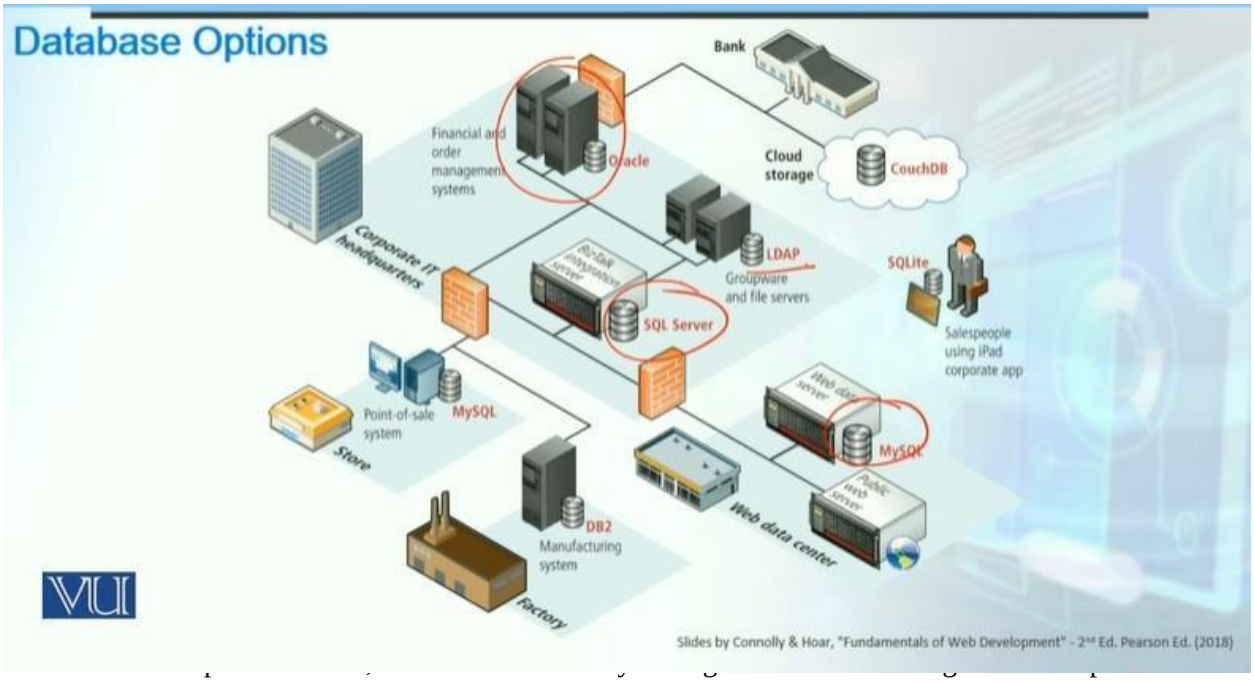
Let see how websites used database. Our user will send the request to the server site through PHP script. Mostly, one script deals with the different types of objects like in the above example, one script is displaying information of different places, with this we will pass some information to the query string.



"19", we will pass information of place id as "DisplayImage.php?id=19". This request will go to PHP script which will access the database. To access the database the server site programming language will use database API which is used to access the database. So, we will pass the required information to the API, then API will send the request to the actual database to fetch the required data. The fetched data will be passed to the script, which then embed the dynamic information into HTML, so the page will be opened at client side at browser. The user will think that all web pages are different, but at the backend script generating these webpages with different information would be the same.

Database Options:

When we talk about database we have many options, such as different organizations are using different options and at times one organizations uses multiple options. For example, some database use DBMS Oracle, some use MySQL or SQL server. If organization is using SQL server, then the information about access grant is usually kept in LDAP server.



internet access is not available, then the apps of their devices will use local database, such as SQLite which is most popular and can be used in mobile apps. In certain situations, database can be placed in cloud then insert or retrieve information in database through cloud. So, there are many options when we are dealing or working with database.

Lesson 115 : SQL - Select, Insert, Update, Delete

When we interact with database, most of the time its language is SQL. In SQL we use different statements.

SELECT Statement:

Most basic statement of database is "SELECT", it is used when we have to fetch some data from the database. For example, we have table having information related to book like book title, ISBN, author information and price etc. This all information is organized columns or fields. Let's say we want to retrieve information of each book title and ISBN, we will follow the following syntax:

SELECT filed name FROM table name; SELECT

ISBN10, Title FROM Books;

If we want to retrieve data from all fields of data then "*" is used, the query will be as follows: SELECT *

FROM Books;

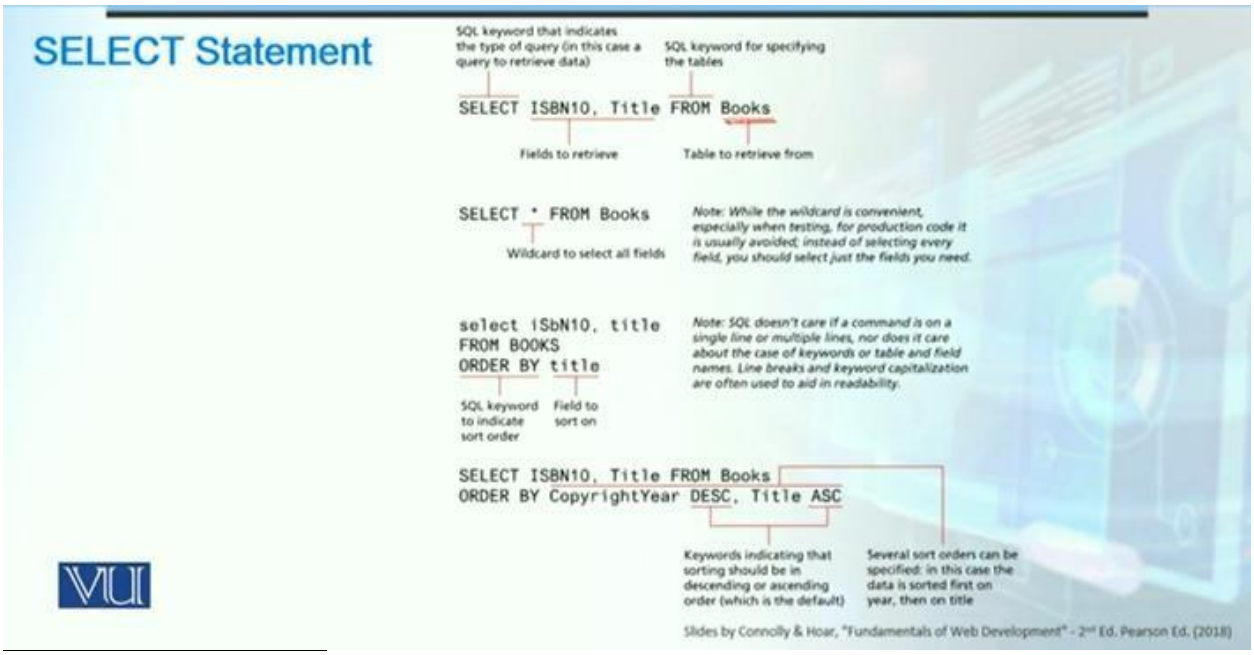
If want to sort the fetched data on the bias of any filed, like we want to sort the data according to title of book then “ORDER BY” is used, the query will be as follows:

SELECT ISBN10, Title FROM Books ORDER BY title;

Sorting can be done on multiple criteria, like sorting some fields on descending order or ascending order. For example, to show the copyright in descending order and title in ascending order, the SQL query will be:

SELECT ISBN10, Title FROM Books ORDER BY CopyrightYear DESC, Title ASC;

Here, you can see that one of the filed” CopyrightYear” is not selected in the SELECT statement, this shows that you can do sorting on the basis of field if it is not selected in SELECT statement.

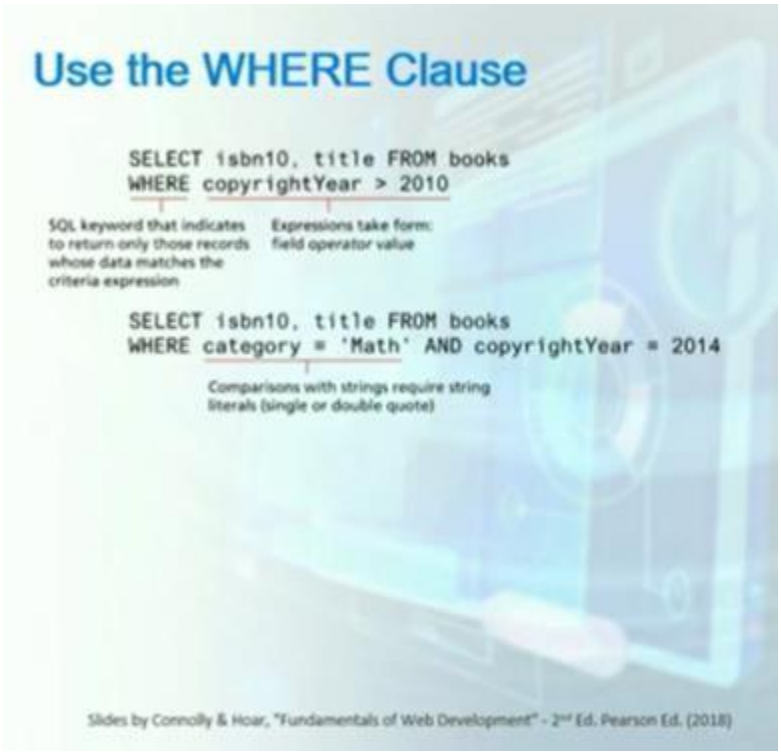


If we want to fetch data from database on some conditions then WHERE clause is used. For example, if we want to fetch the ISBN and title of books which have copy right year is after 2010 and on ward. So, the query will be:

SELECT ISBN10, title FORM Books WHERE copyrightYear > 2010;

Similary , we can apply multiple conditions, for example, to fetch the record of books of maths which have copy right year 2014.

SELECT ISBN10, title FROM Books WHERE category = ‘Math’ AND copyrightYear = 2014;

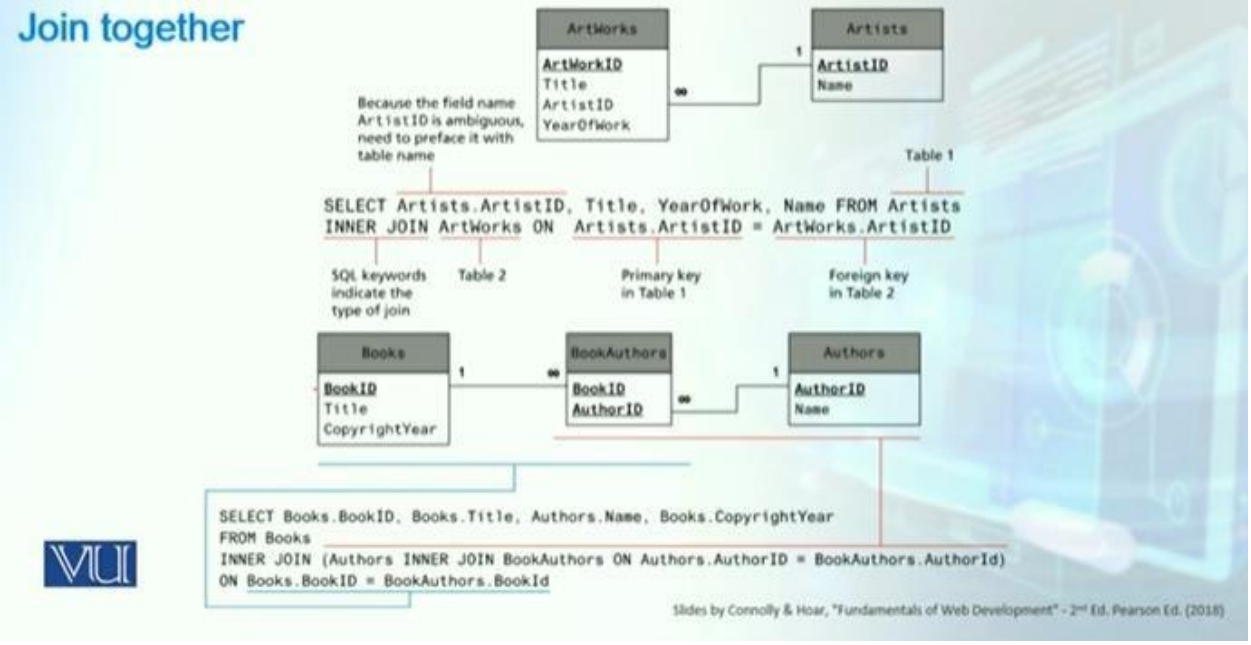


Join in Database:

Sometimes, information is distributed in multiple tables. Lets say we have books , their informaiton is in Books table, book authors informations in in Authors table, then which author has wrriten which book, this informaiton will be in third table BookAuthors. Another example is that the informaiton of artist is in a table Artists, and informaiton about artwork of these artists in another table called Artworks.

In normalization we will do this to resolve some problems. If we want to fetch the information from multiple tables then in databas we use concopt of joins. In this we will select the tables from where we have to fetch the data. Lets say, to fetch the data form artist table and join it to the artworks table, for this we will use INNER JOIN on Artwork table and select the common filed of both tables by using ON keyword. Like in these tables the commons field in ArtistID, so the query will be:

SELECT Artist.ArtistID, Title, YearOfWork, Name FROM Artists INNER JOIN ArtWorks ON Artists.ArtistID = ArtWorks.ArtistID;



Simliary, join can be on two or more tables. For example, to join three table related to books, the Books and Book authors will join on the bais of book ID, while Book authos and authors joins on the bais of author ID. The query for this will be like:

SELECT Books.BookID, Books.Title, Authors.Name, Books.CopyrightYear FROM Books INNER JOIN (Authors INNER JOIN BookAuthors ON Authors.AuthorID = BookAuthors.AuthorId) ON Books.BookID = BookAuthors.BookId;

Member Group By:

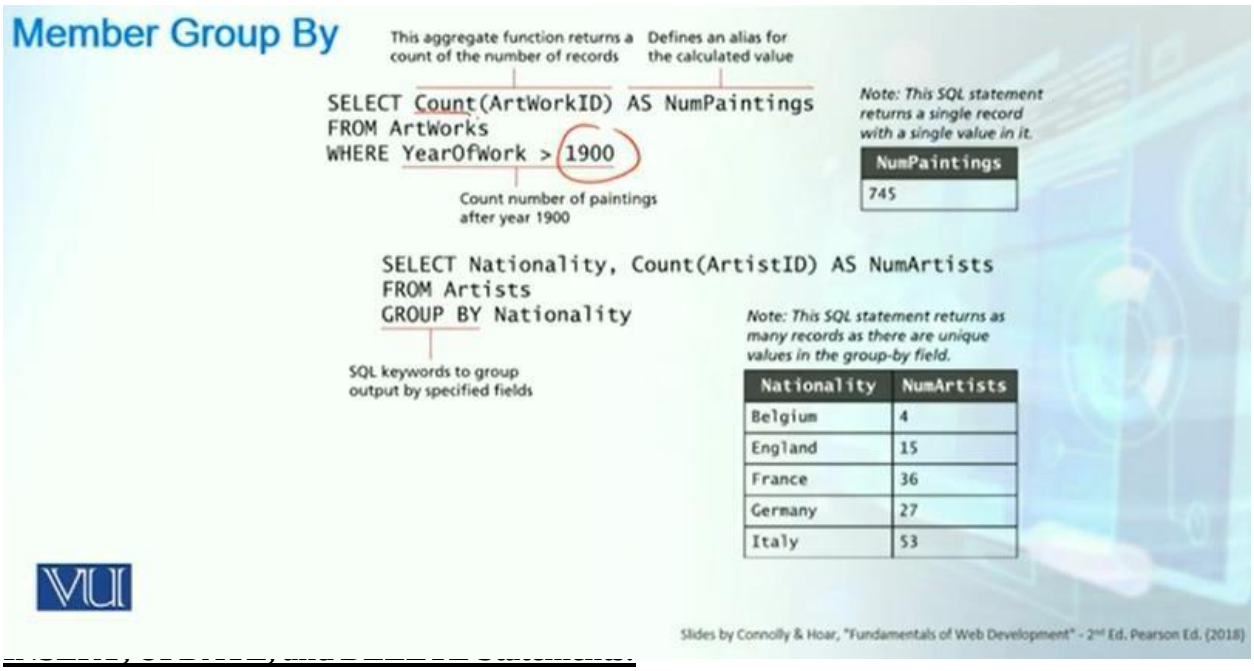
Another important concpet in database is of grouping. Lets say, we want to see that that how many artworks in the table of ArtWorks are made after year 1900. For this we will write SELECT statement, count is used as aggregation function to count the number of paintings. The query will be:

SELECT Count(ArtWorksID) AS NumPaintings FROM ArtWorks WHERE YearOfWork > 1900;

If we want to make this serach specfic to artist that which artist has produce how many artworks then we will use GROUP BY clause. The query will be:

SELECT Nationality, Count(ArtistID) AS NumArtists FORM Artists GROUP BY Nationality;

Here, grouping is done on the basis of nationality, so all records are grouped in the baise of nationality so we will get the number of artworks of each nation.



To store the information in database, INSERT statement is used. The syntax for this query will be:

INSERT INTO table name (name of fields) VALUES (values to be inserted); For

example,

INSERT INTO ArtWorks (Title, YearOfWork, ArtistID) VALUES ('Night Watch', 1642, 105);

It is not mandatory to mention all field, only mention those fields name in which you want to insert data. Another syntax for this query is as follows:

INSERT INTO table name SET filed name = vlaue, filed name = vlaue….; For

example,

INSERT INTO ArtWorks SET Title = 'Night Watch', YearOfWork=1642, ArtistID=105;

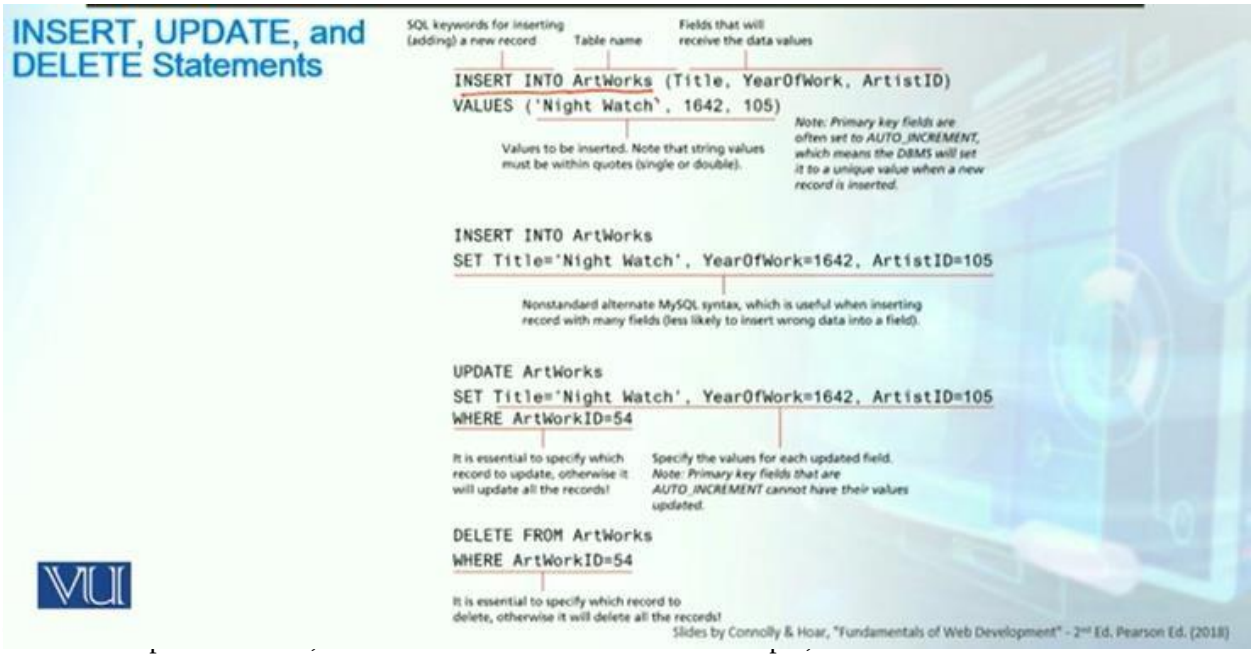
Similarly, to update the record, UPDATE statement is used, syntax is:

UPDATE table name SET field name = vlaue, filed name = vlaue WHERE record to be updated.

WHERE in used to specify that for which record, we want this updation. It is mandatory to use WHERE clause, if you don't use this clause all related record in your database will be updated.

For Example,

UPDATE ArtWorks SET Title='Night Watch', YearOfWork=1642, ArtistID =105 WHERE ArtWorkID=54;

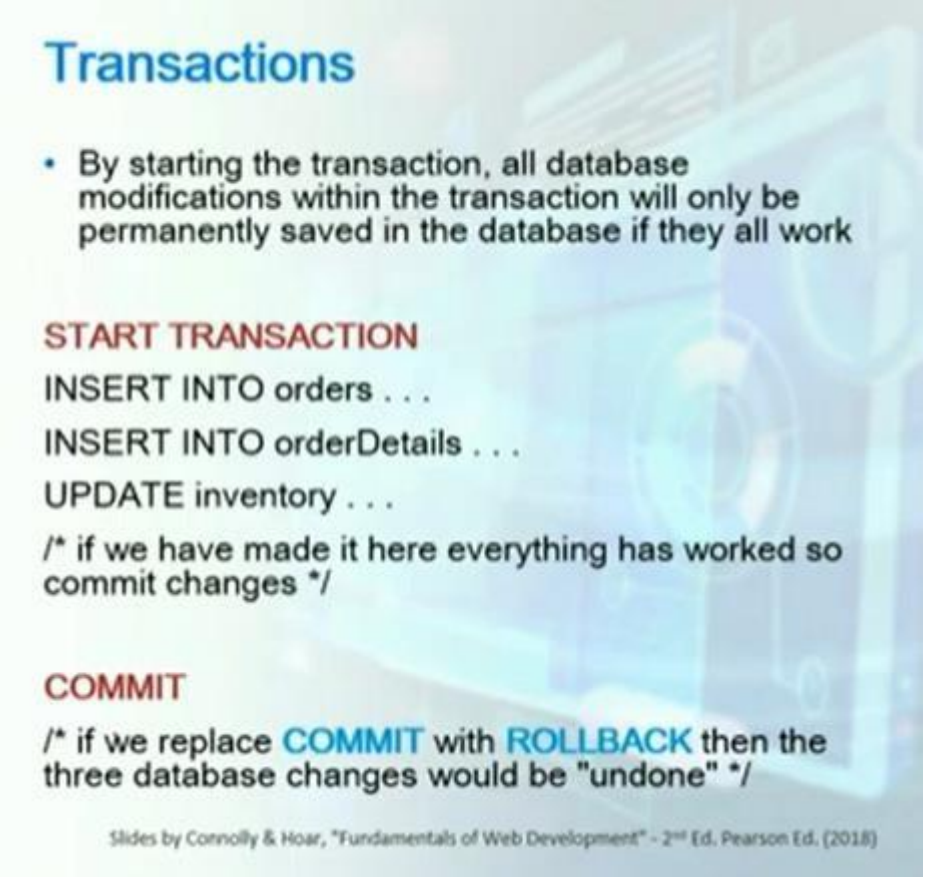


FORM ArtWorks WHERE ArtWordID=54.

Similar to UPDATE statement, you have to use the WHERE clause in DELETE statement, otherwise all your records will be deleted.

Transactions:

Another important concept in SQL is of transactions, specially when you are working with critical systems like banking's or with collection of statements where successful execution of all statements is important. Mechanism of transaction is simple, you start the transaction and then write statements, and successful execution is must, if any of the statements fails to execute then you will stop this transaction and none of the statement will be executed. If you don't use transaction this will results in inconsistency of data as some statements will be executed and some will fail to execute.



In transaction if all statements executes successfully then you will COMMIT this and if all statement doesn't execute then you will use ROLLBACK.

Lesson 116 : Introduction to MySQL

In web development, we have different options for databases, sometimes we are given option to choose the database, while sometime we have to use the given database. Most popular database used in web development is MySQL, which is now known as MariaDB. But the software stack used for this, it has MariaDB at backend, but the tool to access this or the code we write in PHP to interact with database is the same as used in MySQL.

Command-Line Interface:

There are multiple ways to access the MySQL, when you install the database, a command line utility will be installed with this which is called "Command-Line Interface". This utility works irrespective of Window, Linux or MAC. Through this utility we connect with database and using command interact with database.

To launch and interactive MySQL command-line session, you must specify the host, username, and database name to connect. The command will be as follows:

mysql -h 192.186.1.14 -u bookUser -p

In windows to connect with database you have to access file named "mysql.exe", path to access the "mysql" file is **Xamp>MySQL>Bin>mysql.exe.** In the same folder, there will be another file name "mysqld.exe", it is server file so it cannot be used for connection. To connect with database, you need to provide some parameters like host such as "-h 192.168.1.14". This host can be an IP address where the DB server is hosted, it can be local host or 127.0.0.1 if database server is installed on the same machine from where you are accessing it.

Another parameter is user name of database like "-u bookUser" and then password after "-p" but it is not recommended, if you give password, it will become part of command line history.
However, if you write only "-p", then utility on command prompt will ask for password, in case of valid password you will be connected to database and a command line interface will be shown.

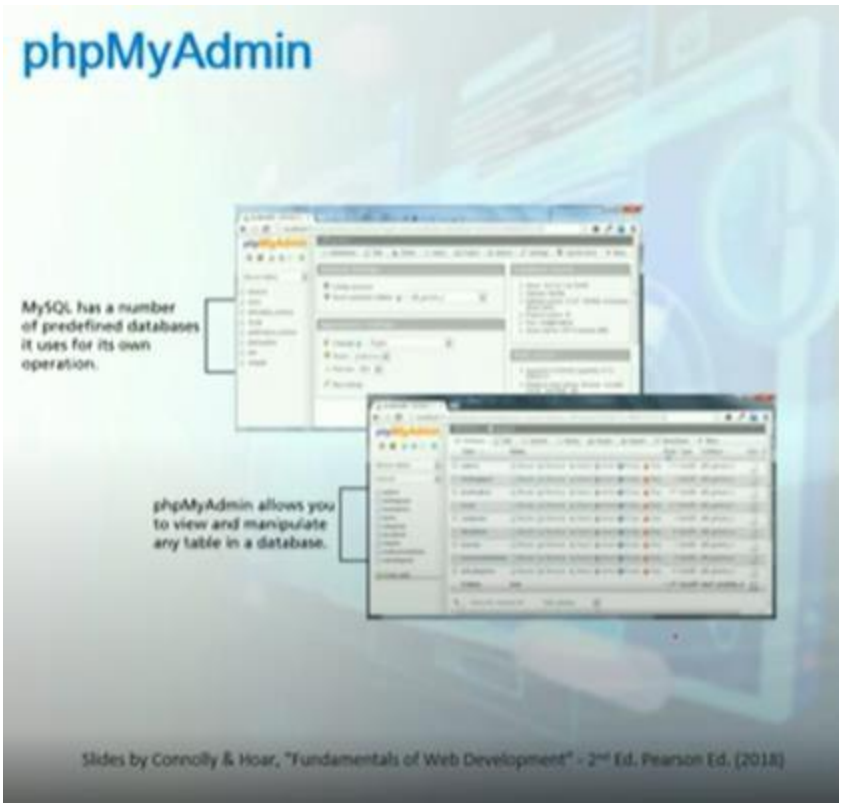
Pre-defined statements can also b executed through command line utility. Lets' say these statements are saved in "commands.sql", which is the text file having SQL queries. To execute the queries, write the "<" after passing parameters and then after a space write the name of file having statements to be executed on server. For example,

mysql -h 192.168.1.14 -u bookUser -p < commands.sql

After valid authentication of above command all statements written in the file will be executed one by one.

phpMyAdmin:

Sometimes, use of command line interface becomes difficult as you have to do all work by typing and memorize the commands as well. Alternative to command line interface is some graphical interface like in MySQL, most popular utility is phpMyAdmin. It is web-based utility but used to manage the MySQL, it is installed with XAMMP.



Its interface is very interactive, you can see all databases, by selecting a database you will see its tables. You can also create new table, execute queries, can save the database and you can also import/export database.

MySQL Workbench:

It is desktop based graphical interface with MySQL. In this you can see the table connections if the primary key/foreign keys are set appropriately. You can also see database schema, create database, generate query, create tables and can perform all other activities related to database.

Lesson 117 : phpMyAdmin

On installing XAMMP, the utility to manage the database in PHP i.e., phpMyAdmin will install by default.

Command-Line Utility/Console based Utility:

To access the MySQL, command-line or console-based utility is used. If you want to access MySQL through the console, write 'cmd' in Windows, while in Linux terminal it is used. The steps are as follows:

1. Go to the folder in which you have installed MySQL via XAMMP.
2. Next there will be a folder of SQL and then a Bin folder.

3. In the bin folder, there is a file mysql.exe.
4. To access the mysql.exe file write this command “mysql -h localhost -u root”
5. After this, it will be connected to mariaDB.

Some other commands which are used are:

- By writing “help” you can view different commands that are available.
- To check the available database, write the command “show databases;”
- To connect with any database, write “use database name;”
- We can see tables and can make queries as well.

Graphical Interface for database:

Alternative to the command-line interface is phpMyAdmin. To open phpMyAdmin, open the browser, if XAMMP is installed then through the local host you can open it. There will be a link on the top right “phpMyAdmin ". Before clicking this link make sure that the database server is started. To check this, go to control panel of XAMMP, here MySQL must be in green color and status is “started”, if it is started then you will see “start” which means database is running, otherwise it will show “stop” button.

If the button is green that means the database is running, so click on phpMyAdmin to access the database, here you can see all databases on the left side of the panel. To create a new database “New” option is also available. By selecting an existing database, you can create new tables using the create table interface.

For example, to view the “mysql” database, there are already many tables to manage the dbms. By clicking on “SQL” you can write any query and its result will be displayed. Moreover, you can export/import the tables, can set privileges for users and perform many other operations.

Lesson 118 : Accessing MySQL in PHP

In web development where we have to store the information in the database then, we have to connect the PHP script with the database. Let's see, how can we fetch the data from MySQL while using PHP?

PHP MySQL APIs:

Earlier, PHP had an extension named “MySQL extension” to connect with the database. This was the original extension to the PHP for working with MySQL and has been replaced with the newer “mysqli” extension.

mysqli Extension:

This extension provides both a procedural and an object-oriented approach. This extension also supports most of the latest features of MySQL. But the disadvantage is that if your website changes from MySQL to any other database, the code will not work for the new database as it was specifically developed for MySQL. As an alternative to this, another generic approach is used which is called "PHP data objects".

PHP data objects (PDOs):

This provides an abstraction layer that with the appropriate drivers can be used with any database, and not just MySQL databases. However, it is not able to make use of all the latest features of MySQL. In PDPOs, the PHP code will not change with change of database, only database connection will change i.e., connection string and parameters will be modified.

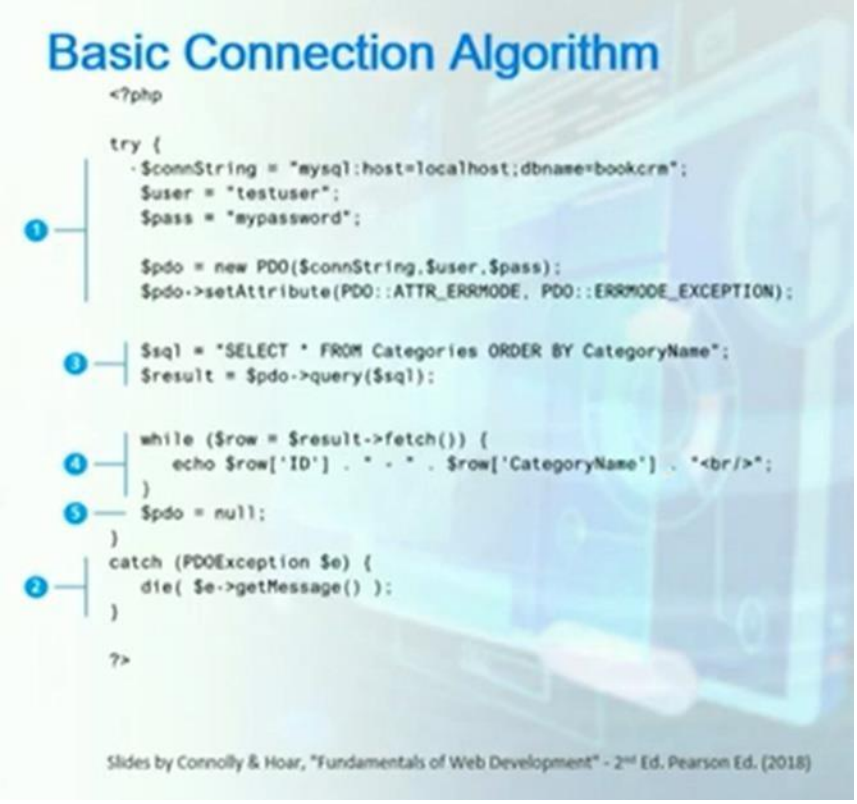
Deciding on a Database API:

While PDO is unable to take advantage of some features of MySQL, there is a lot of merit to the fact that PDO can create database-independent PHP code. But there could be a scenario where such features are required of MySQL which are not available as PDOs, so you can use "mysqli", otherwise use of PDOs are recommended. Like many things in the web world, there is not a single best choice. As the chapter (and book) proceed, we will standardize on the object-oriented, database-independent PDO approach.

Basic Connection Algorithm:

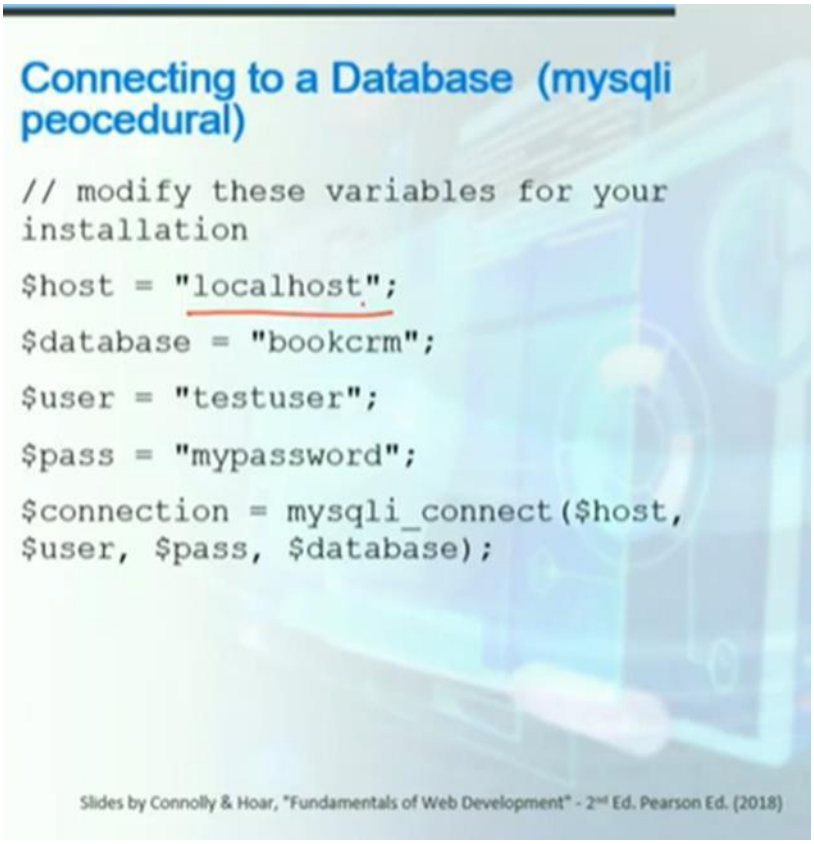
To connect with database there are some simple steps which are as follows:

1. Connect to the database
2. Handle connection errors
3. Execute the SQL query
4. Process the results
5. Free resources and close connection



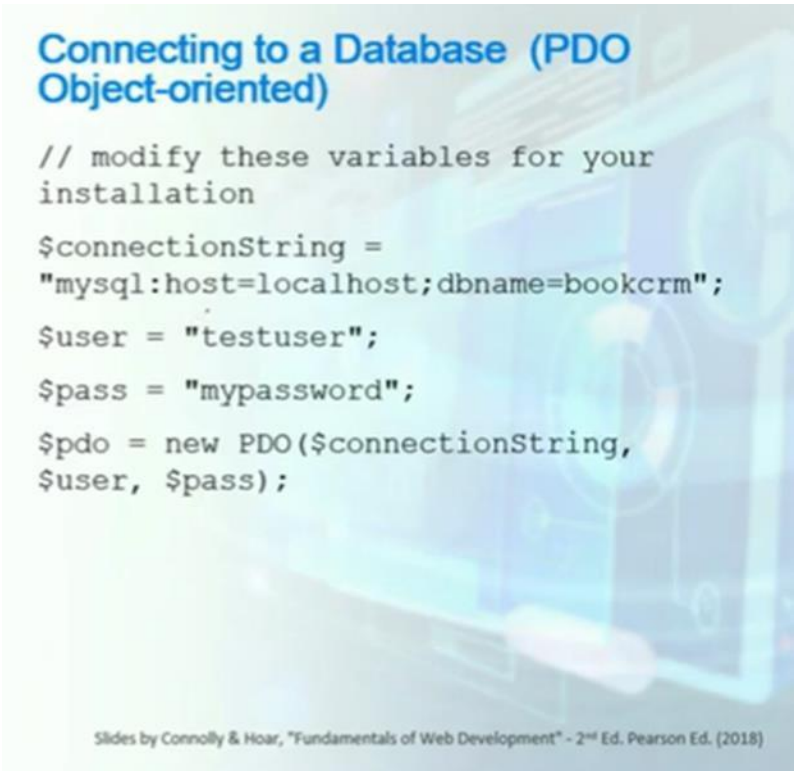
The mechanism to connect with the database in “mysqli” is a procedural mechanism. You will have some functions and in each function pass the connection object. Firstly, set the parameters related to connection e.g., “$host = “localhost”;” this shows that server is on local host, which means web and database server are on the same machine. But this is not necessary in a production environment, servers can be on different machines, in that case you have to give the IP address.

After that give the name of the database, username and password. Write a connection string after this like “$connection = mysqli_connect ($host, $user, $pass, $database);”. Following all your interaction with the database will be done through this connection object. One important thing is that if you have more than one script which is interacting with the database then save the connection related information in the file and use it where required. By this, you will be able to avoid repetition of the same code in every script. Disadvantage of repeating is that if one thing changes in the database you have to update all the scripts. However, if the information is kept in only scripts and you are including it into other scripts then only those scripts need to be updated.

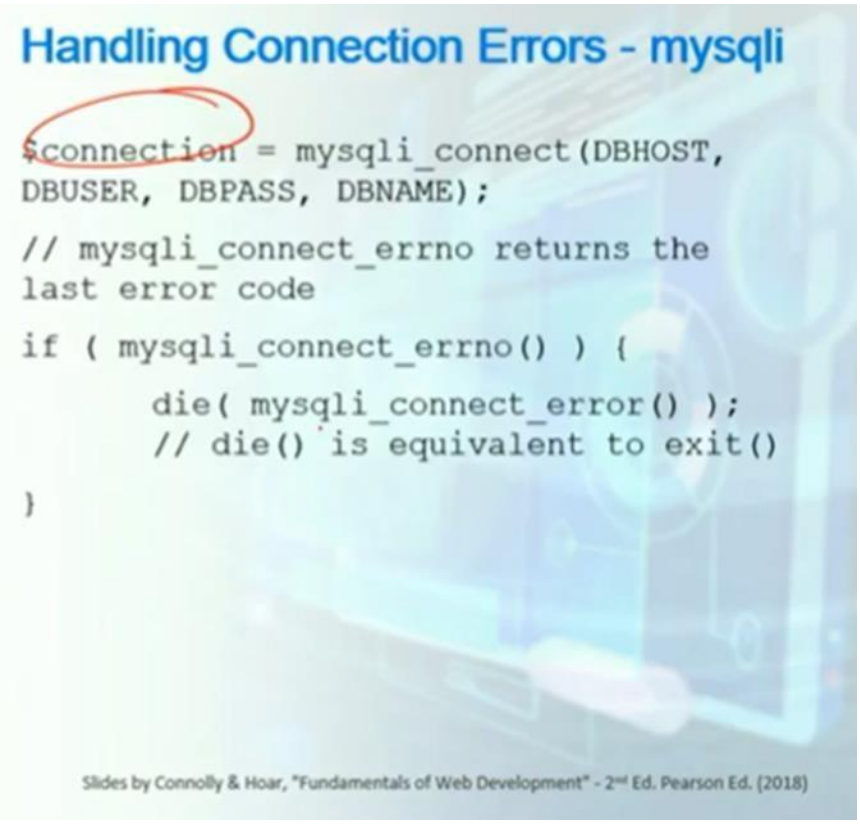


To do the same thing using PDOs, following steps will be followed:

- First tell the name of the database system and host in the connection string.
- Give a User name which will use the database.
- Give a password.
- To establish the connection, create the object of PDO and pass the connection string to it.
- All subsequent database interaction will be done through this object.



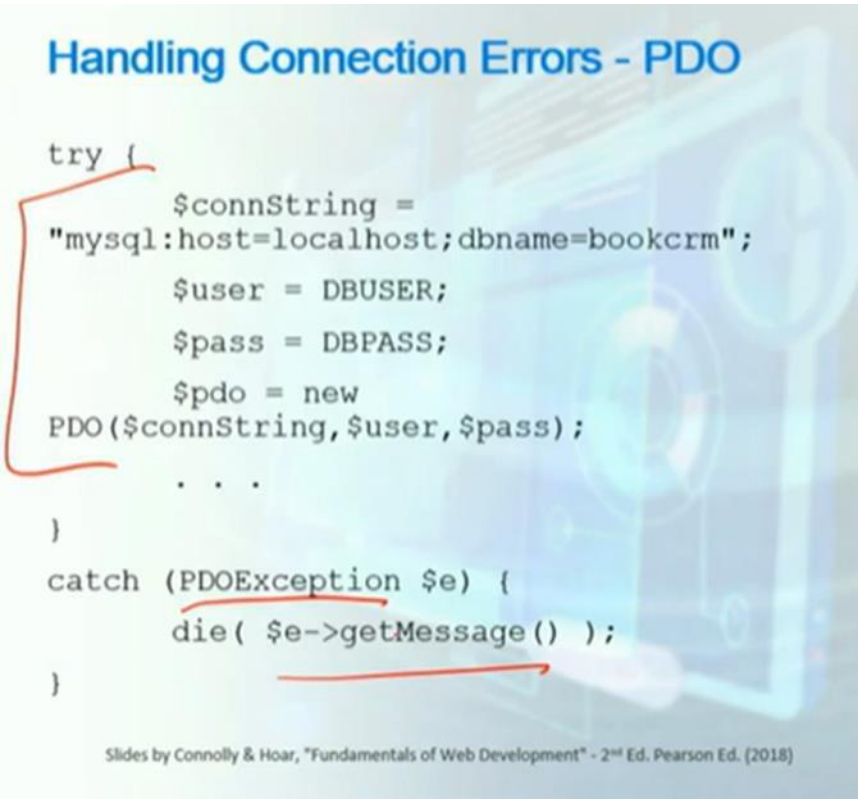
If some error occurs while establishing a connection you can handle them through mysqli. To check whether any error occurs at time of establishing connection call the following function "mysqli_connect_errno ()". If the error occurs this statement will be true otherwise false. In case of an error in connection you will get the details through the "mysqli_connect_error ()" function and stop exaction at this point. To stop the execution of subsequent statements, write "die" before function. Exit() can also be used in place of die().



PDO handles the errors in a different way, that if any error is generated while establishing a connection, PDO will throw an exception. You will catch this exception and display and appropriate message like this:

```
Catch (DPOException $e) { die
        ($e->getMessage() );
}
```

Connection related statements will be in the “try” block, and exceptions will be in the “catch” block. If an exception is thrown then you will go to the catch block, otherwise statements in the catch block will not be executed.



Lesson 119 : Fetching data

Accessing MySQL in PHP Fetching Data:

Let's see how we can fetch data from database systems (MySQL) into PHP. To fetch data in PHP two mechanisms are available i.e., procedural method through mysqli and other is object oriented.

Executing the Query:

For both of the above-mentioned methods we have to write query which we save in a variable like this:

$sql = “SELECT * FROM Categories ORDER BY CategoryName”; In

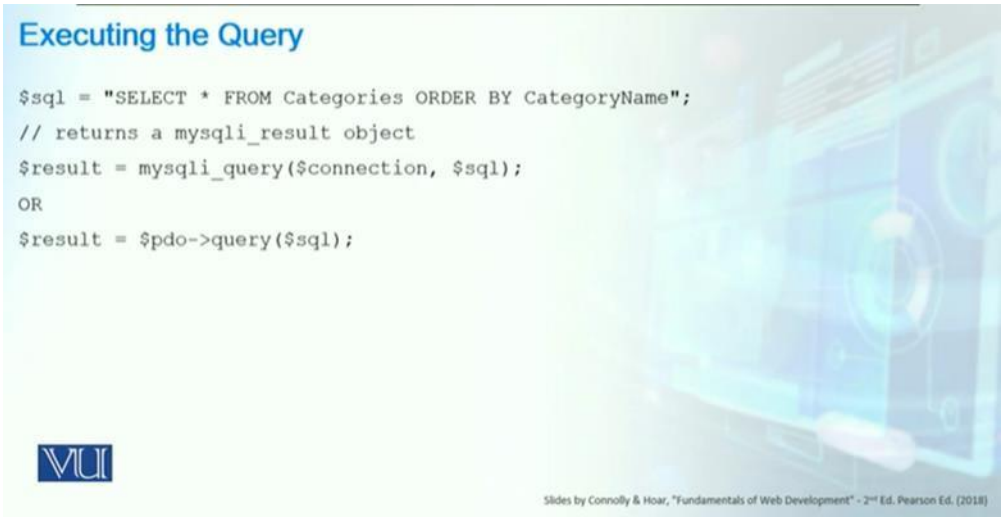
case of using mysqli, query function will be used like this:

$result = mysqli_query (&connection, $sql)

In this function we will pass the connection object and sql query.

This query will execute on the server and return the result. In case of using PDO, no need to pass the connection object as it follows the object-oriented approach. Ther query will be like this:

$result = $pdo->query($sql);



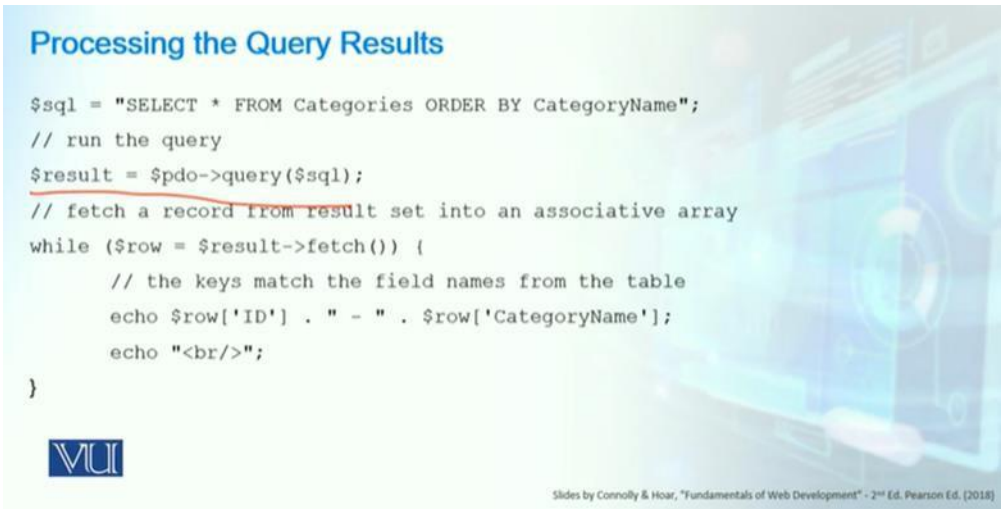
The PDO object will execute the query function and pass the SQL statement, this statement will execute on the server and return the results.

Processing the Query Results:

After retrieving the result, we have to process the results. In the result object there is a fetch () function which is basically a result set, on calling fetch () function it will return the first-row form retrieved results.

while ($row = $result->fetch ())

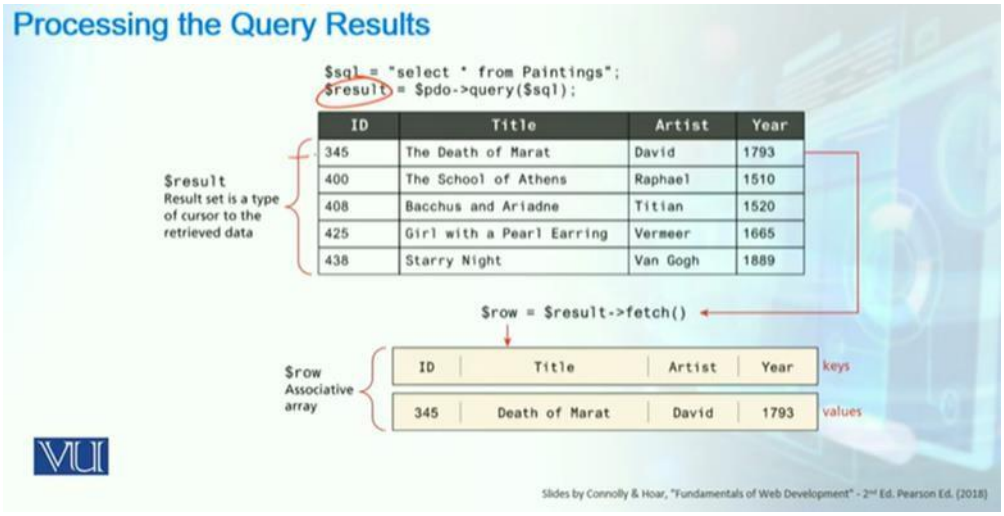
The retrieved result will save in $row. Suppose, there is no result or we have processed all results then $result->fetch () will return a null object and $row=null will get us out of the loop. However, if there are results save in $row, as it is an associative array so its index will be column name. Like in the database we have column name ID so we will get the value of the first row by writing $row ['ID'], same is for category name. In the same fashion all rows will be processed and we will get out of loop.



Result is basically a result set, i.e., type of the cursor to retrieve the data. By this we access all rows one by one.

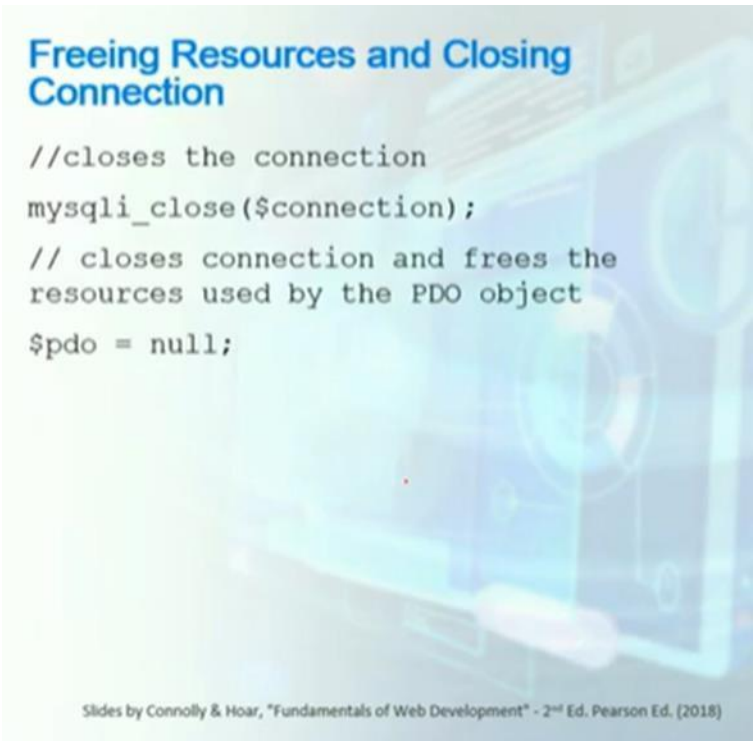
$row = $result->fetch ().

This is an associative array, for one record we will get one row and, in this record, the index will be column name.



Freeing Resources and Closing Connection:

It is important that when you are done with fetching data from the database, then close the connection of the database in the following way.

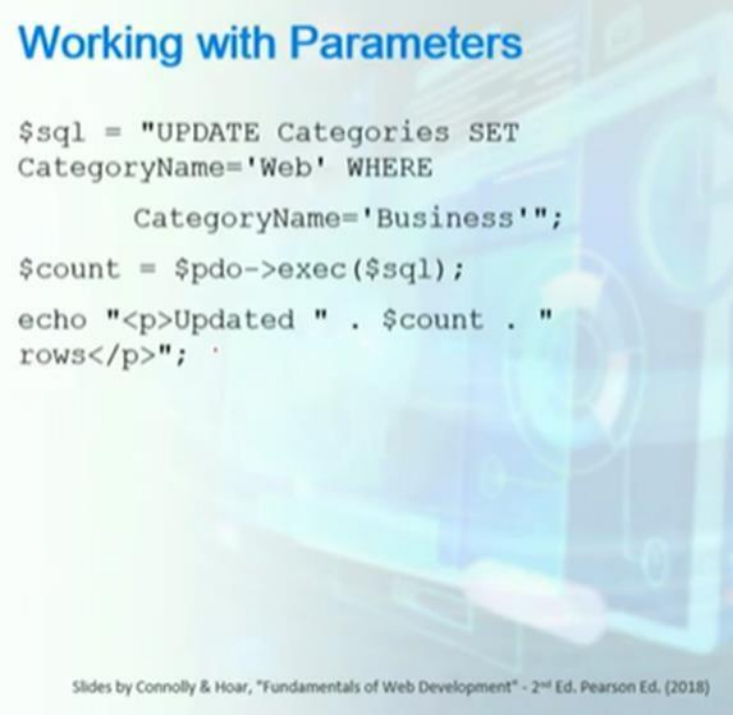


Working with Parameters:

When we execute the query on the server, we have to pass the parameters. There are many ways to pass parameters. For example, to change the name of the category name in the following query.

$sql = "UPDATE Categories SET CategoryName = 'Web' WHERE CategoryName = 'Business'";

Using PDO, call the exec() function and pass the query to it, it will return the count of rows which get affected as a result of this execution.



Working with Parameters – Technique 1? Placeholders:

When you are storing any value in a database or taking value from a user to write SQL query, it is recommended to use placeholders. The placeholders are empty spaces used for passing values, if you don't use placeholders your website gets prone to hacking and specifically for attacks of SQL injections. Using placeholders, you can prevent them to some extent.

For example, we have a table named "books" and want to place values in some fields like:

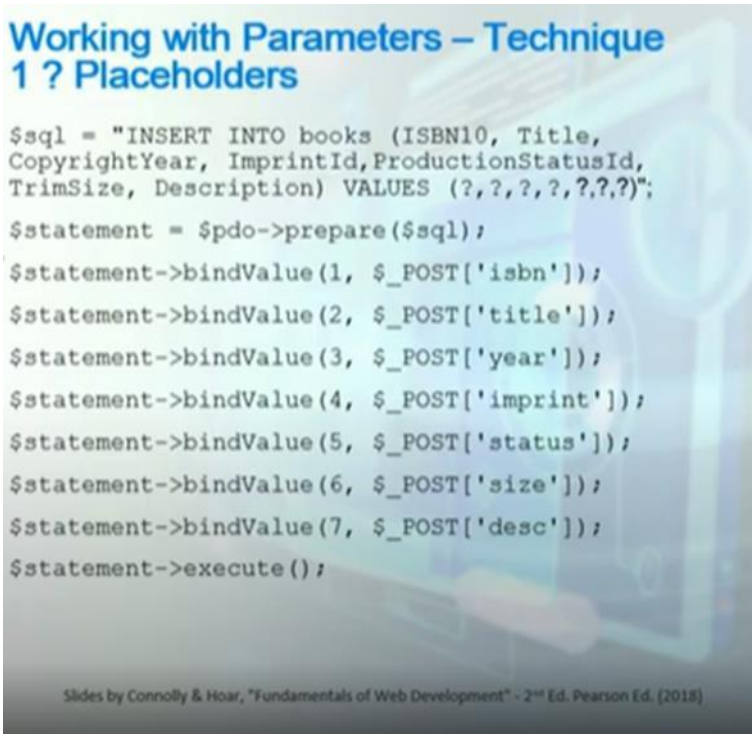
$sql = INSERT INTO books (ISBN0, Title, CopyrightYear, ImprintId, ProductionStatusId, TrimSize, Description) VALUES (?, ?, ?, ?, ?, ?, ?)";

The question marks here are placeholders that need to be filled later on. Before execution of the statement, call the prepare () function and pass the query to this.

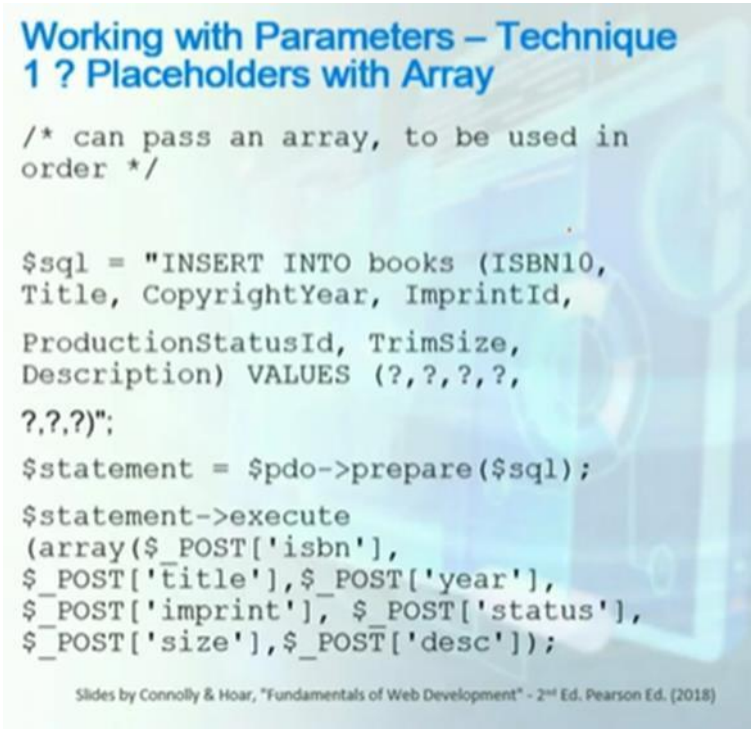
$statement = $pdo->prepare ($sql);

by this prepared statement will get to know how many placeholders are using and how many values it has to accept. After this, for each value we use the bindValue() function like, $statement ->bindValue (1, $_POST ['isbn']);

Do the same for remaining values. The values will be replaced by question marks (placeholder) and upon execution will save in the database.



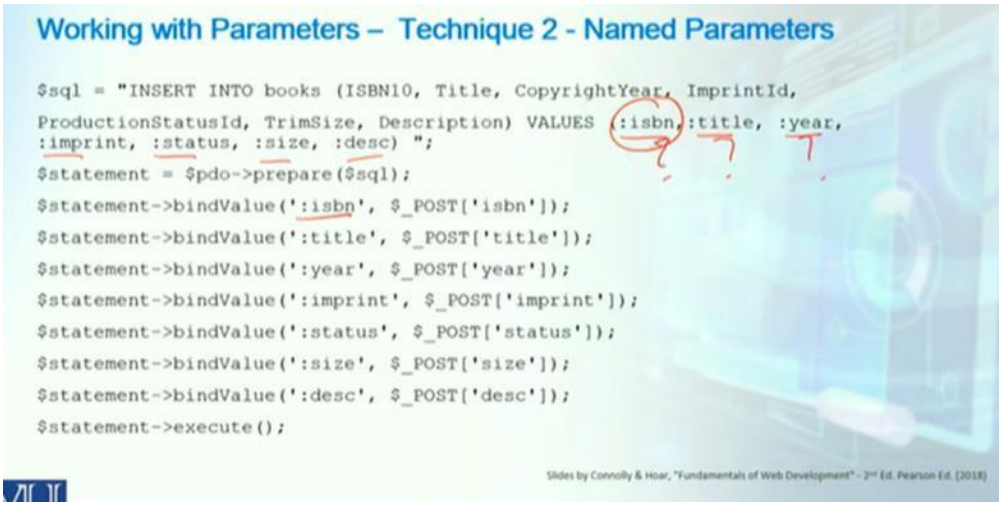
Calling function for each place holder severalty will be time consuming, to solve this issue instead of binding individual values make an array for all values. For this, the main mechanism will remain the same, call the execute function after prepare statement and pass the array to the execute () function.



Working with Parameters- Technique 2 – Named Parameters:

Named parameters can also be used with placeholders. For this, in query write: with the name of value, you want to store like this:

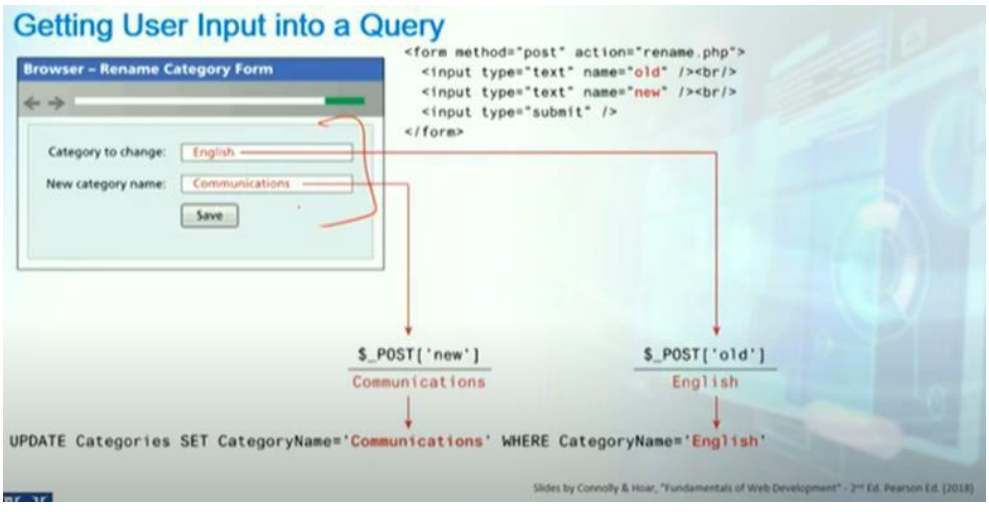
$sql = “INSERT INTO books (ISBN10, Title, CopyrightYear, ImprintId, ProductionStatusId, TrimSize, Description) VALUES (: isbn, :title, :year, :imprint, :status, :size, :desc) “;



After this pass the name of the placeholder in bindValue() function, instead of question marks. In case of using “?” There is a chance of producing a logical bug if the order of values changes, while using the named parameter as placeholder only the appropriate value will go to its place.
You can use an array with the named parameters, the main mechanism will remain the same, while in the execute function, mention the value of the named parameters.

Getting User Input into a Query:

HTML forms are used to get the input from the user, this example is just for learning. In real production you have to take care of the values that are valid or not.



Lesson 120 : Performing transactions

Accessing MySQL in PHP Performing Transactions:

Transaction plays an important role in database management systems, particularly when we are dealing with a group of statements. If you want that group of statements to remain related to each other while executing, and if one statement fails to execute, the remaining statements will stop.
For this every database system provides the facility of transactions.

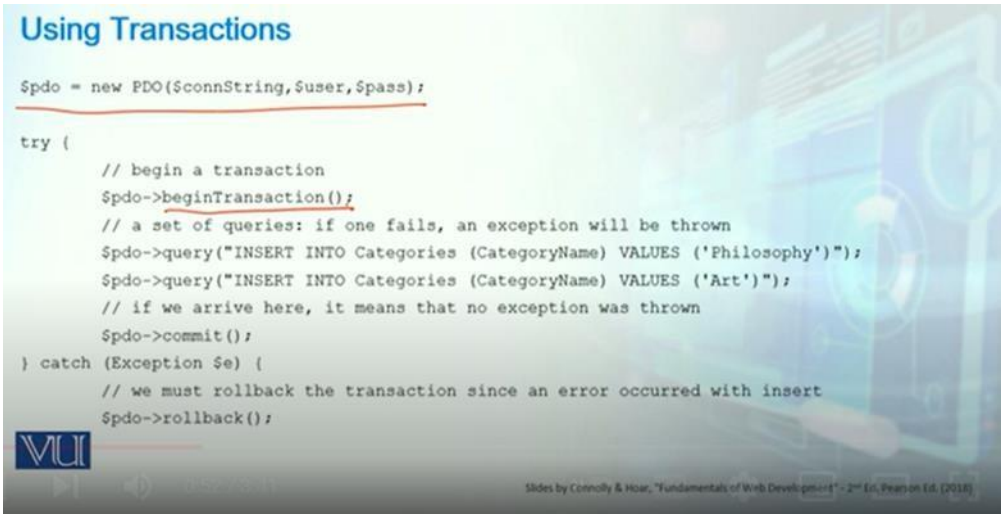
Using Transactions in MySQL:

To use a transaction PDO object is used, which establishes the connection with the database. Once a connection is established, call the function "$pdo->beginTransaction();", it will initiate the transactions.

After initiating transactions, we can execute multiple queries, let's say we have executed some queries and called the commit function as "$pdo->commit ();".

Commit function call will execute all the queries and upon successful execution the program logic will carry on. But if any error occurs in execution of even a single statement/query then an exception will be thrown. Exceptions will be catched by the catch block and will call the roll back function like "$pdo->rollback();". Roll back function will reverse all statements and the database will have no impact.

Suppose, we have two statements, one is used to insert the category of "Philosophy" and the other is "Art". Let's say the first statement executes successfully, while the second statement will give an error so an exception will be thrown, control will shift to catch block and roll back function will be called. Now, the roll back function stops execution of the first statement as well. The value of category" Philosophy" is stored in the database only in a case when there is no exception at the time of calling the commit function.

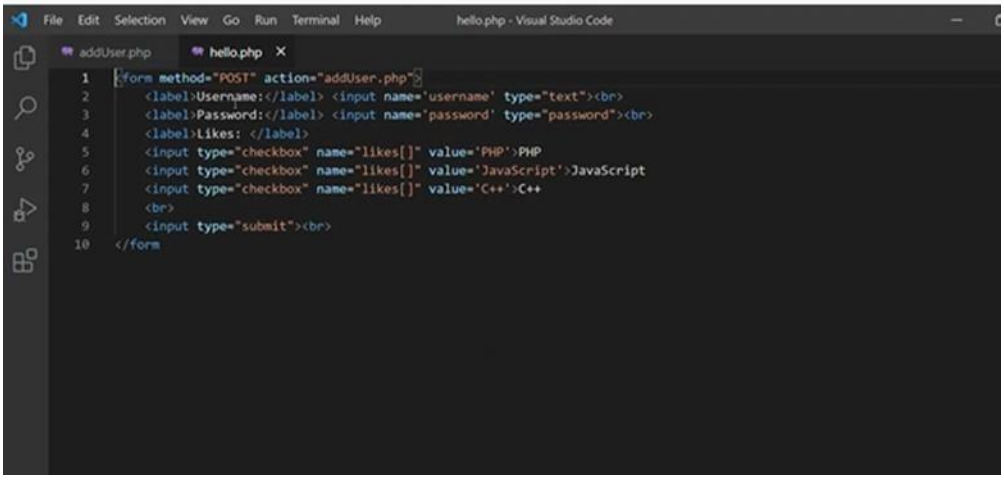


This process is very useful particularly when you are working with a bank database. For example, you have drawn some amount from an account and transferred it into another account. In this case execution of both statements is necessary, in case of error, amount will be deducted from an account and will not be transferred to another account. If you are using a transaction then if an error occurs in the second statement then the first statement will also be rolled back and that deducted amount will be undo.

Lesson 121 : Example: database creation

Example, Database Creation:

Let's work on an example, in which we will create a database and insert values in it. In the previous lecture, we created a form and took input for username and password. After this, in action, user information in PHP was sent to the server, there was a script "addUser.php" that took that information or form data and then displaying on browser.



This page in browser will look like this:



Let's say if we want to save this information in a database. For this, we have to access MySQL using phpMyAdmin. Open the local host to access phpMyAdmin, you can open this directly by typing "localhost/phpmyadmin/". After opening phpMyAdmin interface follows these steps:

- To create a database, go to Databases.
- After creating the database, by default it will ask to add a table.
- To add go to the "Crate table" tab just write the name of the table.
- Whenever you save the record in the database, you add one column for id which is called primary key. You can not take this id form user; it will be created automatically and the database will set it as auto increment.
- So here we need three columns, id, user name, and password so we set Number of columns as 3.
- After creation of table, set the data type for each column such as for id, set the type as INT (integer), Default value as None, and check the "auto increment" box in Index and set PRIMARY, it will become unique.
- For column "username" set the length 50, data type as VARCHAR, and index as UNIQUE, so that every user name should be unique to avoid multiple users with the same name. After that, set the index name as "username".
- For column "password", type is VARCHAR, length as 50. This type is just for practice purposes.
- Create another table for the "likes" of the user programming language by following the same pattern. Remember here to avoid using the word "like" as it is a keyword in SQL.

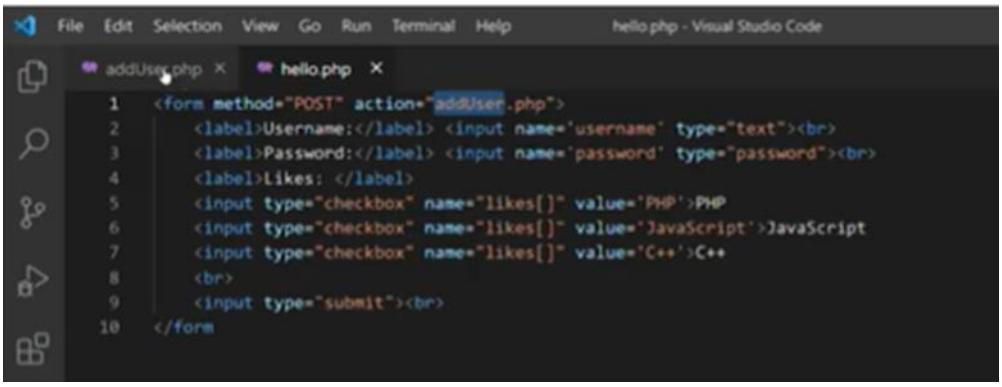
In development, avoid setting the password as text in the database. Before saving the password, hash it using MD5 and store that password hash in the database. The benefit of this is that if you want to recover a password or user forgot password you cannot access the existing password. So, you have to reset the password always.

Lesson 122 : Example: database insertion

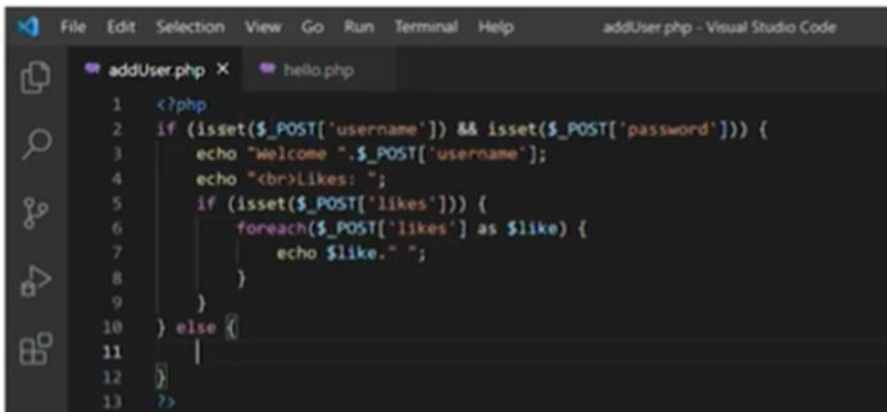
Example – Database Insertion:

After creating the database, read the data from the HTML forms, and send that data to a PHP script to access the data, then insert those values in the database.

Here we have form containing username and password and “addUser.php” is mentioned in its action, when we click on the submit button, the information provided by the user will be submitted to the script.



The following is the script of the “addUser.php”.



In this script the very first thing we have to check is whether the user’s name and password exists or not? So, make sure whenever you take data from the form that data must exist at the php end before putting it into the database. If the data exists then save the values in the variable and save them in the database.

Here, we will create the variable for username and pass and then establish the database connection to insert the values in the database. While creating the database connection, in connection string first mention the type of database which you are accessing like here we are using “pdo”, if you are using any other DBMD then mention that name in place of “sql”. After that mention about the server hosting database, if your database and webserver is on the same machine then write “local host”. After that, write the name of the database. In addition, you have to provide the database user and password which is root for username and password is an empty string at the local host, but in a real development environment you have to create a proper

database user with appropriate privileges with a strong password. After mentioning all requirements, use $pdo for establishing connection.

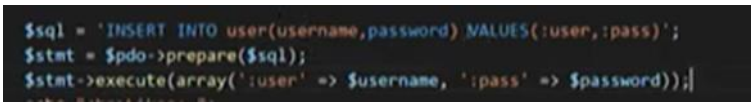


There is a chance that connection doesn't establish, to check this we will write all code in a try catch block like this:



Using try catch we will be able to identify issues causing error, as in case of error it will throw an exception. In the catch block we will catch the PDO exception and use a die or exit method like "die($e->getMessage())". This getMessage() method will read the exception if thrown, display it and exit from the PHP script.

After establishing connection, use the insert statement to insert values in the database. Just make sure to use placeholders in insert statements if you are getting data from the form, it will help you to prevent certain attacks on your website. You can insert direct values but it is never recommended. After writing a sql statement, prepare it through the pdo object. This will give us a statement type object which is used to execute the query by giving values of named placeholders. In case of more than one value in the form of an array.



This insertion step is performed in three steps, it can be done in a single step but not recommended.

If you are not suing named placeholder, using question marks (?) only then the values will be given like this:

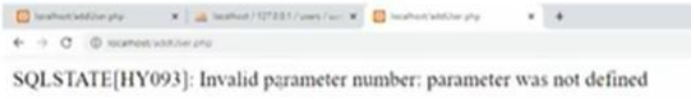
```
$sql = 'INSERT INTO user(username,password) VALUES(?,?)';
$stmt = $pdo->prepare($sql);
$stmt->execute(array($username,$password));
```

In positional placeholder if the order of columns or values get changed it will create a problem, while in using named placeholder it will not create any issue if the order of values/column changed.

Execution of above code will display this page where we input the values which need to be inserted in database.



On submitting the form, we got this error.



This error occurs because we have updated our code with positional placeholder while we are inserting through names placeholders, so after correction code execute it again which will give correct out like this:



To check the values insertion in the database, check the database using phpMyAdmin. Here user id is generated automatically, remember to save the password in hash format.



In the next step we want to insert the values in the “likes” table about the user’s choice in programming languages. But the issue here is that we don’t know the user id while inserting values in php. To resolve this issue, the sql provides us the last insert id so on successful execution of insert statement we can get the id of the user which is inserted last.

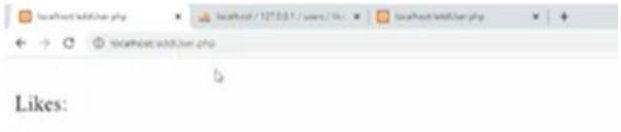
If we have already set the likes variable then we can insert each like in the database. Before this user id is required, to get it we will use a pdo object like “$usrid = $pdo->lastInsertId();” . The insert statement for like table:

```
$userid = $pdo->lastInsertId();
echo "<br>Likes: ";
if (isset($_POST['likes'])) {
    foreach($_POST['likes'] as $like) {
        $sql = 'INSERT INTO likes(userid,likes) VALUES(:userid,:likes)';
        $stmt = $pdo->prepare($sql);
        $stmt->execute(array(':userid' => $userid, ':likes' => $like));
    }
}
```

On executing the above code, it will display the following page where we insert data for user “ahmad”.



On submitting it displays this, which no means no exception is thrown and values are inserted in the database.



Open the database and check the user table first to check the user id of “ahmad” then browse the table “likes” where values are inserted.

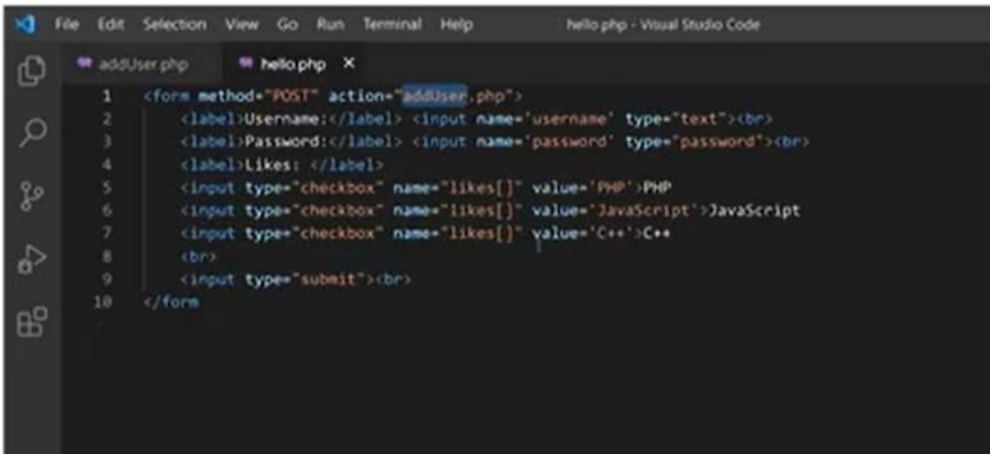


In a normalized database, there will be a separate table for “likes” and have many to many relationships with the “user” table.

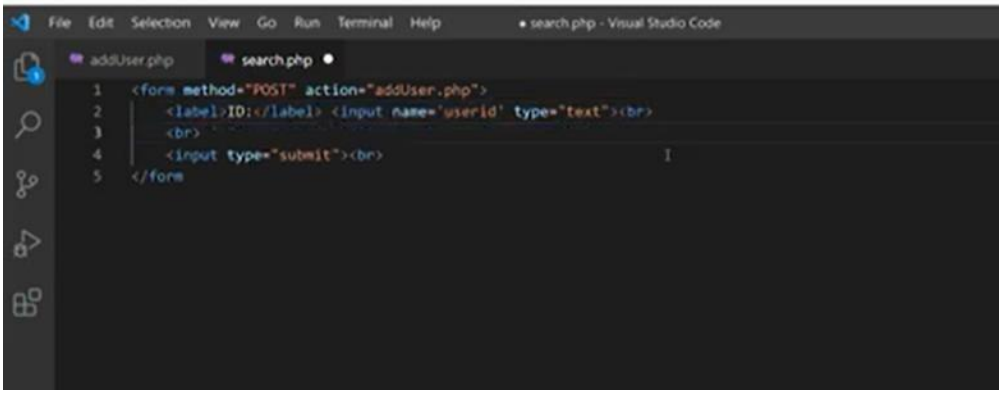
Lesson 123 : Example: database search

Example – Database Search:

After inserting values into the database using PHP, let's see how we can fetch the values from the database and display them in PHP. For this we will make a form for “search”, save it as search.php.



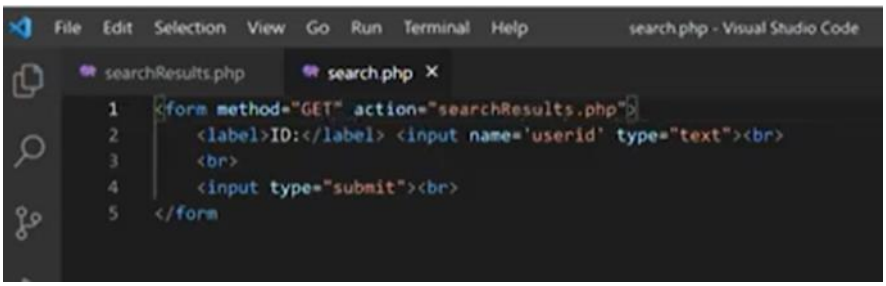
Let’s say pass the user’s name, if it exists then display the user’s name and id. For this create the form and ask for user id as follows:



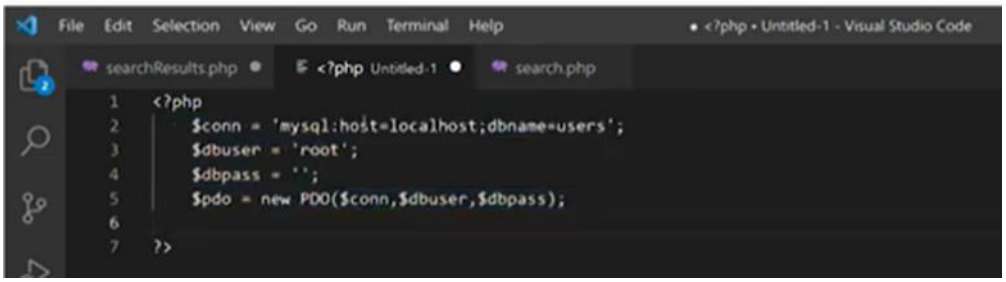
For example, we will pass id “1” here, which will display the name of the user having id “1”.



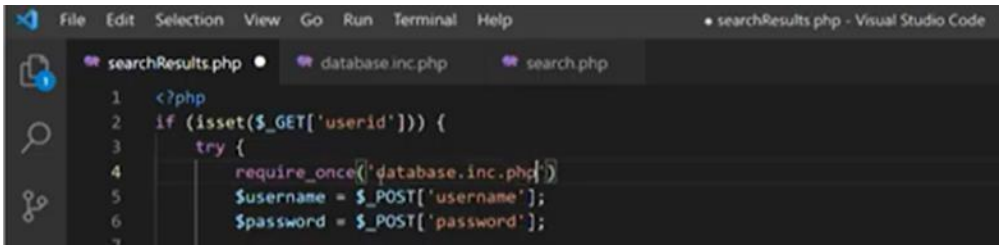
Set the action page as “search_results.php”, also set the method as “GET” and save it. The form we have is “search.php” so when the form is submitted its data goes to action page.



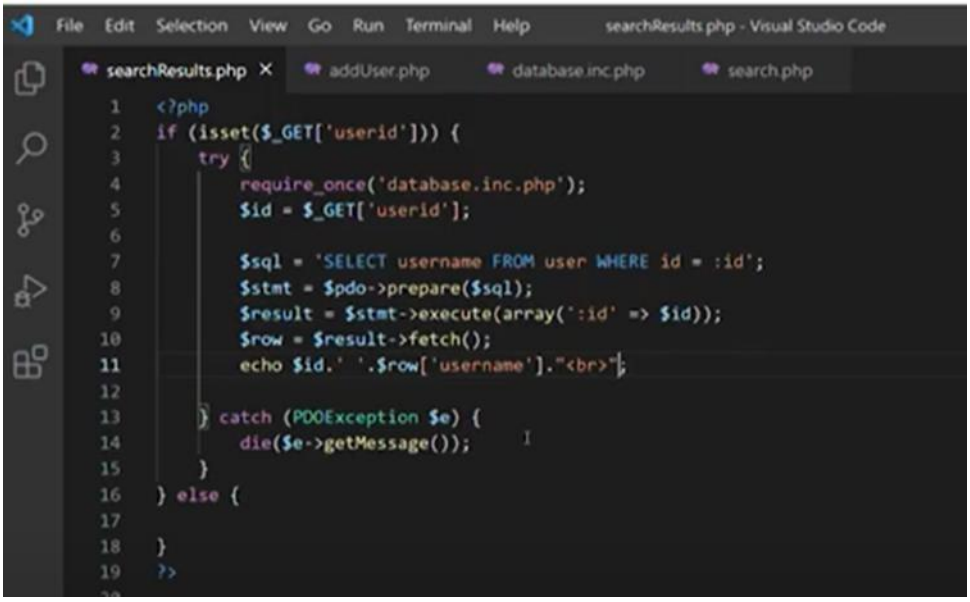
If user id exists and set then create the database connection and specify the user id in this to get the results. The issue of creation of database connection arises if we are dealing with multiple scripts at a time to interact with the database. For this we have to reproduce database connection code for each script, let's say we have to make some changes in db connection then we have to change the complete script. To overcome this issue, create a new file and save the database connectivity code and save it with extension "inc" like "database.inc.php"



After this, include this file where you have to establish the database connection using **require_once (),** so that file is included only once. Do the same with other scripts as well. Remember, if you have a code which is repeating in multiple files, keep that in a single file and include that file where required.



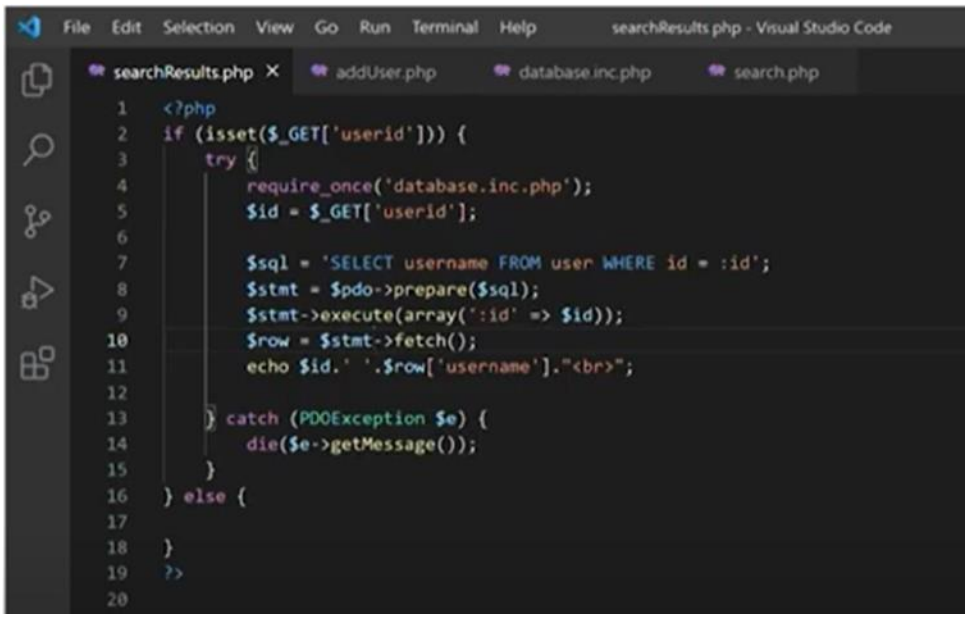
After establishing database connection and getting user id, write the query to fetch the data where id is equal to user id.



On execution of this code, enter the user id as “1”, but it gives a fatal error as result, which is most likely due to the error in the fetch statement. Let’s figure it out.



So, when we are using a fetch statement, we have two options, pdo.query directly or pdo.prepreate or stmt.execute. Here we have used execute, so it will return only true or false value. So, here we will explicitly fetch the result by directly using a statement.



Displaying the code again by giving user id as 1, it will display the following result.



In the same fashion, we can get the information about all users. We can also get information about which user likes which programming language. Let's say if we are fetching records of all users then make the following changes in the code.

```
    $sql = 'SELECT username FROM user WHERE id = :id';
    $stmt = $pdo->prepare($sql);
    $stmt->execute(array(':id' => $id));
    while ($row = $stmt->fetch()) {
        echo $id.' '.$row['username']."<br>";
    }

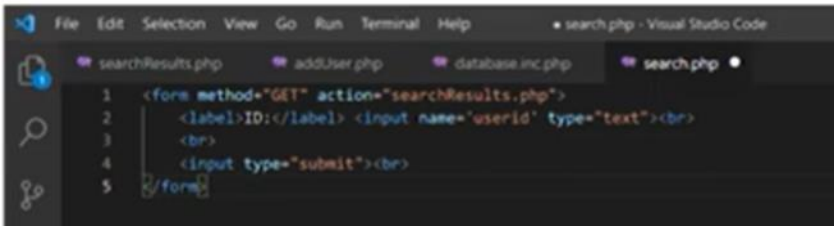
} catch (PDOException $e) {
    die($e->getMessage());
}
```

Remember, if you have to take input from using through form then always use placeholders, although you can access it directly but in that case security issues will arise.

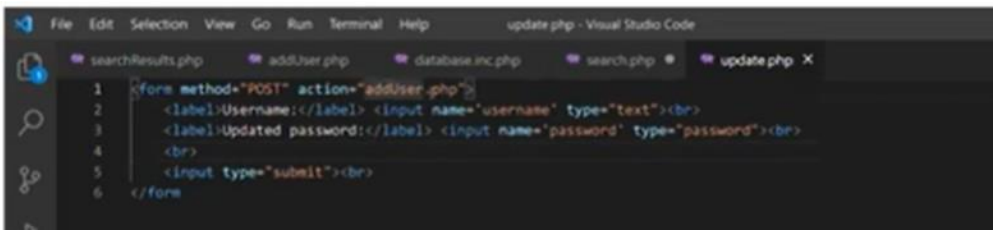
Lesson 124 : Example: database update

Example – Database:

After inserting values in the database lets learn how to update the stored values in the database. Main idea is that the user gives the values in the form which needs to be updated and then these values will be saved in the database. Let's work on the example to understand this. Suppose we have a user and we want to update its password.



Set the field updated password as follows.



After this user will submit this form and call the update user script.

```php
<form method="POST" action="updateUser.php">
    <label>Username:</label> <input name="username" type="text"><br>
    <label>Updated password:</label> <input name="password" type="password"><br>
    <br>
    <input type="submit"><br>
</form
```

It will show the following result on screen.



On giving data in the form and submitting, it will go to updateuser.php and it will update the password of the user given in the input. After this, save the addUser.php with the name of updateUser.php, include the database connectivity in the form of pdo. Now write the update query as follows:

```php
<?php
if (isset($_POST['username']) && isset($_POST['password'])) {
    try {
        require_once('database.inc.php');

        $username = $_POST['username'];
        $password = $_POST['password'];

        $sql = 'UPDATE user SET password = :pass WHERE username = :user';
        $stmt = $pdo->prepare($sql);
        $stmt->execute(array(':user' => $username, ':pass' => $password));
        $userid = $pdo->lastInsertId();
        echo "<br>Likes: ";
```

Execute this code, the return type will show how many records are updated, for this set the count which should be one.

```php
$sql = 'UPDATE user SET password = :pass WHERE username = :user';
$stmt = $pdo->prepare($sql);
$count = $stmt->execute(array(':user' => $username, ':pass' => $password));
```

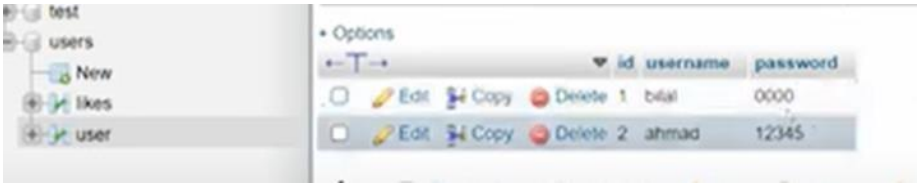
If no record is updated in the result of this execution that means that no record exists against this user. If one record is updated that means the user exists and its password is updated, to check this write the following line of code.

```php
$count = $stmt->execute(array(':user' => $username, ':pass' => $password));
if ($count == 0) {
    echo 'User not found <br>';
} else {
    echo "Record for user $username updated successfully <br>";
}
```

The working of the above code will give following result.



Now check this in the database to verify.



Let say we give the name of the user who doesn't exists, such "khurram" doesn't exist in record then it will display message like this:



We should not have this, so let's figure out why we have got this message? To check this, we will display the $count to check its value, it will be the following output.



In this we are using a prepare and execute statement, if we remove this i.e., directly run the query the count will return us the number of records updated. So, we will use another function like:

```
$sql = 'UPDATE user SET password = :pass WHERE username = :user';
$stmt = $pdo->prepare($sql);
$stmt->execute(array(':user' => $username, ':pass' => $password));
$count = $stmt->rowCount();
if ($count == 0) {
    echo 'User not found <br>';
} else {
    echo "Record for user $username updated successfully <br>";
```

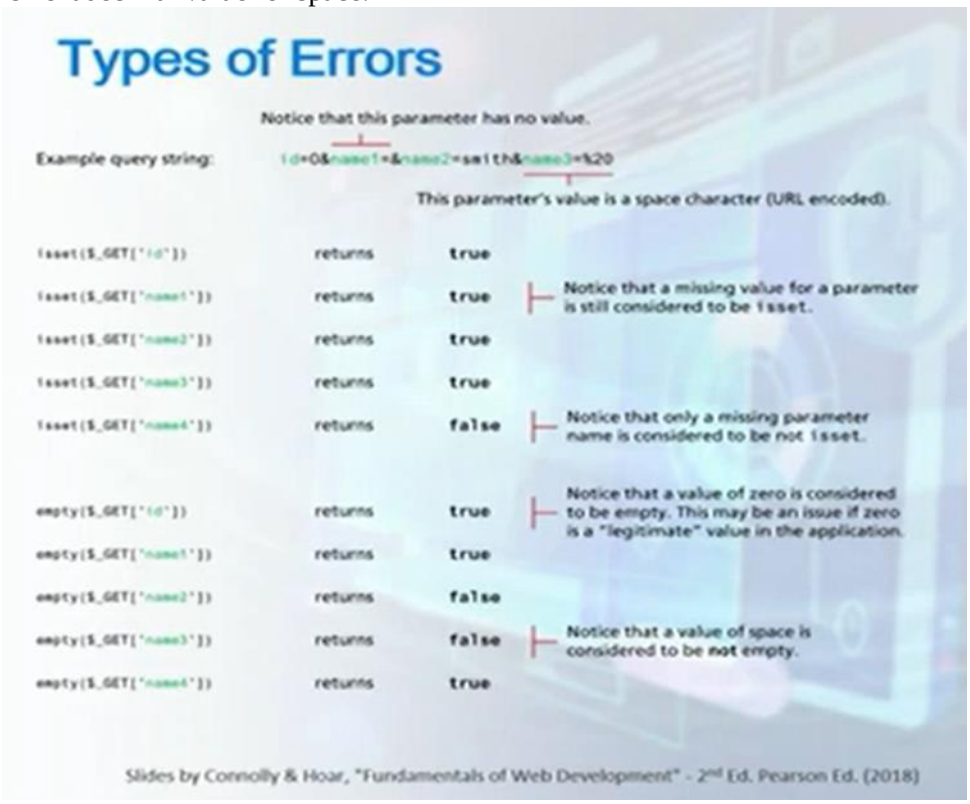
This function will tell us how many rows are updated. So, after this function when we try to update the record of user "khurram" which does exist, it will return us output as "User not found" and if the user exists it will show output as "Record for user ahmad updated successfully".

Remember that the execute statement will return true or false, and to check the number of updated records the rowCount() function will be used.

Week – 10

Lesson 125 : Error handling in PHP; Types of errors

Type of Errors:
There are different types of errors we faced during web development.
Expected errors:
The most common errors expected errors, these errors occur mostly on the base of user input. Like users have not provided the required value or values are not in the required format. Such errors can be checked at runtime and handled logically.
Warnings:
The issues that arise during programming are called warnings and we can deal with them during programming development.
Fatal errors:
The errors which occur after deployment and stop the further execution of program/script are called fatal errors. For example, your web server and database server are running on two different machines, let's say database server went down so when you try to connect with a database it will throw an exception and the program will crash. If the program is handled appropriately,you will give appropriate measures like you can send a message to the user that "Please try again, we are facing some issues". Meanwhile, an email alert can be generated for the database administrator. However, if a fetal occurs you can execute the program further until it is resolved.
As a program it is your responsibility to look into possible errors and check the user's input that it is given in the correct format. For example, if we are taking data from a user either it is in GET form or POST. Here name1 has no value, valid name is given in name2, and in name3 some space like "%20", it is hexadecimal value for space.

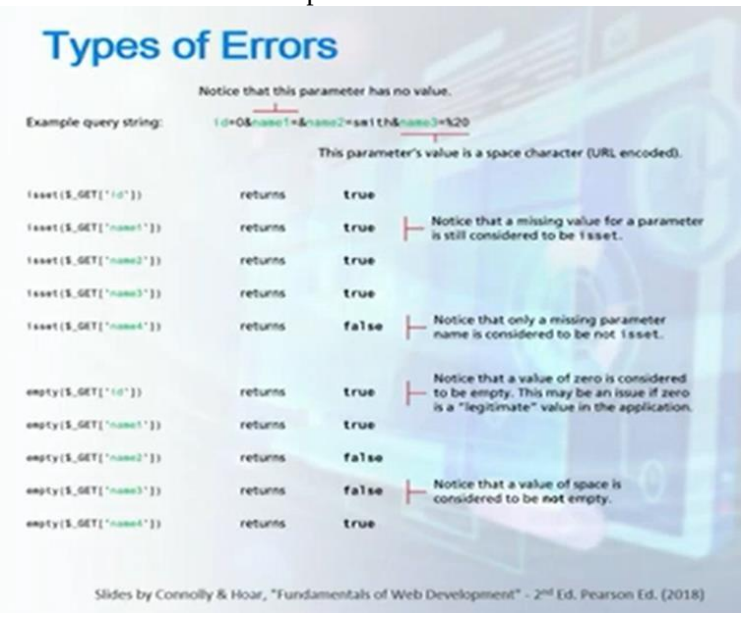


To check the above value, we can use "isset()" and "empty()" functions. The function "isset()" indicates whether the value we want to check or read is available or not, while "empty()" function tells us that if the value exists then it is empty or not. For example, if we pass any variable like id, name1, name2 and

name3 to the “isset ()” function it will return true. This function will not check the value that it exists or not, it is valid or not, it only checks that the variable exists. So, if we pass the variable name4 to the function it will return false as this variable doesn’t exist in the query string.
Similarly, if we pass these variables to “empty ()” function, in case of id it will return true as “id” has zero (0) value and zero stands for false or empty. Name1 is also empty so it will return true, while in name2 and name3 it will return false, and for name4 it will return true as it doesn’t exist.

Lesson 126 : PHP Error Reporting

Type of Errors:
There are different types of errors we faced during web development.
Expected errors:
The most common errors expected errors, these errors occur mostly on the base of user input. Like users have not provided the required value or values are not in the required format. Such errors can be checked at runtime and handled logically.
Warnings:
The issues that arise during programming are called warnings and we can deal with them during programming development.
Fatal errors:
The errors which occur after deployment and stop the further execution of program/script are called fatal errors. For example, your web server and database server are running on two different machines, let’s say database server went down so when you try to connect with a database it will throw an exception and the program will crash. If the program is handled appropriately,you will give appropriate measures like you can send a message to the user that “Please try again, we are facing some issues”. Meanwhile, an email alert can be generated for the database administrator. However, if a fetal occurs you can execute the program further until it is resolved.
As a program it is your responsibility to look into possible errors and check the user’s input that it is given in the correct format. For example, if we are taking data from a user either it is in GET form or POST. Here name1 has no value, valid name is given in name2, and in name3 some space like “%20”, it is hexadecimal value for space.



To check the above value, we can use “isset()” and “empty()” functions. The function “isset()" indicates whether the value we want to check or read is available or not, while “empty()” function tells us that if the value exists then it is empty or not. For example, if we pass any variable like id, name1, name2 and

name3 to the “isset ()” function it will return true. This function will not check the value that it exists or not, it is valid or not, it only checks that the variable exists. So, if we pass the variable name4 to the function it will return false as this variable doesn’t exist in the query string.
Similarly, if we pass these variables to “empty ()” function, in case of id it will return true as “id” has zero (0) value and zero stands for false or empty. Name1 is also empty so it will return true, while in name2 and name3 it will return false, and for name4 it will return true as it doesn’t exist.

Lesson 127 : PHP Error and Exception handling

PHP Error and Exception handling
When a fatal PHP error occurs, program execution will eventually terminate unless it is handled. The PHP documentation provides two mechanisms for handling runtime errors: procedural error handling and object-oriented exception handling.
Procedural Error Handling
In the procedural approach to error handling, the programmer needs to explicitly test for error conditions after performing a task that might generate an error.

```
$connection = mysqli_connect(DBHOST, DBUSER, DBPASS, DBNAME);
$error = mysqli_connect_error();
if ($error != null) {
// handle the error
…
}
```

The advantage of the try … catch mechanism is that it allows the developer to handle a wider variety of exceptions in a single catch block. Yet, even with explicit testing for error conditions, there will still be situations when an unforeseen error occurs. In such a case, unless a custom error handler has been defined, PHP will terminate the execution of the application.
Object-Oriented Exception Handling
When a runtime error occurs, PHP throws an exception. If an exception is not caught, then eventually the PHP environment will handle it by terminating execution with an “Uncaught Exception” message. Like other object-oriented programming languages, PHP uses the try … catch programming construct to programmatically deal with exceptions at runtime.
Example of try … catch block

```
// Exception throwing function
function throwException($message = null,$code = null) {
throw new Exception($message,$code);
}
try {
// PHP code here
$connection = mysqli_connect(DBHOST, DBUSER, DBPASS, DBNAME)
or throwException(“error”);
…
}
catch (Exception $e) {
echo ' Caught exception: ' . $e->getMessage();
echo ' On Line : ' . $e->getLine();
echo ' Stack Trace: '; print_r($e->getTrace());
} finally {
// PHP code here that will be executed after try or after catch
}
```

The finally block is optional. Any code within it will always be executed after the code in the try or in the catch blocks, even if that code contains a return statement. It is typically used if the developer wants certain things done regardless of whether an exception occurred, such as closing a connection or

removing temporary files. However, the finally block is only available in PHP 5.5 and later, which was released in June 2013.

Lesson 128 : Regular Expressions

Regular Expressions

A regular expression is a set of special characters that define a pattern. They are a type of language that is intended for the matching and manipulation of text. In web development they are commonly used to test whether a user's input matches a predictable sequence of characters, such as those in a phone number, postal or zip code, or email address.

Regular Expression Syntax

A regular expression consists of two types of characters: literals and metacharacters. A literal is just a character you wish to match in the target (i.e., the text that you are searching within). A metacharacter is a special symbol that acts as a command to the regular expression parser. There are 14 metacharacters described in the php implementation. To use a metacharacter as a literal, you will need to escape it by prefacing it with a backslash (\).

Table : Common Regular Expression Patterns

Pattern	Description
^ qwerty $	If used at the very start and end of the regular expression, it means that the entire string (and not just a substring) must match the rest of the regular expression contained between the ^ and the $ symbols.
\t	Matches a tab character.
\n	Matches a new-line character.
.	Matches any character other than \n.
[qwerty]	Matches any single character of the set contained within the brackets.
[^qwerty]	Matches any single character not contained within the brackets.
[a-z]	Matches any single character within range of characters.
\w	Matches any word character. Equivalent to [a-zA-Z0-9].
\W	Matches any nonword character.
\s	Matches any white-space character.
\S	Matches any nonwhite-space character.
\d	Matches any digit.
\D	Matches any nondigit.
*	Indicates zero or more matches.
+	Indicates one or more matches.
?	Indicates zero or one match.
{n}	Indicates exactly n matches.
{n,}	Indicates n or more matches.
{n, m}	Indicates at least n but no more than m matches.
|	Matches any one of the terms separated by the | character. Equivalent to Boolean OR.
()	Groups a subexpression. Grouping can make a regular expression easier to understand.

Lesson 129 : Regular Expressions – Examples

Regular Expressions – Examples

In PHP, regular expressions are contained within forward slashes. So, for instance, to define a regular expression, you would use the following:

$pattern = '/ran/';
It should be noted that regular expression pattern checks are case sensitive. Regular expressions can be complicated to visually decode; to help, this section will use the convention of alternating between red and blue to indicate distinct subpatterns in an expression and black text for literals.
This regular expression will find matches in all three of the following strings.
'randy connolly'
'Sue ran to the store'
'I would like a cranberry'
To perform the pattern check in PHP, you would write something similar to the following:
$pattern = '/ran/';
$check = 'Sue ran to the store';
if (preg_match($pattern, $check)) {
echo 'Match found!';
}
To perform the same pattern check in JavaScript, you would write something similar to the following:
var pattern = /ran/;
if (pattern.test('Sue ran to the store')) {
document.write('Match found!');
}
In JavaScript a regular expression is its own data type. Just as a string literal begins and ends with quote characters, in JavaScript, a regular expression literal begins and ends with forward slashes.

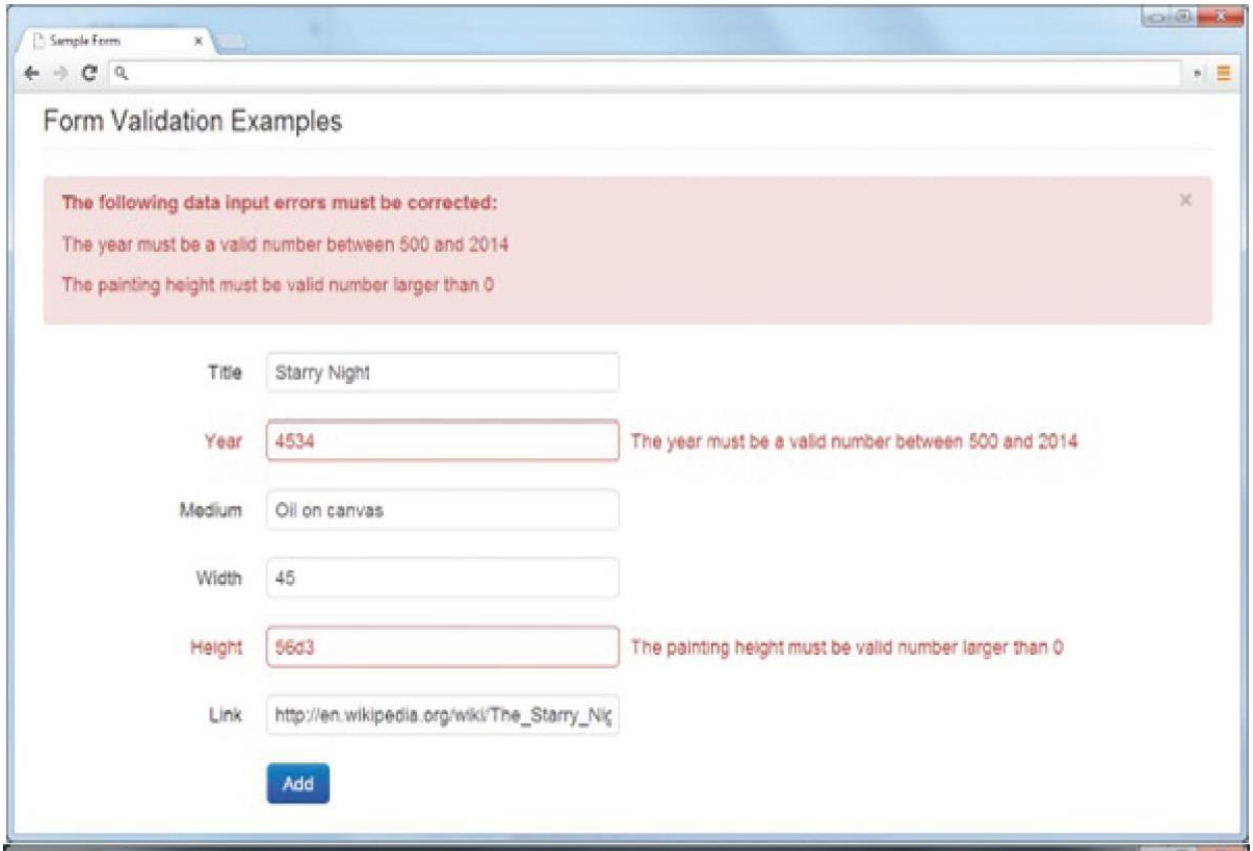
Lesson 130 : PHP Filters

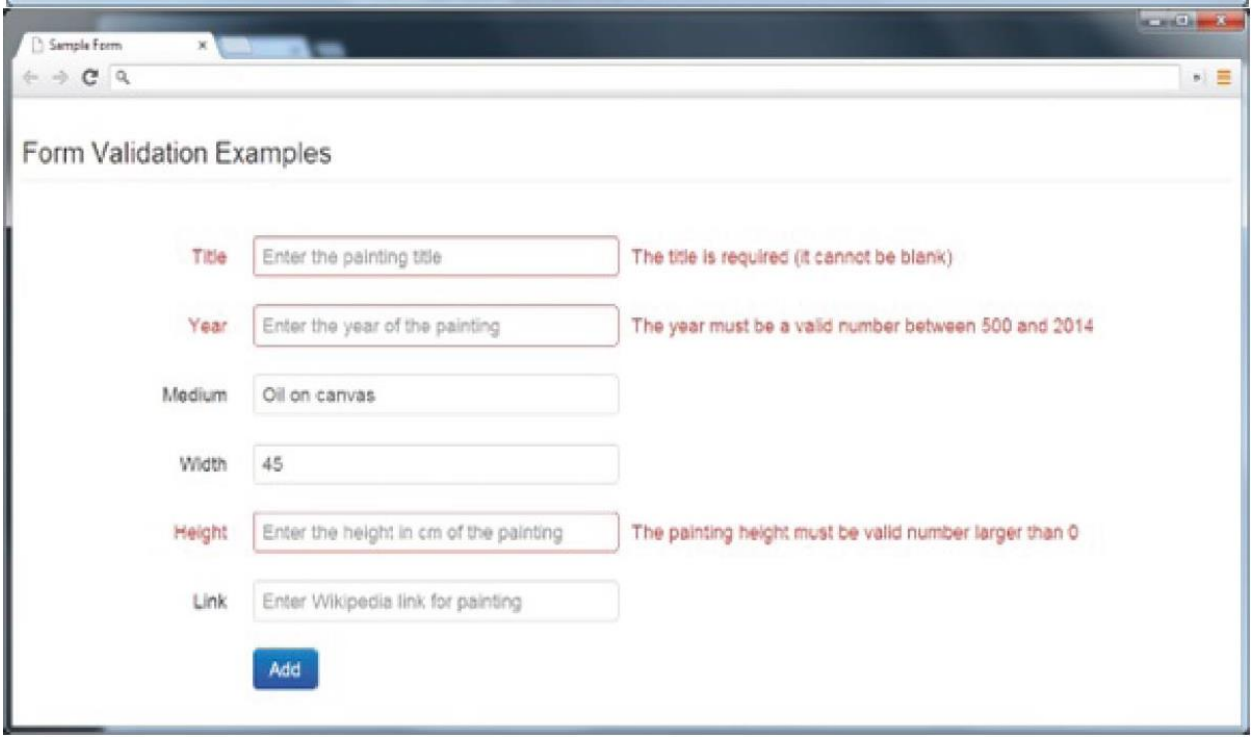
PHP Filters
In PHP, filters are used to validate the input against pre-defined patterns. There are two types of filters: validate filters and sanitize filters. Following function is used for aforementioned task:
Filter_var($value,FILTER)
Validate Filters
• Check input against a pre-defined pattern
• Validate email (FILTER_VALIDATE EMAIL)
$email = filter_var('abc@mail.com', FILTER_VALIDATE_EMAIL);
//$email = abc@mail.com
$email = filter_var('abc@.com', FILTER_VALIDATE_EMAIL);
//$email=null
Other validate filters
FILTER_VALIDATE_BOOL, FILTER_VALIDATE_FLOAT, FILTER_VALIDATE_INT
FILTER_VALIDATE_MAC
FILTER_VALIDATE_IP
FILTER_VALIDATE_URL
FILTER_VALIDATE_REGEXP
Validate against a regular expression (FILTER_VALIDATE_REGEXP)
$cnic = filtervar('12345-1234567-8', FILTER_VALIDATE_REGEXP, array("options" => array("regexp" => "/"\\d{5}-?\\d{7}-?\\d$/")));
if($cnic == null) {
echo "Invalid CNIC";
} else {
echo "Valid CNIC $cnic";
}
Sanitize Filters
Firstly, Remove or replace specific characters
FILTER_SANITIZE_EMAIL

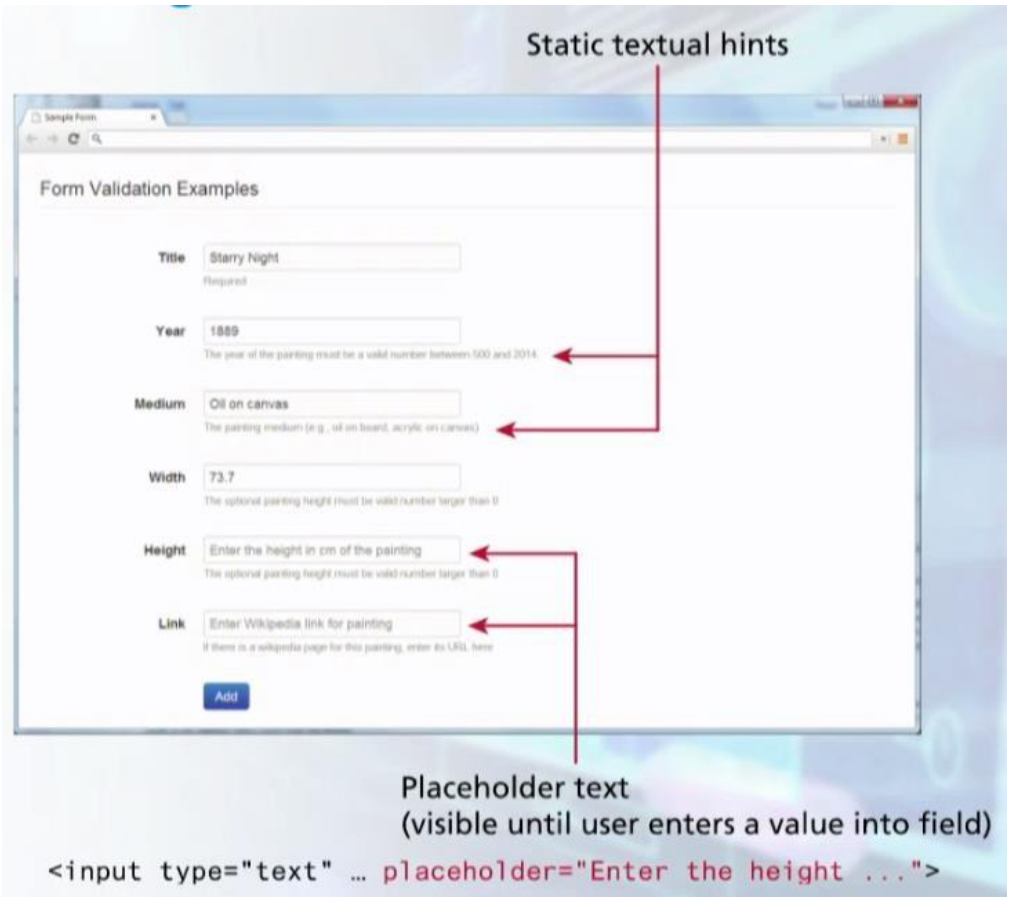
Removes all characters except letters, digits and !#$°/0&'*+-=?Ai{1}7©.[]
Filter_var(‘my(email)@mail/.com/’, SANITIZE) == 'myemail@mail.com'
FILTER_SANITIZE_STRING
Strips HTML tags
filter_var('<h1>A" B</h1>',FILTER_SANITIZE_FULL_SPECIAL_CHARS)=='A" ' B '
FILTER_SANITIZE_FULL_SPECIAL_CHARS
filter_var('<h1>A"\'B</h1>',FILTER_SANITIZE_FULL_SPECIAL_CHARS)== '&1t;h1>A" ' B&It;/h 1 >'
Following are few sanitize filters:
FILTER_SANITIZE_ADD_SLASHES
FILTER_SANITIZE_ENCODED
FILTER_SANITIZE_NUMBER_FLOAT
FILTER_SANITIZE_NUMBER_INT
FILTER_SANITIZE_SPECIAL_CHARS
FILTER_SANITIZE_URL

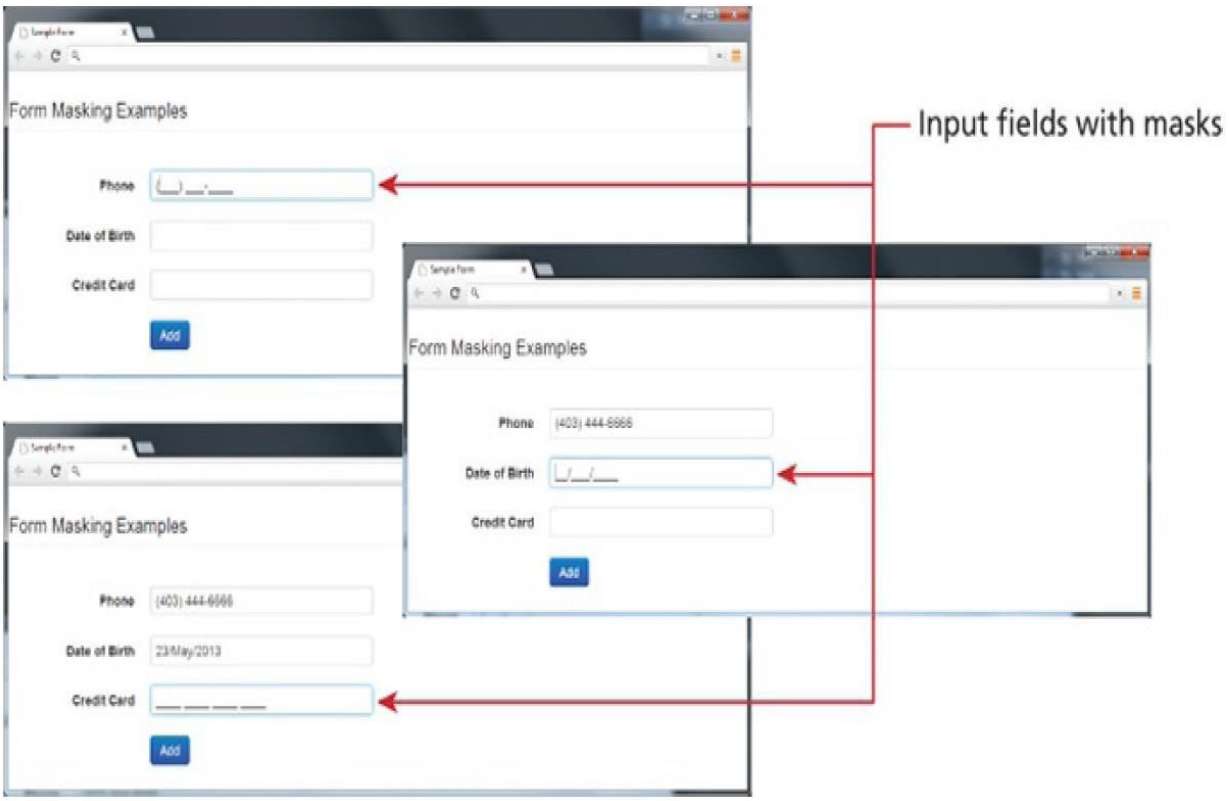
Lesson 131 : Validating user input

Validating User Input
The following list indicates most of the common types of user input validation.
Required information: Some data fields just cannot be left empty. For instance, the principal name of things or people is usually a required field. Other fields such as emails, phones, or passwords are typically required values.
Correct data type. While some input fields can contain any type of data, other fields, such as numbers or dates, must follow the rules for its data type in order to be considered valid.
Correct format: Some information, such as postal codes, credit card numbers, and social security numbers have to follow certain pattern rules. It is possible, however, to go overboard with these types of checks.
Comparison: Some user-entered fields are considered correct or not in relation to an already-inputted value. Perhaps the most common example of this type of validation is entering passwords: most sites require the user to enter the password twice and then a comparison is made to ensure the two entered values are identical.
Range check: Information such as numbers and dates have infinite possible values: However, most systems need numbers and dates to fall within realistic ranges. For instance, if you are asking a user to input her birthday, it is likely you do not want to accept January 1, 214 as a value; it is quite unlikely she is 1800 years old! As a result, almost every number or date should have some type of range check performed.
Custom: Some validations are more complex and are unique to a particular application. Some custom validations can be performed on the client side.
Notifying the User


Sample Form
Form Validation Examples
The following data input errors must be corrected:
The year must be a valid number between 500 and 2014
The painting height must be valid number larger than 0
Title
Starry Night
Year
4534
The year must be a valid number between 500 and 2014
Medium
Oil on canvas
Width
45
Height
56d3
The painting height must be valid number larger than 0
Link
http://en.wikipedia.org/wiki/The_Starry_Nig
Add


Sample Form
Form Validation Examples
Title
Enter the painting title
The title is required (it cannot be blank)
Year
Enter the year of the painting
The year must be a valid number between 500 and 2014
Medium
Oil on canvas
Width
45
Height
Enter the height in cm of the painting
The painting height must be valid number larger than 0
Link
Enter Wikipedia link for painting
Add

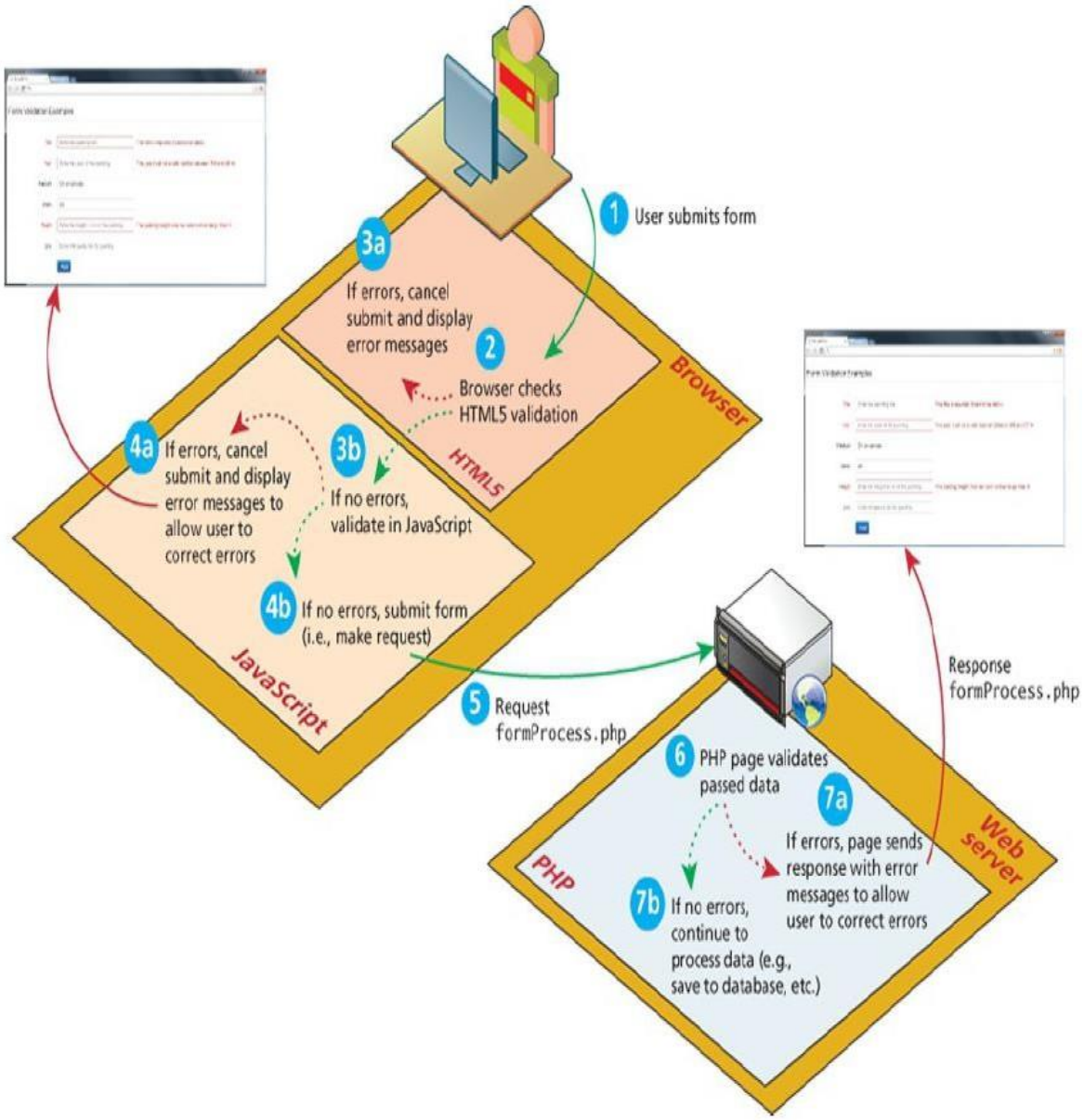

Static textual hints
Form Validation Examples
Title
Starry Night
Year
1889
Medium
Oil on canvas
Width
73.7
Height
Enter the height in cm of the painting
Link
Enter Wikipedia link for painting
Add
Placeholder text
(visible until user enters a value into field)
<input type="text" … placeholder="Enter the height ...">
Pop-up tool tip
(appears when mouse
hovered over icon)
Form Validation Examples
Required
Title
Enter the painting title
Year
Enter the year
Medium
Enter the medium type of the painting
Width
Enter the width in cm of
Height
Enter the height in cm o
Link
Enter Wikipedia link for painting
Add
Hint
If there is a wikipedia page for this painting, enter its URL here
Pop-over



Lesson 132 : Where to perform validation

Where to perform validation

Validation can be performed at three different levels. With HTML5, the browser can perform basic validation with no need for any JavaScript. However, since the validation that can be achieved in HTML5 is quite basic, most web applications also perform validation in the browser using JavaScript. The advantage of validation using JavaScript is that it reduces server load and provides immediate feedback to the user. Unfortunately, JavaScript validation cannot be relied on: for instance, it might be turned off on the user's browser. For these reasons, validation must always be done on the server side. Indeed, you should always perform the same validity checks on both the client in JavaScript and on the server in PHP, but server-side validation is the most important since it is the only validation that is guaranteed to run. Below figure illustrates the interaction of the different levels of validation.



Validation at the PHP Level

No matter how good the HTML5 and JavaScript validation, client-side prevalidation can always be circumvented by hackers, or turned off by savvy users. Validation on the server side using PHP is the most important form of validation and the only one that is absolutely essential. In this case, we will be validating the query string parameters rather than the form elements directly as with JavaScript.

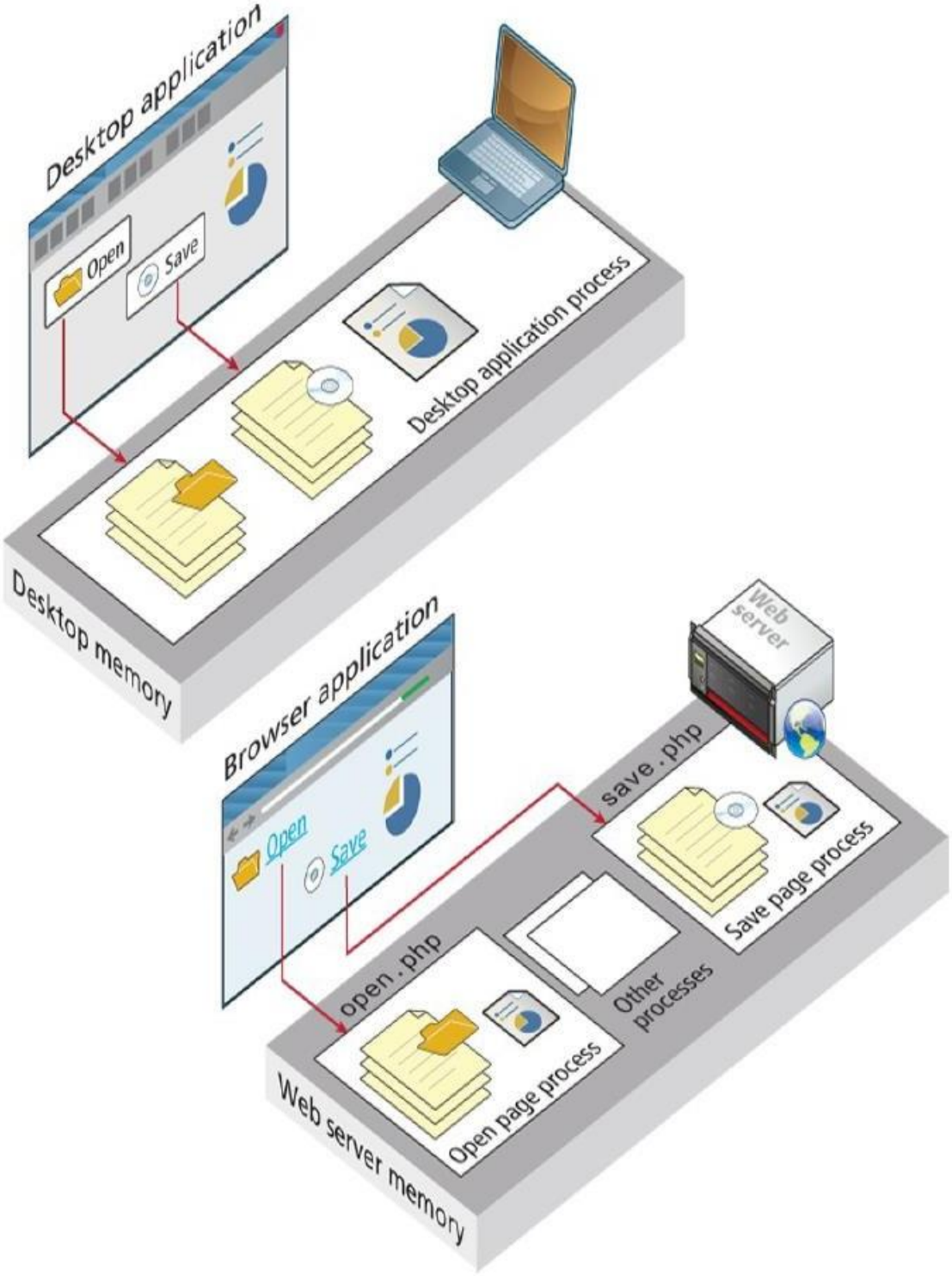
Lesson 133 : The problem of state management; State management on server side

The Problem of State Management

Almost all applications need to process user inputs, output information, and read and write from databases or other storage media. But in this chapter we will be examining a development problem that is unique to the world of web development: how can one request share information with another request?

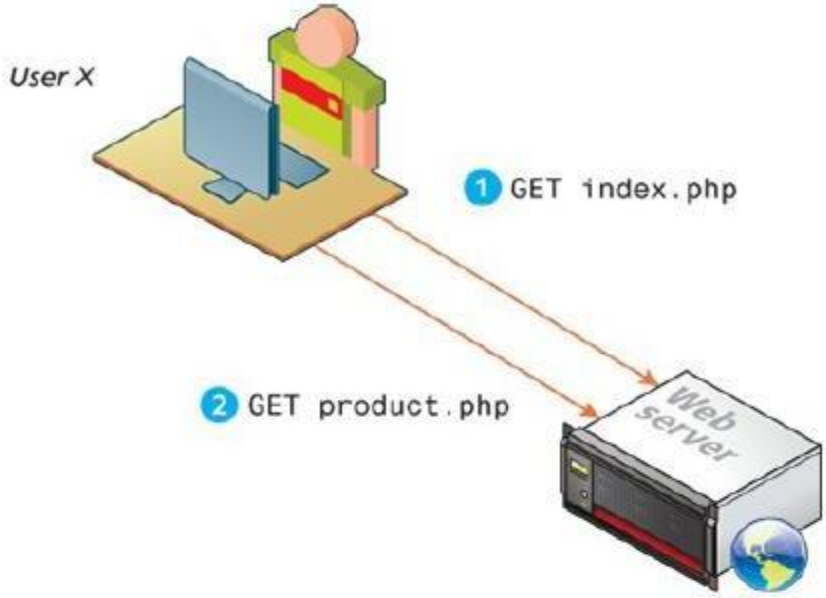
Single-user desktop applications do not have this challenge at all because the program information for the user is stored in memory and can thus be easily accessed throughout the application. Yet one must always

remember that web applications differ from desktop applications in a fundamental way. Unlike the unified single process that is the typical desktop application, a web application consists of a series of disconnected HTTP requests to a web server where each request for a server page is essentially a request to run a separate program, as shown in below figure.

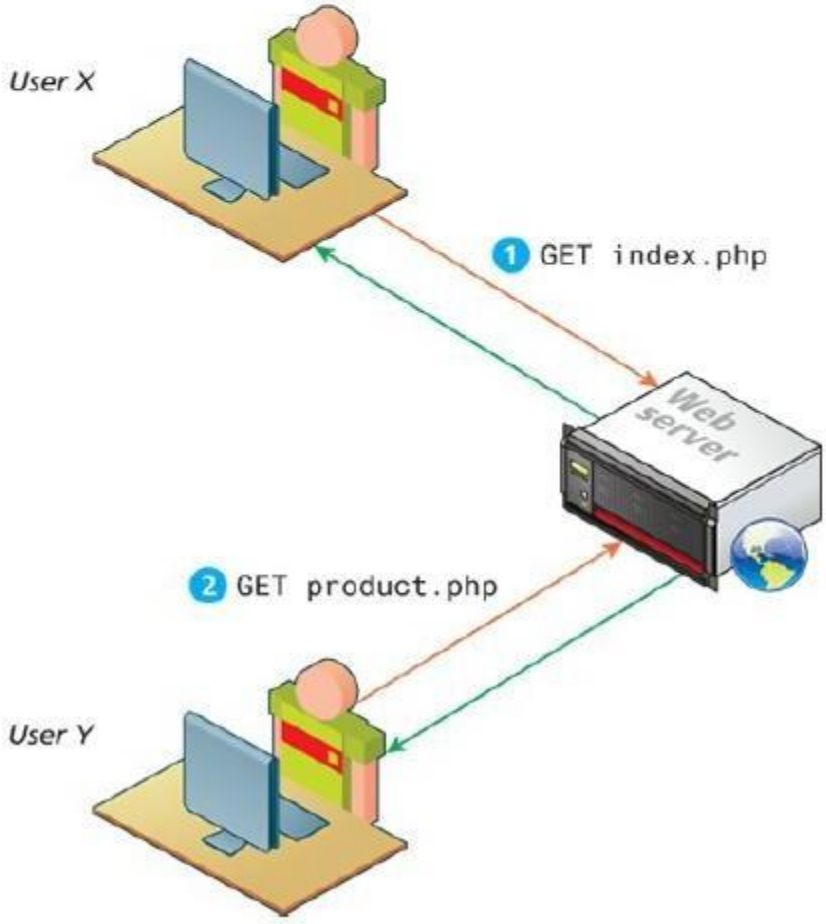


Furthermore, the web server sees only requests. The HTTP protocol does not, without programming intervention, distinguish two requests by one source from two requests from two different sources, as

shown in below figure



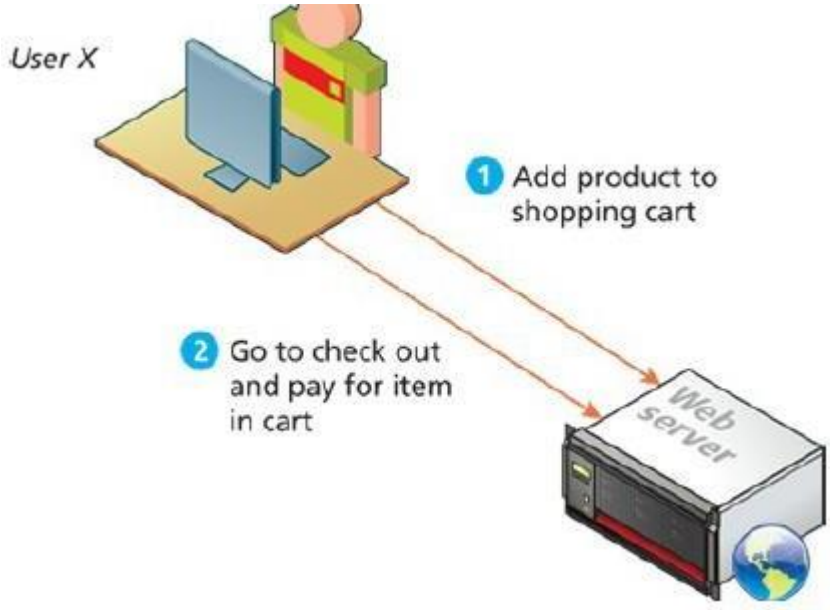
... is for the server not really any different than ...



.

While the HTTP protocol disconnects the user's identity from his or her requests, there are many occasions when we want the web server to connect requests together. Consider the scenario of a web

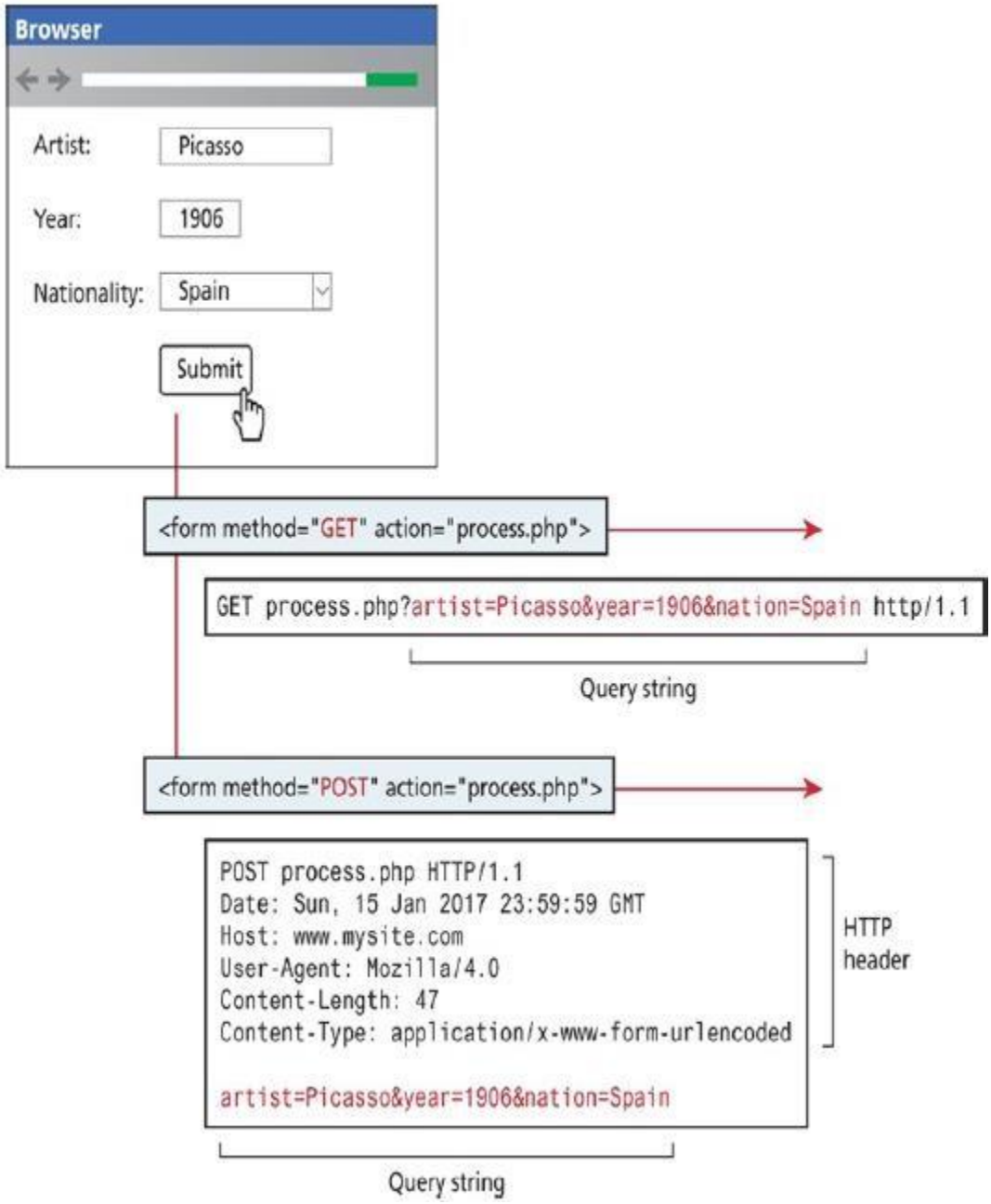
shopping cart, as shown in the below Figure. In such a case, the user most certainly wants the server to recognize that the request to add an item to the cart and the subsequent request to check out and pay for the item in the cart are connected to the same individual.



Lesson 134 : State management using query string

State management using query string

A web page can pass query string information from the browser to the server using one of the two methods: a query string within the URL (GET) and a query string within the HTTP header (POST). Below Figure reviews these two different approaches.



Dynamic vs Static URLs

• Dynamic URLs (i.e., query strings)

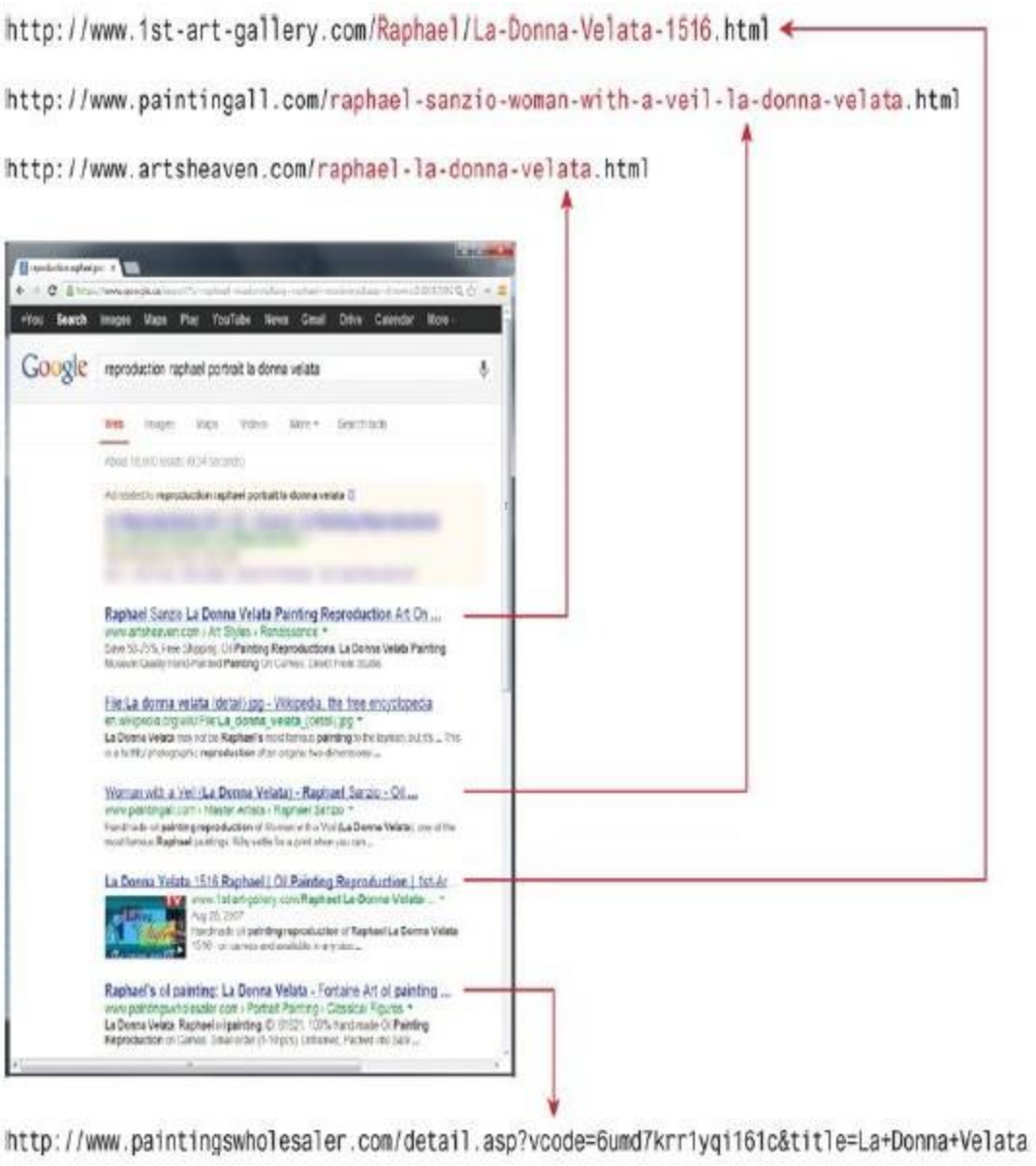
A pretty essential part of web application development

• Static URLs

The appearance of search terms within the URL does seem to improve its relative position. Another benefit to static URLs is that users tend to prefer them.

• Can use URL rewriting to make static URLs be dynamic

Depending on your web development platform, there are different ways to implement URL rewriting. On web servers running Apache, the solution typically involves using the mod_rewrite module in Apache along with the .htaccess file. The mod_rewrite module uses a rule-based rewriting engine that utilizes Perl-compatible regular expressions to change the URLs so that the requested URL can be mapped or redirected to another URL internally.



Lesson 135 : Cookies

Cookies

Cookies are a way to **save information on the user's computer** using their **browser**. This helps websites **remember things** about the user, like their preferences or login details.

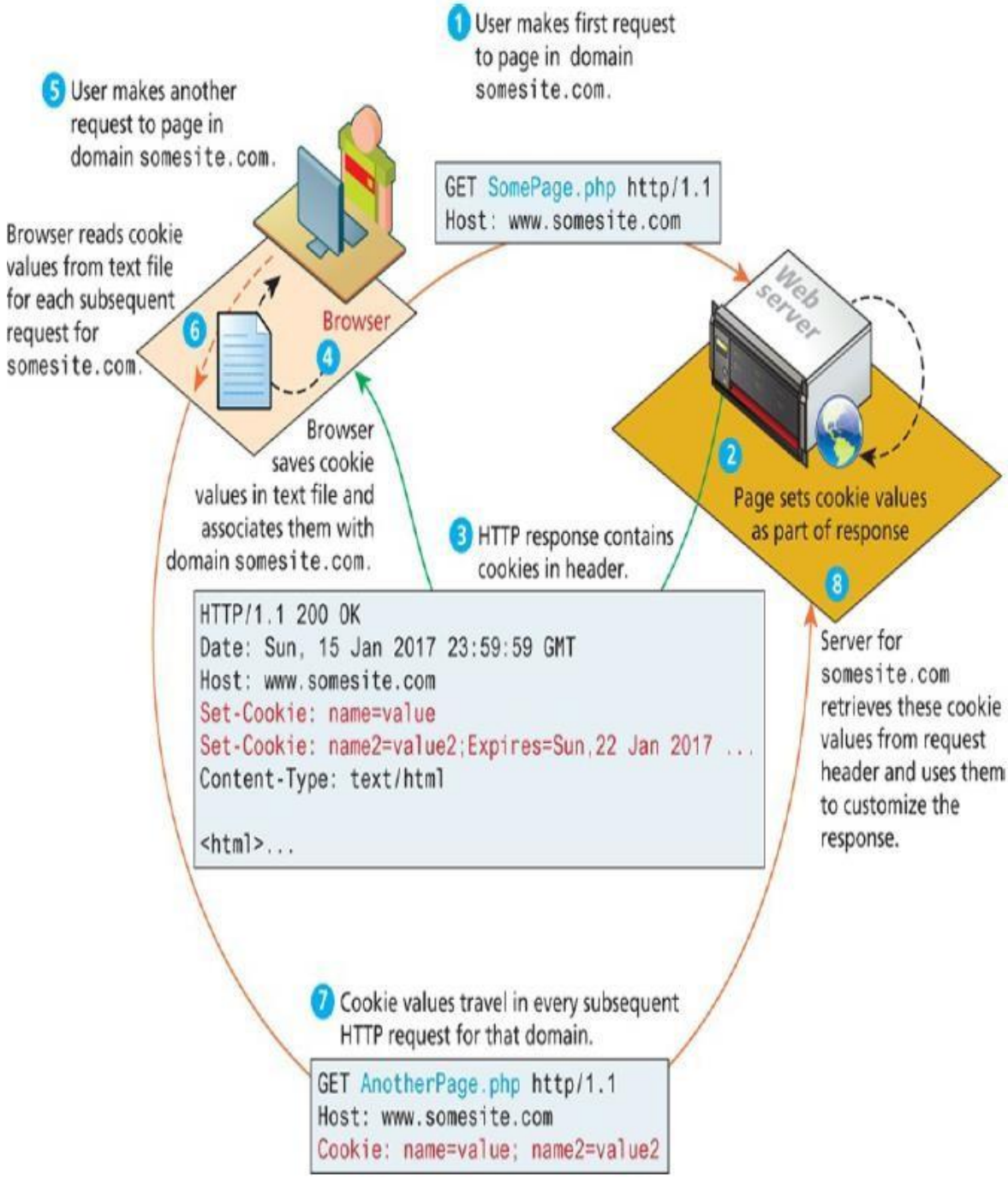
Cookies are meant to **store information for a long time**. They are **linked to a website's domain**, not just one page. So, **no matter which page you visit on the site**, the browser and the website can **share cookie data**.

Each website gets its own cookies. The browser **keeps cookies for each site separate**, so cookies from one site **won't be shared** with another site.

.

How Do Cookies Work

While cookie information is stored and retrieved by the browser, the information in a cookie travels within the HTTP header. Below figure illustrates how cookies work.



Due to the limitations of cookies (both in terms of size and reliability), your site's correct operation should not be dependent upon cookies. Nonetheless, the user's experience might be improved with the judicious use of cookies.

It is important to note that cookies must be written before any other page output.

```
<?php
// add 1 day to the current time for expiry time
$expiryTime = time()+60*60*24;
// create a persistent cookie
$name = "Username";
```

```
$value = "Ricardo";
setcookie($name, $value, $expiryTime);
?>
```

The setcookie() function also supports several more parameters, which further customize the new cookie. Notice that when we read a cookie, we must also check to ensure that the cookie exists. In PHP, if the cookie has expired (or never existed in the first place), then the client's browser would not send anything, and so the $_COOKIE array would be blank.

Lesson 136 : Cookies – Example

Reading a cookie

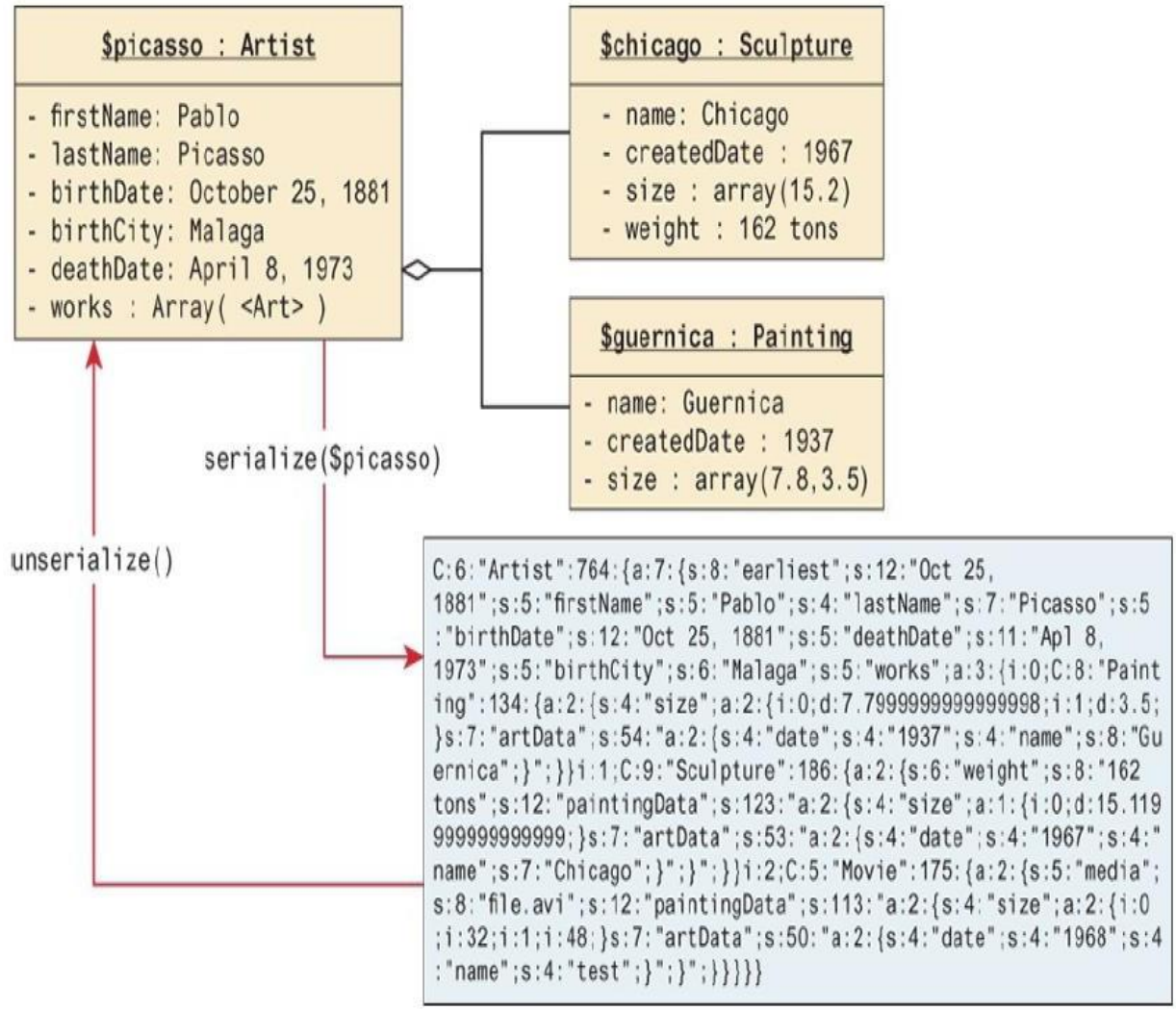
```
<?php
if( !isset($_COOKIE['Username']) ) {
//no valid cookie found
}
else {
echo "The username retrieved from the cookie is:";
echo $_COOKIE['Username'];
}
?>
```

Many sites provide a "Remember Me" checkbox on login forms, which relies on the use of a persistent cookie. This login cookie would contain the user's username but not the password. Instead, the login cookie would contain a random token; this random token would be stored along with the username in the site's back-end database. Every time the user logs in, a new token would be generated and stored in the database and cookie. Another common use of cookies is to track a user's browsing behavior on a site. Some sites will store a pointer to the last requested page in a cookie; this information can be used by the site administrator as an analytic tool to help understand how users navigate through the site.

Lesson 137 : Serialization

Serialization

Serialization is the process of taking a complicated object and reducing it down to zeros and ones for either storage or transmission. Later that sequence of zeros and ones can be reconstituted into the original object as illustrated in the below Figure.



Artist class modified to implement the Serializable interface

```
class Artist implements Serializable {
//...
// Implement the Serializable interface methods
public function serialize() {
// use the built-in PHP serialize function
return serialize(
array(“earliest” =>self::$earliestDate,
“first” => $this->firstName,
“last” => $this->lastName,
“bdate” => $this->birthDate,
“ddate” => $this->deathDate,
“bcity” => $this->birthCity,
“works” => $this->artworks
)
);
}
public function unserialize($data) {
// use the built-in PHP unserialize function
$data = unserialize($data);
```

```
self::$earliestDate = $data['earliest'];
$this->firstName = $data['first'];
$this->lastName = $data['last'];
$this->birthDate = $data['bdate'];
$this->deathDate = $data['ddate'];
$this->birthCity = $data['bcity'];
$this->artworks = $data['works'];
}
//...
}
```

Note that in order for our Artist class to save successfully, the Art, Painting, and other classes must also implement the Serializable interface. It should be noted that references to other objects stored at the same time will be preserved while references to objects not serialized in this operation will be lost. Although nearly unreadable to most people, this data can easily be used to reconstitute the object by passing it to unserialize(). If the data above is assigned to $data, then the following line will instantiate a new object identical to the original:

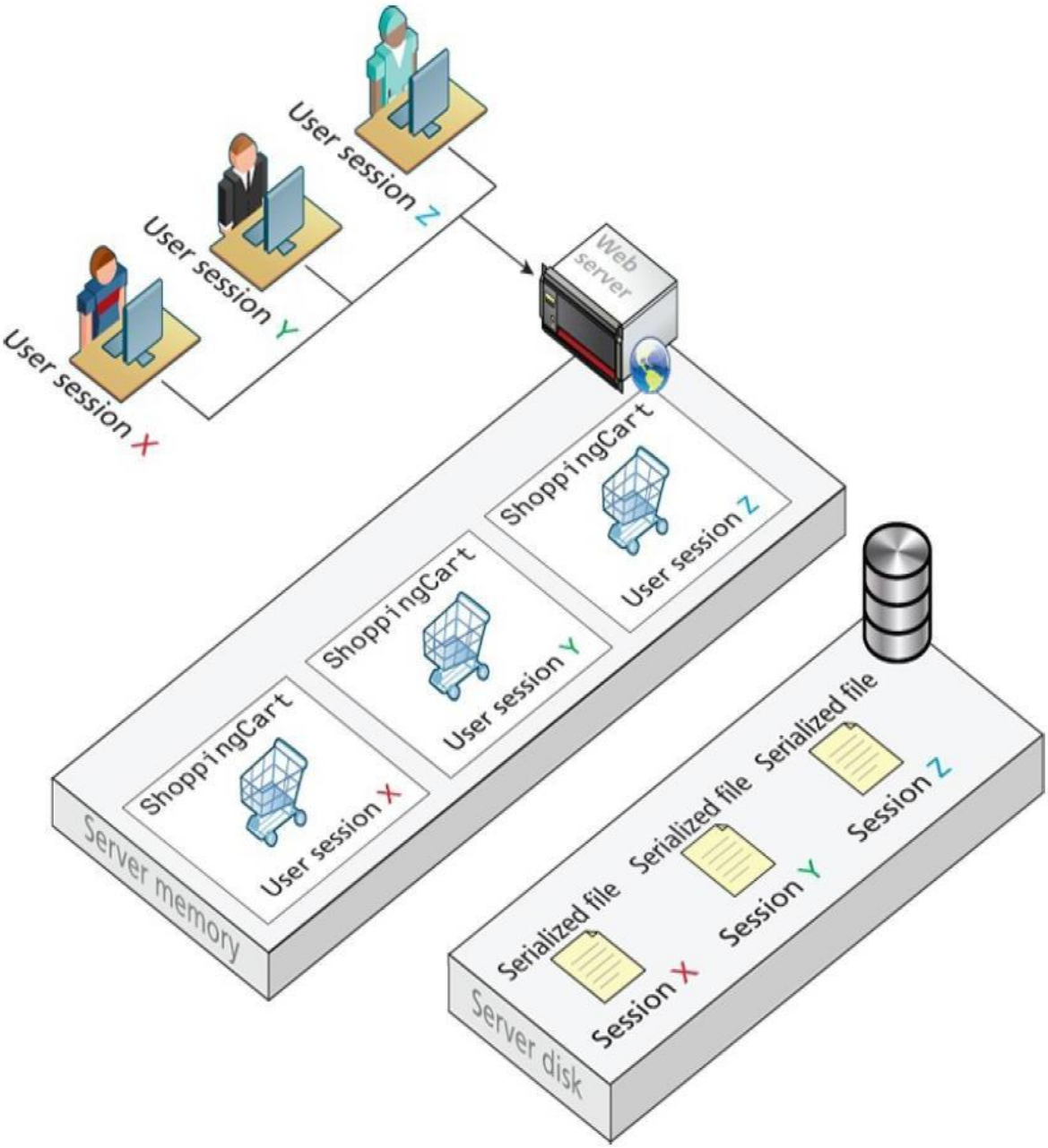
```
$picassoClone = unserialize($data);
```

Week – 11

Lesson 138 : Session state; using sessions

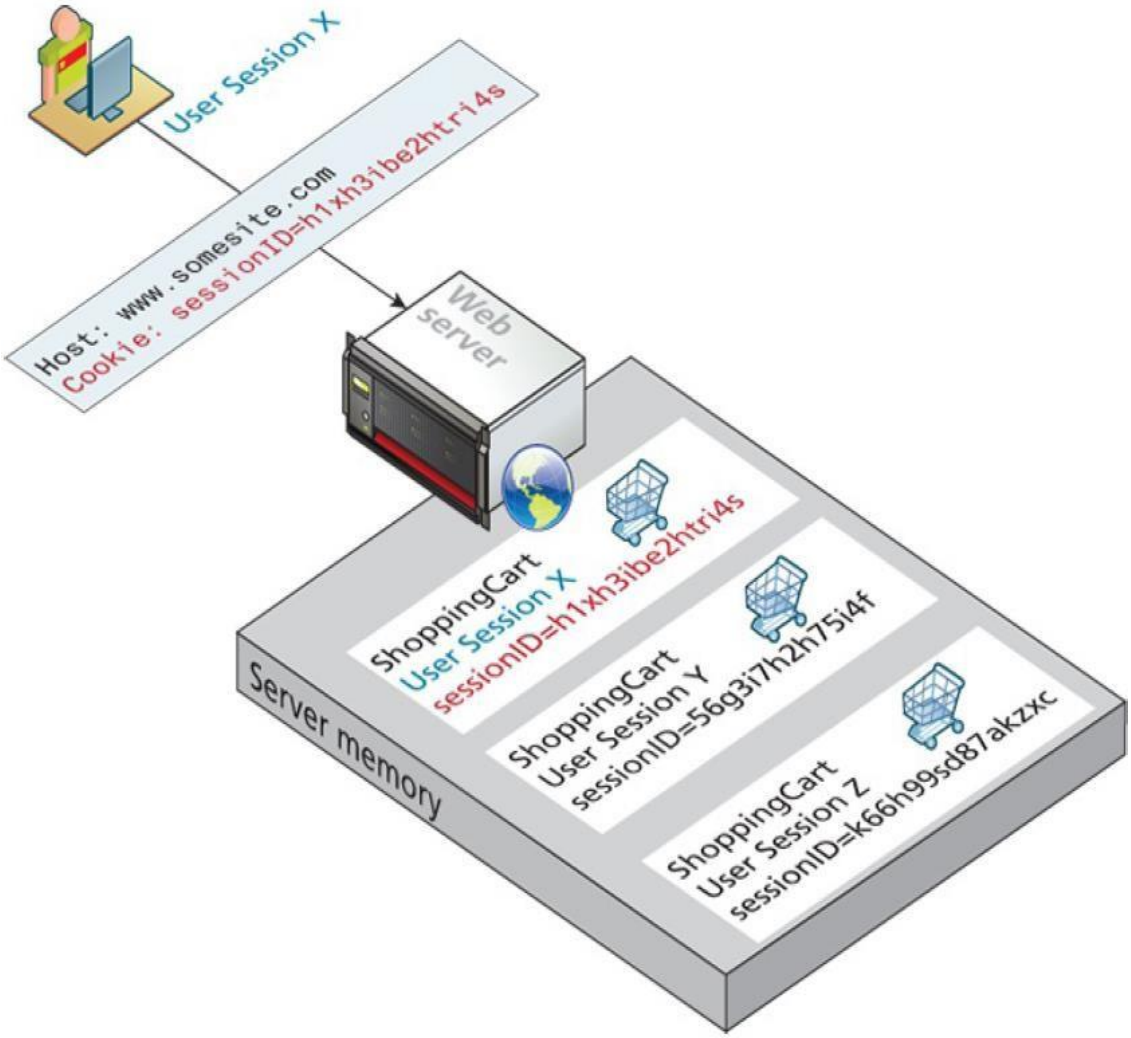
Session State using sessions

All modern web development environments provide some type of session state mechanism. Session state is a server-based state mechanism that lets web applications store and retrieve objects of any type for each unique user session. That is, each browser session has its own session state stored as a serialized file on the server, which is deserialized and loaded into memory as needed for each request, as shown in the below figure.



How Does Session State Work?

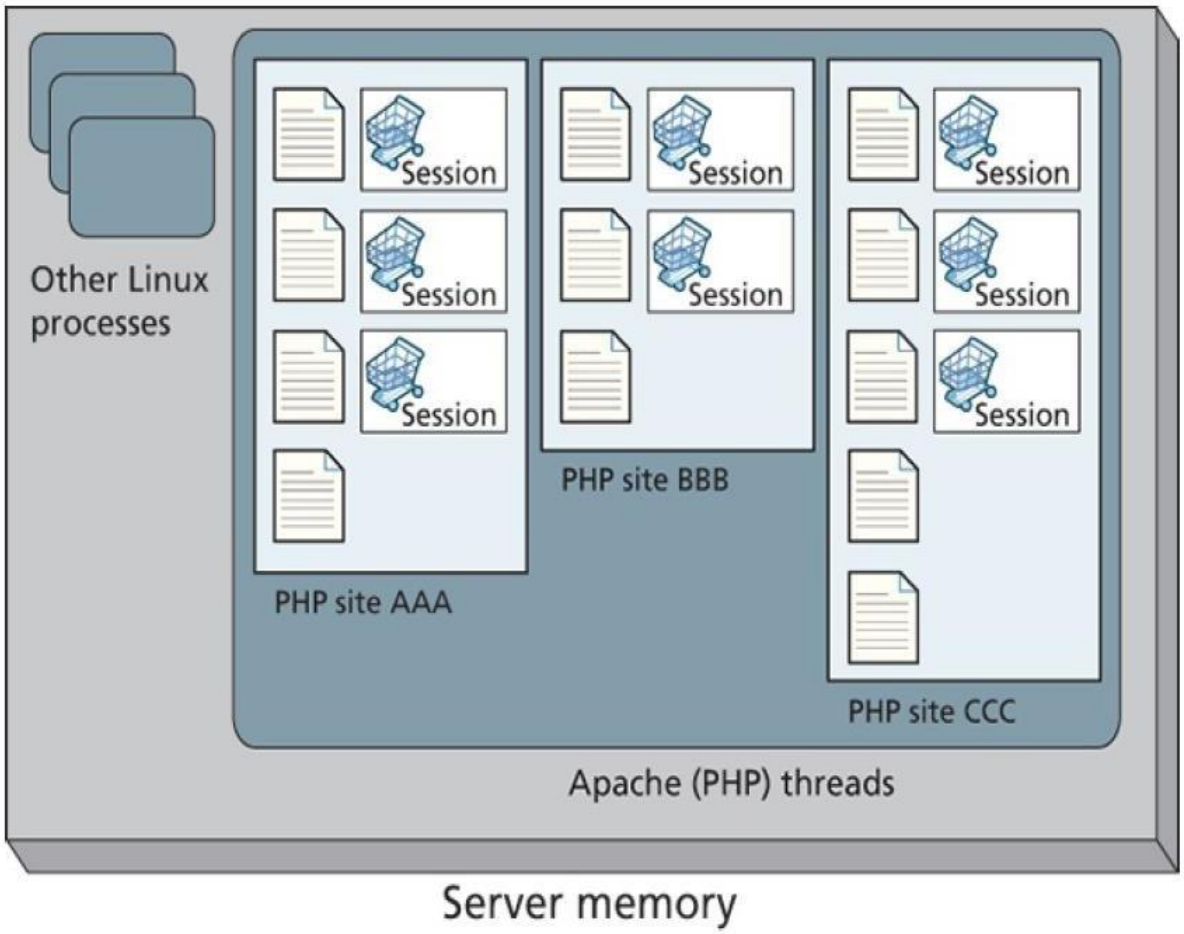
Modern development environments such as ASP.NET and PHP make session state remarkably easy to work with, it is tempting to see session state as a one-stop solution to all web state needs. However, if we take a closer look at how session state works, we will see that session state has the same limitations and issues as the other state mechanisms. The first thing to know about session state is that it works within the same HTTP context as any web request. The server needs to be able to identify a given HTTP request with a specific user request. Since HTTP is stateless, some type of user/session identification system is needed. Sessions in PHP (and ASP.NET) are identified with a unique session ID. In PHP, this is a unique 32-byte string that is by default transmitted back and forth between the user and the server via a session cookie, as shown in below figure.



Lesson 139 : Using Session – Example

Session Storage and Configuration

Each user's session information is kept in serialized files, one per session. It is possible to configure many aspects of sessions including where the session files are saved. For a complete listing refer to the session configuration options in php.ini. The decision to save sessions to files rather than in memory addresses the issue of memory usage that can occur on shared hosts as well as persistence between restarts. Many sites run in commercial hosting environments that are also hosting many other sites.



Session state is ideal for storing more complex objects or data structures that are associated with a user session. The classic example is a shopping cart. While shopping carts could be implemented via cookies or query string parameters, it would be quite complex and cumbersome to do so. In PHP, session state is available to the developer as a superglobal associative array, much like the $_GET, $_POST, and $_COOKIE arrays.3 It can be accessed via the $_SESSION variable, but unlike the other superglobals, you have to take additional steps in your own code in order to use the $_SESSION superglobal. To use sessions in a script, you must call the session_start() function at the beginning of the script.

Accessing session state

```
<?php
session_start();
if ( isset($_SESSION['user']) ) {
// User is logged in
}
else {
// No one is logged in (guest)
}
?>
```

Session state is typically used for storing information that needs to be preserved across multiple requests by the same user. Since each user session has its own session state collection, it should not be used to store large amounts of information because this will consume very large amounts of server memory as the number of active sessions increase. As well, since session information does eventually time out, one should always check if an item retrieved from session state still exists before using the retrieved object..

Checking session existence

```
<?php
include_once(“ShoppingCart.class.php”);
session_start();
// always check for existence of session object before accessing it
if ( !isset($_SESSION[“Cart”]) ) {
// session variables can be strings, arrays, or objects, but
// smaller is better
$_SESSION[“Cart”] = new ShoppingCart();
}
$cart = $_SESSION[“Cart”];
?>
```

Lesson 140 : HTML5 web storage

HTML5 web storage

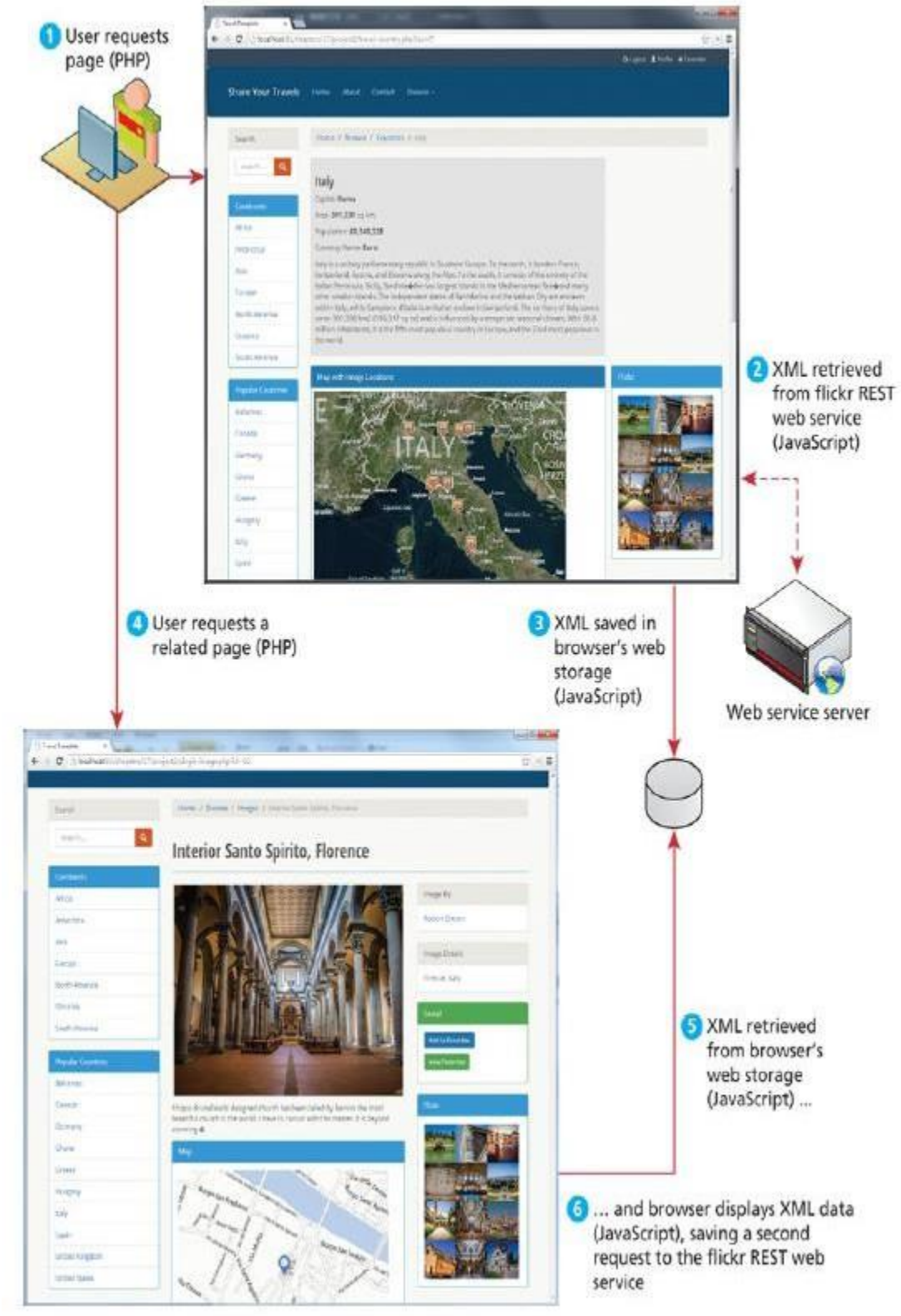
Web storage is a new JavaScript-only API introduced in HTML5.4. In addition, web storage is not limited to the 4K size barrier of cookies; the W3C recommends a limit of 5MB but browsers are allowed to store more per domain. Currently web storage is supported by current versions of the major browsers, including IE8 and above. However, since JavaScript, like cookies, can be disabled on a user's browser, web storage should not be used for mission-critical application functions.

Why Would We Use Web Storage?

Cookies have the disadvantage of being limited in size, potentially disabled by the user, vulnerable to XSS and other security attacks, and being sent in every single request and response to and from a given domain. On the other hand, the fact that cookies are sent with every request and response is also their main advantage: namely, that it is easy to implement data sharing between the client browser and the server. Unfortunately with web storage, transporting the information within web storage back to the server is a relatively complicated affair involving the construction of a web service on the server and then using asynchronous communication via JavaScript to push the information to the server.

A better way to think about web storage is not as a cookie replacement but as a local cache for relatively static items available to JavaScript. One practical use of web storage is to store static content downloaded asynchronously such as XML or JSON from a web service in web storage, thus reducing server load for subsequent requests by the session.

Below figure illustrates an example of how web storage could be used as a mechanism for reducing server data requests, thereby speeding up the display of the page on the browser, as well as reducing load on the server.



Writing web storage

<form … >

```
<h1>Web Storage Writer</h1>
<script language="javascript" type="text/javascript">
if (typeof (localStorage) === "undefined" ||
typeof (sessionStorage) === "undefined") {
alert("Web Storage is not supported on this browser…");
}
else {
sessionStorage.setItem("TodaysDate", new Date());
sessionStorage.FavoriteArtist = "Matisse";
localStorage.UserName = "Ricardo";
document.write("web storage modified");
}
</script>
<p><a href="WebStorageReader.php">Go to web storage reader</a></p>
</form>
```

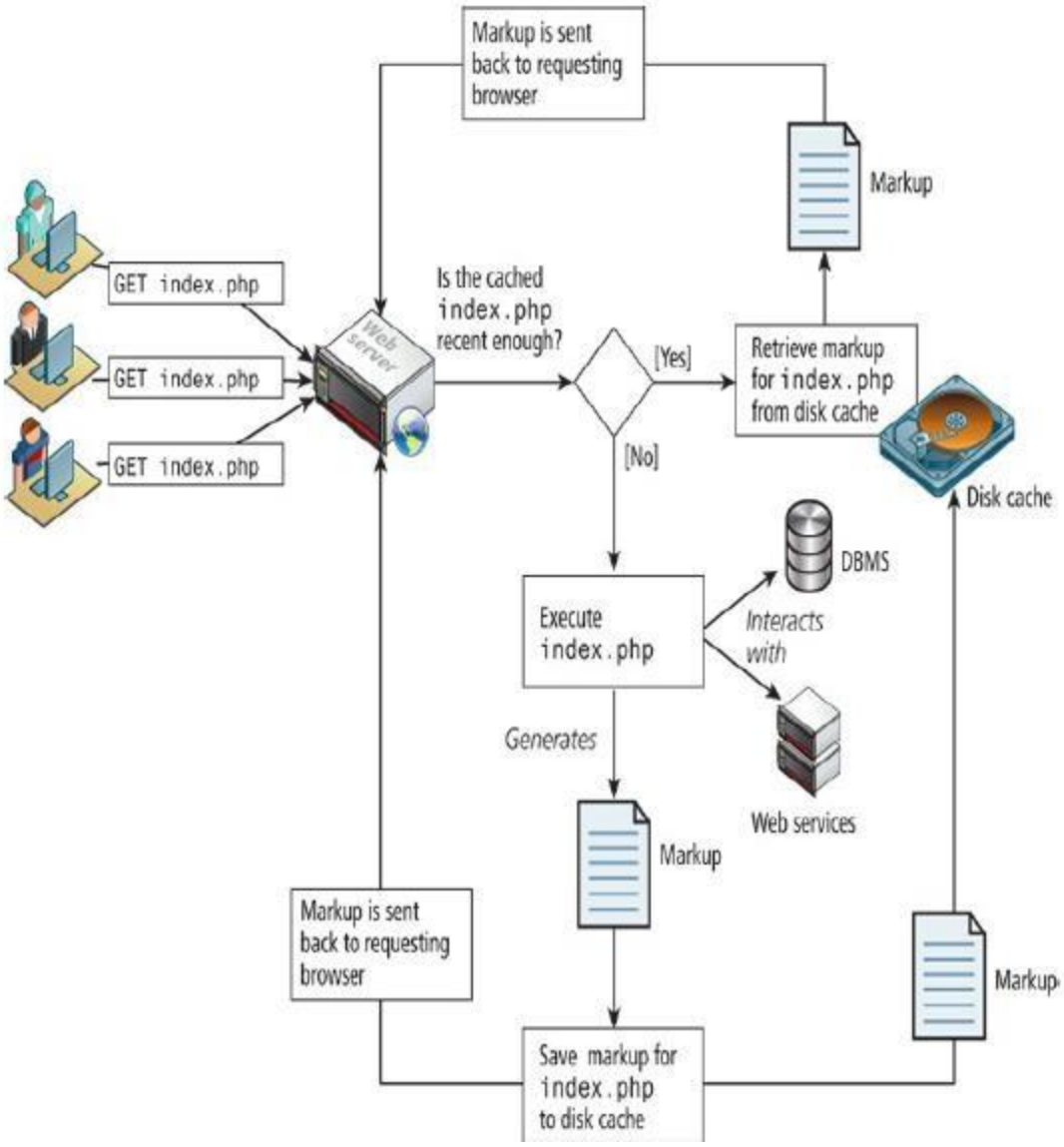
Reading web storage

```
<form id="form1" runat="server">
<h1>Web Storage Reader</h1>
<script language="javascript" type="text/javascript">
if (typeof (localStorage) === "undefined" ||
typeof (sessionStorage) === "undefined") {
alert("Web Storage is not supported on this browser…");
}
else {
var today = sessionStorage.getItem("TodaysDate");
var artist = sessionStorage.FavoriteArtist;
var user = localStorage.UserName;
document.write("date saved=" + today);
document.write("<br/>favorite artist=" + artist);
document.write("<br/>user name = " + user);
}
</script>
</form>
```

Lesson 141 : Caching

Caching

Caching is a vital way to improve the performance of web applications. Your browser uses caching to speed up the user experience by using locally stored versions of images and other files rather than re-requesting the files from the server. While important, from a serverside perspective, a server-side developer only has limited control over browser caching. In this type of caching, the contents of the rendered PHP are written to disk for fast retrieval. This can be particularly helpful because it allows PHP to send a page response to a client without going through the entire page processing life cycle again. Page output caching is especially useful for pages whose content does not change frequently but which require significant processing to create.



Application Data Caching

One of the biggest drawbacks with page output caching is that performance gains will only be had if the entire cached page is the same for numerous requests. However, many sites customize the content on each page for each user, so full or partial page caching may not always be possible. An alternate strategy is to use application data caching in which a page will programmatically place commonly used collections of data that require time intensive queries from the database or web server into cache memory, and then other pages that also need that same data can use the cache version rather than re-retrieve it from its original location.

Lesson 142 : PHP with AJAX

PHP with Ajex

AJAX requests are initiated on client side using JavaScript. The request is received by the server-side script (PHP), just like form data is received. The result is sent to the client side. The client receives the

input and update the page.
JavaScript Side
An asynchronous (non-blocking) XML Http Request (XHR) is sent
•States of XHR object
0 UNSET
1 OPENED
2 HEADERS RECEIVED
3 LOADING
4 DONE

Example - JavaScript Request

```
var xhr = new XMLHttpRequest();
xhr.onreadystatechange = function() {
if (this.readyState == XMLHttpRequest.DONE
&&(this.status>=200&&this.status < 400)) {
document.getElementById("updates").innerHTML = this.responseText;
}
xhr.open("GET", "http://localhost/getupdates.php?q=" + str, true);
xhr.send();
```

Example - jQuery Request

```
var xhr = new XMLHttpRequest();
$.get("http://localhost/getupdates.php?q=" + str,
function(data, status){
$("#updates").html(data);
}
)
.done(function() {...} )
.fail(function() {...} )
.always(function() {...} )
;
```

Lesson 143 : PHP with AJAX case study (e.g., Live search)

PHP with Ajex-Example

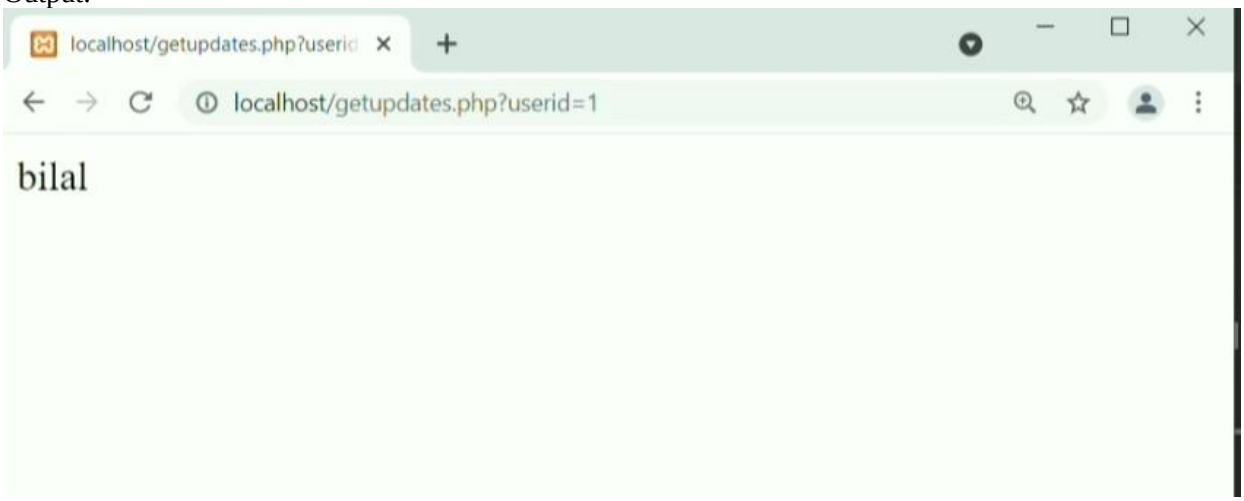
```php
<?php
if(isset($_GET['userid'])) {
    try {
        require_once('database.inc.php');

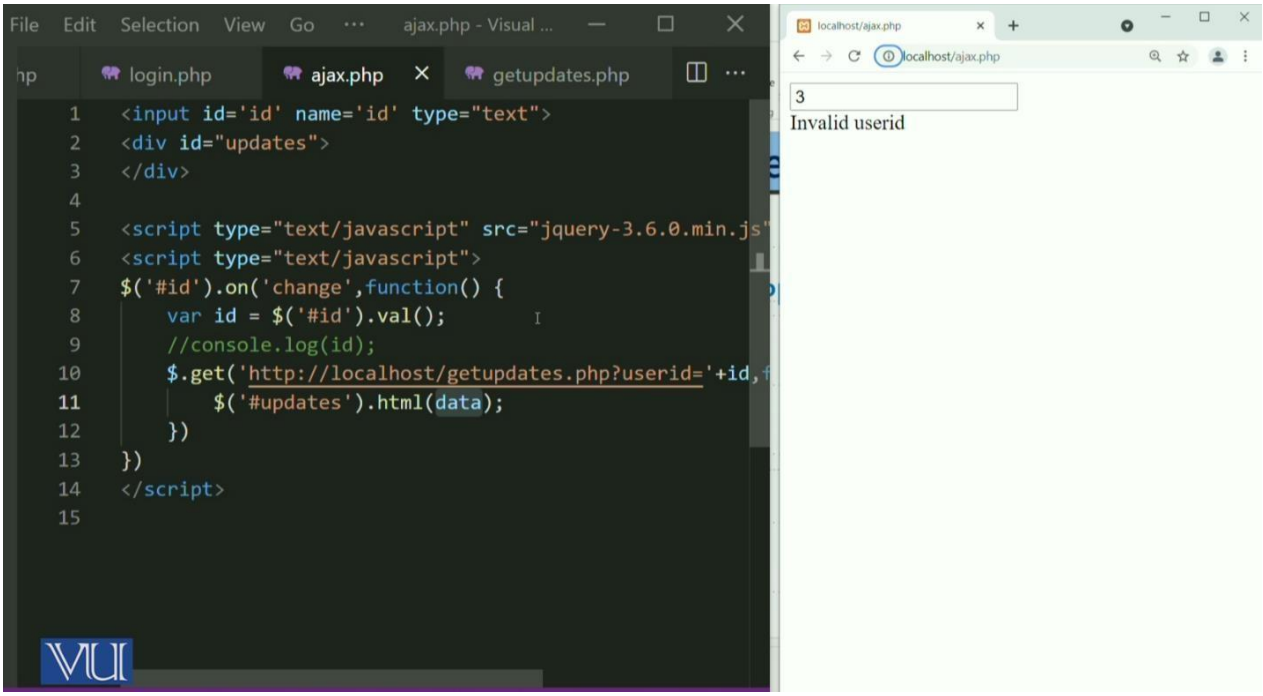
        $id = $_GET['userid'];

        $sql = 'SELECT * FROM user WHERE id = :id LIMIT 1';
        $stmt = $pdo->prepare($sql);
        $stmt->execute(array(':id' => $id));
        $row = $stmt->fetch();
        if ($row == null) {
            echo "Invalid userid";
        } else {
            echo $row['username'];
        }


    } catch (PDOException $e) {
        die($e->getMessage());
    }
```

Output:





Lesson 144 : Web Application Design – Layering

Web Application Design

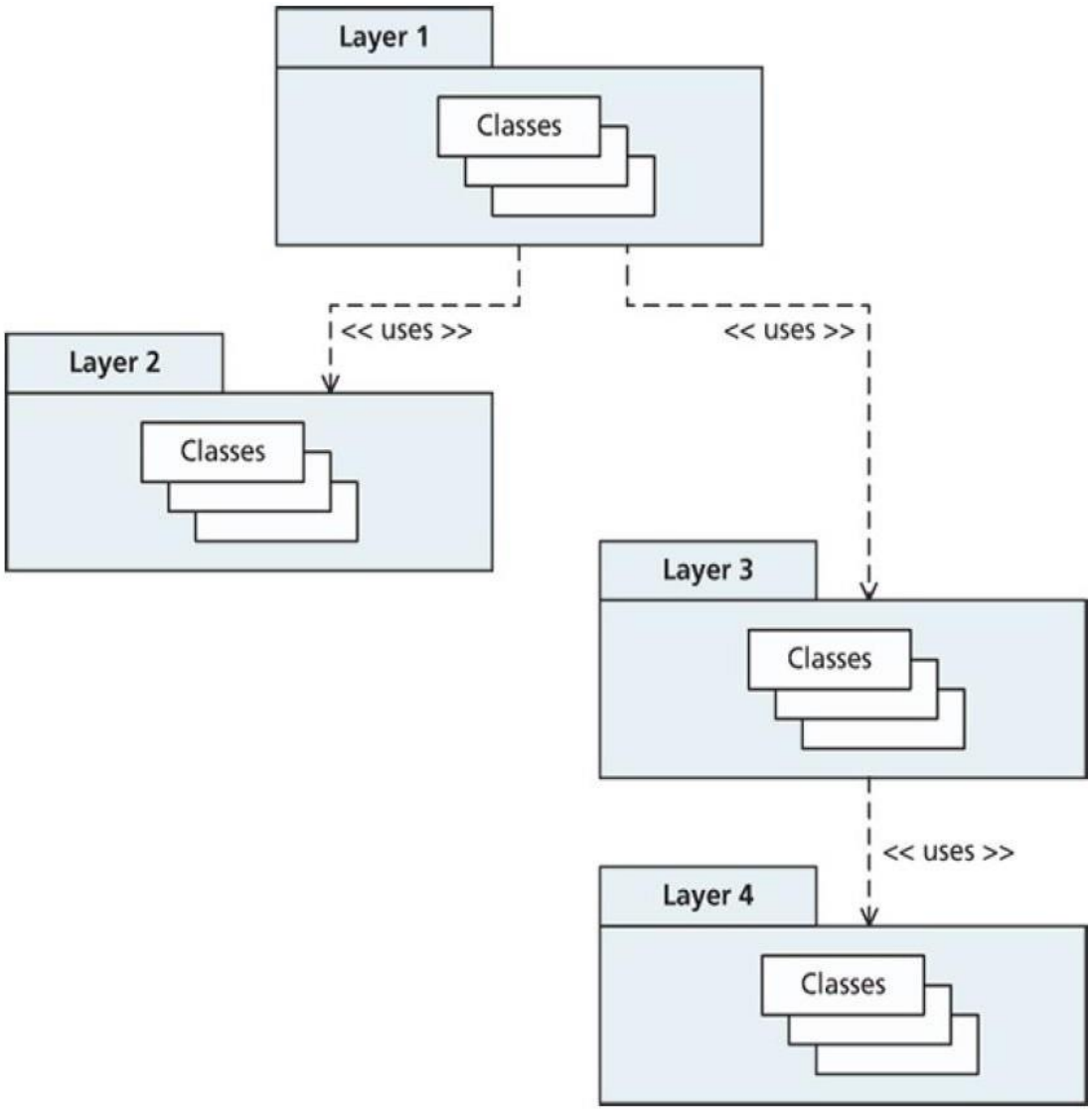
Challenges in Designing Web Applications

It is quite possible to create complex web applications with little to no class design. The page-oriented development approach is such that each page contains most of the programming code it needs to perform its operations. Rapidly thought-out systems are rarely able to handle unforeseen changes in an elegant way. A well-designed application infrastructure up front can make your web application easier to modify and maintain, easier to grow and expand in functionality, less prone to bugs, and thus, ultimately, in the long run easier to create.

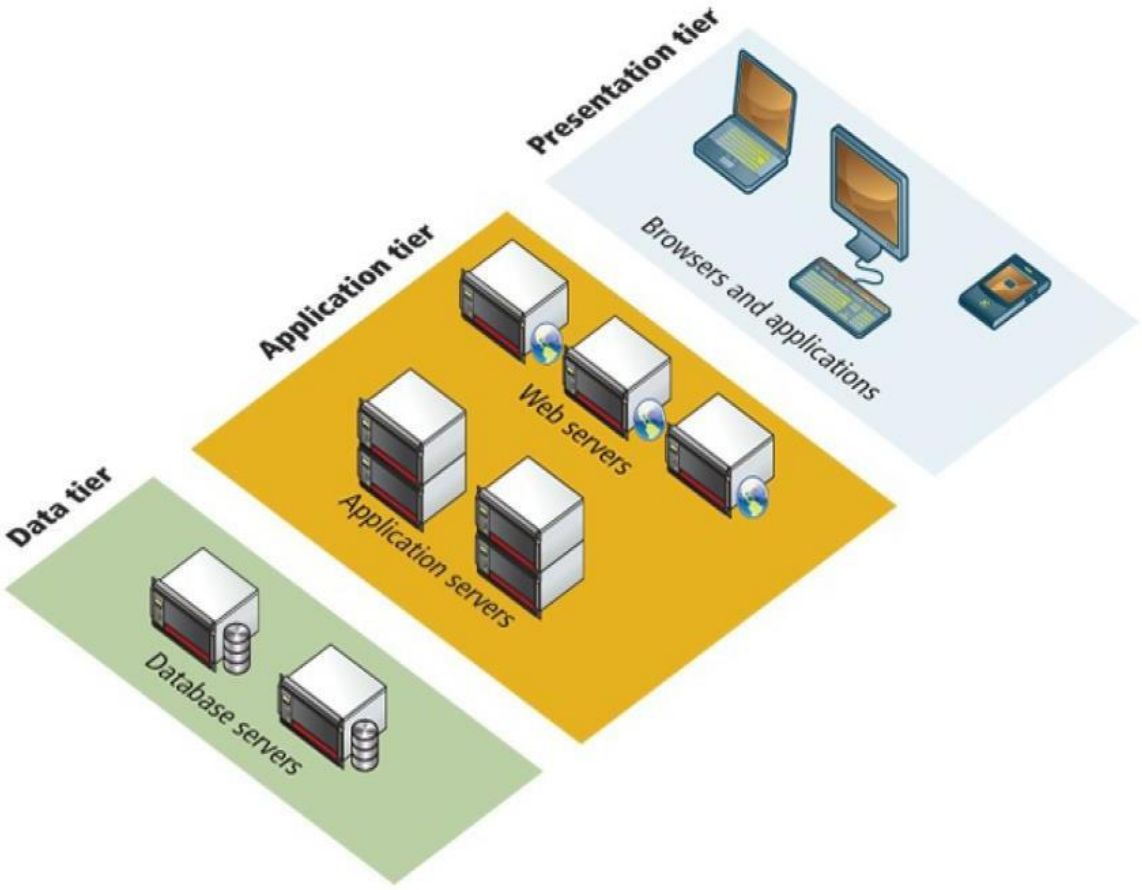
What Is a Layer?

A layer 7 in the context of application development, is simply a group of classes that are functionally or logically related. Each layer in an application should demonstrate cohesion. Distribute the functionality of your software among classes so that coupling is minimized. A dependency is a relationship between two elements where a change in one affects the other.

In the layered design approach, each class within the layer has a limited number of dependencies. A dependency (also referred to in UML as the uses relationship) is a relationship between two elements where a change in one affects the other. In the illustration given in below figure , the various layers have dependencies with classes only in layers“below”them, that is, with layers whose abstractions are more “lower level” or perhaps more dependent upon externalities such as databases or web services.



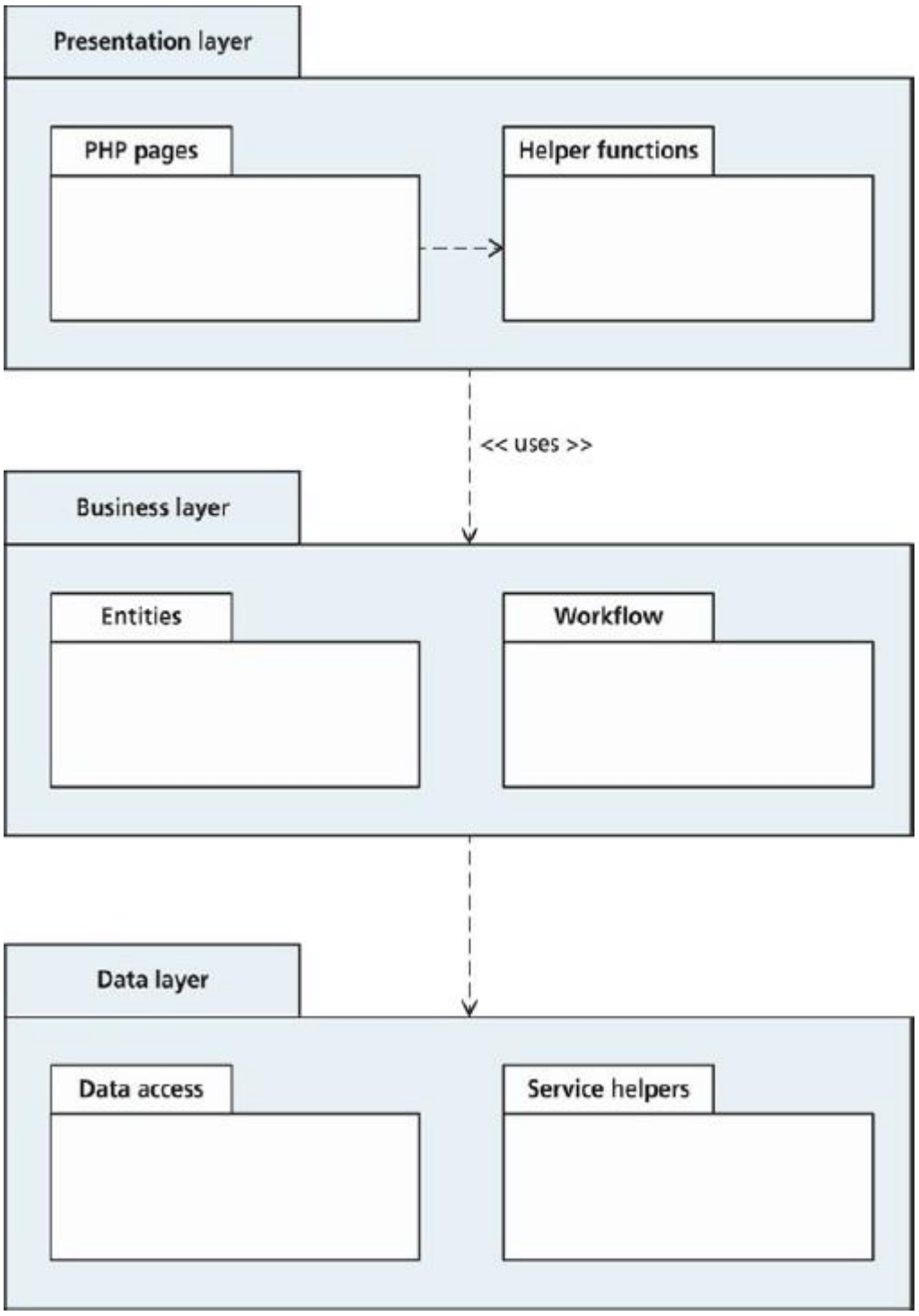
These different tiers most often refer to different places in a network. For example, a typical web application can be considered a three-tier architecture: the user's workstation is the presentation tier, the web server is the application tier, and the DBMS running on a separate data server is the database tier, as shown in below Figure. The rest of the chapter will use the word tier in this latter sense, and use the word layer when referring to the conceptual grouping of classes within an application.



Common Layering Schemes
Presentation: Principally concerned with the display of information to the user, as well as interacting with the user.
Domain/Business: The main logic of the application. Some developers call this the business layer since it is modeling the rules and processes of the business for which the application is being written.
Data Access: Communicates with the data sources used by the application. Often a database, but could be web services, text files, or email systems.

Lesson 145 : Web Application Design - 3-Layer Model

The two-layer paradigm has various flaws, thus many developers prefer a three-layer model in which a business layer (also known as a domain layer) is in charge of executing business rules and processes. The high-level design of a three-layer model is shown in the diagram below.

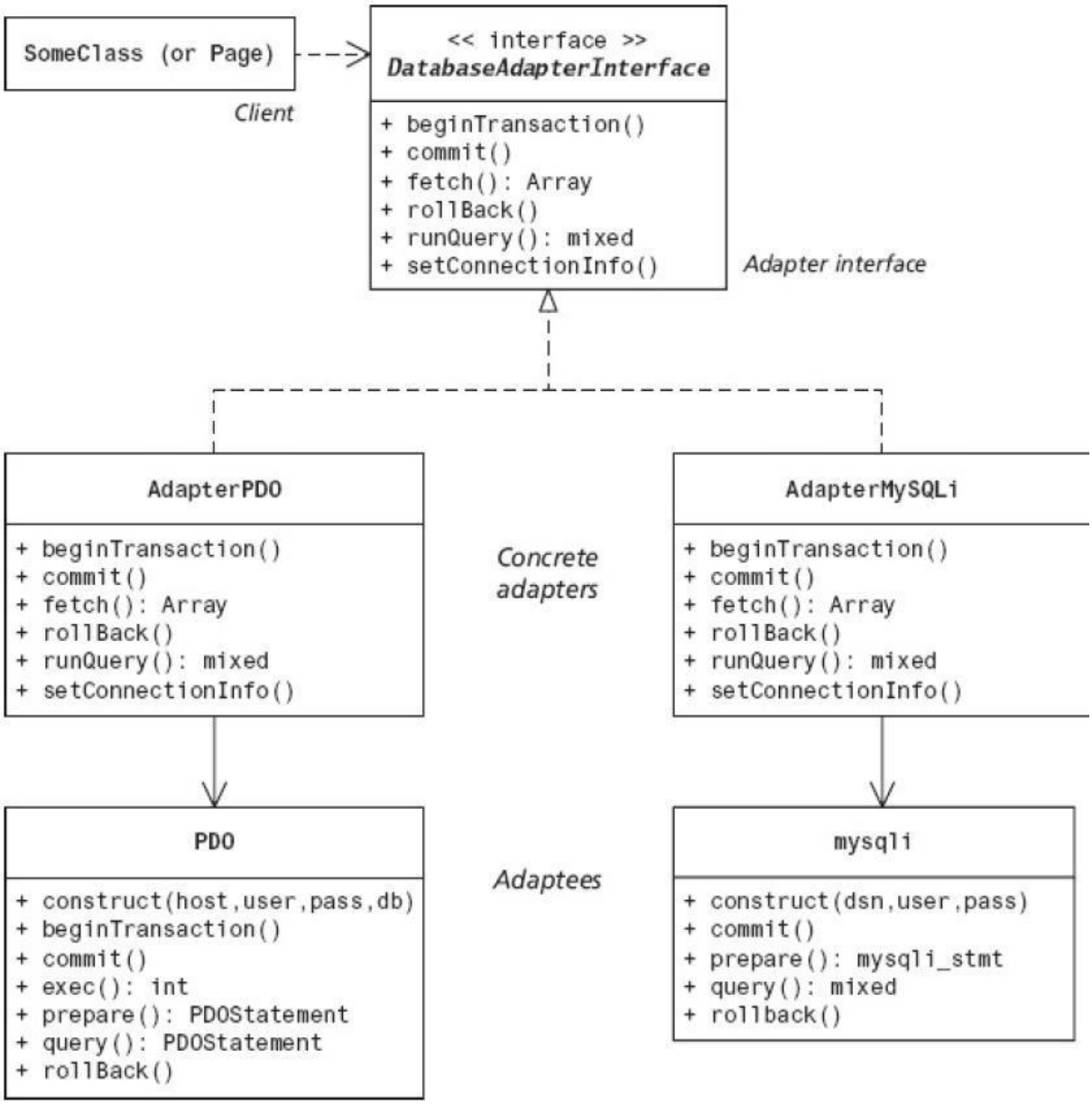


Some authors refer to the classes in a three-layer model's "middle" layer as business objects, while others refer to them as entities or domain objects. Business objects, whatever they're called, represent the data and behaviour of things that correspond to the business's conceptual domain. A simple domain layer would consist of domain/business objects that are very similar to the database table.

Lesson 146 : Software Design Patterns - Adapter pattern; Simple Factory Pattern

Adapter Pattern

The Adapter pattern is used to change a set of classes' interfaces to a different but preferable interface. The key advantage of this pattern is that it separates the client (in the context of discussing patterns, the term client refers to the classes that use the pattern classes) from the consuming class's interface. The Adapter pattern is commonly used in web applications to leverage a database API (such as PDO or mysqli) without having to keep connecting the pages to that database API. Real-world websites, as discussed previously in the chapter, update the database or the API used to access it as the site increases in complexity or the size of its data or requests. The majority of the programme is protected against future change by using an Adapter. Indeed, writing (or reusing) a database API adaptor is one of the first things several designers do when starting a new web application project. The design of a sample database adapter is shown in the diagram below.



So, how would this adapter's code look like? The Adapter pattern, as shown in the diagram above, must first create an interface. We want the adapter to represent the capabilities that any database adapter will

require in this case. Not only can you open and delete connections, but you can also conduct SELECT, UPDATE, INSERT, and DELETE queries and manage transactions.

Simple Factory Pattern

The Adapter pattern was used in the previous section to eliminate a dependent on a changing interface. Unfortunately, during the instantiation of the concrete adapter, a sort of dependency was introduced into the client code. The Simple Factory pattern is used as a solution to this problem. A factory is a custom class that creates subclasses (or concrete implementations of an interface) so that clients aren't tied to specific subclasses or implementations. Factory patterns come in a variety of shapes and sizes. The Factory Method and the Abstract Factory are two patterns identified in the Design Patterns book. The Simple Factory pattern is a simplified version of the other two factories, as its name implies. A Factory Method with early binding can be developed in programming languages like C# or Java using conditional logic similar to the following pseudo-code:

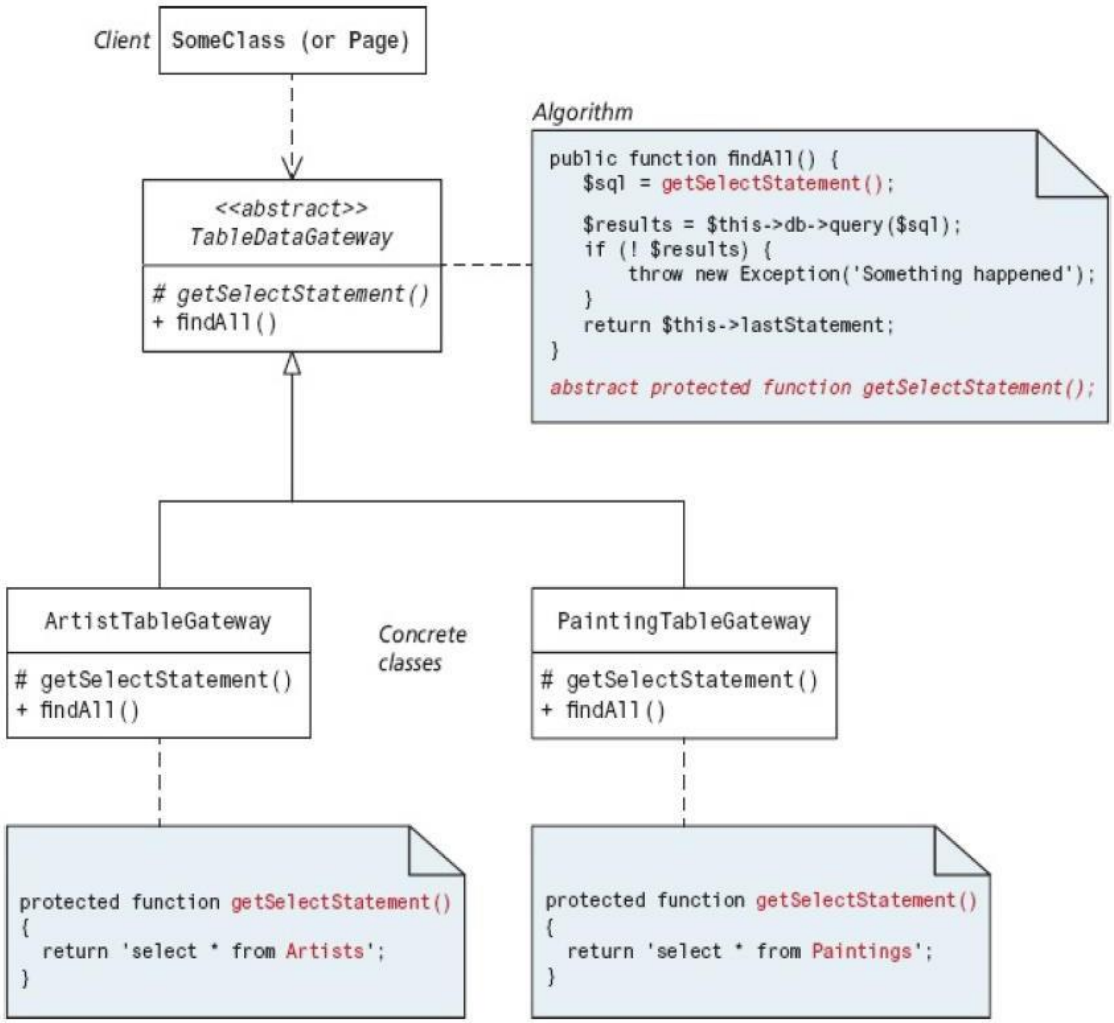
```
If requested == 'PDO' Then
Return new PDOAdapter()
Else If requested == 'oracle' Then
Return new OracleAdapter()
Else If requested 'odbc' then
Return new OdbcAdapter()
etc.
```

Because PHP is a late-binding language, you can design a factory class that avoids conditional logic by supplying the specific class name to instantiate dynamically at run time.

Lesson 147 : Software Design Patterns - Template method and dependency injection ...

Template Method Pattern

One of the most important of the 23 traditional design patterns is the Template Method. Many object-oriented developers, in fact, utilise this technique without even realising it. This technique involves defining an algorithm in an abstract superclass and deferring to the subclasses the algorithm stages that can vary. The following Figure, for example, depicts the design of a sample data access layer that employs the Template Method pattern.



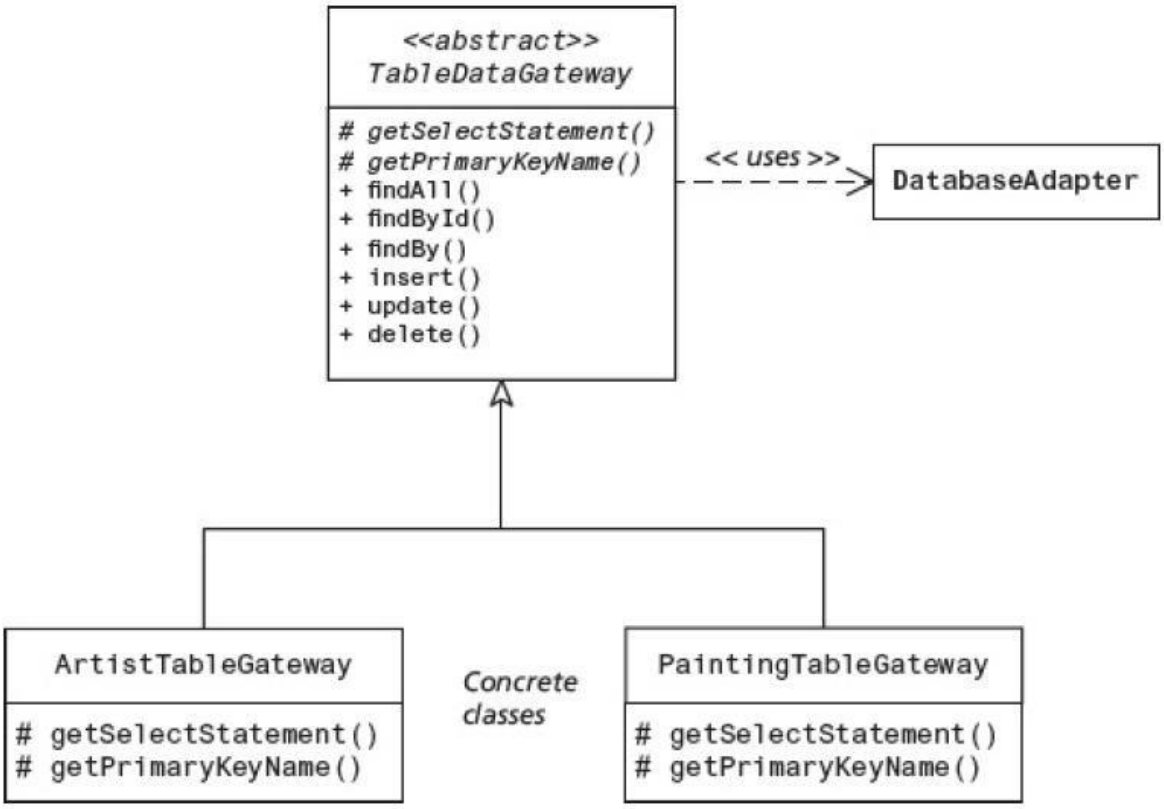
Dependency Injection

Although Dependency Injection is not one of the original 23 design patterns identified in the Design Patterns book, it has become one of the most essential software design patterns (and thankfully one of the simplest). It was first identified and named by Martin Fowler5; its purpose is to reduce the number of dependencies within a class, by passing (injecting) potential dependencies into a class rather than hard-coding them into the class.

Lesson 148 : Data and Domain Patterns - Table Data Gateway, domain model and active ...

Table Data Gateway Pattern

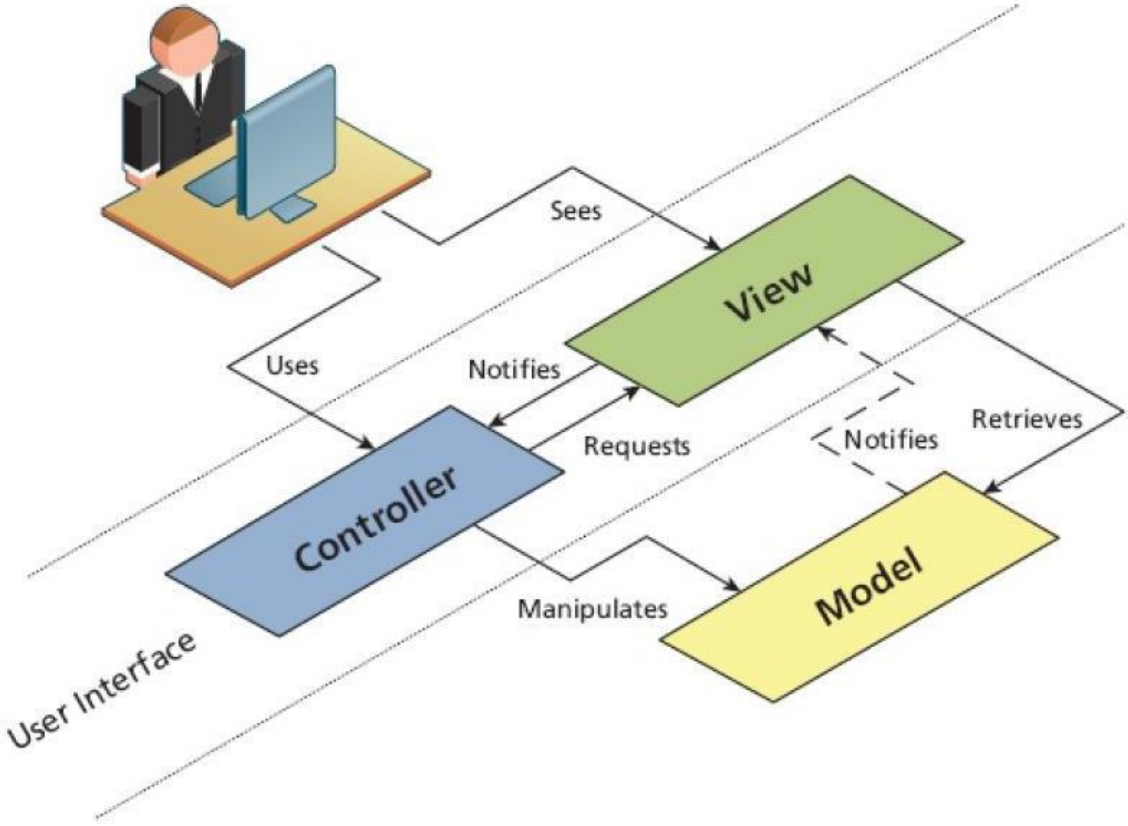
The Table Data Gateway pattern by Fowler is similar to a data access object. A gateway is essentially an object that encapsulates external resource access. As a result, a table data gateway allows CRUD access to a database table (or perhaps joined tables). The diagram below shows how this approach can be used to build the foundations of a data access layer. Notice how the superclass contains the majority of the common code (and uses the Template Method pattern), while each subclass defines the code specific to that table.



Domain Model Pattern
The Domain Model pattern is a natural fit for programmers who are experienced with object-oriented design. The developer creates an object model, which is a collection of connected classes that represent items in the application's issue domain. A domain model's classes will contain both data and behaviour, making them the ideal place to apply business rules.

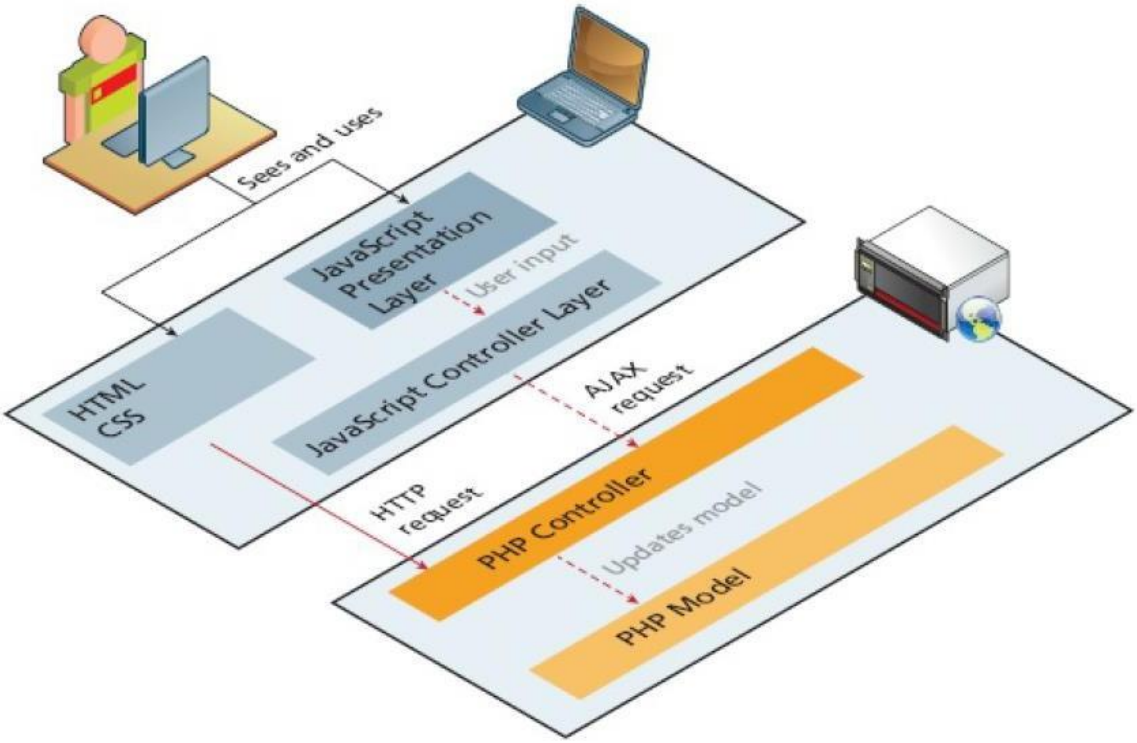
Lesson 149 : Presentation Patterns - MVC and Front Controller patterns

Model-View-Controller (MVC) Pattern
The Model-View-Controller (MVC) pattern predates the entire pattern movement, as it was first developed as a user-interface framework for the SmallTalk (early object-oriented language) platform in the 1970s. It influenced the concept and design of many future user interface frameworks significantly. The MVC design comes in a variety of subtle (and not so subtle) versions, including several for PHP and JavaScript. The MVC design separates an application into three classes that serve three functions: model, view, and controller. The Model-View-Controller (MVC) pattern was created in the 1970s as a user-interface framework for the SmallTalk (early object-oriented language) platform. It has a considerable impact on the concept and design of many future user interface frameworks. The MVC design is available in a variety of subtle (and not so subtle) variations, including PHP and JavaScript implementations. The model, view, and controller (MVC) design divides an application into three classes that serve three functions.

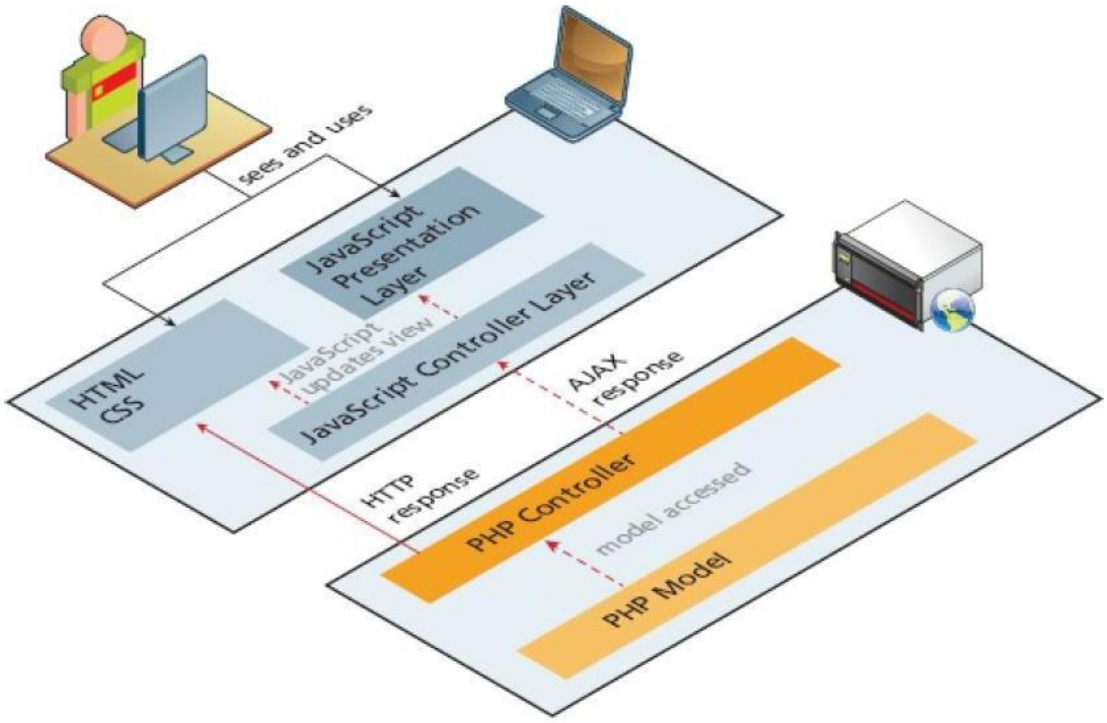


Lesson 150 : Designing and Implementing an MVC based Website

It should be remembered that the MVC design was created for desktop applications in which the views could utilise the Observer pattern (or something similar like event listeners) to update themselves anytime the model changed. When the MVC pattern is used in a web environment, things become more challenging. The model in MVC is relatively straightforward: it's very similar to the domain model covered in the previous section (though it could also be just the gateway classes). However, some features of the paradigm may also be implemented in JavaScript on modern AJAX-based websites. The more difficult question is: what is the relationship between the View and the Controller? The View includes not only HTML and CSS, but also the PHP that creates it and presentation-oriented JavaScript. As seen in Figure , the Controller is most likely partially developed in JavaScript and partially in PHP.



Other distinctions exist between a web MVC and a desktop MVC. Because the model is primarily (or totally) on a different computer than the views, there is no mechanism for the views to listen for changes in the model as in the standard MVC approach. Another distinction is that with desktop applications, the model is a collection of objects that remain populated during the application's lifespan. These objects appear in a PHP application just when the script is running and disappear once the request is handled. When developing a web MVC application, one of the most important design decisions is whether the controller will be a server-side PHP controller or a client-side JavaScript controller. It is possible for the controller to be both as illustrated in following Figure.


sees and uses
JavaScript Presentation Layer
HTML CSS
JavaScript updates view
JavaScript Controller Layer
AJAX response
HTTP response
PHP Controller
model accessed
PHP Model

Week – 12

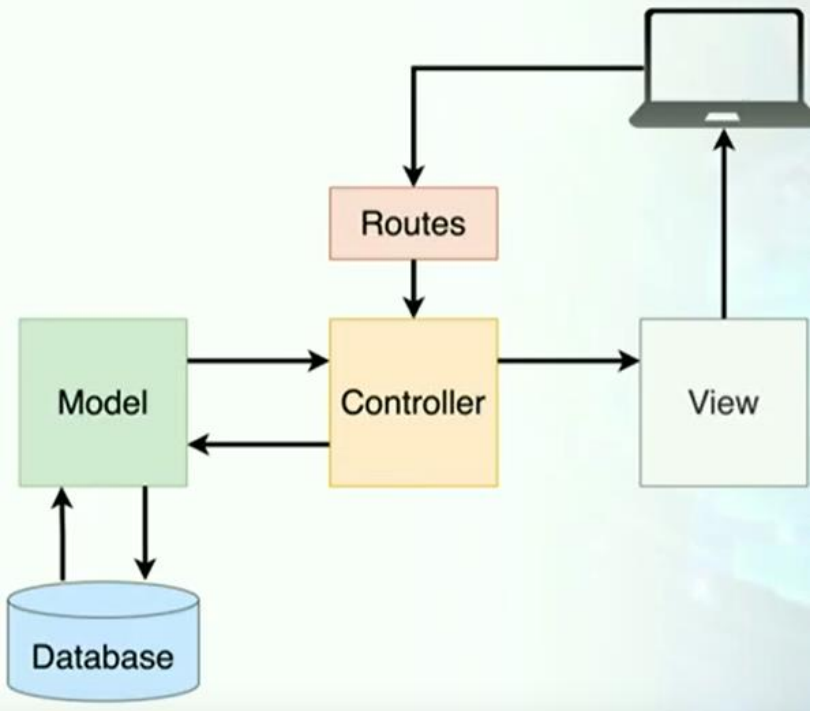
Lesson 151: Laravel Introduction

Laravel Introduction:

Laravel is a php based framework. It is quite a complex framework that provides a lot of features and tools. You can say that it has its own ecosystem. Here we will cover some basics of Laravel. Here are some features of Laravel:

- It is a Php based framework.
- It is Open source
- It follows MVC architecture
- It uses template engine for layouts/views
- It also provides command line utility (artisan) to do plenty of tasks.
- You can easily integrate it with database
- It uses composer as dependency and package manager

Figure below shows the Laravel architecture



User will send a URL request e.g. user can ask about "a list of products". We have a "Route" class in Laravel framework that will manage this request and will decide the controller that will be invoked on the basis of the user request. For example, if user asked for the list of the products, the "Product Controller" will be activated which will send a request to "Product model" to have a list of all the products. Model will then fetch the products from the database, make their list and will send back to controller. Controller will then pass the list to the view. The view is a template that defines how the data will be displayed to the user. The data will then be sent to the user in HTML format which will be rendered in the browser.

Lesson 152: Laravel Installation

Laravel Installation:

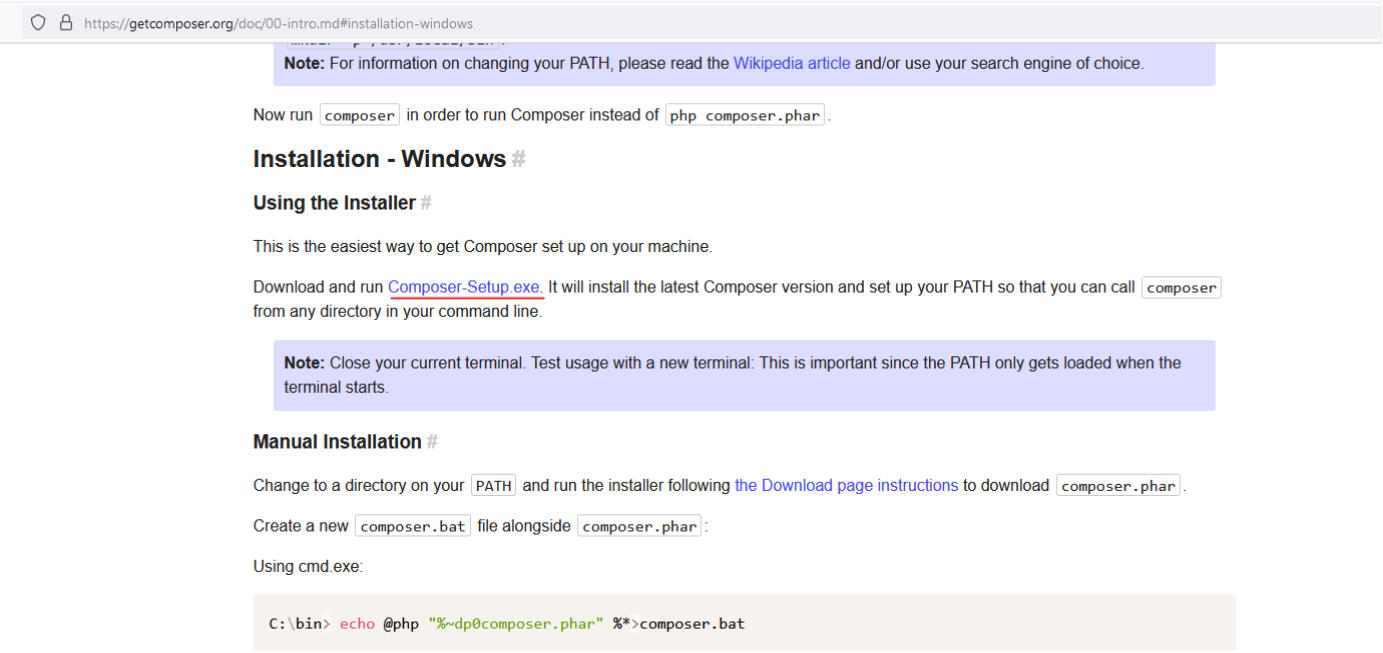
Normally Laravel can be downloaded in zip format. However, Laravel being a PHP framework, depends on some PHP scripts that may not be present in the zip file. For this purpose you will use a dependency manager called the "Composer". When you will install Laravel through composer, then all the dependencies will also be installed on your machine.

For installation of composer do the following:

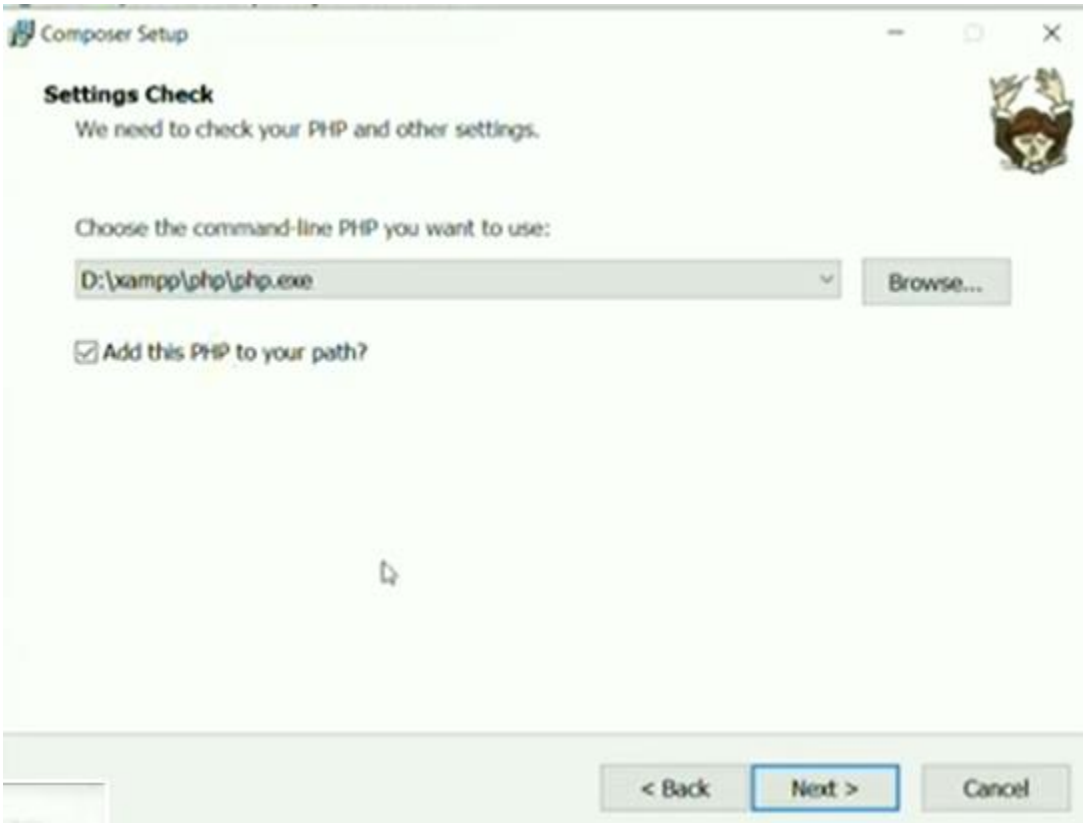
Open the link:

https://getcomposer.org/doc/00-intro.md#installation-windows

From the page (that is opened), click "Composer-Setup.exe" as shown in the figure below:



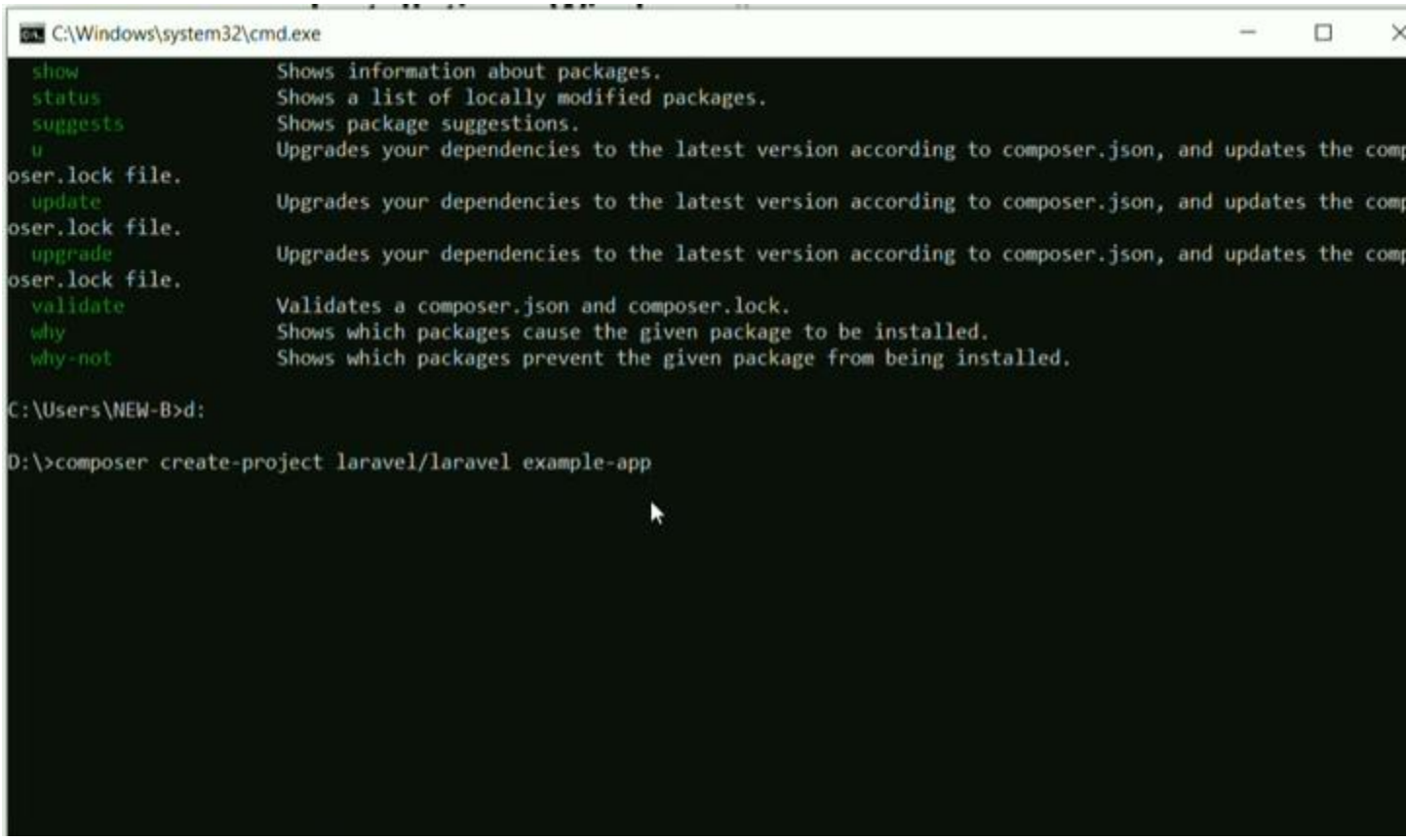
Composer-Setup.exe will be downloaded. Execute this file by accepting the default options. Note that, at an intermediate stage, you will have to enter the path of the PHP file as shown below:



Note: your path may be different.

Once the composer is installed, you can verify the installation by typing “composer” at the command prompt. If the installation was successful, you will see the composer related information on command prompt else you will get “Command not found” message.

Once the composer is installed, you will use the composer to install Laravel. For this purpose type the following command, that will install the Laravel and create a Laravel project named “example-app”.



Once the project is created, you can navigate the installation folder to check different of the directories and files in the example-app project.

Now to run the Laravel, go to the command prompt, go to the project folder (in our case it was D:\example-app) and type the following command:

D:\example-app>php artisan serve

You will get URL of the website (your project). You can open the URL in the browser and see the default webpage.

Lesson 153: Laravel: Introduction to Routes

Whenever the user sends a request, it is received by the Route class. However, before discussing the Routs, lets discuss the Representational State Transfer (REST) architecture. REST is a way of implementing web services. REST is comprised of the objects which we call resources. We can perform different operations on these objects using the HTTP methods. Mainly the following four operations are performed:

POST (Create)

GET (Retrieve)

PUT (Update)

DELETE (Delete)

For example, the following URL will return the product having product id=1

http://www.myshop.com?pid=1

As you can see that we have sent the request using the get method (you can see the string query above), so the retrieve method will be called.

In Laravel, the Routes are handled through the Route class. Routes are defined in the folder named "routes/" in Laravel project.

You can check the "routes" folder in the previously installed "example-app" project. Open the main project folder and you will see the "routes" folder. The folder contains many files. The main file that we will be using is "web.php" file. The file contains the routes that are used when the project is accessed through the web.

Now open the "web.php" file and you will see the following code:

```
Route::get('/',function() {

return view('welcome');

});
```

Enter the following URI in browser:

227.0.0.1:8000

(Note: we assume that this is the URI of the project)

You will see the welcome view in the browser. You can verify the view in the "views" folder.

'/' in the above code refers to the main (root) folder of the project.

Now let's define a new Route named "about". You will define it as follows:

```
Route::get('/about',function() {

return 'About Us!';

});
```

Now enter the URI:

227.0.0.1:8000/about

You will see the "About Us!" message in the browser.

You can create the navigation between "home" and the "about" pages as follows:

Route::get(‘/’,function() {

return ‘Home
 About Us’;

});

Route::get(‘/about’,function() {

return ‘About Us
 Home’;

});

Lesson 154 : Laravel – Routes with Variables
Laravel – Routes with Variables

We create routes using URLs. If we want to handle to handle variable using routes.

Let’s define a route for Users:

Creating a simple Route “/users”

```
Route::get('/users', function () {
    return 'List of users<br><a href="'.url('/').'">Home</a>';
});
```

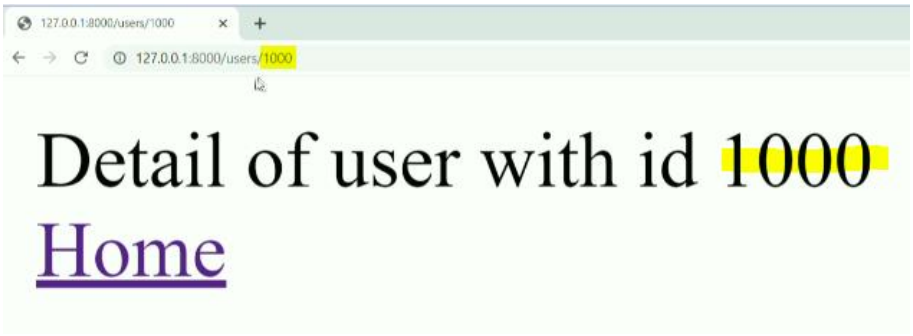
Output:



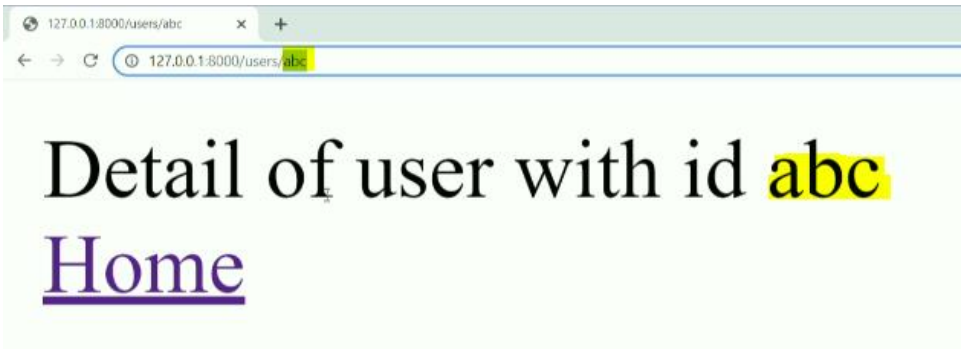
Routes with Variable:

```
Route::get('/users/{id}', function ($id) {
    return 'Detail of user with id '.$id.'<br><a href="'.url('/').'">Home</a>';
});
```

Output:



This variable will be passed, it doesn't matter it's a number or a string. But we can also define a pattern for this variable.



Define different patterns for the variable:

```
Route::get('/users/{id}', function ($id) {
    return 'Details of user with id '.$id.'<br><a href="'.url('/').'">Home</a>';
})->where('id','[0-9]+');

Route::get('/users/{name}', function ($name) {
    return 'Welcome '.$name.'<br><a href="'.url('/').'">Home</a>';
})->where('id','[a-zA-Z ]+');
```

If we pass a string in the URL:



If we write a number in the URL:

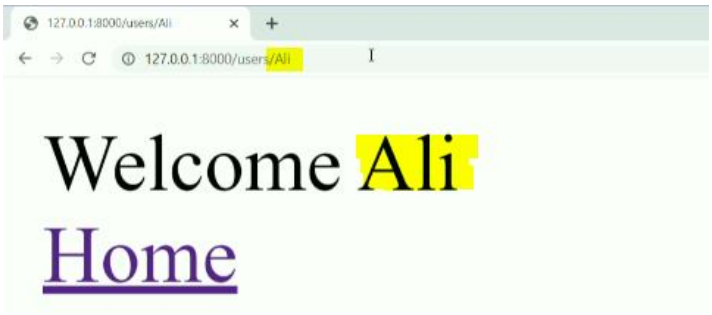


Optional Variable:

```
Route::get('/users/{name?}', function ($name=null) {
    if ($name == null)
        $name = 'Guest';
    return 'Welcome '.$name.'<br><a href="'.url('/').'">Home</a>';
})->where('id','[a-zA-Z ]+');
```

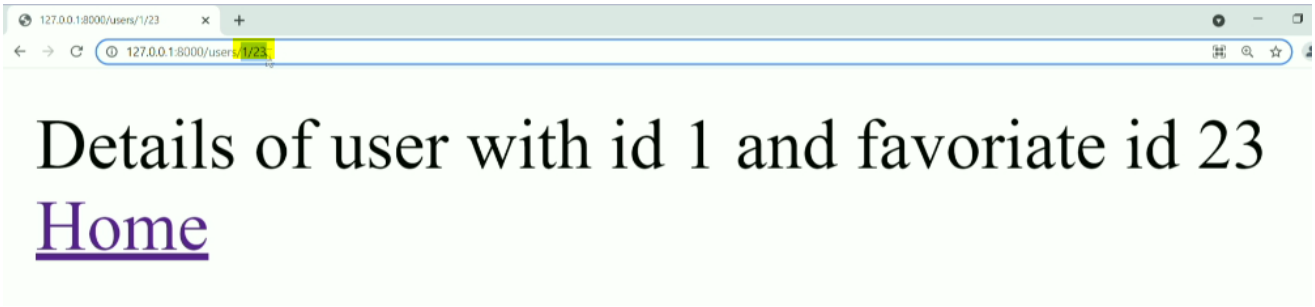
If we will no pass any string so, “Guest” will be passed:





Receive Multiple Variables:

```
Route::get('/users/{id}/{favid}', function ($id,$favid) {
    return 'Details of user with id '.$id.' and favoriate id <br><a href="'.url('/').'">Home</a>';
})->where('id','[0-9]+');
```



Lesson 155: Laravel – Views:

Laravel – Views:

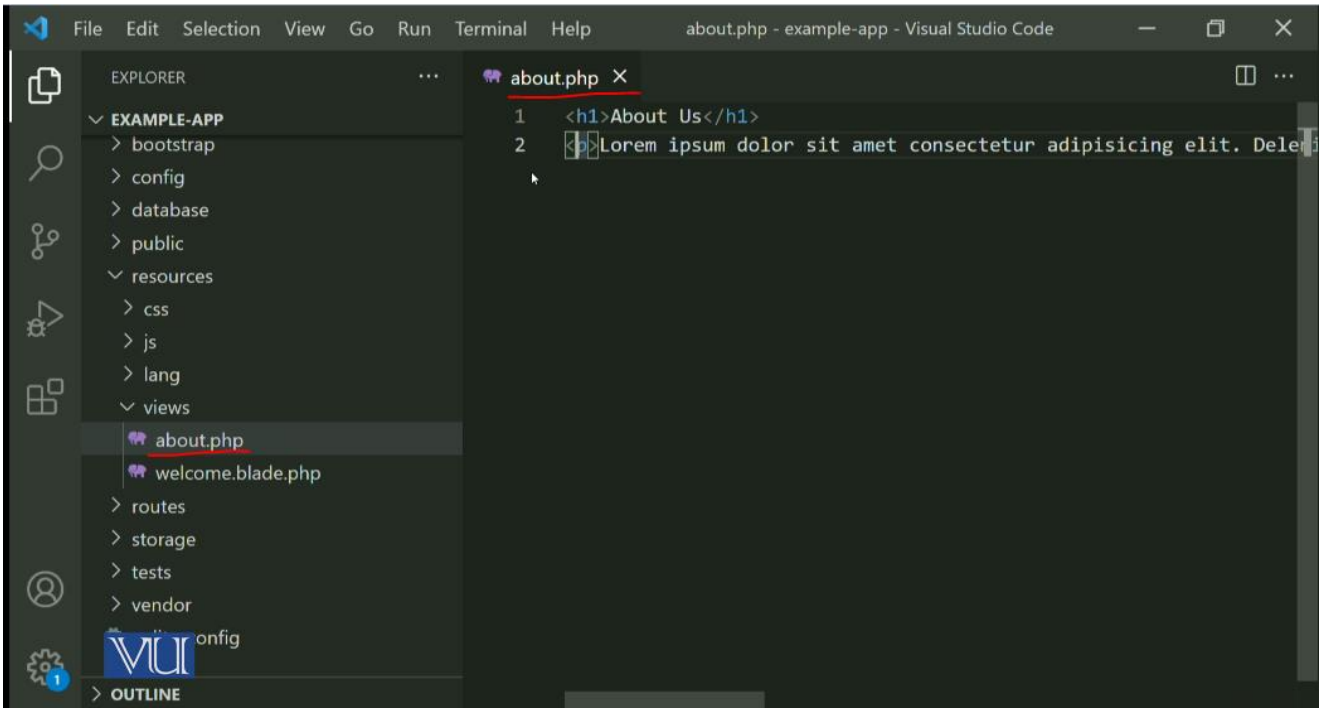
Views are located in resources/view

Types:

- Plain PHP
- Blade Template

To directly access

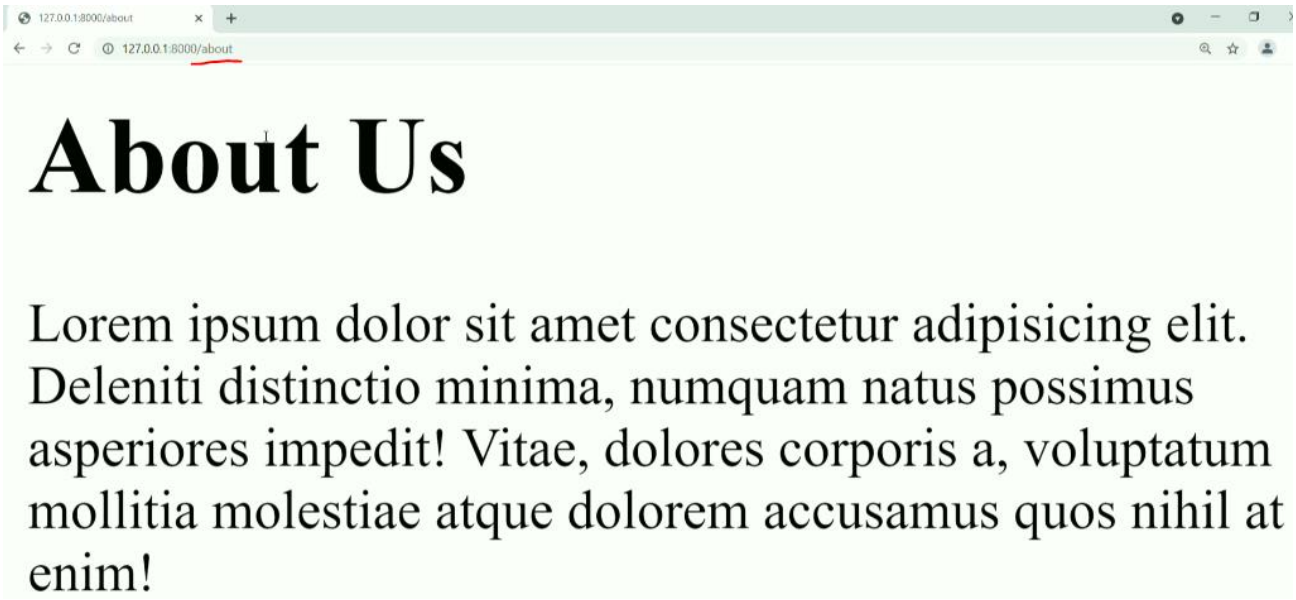
Create a file “about.php” under Views folder.



Open Web.php and write the following code:

```
Route::get('/about', function () {
    return view('about');
});
```

Output:

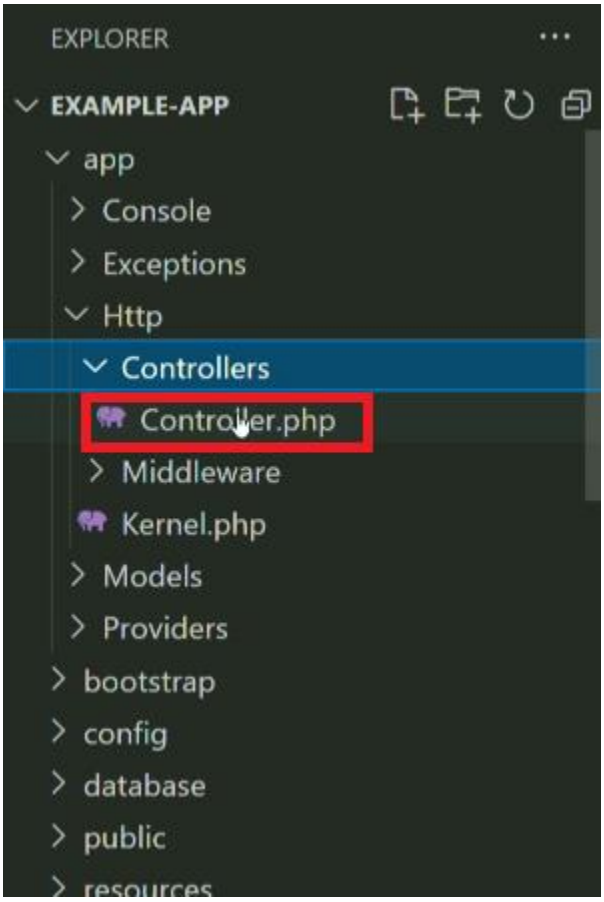


You can also use View Function:

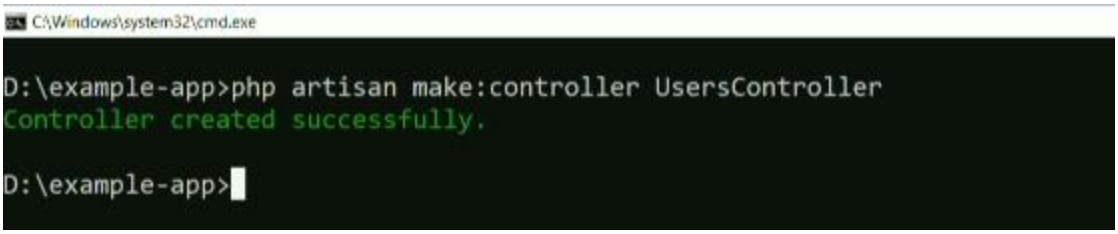
```
Route::view('/about', 'about');
```

Lesson 156: **Laravel – Controllers**

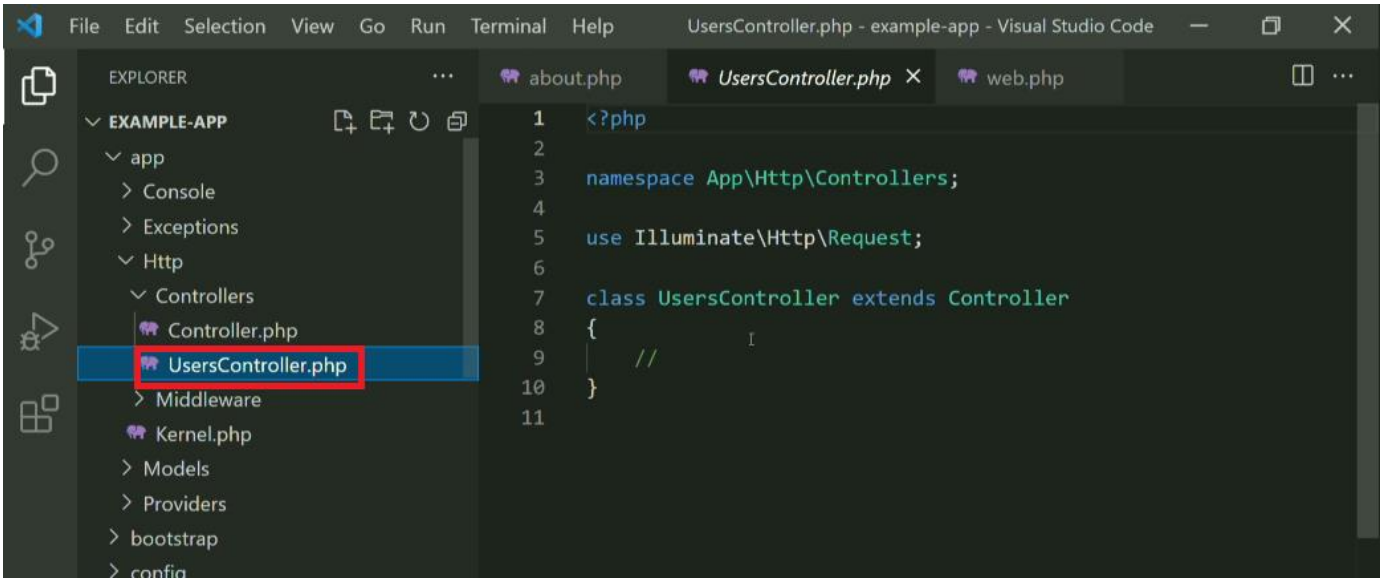
Laravel – Controller



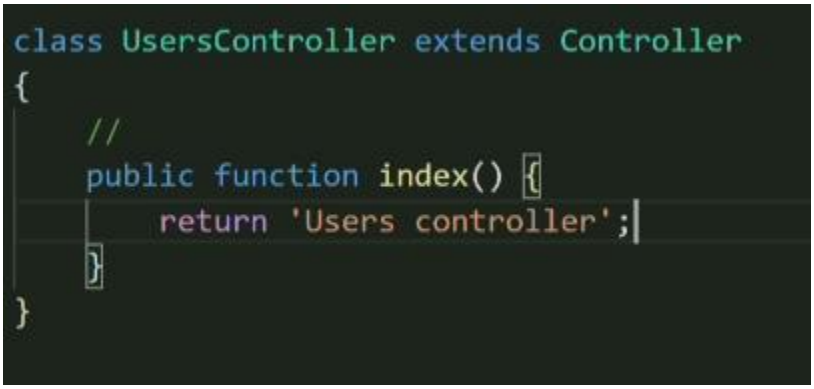
Open cmd.exe. The following command will create a controller named “UserController”.



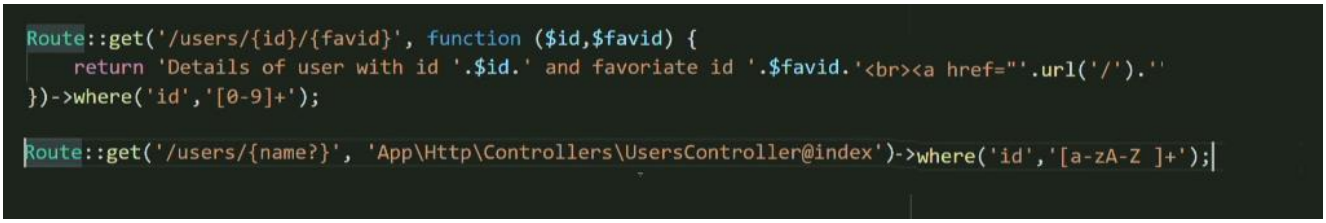
A new controller file is created.



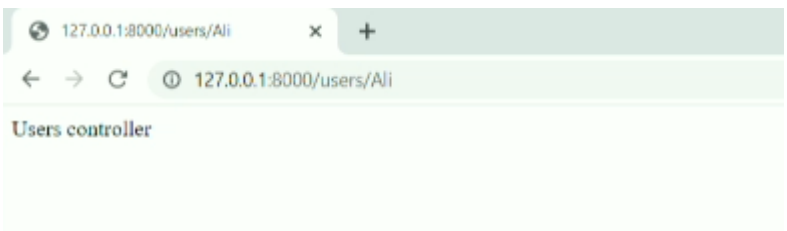
UsersController.php



Controller in Web.php



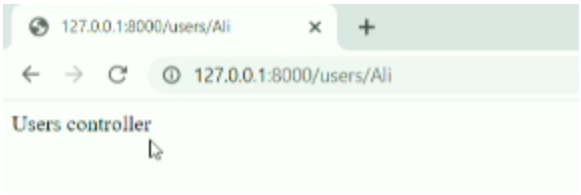
Output:



You can also invoke the controller using the following code:

```
use App\Http\Controllers\UsersController;
Route::get('/users/{name?}',[UsersController::class,'index'])->where('id','[a-zA-Z ]+');
```

Output:



Lesson 157: Laravel – Controller with CRUD Operations

Laravel – Controller with C$RUD Operations

Some operations are redundant, for example, Create, Read, Update and Delete. Let's see how to make a controller for these operators.

Open cmd.exe and write the following Command for the CRUD operations controller.



A new controller will be created under the Controller folder, having all the functions created for Read, Write, Update and Delete.



If we want to check, how many Controllers are created. We can use the following command. Its shows how many routes we defined and how many of them are attached with the Controller.

```
D:\example-app>php artisan make:controller AuthorsController --resource
Controller created successfully.

D:\example-app>php artisan route:list
+--------+----------+--------------------+------+-----------------------------------------+------------+
| Domain | Method   | URI                | Name | Action                                  | Middleware |
+--------+----------+--------------------+------+-----------------------------------------+------------+
|        | GET|HEAD | /                  |      | Closure                                 | web        |
|        | GET|HEAD | about              |      | Closure                                 | web        |
|        | GET|HEAD | api/user           |      | Closure                                 | api        |
|        |          |                    |      |                                         | auth:api   |
|        | GET|HEAD | users/{id}         |      | Closure                                 | web        |
|        | GET|HEAD | users/{id}/{favid} |      | Closure                                 | web        |
|        | GET|HEAD | users/{name?}      |      | App\Http\Controllers\UsersController@index | web        |
+--------+----------+--------------------+------+-----------------------------------------+------------+

D:\example-app>
```

Lesson 158: Laravel – Redirects

Laravel – Redirects

Sometimes, we need to redirect a user. We can direct a user using the following code. There are different methods to redirect in Laravel. The following code will redirect “redirect/helper to the Hokm e page.

```
Route::get('redirect/helper', function() {
    return redirect()->to('/');
});
```

Another way to redirect the web page:

```
Route::get('redirect/shortcut', function() {
    return redirect('/');
});
```

Another way to redirect the web page:

```
Route::get('redirect/facade', function() {
    return Redirect::to('/');
});
```

```
Route::redirect('redirect/route','/');
```

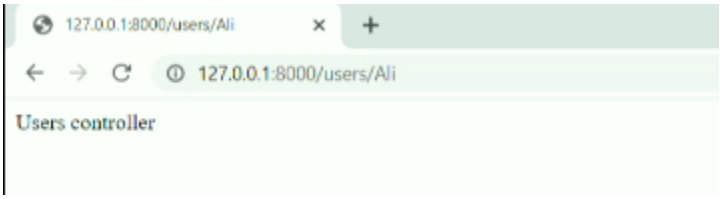
Redirect with a parameter:

```
Route::get('redirect/params', function() {
  return redirect()->route('users.show',['name' => 'Ali']);
});
```

We haven't created any view for the user. So, we are going to create a view for the user at first.

```
use App\Http\Controllers\UsersController;
Route::get('/users/{name?}',[UsersController::class,'index'])
->where('id','[a-zA-Z ]+')
->name('users.show');
```

Here name "Ali" will be passed as a parameter.



Lesson 159: Laravel – Blade Templates

We will use our previous file "about.php". Just rename the file to "about.blade.php".

Edit the file and whatever needs to be dynamic, will be used in Blade Template using {{variable_name}}



Route in web.php:

```php
Route::get('/about', function () {
    return view('about')->with(['data' => 'for about us']);
});
```

Output: “for about us” used as data.

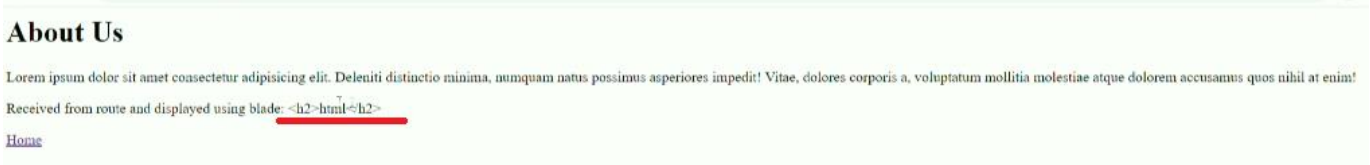


Let’s pass an HTML tag as a parameter and see the output:

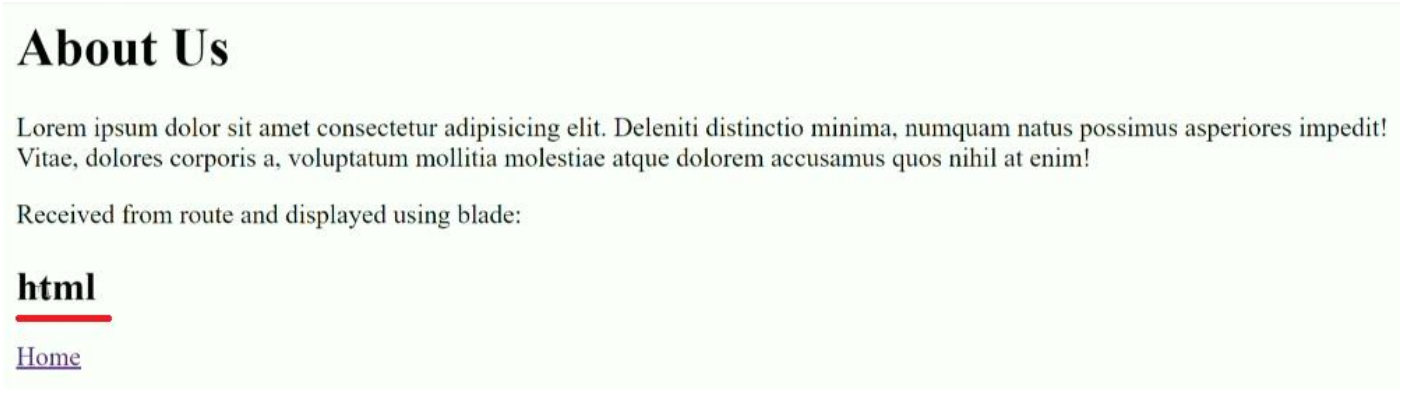
```php
Route::get('/about', function () {
    return view('about')->with(['data' => '<h2>html</h2>']);
});
```

Output:

Here <h2> is processed as a html tag.



We can also show the data as it is, on the webpage using the !! and the data will be shown here as h2.




```blade
<h1>About Us</h1>
<p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Deleniti distinct
<p>Received from route and displayed using blade: {!! $data !!}</p>
<a href="{{ url('/') }}">Home</a>
```

Lesson 160: Laravel – Conditions in Blade

Laravel – Conditions in Blade

We can use different conditions in Laravel Blade Template. Let's see how can we use them.

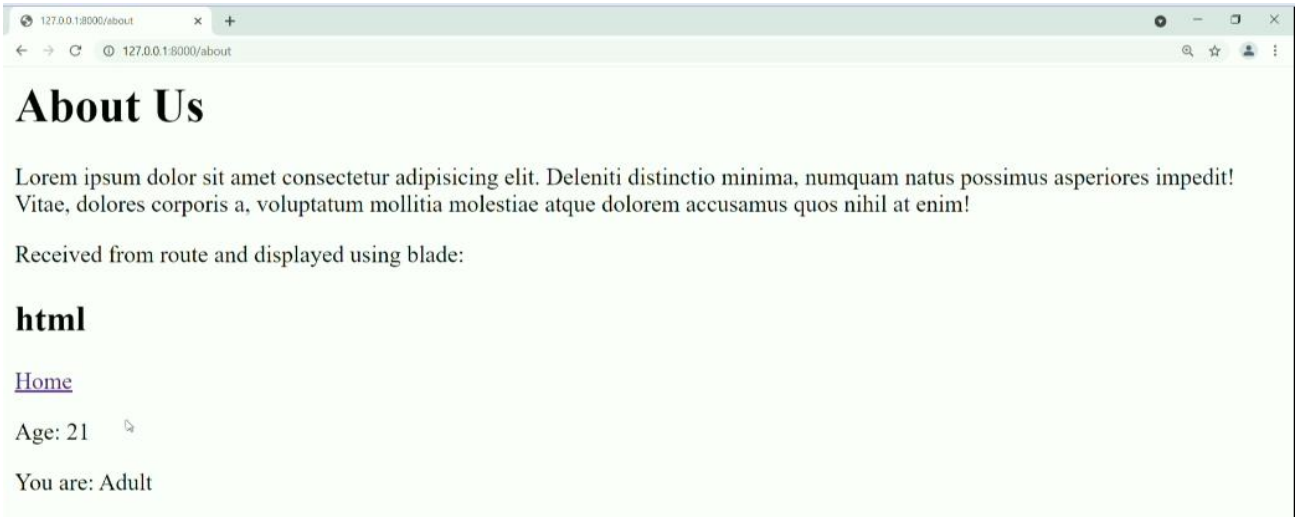
Passing a parameter age with value 21 in Web.php

```
Route::get('/about', function () {
    return view('about')->with(['data' => '<h2>html</h2>', 'age' => 21]);
});
```

about.blade.php | web.php

```
1   <h1>About Us</h1>
2   <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Deleniti distinct
3   <p>Received from route and displayed using blade: {!! $data !!}</p>
4   <a href="{{ url('/') }}">Home</a>
5   <p>Age: {{ $age }}</p>          Receiving the parameter
6   <p>You are:
7   @if ($age < 18)                 if condition
8   Young
9   @elseif ($age < 60)
10  Adult
11  @else
12  Senior
13  @endif
14  </p>    endif
```

Output:



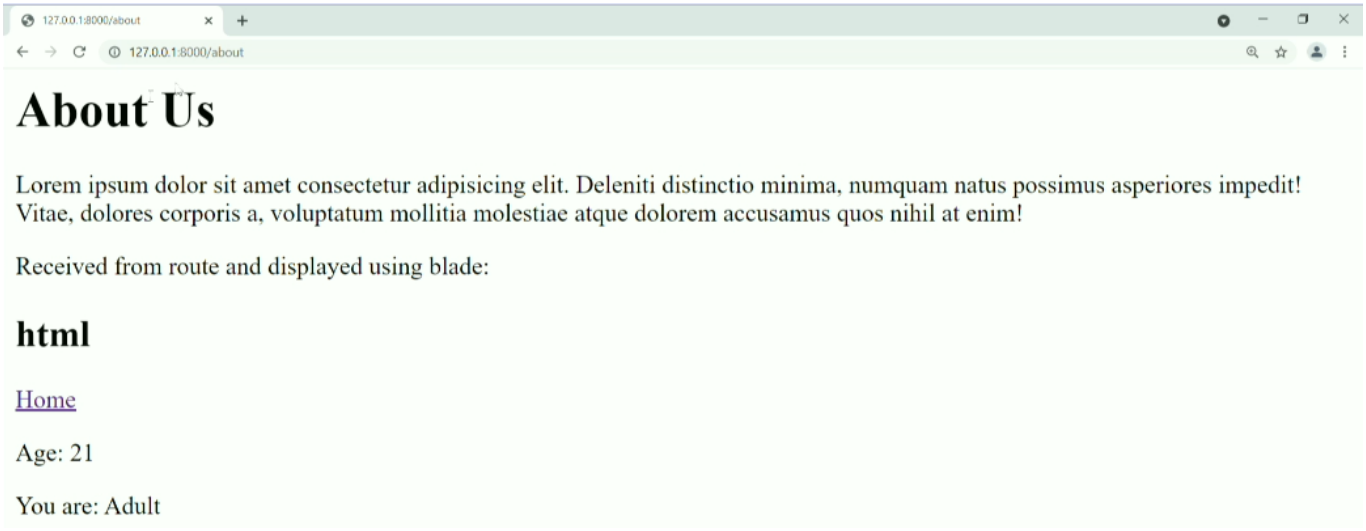
Use of @unless:

This can be used as NOT operator.

about.blade.php | web.php

```
1   <h1>About Us</h1>
2   <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Deleniti distinct
3   <p>Received from route and displayed using blade: {!! $data !!}</p>
4   <a href="{{ url('/') }}">Home</a>
5   <p>Age: {{ $age }}</p>
6   <p>You are:
7   @if ($age < 18)
8   Young
9   @elseif ($age < 60)
10  Adult
11  @else
12  Senior
13  @endif
14
15  @unless($age >= 18)
16  You are elligible.
17  @endunless
18  </p>
```

Output:



Lesson 161: Laravel – Loops in Blade

Laravel – Loops in Blade

Create a Route on about with an array parameter in **“web.php”.**

```php
Route::get('/about', function () {
    return view('about')->with(
        ['data' => '<h2>html</h2>',
        'age' => 15,
        'languages' => ['PHP', 'JavaScript', 'C++', 'Java', 'C#']
        ]
    );
});
```

Just to check of the array is passed or not in "**About.blade.php"**.

about.blade.php ● web.php

```blade
<h1>About Us</h1>
<pre>
    {{ print_r($language )}}
</pre>
```

@For loop:

```blade
@for ($i=0;$i<count($languages);$i++)
  {{ $languages[$i] }}<br>
@endfor
```

Output:



@For Each:

```
<h2>Foreach</h2>
@foreach($languages as $language)
    {{ $language }}<br>
@endforeach
```

Foreach

PHP
JavaScript
C++
Java
C#

@While: This loop will empty the array at the end. So, this loop will be used in such situation we want to empty the array at the end.

```
<h2>While</h2>
@while ($language = array_pop($languages))
    {{ $language }}<br>
@endwhile
```

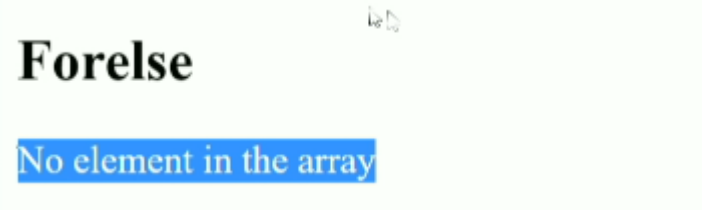
While

C#
Java
C++
JavaScript
PHP

@Forelse:

```
<h2>Forelse</h2>
@forelse($languages as $language)
    $language<br>
@empty
    No element in the array<br>
@endforelse
```

Output:



As @While left no element in this array. So, this @empty section will be displayed on the screen.

Lesson 162: Laravel – Parent Template

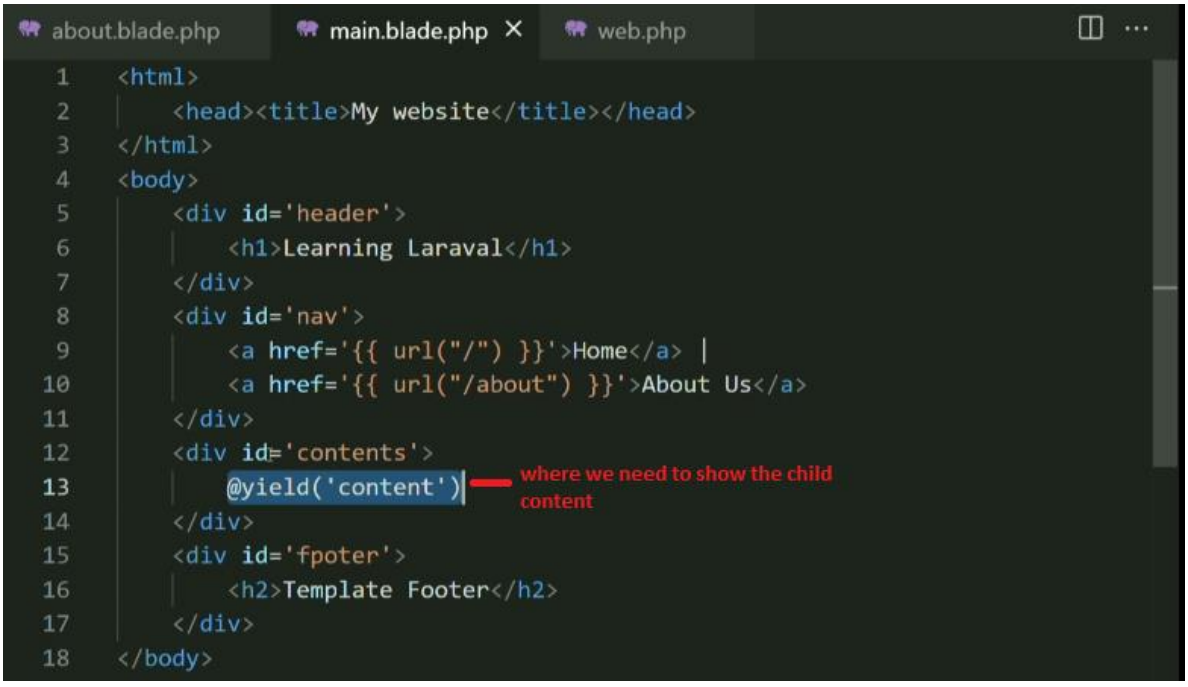
Laravel – Parent Template

We basically define the basic template in the Parent Template. For example, header, footer, navigation bar, left bar, right bar and main contents section.

We will specify in the child template which one is its parent and will use the variables accordingly.

Create a “layouts” folder under “Views”.

Create a file “main.blade.php” in “layouts” folder.



Lesson 163: Laravel Child Templates

Laravel Child Templates

You have already studied the parent templates. Suppose the following directive in the parent template:

@yield(‘content’)

Now we will discuss how to create the child template to fill the contents in the parent template. Note that the contents of the child template are used to fill the parent template.

For this you will have to first mention that which layout this template is using. You can mention the layout as follows:

@extends(‘layouts.main’)

Next you will define the section that will fill the parent template. For this you will use the “sections” as follows:

@section (‘content’)

<h1>Contents</h1>

<P> These are sample contents</P>

@endsection

Note that the sample contents mentioned above will be placed at the location where you have used @yiled(‘content’) directive.

Furthermore, you can define the title of the page as well. This can be done as follows:

@section (‘content’,’Books’)

<h1>Contents</h1>

<P> These are sample contents</P>

@endsection

Now the title of the page will be “Books”.

Week – 13

Lesson 164: Laravel Working with database

Laravel Working with database

Almost every modern web application interacts with a database. Laravel makes interacting with databases extremely simple across a variety of supported databases using raw SQL, a fluent query builder, and the Eloquent ORM. Currently, Laravel provides first-party support for five databases:

MariaDB 10.2+ (Version Policy)

MySQL 5.7+ (Version Policy)

PostgreSQL 9.6+ (Version Policy)

SQLite 3.8.8+

SQL Server 2017+ (Version Policy)

Configuration of Laravel Database

The configuration for Laravel's database services is located in your application's config/database.php configuration file. In this file, you may define all of your database connections, as well as specify which connection should be used by default. Most of the configuration options within this file are driven by the values of your application's environment variables. Examples for most of Laravel's supported database systems are provided in this file.

Configuration Using URLs

Typically, database connections are configured using multiple configuration values such as host, database, username, password, etc. Each of these configuration values has its own corresponding environment variable. This means that when configuring your database connection information on a production server, you need to manage several environment variables.

Environment Configuration

It is often helpful to have different configuration values based on the environment where the application is running. For example, you may wish to use a different cache driver locally than you do on your production server.

To make this a cinch, Laravel utilizes the DotEnv PHP library. In a fresh Laravel installation, the root directory of your application will contain a .env.example file that defines many common environment variables. During the Laravel installation process, this file will automatically be copied to .env.

While configuring database also check database setting in .env file for the following variables

DB_CONNECTION, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME,

and DB_PASSWORD.

Laravel Migrations

Migrations are like version control for your database, allowing your team to modify and share the application's database schema. Migrations are typically paired with Laravel's schema builder to

build your application's database schema. If you have ever had to tell a teammate to manually add a column to their local database schema, you've faced the problem that database migrations solve.

Generating Migrations

To create a migration, use the make:migration Artisan command:

```
php artisan make:migration create_users_table
```

The new migration will be placed in your database/migrations directory. Each migration file name contains a timestamp, which allows Laravel to determine the order of the migrations.

Lesson 165: Creating Tables

Creating Tables

To create a table we will run migration command first

```
php artisan make:migration create_users_table
```

The new migration will be placed in your database/migrations directory. Each migration file name contains a timestamp, which allows Laravel to determine the order of the migrations.

Creating Tables

To create a new database table, use the create method on the Schema facade. The create method accepts two arguments: the first is the name of the table, while the second is a closure which receives a Blueprint object that may be used to define the new table:

```
use Illuminate\Database\Schema\Blueprint;

use Illuminate\Support\Facades\Schema;


Schema::create('users', function (Blueprint $table) {

$table->id();

$table->string('name');

$table->string('email');

$table->timestamps();

});
```

When creating the table, you may use any of the schema builder's column methods to define the table's columns.

Lesson 166: Running Migrations

Running Migrations

To run all of your outstanding migrations, execute the migrate Artisan command:

```
php artisan migrate
```

If you would like to see which migrations have run thus far, you may use the migrate:status Artisan command:

```
php artisan migrate:status
```

Note: If you receive a "class not found" error when running migrations, try running the composer dump-autoload command.

Rolling Back Migrations

To roll back the latest migration operation, you may use the rollback Artisan command. This command rolls back the last "batch" of migrations, which may include multiple migration files:

```
php artisan migrate:rollback
```

You may roll back a limited number of migrations by providing the step option to the rollback command. For example, the following command will roll back the last five migrations:

```
php artisan migrate:rollback --step=5
```

The migrate:reset command will roll back all of your application's migrations:

```
php artisan migrate:reset
```

Lesson 167: Adding a Foreign key

Adding a Foreign key

Foreign Key Constraints

Laravel also provides support for creating foreign key constraints, which are used to force referential integrity at the database level. For example, let's define a user_id column on the posts table that references the id column on a users table:

```
use Illuminate\Database\Schema\Blueprint;

use Illuminate\Support\Facades\Schema;
```

```
Schema::table('posts', function (Blueprint $table) {

$table->unsignedBigInteger('user_id');


$table->foreign('user_id')->references('id')->on('users');

});
```

Since this syntax is rather verbose, Laravel provides additional, terser methods that use conventions to provide a better developer experience. When using the foreignId method to create your column, the example above can be rewritten like so:

```
Schema::table('posts', function (Blueprint $table) {

$table->foreignId('user_id')->constrained();

});
```

The foreignId method creates an UNSIGNED BIGINT equivalent column, while the constrained method will use conventions to determine the table and column name being referenced. If your table name does not match Laravel's conventions, you may specify the table name by passing it as an argument to the constrained method:

```
Schema::table('posts', function (Blueprint $table) {

$table->foreignId('user_id')->constrained('users');

});
```

You may also specify the desired action for the "on delete" and "on update" properties of the constraint:

```
$table->foreignId('user_id')

->constrained()

->onUpdate('cascade')
```

Lesson 168: Running Raw SQL

Running Raw SQL

You can insert data in 2 ways:

Method 1:

Open phpMyAdmin

Select the database you'd like to run a query on

You must select a database from the left column first

Select SQL

Look for the database name again above the text field, confirm this is the correct database you intend to run a query on

Write or paste your query in the text field

Click GO

Some queries will require a second confirmation in order to run.

Method 2:

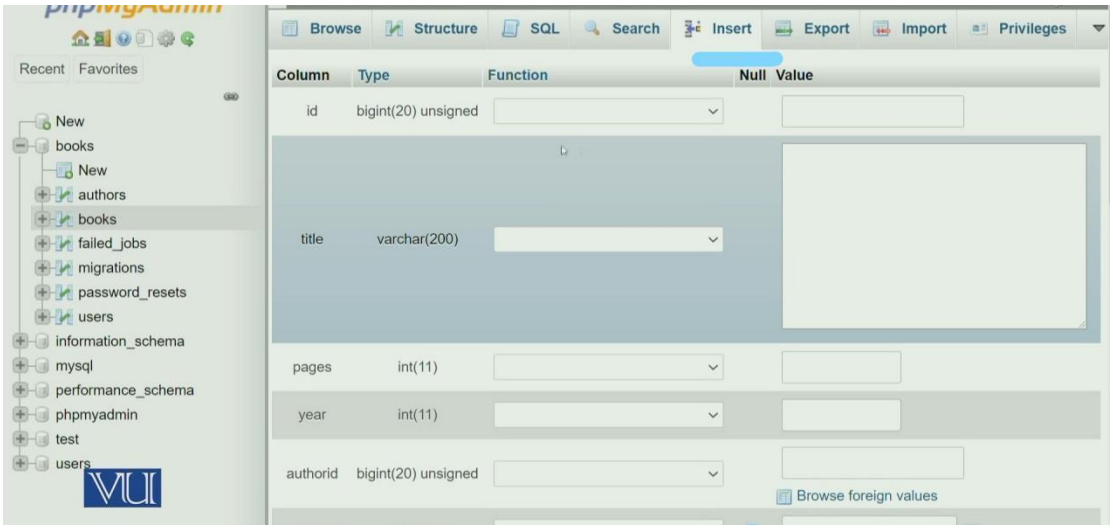
Open phpMyAdmin

Select the database you'd like to run a query on

You must select a database from the left column first

Select Insert

Look for the database name again above the text field, confirm this is the correct database table in which you intend to add data



Insert data in input field and Click GO

Resource Controller:

If you think of each Eloquent model in your application as a "resource", it is typical to perform the same sets of actions against each resource in your application.

Laravel resource routing assigns the typical create, read, update, and delete ("CRUD") routes to a controller with a single line of code. To get started, we can use the make:controller Artisan command's --resource option to quickly create a controller to handle these actions:

```
php artisan make:controller BookController --resource
```

Lesson 169: **Laravel Views and Data**

Laravel Views Introduction

It's not practical to return entire HTML documents strings directly from your routes and controllers. Thankfully, views provide a convenient way to place all of our HTML in separate files. Views separate your controller / application logic from your presentation logic and are stored in the resources/views directory.

A simple view may look like this

<!-- View stored in resources/views/greeting.blade.php -->

```
<html>
    <body>
        <h1>Hello, {{ $name }}</h1>
    </body>
</html>
```

You may create a view by placing a file with the .blade.php extension in your application's resources/views directory. The .blade.php extension informs the framework that the file contains a Blade template. Blade templates contain HTML as well as Blade directives that allow you to easily echo values, create "if" statements, iterate over data, and more.

Once you have created a view, you may return it from one of your application's routes or controllers using the global view helper:

```
Route::get('/', function () {
return view('greeting', ['name' => 'VU']);
});
```

To fetch data for view from database in controller you may use this method given below. It

```
class UserController extends Controller
{
    /**
     * Show a list of all of the application's users.
     *
     * @return \Illuminate\Http\Response
     */
    public function index()
    {
        $users = DB::table('users')->get();


        return view('user.index', ['users' => $users]);
    }
}
```

Here “return view('user.index', ['users' => $users]);” will pass data to view user after fetching data from data base.

Lesson 170: Laravel Query Builder

Laravel Query Builder

Laravel's database query builder provides a convenient, fluent interface to creating and running database queries. It can be used to perform most database operations in your application and works perfectly with all of Laravel's supported database systems.

The Laravel query builder uses PDO parameter binding to protect your application against SQL injection attacks. There is no need to clean or sanitize strings passed to the query builder as query bindings.

Retrieving All Rows From A Table

You may use the table method provided by the DB facade to begin a query. The table method returns a fluent query builder instance for the given table, allowing you to chain more constraints onto the query and then finally retrieve the results of the query using the get method:

```
<?php


namespace App\Http\Controllers;


use App\Http\Controllers\Controller;

use Illuminate\Support\Facades\DB;


class UserController extends Controller

{

/**

* Show a list of all of the application's users.

*

* @return \Illuminate\Http\Response

*/

public function index()

{

$books= DB::table(books)->get();


return view(books.index', [books=> $books]);

}

}
```

If you don't need an entire row, you want to apply a condition that only those books which are entered after or in 2021 then this method will return the only those values.

```
$books= DB::table('books')->where('year','>=', 2021)->get();
```

Joins

The query builder may also be used to add join clauses to your queries. To perform a basic join, you may use the join method on a query builder instance. The first argument passed to the join method is the name of the table you need to join to, while the remaining arguments specify the column constraints for the join. You may even join multiple tables in a single query:

```
*/
public function index()
{
    // $books = DB::select('SELECT * FROM books');
    $books = DB::table('books')
                ->join('authors','books.authorid','=','authors.id')
                ->select('books.*','authors.name')
                ->get();
    return view('books.index',['books' => $books]);
}
```

Lesson 171: Laravel – Eloquent

Eloquent ORM Introduction

Laravel includes Eloquent, an object-relational mapper (ORM) that makes it enjoyable to interact with your database. When using Eloquent, each database table has a corresponding "Model" that is used to interact with that table. In addition to retrieving records from the database table, Eloquent models allow you to insert, update, and delete records from the table as well.

Table Names

After glancing at the example above, you may have noticed that we did not tell Eloquent which database table corresponds to our Flight model. By convention, the "snake case", plural name of the class will be used as the table name unless another name is explicitly specified. So, in this case, Eloquent will assume the Flight model stores records in the flights table, while an AirTrafficController model would store records in an air_traffic_controllers table.

If your model's corresponding database table does not fit this convention, you may manually specify the model's table name by defining a table property on the model:

```
<?php


namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Model;


class Flight extends Model

{

/**

* The table associated with the model.

*

* @var string

*/

protected $table = 'my_flights';

}
```

Retrieving Models

Once you have created a model and its associated database table, you are ready to start retrieving data from your database. You can think of each Eloquent model as a powerful query builder allowing you to fluently query the database table associated with the model. The model's all method will retrieve all of the records from the model's associated database table:

```
use App\Models\Flight;


foreach (Book::all() as $book) {

echo $book->name;

}
```

Building Queries

The Eloquent all method will return all of the results in the model's table. However, since each Eloquent model serves as a query builder, you may add additional constraints to queries and then invoke the get method to retrieve the results:

```
$books= Book::where('active', 1)

->orderBy('name')

->take(10)
```

```
->get();
```

For Insert, Update, Delete

Insert

$book = new Book;

$book->title = "Introduction to Www"

$book->save ();

Update

$book = Book: :find (1);

$book->title = "Introduction to CS";

$book->save ()):

Delete

$book = Book: :find (1);

Sbook->delete ():

Lesson 172: Laravel Eloquent (Getting data)

Laravel Eloquent (Getting data)

You may generate various other types of classes when generating a model, such as factories, seeders, policies, controllers, and form requests.

To get started, let's create an Eloquent model. Models typically live in the app\Models directory and extend the Illuminate\Database\Eloquent\Model class. You may use the make:model Artisan command to generate a new model:

Create BookController first using following command

php artisan make: controller BookController --resource

Now create eloquent model with --factory

php artisan make: model Book --factory

Previously we were fetching data by conventional laravel query builder now by fetching data by Eloquent room we will be using following method

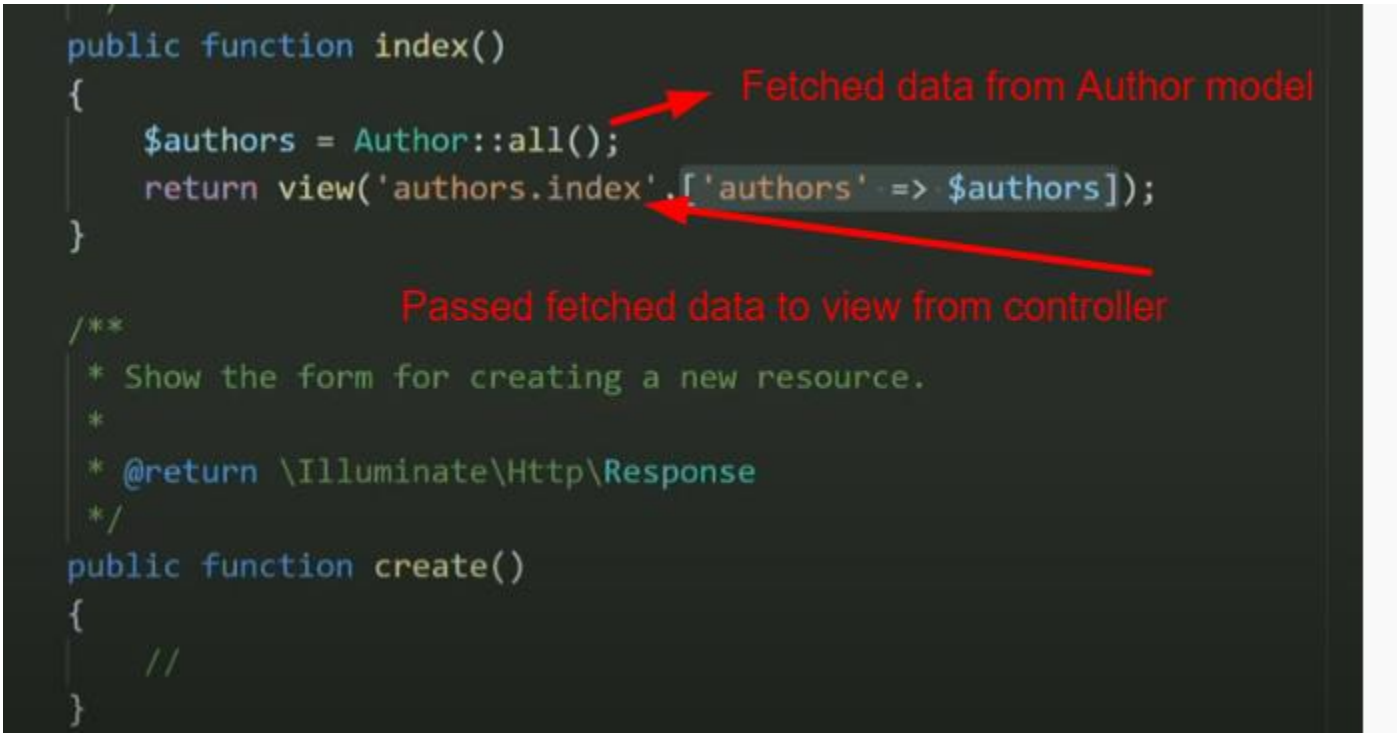


This will return all data in books table and pass it to the view books.index using Eloquent.

Lesson 173: Laravel Eloquent (Viewing data)

Laravel Eloquent (Viewing data)

Make new controller as par Eloquent naming convictions and fetch data in your controller first as shown below in figure



After fetching data from data base using Eloquent, data is now passed to the index view.

Alternatively ::all() can also be used in model, when model is associated with database table

, you are ready to start retrieving data from your database. You can think of each Eloquent model as a powerful query builder allowing you to fluently query the database table associated with the model. The model's all method will retrieve all of the records from the model's associated database table:

```
use App\Models\Author;


foreach (Author::all() as $author) {

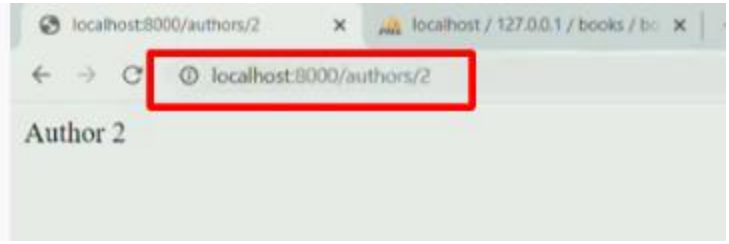
echo $author->name;

}
```

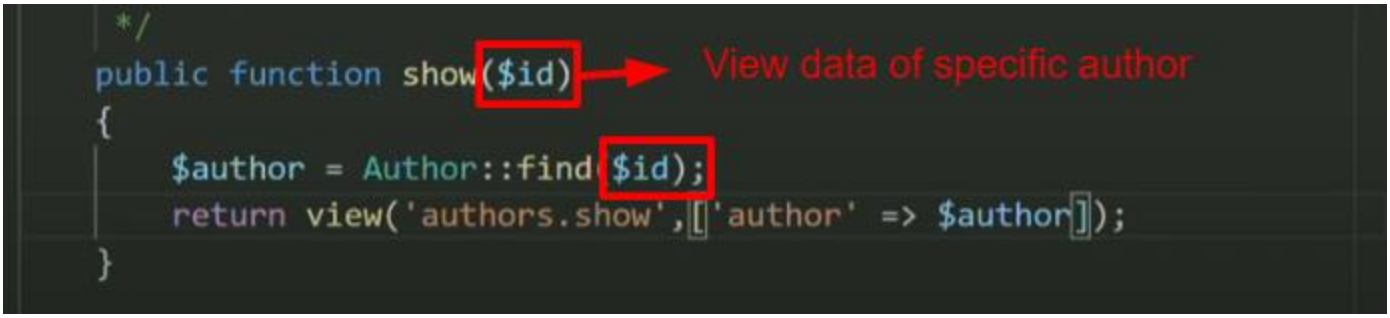
View data of specific ID

To view data of specific author will provide id of the the author of which data we need.

Resource controller will detect ID from the URL.



ID will be passed to the show function in our resource controller and controller will fetch data from database using model and pass it to blade view.



Lesson 174: Laravel - Eloquent (Relations)

Laravel - Eloquent (Relations)

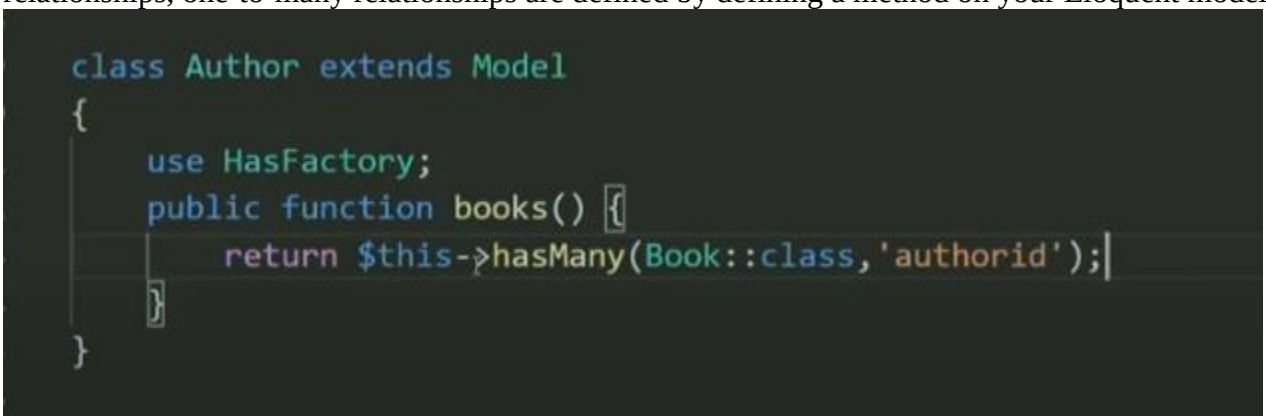
Eloquent relationships are defined as methods on your Eloquent model classes. Since relationships also serve as powerful query builders, defining relationships as methods provides powerful method chaining and querying capabilities. For example, we may chain additional query constraints on this posts relationship:

$user->posts()->where('active', 1)->get();

But, before diving too deep into using relationships, let's learn how to define each type of relationship supported by Eloquent.

One To Many

A one-to-many relationship is used to define relationships where a single model is the parent to one or more child models. For example, one author has many books. Like all other Eloquent relationships, one-to-many relationships are defined by defining a method on your Eloquent model:



Lesson: 175: **Laravel Form Request (Create)**

Laravel Form Request (Create)

Blade is the simple, yet powerful templating engine that is included with Laravel. Unlike some PHP templating engines, Blade does not restrict you from using plain PHP code in your templates. In fact, all Blade templates are compiled into plain PHP code and cached until they are modified, meaning Blade adds essentially zero overhead to your application. Blade template files use the .blade.php file extension and are typically stored in the resources/views directory.

CSRF Field

Anytime you define an HTML form in your application, you should include a hidden CSRF token field in the form so that the CSRF protection middleware can validate the request. You may use the @csrf Blade directive to generate the token field:

<form method="POST" action="/profile">

@csrf

...

</form>

Method Field

Since HTML forms can't make PUT, PATCH, or DELETE requests, you will need to add a hidden _method field to spoof these HTTP verbs. The @method Blade directive can create this field for you:

<form action="/foo/bar" method="POST">

@method('PUT')

</form>

Blade views may be returned from routes or controller using the global view helper. Of course, as mentioned in the documentation on views, data may be passed to the Blade view using the view helper's second argument:

```
Route::get('/', function () {

return view('greeting', ['name' => 'Finn']);

});
```

In our lecture video we will be writing these two function for form view and for form store .

```
public function create()
{
    return view('authors.create');
}

/**
 * Store a newly created resource in storage.
 *
 * @param  \Illuminate\Http\Request  $request
 * @return \Illuminate\Http\Response
 */
public function store(Request $request)
{
    $author = new Author;
    $author->name = $request->input('name');
    $author->email = $request->input('email');
    $author->category = $request->input('category');
    $author->save();
}
```

Form create will pass the control of application blade view template to view the form. After for submission it route will direct the control to store controller function to store the form.

Lesson 176: **Laravel Form Request (Update)**

Laravel Form Request (Update)

This controller method will process the edit form. It is very similar to store(). We will validate, update, and redirect.

Create a new function ‘’edit’’ in controller, this function will be getting an id from view, on the basis of that id function edit will request data from author table only of that specific id.

```
public function edit($id)
{
    $author = Author::find($id);
    return view('authors.edit',['author' => $author]);
}
```

After getting data from database, controller function will pass data object to view where data will be represented in editable form.

After submission of form data will be submitted to controller function update which will update data in database accordingly using controller function given below.

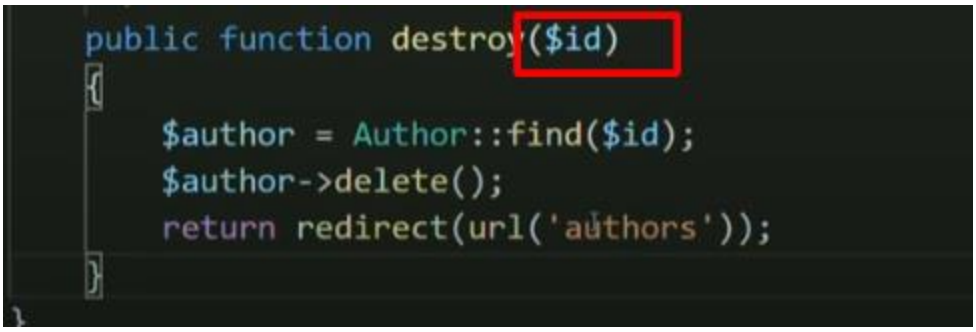
```
public function update(Request $request, $id)
{
    $author = Author::find($id);
    $author->name = $request->input('name');
    $author->email = $request->input('email');
    $author->category = $request->input('category');
    $author->save();
    return view('authors.show',['author' => $author]);
}
```

Lesson 177: **Laravel Form Request (Delete)**

Laravel Form Request (Delete)

When Form is displayed with delete button, on pressing delete button we can pass id of instance which needs to be deleted to the controller function destroy directly. This could also be done using JavaScript.

When Id is passed to controller function destroy then function after loading author model will execute delete() function. On execution of delete() function in model delete query will run and instance will be deleted.



Week 14

Laravel CRUD Operations

You have already studied about the Create, Update and Delete operations in previous topics. Here we will continue the same code and will add the links to let users perform these operations.

Consider the code given below:

```
@foreach ($authors as $author)

<tr>

        <td> {{ $author->id}}</td>

        <td> {{ $author->name}}</td>

        <td> {{ $author->email}}</td>

        <td> {{ $author->category}}</td>

        <td>

                <a href='{{url("authors/".$author->id) }}'>Details</a> |

                <a href='{{url("authors/".$author->id."/edit")
}}'>Edit</a> |

                <a href='{{url("authors/".$author->id)."/delete"
}}'>Delete</a> |

          </td>

</tr>
```

@endforeach

In the above code, we have executed “foreach” loop to display the information of each author. For each author we will display the author id, name, email and category. Furthermore, now we will add the following links against each author:

Details: Clicking this link will display the details of each author. The information will be displayed using author id.

Edit: Click this link will help us edit the details of the author.

Delete: Clicking this link will help us delete a particular author.

Note: As all these tasks are performed in loop, the links will appear

Security principals

Information security is normally mentioned in the form of a triangle having three dimensions as shown below:

We need to consider all of these three dimensions for implementing the security of a webpage.

Confidentiality refers to the privacy of the data. It means that the private data of a user should only be accessible to that user. No other user should have the access to the private data of other users.

Integrity means that the data should be correct and secure. This means that the unauthorized access and modifications should be prevented.

Availability means that the website and the data should always be available to the authorized users whenever requested.

Risk Assessment and Management:

It is important to assess the risk in the web development domain. Risk assessment means that we measure that how the risk is likely to occur and what will be the impact of the risk

We can discuss the security principals from different angels. We will discuss the following aspects w.r.t. risks:

- Actors
- Impact
- Threat
- Vulnerabilities

Actors:

Generally there are three categories of Actors.

- Internal actors are those people that work within the organization e.g. cashier, IT staff, CEO etc.
- External actors are the people outside the organization. Majority of the attacks are launched by these people. Normally these people have bad intentions and high skills to penetrate within the system of the organization
- Partner actors Patterns are the people an organization works with. Partners may also be source of an attack and may result in consequences as the partners have access to the data of the organization (system) in majority of the cases.

Impact:

Impact refers to the cost and loss that we will have to bear if attack is successful, for example, which systems were affected and which data was stolen. Impact relates to loss of the availability, confidentiality and/or integrity.

- A loss of availability: It prevents users from accessing some or all of the systems.

- A loss of confidentiality: It means your data is exposed to unauthorized users.
- A loss of integrity: it means that your data is changed and you don’t have the accurate data now e.g. placing fake orders, changing the home address of the user etc.

Threats:

There are different categories of threats. Normally threats are categorized using the “STRIDE” mnemonic.

- Spoofing—The attacker uses someone else’s information to access the system.
- Tampering—The attacker modifies some data in non-authorized ways.
- Repudiation—The attacker removes all trace of their attack, so that they cannot be held accountable for other damages done.
- Information disclosure—The attacker accesses data they should not be able to.
- Denial of service—The attacker prevents real users from accessing the systems.
- Elevation of privilege—The attacker increases their privileges on the system thereby getting access to things they are not authorized to do.

Vulnerabilities:

Vulnerabilities refer to the security holes in your system. Following are top five classes of vulnerability from the Open Web Application Security Project:

1. Injection

2. Broken authentication and session management

3. Cross-site scripting

4. Insecure deserialization

5. Security misconfiguration

Security Policy:

For each system in an organization, there should be a well-defined policy related to the usage and security of the system. A policy can be defined with respect to the following perspectives:

- Usage policy means that what systems are available for the users to use and under which circumstances.
- Authentication policy defines that how the users are granted access to the systems.
- Legal policies includes a number of things e.g. data retention and backup policies, accessibility requirements etc.

Whatever the circumstances are, the business should continue. So, it is better to make plans and follow the good practices to keep the system going in case of hard security circumstances.

Following are some good practices that should be considered:

- Admin Password Management
- Backups and Redundancy
- Geographic Redundancy
- Stage Mock Events
- Auditing

Reference:

Connolly & Hoar, "Fundamentals of Web Development" - 2nd Ed. Pearson Ed. (2018)

Security by design

Security needs to be implemented in each phase of software development e.g. when you are collecting the requirements, you need to check your security requirements at the same stage, what should be your security policy etc. similarly, you need to find out and implement the security requirements at design phase e.g. you need assess the threats and risks, you need to devise the plans for redundancy management etc. The same applies to the web development

Following are some points that you will have to consider at each phase:

Requirements:

- Privacy needs
- Security Policy
- CIA Triad

Design

- Threat assessment
- Risk Assessment
- Redundancy Planning

Implementation

- Pair Programming
- Code Reviews
- Defensive Programming

Testing

- Security Unit Tests
- Vulnerability Tests
- Test Cases

Deployment

- Penetration Testing

- Attach thyself
- Default Values

Reference:

Connolly & Hoar, "Fundamentals of Web Development" - 2nd Ed. Pearson Ed. (2018)

Authentication:
To maintain confidentiality and integrity, the identity of an individual must be verified. The process of verifying individual identity is known as authentication. Authentication is the process to confirm what someone say they are. On successful authentication the user is allowed to access the required resources.

Authentication Factors:

The things that you ask from someone to verify their identity are known as authentication factors. There are three factors of authentications factors

1. Knowledge: the things that you know like username and passwords, Security questions, among others. This information can be found by someone else and can shared easily.
2. Ownership: the things you have that are certificates, access card, USB keys, software tokens, email, SMS among others. These factors are vulnerable to be stolen, Snatched like other possession. Moreover, some these factors can be duplicated and can be misused.
3. Inherence: something you are like typing speed, Key pattern intervals, Biometrics (Retinas, Facial recognition, fingerprints among others). These authentication factors are less vulnerable and are difficult to forge and shared.

Authentication Factors (usage perspective):

Authentication can be Single factor or Multi-factor authentication. In single-factor authentication, a single of the above factor is used validate the users claim. In case of single factor authentication, the knowledge factor is comparatively more vulnerable than ownership and inherence factor. Single-factor based authentication can be implemented as knowing the password and relies on the strength of the password. Password can be easily learnt by the person looking over there during the password entry and can be retrieved from the browser or through phishing scams and other attacks. In the scenario where more authentication confidence is required like banks transaction or accessing sensitive information, multi-factor authentication need to be considered.

Multi-factor authentication requires at-least two distinct factors like one from knowledge and other form ownership. This kind of authentication increase security and reduces attacks threats. This is because to break multifactor authentication, the attacker needs to have two different attack vectors. ATM machine is an example of multi-factor authentication where the customer need to use the ATM card and enter PIN as well.

Type of authentication

There are various ways a user can be authenticated - some of which include, Session-based authentication, Cookie-based authentication, Token-based authentication, Claims-based authentication and Hypertext Transfer Protocol (HTTP) authentication mechanisms.

HTTP authentication:

HTTP authentication is one of the simplest ways to identify users logged into a system. Common types of HTTP authentication include: Basic, Digest and Form Based

- HTTP basic authentication: The users need to provide username and password while requesting to the server and are sent in the authorization header. The user's name and passwords are

concatenated into a single string username: password and encoded in Base64.

- HTTP Digest authentication: In this technique all the authentication credentials like http method, username, password, and requested URIs are encrypted using MD5. However, this authentication is still vulnerable to main-in-the-middle attack.
- Form based authentication: In Form-based authentication, the user credentials (username and password) are sent in a plain text (like Basic authentication) using forms, which can lead to exposure of usernames and passwords except connecting via HTTPS/SSL. One benefit is that, the developer is allowed to customize the authentication and error pages sent by the browser.

Third party authentication:

To store and maintain authentication credentials even if it is username and password is not so easy. Moreover, direct log in through username and password is not safe and secure. Therefore, most of the larger organizations use third party authentication services for authentication of users. Third party authentication service providers are the organization who have already invested in the authentication and allows you to use their system to authenticate your users.

Open Authentication (OAuth) is one of the most popular third party authentication services. OAuth is "An open protocol to allow secure authorization in a simple and standard method from web, mobile and desktop applications". OAuth allows users to authenticate a site by using credentials from another site and have many versions. OAuth allows users to share specific data with an application while keeping their usernames, passwords, and other information private. For example, an application can use OAuth services to obtain permission from users to store files in their Google Drives.

References:

https://www.oauth.com/
https://developers.google.com/identity/protocols/oauth2

Cryptography:

Cryptography is an information security tactic used to protect enterprise information and communication from cyber threats through the use of codes. It is considered the art of hiding information to prevent unauthorized access to your data.

This practice refers to secure information and communication techniques derived from mathematical concepts and a set of rule-based calculations, called algorithms, to transform messages in ways that are hard to decipher. These algorithms are then used for cryptographic key generation, digital signing, verification to protect data privacy, web browsing on the internet, and confidential communication like credit card transactions and emails.

Cryptography achieves several information security-related objectives including confidentiality, integrity, and authentication, and non-repudiation.

Basic Concepts:

Cryptography: The art or science encompassing the principles and methods of transforming an intelligible message into one that is unintelligible, and then retransforming that message back to its original form

Plain text: The original intelligible message
Cipher text: The transformed message
Cipher: An algorithm for transforming an intelligible message into one that is unintelligible by transposition and/or substitution methods
Key: Some critical information used by the cipher, known only to the

sender & receiver
Encipher (encode): The process of converting plaintext to cipher text using a cipher and a key
Decipher (decode): The process of converting cipher text back into plaintext using a cipher and a key
Cryptanalysis: The study of principles and methods of transforming an unintelligible message back into an intelligible message without knowledge of the key.
Code: An algorithm for transforming an intelligible message into an unintelligible one using a code-book

Ciphers: All the encryption algorithms are based on two general principles: substitution, in which each element in the plaintext is mapped into another element, and transposition, in which elements in the plaintext are rearranged.

Public Key Cryptography:

There are two main methods to encrypt and decrypt a message that need to securely communicated between sender and receiver.

Symmetric Key: In symmetric key algorithms, the encryption and decryption keys are known both to sender and receiver. The encryption key is shared and the decryption key is easily calculated from it. The big challenge in the symmetric key ciphers is the exchange of secret before starting the communication.

Public key cryptography:

The public key ciphers, the issue of exchanging the secret has been resolved by using two different keys i.e., public key used for encryption and private key used for decryption.
In public key cryptography, encryption key is made public, but it is computationally infeasible to find the decryption key without the information known to the receiver.

Digital Signature:

Digital signature is a secure method used to validate the digital document send by the sender from three perspective:

- To validate the sender who claim the digital document also known as authenticity.
- The received document is same as send by the sender i.e., not changed in the middle which is termed as integrity of the document
- The sender cannot deny the sent document which is termed as nonrepudiation.

Digital document can be signed by using the concept of public key cryptography. It can be done by encrypting the hash of the transmitted message and the receiver apply the same technique to get the hash and decrypting the signature.

References:

https://www.triskelelabs.com/
https://www.vssut.ac.in/lecture_notes/lecture1428550736.pdf

HTTPS:
Hyper Text Transfer Protocol Secure (HTTPS) is an HTTP protocol running on the top of Transport Layer Security (TLS). TLS is an improved form of Secure Socket Layer (SSL).

Secure Handshakes:

The foundation for establishing a secure link happens during the initial handshake. This handshake must occur on an IP address level, so while you can host multiple secure sites on the same server, each domain must have its own IP address in order to perform the low-level handshaking as shown in the following figure.

Certificates:

The certificate that is transmitted during the handshake is actually an X.509

certificate, which contains many details including the algorithms used, the

domain it was issued for, and some public key information. The complete

X.509 specification can be found in the International Telecommunication

Union's directory of public key frameworks.

From HTTP to HTTPS:

Despite all the advantages of a secure site (including a modest boost from some search engines in ranking, and an increasing trend to serve all websites over https), there are many considerations to face when migrating or setting up a secure site. Coordinating the migration of a website can be a complex endeavor involving multiple divisions of a company. In addition to marketing materials being updated in the physical world to use the new URL, there are some nontechnical issues that need to be addressed like the annual budget to purchase and renew a certificate from a certificate authority. In addition to the business questions there are also some technical considerations in migrating to HTTPS.

References:

R. Connolly and R. Hoar, Fundamentals of Web Development, 2nd ed., Pearson, 2017.

Security Best Practices:

With all our previous discussion of security thinking, cryptographic principles, and authentication in mind, it's now time to discuss some practical things you can do to harden your system against attacks.

A system will be targeted either purposefully or by chance. The majority of attacks are opportunistic attacks where a scan of many systems identifies yours for vulnerabilities. Targeted attacks occur less often, but are by their nature more difficult to block. Either way there are some great techniques to make your system less of a target.

Auditing:

Auditing is the process by which a third party is invited (or required) to check over your systems to see if you are complying with regulations and your claims. Auditing happens in the financial sector regularly, with a third-party auditor checking a company's financial records to ensure everything is as it should be. Oftentimes, simply knowing an audit will be done provides

incentive to implement proper practices.

The practice of logging, where each request for resources is stored in a secure

log, provides auditors with a wealth of data to investigate. Linux, by default,

stores logs related to ssh (Secure Shell) and other network access. You can exert control

over these and the logging of your Apache server.

Another common practice is to use databases to track when records are edited

or deleted by storing the timestamp, the record, the change, and the user who was logged in. These logs are often stored in separate, audit tables.

References:

R. Connolly and R. Hoar, Fundamentals of Web Development, 2nd ed., Pearson, 2017.

Common threats vectors:

These are the common factors that an attacker uses to penetrate to a web application.

Brute force attacks:

A brute force attack uses trial-and-error to guess login info, encryption keys, or find a hidden web page. Hackers work through all possible combinations hoping to guess correctly.

This is an old attack method, but it's still effective and popular with hackers. Because depending on the length and complexity of the password.

Automated tools help with brute force attacks. These use rapid-fire guessing that is built to create every possible password and attempt to use them. Brute force hacking software can find a single dictionary word password within one second.

Protection from Brute force attacks:

XML:

XML stands for eXtensible Markup Language and was designed to store and transport data. It is a markup language, but unlike HTML, XML can be used to mark up any type of data. One key benefit of XML data is that as plain text, it can be read and transferred between applications and different operating systems. XML is used on the web server to communicate asynchronously with the browser. Used as a data interchange format for moving information between systems over the Internet.

Well-Formed XML

- Element names are composed of any of the valid characters
- Element names can’t start with a number
- There must be a single-root element
- All elements must have a closing element (or be self-closing)
- Elements must be properly nested
- Elements can contain attributes
- Attribute values must always be within quotes
- Element and attribute names are case sensitive

Example

```
<?xml version="1.0" encoding="ISO-8859-1"?>
   <art>
      <painting id="290">
         <title>Balcony</title>
         <artist>
            <name>Manet</name>
            <nationality>France</nationality>
         </artist>
         <year>1868</year>
         <medium>Oil on canvas</medium>
      </painting>
   </art>
```

Valid XML:

- A valid XML document is one that is well formed and whose element and content conform to the rules of either its document type definition (DTD) or its schema
- DTDs tell the XML parser which elements and attributes to expect in the document as well as the order and nesting of those elements

- A DTD can be defined within an XML document or within an external file

Document Type Definition (DTD):

Example of XML Schema:

- XML

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<art>
  <painting id="290">
    <title>Balcony</title>
    <artist>
      <name>Manet</name>
      <nationality>France</nationality>
    </artist>
    <year>1868</year>
    <medium>Oil on canvas</medium>
  </painting>
</art>
```

- Corresponding XML Schema

```
<?xml version = "1.0" encoding = "UTF-8"?>

<xs:schema xmlns:xs =
"http://www.w3.org/2001/XMLSchema">

  <xs:element name="art">

    <xs:complexType>

      <xs:sequence>

        <xs:element name="painting"
maxOccurs="unbounded" minOccurs="0">

          <xs:complexType>

            <xs:attribute type="xs:integer"
name="id"/>

            <xs:sequence>

              <xs:element type="xs:string"
name="title"/>

              <xs:element name="artist">

                <xs:complexType>

                  <xs:choice>

                    <xs:element type="xs:string"
name="name"/>

                    <xs:element type="xs:string"
name="nationality"/>

                  </xs:choice>

                </xs:complexType>
```

```
                </xs:element>

                <xs:element type="xs:short"
name="year"/>

                <xs:element type="xs:string"
name="medium"/>

              </xs:sequence>

            </xs:complexType>

          </xs:element>

        </xs:sequence>

      </xs:complexType>

    </xs:element>

</xs:schema>
```

References:

R. Connolly and R. Hoar, Fundamentals of Web Development, 2nd ed., Pearson, 2017.

Xpath:

- XML Path Language
- Used for accessing elements and attributes in an XML document
- Widely used in JavaScript for accessing certain elements/attributes of HTML documents

XPath expressions:

Expression	Description
nodename	All nodes with the name "nodename"
/	From root
//	Anywhere in the document
.	Current node
..	Parent node
@	Attribute
[]	Predicate

‹Path Examples

-

```
<?xml version="1.0" encoding="ISO-
8859-1"?>

<art>

  <painting id="290">

    <title>Balcony</title>

    <artist>

      <name>Manet</name>
```

```
        <nationality>France</nationa
lity>

    </artist>

    <year>1868</year>

    <medium>Oil on canvas</medium>

  </painting>

</art>
```

- XML

-
- Corresponding XPath
  - /art
    Selects element "art"
  - //year
    Selects element "year"
  - //name/..
    Selects element "artist"
  - //painting/@id
    Selects attribute "id" in painting
  - //painting[year>1800]
    Selects "painting" elements with year > 1800

XSLT:

- eXtensible Stylesheet Language Transformation (XSLT)
- Used for transforming an XML document into another form
- Defined as an XML document
- Uses XPath for selecting elements and built-in functions for processing/transforming them

```
<?xml version="1.0" encoding="UTF-8"?>

<xsl:stylesheet version="1.0"

xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
>

<xsl:template match="/art">

  <h1>Paintings</h1>

  <ul>

    <xsl:for-each select="painting">

      <li>

        <xsl:value-of select="title"/> by

        <xsl:value-of select="artist/name"/>

      </li>

    </xsl:for-each>

  </ul>

</xsl:template>

</xsl:stylesheet>
```

References:

1. R. Connolly and R. Hoar, Fundamentals of Web Development, 2nd ed., Pearson, 2017.

XML Processing:
Every language has some libraries to process the XML document.

XML processing in Javascript:
XML processing using JavaScript: JavaScript provides two ways to process the XML document. 1) The approach that used to process the entire XML document is known as in-memory approach. The second method is to extract/pull the few lines or elements at a time is known as event or pull approach.

Let say we are receiving an XML document “art.xml” through ajax query. Request is sent to the server through send() method and object to the XML document “xmlDoc” will be received as response from the server. Further, all elements with name “painting” will be pulled and will be store in an object named “paintings” which can be used for further processing.

```
var xmlhttp = new XMLHttpRequest()

xmlhttp.open("GET","art.xml",false);

xmlhttp.send();

var xmlDoc = xmlhttp.responseXML;

var paintings =
xmlDoc.getElementsByTagName("painting");
```

XML processing in jQuery:

Let suppose the XML document is in string form, first string is converted into the XML document and stored in the DOM object. After that the DOM object is converted into jQuery object and then all elements with name “painting” will be extracted and stored in the “paintings” which can be used for further processing.

```
var art = '<?xml ...';

// create the DOM object

var xmlDoc = $.parseXML(art);

// convert DOM object to jQuery object

var xml = $(xmlDoc);

// find all the painting elements

var paintings = xml.find("painting");
```

XML processing in PHP:

There are two ways to process XML document in PHP i.e., SimpleXML and XMLReader.

1. SimpleXML

Let suppose there is an XML file “art.xml”. The if condition is used to check wether the file is available or not. If file is available then XML file is loaded to an object, then first element is accessed and printed using “echo” command.

```
<?php

//xml file with name art.xml

$filename = 'art.xml';

// condition to check the file is available

if (file_exists($filename)) {

//if yes then file is loaded to “$art”

  $art = simplexml_load_file($filename);
```

```
    // access a single element
    $painting = $art->painting[0];
     // display elemements
    echo '<h2>'.$painting->title.'</h2>';
  }
```

2. XMLReader:

```
    $art = simplexml_load_file($filename);
    $titles = $art->xpath('/art/painting/title');
    foreach ($titles as $t) {
      echo $t . '<br/>';
    }
    $names = $art->xpath('/art/painting[year>1800]/artist/name');
    foreach ($names as $n) {
      echo $n . '<br/>';
}
```

References:

1. R. Connolly and R. Hoar, Fundamentals of Web Development, 2nd ed., Pearson, 2017.

JSON

JSON stands for JavaScript Object Notation and is a most commonly used text format for storing and transporting data. JSON is "self-

describing" and easy to understand. JSON objects are in the key value pair form.

As compared to XML, JSON is quicker to read and write, and easy to parse. For AJAX applications, JSON is faster and easier than XML

Using JSON in Javascript:

JSON is parsed into a ready-to-use JavaScript object.

Example of JSON in Javascript:

```
//Let JSON is in text document

var text =

  '{"artist": {

            "name":"Manet",

            "nationality":"France"

    }

   }';

//converting text into javascript object

var a = JSON.parse(text);

// can access value using the object

alert(a.artist.nationality);

//or a['artist']['nationality']
```

Using JSON in PHP:

```
<?php
// convert JSON string into PHP object
$text =
 '{"artist": {
            "name":"Manet",
            "nationality":"France"
    }
  }';
$anObject = json_decode($text);
// check for parse errors
if (json_last_error() == JSON_ERROR_NONE) {
  echo $anObject->artist->nationality;
}
?>
```

References:

1. R. Connolly and R. Hoar, Fundamentals of Web Development, 2nd ed., Pearson, 2017.

https://www.w3schools.com/js

WEEK 15

Overview of web services:

When you are developing a website, most of time you need to use some services from an external or internal source which is known as web services. These sources may include external finance system, internal legacy system, travel service, and map services among others as shown in the following figure.

SOAP base web services:

SOAP stands for Simple Object Access Protocol which is an XML-based protocol having the main benefit of implementing the SOAP Web Service as its security. SOAP offers a wrapper for sending web service-based messages over the Internet with the help of HTTP protocol. All its messages are usually in XML format. The SOAP message consists of:

1. SOAP document has a root element termed as the <Envelope> element. This element is the initial element used in an XML document.

1. SOAP document has a root element termed as the <Envelope> element. This element is the initial element used in an XML document.
2. Then next to the "envelope," which is categorized into two parts. The former is the 'header,' and the later is the 'body.' The header includes the routing-data that is essentially the information telling the XML document to whom or for which the client needs to be sent.
3. Lastly, the body includes the actual message

Rest web services:

REST service is not a collection of a paradigm or specific rules; it is instead the style architecture for software. Those apps that are designed using this architecture are collectively termed as RESTful web services. Contrasting SOAP, which aims at actions, REST deals majorly on the resources.

It establishes the resources through the use of URL as well as depends on the nature of the transport protocol (like HTTP's: GET, PUT, POST, DELETE, etc.) used to perform the resources. The allocation of resources in REST depends on the URL. It is more like conventions-based application.

1. Identity. Each web service request must identify who is making the request.
2. Authentication. Each web service request must provide additional evidence that they are who they say they are.
3. API Keys
4. https://dev.virtualearth.net/REST/v1/Locations?o=json&query=British%20Museum,+Great+Russell+Street,+London,+WC1B+3DG,+UK&key=[BING API KEY HERE]

References:

1. R. Connolly and R. Hoar, Fundamentals of Web Development, 2nd ed., Pearson, 2017.

https://www.w3schools.in/restful-web-services/types-of-web-services

Consuming Web services in PHP:

Webservice can be used for different purposes using the URI provided by that webservice. Every webservice can provide multiple URIs, functionalities, resources that one need to consume. For example, you

want to use the image sharing website named “flicker” which provide high quality images and API for accessing those images. For this, you need to generate and send a query and results will be fetched based on that query which are actually the images which can be displayed on your own web page. In this way you are displaying images on your own web page that are provided by another source (shown in the following figure).

Consuming JSON based web service:

Example 1:

Let suppose we want to use a map related JSON based webservice. For this purpose, there is need to provide latitude and altitude that particular location that we want to display. If location found, JSON the name of location for encoded latitude and altitude values will be returned. To find the surrounding location of the already provided latitude and altitude values, another webservice will used. The JSON encoded list of names of the surrounding will be returned. In this way, you will get maps related to the provided latitude and altitude and can display in your web page. Similarly, you can interact with multiple webservices and use them in your web page.

Example 2:

References:

1. R. Connolly and R. Hoar, Fundamentals of Web Development, 2nd ed., Pearson, 2017.

2. https://www.w3schools.com/js

Creating a JSON Web Service

1. Consider the URL and format of requests
2. Tell the browser to expect JSON rather than HTML
3. Header('Content-Type: application/json');
4. Use json_encode() to format.
5. Implement JsonSerializable

Example:

There is Class named “Country” which contains country related information. This class extends “DomainObject” class and implements “JsonSerializable” interface. To generate some response, a jsonSerialize() method will be implemented. In this function key value pairs will passed in the form of array and will be converted into the Json and those who request these web service will receive these information as shown in below figure.

```
class Country extends DomainObject implements JsonSerializable

{

  /*

  This method is called by the json_encode() function that is part of PHP

  */

public function jsonSerialize() {

return ['iso' => $this->ISO,

'name' => $this->CountryName,

'value' => $this->CountryName,
```

```
'area' => $this->Area,

'population' => $this->Population,

'continent' => $this->Continent,

'capital' => $this->Capital

]; }

}
```

Output:

References:

1. R. Connolly and R. Hoar, Fundamentals of Web Development, 2nd ed., Pearson, 2017.

Web services using AJAX:

Let there some webservices which can be our own developed or external webservice and we want to use those web service in Javasript using AJAX.

The following figure shows a simple webservice where user type first two letters of a country, a list of countries name having first two letters as “ca” will appear. The user selects the target name or type them.

Similarly, if some want to used google map you need to use the required API. For this, you need to use the URI of google map API and mention

the information you need to access. In this way you get the required data in the form images or in embedded map object form as shown in the following figure.

Code example:

The HTML page used for displaying the map having an element map with required height and width which is 500px and 600px respectively. In the script the URI and required key need to be defined in the script tag. To use the Map service, a javascript function is defined with required parameters like latitude and altitude and the previously defined element “map” with some added properties like alignment, zoom among others.

```
<!doctype html>
<html>
<head>
  <style>
    #map {
      height: 500px;
      width: 600px
    }
  </style>
  <script
```

```
        src="https://maps.googleapis.com/maps/api/js?key=YOUR_API_
        KEY" async>

  </script>

  <script>

     $(function() {

     // hard-coded latitude and longitude for demonstration purposes

      var ourLatLong = {lat: 51.011179 , lng: -114.132866 };

       var ourMap =
new  google.maps.Map(document.getElementById('map'),

      {
         center: ourLatLong,

         scrollwheel: false,

         zoom: 14

      });});

</script>

</head>

<body>

   <h2>Our Location</h2>

   <h3>This is where we work ... </h3>

   <div id="map"></div>

</body>
```

</html>

References:

1. R. Connolly and R. Hoar, Fundamentals of Web Development, 2nd ed., Pearson, 2017.

JavaScript Frameworks

1. JavaScript frameworks provide structure for developing JavaScript applications (front-end)
2. Popular frameworks
3. Angular
4. Ember
5. Vue
6. React
7. …

JavaScript Front-End Frameworks

1. **Ember** forces developers to adopt a known and well-regarded approach to structuring and implementing a web application. It uses a variant of the MVC pattern
2. **Angular** has many similarities to Ember (i.e., models, templates, and routing), and has the added advantage of being partially maintained by Google.
3. **React** is developed by Facebook. Unlike Ember and Angular, React is not a complete MVC-like framework; instead it focuses on the view.

JavaScript for Backend (Node.js)

1. Alternative to LAMP stack
2. MEAN stack
    - MongoDB-Express-Angular-Node.js
3. MERN stack
    - MongoDB-Express-React-Node.js

Introduction to Node.js:

1. Started in 2009
2. Nod.js is an open-source server environment that runs JavaScript outside the browser, can be run on every platform (windows, Linux, mac os x among others).
3. Node.js eliminates the waiting, and simply continues with the next request.
4. Node.js runs single-threaded, non-blocking, asynchronous programming, which is very memory efficient.
5. Also used for non-web development, e.g., VS Code

Prominent Nod.js based web applications:

- LinkedIn switched from Ruby on Rails to Node.js
  - Up to 20 times faster
  - Servers reduced from 30 to 3
- PayPal switched from Java to Node.js
  - Developed in less time, with less developers
  - 33% less lines of code
  - 40% less files
  - Doubled performance (requests/second), with less processors
    - Fast response time, 35% less response time

Push based applications:

Node.js basically used for push-based applications. In push-based applications a user generates a message and send to the server. The server processes the received message and then pushes the processed message to all the concern listeners.

Blocking thread-based architecture:

In traditional server-based web development languages like php, Asp.net among others are based on blocking threaded architecture. In these

languages, there are limited number of threads which executes task/process in parallel. Every thread performs a single task at a time means when a single completes their execution only then new task/process can be served. In this way when there more task in the queue to be executed on limited number of threads. In that case tasks in the queue will have to wait until the threads get free by completing execution of already assigned tasks. So, this results in high waiting time users' task in queue.

Node.js Single-Thread Architecture:

The Nod.js support single-threaded architecture. In this architecture, tasks are executed in a loop which serve the new tasks in the queue without waiting for the completion of task which is already under process and their execution not completed yet. In this way, Nod.js reduce the waiting time of the task in the queue.

Node Package Manager (NPM)

- Package manager for JavaScript
- Used for installing JavaScript libraries and frameworks
- Can handle dependencies
- Usage

npm install // command to install npm package

npm install packagename // To install a particular (by mentioning their name) npm package

npm install packagename@version // To install a particular version of npm package

```
npm update // To update the npm package

npm list   // To show all the packages
```

This a simple Nod.js program (hello.js) that listen to the request and display a hello message as response to that message.

```
// Load the http module to create an HTTP server

var http = require('http'); // including the required package

// Configure HTTP server to respond with Hello World to all requests

// “function (request, response)” is the call back function that execute

//when process completes.

var server = http.createServer(function (request, response) {

    response.writeHead(200, {"Content-Type": "text/plain"});

    response.write("Hello this is our first node.js application\n");

    response.end();

});

// Listen on port 7000 on localhost

server.listen(7000, "localhost");

// display a message on the terminal

console.log("Server running at http://127.0.0.1:7000/");
```

How to execute the “hello.js” program?

You have to open terminal or command prompt and write the command like: node hello.js, the program will run. The message will be displayed

by adding the IP address in the browser as shown in the following figures.

Simple Node.js program 2:

- To save a text message into a file
- data is written in the file 'message.txt'
- When writing is complete, anonymous function is "called back"

```
const fs = require('fs') // include package “fs”

// file name: “message.txt”, call back function: “function (err)”
fs.writeFile('message.txt',  'Hello World!', function (err) {

  if (err) {  // if error exist show error message or display message

                // “writing is done”

    console.error(err);

    process.exit(1);

  } else {

    console.log('Writing is done.');

  }

})
```

Express.js:

Its bit difficult to handle web requests using Nod.js only, therefore, to resolve this difficulty, a number libraries can be used to facilitate the Nod.js development. One these libraries is Express.js library which is also known as Minimalist web application framework for Node.js.

Express.js has support for handling:

- Routes
- Requests/Responses
- Views

Example:

```
const express = require('express') // including express package

const app = express()       // create express object

const port = 3000           // mention port number

// for get request, get method is called with name and call back function

 app.get('/:name', function(req, res) {

// reading user name using “req.params.name”, concatenate with word
//“Hello”, and will sent through response object.

  res.send('Hello '.req.params.name)

})

// To run the server which will listen through port “port”

app.listen(port, function (){

 console.log("http://localhost:".${port})

})
```

Background:

There are many JavaScript’s front-end frameworks that exist that we use to develop the interfaces. The main idea behind it is to generate interfaces by using JavaScript or HTML. We can generate interfaces

through HTML and then Java script will be used to fill the data. The question arises here how can we get data from the server-side. The Client-side sends the request through the browser to the server and renders the fetched data from the server at the appropriate place. There are many JavaScript front-end frameworks available but we are going to discuss the relatively early days framework which is Angular.

Introduction:

Angular is a popular browser-based and open-source framework. Although it is open source, many of its lead developers are employees of Google. It follows the MVC framework. It also stands for "A" in the terms of MEAN Stack but it does not mean that you have to use angular whenever there is MEAN in full-stack. As we have discussed in the previous lecture that there are many other options which can be used like react, view or amber can be used. It is independent of other components of Stack and can be used without any of them.

Angular 1 was written in JavaScript. Angular 2 is not an upgrade of angular 1 but it is completely rewritten. Angular 2 required developers to switch from java script to Type Script which is a superset of JavaScript. In TypeScript, we can statically assign the types of data variables but this is not possible in JavaScript because JS is managing the types of data variables at run time. Angular not restrict the developer to use the TypeScript only but it also allows them to write in Dart whose syntax is like C language. AngularJS uses JavaScript and we are going to discuss AngularJS because it uses the regular JavaScript. It has a large user base of committed developers.

Why Angular JS?

One of the reasons for the developer's interest in AngularJS was that it is well suited in fact angular and front-end frameworks are well suited for single-page applications. Single-page applications are web

applications that typically consist of only a single page that is updated dynamically. It means all the functionality is available on a single page and record components fetched the required data from the server and fill it in appropriate places. For example, when you open Gmail, the interface has been generated. A new page is not going to open when you click on the inbox. It only shows the list of emails.

Example:

Here is a very simple angular application. Angular can be included by saving in the local system or you can include it through the content delivery network.

<html ng-app>

//Here we are telling that HTML document is basically an angular app. ng stands for angular. ng-app is a directive for designating the root AngularJS element.

<head>

<title> Chapter 20 </title>

<script src=https://code.angularjs.org/1.5.0/angular.min.js>

</script>

</head>

<body>

Enter your name: <input type="text" ng-model="name" />

// Text field is created here and angular model has been defined for this text field and "name" is the given name of the model. The model could

be used for giving data. Ng-model=”name” is a directive for saving the field value in the model.

 You entered: {{ name }}

//we are going to access the model here. Model name should be written in the double curly brackets and it displays the data stored in that model. {{ name }} is a data binding expression.

<hr>

Enter your city: <input type=”text” ng-model=”city” />

 You entered: {{ city }}

//One more angular model has been defined and the name of the model is city. Model can be accessed by using its name in the double curly bracket.

</body>

</html>

You have created the models on the input field and linked them to the paragraph tag. Paragraph tag will display the value stored in the model. As soon as you are going to write anything in the input field, it will display in paragraph.

Connolly & Hoar, "Fundamentals of Web Development" - 2nd Ed. Pearson Ed. (2018)

Let's extend this example by adding in an AngularJS controller. Angular is MVC framework and it has three components i.e., Model, View, and

Controller. Controller is basically managing the request. Let's write the code for simple controller, fetch the data from controller and display it.

Adding a controller

<html ng-app="demo">

//Now this directive is specifying the module used in the application.

<body ng-controller="myController">

//This element is going to use a controller (myController) to get its data.

City Search: <input type="text" ng-model="search" />

//Text field has been generated and search model has been defined for this text field which saves the user's input in a model property named search

<table>

<tr ng-repeat="city in cities | filter:search |orderBy: 'name' ">

//

A directive to loop through a collection named cities (which is defined in the controller). We have cities array or the collection. This repeat is going to iterate all cities present in cities collection one by one. This is just like the for each loop. Then we are going to filter the data on a search basis. The value of search we are getting from the text field. Then we are going to sort the data in order on the basis of cities name. It uses filters to alter how this element works. In the example, the filter filter and the orderBy

filter are used to modify how the ng-repeat works. Here the search refers to data items in the model.

```
<td> {{ city.name }} <\td>

<td> {{ city.country }} <\td>
```

//It bind the data to values in the collection.

```
  </tr>

</table>

</body>

</html>
```

How controller works?

Controller has a collection of cities. We are going to fetch cities from the controller and filter hose values and display them in the form of a table.

```
Var myapp = angular.module('demo', []);
```

//First we are going to create the app and then create the module using angular.module. A module is an AngularJS container for the different components used in the application.

```
Myapp.controller('myController, function ($scope)
```

// We can create controller through the app. Add a controller to the module named myController. The $scope variable is passed or injected into the controller by AngularJS. $scope is going to use the access the variable stored in the controller.

```
{
```

```
$scope.cities = [{name: ‘Calgary’, country: ‘Canada’},
                 {name: ‘Toronto’, country: ‘Canada’},
                 {name: ‘Boston’, country: ‘United States’},
                 {name: ‘Seattle’, country: ‘United States’},
                 {name: ‘Almeria’, country: ‘Spain’},
                 {name: ‘Barcelona’, country: ‘Spain’}];
```

//$scope.cities variable is used to store the model (data). Here we are defining an array of object literal named cities. The body tag of controller access the all data available in $scope.cities.

});

Connolly & Hoar, "Fundamentals of Web Development" - 2nd Ed. Pearson Ed. (2018)

Getting Data from a Web Service

```
var myapp = angular.module('demo',[]);

//First create the app and create the module using angular.module.

// notice that our callback function is passed the $http parameter.

// parameters are injected by the AngularJS system "behind-the-scenes"

myapp.controller('myController', function ($scope, $http) {

// Create the controller through app and pass the anonymous function. Angular injects two parameters in this function. One parameter is scope
```

that stores the data and the other parameter is HTTP which is going to deal with the HTTPS requests and responses.

```
var url =
'http://www.randyconnolly.com/funwebdev/services/travel/countries.
php?continent=EU;

// we are going to specify the URLs in the function and fetch the data
using jQuery from that URL. It retrieves country data from web services.

$http.get(url).then(function (response) {

$scope.countries = response.data

// get will get the data from the URL and after the completion of get
command, it will execute the function. Data fetched from the web
services if going to store in scope.countries variable. Use the body
element to display the data.

});

});
```

Web Sockets:

As we have discussed that we can fetch the data from the server-side while staying at the browser through JavaScript by using ajax.

Ajax is HTTP-based and HTTP is a half-duplex protocol which means data can flow back and forth between two devices, but not simultaneously. The sender can send data at one time and the receiver can receive data at another time, they both can send and receive the data but only one can request will be performed at a time. Both requests cannot be performed simultaneously.

We have one other protocol which is Web Sockets and it is a full-duplex communication protocol. Enables interaction between a web browser and web server. It allows an application program to send and receive information simultaneously. We can send the request and fetch the data at the same time.

Let's see through examples. We will send the data from the server and get the data from the client on the other side. We are going to use node.js to implement the server-side but you can use any other scripting language too.

The idea behind is that the server listens to the requests of web sockets and then responds accordingly. We are going to develop an application that will return the same message to the client which we receive from the client and concatenate the string "message received" with the message.

Server-Side Example (Node.js)

```
const express = require('express');

const app = express();

// We need express and app will be created on its basis.

const http = require('http');

const server = http.createServer(app);

// We need HTTP for request response and will create server through it.

const {server} = require("socket.io");

//We need socket.io library. It is used for implementing the sockets.

const io = new Server(server);
```

// We will create server of this library and create the object io on the basis of this server. Then the HTTP server will be passed to the socket server.

// This object has different event handler on it.

io.**on (‘connection’**, (socket) => {

console.log(‘A user connected’);});

//The first event handler is to on the connection. When the connection of the web socket will establish it will print the message (user connected). As you can see, we have the call-back function. We can give the call back function by giving the arrow notation => and you can also write it like function(socket).

io.**on (‘message**, function(message) => {

console.log(‘message from client: ’ + message);

io.**emit**(‘Message received ‘ + message});

//Second event is used to define the message. As soon as a message will be received, the call back function will be called and inject the message as a parameter and display it on the console. This message will be displayed on the server-side. To display this message on the client-side we are going to use the io.emit event handler.

server.listen(7000, () => {

console.log(‘localhost:7000’);});

Client-Side Example (javaScript)

//Now we are going to communicate with the server through the client-side. Create the object of the web socket and pass the IP address of the

server. Usually, WSS replaces the HTTP protocol. WS is WebSocket and WSS is web socket secure connection.

var client = new WebSocket ("http://localhost:7000");

//Then define the event handler. When the connection will open send the message to the server.

//sending message on socket open

client.**onopen** = function() {

client.**send** ("Message sent to the server");

}

// This message will be sent to the server and server sends the message back. To receive the message, onmessage event handler will be used.

client.**onmessage** = function (event) {

console.log(event.**data**);

}

//Now end the communication with the server and don't want to send and receive the message, close connection.

client.close();

Web Server Hosting

When we finished the development of the website, the next step is to host the website. Website hosting means we have to copy the website on a web server so that the users can visit our website through the web server.

The hosting of the website is very important and crucial because users will be interacting with your website through the server (host). Users will consider your website slow, unresponsive, and unavailable if the hosting is poor. It is not necessary to buy expensive hosting. The solution is to understand the different types of hosting and choose the best option according to your website.

The three broad categories of web hosting are shared hosting, collocated hosting, and dedicated hosting. Within each of these categories, there are subcategories, which altogether provide you with more than enough choices to make a selection that works for your situation.

Shared Hosting:

The most common and most cheap hosting is shared web hosting. This hosting will provide the space on the server for your website that will host many websites on a single machine. All those websites use the resources of the same machine. At times, there might be hundreds or thousands of websites hosted on a single machine. This hosting is suitable especially for small websites or websites that have less traffic. Shared web hosting is further divided into two categories: Simple shared hosting and virtualized shared hosting.

Simple shared hosting

In simple shared hosting, all the websites are deployed on a single machine and it usually has a single Database management system. All the websites use the resources i.e., CPU, RAM, bandwidth, or hard disk space of the single machine. In simple hosting clients receive access to a webserver but they cannot increase their privileges to any part of the machine.

Lack of control and poor performance can be disadvantages of simple shared hosting. Static HTML sites or default WordPress installations

don’t have the problems of control. But if you want to install some software, shared hosts do not permit it. You cannot use a particular version of the software, but rather use what everybody is using.

Poor performance is a more common problem with shared hosting. As all the websites are using a single machine and sharing the same resources, performance problems will arise. Shared servers work fine if they have a small number of websites hosted. some shared servers host thousands of websites on a single machine and which results in poor performance.

Virtualized shared hosting

We have one other mechanism to host the website virtualized shared hosting. In this hosting, we create virtual machines on a single machine. Each virtual machine is assigned its own resources from the resources of a single machine. In virtualized hosting, each website will have its own virtualized machine.

The advantage of virtualized shared hosting is that each website has its own virtual machine, they are given the freedom to install or configure the software. Disadvantage+ is still the same in the end all the resources are shared.

Dedicated Hosting:

In dedicated hosting, a physical machine is rented or allocated to you, and that web hosting service is going to manage that machine. You can install software of your choice on that machine and all the resources of the machine are going to be used by your website. It has an advantage over shared hosting is that you have given the complete physical machine to control and you don’t have to handle the issues that arise due to shared resources.

Hardware is normally standardized by the web hosting service provider and the host is going to take care of hardware issues. The disadvantage of dedicated hosting is the lack of control over the hardware and a restriction on accessing the hardware. It might not suitable for your particular needs.

Collocated Hosting:

Collocated hosting is almost like dedicated hosting, but you can purchase, build, and manage the machine yourself. In collocation, you can use the infrastructure of a web hosting service but you can manage the hardware by yourself, unlike the dedicated hosting. We have physical access to the computer. Web hosting services provide the services like energy requirements, internet connection, electricity, security requirements, fire suppression system, climate control, or air conditioning requirements.

The advantage of collocated hosting goes beyond a dedicated server with not only full control over the operating system, software version, firewalls, and policies but also the physical machine.

The disadvantage is that you must control everything yourself, with little or no support from a third party. These web hosting service providers are costly and they are not going to provide services for the hardware issues.

In-House Hosting:

In in-house hosting, you have to manage the webserver yourself entirely in-house. You will use your personal machine to host the website and your website is going to use all the resources of that machine.

For example, if you are using a DSL connection and you asked your service provider to assign you the static IP address that is not going to change whenever you connect to the internet. You can enter it in the domain name server and host your website here. It has an advantage in

terms of control. It may seem appealing to host your website by using your own home-based machine but there are many issues that can arise. You have to manage all the requirements by yourself. Most of the time, the bandwidth of home-based resources will be less, it has limited resources, and you have to keep your machine online 24/7 to be accessible. This option is good for small websites and for websites where a 24/7 online presence is not important.

Cloud Hosting:

The recent hosting mechanism is cloud-based hosting. In cloud-based hosting, your website is hosted on multiple machines and a third party manages it. For example, Amazon and many other companies provide the services to run your website on multiple machines.

The website has thousands or millions of users, you can manage all its request on one machine and that's why you have to distribute its request on multiple machines to keep your website scalable.

There are different options to host the website. Let's see how to use a free web hosting service to host the website. This is just to get an idea to show how the website can be hosted. But you cannot host a professional website on a free web hosting service because it has many limitations. It will show the ads and free website hosting services cannot give the support of secure connections like HTTPS or SSL. If the website is dealing with users' data, then the data will not be transferred securely and will be exposed. So only use free hosting service for demonstration and learning purposes.

Let's go through one example to demonstrate how the website can be hosted on web hosting.

We are using a free web hosting service (infinityfree). This is free website hosting and it has different limitations.

You will sign up for it and after signing up, you are going to log in to your account. When you log in the system will display the screen, which shows the active accounts, and right now it will show the 1 active account out of 3 accounts. We have created the website already because it can take 2-3 days to enter your website in DNS. A username will be assigned to your website and you can manage your website.

You can see the control panel information and see the information (username and password) related to establishing a database connection.

Open the control panel and you can access different tools here from which you can upload the files to a web server. You can perform many other activities through the control panel. InfinityFree panel is a C panel but this C panel is a very basic version and many features are disabled because this is a free service. When you pay for some good web hosting service and buy paid account then it has many additional options.

In this control panel, you have a preferences panel in which you can perform different settings for your website. There are different options to upload the file. You can upload the file through an online file manager or you can use the option of FTP client. If you want to use the FTP client option then you have to go for File Zilla which is a free and open client for accessing the FTP sites. It has a database-related section and you can create the different databases. It also has the support of phpMyAdmin through which you can manage the database.

Let’s upload the file to the webserver and see how it will be displayed on the website. Click on the file manager. It will show the basic interface to upload the file. We have httd docs folder here and you can upload your files in this folder. These files will be hosted on the server and will be displayed on your website. At the bottom of the page, it has an upload file option choose the file from local storage and upload it. After uploading the file be accessible on the URL which you used while creating the website. You can see the file on your website. We can also change the file name with the option to rename it in the hosting service. You can upload your website, PHP scripts, establish a database connection, enter the data, or retrieve the data.

I would suggest you develop the home page or the personal website and upload the dynamic content.

It is obvious that when you are going to use the free hosting service, you will face limitations. You cannot install the Laravel in infinityfree but you can deal with the PHP scripts and establish a database connection.

Domain and Name Server Administration

Introduction:

Internet functioning is based on the IP Addresses. Each website has IP address and we can access the website through that IP address. It is very difficult to remember the IP addresses because they keep changing. We have domain name system to access the website in which we link the IP address to the URL and all this mechanism done through the domain name system.

We can register the URL of our website through Domain name system. We have different domain name registrar that register our website. We have some popular registrars i.e., GoDaddy, TuCows, Network solutions and many more.

Most of the time registrar also provide the hosting options with the domain name registration although they are two different things. You can host your website on some other web server and use other domain name registrar to register the website. Most of the time, web hosting provider also assign the domain name along with hosting the website.

Search Engines

Search engine has an important role in the world wide web through which users search the websites. Before getting into the details of search engines, let's look into what happened before the search engines.

In the days before search engines, there was no capacity to search the entire world wide web. There were very few websites when the world wide web was created and this number increased to a thousand or more than ten thousand websites. It was not easy to keep the records of this large number and browse them. We had directories to browse. The first directory was “vlib” which was created by Tim Berners-Lee. Websites were listed down in vlib and you can access these websites. The most well-known directory is the Yahoo directory, which listed down and categorized the websites. For example, if you want to visit the university website, you have to select the education category and then the university category and here you find the websites of the universities.

There was one more option which is an open directory project. Directory websites were good but they lacked the ability to search quickly. “dmoz” is the open directory project in which websites are listed down as per categories. Users can share the website information through email, message boards, and chats to inform each other about the websites.

Now, if you have to search the specific websites let's say you want a website related to automobiles, you can visit only those websites which are listed in the directory and those about them you know already.

Search engines solve the problem using the web crawler. Web crawler is the first component of the search engine. Search engines looked for the available websites using a web crawler and then indexed them. If someone searches for the website of automobiles, the search engine will look for the websites that are related to automobiles or have the term automobiles in their content and then show the result.

Search Engine Overview

Components of search engine

Search engine has multiple components but the main component is a web crawler. Web crawl crawls the existing websites. There are the ancho tags in the website through which we users move from website to website. It downloads the page, identifies the anchor tags, and adds them to the database. Crawlers are also known as web spiders, worms, or robots. Around 50% of the websites can be accessed through this component.

URL submission is another mechanism to search the website. A new website has been uploaded and it doesn't have anchor text right now. The other websites cannot point to the new website. Search engines cannot find the existence of the website. Here comes the URL submission where you add the URL of your website which tells the search engine about the existence of the website. Now, it should be made part of search engine index and users can access them.

Indexes are also the mechanism to search the website. Search engine saves the content of the website and creates the index on it. These indexes will be stored in the database and find the keywords in the

content of the website. After that, queries can be executed quickly on those indexes to recover the complete records.

The query server handles requests from the users for a particular query. Search engine shows the results of the query. Now, it is very important to rank the result of that query. For example, there are more than 10 million websites on automobiles, which of these will be the top 10 from them. Search engines solve this problem by using ranking algorithms.

Page Rank

Page rank is an algorithm that is used for ranking websites. It was published by Google's founder. This algorithm is basic for search engine ranking. In fact, search engines use many other factors for ranking websites as well but the most important factor is the page rank algorithm. It works on the graphs that model all the websites and their connection with other websites. The graph looks at the hyperlinks between websites. Let's look at the example and create the graph for the example. We have four websites i.e., HEC, VU, QAU, and MOE.

We have four websites so we will have 4 vertices in the graph. Then edges will be created for the vertices on the basis of connection with other websites. One website has a link to another website. For example, "VU and QAU" websites have the link of "HEC" and it will be shown in a graph by drawing the edge from VU towards HEC and from QAU towards HEC. The 'HEC" website has the links for the links VU, QAU, and MOE, and connections will be created from "HEC" to these three websites. The following diagram shows the graph of the example. Each website becomes a vertex and each connection to another website becomes the edge.

Figure 6: Example

How the page rank algorithm interprets the graph? The main idea is that authentic websites are pointed by authentic websites. Now the question arises of how to identify the authentic website and which authentic website is pointing to which websites? We can see from the graph that one website is pointing to which website but how to calculate the authenticity and importance. In the following diagram, you can see the different websites pointing to other websites. Now look at the website named as 1 and is linked by the one website. Look at the website named 2 which is also linked by the one website. But both websites have different page ranks although they have the same number of websites pointing it. Page rank is represented by the size of the vertex for this particular website. Website 2 is linked by a yellow vertex and its size is larger than the website pointing towards website 1. The bigger size of the vertex represents more importance. Website 2 has a larger page rank than website 1 because it is linked by the important vertex.

Figure 7: Example

How it works?

We can calculate the authenticity and importance through iterations. In iteration 0, we assign the same page rank to all websites i.e., HEC, VU, QAU, and MOE. Each vertex is assigned the ranking of 1/n. We have four websites for the current example so "n" will be 4 and the page rank for all websites will be 1/4.

In iteration 1 each vertex will be visited and its page rank will be distributed among its outgoing vertices. HEC vertex is in consideration, its page rank is 1/4 and it has three outgoing links towards VU, QAU, and

MOE. This page ranking of 1/4 will be distributed among these three vertices. Each outgoing vertex of HEC will have a ranking of 1/12. Now consider the QAU vertex, it has the page rank of 1/4 and it has one outgoing link towards HEC. Its page rank of 1/4 is going to be distributed to one outgoing link and HEC will have the page rank of 1/4 from QAU. MOE vertex has no outgoing link and its page rank will not be distributed.

The sum of all the page ranks should be 1. In the first iteration, the HEC vertex has a ranking of 1/4 assigned from QAU and 1/4 assigned from VU, and page rank will be 1/2 for HEC. Other vertices have a 1/12 page rank. This process is a repetitive process. In this next iteration, consider HEC which has 1/2 page ranking. This rank will be distributed among 3 vertices and each vertex will be awarded 1/6 page rank from HEC. This process will be repeated and page rank will stop changing at one point. We cannot perform more iterations because page ranks have been fixed. They won't change anymore and we call it page rank convergence. The page rank algorithm runs when either page ranks are supposed to be converged or a certain number of iterations will be completed. For example, we would find page rank for 1000 iterations and after 1000 iterations the score will be fixed.

The basic version of page rank and the problem is that if the vertex has no outgoing link, then ultimately the page rank of that website will be 1and 0 page rank for other websites. This problem will be solved using a scaled version of page rank. In the scaled version in each iteration, each vertex has been assigned the page rank. Some page ranks are assigned on the basis of calculation and other page ranks are equally distributed among the vertices. This version does not create the sync holes and page rank cannot be accumulated at one point. This is the one mechanism to rank the websites and it is not easy to manipulate them. We can create the links to other websites on our websites but it would not be easy to add the link of your website to the important or authentic websites.

Week 16

Search Engine Optimization

Our website ranking is also depending on the content and design of our website. We can increase the ranking of our website and this is called search engine optimization (SEO). The main purpose of SEO is to increase the ranking of a website. It makes a website more appealing to search engines. If some user searches for something related to our website, then our website should be displayed on the top results. We have two categories of SEO i.e., White-hat SEO and Black-hat SEO.

White-hat SEO is an ethically allowed and accepted way to increase the rank of a website but black-hat SEO is not recommended. It is unethical and search engines can blacklist the website which is using the black-hat SEO.

White-hat SEO Techniques:

White-hat SEO techniques to increase the rank of a website in the search results are ethical and obvious. Website ranking can be increased through page ranking which finds the importance of website. It is very challenging for the search engine to find the best result for millions or hundreds of millions of websites. Search engine looks for many other factors i.e., design and content of the website. Let's have a look at which technique is good to use to increase the ranking of website.

Title and meta-data

The title of the website is the most important to optimize for search engines. Title should be appropriate. Title should be descriptive, to the point, short in length, and related to the web page. Your web page is identified by the title in search engine. Search engine is going to decide the category of your web page on the basis of title.

Metadata basically contains the summary of your website and it should be properly described. You are going to define the description of the website in metadata. Search engine uses this description when displaying the sites in search results.

We can also control the behavior of the search engine by defining the robots. You can allow or disallow the robots to access the website. Search engines do not index your website if you disallow the robots' options in the metadata.

Designing URLs

Unform Resource Locator (URL) is used to identify the resources on the WWW. They are also used to index your website and gather additional information about your pages. URLs of your website should be descriptive. Some URLs work fine for the website but cannot be understandable and readable by humans easily. If you are going to use the query string in the URL i.e., you create the URL like example /index/users.php?userID=123.

This will work fine and return the correct result. If the user followed this link to reach the page, they will see the expected output, but it is difficult for them to understand the URL and to know what they are

going to see without a reference. Search engines consider the understanding of the URL for the user. You have to make the URL meaningful.

In the former example, we are looking for some user and the URL has the userID. UserID can be changed with some descriptive name which will be more understanding. userID=123 can be changed example /index/users/userID/webVU. Search engines will consider your website for top ranking if the URLs and path of web page have meaningful and understanding words. File paths should be descriptive as the URL of the website.

Website Design

The design, structure, and layout of the website have a huge impact on the website's visibility to search engines. The layout of the website should be proper. Website should be readable; color contrast should be good. It should not be like light background had a light color font and it will be readable or visible at all that's why should focus on the color contrast. The website should be properly structured and should use the proper nav, side, and sections tags so that website should be well defined.

Your website's UI should not be created through JavaScript or Flash because it will suffer from poor indexing. This is because crawlers cannot interpret the contents generated through JavaScript. Crawlers can only access and scrape the content of HTML documents. There are ways for example in react or angular where JavaScript content can be accessed but then those search engines have that type of support. If your content is not available to the search engine then your website will be invisible to the search engine and ultimately your website cannot be ranked.

Sitemaps

Sitemaps can be used to capture the website structure. Sitemaps are the XML files that contain the layout of your website. The search engine gets the layout of your website and got to know the content and location of web pages of your website.

Anchors and Images

Anchor text is the text inside the anchor tags, which the user sees as a hyperlink. Anchor text should not be like "click here" or "click here to find more". You should avoid such texts and add some meaningful text. Anchor text describes the link and it should be appropriate.

Many search engines have a separate site to search for images. Unlike an HTML page, it is much more difficult to index an image. Search engines cannot process the contents of the image. There are some elements of the images that are indexed including the filename and the alt text. Filename of the image should be appropriate and must define the image. For example, if there is an image of rose, do not write the filename as xyz.jpg instead of rose.png. File name and alt text should be meaningful and describes the image.

Content

The content of the website is also important for the website ranking. To increase the visibility of your pages in search results, you should use original and fresh content. Do not use the copied content.

Freshness of the content is most important factor. If the website has not b updated for the last 10 to 20 years, the search engines will consider this website an old website and it will not give a good ranking. If you keep updating your website then the search engines will increase your ranking. Many websites use user-generated content. They have comments, feedback, or review module on the website. These contents add to the website which keeps the content of the webpage fresh and it will increase the ranking of your website.

Black-Hat SEO

Black-hat SEO techniques are used to increase the website ranking in unethical ways. These techniques are constantly evolving as people try to exploit weaknesses in secret algorithms. Search engines may punish or ban your website from their result for using the black-hat SEO techniques. The main purpose of these techniques is to improve the website ranking. Black-hat SEO techniques are unethical techniques that should not be used. You should know these techniques so that you can avoid them.

Content Spamming:

Content spamming is using the content or the keywords in your website which are not related to your website. It is the technique that manipulates search engine results by adding irrelevant content and keywords to your website. Let's say your website is related to automobiles and the content and keywords in the website are related to health, education, tourism, or food. These irrelevant content and keywords are added to the website so that when users research related terms then your website should be shown in the related results. We should not add the irrelevant content to your website.

Some people also use the technique where you purposely add keywords to the site in a most unnatural way and this is known as keyword stuffing. In this technique, meaningful content is replaced by the content which is primarily for robots, not humans.

People also took measures to stuff as many words as possible into their web pages. Most of the time people use keywords that are popular and trending. All these techniques should be avoided.

Paid link is also a black-hat SEO technique in which users pay for the links. They pay to add the link of their website to other websites using anchor tags and the text of the anchor tag should be strong enough to improve the ranking. Purchased advertisement on the website is not considered paid link unless and until they are hidden.

Doorway pages are the pages written to be indexed by search engines and included in the search results. These pages are basically computer-generated pages crammed full of keywords. These pages are not human-understandable and they try to boost the search engine.

Link Spam

Since PageRank is basically using the links which tell which website is linked by which website of which website is pointing to which website. If the website has many in-links it means many websites are pointing to it and it ultimately increases the ranking. Users of black-hat SEO techniques manipulate the links by adding links to your website on other websites.

Hidden links are straightforward as hidden content. They try to hide the link from real users by matching the link color with background color. This technique manipulates the search engine without impacting the human reader.

There are websites that allow the users to post comments, feedback, or review i.e., YouTube, Facebook, or Instagram. Black-hat SEO technique users pay the users to comment and post the link of the website in the comment to increase the ranking.

Link farms and pyramids utilize paid links to manipulate the PageRank. It is the most effective way to get your website rank increased. Link farm is a set of websites that all interlink each other. For example, you have 10 websites and those 10 websites start pointing to each other which is intended to increase the PageRank which ultimately increases the ranking of the website. Link farms are very easy to detect and to avoid the detection people use link pyramids. In link pyramid, every website does not point to each other. Some websites point to other websites and some do not. It has the intention to promote one or two websites.

Google bombing is the technique to use the anchor text in links. Search engines process the anchor text and associate it with the related result. It is also used to defame the website and to increase the ranking of a specific website. They start using irrelevant anchor text to defame the website and relevant anchor text to improve the ranking. These techniques must be avoided.

Google Bowling:

Google bowling is used to defame the websites of your competitors. This is the dirtiest and most immoral technique because it weakens the website to remove it or blacklist it from search engine results. They start applying the black-hat SEO techniques on that website. Apparently, the ranking of that website will be improved but if the search engine identified those techniques, they will remove or black list the website after identifying.

Cloaking:

Cloaking is the technique to show the content on the basis of users. This technique serves the content differently from regular users. The header of the website request contains the user agent information i.e., window, browser name, or the version. When the authentic user visits your website and you can find it from the user agent information then you show them the different content which will be using the black-hat SEO technique. If the search engine is visiting your website, then you show the content which can't expose the black-hat SEO techniques.

Duplicate content

Content generation is not an easy task. The users of the black-hat SEO technique copy the content from different websites and paste it into their own website which is shown in the form of new content. It makes your website unauthentic and if the search engine finds it, it will get your website black listed.

Online Social Networks

Social networks are systems designed to bring people together. In this era, there is no one who is not using social networks. Famous social networks include Facebook, Instagram, Snapchat, Twitter, LinkedIn, and among the sea of others. Social media has penetrated our lives. As a web developer, you have to use the social network tool positively for your website. Social networks allow users to interact with each other and they can share the content with each other. social networks include the following characteristics.

Social media networks are free for registration and for using a well. It is essential for social media platforms and the best way to attract the users is to make it free.

Social media allow users to manage their profiles. Everyone has something to say about themselves and every social network provides a place to say it. Social media allows the user to easily setup at the individual level as well as organizational level.

Social media allows users to generate revenue through monetization. Social media allows the developers to access the platform programmatically because it will increase their exposure. Developers can use the social media content or link it with other social media platforms.

Social Media presence:

Today, social media presence is very important because many users are not browsing the website but they are using the social media platforms. Your presence on social media increases the user traffic to your website. If you are not present on social media then you are restricting the user's traffic.

It is very easy to create a home page on social media. All social media platforms allow creating and management of the profile of individuals, companies, groups, or the organization.

Social media allows adding your profile link to the home page of your website. You can show your social media presence on your website. You can also use the logos from the social media platform itself.

You can use the URL shortener to shorten the URL of the link. Shorter URLs are preferable to long URLs since they leave more room for other content. It might appear random to add extra characters while shortening the URL but it has the advantage of providing the statistics of user's visits to the website.

You can add the social media plugins on your website then Facebook's social plugins provide a wide range of things like buttons, an activity feed, and comments. If you want to create the like button or want to display the comments related to specific posts you can easily use the Facebook plugin. You have to register as a Facebook developer. Facebook uses the Facebook Markup Language (FBML) for its plugins. You can also use the plugins by adding the specific elements in the existing div or in HTML elements.

<div class="fb-like"

data-href=https://example.com/page.html

data-width="200"

data-layout="standard"

data-action=”like”

data-size=”small”

data-share=”true”>

You have the webpage example.com/page.html. If you want to create the like button it has a simple mechanism. You will send a request to Facebook as a developer. You will get the key and use the JavaScript code of Facebook to the webpage and then display the div on the page. This like button will appear.

Twitter is another popular social media platform. You have seen a news website available on Twitter and if the Prime Minister or any other important person retweets it, then that person will be embedded. This embedding will be done through Twitter widgets. Twitter follows the same pattern of including a javascript library and then using the tags to embed simple social widgets. However, Twitter has a different approach to embedding social widgets into a page. They prefer most users to paste code from a box, rather than try to explain how to create widgets.

 Tweet

<script async src=https://platform.twitter.com/widgets.js” charset=”urf-8”></script>

Here is a small example to create the share tweet button on the web page. When a user clicks on it and user can tweet about that web page from its own account. You have to create the simple anchor tag with a specific class, add the URL, and then the text message which wants to be retweeted. Twitter’s JavaScript link should be included after the anchor tag. There are many other plugins available.

Monetization with Ads

High-volume websites can generate revenues. Website is going to display the advertisement on it and the type of the advertisement depends on the number of users visiting your website. You can generate revenue through this advertisement. Let’s have a look at what is advertisement economy and how can we place the ads on websites.

You can agree with the company and display the advertisement of that company on your website. This might be an option but options to have an agreement with the companies are limited. If your site ever gets enough or is sufficiently local, you can create and manage your own client account through your own home-based advertising network.

The vast majority of advertising is done through advertising networks. These advertising networks connect the website and the company that wants to display their advertisement without getting them to know each other. There are many advertising networks that you can choose from.

Advertisement Types

There are many types of web advertisements that go beyond the basics. The most common types of advertisement are graphics, text, banners, and dynamics.

Graphic ads are the ones that serve a static image to the web browser. The image might contain text and graphics that allow the user to click the ad and will direct them to the company's website. These ads take more bandwidth and space.

There are also text ads which have simply text-based. User clicks on the text and is redirected to the company's website. They also encourage the users to click it like graphics ads. They are popular because they take less bandwidth and space.

Dynamic ads are interacting ads. This can range from a simple GIF graphics ad all the way up to complex Flash widgets. It allows the users to interact right on your page. You have seen the games as an ad through which you can interact. It can be flash files, components, or the HTML 5 code which allows the dynamic interaction. Dynamic ads take higher bandwidth, space, and computation needs.

Creating Ads

You have to register for an advertisement network (AdSense is one of them). You will get the code from the platform and place it on your web page through which ad will be displayed. The actual advertisements are normally a little piece of JavaScript to embed on your page.

Web Ads Economy

The website owner can display ads in exchange for money. We have different commodities which are used to calculate the revenue. The website owner has three commodities: View, Clicks, and Actions.

An Ad View is a viewing of an advertisement by the user. It is based on one loading of the page. There can be many ads on the page but the impression is counted for each one. Ad Click is the user clicks on the ad and directs it to the URL associated with an ad. Action is when a user performs some action through that ad.

Web Commodity markets

On the basis of commodities, we can calculate the rates. Advertisers can pay for their advertisements. The **cost per click (CPC)** will decide the money each can have regardless of how many times it must be clicked. The **cost per Mille (CPM)** is the strategy in which the cost per thousand views will be decided. The **cost per action (CPA)** is related to the cost of action performed by the advertisement i.e., buying the product. This is the most paying commodity in which if a user buys some product, then the website gets the fixed amount for that action. The **click-through rate (CTR)** is the relationship between CPM and CPC. It is the percentage of views. You have a thousand users on the website and how many of them click on the advertisement. On the web, click-through rates are very low. There is the change that 1 out of 1000 users visiting the website. The higher the click-through rate, the more revenue.

There is one other option which is an auction. In this strategy, ad networks find the popular website to place the ad on, and businesses or people compete with each other to place their ad on that website. Ad networks automate the process. They asked the businesses how much they can pay for the ad and they

also have information on websites. They connect the relevant people to relevant content. Websites get their revenue and ad networks get their potential customers.

Web Analytics

It is very important to get the information about your website's users and how many users are visiting your website after hosting the website. We can analyze the website through its usage. Web analytics provides information about the websites to help the owner to make and access changes to their sites. Web analytics basically provide us insight into the usage of our website. There are many aspects that can be analyzed. Some examples are:

- Tracking of the bandwidth usage. Which web page or the media is consuming how much bandwidth. Identification of the sites that are referring your website to the users. From where the traffic is coming to your website. Which websites are directing the users to your website?
- Analyzing search engine crawler traffic. Which search engines are directing traffic to your website.
- Identification of popular domain from multiple domains of your website.
- Identification of the pages which are visited more often.
- Identification of search terms that includes your website in the result. When user searches on Google and your website is also included in that result then you can also analyze those search terms that lead to your website in result.
- You can also monitor the behavior of the users. Which users enter on which web page. How much time they are spending on each page and then where they go after it.
- Demographics can also be analyzed. You can analyze the location of the users, their genders and so on. You can identify which gender is visiting your website more, which age group, and many more. Of course, these demographics might not be accurate and it is very difficult to obtain the demographics of each user so this aspect gives you limited information.

Web Metrics

Web analytics analyses the data of the website. It helps to measure and compare various aspects of web traffic by using different metrics.

Page Views: it is the most important metric which counts the number of visits by each user. It counts the page requests and even if requested multiple times by the same user or the IP address. If the one user visits one page 10 times, then the page view will be counted as 10.

Unique Page Views: It counts the web page request but it will consider the one count for the multiple requests by the same user or the same IP address. If the one user visits one page 10 times, then the unique page view will be counted as 1.

Average Visit Duration: This metric tells about the time users spend on your website. It is also an important metric. The longer the visit indicated the good quality of the website.

Bounce Rate: Bounce rate is the rate at which users bounce back from your website. It is basically the percentage of visitors who leave your website without visiting the other pages. The high bounce rate means your website is not able to retain users. It has many reasons maybe your website has not have the

content he is searching for or your website is not that interesting and the user went away from the website.

Internal Analytics

You can also perform the analytics by yourself. We all know that web servers keep the log files which has the information about the IP addresses that visited your website. There is a lot of data in the log files that can help you see the patterns and trends in the data requests. You can identify the location of the request through the IP address. You can also get the information through the user agents analysis present in the headers. User-agent helps to find from which operating system requests are coming on the web pages, the screen size of the users, number of mobile users, number of users using a browser, and so on. You can also perform the frequency analysis which tells the time on which website has more users visiting the website or the number of users visiting the website on daily basis. These analytics can be provided by the web hosting company.

Third Party Analytics

Internal analytics is good option but the third party also provides the services to analyze your website. Third-party has all of the metrics available internally and much more matrices. Google and Bing provide analytics services. They also analyze the same traffic data as we use in internal analytics but rather than collect it from server logs, they maintain their own logs and use that data. You can register on the analytics engine and they provide the small piece of script that you embed into each page of your site. This script tracks the request directly, analyzes the data, and provides you analytics in the form of reports. These analytics engines provide the facility of flow analysis which tells the flow of users using the website. It tells from which web page the user started browsing and then visited which web pages. It is important from a business perspective. It also provides the in-page analytics which provides the information about the elements on the web page used by the user, scrolling depth, or the time spent on each page. All this detailed information provided by the third-party analytics.

Google analytics- demo

Figure 8: Dashboard of Google Analytics

Let's see the example of using third-party analytics. We are going to use the Google analytics engine which provides free analytical services. You can analyze the usage of your website and gain insight into the website by using the dashboard of Google analytics. Let's go through an example. You must use the analytics.google.com to use the Google analytics. Open it and log in by using your email. Following is the interface after logging in.

Click on the start measuring option, specify the account name, and choose the default option. Check all the boxes and mention the name of your property.

Figure 9: Start Measuring Option

Your property is website, name is 41web.io and monitor the website. You can also specify the time zone and currency.

Figure 10: Specify the Time Zone and Currency

You can also write about your business. You can choose the data stream for your website. Enter the URL of your website and mention the metrics that you want to measure. Data stream has been created and measurement ID has been assigned.

Figure 11: Data Stream

We can use both options global site tag or the google tag manager. In this example, we are using the global site tag. Google Analytics gives you the script, copies it, and embeds it in the website header file. It has been connected to the website.

Figure 12: Global Site Tag

Figure 13: Real Time Report

We have a report option to generate the reports. Click the reports and click real-time again. It will show the analytics about the website. One user visited our website in the last 18 minutes and accessed it from Lahore.

Figure 14: Page View Analysis

Then we want to have 2 users and open the website in another browser but it still shows that there is the only one user who visited the website. This is analyzed on the basis of IP address. If we visit the website from another computer then it will show the two users. On the right side, it is displaying different metrics. There are 2 page views because one is from chrome and the other Is from opera. When the website has multiple pages then it shows the analytics about all pages.

Figure 15: Engagement Overview

Click on the engagement interface to see the user's engagement on your website. It has different options here.

It will show the one-page information when clicking on the page and screen option. It will show the information of the page with its title when the website has multiple pages. You can also select the time period in which data will be analyzed.

Figure 16: Page and Screen Option

Figure 17: Demographics Overview

You can also use the demographics analysis. It will show the analytics about multiple users too. It will show how many users are from which country or the city. It will provide the gender information of which gender is visiting which web page.

As we discussed before that it is difficult to get information about demographics. The advantage of using Google Analytics is that Google has the information about its users. When a user visits your website then Google uses its own information and provides the demographics analysis in detail.